Backend Design Patterns

A Complete Guide

Master GoF Patterns, Architecture Patterns, and Microservices Patterns -- With Java and Python

2025 Edition • 32 Chapters

Table of Contents

Chapter 1 — What Are Backend Patterns
Chapter 2 — Singleton
Chapter 3 — Factory Method
Chapter 4 — Abstract Factory
Chapter 5 — Builder
Chapter 6 — Prototype
Chapter 7 — Adapter
Chapter 8 — Decorator
Chapter 9 — Facade
Chapter 10 — Proxy
Chapter 11 — Bridge
Chapter 12 — Composite
Chapter 13 — Flyweight
Chapter 14 — Strategy
Chapter 15 — Observer
Chapter 16 — Command
Chapter 17 — Template Method
Chapter 18 — Chain of Responsibility
Chapter 19 — State
Chapter 20 — Mediator
Chapter 21 — Iterator
Chapter 22 — Visitor
Chapter 23 — Repository Pattern
Chapter 24 — Unit of Work
Chapter 25 — CQRS
Chapter 26 — Event Sourcing
Chapter 27 — Circuit Breaker

Chapter 28 — Saga
Chapter 29 — API Gateway
Chapter 30 — Service Discovery
Chapter 31 — Project: E-Commerce
Chapter 32 — Glossary

Chapter 1: What Are Backend Design Patterns?

What You Will Learn

- What design patterns are and where they came from
- The three categories of design patterns: creational, structural, and behavioral
- The difference between architecture patterns and design patterns
- The SOLID principles and how they connect to patterns
- Why patterns matter specifically in backend development
- How to read and use a pattern template
- What anti-patterns are and how to avoid them
- When NOT to use patterns (the over-engineering trap)

Why This Chapter Matters

Every backend developer eventually faces the same problems: how do I structure this code so it does not turn into a tangled mess? How do I make this system handle more traffic without rewriting everything? How do I explain my design decisions to a new team member?

Design patterns are the answer to all three questions. They are proven solutions to recurring problems, and they give you a shared vocabulary with every other developer who has learned them. Before you write a single line of pattern code, you need to understand what patterns are, where they came from, and when they help versus when they hurt.

This chapter is the foundation. Skip it, and every other chapter will feel like memorizing recipes without understanding cooking.

1.1 What Is a Design Pattern?

A design pattern is a reusable solution to a commonly occurring problem in software design. It is not a finished piece of code you can copy and paste. It is a template, a blueprint, a description of how to solve a problem that you can adapt to your specific situation.

Think of it like this: if someone tells you to "use a queue to handle background jobs," they are describing a pattern. They are not giving you exact code. They are giving you a proven approach that you adapt to your language, framework, and requirements.

The Key Characteristics of a Pattern

```
+-------------------------------------------------------+
|              DESIGN PATTERN                           |
+-------------------------------------------------------+
|                                                       |
|  1. NAME        - A shared vocabulary term            |
|  2. PROBLEM     - When to apply the pattern           |
|  3. SOLUTION    - The arrangement of classes/objects  |
|  4. CONSEQUENCES - Trade-offs and results             |
|                                                       |
+-------------------------------------------------------+
```

A pattern is NOT:

- A library or framework
- A specific algorithm (like quicksort)
- A finished piece of code
- A silver bullet that solves everything

A pattern IS:

- A description of a solution to a recurring problem
- Language-independent (though implementations differ)
- A communication tool between developers
- A way to capture expert knowledge

1.2 The Gang of Four: Where It All Started

In 1994, four authors published a book that changed software engineering forever: Design Patterns: Elements of Reusable Object-Oriented Software. The authors were Erich Gamma,

Richard Helm, Ralph Johnson, and John Vlissides. Developers quickly started calling them the "Gang of Four" or simply "GoF."

The GoF did not invent these patterns. They observed them. They studied experienced developers, looked at successful software systems, and documented the recurring solutions they found. Their contribution was cataloging and naming these patterns so that every developer could learn and communicate them.

```
+-------------------------------------------------------------+
|           THE GANG OF FOUR (GoF) - 1994                     |
+-------------------------------------------------------------+
|                                                             |
|  Erich Gamma ----+                                          |
|  Richard Helm ---+---> 23 Design Patterns Cataloged         |
|  Ralph Johnson --+                                          |
|  John Vlissides -+                                          |
|                                                             |
|  Key Insight: "Program to an interface, not an              |
|               implementation."                              |
|                                                             |
|  Key Insight: "Favor object composition over                |
|               class inheritance."                           |
|                                                             |
+-------------------------------------------------------------+
```

The GoF cataloged 23 patterns, divided into three categories. These 23 patterns remain relevant today, even though the book was written before Java 1.0 was released. The problems they solve have not changed, even as languages and frameworks have evolved.

1.3 The Three Categories of Design Patterns

The GoF organized their 23 patterns into three categories based on what kind of problem each pattern solves.

```
+--------------------+--------------------+---------------------+
|    CREATIONAL      |    STRUCTURAL      |    BEHAVIORAL       |
|    (5 patterns)    |    (7 patterns)    |    (11 patterns)    |
+--------------------+--------------------+---------------------+
|                    |                    |                     |
| HOW objects        | HOW objects        | HOW objects         |
| are CREATED        | are COMPOSED       | COMMUNICATE         |
|                    |                    |                     |
+--------------------+--------------------+---------------------+
| - Singleton        | - Adapter          | - Observer          |
```

```
| - Factory Method  | - Bridge           | - Strategy         |
| - Abstract Factory| - Composite        | - Command          |
| - Builder         | - Decorator        | - Iterator         |
| - Prototype       | - Facade           | - State            |
|                   | - Flyweight        | - Template Method  |
|                   | - Proxy            | - Chain of Resp.   |
|                   |                    | - Mediator         |
|                   |                    | - Memento          |
|                   |                    | - Visitor          |
|                   |                    | - Interpreter      |
+-------------------+--------------------+--------------------+
```

Creational Patterns

Creational patterns deal with object creation mechanisms. Instead of creating objects directly with `new`, these patterns provide ways to create objects that increase flexibility and reuse.

Backend relevance: Every backend system creates objects constantly: database connections, request handlers, configuration objects, service instances. Creational patterns help you control how, when, and how many of these objects get created.

Structural Patterns

Structural patterns deal with how classes and objects are composed to form larger structures. They help you build flexible and efficient class hierarchies.

Backend relevance: Backend systems are built from many components: caches, databases, APIs, message queues. Structural patterns help you combine these components cleanly, add new capabilities without modifying existing code, and simplify complex subsystems.

Behavioral Patterns

Behavioral patterns deal with how objects communicate and distribute responsibility. They help you define clear protocols for object interaction.

Backend relevance: Request processing, event handling, workflow orchestration, and state management are all communication problems. Behavioral patterns give you proven ways to handle them.

1.4 Architecture Patterns vs. Design Patterns

This is a distinction that confuses many developers. Architecture patterns and design patterns solve problems at different scales.

```
+-----------------------------------------------------------------+
|                     SCALE OF PATTERNS                           |
+-----------------------------------------------------------------+
|                                                                 |
|  ARCHITECTURE PATTERNS (System-level)                           |
|  +-------------------------------------------------------------+ |
|  | - Microservices     - Event-Driven Architecture            | |
|  | - Layered (N-Tier)  - CQRS                                 | |
|  | - Hexagonal         - Serverless                           | |
|  | - MVC               - Pipe and Filter                      | |
|  +-------------------------------------------------------------+ |
|  | Scope: Entire application or system                        | |
|  | Decided by: Architects, tech leads                         | |
|  | Changed: Rarely (expensive to change)                      | |
|  +-------------------------------------------------------------+ |
|                                                                 |
|  DESIGN PATTERNS (Class/Object-level)                           |
|  +-------------------------------------------------------------+ |
|  | - Singleton          - Factory Method                      | |
|  | - Observer           - Strategy                            | |
|  | - Builder            - Decorator                           | |
|  +-------------------------------------------------------------+ |
|  | Scope: A few classes or objects                            | |
|  | Decided by: Individual developers                          | |
|  | Changed: Frequently (low cost to change)                   | |
|  +-------------------------------------------------------------+ |
|                                                                 |
+-----------------------------------------------------------------+
```

Architecture patterns answer questions like:

- How do we structure the entire application?
- How do services communicate?
- How do we handle data flow across the system?

Design patterns answer questions like:

- How do I create this object flexibly?
- How do I add behavior to this class without modifying it?
- How do I decouple these two classes?

They work together. Inside a microservices architecture, each service uses design patterns internally. A layered architecture uses the Facade pattern at layer boundaries. An event-driven architecture uses the Observer pattern at its core.

A Concrete Example

Imagine you are building an e-commerce backend:

```
ARCHITECTURE DECISION:
  "We will use a microservices architecture
   with an API Gateway."

                +-------------+
                | API Gateway |
                +------+------+
                       |
           +------------+------------+
           |            |            |
      +----+----+  +----+----+  +---+-----+
      |  Order  |  | Payment |  | Inventory|
      | Service |  | Service |  | Service  |
      +---------+  +---------+  +----------+

DESIGN DECISIONS (inside each service):
  - Singleton for database connection pool
  - Factory Method for creating payment processors
  - Builder for constructing complex order objects
  - Observer for inventory change notifications
```

1.5 SOLID Principles Overview

Before diving into patterns, you need to understand SOLID. These five principles guide good object-oriented design, and almost every design pattern embodies one or more of them.

```
+---+-----------------------------------+----------------------------+
| # | PRINCIPLE                         | ONE-LINE SUMMARY           |
+---+-----------------------------------+----------------------------+
| S | Single Responsibility Principle   | One class, one reason to   |
|   |                                   | change.                    |
+---+-----------------------------------+----------------------------+
| O | Open/Closed Principle             | Open for extension, closed |
|   |                                   | for modification.          |
+---+-----------------------------------+----------------------------+
| L | Liskov Substitution Principle     | Subtypes must be           |
|   |                                   | substitutable for their    |
|   |                                   | base types.                |
+---+-----------------------------------+----------------------------+
| I | Interface Segregation Principle   | Many small interfaces beat |
|   |                                   | one fat interface.         |
+---+-----------------------------------+----------------------------+
| D | Dependency Inversion Principle    | Depend on abstractions,    |
|   |                                   | not concretions.           |
+---+-----------------------------------+----------------------------+
```

Let us look at each one with backend-relevant examples.

S - Single Responsibility Principle (SRP)

A class should have only one reason to change.

Java - Before (violates SRP):

```java
// BAD: This class does three things
public class UserService {

    public void createUser(String name, String email) {
        // 1. Validates input
        if (name == null || email == null) {
            throw new IllegalArgumentException("Name and email required");
        }

        // 2. Saves to database
        String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
        jdbcTemplate.update(sql, name, email);

        // 3. Sends welcome email
        String subject = "Welcome!";
        String body = "Hello " + name + ", welcome to our platform!";
        emailClient.send(email, subject, body);
    }
}
```

Java - After (follows SRP):

```java
// GOOD: Each class has one responsibility

public class UserValidator {
    public void validate(String name, String email) {
        if (name == null || email == null) {
            throw new IllegalArgumentException("Name and email required");
        }
    }
}

public class UserRepository {
    public void save(String name, String email) {
        String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
        jdbcTemplate.update(sql, name, email);
    }
}

public class WelcomeEmailSender {
    public void send(String name, String email) {
```

```
        String subject = "Welcome!";
        String body = "Hello " + name + ", welcome to our platform!";
        emailClient.send(email, subject, body);
    }
}

public class UserService {
    private final UserValidator validator;
    private final UserRepository repository;
    private final WelcomeEmailSender emailSender;

    public void createUser(String name, String email) {
        validator.validate(name, email);
        repository.save(name, email);
        emailSender.send(name, email);
    }
}
```

Python - Before (violates SRP):

```python
# BAD: This class does three things
class UserService:
    def create_user(self, name: str, email: str):
        # 1. Validates input
        if not name or not email:
            raise ValueError("Name and email required")

        # 2. Saves to database
        cursor = self.db.execute(
            "INSERT INTO users (name, email) VALUES (?, ?)",
            (name, email)
        )

        # 3. Sends welcome email
        self.email_client.send(
            to=email,
            subject="Welcome!",
            body=f"Hello {name}, welcome to our platform!"
        )
```

Python - After (follows SRP):

```python
# GOOD: Each class has one responsibility

class UserValidator:
    def validate(self, name: str, email: str):
        if not name or not email:
            raise ValueError("Name and email required")
```

```
class UserRepository:
    def __init__(self, db):
        self.db = db

    def save(self, name: str, email: str):
        self.db.execute(
            "INSERT INTO users (name, email) VALUES (?, ?)",
            (name, email)
        )

class WelcomeEmailSender:
    def __init__(self, email_client):
        self.email_client = email_client

    def send(self, name: str, email: str):
        self.email_client.send(
            to=email,
            subject="Welcome!",
            body=f"Hello {name}, welcome to our platform!"
        )

class UserService:
    def __init__(self, validator, repository, email_sender):
        self.validator = validator
        self.repository = repository
        self.email_sender = email_sender

    def create_user(self, name: str, email: str):
        self.validator.validate(name, email)
        self.repository.save(name, email)
        self.email_sender.send(name, email)
```

O - Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification.

This means you should be able to add new behavior without changing existing code. The Factory Method and Strategy patterns are direct applications of this principle.

L - Liskov Substitution Principle (LSP)

Objects of a superclass should be replaceable with objects of a subclass without breaking the application.

If your code works with a `PaymentProcessor` interface, it should work with `StripeProcessor`, `PayPalProcessor`, or any other implementation without surprises.

I - Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use.

Instead of one giant `DatabaseOperations` interface with 50 methods, create smaller interfaces like `Readable`, `Writable`, and `Deletable`.

D - Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Instead of `OrderService` depending directly on `MySQLDatabase`, both should depend on a `Database` interface.

```
BEFORE (Dependency Inversion violated):

  +---------------+         +------------------+
  | OrderService  |-------->| MySQLDatabase    |
  +---------------+         +------------------+
  High-level module         Low-level module
  depends on                (concrete class)
  concrete class

AFTER (Dependency Inversion applied):

  +---------------+         +------------------+
  | OrderService  |-------->|  <<interface>>   |
  +---------------+         |    Database      |
                            +------------------+
                                    ^
                                    |
                            +------------------+
                            | MySQLDatabase    |
                            +------------------+

  Both depend on the abstraction (Database interface).
  OrderService is easy to test with a mock database.
```

1.6 Why Patterns Matter in Backend Development

Backend development has unique challenges that patterns address directly.

1. Scalability

Backend systems must handle growing load. Patterns like Singleton (connection pools), Flyweight (shared objects), and Proxy (lazy loading) help you manage resources efficiently.

```
Without patterns:
  Request 1 --> new DBConnection()  --> DB
  Request 2 --> new DBConnection()  --> DB
  Request 3 --> new DBConnection()  --> DB
  ...
  Request 10000 --> new DBConnection() --> DB OVERWHELMED!

With Singleton + Object Pool:
  Request 1 ----+
  Request 2 ----+--> ConnectionPool (Singleton)
  Request 3 ----+       |
  ...           |       +--> Connection 1 --> DB
  Request 10000-+       +--> Connection 2 --> DB
                        +--> Connection 3 --> DB
                        (reuses connections)
```

2. Maintainability

Backend code lives for years. Patterns make code easier to understand and modify because every developer recognizes them.

```
WITHOUT patterns (in a code review):
  Developer A: "Why is there a private constructor here?"
  Developer B: "Oh, I wanted only one instance."
  Developer A: "But what about thread safety?"
  Developer B: "Uh..."

WITH patterns (in a code review):
  Developer A: "I see you used the Singleton pattern."
  Developer B: "Yes, with double-checked locking for thread safety."
  Developer A: "Got it. Looks good."
```

3. Team Communication

Patterns are a shared vocabulary. Saying "use a Builder for the query object" communicates more information in fewer words than describing the entire approach from scratch.

4. Testability

Many patterns naturally lead to testable code. Factory patterns let you inject mock objects. Strategy patterns let you swap algorithms in tests. Observer patterns let you verify events were fired.

5. Framework Integration

Modern frameworks are built on patterns. Spring uses Singleton, Factory, Proxy, and Template Method extensively. Django uses Template Method, Observer, and Strategy. Understanding patterns helps you understand your framework.

```
+-------------------------------+---------------------------+
| SPRING FRAMEWORK              | PATTERNS USED             |
+-------------------------------+---------------------------+
| @Component, @Service          | Singleton                 |
| @Bean                         | Factory Method            |
| @Autowired                    | Dependency Injection      |
| AOP / @Transactional          | Proxy                     |
| JdbcTemplate                  | Template Method           |
| ApplicationEvent              | Observer                  |
+-------------------------------+---------------------------+

+-------------------------------+---------------------------+
| DJANGO FRAMEWORK              | PATTERNS USED             |
+-------------------------------+---------------------------+
| Views (CBV)                   | Template Method           |
| Signals                       | Observer                  |
| Middleware                     | Chain of Responsibility   |
| QuerySet                      | Builder / Iterator        |
| Settings module               | Singleton (module-level)  |
+-------------------------------+---------------------------+
```

1.7 The Pattern Template

Throughout this book, every pattern follows a consistent template. Here is how to read it.

```
+-----------------------------------------------------------------+
|                    PATTERN TEMPLATE                             |
+-----------------------------------------------------------------+
|                                                                 |
|  PATTERN NAME                                                   |
|  Also known as: Alternative names                               |
|                                                                 |
|  CATEGORY: Creational / Structural / Behavioral                 |
|                                                                 |
|  INTENT                                                         |
|  One or two sentences describing what the pattern does.         |
|                                                                 |
|  PROBLEM                                                        |
|  The specific problem this pattern solves.                      |
|  Includes a "Before" code example showing the pain.             |
```

```
|                                                                  |
|  SOLUTION                                                        |
|  How the pattern solves the problem.                             |
|  Includes ASCII diagram of the pattern structure.                |
|                                                                  |
|  PARTICIPANTS                                                    |
|  The classes/interfaces involved and their roles.                |
|                                                                  |
|  CODE EXAMPLES                                                   |
|  Java and Python implementations side by side.                   |
|  Each with line-by-line explanations.                            |
|  Each with sample output.                                        |
|                                                                  |
|  REAL-WORLD BACKEND USE CASE                                     |
|  A concrete backend scenario where this pattern shines.          |
|                                                                  |
|  WHEN TO USE / WHEN NOT TO USE                                   |
|  Clear guidance on applicability.                                |
|                                                                  |
|  COMMON MISTAKES                                                 |
|  Pitfalls that developers frequently fall into.                  |
|                                                                  |
|  BEST PRACTICES                                                  |
|  Tips for getting the most out of the pattern.                   |
|                                                                  |
+------------------------------------------------------------------+
```

1.8 Anti-Patterns: When Good Intentions Go Wrong

An anti-pattern is a common response to a recurring problem that is usually ineffective and risks being counterproductive. In other words, it is a pattern that looks like a good idea but turns out to be a bad one.

Common Backend Anti-Patterns

1. God Object / God Class

One class that knows everything and does everything.

```
+----------------------------------------------------+
|              GOD CLASS (ANTI-PATTERN)              |
+----------------------------------------------------+
|                                                    |
|  class ApplicationManager:                         |
|      def handle_request()                          |
```

```
|      def validate_input()                        |
|      def query_database()                        |
|      def send_email()                            |
|      def generate_report()                       |
|      def process_payment()                       |
|      def manage_cache()                          |
|      def log_activity()                          |
|      def authenticate_user()                     |
|      def ... (200 more methods)                  |
|                                                  |
+--------------------------------------------------+
|  PROBLEM: Impossible to test, maintain, or       |
|  understand. Every change risks breaking         |
|  something unrelated.                            |
+--------------------------------------------------+
```

2. Spaghetti Code

Code with no clear structure, where control flow jumps around unpredictably.

3. Golden Hammer

Using the same pattern or technology for every problem because you are familiar with it.

```
Developer: "I know Singleton really well!"

  Need a config?      --> Singleton!
  Need a logger?      --> Singleton!
  Need a user object? --> Singleton!  <-- WRONG
  Need a request?     --> Singleton!  <-- VERY WRONG
  Need anything?      --> Singleton!  <-- STOP
```

4. Premature Optimization

Applying complex patterns to solve performance problems that do not exist yet.

5. Copy-Paste Programming

Duplicating code instead of finding the right abstraction.

1.9 When NOT to Use Patterns: The Over-Engineering Trap

This might be the most important section in the entire book.

Patterns are tools. Like any tool, they can be misused. The most common misuse is applying patterns where they are not needed.

Signs You Are Over-Engineering

```
+-----------------------------------------------------------------+
|              OVER-ENGINEERING WARNING SIGNS                      |
+-----------------------------------------------------------------+
|                                                                 |
|  1. "We might need this someday"                                |
|     --> YAGNI: You Aren't Gonna Need It                         |
|                                                                 |
|  2. You have more interfaces than implementations               |
|     --> An interface with one implementation is overhead         |
|                                                                 |
|  3. Simple operations require navigating 10+ files              |
|     --> Patterns should simplify, not complicate                 |
|                                                                 |
|  4. New team members take weeks to understand the code           |
|     --> If patterns make code harder to read, remove them        |
|                                                                 |
|  5. You are adding a pattern because the book says so            |
|     --> Only add patterns to solve real problems                 |
|                                                                 |
+-----------------------------------------------------------------+
```

The Pragmatic Approach

Java - Over-Engineered (DO NOT do this):

```java
// A simple greeting... with five files and three interfaces

public interface GreetingStrategy {
    String greet(String name);
}

public class EnglishGreetingStrategy implements GreetingStrategy {
    public String greet(String name) {
        return "Hello, " + name;
    }
}

public interface GreetingFactory {
    GreetingStrategy createStrategy(String language);
}

public class DefaultGreetingFactory implements GreetingFactory {
    public GreetingStrategy createStrategy(String language) {
```

```
        if ("en".equals(language)) {
            return new EnglishGreetingStrategy();
        }
        throw new UnsupportedOperationException();
    }
}

public class GreetingService {
    private final GreetingFactory factory;

    public GreetingService(GreetingFactory factory) {
        this.factory = factory;
    }

    public String greet(String name, String language) {
        return factory.createStrategy(language).greet(name);
    }
}

// All of this... to print "Hello, World"
```

Java - Just Right:

```java
// Simple and clear. Add patterns WHEN complexity demands it.
public class GreetingService {

    public String greet(String name) {
        return "Hello, " + name;
    }
}

// When you actually need multiple languages,
// THEN add the Strategy pattern.
```

The Decision Framework

Ask yourself these questions before applying a pattern:

```
+-----------------------------------+
|   Should I Use a Pattern Here?    |
+-----------------------------------+
          |
          v
+-----------------------------------+
| Do I have a real problem that     |
| a pattern solves?                 |
+---+-------------------------------+
    |                   |
   YES                  NO --> STOP. Keep it simple.
```

```
    |
    v
+-----------------------------------+
| Will this pattern make the code   |
| easier to understand?             |
+---+-------------------------------+
    |                   |
   YES                  NO --> STOP. Find a simpler solution.
    |
    v
+-----------------------------------+
| Is the added complexity worth     |
| the flexibility I gain?           |
+---+-------------------------------+
    |                   |
   YES                  NO --> STOP. Simpler is better.
    |
    v
+-----------------------------------+
| USE THE PATTERN.                  |
+-----------------------------------+
```

1.10 Patterns in This Book

This book covers the most important patterns for backend developers. Here is the roadmap:

```
PART I: CREATIONAL PATTERNS
+-------------------------------------------------------------+
| Ch 2:  Singleton       - One instance to rule them all      |
| Ch 3:  Factory Method  - Let subclasses decide              |
| Ch 4:  Abstract Factory- Families of related objects        |
| Ch 5:  Builder         - Step-by-step construction          |
| Ch 6:  Prototype       - Clone existing objects             |
+-------------------------------------------------------------+

PART II: STRUCTURAL PATTERNS (Future chapters)
+-------------------------------------------------------------+
| Adapter, Decorator, Proxy, Facade, Composite                |
+-------------------------------------------------------------+

PART III: BEHAVIORAL PATTERNS (Future chapters)
+-------------------------------------------------------------+
| Observer, Strategy, Command, Chain of Responsibility,       |
| Template Method, State                                      |
+-------------------------------------------------------------+
```

Each pattern chapter follows the same structure, so you always know where to find the information you need.

Quick Summary

Concept	Key Takeaway
Design patterns	Reusable solutions to recurring problems, not copy-paste code
Gang of Four	Four authors who cataloged 23 patterns in 1994
Three categories	Creational (object creation), Structural (composition), Behavioral (communication)
Architecture vs. design patterns	Architecture = system-level, Design = class/object-level
SOLID principles	Five principles that guide good OO design; patterns embody them
Anti-patterns	Common "solutions" that cause more problems than they solve
Over-engineering	The biggest risk of learning patterns; apply them only when needed

---

Key Points

1. Design patterns are proven solutions, not invented solutions. They are discovered by observing what works in practice.
2. The Gang of Four cataloged 23 patterns in three categories. Every pattern in this book traces back to their work.
3. SOLID principles are the foundation. If you understand SOLID, patterns will make intuitive sense.
4. Patterns are a communication tool. Saying "Singleton" is faster and more precise than explaining the entire concept from scratch.

5. The biggest danger of learning patterns is over-engineering. Apply a pattern only when it solves a real problem and makes the code simpler to understand.
6. Architecture patterns and design patterns work at different scales. You will use both, but this book focuses on design patterns.
7. Anti-patterns are just as important to recognize as patterns. Knowing what NOT to do saves you from expensive mistakes.

Practice Questions

1. What are the three categories of design patterns defined by the Gang of Four? Give one example pattern from each category and explain what kind of problem it addresses.
2. Explain the difference between an architecture pattern and a design pattern. Give a scenario where you would use both together in a backend system.
3. You join a team where a single `ApplicationManager` class has 3,000 lines of code and handles user authentication, payment processing, email sending, and report generation. Which SOLID principle does this violate? What would you do to fix it?
4. A junior developer on your team wants to implement the Abstract Factory pattern for a service that currently has only one type of database and no plans to add another. Is this a good idea? Explain your reasoning.
5. Name three backend-specific reasons why design patterns are important. For each reason, give a concrete example.

Exercises

Exercise 1: Identify the Principle

Look at the following code and identify which SOLID principle(s) it violates. Rewrite it to follow the principles.

```python
class ReportService:
    def generate_pdf_report(self, data):
        # Validate data
        if not data:
            raise ValueError("No data")
```

```
        # Query database for additional info
        extra = self.db.query("SELECT * FROM metadata")

        # Generate PDF
        pdf = PDFLibrary.create(data, extra)

        # Send email with attachment
        self.email.send("admin@company.com", "Report", attachment=pdf)

        # Log to file
        with open("report.log", "a") as f:
            f.write(f"Report generated at {datetime.now()}\n")

        return pdf
```

Exercise 2: Pattern or Over-Engineering?

For each scenario below, decide whether applying a design pattern is appropriate or over-engineering. Justify your answer.

1. A REST API endpoint that returns a hardcoded "OK" health check response. A colleague suggests using the Strategy pattern so you can swap health check implementations.
2. A payment service that currently supports Stripe. The product roadmap shows PayPal and Square integrations coming in the next quarter. A colleague suggests using the Factory Method pattern.
3. A configuration loader that reads from a YAML file at startup. A colleague suggests using the Singleton pattern to ensure only one config object exists.

Exercise 3: Map the Framework

Pick a backend framework you use (Spring, Django, Flask, Express, etc.). Find three places where the framework uses design patterns. For each one, name the pattern and explain what problem it solves in the framework.

What Is Next?

In the next chapter, we dive into our first design pattern: the Singleton. It is the simplest creational pattern, the most debated, and the most frequently misused. You will learn when it genuinely helps (database connection pools, configuration managers) and when it causes

more harm than good (hidden dependencies, testing nightmares). We will implement it in both Java and Python, handle thread safety, and see how modern frameworks like Spring handle singletons for you.

Chapter 2: The Singleton Pattern

What You Will Learn

- What the Singleton pattern is and what problem it solves
- How to implement Singleton in Java (private constructor, double-checked locking, enum)
- How to implement Singleton in Python (`__new__`, metaclass, module-level)
- How Spring handles singletons with `@Scope("singleton")`
- Why singletons can hurt testability and hide state
- Thread-safety concerns and how to address them
- When to use Singleton and when to avoid it

Why This Chapter Matters

The Singleton is the most well-known design pattern and often the first one developers learn. It is also the most controversial. Used correctly, it solves real problems: you do not want 50 database connection pools or multiple logger instances competing for the same file. Used incorrectly, it creates hidden dependencies, makes testing painful, and introduces subtle concurrency bugs.

Every backend developer will encounter singletons. Your framework almost certainly uses them. Understanding how they work, when they help, and when they hurt is essential knowledge.

The Problem

Imagine you are building a backend service that connects to a database. Every time a request arrives, your code creates a new database connection.

Before (No Singleton) - The Pain:

```java
// Java - Every request creates a new connection
public class OrderService {

    public Order getOrder(int id) {
        // NEW connection every single time
        Connection conn = DriverManager.getConnection(
            "jdbc:postgresql://localhost:5432/shop",
            "admin",
            "password"
        );
        // ... use connection
        conn.close();
        return order;
    }
}
```

```python
# Python - Every request creates a new connection
class OrderService:

    def get_order(self, order_id: int):
        # NEW connection every single time
        conn = psycopg2.connect(
            host="localhost",
            database="shop",
            user="admin",
            password="password"
        )
        # ... use connection
        conn.close()
        return order
```

What happens under load?

```
Request 1   --> new Connection() --> DB  (connection #1)
Request 2   --> new Connection() --> DB  (connection #2)
Request 3   --> new Connection() --> DB  (connection #3)
...
Request 500 --> new Connection() --> DB  (connection #500)
...
Request 1000 --> CRASH! Database refuses new connections.

+-------------------------------------------------------------+
| PostgreSQL default max_connections = 100                    |
| MySQL default max_connections = 151                         |
|                                                             |
| You ran out of connections because every request            |
| created a new one instead of sharing a pool.                |
+-------------------------------------------------------------+
```

The same problem appears with:

- Configuration objects (reading the same config file 1,000 times)
- Logger instances (multiple loggers fighting over the same log file)
- Cache managers (multiple caches storing duplicate data)
- Thread pools (creating pools of pools)

The Solution: Singleton Pattern

The Singleton pattern ensures a class has only one instance and provides a global point of access to it.

```
+-----------------------------------------------------------------+
|                    SINGLETON PATTERN                            |
+-----------------------------------------------------------------+
|                                                                 |
|  INTENT: Ensure a class has only one instance and provide      |
|          a global point of access to it.                        |
|                                                                 |
|  CATEGORY: Creational                                           |
|                                                                 |
+-----------------------------------------------------------------+

  +-----------------------------+
  |        Singleton            |
  +-----------------------------+
  | - instance: Singleton       |  <-- static/class variable
  +-----------------------------+
  | - Singleton()               |  <-- private constructor
  | + getInstance(): Singleton  |  <-- public static method
  | + operation(): void         |
  +-----------------------------+

  How it works:
  +------------+     +-------------------+
  | Client A  |---->|                   |
  +------------+     |   getInstance()   |---> Same instance
  +------------+     |                   |
  | Client B  |---->|                   |
  +------------+     +-------------------+
  +------------+
  | Client C  |---->  (all get the same object)
  +------------+
```

Java Implementation

Approach 1: Basic Singleton (Not Thread-Safe)

```java
public class DatabaseConnectionPool {

    // Step 1: Static variable to hold the single instance
    private static DatabaseConnectionPool instance;

    // Step 2: Private constructor prevents external instantiation
    private DatabaseConnectionPool() {
        System.out.println("Initializing connection pool...");
        // In real code: create a pool of database connections
    }

    // Step 3: Public static method to get the instance
    public static DatabaseConnectionPool getInstance() {
        if (instance == null) {                    // Line A
            instance = new DatabaseConnectionPool(); // Line B
        }
        return instance;
    }

    public void executeQuery(String sql) {
        System.out.println("Executing: " + sql);
    }
}
```

Line-by-line explanation:

- `private static DatabaseConnectionPool instance;` -- A static field holds the one and only instance. It starts as `null`.
- `private DatabaseConnectionPool()` -- The constructor is `private`. No code outside this class can call `new DatabaseConnectionPool()`.
- `getInstance()` checks if `instance` is `null`. If yes, it creates one. If no, it returns the existing one.

Usage and output:

```java
public class Main {
    public static void main(String[] args) {
        // Both variables point to the SAME object
        DatabaseConnectionPool pool1 = DatabaseConnectionPool.getInstance();
        DatabaseConnectionPool pool2 = DatabaseConnectionPool.getInstance();

        pool1.executeQuery("SELECT * FROM orders");
```

```
        System.out.println("Same instance? " + (pool1 == pool2));
    }
}
```

```
Output:
Initializing connection pool...
Executing: SELECT * FROM orders
Same instance? true
```

Notice that "Initializing connection pool..." prints only once, even though `getInstance()` is called twice.

The problem: This is NOT thread-safe. Two threads could both see `instance == null` at Line A and both create new instances at Line B.

```
Thread 1: if (instance == null)  --> true  (instance is null)
Thread 2: if (instance == null)  --> true  (still null, Thread 1 hasn't finished)
Thread 1: instance = new DatabaseConnectionPool()  --> creates instance #1
Thread 2: instance = new DatabaseConnectionPool()  --> creates instance #2!

Result: TWO instances exist. Singleton violated.
```

Approach 2: Thread-Safe with Double-Checked Locking

```java
public class DatabaseConnectionPool {

    // 'volatile' ensures all threads see the updated value
    private static volatile DatabaseConnectionPool instance;

    private final List<Connection> connections;

    private DatabaseConnectionPool() {
        System.out.println("Initializing connection pool...");
        connections = new ArrayList<>();
        for (int i = 0; i < 10; i++) {
            connections.add(createConnection());
        }
    }

    public static DatabaseConnectionPool getInstance() {
        // First check: avoid locking if instance already exists
        if (instance == null) {                           // Check 1
            synchronized (DatabaseConnectionPool.class) {  // Lock
                // Second check: another thread might have created
                // the instance between Check 1 and the Lock
```

```
                if (instance == null) {                    // Check 2
                    instance = new DatabaseConnectionPool();
                }
            }
        }
        return instance;
    }

    public Connection getConnection() {
        // Return a connection from the pool
        return connections.remove(0);
    }

    public void returnConnection(Connection conn) {
        connections.add(conn);
    }

    private Connection createConnection() {
        // Simplified - in real code, use actual DB connection
        return new Connection();
    }
}
```

Line-by-line explanation:

- `volatile` -- This keyword ensures that when one thread writes to `instance`, all other threads immediately see the new value. Without it, threads might cache a stale `null` value.
- First `if (instance == null)` (Check 1) -- A quick check without locking. If the instance already exists, we skip the expensive `synchronized` block entirely. This is why it is called "double-checked": we check twice.
- `synchronized (DatabaseConnectionPool.class)` -- Only one thread can enter this block at a time. This prevents two threads from creating two instances.
- Second `if (instance == null)` (Check 2) -- Inside the lock, we check again. Between Check 1 and acquiring the lock, another thread might have created the instance.

```
WHY DOUBLE-CHECKED LOCKING?

Without it (just synchronized):
  Thread 1: lock -> check -> create -> unlock
  Thread 2: lock -> check -> return  -> unlock   (SLOW: waits for lock)
  Thread 3: lock -> check -> return  -> unlock   (SLOW: waits for lock)
  Thread 4: lock -> check -> return  -> unlock   (SLOW: waits for lock)
  ... every call pays the locking cost

With double-checked locking:
  Thread 1: check1(null) -> lock -> check2(null) -> create -> unlock
  Thread 2: check1(null) -> lock -> check2(exists) -> return -> unlock
  Thread 3: check1(exists) -> return   (FAST: no lock needed)
```

```
  Thread 4: check1(exists) -> return   (FAST: no lock needed)
  ... after creation, no thread ever locks again
```

Approach 3: Enum Singleton (Recommended in Java)

Joshua Bloch, author of Effective Java, recommends this as the best way to implement Singleton in Java.

```java
public enum DatabaseConnectionPool {

    INSTANCE;  // The single instance

    private final List<Object> connections = new ArrayList<>();

    // Constructor runs once when the enum is loaded
    DatabaseConnectionPool() {
        System.out.println("Initializing connection pool...");
        for (int i = 0; i < 10; i++) {
            connections.add(createConnection());
        }
    }

    public Object getConnection() {
        if (connections.isEmpty()) {
            throw new RuntimeException("No available connections");
        }
        return connections.remove(0);
    }

    public void returnConnection(Object conn) {
        connections.add(conn);
    }

    private Object createConnection() {
        return new Object(); // Simplified
    }
}
```

Usage:

```java
public class Main {
    public static void main(String[] args) {
        // Access via INSTANCE - clean and simple
        Object conn = DatabaseConnectionPool.INSTANCE.getConnection();
        System.out.println("Got connection: " + conn);
        DatabaseConnectionPool.INSTANCE.returnConnection(conn);
    }
}
```

```
Output:
Initializing connection pool...
Got connection: java.lang.Object@1b6d3586
```

Why enum Singleton is best:

```
+-----------------------------------------------------------+
|  ENUM SINGLETON ADVANTAGES                                |
+-----------------------------------------------------------+
|                                                           |
|  1. Thread-safe by default (JVM handles it)               |
|  2. Serialization-safe (no duplicate on deserialization)  |
|  3. Reflection-safe (cannot create via reflection)        |
|  4. Concise - no boilerplate code                         |
|  5. Lazy initialization by the JVM                        |
|                                                           |
+-----------------------------------------------------------+
```

Python Implementation

Approach 1: Using `__new__`

```python
class DatabaseConnectionPool:
    """Singleton connection pool using __new__."""

    _instance = None

    def __new__(cls):
        if cls._instance is None:
            print("Initializing connection pool...")
            cls._instance = super().__new__(cls)
            cls._instance._connections = [f"conn_{i}" for i in range(10)]
        return cls._instance

    def get_connection(self) -> str:
        if not self._connections:
            raise RuntimeError("No available connections")
        return self._connections.pop()

    def return_connection(self, conn: str):
        self._connections.append(conn)
```

Line-by-line explanation:

- `_instance = None` -- Class variable that holds the single instance. `None` means no instance has been created yet.
- `__new__(cls)` -- This special method is called BEFORE `__init__`. It controls object creation. By overriding it, we control whether a new object is actually created.
- `if cls._instance is None` -- First time: `_instance` is `None`, so we create the object. Every subsequent time: `_instance` already exists, so we return it.
- `super().__new__(cls)` -- Calls the parent class `object.__new__()` to actually create the instance.
- We initialize `_connections` inside `__new__`, not in `__init__`, because `__init__` would run every time someone calls `DatabaseConnectionPool()`, resetting the connections.

Usage and output:

```python
pool1 = DatabaseConnectionPool()
pool2 = DatabaseConnectionPool()

conn = pool1.get_connection()
print(f"Got connection: {conn}")
print(f"Same instance? {pool1 is pool2}")
```

```
Output:
Initializing connection pool...
Got connection: conn_9
Same instance? True
```

Approach 2: Using a Metaclass

```python
class SingletonMeta(type):
    """A metaclass that creates Singleton instances."""

    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            # Create the instance using the normal mechanism
            instance = super().__call__(*args, **kwargs)
            cls._instances[cls] = instance
        return cls._instances[cls]


class AppConfig(metaclass=SingletonMeta):
    """Application configuration - singleton via metaclass."""

    def __init__(self):
```

```
        print("Loading configuration...")
        self.database_url = "postgresql://localhost:5432/myapp"
        self.cache_ttl = 300
        self.debug_mode = False
        self.max_retries = 3

    def get(self, key: str):
        return getattr(self, key, None)

    def set(self, key: str, value):
        setattr(self, key, value)
```

Line-by-line explanation:

- `class SingletonMeta(type)` -- A metaclass is a "class of a class." It controls how classes themselves are created and called.
- `_instances = {}` -- A dictionary that maps each class to its singleton instance. This allows the metaclass to be reused for multiple singleton classes.
- `__call__` -- This method is invoked when you write `AppConfig()`. The metaclass intercepts the call and returns the existing instance if one exists.

Usage and output:

```python
config1 = AppConfig()
config2 = AppConfig()

config1.set("debug_mode", True)
print(f"Debug mode from config2: {config2.get('debug_mode')}")
print(f"Same instance? {config1 is config2}")
```

```
Output:
Loading configuration...
Debug mode from config2: True
Same instance? True
```

Notice that "Loading configuration..." prints only once, and changing a value through `config1` is immediately visible through `config2` because they are the same object.

Approach 3: Module-Level Singleton (Pythonic Way)

In Python, a module is only loaded once. This means any object created at the module level is naturally a singleton.

```python
# config.py - The entire module IS the singleton

import json

class _AppConfig:
    """Internal config class. The singleton is the module-level instance."""

    def __init__(self, config_path: str = "config.json"):
        print("Loading configuration...")
        self._data = self._load(config_path)

    def _load(self, path: str) -> dict:
        try:
            with open(path) as f:
                return json.load(f)
        except FileNotFoundError:
            return {
                "database_url": "postgresql://localhost:5432/myapp",
                "cache_ttl": 300,
                "debug_mode": False,
            }

    def get(self, key: str, default=None):
        return self._data.get(key, default)

    def set(self, key: str, value):
        self._data[key] = value


# The singleton instance - created when the module is first imported
config = _AppConfig()
```

Usage:

```python
# service_a.py
from config import config

print(config.get("database_url"))

# service_b.py
from config import config

config.set("debug_mode", True)
```

Both `service_a.py` and `service_b.py` get the same `config` object because Python only executes the module code once.

```
+-----------------------------------------------------------+
|  PYTHONIC SINGLETON: MODULE-LEVEL APPROACH                |
+-----------------------------------------------------------+
```

```
|                                                          |
|  PROS:                                                   |
|  - Simplest approach                                     |
|  - No special classes or metaclasses needed               |
|  - Thread-safe (Python's import system handles it)        |
|  - Naturally discoverable                                |
|                                                          |
|  CONS:                                                   |
|  - Instance is created at import time (eager, not lazy)  |
|  - Harder to mock in tests (but doable with patch)       |
|                                                          |
+----------------------------------------------------------+
```

Thread Safety in Python

Python has the Global Interpreter Lock (GIL), which means only one thread executes Python bytecode at a time. However, the GIL does NOT make your singleton automatically thread-safe in all cases.

```python
import threading

class ThreadSafeConnectionPool:
    """Thread-safe singleton with a lock."""

    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                # Double-checked locking - same idea as Java
                if cls._instance is None:
                    print("Initializing connection pool...")
                    cls._instance = super().__new__(cls)
                    cls._instance._connections = [
                        f"conn_{i}" for i in range(10)
                    ]
                    cls._instance._pool_lock = threading.Lock()
        return cls._instance

    def get_connection(self) -> str:
        with self._pool_lock:
            if not self._connections:
                raise RuntimeError("No available connections")
            return self._connections.pop()

    def return_connection(self, conn: str):
        with self._pool_lock:
            self._connections.append(conn)
```

Spring Framework: @Scope("singleton")

Spring beans are singletons by default. You do not need to implement the pattern yourself.

```java
import org.springframework.stereotype.Component;

@Component  // Spring creates ONE instance and shares it
public class UserRepository {

    public UserRepository() {
        System.out.println("UserRepository created");
    }

    public User findById(int id) {
        // ... database query
        return new User(id, "John");
    }
}
```

```java
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    private final UserRepository userRepository;

    // Spring injects the SAME UserRepository instance everywhere
    public OrderService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

```java
@Service
public class PaymentService {

    private final UserRepository userRepository;

    // Same UserRepository instance as OrderService got
    public PaymentService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

```
Spring Container:
+-------------------------------------------------------------+
|                                                              |
|  Bean: userRepository (SINGLETON scope)                      |
|  +-----------------------------------------------------+  |
|  |  UserRepository instance                            |  |
|  +-----------------------------------------------------+  |
|        ^                    ^                                |
|        |                    |                                |
|  +-----+------+     +------+-------+                       |
|  | OrderService|     | PaymentService|                      |
|  +------------+     +--------------+                       |
|  (both share the same UserRepository)                       |
|                                                              |
+-------------------------------------------------------------+
```

You can explicitly set the scope:

```java
@Component
@Scope("singleton")   // This is the DEFAULT - same as no @Scope
public class CacheManager {
    // ...
}

@Component
@Scope("prototype")   // NEW instance every time it is injected
public class RequestHandler {
    // ...
}
```

```
+-----------------------------------+-----------------------------------+
|  SINGLETON SCOPE (default)        |  PROTOTYPE SCOPE                  |
+-----------------------------------+-----------------------------------+
|  One instance per container       |  New instance every injection     |
|  Shared across all consumers      |  Each consumer gets its own       |
|  Created at startup (eager)       |  Created on demand (lazy)         |
|  Good for: stateless services     |  Good for: stateful objects       |
+-----------------------------------+-----------------------------------+
```

Real-World Backend Use Case: Application Logger

A logger is one of the most natural uses of the Singleton pattern. You want exactly one logger instance to manage log files, formatting, and output destinations.

Java Implementation:

```java
public enum Logger {
    INSTANCE;

    private final List<String> logEntries = new ArrayList<>();
    private LogLevel currentLevel = LogLevel.INFO;

    public enum LogLevel {
        DEBUG, INFO, WARN, ERROR
    }

    public void setLevel(LogLevel level) {
        this.currentLevel = level;
    }

    public void debug(String message) {
        log(LogLevel.DEBUG, message);
    }

    public void info(String message) {
        log(LogLevel.INFO, message);
    }

    public void warn(String message) {
        log(LogLevel.WARN, message);
    }

    public void error(String message) {
        log(LogLevel.ERROR, message);
    }

    private void log(LogLevel level, String message) {
        if (level.ordinal() >= currentLevel.ordinal()) {
            String entry = String.format(
                "[%s] %s: %s",
                java.time.LocalDateTime.now(),
                level,
                message
            );
            logEntries.add(entry);
            System.out.println(entry);
        }
    }

    public List<String> getLogEntries() {
        return Collections.unmodifiableList(logEntries);
    }
}
```

Usage:

```java
// In OrderService.java
Logger.INSTANCE.info("Processing order #1234");

// In PaymentService.java
Logger.INSTANCE.info("Payment received for order #1234");

// In ErrorHandler.java
Logger.INSTANCE.error("Failed to connect to payment gateway");
```

```
Output:
[2024-01-15T10:30:45] INFO: Processing order #1234
[2024-01-15T10:30:46] INFO: Payment received for order #1234
[2024-01-15T10:30:47] ERROR: Failed to connect to payment gateway
```

Python Implementation:

```python
import datetime
from enum import IntEnum

class LogLevel(IntEnum):
    DEBUG = 0
    INFO = 1
    WARN = 2
    ERROR = 3

class Logger:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._log_entries = []
            cls._instance._level = LogLevel.INFO
        return cls._instance

    def set_level(self, level: LogLevel):
        self._level = level

    def debug(self, message: str):
        self._log(LogLevel.DEBUG, message)

    def info(self, message: str):
        self._log(LogLevel.INFO, message)

    def warn(self, message: str):
        self._log(LogLevel.WARN, message)

    def error(self, message: str):
        self._log(LogLevel.ERROR, message)
```

```
    def _log(self, level: LogLevel, message: str):
        if level >= self._level:
            entry = f"[{datetime.datetime.now()}] {level.name}: {message}"
            self._log_entries.append(entry)
            print(entry)

    @property
    def log_entries(self):
        return list(self._log_entries)
```

Usage:

```python
# In order_service.py
logger = Logger()
logger.info("Processing order #1234")

# In payment_service.py
logger = Logger()
logger.info("Payment received for order #1234")

# Both 'logger' variables are the SAME object
```

When Singletons Hurt

Singletons are not always the right choice. Here are the real dangers.

1. Hidden Dependencies

```java
// BAD: Hidden dependency on Singleton
public class OrderService {

    public void processOrder(Order order) {
        // Where does this dependency come from?
        // You cannot see it in the constructor.
        DatabaseConnectionPool.getInstance().executeQuery(
            "INSERT INTO orders ..."
        );
        Logger.INSTANCE.info("Order processed");
        CacheManager.getInstance().invalidate("orders");
    }
}

// BETTER: Explicit dependencies via constructor injection
```

```
public class OrderService {

    private final DatabaseConnectionPool pool;
    private final Logger logger;
    private final CacheManager cache;

    // Dependencies are VISIBLE and EXPLICIT
    public OrderService(DatabaseConnectionPool pool,
                        Logger logger,
                        CacheManager cache) {
        this.pool = pool;
        this.logger = logger;
        this.cache = cache;
    }

    public void processOrder(Order order) {
        pool.executeQuery("INSERT INTO orders ...");
        logger.info("Order processed");
        cache.invalidate("orders");
    }
}
```

2. Testing Nightmare

```java
// Hard to test: How do you replace the real database?
public class OrderServiceTest {

    @Test
    void testProcessOrder() {
        // PROBLEM: This uses the REAL database connection pool
        // You cannot replace it with a mock
        OrderService service = new OrderService();
        service.processOrder(new Order());
        // This test hits the real database!
    }
}
```

```python
# Hard to test: global state persists between tests
def test_logger_counts():
    logger = Logger()
    logger.info("Test message")

    # PROBLEM: If another test ran first, log_entries
    # already has entries from that test!
    assert len(logger.log_entries) == 1  # FAILS!
```

3. Global Mutable State

```
+-----------------------------------------------------------+
|  THE GLOBAL STATE PROBLEM                                 |
+-----------------------------------------------------------+
|                                                           |
|  Thread A: config.set("max_retries", 5)                   |
|  Thread B: config.set("max_retries", 3)                   |
|  Thread C: retries = config.get("max_retries")            |
|                                                           |
|  What does Thread C get? 5? 3? It depends on timing.    |
|  This is a race condition caused by shared mutable        |
|  state in a singleton.                                    |
|                                                           |
+-----------------------------------------------------------+
```

When to Use / When NOT to Use

```
+-----------------------------------+-----------------------------------+
|  USE SINGLETON WHEN               |  DO NOT USE SINGLETON WHEN        |
+-----------------------------------+-----------------------------------+
|                                   |                                   |
| - Exactly one instance makes      | - You need different instances    |
|   logical sense (connection       |   for different contexts (e.g.,   |
|   pool, config, logger)           |   per-request state)              |
|                                   |                                   |
| - Creating multiple instances     | - You are using it as a lazy      |
|   would waste resources or        |   way to avoid passing            |
|   cause conflicts                 |   dependencies                    |
|                                   |                                   |
| - The object is truly stateless   | - The singleton has mutable       |
|   or has read-only state          |   state accessed by multiple      |
|                                   |   threads                         |
|                                   |                                   |
| - Your framework manages the      | - You are making it a singleton   |
|   lifecycle for you (Spring)      |   "just in case"                  |
|                                   |                                   |
+-----------------------------------+-----------------------------------+
```

Common Mistakes

1. Forgetting thread safety. The basic `if (instance == null)` check is not atomic. In multi-threaded backend services, always use thread-safe approaches.

2. Using Singleton for everything. Not everything needs to be a singleton. Request objects, user sessions, and domain entities should NOT be singletons.
3. Storing request-specific state in a Singleton. A singleton is shared across all requests. If you store `currentUser` in a singleton, all users will see the same user.
4. Ignoring the testing cost. If your singleton cannot be mocked or replaced in tests, you will end up with slow, fragile, integration-like unit tests.
5. Singleton as a global variable. If you are using Singleton just to avoid passing objects around, you are using global variables with extra steps.

Best Practices

1. Prefer framework-managed singletons. In Spring, use `@Component` or `@Service`. In Python frameworks, use dependency injection. Let the framework handle the lifecycle.
2. Use constructor injection. Even if the underlying object is a singleton, inject it through constructors. This makes dependencies visible and testable.
3. Make singletons immutable when possible. If configuration is loaded once and never changes, make it read-only. This eliminates thread-safety concerns.
4. In Java, prefer enum Singleton. It is thread-safe, serialization-safe, and reflection-safe with zero boilerplate.
5. In Python, prefer module-level singletons. It is the most Pythonic approach and naturally thread-safe during import.
6. Document that a class is a Singleton. Not everyone will notice the private constructor or `__new__` override. A clear comment or docstring helps.

Quick Summary

Aspect	Details
Pattern name	Singleton
Category	Creational
Intent	Ensure exactly one instance exists

Problem it solves	Multiple instances wasting resources or causing conflicts
Java approaches	Private constructor + `getInstance()`, double-checked locking, enum (recommended)
Python approaches	`__new__`, metaclass, module-level (recommended)
Spring	Singleton scope is the default for all beans
Main danger	Hidden dependencies, testing difficulty, global mutable state

---

Key Points

1. The Singleton pattern guarantees exactly one instance of a class exists in the application.
2. In Java, the enum approach is the safest and most concise way to implement Singleton. It handles thread safety, serialization, and reflection automatically.
3. In Python, the module-level approach is the most idiomatic. A module is loaded once, so any object created at module level is naturally a singleton.
4. Thread safety is critical in backend applications. The basic `if null` check is not thread-safe. Use double-checked locking, enums, or framework support.
5. The biggest danger of Singleton is not the pattern itself but how developers misuse it: as a replacement for proper dependency injection, as a global variable, or as a place to store mutable shared state.
6. Modern frameworks like Spring manage singletons for you. Prefer framework-managed singletons over hand-rolled implementations.

Practice Questions

1. Why is the basic Singleton implementation (without synchronization) not thread-safe? Describe a specific scenario where two threads could create two instances.
2. Explain what the `volatile` keyword does in the double-checked locking Singleton in Java. What would happen without it?

3. A developer stores the currently authenticated user in a Singleton `SessionManager`. Why is this dangerous in a backend serving multiple users concurrently?
4. Compare the enum Singleton approach in Java with the module-level approach in Python. What are the advantages and disadvantages of each?
5. Your team uses Spring Boot. A junior developer writes a hand-rolled Singleton with `private static instance` and `getInstance()` for a repository class. What would you suggest instead, and why?

Exercises

Exercise 1: Thread-Safe Configuration Manager

Implement a thread-safe `ConfigurationManager` singleton in both Java and Python. It should:

- Load configuration from a properties/JSON file on first access
- Provide `get(key)` and `getOrDefault(key, default)` methods
- Be immutable after initialization (no `set` method)
- Be thread-safe

Exercise 2: Singleton vs. Dependency Injection

Refactor the following code to remove the Singleton dependency and use constructor injection instead. Then write a unit test that uses a mock.

```java
public class NotificationService {
    public void sendNotification(String userId, String message) {
        User user = UserRepository.getInstance().findById(userId);
        EmailClient.getInstance().send(user.getEmail(), message);
        Logger.getInstance().info("Notification sent to " + userId);
    }
}
```

Exercise 3: Connection Pool with Metrics

Extend the `DatabaseConnectionPool` singleton to track:

- Total connections created
- Currently active connections
- Peak concurrent connections

Make sure the metrics tracking is thread-safe. Implement in both Java and Python.

What Is Next?

In the next chapter, we explore the Factory Method pattern. While the Singleton controls how many instances exist, the Factory Method controls which class gets instantiated. You will learn how to decouple your code from concrete classes, making it easy to add new types (email, SMS, push notifications) without modifying existing code. This is the Open/Closed Principle in action.

Chapter 3: The Factory Method Pattern

What You Will Learn

- What the Factory Method pattern is and the problem it solves
- The difference between Simple Factory and Factory Method
- How to implement Factory Method in Java with a notification system example
- How to implement Factory Method in Python with a payment processing example
- How Spring's `@Bean` annotation acts as a factory method
- How Factory Method applies the Open/Closed Principle
- When to use Factory Method and when simpler approaches are better

Why This Chapter Matters

Every backend system creates objects. You create database connections, HTTP clients, notification senders, payment processors, serializers, and hundreds of other objects. The question is not whether you create objects but how you create them.

When your code is tightly coupled to concrete classes, adding a new type means modifying existing code. Modify existing code, and you risk breaking what already works. The Factory Method pattern breaks this coupling. It lets subclasses decide which class to instantiate, so you can add new types without touching a single line of existing code.

This is the Open/Closed Principle in action, and it is one of the most frequently used patterns in production backend systems.

The Problem

Imagine you are building a notification service. It starts simple: just send emails.

Before (Tightly Coupled) - Java:

```java
public class NotificationService {

    public void send(String type, String recipient, String message) {
        if ("email".equals(type)) {
            System.out.println("Connecting to SMTP server...");
            System.out.println("Sending email to " + recipient);
            System.out.println("Subject: Notification");
            System.out.println("Body: " + message);

        } else if ("sms".equals(type)) {
            System.out.println("Connecting to SMS gateway...");
            System.out.println("Sending SMS to " + recipient);
            System.out.println("Message: " + message);

        } else if ("push".equals(type)) {
            System.out.println("Connecting to push service...");
            System.out.println("Sending push to device " + recipient);
            System.out.println("Payload: " + message);
        }
    }
}
```

Before (Tightly Coupled) - Python:

```python
class NotificationService:

    def send(self, notification_type: str, recipient: str, message: str):
        if notification_type == "email":
            print("Connecting to SMTP server...")
            print(f"Sending email to {recipient}")
            print(f"Subject: Notification")
            print(f"Body: {message}")

        elif notification_type == "sms":
            print("Connecting to SMS gateway...")
            print(f"Sending SMS to {recipient}")
            print(f"Message: {message}")

        elif notification_type == "push":
            print("Connecting to push service...")
            print(f"Sending push to device {recipient}")
            print(f"Payload: {message}")
```

What is wrong with this code?

```
PROBLEMS WITH THE IF/ELSE APPROACH:

1. OPEN/CLOSED VIOLATION
   Adding Slack notifications means modifying this class.
   Adding WhatsApp means modifying it again.
   Every new type = change existing, tested code.

2. SINGLE RESPONSIBILITY VIOLATION
   This one method knows how to send emails, SMS, and push.
   It has three reasons to change.

3. TESTING DIFFICULTY
   You cannot test email sending without also loading
   SMS and push code.

4. CODE GROWTH
   After 10 notification types, this method is 200+ lines.

   send()
   +-- if "email"     (30 lines)
   +-- if "sms"       (25 lines)
   +-- if "push"      (20 lines)
   +-- if "slack"     (35 lines)
   +-- if "whatsapp"  (30 lines)
   +-- if "teams"     (25 lines)
   +-- if "webhook"   (40 lines)
   +-- ...
```

The Solution: Factory Method Pattern

The Factory Method pattern defines an interface for creating an object, but lets subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.

```
+-----------------------------------------------------------------+
|                 FACTORY METHOD PATTERN                          |
+-----------------------------------------------------------------+
|                                                                 |
|  INTENT: Define an interface for creating an object, but let  |
|          subclasses decide which class to instantiate.         |
|                                                                 |
|  CATEGORY: Creational                                           |
|                                                                 |
+-----------------------------------------------------------------+

  UML-like Structure:
```

```
+-------------------------+          +----------------------+
|     Creator             |          |     Product          |
|  (abstract class)       |          |   (interface)        |
+-------------------------+          +----------------------+
| + factoryMethod():      |          | + send()             |
|     Product  {abstract} |          | + validate()         |
| + someOperation()       |          +----------------------+
+-------------------------+                   ^
         ^                                    |
         |                           +---------+---------+
+--------+--------+                |                   |
|                 |         +------+------+    +-------+------+
|  EmailCreator   |         | EmailNotif  |    | SMSNotif     |
|  SMSCreator     |         +-------------+    +--------------+
|  PushCreator    |
+-----------------+

The Creator declares the factory method.
Each ConcreteCreator overrides it to return
a different ConcreteProduct.
```

Java Implementation: Notification System

Step 1: Define the Product Interface

```java
/**
 * Product interface - all notification types implement this.
 */
public interface Notification {

    /**
     * Validate the recipient format for this notification type.
     * Email expects an email address, SMS expects a phone number, etc.
     */
    boolean validate(String recipient);

    /**
     * Send the notification to the recipient.
     */
    void send(String recipient, String message);

    /**
     * Return the notification type name for logging.
     */
    String getType();
}
```

Step 2: Create Concrete Products

```java
/**
 * Concrete Product: Email notification.
 */
public class EmailNotification implements Notification {

    @Override
    public boolean validate(String recipient) {
        // Simple email validation
        boolean valid = recipient != null && recipient.contains("@");
        if (!valid) {
            System.out.println("Invalid email address: " + recipient);
        }
        return valid;
    }

    @Override
    public void send(String recipient, String message) {
        System.out.println("=== EMAIL NOTIFICATION ===");
        System.out.println("Connecting to SMTP server smtp.example.com:587");
        System.out.println("To: " + recipient);
        System.out.println("Subject: Notification");
        System.out.println("Body: " + message);
        System.out.println("Email sent successfully.");
    }

    @Override
    public String getType() {
        return "EMAIL";
    }
}
```

```java
/**
 * Concrete Product: SMS notification.
 */
public class SMSNotification implements Notification {

    @Override
    public boolean validate(String recipient) {
        boolean valid = recipient != null && recipient.matches("\\+?\\d{10,15}");
        if (!valid) {
            System.out.println("Invalid phone number: " + recipient);
        }
        return valid;
    }

    @Override
    public void send(String recipient, String message) {
```

```
        System.out.println("=== SMS NOTIFICATION ===");
        System.out.println("Connecting to Twilio API...");
        System.out.println("To: " + recipient);
        // SMS messages are limited to 160 characters
        String smsBody = message.length() > 160
            ? message.substring(0, 157) + "..."
            : message;
        System.out.println("Message: " + smsBody);
        System.out.println("SMS sent successfully.");
    }

    @Override
    public String getType() {
        return "SMS";
    }
}
```

```java
/**
 * Concrete Product: Push notification.
 */
public class PushNotification implements Notification {

    @Override
    public boolean validate(String recipient) {
        boolean valid = recipient != null && recipient.startsWith("device:");
        if (!valid) {
            System.out.println("Invalid device token: " + recipient);
        }
        return valid;
    }

    @Override
    public void send(String recipient, String message) {
        System.out.println("=== PUSH NOTIFICATION ===");
        System.out.println("Connecting to Firebase Cloud Messaging...");
        System.out.println("Device: " + recipient);
        System.out.println("Payload: {\"title\": \"Notification\", "
            + "\"body\": \"" + message + "\"}");
        System.out.println("Push notification sent successfully.");
    }

    @Override
    public String getType() {
        return "PUSH";
    }
}
```

Step 3: Define the Creator (Factory Method)

```java
/**
 * Creator - declares the factory method.
 *
 * The key insight: the Creator does NOT know which concrete
 * Notification it will create. That decision is deferred to
 * subclasses.
 */
public abstract class NotificationCreator {

    /**
     * The Factory Method - subclasses override this to create
     * the appropriate Notification type.
     */
    public abstract Notification createNotification();

    /**
     * Business logic that uses the factory method.
     * This method does NOT change when new notification types
     * are added.
     */
    public void sendNotification(String recipient, String message) {
        // The factory method creates the right type
        Notification notification = createNotification();

        System.out.println("Preparing " + notification.getType()
            + " notification...");

        // Validate before sending
        if (!notification.validate(recipient)) {
            System.out.println("Notification not sent: "
                + "invalid recipient.");
            return;
        }

        // Send the notification
        notification.send(recipient, message);

        System.out.println("Notification logged to audit trail.");
    }
}
```

Step 4: Create Concrete Creators

```java
public class EmailNotificationCreator extends NotificationCreator {
    @Override
    public Notification createNotification() {
        return new EmailNotification();
    }
}
```

```
public class SMSNotificationCreator extends NotificationCreator {
    @Override
    public Notification createNotification() {
        return new SMSNotification();
    }
}

public class PushNotificationCreator extends NotificationCreator {
    @Override
    public Notification createNotification() {
        return new PushNotification();
    }
}
```

Step 5: Usage

```java
public class Main {
    public static void main(String[] args) {
        // The client code works with creators through the base class
        NotificationCreator emailCreator = new EmailNotificationCreator();
        NotificationCreator smsCreator = new SMSNotificationCreator();
        NotificationCreator pushCreator = new PushNotificationCreator();

        emailCreator.sendNotification(
            "user@example.com",
            "Your order has been shipped!"
        );

        System.out.println();

        smsCreator.sendNotification(
            "+15551234567",
            "Your order has been shipped!"
        );

        System.out.println();

        pushCreator.sendNotification(
            "device:abc123xyz",
            "Your order has been shipped!"
        );
    }
}
```

Output:

```
Preparing EMAIL notification...
=== EMAIL NOTIFICATION ===
Connecting to SMTP server smtp.example.com:587
```

```
To: user@example.com
Subject: Notification
Body: Your order has been shipped!
Email sent successfully.
Notification logged to audit trail.

Preparing SMS notification...
=== SMS NOTIFICATION ===
Connecting to Twilio API...
To: +15551234567
Message: Your order has been shipped!
SMS sent successfully.
Notification logged to audit trail.

Preparing PUSH notification...
=== PUSH NOTIFICATION ===
Connecting to Firebase Cloud Messaging...
Device: device:abc123xyz
Payload: {"title": "Notification", "body": "Your order has been shipped!"}
Push notification sent successfully.
Notification logged to audit trail.
```

The Open/Closed Principle in Action

Now, suppose you need to add Slack notifications. Here is what you change:

```
FILES MODIFIED:  0   (zero!)
FILES CREATED:   2   (SlackNotification.java, SlackNotificationCreator.java)

BEFORE                              AFTER
+----------------------+            +----------------------+
| Notification (intf) |            | Notification (intf) |  <-- unchanged
+----------------------+            +----------------------+
| EmailNotification   |            | EmailNotification   |  <-- unchanged
| SMSNotification     |            | SMSNotification     |  <-- unchanged
| PushNotification    |            | PushNotification    |  <-- unchanged
+----------------------+            | SlackNotification   |  <-- NEW
                                    +----------------------+
+----------------------+            +----------------------+
| NotificationCreator |            | NotificationCreator |  <-- unchanged
+----------------------+            +----------------------+
| EmailCreator        |            | EmailCreator        |  <-- unchanged
| SMSCreator          |            | SMSCreator          |  <-- unchanged
| PushCreator         |            | PushCreator         |  <-- unchanged
+----------------------+            | SlackCreator        |  <-- NEW
                                    +----------------------+
```

```java
// NEW FILE: SlackNotification.java
public class SlackNotification implements Notification {
    @Override
    public boolean validate(String recipient) {
        return recipient != null && recipient.startsWith("#");
    }

    @Override
    public void send(String recipient, String message) {
        System.out.println("=== SLACK NOTIFICATION ===");
        System.out.println("Channel: " + recipient);
        System.out.println("Message: " + message);
    }

    @Override
    public String getType() {
        return "SLACK";
    }
}

// NEW FILE: SlackNotificationCreator.java
public class SlackNotificationCreator extends NotificationCreator {
    @Override
    public Notification createNotification() {
        return new SlackNotification();
    }
}
```

Zero existing files changed. Zero risk of breaking existing functionality. This is the power of Factory Method.

Python Implementation: Payment Processing

```python
from abc import ABC, abstractmethod


# ---- Product Interface ----

class PaymentProcessor(ABC):
    """Abstract product: all payment processors implement this."""

    @abstractmethod
    def validate_payment(self, amount: float) -> bool:
        """Check if the payment details are valid."""
        pass
```

```
    @abstractmethod
    def process(self, amount: float, currency: str) -> dict:
        """Process the payment and return a result."""
        pass

    @abstractmethod
    def get_name(self) -> str:
        """Return the processor name for logging."""
        pass


# ---- Concrete Products ----

class StripeProcessor(PaymentProcessor):
    """Concrete product: Stripe payment processing."""

    def validate_payment(self, amount: float) -> bool:
        if amount <= 0:
            print(f"Stripe: Invalid amount ${amount:.2f}")
            return False
        if amount > 999999.99:
            print("Stripe: Amount exceeds maximum")
            return False
        return True

    def process(self, amount: float, currency: str) -> dict:
        print(f"Stripe: Connecting to api.stripe.com...")
        print(f"Stripe: Processing ${amount:.2f} {currency}")
        print(f"Stripe: Payment successful!")
        return {
            "status": "success",
            "processor": "stripe",
            "transaction_id": "stripe_txn_abc123",
            "amount": amount,
            "currency": currency,
        }

    def get_name(self) -> str:
        return "Stripe"


class PayPalProcessor(PaymentProcessor):
    """Concrete product: PayPal payment processing."""

    def validate_payment(self, amount: float) -> bool:
        if amount <= 0:
            print(f"PayPal: Invalid amount ${amount:.2f}")
            return False
        if amount > 10000.00:
            print("PayPal: Amount exceeds single transaction limit")
            return False
        return True

    def process(self, amount: float, currency: str) -> dict:
        print(f"PayPal: Connecting to api.paypal.com...")
```

```
        print(f"PayPal: Creating payment for ${amount:.2f} {currency}")
        print(f"PayPal: Payment authorized!")
        return {
            "status": "success",
            "processor": "paypal",
            "transaction_id": "paypal_txn_xyz789",
            "amount": amount,
            "currency": currency,
        }

    def get_name(self) -> str:
        return "PayPal"


class SquareProcessor(PaymentProcessor):
    """Concrete product: Square payment processing."""

    def validate_payment(self, amount: float) -> bool:
        if amount <= 0:
            print(f"Square: Invalid amount ${amount:.2f}")
            return False
        return True

    def process(self, amount: float, currency: str) -> dict:
        print(f"Square: Connecting to connect.squareup.com...")
        print(f"Square: Charging ${amount:.2f} {currency}")
        print(f"Square: Charge completed!")
        return {
            "status": "success",
            "processor": "square",
            "transaction_id": "square_txn_def456",
            "amount": amount,
            "currency": currency,
        }

    def get_name(self) -> str:
        return "Square"


# ---- Creator (Factory Method) ----

class PaymentService(ABC):
    """
    Creator class with the factory method.

    The key insight: PaymentService contains business logic
    (validate, process, log) but does NOT know which specific
    PaymentProcessor it will use.
    """

    @abstractmethod
    def create_processor(self) -> PaymentProcessor:
        """Factory method - subclasses decide which processor to create."""
        pass
```

```
    def charge(self, amount: float, currency: str = "USD") -> dict:
        """
        Business logic that uses the factory method.
        This method NEVER changes when new processors are added.
        """
        processor = self.create_processor()
        print(f"\n--- Processing payment via {processor.get_name()} ---")

        # Validate
        if not processor.validate_payment(amount):
            return {"status": "failed", "reason": "validation_error"}

        # Process
        result = processor.process(amount, currency)

        # Log (common to all processors)
        print(f"Payment logged: {result['transaction_id']}")

        return result


# ---- Concrete Creators ----

class StripePaymentService(PaymentService):
    def create_processor(self) -> PaymentProcessor:
        return StripeProcessor()


class PayPalPaymentService(PaymentService):
    def create_processor(self) -> PaymentProcessor:
        return PayPalProcessor()


class SquarePaymentService(PaymentService):
    def create_processor(self) -> PaymentProcessor:
        return SquareProcessor()
```

Usage and output:

```python
def main():
    # Client code works with the base PaymentService class
    services = [
        StripePaymentService(),
        PayPalPaymentService(),
        SquarePaymentService(),
    ]

    for service in services:
        result = service.charge(49.99)
        print(f"Result: {result['status']}\n")
```

```
if __name__ == "__main__":
    main()
```

```
Output:

--- Processing payment via Stripe ---
Stripe: Connecting to api.stripe.com...
Stripe: Processing $49.99 USD
Stripe: Payment successful!
Payment logged: stripe_txn_abc123
Result: success

--- Processing payment via PayPal ---
PayPal: Connecting to api.paypal.com...
PayPal: Creating payment for $49.99 USD
PayPal: Payment authorized!
Payment logged: paypal_txn_xyz789
Result: success

--- Processing payment via Square ---
Square: Connecting to connect.squareup.com...
Square: Charging $49.99 USD
Square: Charge completed!
Payment logged: square_txn_def456
Result: success
```

Factory Method vs. Simple Factory

Many developers confuse Factory Method with Simple Factory. They are different.

Simple Factory (Not a GoF Pattern)

A Simple Factory is a single class with a method that creates objects based on a parameter. It is useful but does not follow the Open/Closed Principle.

```java
// Simple Factory - a single class with creation logic
public class NotificationFactory {

    public static Notification create(String type) {
        switch (type) {
            case "email": return new EmailNotification();
            case "sms":   return new SMSNotification();
            case "push":  return new PushNotification();
```

```
            default:
                throw new IllegalArgumentException(
                    "Unknown notification type: " + type
                );
        }
    }
}

// Usage:
Notification n = NotificationFactory.create("email");
n.send("user@example.com", "Hello!");
```

```python
# Simple Factory in Python
class NotificationFactory:

    @staticmethod
    def create(notification_type: str) -> Notification:
        factories = {
            "email": EmailNotification,
            "sms": SMSNotification,
            "push": PushNotification,
        }
        creator = factories.get(notification_type)
        if creator is None:
            raise ValueError(f"Unknown type: {notification_type}")
        return creator()

# Usage:
n = NotificationFactory.create("email")
n.send("user@example.com", "Hello!")
```

Side-by-Side Comparison

```
+-------------------------------+-------------------------------+
|       SIMPLE FACTORY          |       FACTORY METHOD          |
+-------------------------------+-------------------------------+
|                               |                               |
| One class creates all types   | Each type has its own creator |
|                               |                               |
| Uses if/else or switch        | Uses inheritance/polymorphism |
|                               |                               |
| Adding a type = modify the    | Adding a type = add new class |
| factory (violates OCP)        | (follows OCP)                 |
|                               |                               |
| Simpler to understand         | More classes, more flexible   |
|                               |                               |
| Good for: few types that      | Good for: many types, types   |
| rarely change                 | added frequently, frameworks  |
|                               |                               |
```

```
+-------------------------------+-------------------------------+

SIMPLE FACTORY:

  +------------------------+
  | NotificationFactory    |
  +------------------------+         +-------------------+
  | + create(type): Notif |-------->| Notification      |
  +------------------------+         +-------------------+
  | if "email" -> Email    |              ^    ^    ^
  | if "sms"   -> SMS      |              |    |    |
  | if "push"  -> Push     |         Email SMS Push
  +------------------------+

FACTORY METHOD:

  +----------------------+         +-------------------+
  | NotificationCreator |         | Notification      |
  | {abstract}          |-------->| {interface}       |
  +----------------------+         +-------------------+
  | + create(): Notif    |              ^    ^    ^
  +----------------------+              |    |    |
       ^      ^      ^            Email SMS Push
       |      |      |
  Email   SMS    Push
  Creator Creator Creator
```

When to use Simple Factory: You have a small, stable set of types (fewer than 5) that rarely changes. The simplicity is worth the OCP violation.

When to use Factory Method: You expect new types to be added over time, you want to follow the Open/Closed Principle, or you are building a framework/library.

Spring: @Bean as Factory Method

In Spring, the `@Bean` annotation on a method is essentially a factory method. Spring calls that method to create the bean.

```java
@Configuration
public class AppConfig {

    // This is a factory method - Spring calls it to create the bean
    @Bean
    public PaymentProcessor paymentProcessor() {
        String provider = System.getenv("PAYMENT_PROVIDER");
```

```
        switch (provider) {
            case "stripe":
                return new StripeProcessor("sk_live_xxx");
            case "paypal":
                return new PayPalProcessor("client_id", "secret");
            default:
                return new StripeProcessor("sk_test_xxx");
        }
    }

    // Another factory method
    @Bean
    public NotificationSender notificationSender() {
        return new EmailNotificationSender(
            smtpHost(),
            smtpPort()
        );
    }

    @Bean
    public String smtpHost() {
        return System.getenv("SMTP_HOST");
    }

    @Bean
    public int smtpPort() {
        return Integer.parseInt(
            System.getenv().getOrDefault("SMTP_PORT", "587")
        );
    }
}
```

```java
@Service
public class CheckoutService {

    private final PaymentProcessor processor;

    // Spring injects the object created by the @Bean factory method
    public CheckoutService(PaymentProcessor processor) {
        this.processor = processor;
    }

    public void checkout(Order order) {
        processor.process(order.getTotal(), "USD");
    }
}
```

```
Spring's Factory Method flow:

  @Configuration class
```

```
+---------------------------+
| @Bean                     |
| paymentProcessor()  ------+---> creates StripeProcessor
+---------------------------+            |
                                         v
@Service class                    +--------------+
+---------------------------+    | Spring       |
| CheckoutService           |<---| Container    |
| (PaymentProcessor proc)   |    | (injects)    |
+---------------------------+    +--------------+

The CheckoutService never knows whether it gets
Stripe, PayPal, or Square. It depends on the
PaymentProcessor abstraction.
```

Real-World Backend Use Case: Log Exporter

Different environments need different log export formats. Development uses console output. Staging uses JSON files. Production uses a centralized logging service.

```java
// Product interface
public interface LogExporter {
    void export(List<LogEntry> entries);
    String getFormat();
}

// Concrete products
public class ConsoleLogExporter implements LogExporter {
    public void export(List<LogEntry> entries) {
        entries.forEach(e -> System.out.println(
            e.getTimestamp() + " | " + e.getLevel()
            + " | " + e.getMessage()
        ));
    }
    public String getFormat() { return "CONSOLE"; }
}

public class JsonFileLogExporter implements LogExporter {
    public void export(List<LogEntry> entries) {
        // Write entries as JSON to a file
        String json = new ObjectMapper()
            .writeValueAsString(entries);
        Files.writeString(Path.of("logs/app.json"), json);
    }
    public String getFormat() { return "JSON_FILE"; }
}
```

```
public class CloudLogExporter implements LogExporter {
    public void export(List<LogEntry> entries) {
        // Send entries to centralized logging (e.g., Datadog)
        httpClient.post("https://api.datadog.com/v1/logs", entries);
    }
    public String getFormat() { return "CLOUD"; }
}

// Creator
public abstract class LogManager {

    public abstract LogExporter createExporter();

    public void flush(List<LogEntry> entries) {
        LogExporter exporter = createExporter();
        System.out.println("Exporting " + entries.size()
            + " entries via " + exporter.getFormat());
        exporter.export(entries);
    }
}

// Concrete creators - one per environment
public class DevLogManager extends LogManager {
    public LogExporter createExporter() {
        return new ConsoleLogExporter();
    }
}

public class StagingLogManager extends LogManager {
    public LogExporter createExporter() {
        return new JsonFileLogExporter();
    }
}

public class ProductionLogManager extends LogManager {
    public LogExporter createExporter() {
        return new CloudLogExporter();
    }
}
```

```python
# Python equivalent with environment-based selection

from abc import ABC, abstractmethod
from typing import List
import json
import os


class LogEntry:
    def __init__(self, timestamp: str, level: str, message: str):
        self.timestamp = timestamp
```

```
        self.level = level
        self.message = message


class LogExporter(ABC):
    @abstractmethod
    def export(self, entries: List[LogEntry]):
        pass

    @abstractmethod
    def get_format(self) -> str:
        pass


class ConsoleLogExporter(LogExporter):
    def export(self, entries: List[LogEntry]):
        for e in entries:
            print(f"{e.timestamp} | {e.level} | {e.message}")

    def get_format(self) -> str:
        return "CONSOLE"


class JsonFileLogExporter(LogExporter):
    def export(self, entries: List[LogEntry]):
        data = [vars(e) for e in entries]
        with open("logs/app.json", "w") as f:
            json.dump(data, f, indent=2)

    def get_format(self) -> str:
        return "JSON_FILE"


class LogManager(ABC):
    @abstractmethod
    def create_exporter(self) -> LogExporter:
        pass

    def flush(self, entries: List[LogEntry]):
        exporter = self.create_exporter()
        print(f"Exporting {len(entries)} entries "
              f"via {exporter.get_format()}")
        exporter.export(entries)


class DevLogManager(LogManager):
    def create_exporter(self) -> LogExporter:
        return ConsoleLogExporter()


class StagingLogManager(LogManager):
    def create_exporter(self) -> LogExporter:
        return JsonFileLogExporter()
```

```
# Select manager based on environment
def get_log_manager() -> LogManager:
    env = os.getenv("APP_ENV", "development")
    managers = {
        "development": DevLogManager,
        "staging": StagingLogManager,
    }
    manager_class = managers.get(env, DevLogManager)
    return manager_class()
```

When to Use / When NOT to Use

```
+-----------------------------------+-----------------------------------+
|  USE FACTORY METHOD WHEN          |  DO NOT USE WHEN                  |
+-----------------------------------+-----------------------------------+
|                                   |                                   |
| - You do not know ahead of time   | - You have only one concrete      |
|   which exact class to create     |   class and no plans for more     |
|                                   |                                   |
| - You want subclasses to specify  | - The creation logic is trivial   |
|   which objects to create         |   (just `new SomeClass()`)        |
|                                   |                                   |
| - You need to follow the          | - Adding the pattern makes the    |
|   Open/Closed Principle           |   code harder to understand       |
|                                   |                                   |
| - You are building a framework    | - You have fewer than 3 types     |
|   or library where users add      |   and they rarely change          |
|   their own types                 |   (Simple Factory is enough)      |
|                                   |                                   |
| - Different environments need     | - You are over-engineering a      |
|   different implementations       |   simple creation scenario        |
|   (dev/staging/prod)              |                                   |
|                                   |                                   |
+-----------------------------------+-----------------------------------+
```

Common Mistakes

1. Using Factory Method when Simple Factory is enough. If you have three notification types and they have not changed in two years, a Simple Factory is simpler and perfectly fine.

2. Creating a factory for every class. Not every object needs a factory. If there is only one implementation and no reason to expect others, use `new` directly.
3. Putting too much logic in the factory method. The factory method should create an object and return it. Business logic belongs in the Creator's other methods or in the Product itself.
4. Forgetting to program to the interface. If your client code references `EmailNotification` directly instead of `Notification`, the factory provides no benefit.
5. Confusing Factory Method with Abstract Factory. Factory Method creates ONE product. Abstract Factory creates FAMILIES of related products. We cover Abstract Factory in the next chapter.

Best Practices

1. Start with Simple Factory, evolve to Factory Method. Begin with the simplest approach. When you add the third or fourth type, refactor to Factory Method.
2. Name your factories clearly. `NotificationCreator` is better than `NotificationFactory` for the Factory Method pattern, because it emphasizes the creator/product relationship.
3. Use the Product interface everywhere. Client code should never reference concrete product classes. Always work with the `Notification` or `PaymentProcessor` interface.
4. Combine with Dependency Injection. In Spring or similar frameworks, let the DI container choose the concrete creator based on configuration. This moves the decision out of code entirely.
5. Keep the factory method simple. It should do one thing: create and return a product. If it needs complex setup, move that to the product's constructor or an initialization method.

Quick Summary

Aspect	Details
Pattern name	Factory Method
Category	Creational
Intent	Let subclasses decide which class to instantiate

Problem it solves	Tight coupling to concrete classes; if/else chains for object creation
Key principle	Open/Closed: add new types without modifying existing code
Participants	Creator (abstract), ConcreteCreator, Product (interface), ConcreteProduct
vs. Simple Factory	Simple Factory uses one class with if/else; Factory Method uses inheritance
Spring equivalent	`@Bean` methods in `@Configuration` classes

Key Points

1. Factory Method lets subclasses decide which class to instantiate. The Creator defines the factory method signature; ConcreteCreators provide the implementation.
2. The main benefit is the Open/Closed Principle. Adding a new product type requires only new classes, never modifying existing ones.
3. Simple Factory is not a GoF pattern but is useful for simple cases. Upgrade to Factory Method when you need extensibility.
4. In Spring, `@Bean` methods are factory methods. The framework calls them to create beans, and the rest of your code depends on abstractions.
5. Always program to the Product interface. If client code references concrete classes, the factory adds complexity without benefit.

Practice Questions

1. Explain the difference between Simple Factory and Factory Method. When would you choose one over the other?
2. How does the Factory Method pattern support the Open/Closed Principle? Walk through a specific example of adding a new type.
3. A developer writes a `NotificationFactory` with a 500-line `create()` method containing 20 if/else branches. What problems does this cause, and how would you refactor it?
4. Why is it important for client code to depend on the Product interface rather than concrete product classes? What breaks if it does not?

5. How does Spring's `@Bean` annotation relate to the Factory Method pattern?

Exercises

Exercise 1: Serialization Factory

Create a Factory Method implementation for data serialization. Support JSON, XML, and CSV formats. Each serializer should implement a `Serializer` interface with `serialize(data)` and `deserialize(text)` methods. Implement in both Java and Python.

Exercise 2: Simple Factory to Factory Method Refactoring

Start with this Simple Factory and refactor it to use the Factory Method pattern:

```python
class CacheFactory:
    @staticmethod
    def create(cache_type: str):
        if cache_type == "memory":
            return InMemoryCache()
        elif cache_type == "redis":
            return RedisCache("localhost", 6379)
        elif cache_type == "memcached":
            return MemcachedCache("localhost", 11211)
        else:
            raise ValueError(f"Unknown cache type: {cache_type}")
```

Exercise 3: Environment-Aware Factory

Build a Factory Method system that creates different database connection objects based on the environment (development, testing, production). Development should use SQLite, testing should use an in-memory H2/SQLite database, and production should use PostgreSQL. The factory should read the environment from a configuration source.

What Is Next?

In the next chapter, we level up from Factory Method to Abstract Factory. While Factory Method creates a single product, Abstract Factory creates entire families of related products.

You will see how to build a database abstraction layer that creates matched sets of connections, queries, and migrations for PostgreSQL and MySQL, and a cloud provider factory for AWS and GCP that creates consistent sets of storage, compute, and queue services.

Chapter 4: The Abstract Factory Pattern

What You Will Learn

- What the Abstract Factory pattern is and how it differs from Factory Method
- How to create families of related objects that must stay consistent
- How to implement Abstract Factory in Java with a database driver system
- How to implement Abstract Factory in Python with a cloud provider system
- When to use Abstract Factory and when it is overkill
- How Abstract Factory works with dependency injection in real frameworks

Why This Chapter Matters

In the previous chapter, you learned the Factory Method pattern, which creates a single product. But real backend systems rarely deal with isolated objects. A database layer needs a connection, a query builder, and a migration tool -- and they must all be for the same database engine. A cloud integration needs storage, compute, and queue services -- and they must all be from the same provider.

Mixing a PostgreSQL connection with a MySQL query builder would be a disaster. The Abstract Factory pattern prevents this by ensuring that families of related objects are always created together, consistently.

This pattern appears in database drivers, cloud SDKs, UI toolkits, and any system where objects must come from the same family.

The Problem

Imagine you are building a backend service that supports multiple databases. You need to create connections, execute queries, and run migrations. Each database has its own implementation of each component.

Before (Inconsistent Families) - The Pain:

```java
public class DatabaseService {

    public void setupDatabase(String dbType) {
        Connection connection;
        QueryBuilder queryBuilder;
        MigrationRunner migrationRunner;

        if ("postgres".equals(dbType)) {
            connection = new PostgresConnection();
            queryBuilder = new PostgresQueryBuilder();
            migrationRunner = new PostgresMigrationRunner();
        } else if ("mysql".equals(dbType)) {
            connection = new MySQLConnection();
            queryBuilder = new MySQLQueryBuilder();
            migrationRunner = new MySQLMigrationRunner();
        } else if ("sqlite".equals(dbType)) {
            connection = new SQLiteConnection();
            // BUG! Using MySQL query builder with SQLite connection!
            queryBuilder = new MySQLQueryBuilder();
            migrationRunner = new SQLiteMigrationRunner();
        }

        // ... use the components
    }
}
```

```
THE CONSISTENCY PROBLEM:

  PostgreSQL family:
  +-------------+  +-----------------+  +-------------------+
  | PgConnection|  | PgQueryBuilder  |  | PgMigrationRunner |
  +-------------+  +-----------------+  +-------------------+
       OK               OK                     OK

  MySQL family:
  +---------------+  +--------------------+  +----------------------+
  | MySQLConnection|  | MySQLQueryBuilder  |  | MySQLMigrationRunner|
  +---------------+  +--------------------+  +----------------------+
       OK                 OK                       OK

  MIXED (BUG!):
  +---------------+  +--------------------+  +----------------------+
```

```
  | SQLiteConnect |  | MySQLQueryBuilder  |  | SQLiteMigrationRun  |
  +---------------+  +--------------------+  +---------------------+
       SQLite            MySQL!                    SQLite
                     ^^^^^^^^^^^^
                     WRONG FAMILY!
                     MySQL syntax will fail
                     on a SQLite database.
```

The if/else approach lets developers accidentally mix components from different families. The Abstract Factory pattern makes this impossible.

The Solution: Abstract Factory Pattern

The Abstract Factory provides an interface for creating families of related objects without specifying their concrete classes.

```
+-----------------------------------------------------------------+
|              ABSTRACT FACTORY PATTERN                           |
+-----------------------------------------------------------------+
|                                                                 |
|  INTENT: Provide an interface for creating families of         |
|          related or dependent objects without specifying        |
|          their concrete classes.                                |
|                                                                 |
|  CATEGORY: Creational                                           |
|                                                                 |
+-----------------------------------------------------------------+

  UML-like Structure:

  +---------------------------+
  |   AbstractFactory         |
  |   (interface)             |
  +---------------------------+
  | + createConnection()      |
  | + createQueryBuilder()    |
  | + createMigrationRunner() |
  +---------------------------+
        ^             ^
        |             |
  +-----+-----+ +----+------+
  | PostgresDB | | MySQLDB   |
  | Factory    | | Factory   |
  +-----+-----+ +----+------+
        |             |
        |             |
```

```
  Creates:       Creates:
  - PgConnection    - MySQLConnection
  - PgQueryBuilder  - MySQLQueryBuilder
  - PgMigration     - MySQLMigration

  KEY GUARANTEE: Each factory only creates
  objects from the SAME family. You cannot
  accidentally mix PostgreSQL and MySQL.

  +-----------------------------------------------+
  |  Client code                                  |
  |                                               |
  |  DatabaseFactory factory = getFactory(config);|
  |  Connection conn = factory.createConnection();|
  |  QueryBuilder qb = factory.createQueryBuilder();
  |                                               |
  |  Client never knows if it is Postgres or MySQL|
  +-----------------------------------------------+
```

Java Implementation: Database Driver System

Step 1: Define Abstract Product Interfaces

```java
/**
 * Abstract Product A: Database Connection
 */
public interface DatabaseConnection {
    void connect(String host, int port, String database);
    void disconnect();
    boolean isConnected();
    String getDatabaseType();
}

/**
 * Abstract Product B: Query Builder
 */
public interface QueryBuilder {
    QueryBuilder select(String... columns);
    QueryBuilder from(String table);
    QueryBuilder where(String condition);
    QueryBuilder limit(int count);
    String build();
}

/**
```

```
 * Abstract Product C: Migration Runner
 */
public interface MigrationRunner {
    void createTable(String name, Map<String, String> columns);
    void addColumn(String table, String column, String type);
    void runMigration(String migrationSql);
    List<String> getPendingMigrations();
}
```

Step 2: Create Concrete Products for PostgreSQL

```java
// ---- PostgreSQL Family ----

public class PostgresConnection implements DatabaseConnection {
    private boolean connected = false;

    @Override
    public void connect(String host, int port, String database) {
        System.out.println("PostgreSQL: Connecting to "
            + host + ":" + port + "/" + database);
        System.out.println("PostgreSQL: Using libpq protocol v3");
        this.connected = true;
    }

    @Override
    public void disconnect() {
        System.out.println("PostgreSQL: Closing connection");
        this.connected = false;
    }

    @Override
    public boolean isConnected() {
        return connected;
    }

    @Override
    public String getDatabaseType() {
        return "PostgreSQL";
    }
}

public class PostgresQueryBuilder implements QueryBuilder {
    private final StringBuilder query = new StringBuilder();
    private final List<String> parts = new ArrayList<>();

    @Override
    public QueryBuilder select(String... columns) {
        query.append("SELECT ").append(String.join(", ", columns));
        return this;
    }
```

```
    @Override
    public QueryBuilder from(String table) {
        // PostgreSQL uses schema-qualified names
        query.append(" FROM public.").append(table);
        return this;
    }

    @Override
    public QueryBuilder where(String condition) {
        query.append(" WHERE ").append(condition);
        return this;
    }

    @Override
    public QueryBuilder limit(int count) {
        // PostgreSQL LIMIT syntax
        query.append(" LIMIT ").append(count);
        return this;
    }

    @Override
    public String build() {
        return query.toString() + ";";
    }
}

public class PostgresMigrationRunner implements MigrationRunner {
    @Override
    public void createTable(String name, Map<String, String> columns) {
        StringBuilder sql = new StringBuilder();
        sql.append("CREATE TABLE IF NOT EXISTS public.")
           .append(name).append(" (\n");

        List<String> colDefs = new ArrayList<>();
        for (Map.Entry<String, String> col : columns.entrySet()) {
            // PostgreSQL uses SERIAL for auto-increment
            String type = col.getValue()
                .replace("AUTO_INCREMENT", "SERIAL");
            colDefs.add("  " + col.getKey() + " " + type);
        }
        sql.append(String.join(",\n", colDefs));
        sql.append("\n);");

        System.out.println("PostgreSQL Migration:\n" + sql);
    }

    @Override
    public void addColumn(String table, String column, String type) {
        System.out.println("PostgreSQL: ALTER TABLE public." + table
            + " ADD COLUMN " + column + " " + type + ";");
    }

    @Override
    public void runMigration(String migrationSql) {
```

```
        System.out.println("PostgreSQL: Running migration within "
            + "transaction...");
        System.out.println("PostgreSQL: BEGIN;");
        System.out.println("PostgreSQL: " + migrationSql);
        System.out.println("PostgreSQL: COMMIT;");
    }

    @Override
    public List<String> getPendingMigrations() {
        return List.of("001_create_users", "002_add_email_index");
    }
}
```

Step 3: Create Concrete Products for MySQL

```java
// ---- MySQL Family ----

public class MySQLConnection implements DatabaseConnection {
    private boolean connected = false;

    @Override
    public void connect(String host, int port, String database) {
        System.out.println("MySQL: Connecting to "
            + host + ":" + port + "/" + database);
        System.out.println("MySQL: Using MySQL protocol v10");
        this.connected = true;
    }

    @Override
    public void disconnect() {
        System.out.println("MySQL: Closing connection");
        this.connected = false;
    }

    @Override
    public boolean isConnected() {
        return connected;
    }

    @Override
    public String getDatabaseType() {
        return "MySQL";
    }
}

public class MySQLQueryBuilder implements QueryBuilder {
    private final StringBuilder query = new StringBuilder();

    @Override
    public QueryBuilder select(String... columns) {
```

```
        query.append("SELECT ").append(String.join(", ", columns));
        return this;
    }

    @Override
    public QueryBuilder from(String table) {
        // MySQL uses backtick quoting
        query.append(" FROM `").append(table).append("`");
        return this;
    }

    @Override
    public QueryBuilder where(String condition) {
        query.append(" WHERE ").append(condition);
        return this;
    }

    @Override
    public QueryBuilder limit(int count) {
        // MySQL LIMIT syntax (same as PostgreSQL in this case)
        query.append(" LIMIT ").append(count);
        return this;
    }

    @Override
    public String build() {
        return query.toString() + ";";
    }
}

public class MySQLMigrationRunner implements MigrationRunner {
    @Override
    public void createTable(String name, Map<String, String> columns) {
        StringBuilder sql = new StringBuilder();
        sql.append("CREATE TABLE IF NOT EXISTS `")
           .append(name).append("` (\n");

        List<String> colDefs = new ArrayList<>();
        for (Map.Entry<String, String> col : columns.entrySet()) {
            // MySQL uses AUTO_INCREMENT directly
            colDefs.add("  `" + col.getKey() + "` " + col.getValue());
        }
        sql.append(String.join(",\n", colDefs));
        sql.append("\n) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");

        System.out.println("MySQL Migration:\n" + sql);
    }

    @Override
    public void addColumn(String table, String column, String type) {
        System.out.println("MySQL: ALTER TABLE `" + table
            + "` ADD COLUMN `" + column + "` " + type + ";");
    }

    @Override
```

```
    public void runMigration(String migrationSql) {
        System.out.println("MySQL: Running migration...");
        System.out.println("MySQL: " + migrationSql);
    }

    @Override
    public List<String> getPendingMigrations() {
        return List.of("001_create_users", "002_add_email_index");
    }
}
```

Step 4: Define the Abstract Factory

```java
/**
 * The Abstract Factory interface.
 *
 * Each method creates one product from the family.
 * Concrete factories implement ALL methods, ensuring
 * that all products come from the same family.
 */
public interface DatabaseFactory {
    DatabaseConnection createConnection();
    QueryBuilder createQueryBuilder();
    MigrationRunner createMigrationRunner();
}
```

Step 5: Create Concrete Factories

```java
/**
 * Concrete Factory: creates PostgreSQL family of products.
 */
public class PostgresFactory implements DatabaseFactory {

    @Override
    public DatabaseConnection createConnection() {
        return new PostgresConnection();
    }

    @Override
    public QueryBuilder createQueryBuilder() {
        return new PostgresQueryBuilder();
    }

    @Override
    public MigrationRunner createMigrationRunner() {
        return new PostgresMigrationRunner();
```

```
    }
}

/**
 * Concrete Factory: creates MySQL family of products.
 */
public class MySQLFactory implements DatabaseFactory {

    @Override
    public DatabaseConnection createConnection() {
        return new MySQLConnection();
    }

    @Override
    public QueryBuilder createQueryBuilder() {
        return new MySQLQueryBuilder();
    }

    @Override
    public MigrationRunner createMigrationRunner() {
        return new MySQLMigrationRunner();
    }
}
```

Step 6: Client Code

```java
/**
 * The client works ONLY with the abstract interfaces.
 * It never references PostgreSQL or MySQL classes directly.
 */
public class DatabaseService {

    private final DatabaseFactory factory;

    // The factory is injected - client does not choose it
    public DatabaseService(DatabaseFactory factory) {
        this.factory = factory;
    }

    public void initialize(String host, int port, String database) {
        // Create all components from the SAME family
        DatabaseConnection connection = factory.createConnection();
        QueryBuilder queryBuilder = factory.createQueryBuilder();
        MigrationRunner migrationRunner = factory.createMigrationRunner();

        // Connect
        connection.connect(host, port, database);

        // Run pending migrations
        List<String> pending = migrationRunner.getPendingMigrations();
```

```
        System.out.println("Pending migrations: " + pending);

        migrationRunner.createTable("users", Map.of(
            "id", "INT AUTO_INCREMENT PRIMARY KEY",
            "name", "VARCHAR(255) NOT NULL",
            "email", "VARCHAR(255) NOT NULL"
        ));

        // Build a query
        String query = queryBuilder
            .select("id", "name", "email")
            .from("users")
            .where("active = true")
            .limit(10)
            .build();

        System.out.println("Generated query: " + query);

        // Disconnect
        connection.disconnect();
    }
}
```

Step 7: Usage and Output

```java
public class Main {
    public static void main(String[] args) {
        // Choose factory based on configuration
        String dbType = System.getenv().getOrDefault("DB_TYPE", "postgres");

        DatabaseFactory factory;
        if ("mysql".equals(dbType)) {
            factory = new MySQLFactory();
        } else {
            factory = new PostgresFactory();
        }

        // Client code is identical regardless of which database
        DatabaseService service = new DatabaseService(factory);
        service.initialize("localhost", 5432, "myapp");
    }
}
```

Output with PostgreSQL factory:

```
PostgreSQL: Connecting to localhost:5432/myapp
PostgreSQL: Using libpq protocol v3
Pending migrations: [001_create_users, 002_add_email_index]
PostgreSQL Migration:
```

```
CREATE TABLE IF NOT EXISTS public.users (
  id INT SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL
);
Generated query: SELECT id, name, email FROM public.users WHERE active = true LIMIT 10;
PostgreSQL: Closing connection
```

Output with MySQL factory:

```
MySQL: Connecting to localhost:5432/myapp
MySQL: Using MySQL protocol v10
Pending migrations: [001_create_users, 002_add_email_index]
MySQL Migration:
CREATE TABLE IF NOT EXISTS `users` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(255) NOT NULL,
  `email` VARCHAR(255) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
Generated query: SELECT id, name, email FROM `users` WHERE active = true LIMIT 10;
MySQL: Closing connection
```

Notice how the client code (`DatabaseService`) is identical in both cases. The factory ensures every component belongs to the same database family.

Python Implementation: Cloud Provider System

```python
from abc import ABC, abstractmethod
from typing import List, Dict, Any


# ---- Abstract Products ----

class CloudStorage(ABC):
    """Abstract Product A: Object storage service."""

    @abstractmethod
    def create_bucket(self, name: str) -> str:
        pass

    @abstractmethod
    def upload(self, bucket: str, key: str, data: bytes) -> str:
        pass
```

```
    @abstractmethod
    def download(self, bucket: str, key: str) -> bytes:
        pass

    @abstractmethod
    def get_provider(self) -> str:
        pass


class CloudCompute(ABC):
    """Abstract Product B: Compute/VM service."""

    @abstractmethod
    def launch_instance(self, instance_type: str,
                        image: str) -> str:
        pass

    @abstractmethod
    def stop_instance(self, instance_id: str) -> None:
        pass

    @abstractmethod
    def list_instances(self) -> List[str]:
        pass

    @abstractmethod
    def get_provider(self) -> str:
        pass


class CloudQueue(ABC):
    """Abstract Product C: Message queue service."""

    @abstractmethod
    def create_queue(self, name: str) -> str:
        pass

    @abstractmethod
    def send_message(self, queue_url: str,
                     message: str) -> str:
        pass

    @abstractmethod
    def receive_messages(self, queue_url: str,
                         max_count: int = 10) -> List[str]:
        pass

    @abstractmethod
    def get_provider(self) -> str:
        pass


# ---- AWS Concrete Products ----
```

```
class S3Storage(CloudStorage):
    def create_bucket(self, name: str) -> str:
        url = f"s3://{name}"
        print(f"AWS S3: Created bucket {url}")
        return url

    def upload(self, bucket: str, key: str, data: bytes) -> str:
        url = f"s3://{bucket}/{key}"
        print(f"AWS S3: Uploaded {len(data)} bytes to {url}")
        return url

    def download(self, bucket: str, key: str) -> bytes:
        print(f"AWS S3: Downloading s3://{bucket}/{key}")
        return b"mock-data"

    def get_provider(self) -> str:
        return "AWS S3"


class EC2Compute(CloudCompute):
    def launch_instance(self, instance_type: str,
                        image: str) -> str:
        instance_id = "i-0abc123def456"
        print(f"AWS EC2: Launching {instance_type} "
              f"with AMI {image}")
        print(f"AWS EC2: Instance {instance_id} is running")
        return instance_id

    def stop_instance(self, instance_id: str) -> None:
        print(f"AWS EC2: Stopping instance {instance_id}")

    def list_instances(self) -> List[str]:
        return ["i-0abc123def456", "i-0xyz789ghi012"]

    def get_provider(self) -> str:
        return "AWS EC2"


class SQSQueue(CloudQueue):
    def create_queue(self, name: str) -> str:
        url = f"https://sqs.us-east-1.amazonaws.com/123456/{name}"
        print(f"AWS SQS: Created queue {name}")
        return url

    def send_message(self, queue_url: str,
                     message: str) -> str:
        msg_id = "sqs-msg-001"
        print(f"AWS SQS: Sent message to {queue_url}")
        return msg_id

    def receive_messages(self, queue_url: str,
                         max_count: int = 10) -> List[str]:
        print(f"AWS SQS: Polling {queue_url} "
              f"(max {max_count})")
        return ["message-1", "message-2"]
```

```
    def get_provider(self) -> str:
        return "AWS SQS"


# ---- GCP Concrete Products ----

class GCSStorage(CloudStorage):
    def create_bucket(self, name: str) -> str:
        url = f"gs://{name}"
        print(f"GCP GCS: Created bucket {url}")
        return url

    def upload(self, bucket: str, key: str, data: bytes) -> str:
        url = f"gs://{bucket}/{key}"
        print(f"GCP GCS: Uploaded {len(data)} bytes to {url}")
        return url

    def download(self, bucket: str, key: str) -> bytes:
        print(f"GCP GCS: Downloading gs://{bucket}/{key}")
        return b"mock-data"

    def get_provider(self) -> str:
        return "GCP Cloud Storage"


class GCECompute(CloudCompute):
    def launch_instance(self, instance_type: str,
                        image: str) -> str:
        instance_id = "gce-vm-abc123"
        print(f"GCP GCE: Launching {instance_type} "
              f"with image {image}")
        print(f"GCP GCE: Instance {instance_id} is running")
        return instance_id

    def stop_instance(self, instance_id: str) -> None:
        print(f"GCP GCE: Stopping instance {instance_id}")

    def list_instances(self) -> List[str]:
        return ["gce-vm-abc123", "gce-vm-def456"]

    def get_provider(self) -> str:
        return "GCP Compute Engine"


class PubSubQueue(CloudQueue):
    def create_queue(self, name: str) -> str:
        topic = f"projects/my-project/topics/{name}"
        print(f"GCP Pub/Sub: Created topic {name}")
        return topic

    def send_message(self, queue_url: str,
                     message: str) -> str:
        msg_id = "pubsub-msg-001"
        print(f"GCP Pub/Sub: Published message to {queue_url}")
```

```
        return msg_id

    def receive_messages(self, queue_url: str,
                         max_count: int = 10) -> List[str]:
        print(f"GCP Pub/Sub: Pulling from {queue_url} "
              f"(max {max_count})")
        return ["message-1", "message-2"]

    def get_provider(self) -> str:
        return "GCP Pub/Sub"


# ---- Abstract Factory ----

class CloudProviderFactory(ABC):
    """
    Abstract Factory: creates a family of cloud services.

    Each concrete factory creates Storage, Compute, and Queue
    services from the SAME cloud provider.
    """

    @abstractmethod
    def create_storage(self) -> CloudStorage:
        pass

    @abstractmethod
    def create_compute(self) -> CloudCompute:
        pass

    @abstractmethod
    def create_queue(self) -> CloudQueue:
        pass


# ---- Concrete Factories ----

class AWSFactory(CloudProviderFactory):
    def create_storage(self) -> CloudStorage:
        return S3Storage()

    def create_compute(self) -> CloudCompute:
        return EC2Compute()

    def create_queue(self) -> CloudQueue:
        return SQSQueue()


class GCPFactory(CloudProviderFactory):
    def create_storage(self) -> CloudStorage:
        return GCSStorage()

    def create_compute(self) -> CloudCompute:
        return GCECompute()
```

```
    def create_queue(self) -> CloudQueue:
        return PubSubQueue()


# ---- Client Code ----

class InfrastructureManager:
    """
    Client class that works with any cloud provider.
    It only depends on abstract interfaces.
    """

    def __init__(self, factory: CloudProviderFactory):
        self.storage = factory.create_storage()
        self.compute = factory.create_compute()
        self.queue = factory.create_queue()

    def deploy_application(self):
        print(f"\n=== Deploying with {self.storage.get_provider()} "
              f"/ {self.compute.get_provider()} "
              f"/ {self.queue.get_provider()} ===\n")

        # Step 1: Create storage
        bucket = self.storage.create_bucket("my-app-assets")
        self.storage.upload(
            "my-app-assets",
            "config/app.yaml",
            b"database_url: postgres://..."
        )

        # Step 2: Launch compute
        instance_id = self.compute.launch_instance(
            instance_type="medium",
            image="ubuntu-22.04"
        )

        # Step 3: Create message queue
        queue_url = self.queue.create_queue("task-queue")
        self.queue.send_message(queue_url, "deploy:v1.2.3")

        print("\nDeployment complete!")
```

Usage and output:

```python
def main():
    import os
    provider = os.getenv("CLOUD_PROVIDER", "aws")

    factories = {
        "aws": AWSFactory,
        "gcp": GCPFactory,
    }
```

```
    factory_class = factories.get(provider, AWSFactory)
    factory = factory_class()

    manager = InfrastructureManager(factory)
    manager.deploy_application()


if __name__ == "__main__":
    main()
```

Output with AWS factory:

```
=== Deploying with AWS S3 / AWS EC2 / AWS SQS ===

AWS S3: Created bucket s3://my-app-assets
AWS S3: Uploaded 25 bytes to s3://my-app-assets/config/app.yaml
AWS EC2: Launching medium with AMI ubuntu-22.04
AWS EC2: Instance i-0abc123def456 is running
AWS SQS: Created queue task-queue
AWS SQS: Sent message to https://sqs.us-east-1.amazonaws.com/123456/task-queue

Deployment complete!
```

Output with GCP factory:

```
=== Deploying with GCP Cloud Storage / GCP Compute Engine / GCP Pub/Sub ===

GCP GCS: Created bucket gs://my-app-assets
GCP GCS: Uploaded 25 bytes to gs://my-app-assets/config/app.yaml
GCP GCE: Launching medium with image ubuntu-22.04
GCP GCE: Instance gce-vm-abc123 is running
GCP Pub/Sub: Created topic task-queue
GCP Pub/Sub: Published message to projects/my-project/topics/task-queue

Deployment complete!
```

Factory Method vs. Abstract Factory

```
+-------------------------------+-------------------------------+
|       FACTORY METHOD          |       ABSTRACT FACTORY        |
+-------------------------------+-------------------------------+
|                               |                               |
```

```
| Creates ONE product           | Creates FAMILIES of products  |
|                               |                               |
| Uses inheritance              | Uses composition              |
| (subclass overrides method)   | (factory object is passed in) |
|                               |                               |
| One factory method per        | Multiple factory methods in   |
| creator class                 | one factory interface          |
|                               |                               |
| Example: create a single      | Example: create a matched     |
| Notification                  | set of Connection +            |
|                               | QueryBuilder + Migration       |
|                               |                               |
| Simpler                       | More complex, more powerful    |
|                               |                               |
+-------------------------------+-------------------------------+

  FACTORY METHOD:

  Creator                Product
  +---------------+      +----------+
  | create(): P   |----->| Product  |
  +---------------+      +----------+
        ^                     ^
        |                     |
  EmailCreator          EmailProduct


  ABSTRACT FACTORY:

  Factory                    Product A    Product B    Product C
  +--------------------+    +--------+  +--------+  +--------+
  | createA(): A       |--->|   A    |  |   B    |  |   C    |
  | createB(): B       |--->+--------+  +--------+  +--------+
  | createC(): C       |--->    ^           ^           ^
  +--------------------+        |           |           |
        ^                  Concrete A  Concrete B  Concrete C
        |                  (all from the SAME family)
  ConcreteFactory
```

Real-World Backend Use Case: Multi-Tenant Configuration

In a SaaS application, different tenants might need different infrastructure: one on AWS, another on GCP, a third on Azure. The Abstract Factory lets you configure each tenant independently.

```java
public class TenantService {

    private final Map<String, CloudProviderFactory> tenantFactories;

    public TenantService() {
        this.tenantFactories = new HashMap<>();
    }

    public void registerTenant(String tenantId,
                               CloudProviderFactory factory) {
        tenantFactories.put(tenantId, factory);
    }

    public void handleRequest(String tenantId, Request request) {
        CloudProviderFactory factory = tenantFactories.get(tenantId);
        if (factory == null) {
            throw new RuntimeException("Unknown tenant: " + tenantId);
        }

        // All services for this tenant come from the same provider
        CloudStorage storage = factory.createStorage();
        CloudQueue queue = factory.createQueue();

        // Process the request using the tenant's cloud provider
        storage.upload(
            tenantId + "-bucket",
            request.getKey(),
            request.getData()
        );
        queue.sendMessage(
            tenantId + "-queue",
            "process:" + request.getKey()
        );
    }
}

// Configuration
TenantService service = new TenantService();
service.registerTenant("acme-corp", new AWSFactory());
service.registerTenant("globex-inc", new GCPFactory());
```

```
MULTI-TENANT ABSTRACT FACTORY:

  Tenant: "acme-corp"           Tenant: "globex-inc"
  +-------------------+        +-------------------+
  | AWSFactory        |        | GCPFactory        |
  +-------------------+        +-------------------+
  | S3Storage         |        | GCSStorage        |
  | EC2Compute        |        | GCECompute        |
  | SQSQueue          |        | PubSubQueue       |
  +-------------------+        +-------------------+

  Each tenant gets a consistent family of services.
```

```
  No mixing AWS storage with GCP compute.
```

When to Use / When NOT to Use

```
+-----------------------------------+-----------------------------------+
|  USE ABSTRACT FACTORY WHEN        |  DO NOT USE WHEN                  |
+-----------------------------------+-----------------------------------+
|                                   |                                   |
| - You have families of related    | - Objects in the "family" are     |
|   objects that must be used       |   independent and do not need     |
|   together consistently          |   to match                       |
|                                   |                                   |
| - You want to enforce that all    | - You have only one product type |
|   products come from the same     |   (use Factory Method instead)    |
|   family                         |                                  |
|                                   |                                   |
| - You need to support multiple    | - The number of families will     |
|   variants (DB engines, cloud     |   never grow beyond one           |
|   providers, OS platforms)        |                                   |
|                                   |                                   |
| - Client code should be           | - Adding the pattern introduces   |
|   independent of how products     |   more complexity than the        |
|   are created and composed        |   problem warrants                |
|                                   |                                   |
+-----------------------------------+-----------------------------------+
```

Common Mistakes

1. Using Abstract Factory when you only have one product type. If you only create connections (no query builders, no migration runners), Factory Method is simpler and sufficient.
2. Adding a new product to the family later. If you add a `createBackupService()` method to the abstract factory, every concrete factory must implement it. This can be a significant change if you have many families. Plan your product interfaces carefully.
3. Making factories too granular. If every tiny object gets its own factory method, the factory interface becomes bloated. Group related objects logically.
4. Forgetting that Abstract Factory is about consistency. The whole point is that products come in matched sets. If your products do not need to match, you do not need Abstract

Factory.

5. Creating factories with state. Factories should ideally be stateless. They create products; they do not manage them. State management belongs in the client or a separate manager.

Best Practices

1. Keep factory interfaces small. Three to five creation methods per factory is typical. If you have more, consider splitting into multiple factories.
2. Use dependency injection for the factory itself. Pass the factory to client code via the constructor. This makes it easy to switch families via configuration.
3. Consider combining with Singleton. Factories are often stateless and can be singletons. In Spring, they are singletons by default.
4. Name factories after the family, not the pattern. `PostgresFactory` is better than `AbstractDatabaseFactoryImpl`. The name should communicate which family it creates.
5. Test with a test/mock factory. Create an `InMemoryFactory` that returns simple, fast implementations for testing. This is one of the biggest practical benefits of the pattern.

Quick Summary

Aspect	Details
Pattern name	Abstract Factory
Category	Creational
Intent	Create families of related objects without specifying concrete classes
Problem it solves	Mixing objects from different families (PostgreSQL connection with MySQL query builder)
Key guarantee	All products from one factory belong to the same family
vs. Factory Method	Factory Method creates one product; Abstract Factory creates families

Participants	AbstractFactory, ConcreteFactory, AbstractProduct, ConcreteProduct, Client

Key Points

1. Abstract Factory creates families of related objects. The key word is "families" -- objects that must work together.
2. The pattern guarantees consistency. A PostgreSQL factory can only create PostgreSQL products. It is impossible to accidentally mix families.
3. Adding a new family is easy. Create a new concrete factory and new concrete products. No existing code changes.
4. Adding a new product to all families is hard. Every concrete factory must be updated. This is the main trade-off.
5. Client code depends only on abstract interfaces. It never knows whether it is using PostgreSQL or MySQL, AWS or GCP.

Practice Questions

1. What is the key difference between Factory Method and Abstract Factory? When would you choose one over the other?
2. In the database example, what happens if you need to add SQLite support? What new classes do you create? What existing code do you modify?
3. Why is adding a new product type (e.g., `createBackupService()`) to an Abstract Factory more disruptive than adding a new family? How would you mitigate this?
4. A colleague proposes using Abstract Factory for an application that has one database engine (PostgreSQL) and no plans to support others. Is this appropriate? Why or why not?
5. How does Abstract Factory help with testing? Describe how you would create a test factory for the cloud provider example.

Exercises

Exercise 1: Add Azure Support

Extend the Python cloud provider example to support Microsoft Azure. Create `AzureFactory` with `AzureBlobStorage`, `AzureVMCompute`, and `AzureServiceBusQueue`. The client code (`InfrastructureManager`) should work without any changes.

Exercise 2: Document Format Factory

Create an Abstract Factory for document generation. The factory should create three products: `HeaderFormatter`, `BodyFormatter`, and `FooterFormatter`. Implement two families: `HTMLDocumentFactory` (produces HTML tags) and `MarkdownDocumentFactory` (produces Markdown syntax). Build a `DocumentGenerator` client that uses the factory to produce a complete document.

Exercise 3: Test Factory

For the Java database example, create an `InMemoryDatabaseFactory` that returns lightweight in-memory implementations of `DatabaseConnection`, `QueryBuilder`, and `MigrationRunner`. Use this factory to write a unit test for `DatabaseService` that does not require a real database.

What Is Next?

In the next chapter, we tackle the Builder pattern. While Abstract Factory creates entire families of objects, the Builder pattern handles a different problem: constructing a single complex object step by step. You will see how Builder eliminates the "telescoping constructor" anti-pattern, build fluent APIs for query construction and HTTP request building, and learn how Lombok's `@Builder` annotation does the heavy lifting for you in Java.

Chapter 5: The Builder Pattern

What You Will Learn

- What the Builder pattern is and the "telescoping constructor" problem it solves
- How to implement Builder in Java with a QueryBuilder and HttpRequestBuilder
- How to implement Builder in Python with a ReportBuilder
- What a fluent interface is and how Builder enables it
- The Director pattern and when to use it
- How Lombok's `@Builder` annotation generates builders automatically
- When NOT to use Builder (simple objects with few parameters)

Why This Chapter Matters

Every backend developer has encountered a class with too many constructor parameters. A database query might need a table, columns, conditions, sorting, limits, joins, and grouping. An HTTP request needs a URL, method, headers, body, timeout, retry policy, and authentication. A report needs a title, date range, format, filters, sorting, and pagination.

When a constructor has 10 or 15 parameters, the code becomes unreadable and error-prone. Which parameter was the timeout again -- the seventh or the eighth? The Builder pattern solves this by letting you construct complex objects step by step, with clear, readable code that makes mistakes nearly impossible.

This pattern is everywhere in backend development: SQL query builders, HTTP client libraries, test data factories, and configuration objects all use it.

The Problem: Telescoping Constructor

The "telescoping constructor" anti-pattern happens when you create multiple constructors with increasing numbers of parameters to handle optional values.

Before (Telescoping Constructor) - Java:

```java
public class HttpRequest {

    private final String url;
    private final String method;
    private final Map<String, String> headers;
    private final String body;
    private final int timeoutMs;
    private final int maxRetries;
    private final boolean followRedirects;
    private final String authToken;
    private final String contentType;
    private final boolean verifySsl;
    private final String proxyHost;
    private final int proxyPort;

    // Constructor with ALL parameters - who can read this?
    public HttpRequest(String url, String method,
                       Map<String, String> headers,
                       String body, int timeoutMs,
                       int maxRetries, boolean followRedirects,
                       String authToken, String contentType,
                       boolean verifySsl, String proxyHost,
                       int proxyPort) {
        this.url = url;
        this.method = method;
        this.headers = headers;
        this.body = body;
        this.timeoutMs = timeoutMs;
        this.maxRetries = maxRetries;
        this.followRedirects = followRedirects;
        this.authToken = authToken;
        this.contentType = contentType;
        this.verifySsl = verifySsl;
        this.proxyHost = proxyHost;
        this.proxyPort = proxyPort;
    }
}

// Usage - can you tell what each parameter means?
HttpRequest request = new HttpRequest(
    "https://api.example.com/users",  // url
    "POST",                            // method
    null,                              // headers
    "{\"name\": \"John\"}",           // body
    5000,                              // timeout? retries?
    3,                                 // retries? timeout?
    true,                              // follow redirects?
    "Bearer abc123",                   // auth token
```

```
    "application/json",                // content type
    true,                              // verify SSL?
    null,                              // proxy host
    0                                  // proxy port
);
```

Before (Telescoping Constructor) - Python:

```python
class HttpRequest:
    def __init__(self, url, method="GET", headers=None,
                 body=None, timeout_ms=30000, max_retries=0,
                 follow_redirects=True, auth_token=None,
                 content_type=None, verify_ssl=True,
                 proxy_host=None, proxy_port=None):
        self.url = url
        self.method = method
        self.headers = headers or {}
        self.body = body
        self.timeout_ms = timeout_ms
        self.max_retries = max_retries
        self.follow_redirects = follow_redirects
        self.auth_token = auth_token
        self.content_type = content_type
        self.verify_ssl = verify_ssl
        self.proxy_host = proxy_host
        self.proxy_port = proxy_port

# Usage - slightly better with keyword args, but still messy
request = HttpRequest(
    "https://api.example.com/users",
    method="POST",
    body='{"name": "John"}',
    timeout_ms=5000,
    max_retries=3,
    auth_token="Bearer abc123",
    content_type="application/json",
)
```

```
THE TELESCOPING CONSTRUCTOR PROBLEMS:

  1. READABILITY
     new HttpRequest("url", "POST", null, body, 5000, 3, true, ...)
     Which parameter is which? You have to count positions.

  2. PARAMETER CONFUSION
     Was 5000 the timeout or the max retries?
     Easy to swap adjacent parameters of the same type.

  3. NULL FILLER
     You must pass null for optional parameters you do not need.
     new HttpRequest("url", "GET", null, null, 5000, 0, true,
```

```
                      null, null, true, null, 0)

  4. IMMUTABILITY
     Setters solve readability but sacrifice immutability.
     A half-built mutable object is a bug waiting to happen.

  5. VALIDATION
     When do you validate? In the constructor? What if
     required parameters are missing? With 12 parameters,
     validation logic becomes a mess.
```

The Solution: Builder Pattern

The Builder pattern separates the construction of a complex object from its representation, allowing the same construction process to create different representations.

```
+-----------------------------------------------------------------+
|                    BUILDER PATTERN                              |
+-----------------------------------------------------------------+
|                                                                 |
|  INTENT: Separate the construction of a complex object          |
|          from its representation so that the same               |
|          construction process can create different              |
|          representations.                                       |
|                                                                 |
|  CATEGORY: Creational                                           |
|                                                                 |
+-----------------------------------------------------------------+

  UML-like Structure:

  +-------------------+        +-------------------+
  |    Director       |------->|    Builder        |
  | (optional)        |        |    (interface)    |
  +-------------------+        +-------------------+
  | + construct()     |        | + setPartA()      |
  +-------------------+        | + setPartB()      |
                               | + build(): Product |
                               +-------------------+
                                       ^
                                       |
                               +-------------------+
                               | ConcreteBuilder   |
                               +-------------------+
                               | - product         |
                               | + setPartA()      |
                               | + setPartB()      |
```

```
                                | + build(): Product |
                                +--------------------+
                                        |
                                        | creates
                                        v
                                +--------------------+
                                |    Product         |
                                +--------------------+

  How the fluent interface works:

  Product product = new Builder()
      .setPartA("value")     // returns Builder (this)
      .setPartB("value")     // returns Builder (this)
      .setPartC("value")     // returns Builder (this)
      .build();              // returns Product

  Each method returns the builder itself,
  enabling method chaining.
```

Java Implementation 1: HttpRequestBuilder

The Product Class

```java
/**
 * The product: an immutable HTTP request.
 * All fields are final. No setters. Once built, it cannot change.
 */
public class HttpRequest {

    private final String url;
    private final String method;
    private final Map<String, String> headers;
    private final String body;
    private final int timeoutMs;
    private final int maxRetries;
    private final boolean followRedirects;
    private final String authToken;
    private final String contentType;
    private final boolean verifySsl;

    // Package-private constructor - only the Builder can call this
    HttpRequest(Builder builder) {
        this.url = builder.url;
        this.method = builder.method;
```

```
        this.headers = Collections.unmodifiableMap(
            new HashMap<>(builder.headers)
        );
        this.body = builder.body;
        this.timeoutMs = builder.timeoutMs;
        this.maxRetries = builder.maxRetries;
        this.followRedirects = builder.followRedirects;
        this.authToken = builder.authToken;
        this.contentType = builder.contentType;
        this.verifySsl = builder.verifySsl;
    }

    // Getters only - no setters
    public String getUrl() { return url; }
    public String getMethod() { return method; }
    public Map<String, String> getHeaders() { return headers; }
    public String getBody() { return body; }
    public int getTimeoutMs() { return timeoutMs; }
    public int getMaxRetries() { return maxRetries; }
    public boolean isFollowRedirects() { return followRedirects; }
    public String getAuthToken() { return authToken; }
    public String getContentType() { return contentType; }
    public boolean isVerifySsl() { return verifySsl; }

    @Override
    public String toString() {
        return method + " " + url + "\n"
            + "Content-Type: " + contentType + "\n"
            + "Authorization: " + (authToken != null
                ? authToken.substring(0, 10) + "..." : "none")
            + "\n"
            + "Timeout: " + timeoutMs + "ms, "
            + "Retries: " + maxRetries + "\n"
            + "Body: " + (body != null
                ? body.substring(0, Math.min(50, body.length()))
                : "none");
    }

    /**
     * The Builder - a static inner class.
     */
    public static class Builder {

        // Required parameters
        private final String url;

        // Optional parameters with defaults
        private String method = "GET";
        private Map<String, String> headers = new HashMap<>();
        private String body = null;
        private int timeoutMs = 30000;
        private int maxRetries = 0;
        private boolean followRedirects = true;
        private String authToken = null;
        private String contentType = "application/json";
```

```
        private boolean verifySsl = true;

        /**
         * Constructor takes ONLY required parameters.
         */
        public Builder(String url) {
            if (url == null || url.isEmpty()) {
                throw new IllegalArgumentException("URL is required");
            }
            this.url = url;
        }

        /**
         * Each setter returns 'this' for method chaining.
         */
        public Builder method(String method) {
            this.method = method;
            return this;                 // <-- returns Builder
        }

        public Builder header(String key, String value) {
            this.headers.put(key, value);
            return this;
        }

        public Builder body(String body) {
            this.body = body;
            return this;
        }

        public Builder timeoutMs(int timeoutMs) {
            if (timeoutMs <= 0) {
                throw new IllegalArgumentException(
                    "Timeout must be positive"
                );
            }
            this.timeoutMs = timeoutMs;
            return this;
        }

        public Builder maxRetries(int maxRetries) {
            if (maxRetries < 0) {
                throw new IllegalArgumentException(
                    "Retries cannot be negative"
                );
            }
            this.maxRetries = maxRetries;
            return this;
        }

        public Builder followRedirects(boolean followRedirects) {
            this.followRedirects = followRedirects;
            return this;
        }
```

```
        public Builder authToken(String authToken) {
            this.authToken = authToken;
            return this;
        }

        public Builder contentType(String contentType) {
            this.contentType = contentType;
            return this;
        }

        public Builder verifySsl(boolean verifySsl) {
            this.verifySsl = verifySsl;
            return this;
        }

        /**
         * Build the final immutable HttpRequest.
         * Validation happens here.
         */
        public HttpRequest build() {
            // Validate the complete state
            if ("POST".equals(method) || "PUT".equals(method)) {
                if (body == null) {
                    throw new IllegalStateException(
                        method + " requests require a body"
                    );
                }
            }
            return new HttpRequest(this);
        }
    }
}
```

Line-by-line explanation of key concepts:

- The `HttpRequest` constructor is package-private and takes a `Builder`. External code cannot construct it directly.
- Every field in `HttpRequest` is `final`. The object is immutable after construction.
- The `Builder` constructor takes only required parameters (`url`). Everything else is optional with sensible defaults.
- Each setter method returns `this` (the Builder itself). This enables the fluent interface: method chaining.
- `build()` validates the complete state and creates the immutable product.

Usage and Output

```java
public class Main {
    public static void main(String[] args) {

        // Simple GET request - just the URL
        HttpRequest getRequest = new HttpRequest.Builder(
                "https://api.example.com/users"
            )
            .build();

        System.out.println(getRequest);
        System.out.println();

        // Complex POST request - readable and self-documenting
        HttpRequest postRequest = new HttpRequest.Builder(
                "https://api.example.com/users"
            )
            .method("POST")
            .body("{\"name\": \"John\", \"email\": \"john@example.com\"}")
            .contentType("application/json")
            .authToken("Bearer eyJhbGciOiJIUzI1NiJ9.abc123")
            .timeoutMs(5000)
            .maxRetries(3)
            .header("X-Request-ID", "req-12345")
            .header("X-Client-Version", "2.1.0")
            .build();

        System.out.println(postRequest);
    }
}
```

```
Output:
GET https://api.example.com/users
Content-Type: application/json
Authorization: none
Timeout: 30000ms, Retries: 0
Body: none

POST https://api.example.com/users
Content-Type: application/json
Authorization: Bearer eyJ...
Timeout: 5000ms, Retries: 3
Body: {"name": "John", "email": "john@example.com"}
```

Compare the readability:

```
BEFORE (telescoping constructor):
  new HttpRequest("url", "POST", null, body, 5000, 3,
                  true, "Bearer abc", "application/json",
                  true, null, 0);

AFTER (builder):
```

```
new HttpRequest.Builder("url")
    .method("POST")
    .body(body)
    .timeoutMs(5000)
    .maxRetries(3)
    .authToken("Bearer abc")
    .contentType("application/json")
    .build();

Every parameter is labeled. No nulls. No counting.
```

Java Implementation 2: QueryBuilder

```java
public class SqlQuery {

    private final String sql;

    private SqlQuery(String sql) {
        this.sql = sql;
    }

    public String getSql() {
        return sql;
    }

    @Override
    public String toString() {
        return sql;
    }

    public static class Builder {

        private String table;
        private List<String> columns = new ArrayList<>();
        private List<String> conditions = new ArrayList<>();
        private List<String> joins = new ArrayList<>();
        private String orderBy;
        private String orderDirection = "ASC";
        private Integer limit;
        private Integer offset;
        private List<String> groupBy = new ArrayList<>();
        private String having;

        public Builder select(String... columns) {
            this.columns.addAll(Arrays.asList(columns));
            return this;
```

```
        }

        public Builder from(String table) {
            this.table = table;
            return this;
        }

        public Builder where(String condition) {
            this.conditions.add(condition);
            return this;
        }

        public Builder innerJoin(String table, String on) {
            this.joins.add("INNER JOIN " + table + " ON " + on);
            return this;
        }

        public Builder leftJoin(String table, String on) {
            this.joins.add("LEFT JOIN " + table + " ON " + on);
            return this;
        }

        public Builder orderBy(String column, String direction) {
            this.orderBy = column;
            this.orderDirection = direction;
            return this;
        }

        public Builder limit(int limit) {
            this.limit = limit;
            return this;
        }

        public Builder offset(int offset) {
            this.offset = offset;
            return this;
        }

        public Builder groupBy(String... columns) {
            this.groupBy.addAll(Arrays.asList(columns));
            return this;
        }

        public Builder having(String condition) {
            this.having = condition;
            return this;
        }

        public SqlQuery build() {
            if (table == null) {
                throw new IllegalStateException(
                    "Table (FROM) is required"
                );
            }
```

```
        StringBuilder sql = new StringBuilder();

        // SELECT
        sql.append("SELECT ");
        if (columns.isEmpty()) {
            sql.append("*");
        } else {
            sql.append(String.join(", ", columns));
        }

        // FROM
        sql.append("\nFROM ").append(table);

        // JOINs
        for (String join : joins) {
            sql.append("\n").append(join);
        }

        // WHERE
        if (!conditions.isEmpty()) {
            sql.append("\nWHERE ")
               .append(String.join(" AND ", conditions));
        }

        // GROUP BY
        if (!groupBy.isEmpty()) {
            sql.append("\nGROUP BY ")
               .append(String.join(", ", groupBy));
        }

        // HAVING
        if (having != null) {
            sql.append("\nHAVING ").append(having);
        }

        // ORDER BY
        if (orderBy != null) {
            sql.append("\nORDER BY ")
               .append(orderBy)
               .append(" ")
               .append(orderDirection);
        }

        // LIMIT & OFFSET
        if (limit != null) {
            sql.append("\nLIMIT ").append(limit);
        }
        if (offset != null) {
            sql.append("\nOFFSET ").append(offset);
        }

        sql.append(";");
        return new SqlQuery(sql.toString());
    }
}
```

```
}
```

Usage and output:

```java
// Simple query
SqlQuery simple = new SqlQuery.Builder()
    .select("id", "name", "email")
    .from("users")
    .where("active = true")
    .limit(10)
    .build();

System.out.println(simple);
```

```
Output:
SELECT id, name, email
FROM users
WHERE active = true
LIMIT 10;
```

```java
// Complex query with joins, grouping, and pagination
SqlQuery complex = new SqlQuery.Builder()
    .select("u.name", "COUNT(o.id) AS order_count",
            "SUM(o.total) AS total_spent")
    .from("users u")
    .innerJoin("orders o", "u.id = o.user_id")
    .leftJoin("discounts d", "o.discount_id = d.id")
    .where("u.active = true")
    .where("o.created_at > '2024-01-01'")
    .groupBy("u.id", "u.name")
    .having("COUNT(o.id) > 5")
    .orderBy("total_spent", "DESC")
    .limit(20)
    .offset(40)
    .build();

System.out.println(complex);
```

```
Output:
SELECT u.name, COUNT(o.id) AS order_count, SUM(o.total) AS total_spent
FROM users u
INNER JOIN orders o ON u.id = o.user_id
LEFT JOIN discounts d ON o.discount_id = d.id
WHERE u.active = true AND o.created_at > '2024-01-01'
GROUP BY u.id, u.name
HAVING COUNT(o.id) > 5
ORDER BY total_spent DESC
```

```
LIMIT 20
OFFSET 40;
```

Python Implementation: ReportBuilder

```python
from dataclasses import dataclass, field
from typing import List, Optional, Dict, Any
from datetime import date, datetime
from enum import Enum


class ReportFormat(Enum):
    PDF = "pdf"
    CSV = "csv"
    EXCEL = "excel"
    HTML = "html"


@dataclass(frozen=True)
class Report:
    """
    Immutable report object. The frozen=True makes it immutable.
    Only the Builder can create instances.
    """
    title: str
    description: str
    format: ReportFormat
    start_date: date
    end_date: date
    columns: List[str]
    filters: Dict[str, Any]
    sort_by: Optional[str]
    sort_order: str
    page_size: int
    include_totals: bool
    include_charts: bool
    header_text: Optional[str]
    footer_text: Optional[str]

    def __str__(self) -> str:
        lines = [
            f"=== Report: {self.title} ===",
            f"Description: {self.description}",
            f"Format: {self.format.value.upper()}",
            f"Date Range: {self.start_date} to {self.end_date}",
            f"Columns: {', '.join(self.columns)}",
```

```
            f"Filters: {self.filters}",
            f"Sort: {self.sort_by} {self.sort_order}"
            if self.sort_by else "Sort: none",
            f"Page Size: {self.page_size}",
            f"Totals: {'Yes' if self.include_totals else 'No'}",
            f"Charts: {'Yes' if self.include_charts else 'No'}",
        ]
        if self.header_text:
            lines.append(f"Header: {self.header_text}")
        if self.footer_text:
            lines.append(f"Footer: {self.footer_text}")
        return "\n".join(lines)


class ReportBuilder:
    """
    Builder for the Report object.
    Provides a fluent interface for step-by-step construction.
    """

    def __init__(self, title: str):
        """Constructor takes only the required parameter."""
        if not title:
            raise ValueError("Title is required")
        self._title = title
        self._description = ""
        self._format = ReportFormat.PDF
        self._start_date = date.today()
        self._end_date = date.today()
        self._columns: List[str] = []
        self._filters: Dict[str, Any] = {}
        self._sort_by: Optional[str] = None
        self._sort_order = "ASC"
        self._page_size = 50
        self._include_totals = False
        self._include_charts = False
        self._header_text: Optional[str] = None
        self._footer_text: Optional[str] = None

    def description(self, description: str) -> "ReportBuilder":
        self._description = description
        return self

    def format(self, fmt: ReportFormat) -> "ReportBuilder":
        self._format = fmt
        return self

    def date_range(self, start: date,
                   end: date) -> "ReportBuilder":
        if start > end:
            raise ValueError("Start date must be before end date")
        self._start_date = start
        self._end_date = end
        return self
```

```
    def columns(self, *cols: str) -> "ReportBuilder":
        self._columns.extend(cols)
        return self

    def filter(self, key: str, value: Any) -> "ReportBuilder":
        self._filters[key] = value
        return self

    def sort_by(self, column: str,
                order: str = "ASC") -> "ReportBuilder":
        if order not in ("ASC", "DESC"):
            raise ValueError("Order must be ASC or DESC")
        self._sort_by = column
        self._sort_order = order
        return self

    def page_size(self, size: int) -> "ReportBuilder":
        if size <= 0:
            raise ValueError("Page size must be positive")
        self._page_size = size
        return self

    def with_totals(self) -> "ReportBuilder":
        self._include_totals = True
        return self

    def with_charts(self) -> "ReportBuilder":
        self._include_charts = True
        return self

    def header(self, text: str) -> "ReportBuilder":
        self._header_text = text
        return self

    def footer(self, text: str) -> "ReportBuilder":
        self._footer_text = text
        return self

    def build(self) -> Report:
        """Validate and create the immutable Report."""
        if not self._columns:
            raise ValueError("At least one column is required")

        return Report(
            title=self._title,
            description=self._description,
            format=self._format,
            start_date=self._start_date,
            end_date=self._end_date,
            columns=list(self._columns),
            filters=dict(self._filters),
            sort_by=self._sort_by,
            sort_order=self._sort_order,
            page_size=self._page_size,
            include_totals=self._include_totals,
```

```
            include_charts=self._include_charts,
            header_text=self._header_text,
            footer_text=self._footer_text,
        )
```

Usage and output:

```python
# Simple report
simple_report = (
    ReportBuilder("Monthly Sales")
    .description("Sales summary for March 2024")
    .columns("product", "quantity", "revenue")
    .date_range(date(2024, 3, 1), date(2024, 3, 31))
    .build()
)

print(simple_report)
print()

# Complex report
detailed_report = (
    ReportBuilder("Customer Analytics")
    .description("Detailed customer behavior analysis")
    .format(ReportFormat.EXCEL)
    .date_range(date(2024, 1, 1), date(2024, 3, 31))
    .columns("customer_name", "total_orders",
             "total_spent", "avg_order_value",
             "last_order_date")
    .filter("region", "North America")
    .filter("min_orders", 5)
    .filter("status", "active")
    .sort_by("total_spent", "DESC")
    .page_size(100)
    .with_totals()
    .with_charts()
    .header("CONFIDENTIAL - Internal Use Only")
    .footer("Generated by Analytics Engine v2.1")
    .build()
)

print(detailed_report)
```

```
Output:
=== Report: Monthly Sales ===
Description: Sales summary for March 2024
Format: PDF
Date Range: 2024-03-01 to 2024-03-31
Columns: product, quantity, revenue
Filters: {}
Sort: none
Page Size: 50
```

```
Totals: No
Charts: No

=== Report: Customer Analytics ===
Description: Detailed customer behavior analysis
Format: EXCEL
Date Range: 2024-01-01 to 2024-03-31
Columns: customer_name, total_orders, total_spent, avg_order_value, last_order_date
Filters: {'region': 'North America', 'min_orders': 5, 'status': 'active'}
Sort: total_spent DESC
Page Size: 100
Totals: Yes
Charts: Yes
Header: CONFIDENTIAL - Internal Use Only
Footer: Generated by Analytics Engine v2.1
```

The Fluent Interface

The fluent interface is what makes builders readable. Each method returns `this` (or `self`), enabling method chaining.

```
WITHOUT fluent interface:

  Builder builder = new Builder("Report");
  builder.setTitle("Sales");
  builder.setFormat("PDF");
  builder.addColumn("name");
  builder.addColumn("revenue");
  builder.setLimit(100);
  Report report = builder.build();

  Seven statements. Repetitive "builder." prefix.

WITH fluent interface:

  Report report = new Builder("Sales")
      .format("PDF")
      .columns("name", "revenue")
      .limit(100)
      .build();

  One statement. Reads like English.
```

The key is that every method (except `build()`) returns the builder itself:

```java
public Builder method(String method) {
    this.method = method;
    return this;    // <-- THIS is what enables chaining
}
```

```python
def method(self, method: str) -> "Builder":
    self._method = method
    return self     # <-- THIS is what enables chaining
```

The Director Pattern

The Director is an optional part of the Builder pattern. It encapsulates a specific construction sequence, so you can reuse it.

```
+-----------------------------------------------------------------+
|                THE DIRECTOR                                     |
+-----------------------------------------------------------------+
|                                                                 |
|  Without Director:                                              |
|    Each client must know the exact build steps.                 |
|    Duplicated construction code everywhere.                     |
|                                                                 |
|  With Director:                                                 |
|    Common construction sequences are encapsulated.              |
|    Client says "build me a standard GET request"                |
|    and the Director handles the details.                        |
|                                                                 |
+-----------------------------------------------------------------+

  +------------------+      +--------------------+
  |    Director      |----->|    Builder         |
  +------------------+      +--------------------+
  | + buildGetReq()  |      | + method()         |
  | + buildPostReq() |      | + header()         |
  | + buildAuthReq() |      | + timeout()        |
  +------------------+      | + build()          |
                            +--------------------+
```

Java Director:

```java
public class HttpRequestDirector {
```

```
    /**
     * Build a standard API GET request with common defaults.
     */
    public HttpRequest buildStandardGet(String url, String authToken) {
        return new HttpRequest.Builder(url)
            .method("GET")
            .authToken(authToken)
            .contentType("application/json")
            .header("Accept", "application/json")
            .timeoutMs(10000)
            .maxRetries(2)
            .build();
    }

    /**
     * Build a standard JSON POST request.
     */
    public HttpRequest buildJsonPost(String url, String body,
                                     String authToken) {
        return new HttpRequest.Builder(url)
            .method("POST")
            .body(body)
            .authToken(authToken)
            .contentType("application/json")
            .header("Accept", "application/json")
            .timeoutMs(15000)
            .maxRetries(1)
            .build();
    }

    /**
     * Build a file upload request with longer timeout.
     */
    public HttpRequest buildFileUpload(String url, String filePath,
                                       String authToken) {
        return new HttpRequest.Builder(url)
            .method("POST")
            .body("file://" + filePath)
            .authToken(authToken)
            .contentType("multipart/form-data")
            .timeoutMs(60000)    // 60 seconds for uploads
            .maxRetries(3)       // retry on failure
            .build();
    }
}
```

Usage:

```java
HttpRequestDirector director = new HttpRequestDirector();

// One line to create a standard GET request
HttpRequest getUsers = director.buildStandardGet(
```

```
    "https://api.example.com/users",
    "Bearer token123"
);

// One line to create a JSON POST
HttpRequest createUser = director.buildJsonPost(
    "https://api.example.com/users",
    "{\"name\": \"Jane\"}",
    "Bearer token123"
);
```

Python Director:

```python
class ReportDirector:
    """Encapsulates common report construction sequences."""

    @staticmethod
    def build_sales_summary(start: date, end: date) -> Report:
        return (
            ReportBuilder("Sales Summary")
            .description(f"Sales from {start} to {end}")
            .format(ReportFormat.PDF)
            .date_range(start, end)
            .columns("product", "units_sold", "revenue",
                     "profit_margin")
            .sort_by("revenue", "DESC")
            .with_totals()
            .with_charts()
            .footer("Auto-generated report")
            .build()
        )

    @staticmethod
    def build_user_export() -> Report:
        return (
            ReportBuilder("User Export")
            .description("Full user data export")
            .format(ReportFormat.CSV)
            .columns("id", "name", "email", "created_at",
                     "last_login")
            .sort_by("created_at", "DESC")
            .page_size(10000)
            .build()
        )

# Usage
sales = ReportDirector.build_sales_summary(
    date(2024, 1, 1), date(2024, 3, 31)
)
users = ReportDirector.build_user_export()
```

Lombok @Builder

In Java, the Lombok library generates builder code automatically with a single annotation.

```java
import lombok.Builder;
import lombok.Getter;
import lombok.ToString;

@Getter
@Builder
@ToString
public class HttpRequest {

    private final String url;

    @Builder.Default
    private final String method = "GET";

    @Builder.Default
    private final int timeoutMs = 30000;

    @Builder.Default
    private final int maxRetries = 0;

    @Builder.Default
    private final boolean followRedirects = true;

    @Builder.Default
    private final String contentType = "application/json";

    @Builder.Default
    private final boolean verifySsl = true;

    private final String body;
    private final String authToken;

    @Singular
    private final Map<String, String> headers;
}
```

What Lombok generates for you:

```
+-------------------------------------------------------------+
|  WHAT @Builder GENERATES                                    |
+-------------------------------------------------------------+
|                                                             |
|  - A static Builder inner class                             |
|  - A builder() static method to get a new Builder           |
|  - Setter methods on the Builder for each field             |
|  - A build() method that creates the object                 |
|  - @Builder.Default sets default values                     |
```

```
|  - @Singular generates add-one-at-a-time methods for     |
|    collections (header() instead of headers())           |
|                                                          |
+----------------------------------------------------------+
```

Usage with Lombok:

```java
HttpRequest request = HttpRequest.builder()
    .url("https://api.example.com/users")
    .method("POST")
    .body("{\"name\": \"John\"}")
    .authToken("Bearer abc123")
    .header("X-Request-ID", "req-001")
    .header("X-Client", "backend-v2")
    .timeoutMs(5000)
    .maxRetries(3)
    .build();
```

This is identical to hand-written builder code but with zero boilerplate. Lombok generates hundreds of lines of builder code from a single `@Builder` annotation.

Real-World Backend Use Case: API Response Builder

```java
public class ApiResponse<T> {

    private final int statusCode;
    private final String message;
    private final T data;
    private final Map<String, String> metadata;
    private final List<String> errors;
    private final long timestamp;

    private ApiResponse(Builder<T> builder) {
        this.statusCode = builder.statusCode;
        this.message = builder.message;
        this.data = builder.data;
        this.metadata = Collections.unmodifiableMap(builder.metadata);
        this.errors = Collections.unmodifiableList(builder.errors);
        this.timestamp = builder.timestamp;
    }
```

```
    // ... getters ...

    public static class Builder<T> {
        private int statusCode = 200;
        private String message = "OK";
        private T data;
        private Map<String, String> metadata = new HashMap<>();
        private List<String> errors = new ArrayList<>();
        private long timestamp = System.currentTimeMillis();

        public Builder<T> status(int code) {
            this.statusCode = code;
            return this;
        }

        public Builder<T> message(String message) {
            this.message = message;
            return this;
        }

        public Builder<T> data(T data) {
            this.data = data;
            return this;
        }

        public Builder<T> meta(String key, String value) {
            this.metadata.put(key, value);
            return this;
        }

        public Builder<T> error(String error) {
            this.errors.add(error);
            return this;
        }

        public ApiResponse<T> build() {
            return new ApiResponse<>(this);
        }

        // Convenience factory methods
        public static <T> ApiResponse<T> success(T data) {
            return new Builder<T>()
                .status(200)
                .message("OK")
                .data(data)
                .build();
        }

        public static <T> ApiResponse<T> notFound(String msg) {
            return new Builder<T>()
                .status(404)
                .message(msg)
                .build();
        }
```

```
        public static <T> ApiResponse<T> serverError(
                String... errors) {
            Builder<T> builder = new Builder<T>()
                .status(500)
                .message("Internal Server Error");
            for (String e : errors) {
                builder.error(e);
            }
            return builder.build();
        }
    }
}
```

Usage:

```java
// Success response
ApiResponse<User> response = new ApiResponse.Builder<User>()
    .status(200)
    .message("User found")
    .data(user)
    .meta("cache", "HIT")
    .meta("responseTime", "45ms")
    .build();

// Or use the convenience methods
ApiResponse<User> success = ApiResponse.Builder.success(user);
ApiResponse<User> notFound = ApiResponse.Builder.notFound(
    "User not found"
);
ApiResponse<User> error = ApiResponse.Builder.serverError(
    "Database connection failed",
    "Retry limit exceeded"
);
```

When to Use / When NOT to Use

```
+-----------------------------------+-----------------------------------+
|  USE BUILDER WHEN                 |  DO NOT USE WHEN                  |
+-----------------------------------+-----------------------------------+
|                                   |                                   |
| - Object has many parameters      | - Object has 3 or fewer           |
|   (more than 4-5)                |   parameters                     |
|                                   |                                   |
| - Many parameters are optional    | - All parameters are required     |
|   with sensible defaults          |   (no optional ones)              |
```

```
|                                   |                                   |
| - You want immutable objects      | - Mutability is acceptable        |
|   (no setters after construction)|   (just use setters)              |
|                                   |                                   |
| - Construction involves           | - Construction is straightforward|
|   validation or complex logic     |   (just assign fields)            |
|                                   |                                   |
| - You need a fluent API           | - A simple constructor is         |
|   (query builders, config)        |   readable enough                 |
|                                   |                                   |
+-----------------------------------+-----------------------------------+
```

Common Mistakes

1. Using Builder for simple objects. If your class has two or three fields, a constructor is simpler. Builder adds complexity; do not use it where a constructor is clear enough.

```java
// OVER-ENGINEERED: Builder for a simple class
Point p = new Point.Builder().x(10).y(20).build();

// JUST USE A CONSTRUCTOR:
Point p = new Point(10, 20);
```

2. Forgetting to validate in `build()`. The whole point of Builder is controlled construction. If `build()` does not validate, you lose the safety guarantees.
3. Mutable builder reuse. Do not reuse a builder instance after calling `build()`. The builder's internal state may leak into the next object.
4. Not making the product immutable. If the product has setters, the builder provides no advantage over setters. The power of Builder is creating immutable objects.
5. Too many required parameters in the builder constructor. If the Builder constructor takes 5 required parameters, you have the same problem. Keep the builder constructor to 1-2 required fields.

Best Practices

1. Required parameters go in the Builder constructor. Optional parameters have setter methods with defaults.

2. Validate in `build()`, not in individual setters. Some validations depend on the combination of fields (e.g., POST requires a body). These can only be checked when all fields are set.
3. Return immutable objects from `build()`. Use `final` fields, unmodifiable collections, and no setters.
4. Consider a Director for common configurations. If the same builder sequence appears in multiple places, extract it into a Director.
5. In Java, use Lombok `@Builder` for simple cases. Write a custom builder only when you need custom validation, computed fields, or complex logic.
6. In Python, consider `dataclasses` with a builder. Python's `@dataclass(frozen=True)` gives you immutability. Add a builder class when you have many optional parameters.

Quick Summary

Aspect	Details
Pattern name	Builder
Category	Creational
Intent	Construct complex objects step by step
Problem it solves	Telescoping constructors, unreadable parameter lists
Key feature	Fluent interface with method chaining
Director	Optional class that encapsulates common build sequences
Lombok	`@Builder` generates builder code automatically in Java
Result	Immutable, validated objects with readable construction code

---

Key Points

1. Builder solves the telescoping constructor problem by replacing a long parameter list with named, chainable method calls.
2. The fluent interface (each method returns `this`) makes construction code read almost like natural language.
3. Builder produces immutable objects. The product's constructor is private; only the builder can create instances.
4. Validation happens in `build()`, not in individual setters. This allows cross-field validation.
5. The Director pattern encapsulates common build sequences. Use it when the same configuration appears in multiple places.
6. In Java, Lombok's `@Builder` eliminates boilerplate. Use it for straightforward cases; write custom builders for complex validation.
7. Do not use Builder for simple objects. If a class has fewer than 4 parameters and no optional ones, a regular constructor is clearer.

Practice Questions

1. What is the "telescoping constructor" problem? Give an example of how it makes code harder to read and maintain.
2. How does the fluent interface work in the Builder pattern? What does each setter method return, and why?
3. Why should the product object be immutable? What problems can occur if the product has setters?
4. When would you use a Director class? Give a real-world backend example where a Director would be beneficial.
5. Compare hand-written builders with Lombok's `@Builder`. When would you choose one over the other?

Exercises

Exercise 1: Email Message Builder

Create a Builder for an `EmailMessage` class with the following fields: `to` (required), `from` (required), `subject`, `bodyText`, `bodyHtml`, `cc` (list), `bcc` (list), `attachments` (list), `priority` (LOW/NORMAL/HIGH), `replyTo`. Include validation in `build()`: subject and at least one body

(text or HTML) are required. Implement in both Java and Python.

Exercise 2: Database Connection Config Builder

Create a `DatabaseConfig` builder with: `host` (required), `port` (default 5432), `database` (required), `username`, `password`, `maxPoolSize` (default 10), `minPoolSize` (default 2), `connectionTimeoutMs` (default 5000), `idleTimeoutMs` (default 600000), `sslEnabled` (default false), `sslCertPath`. Validation: if SSL is enabled, `sslCertPath` is required. `maxPoolSize` must be >= `minPoolSize`.

Exercise 3: Query Builder with Director

Extend the SQL `QueryBuilder` example to support INSERT, UPDATE, and DELETE operations. Then create a `QueryDirector` with methods like `buildPaginatedList(table, page, pageSize)`, `buildCountQuery(table, conditions)`, and `buildSoftDelete(table, id)`.

What Is Next?

In the next and final chapter of Part I, we explore the Prototype pattern. While Builder constructs objects step by step, Prototype creates objects by cloning existing ones. You will learn when cloning is faster and more practical than building from scratch, the critical difference between shallow and deep copies, and how prototype registries let you manage template objects for test data, configuration templates, and cached prototypes.

Chapter 6: Prototype Pattern -- Clone Objects Efficiently

What You Will Learn

- What the Prototype pattern is and why cloning objects matters
- The difference between deep copy and shallow copy
- How to implement Prototype in Java using `Cloneable`
- How to implement Prototype in Python using the `copy` module
- How to build a Prototype Registry for managing reusable templates
- Real-world applications: template configurations, test data factories

Why This Chapter Matters

Creating objects from scratch is expensive when they require database calls, network requests, or complex initialization. Sometimes you need dozens of objects that are almost identical, differing only in a few fields. Building each one from the ground up wastes time and resources.

The Prototype pattern solves this by letting you clone an existing object and then tweak the copy. Think of it like photocopying a filled-out form and changing just the name on each copy, instead of filling out every form from a blank page.

If you have ever duplicated a configuration file and changed two lines, you already understand the Prototype pattern. This chapter teaches you how to do it properly in code.

The Problem

Imagine you are building a cloud deployment tool. Each server configuration has 40+ fields: operating system, CPU count, memory, disk size, network settings, security groups, monitoring flags, and more. Most servers share 90% of the same configuration.

Without Prototype, you do this:

```
ServerConfig web1 = new ServerConfig();
web1.setOs("Ubuntu 22.04");
web1.setCpu(4);
web1.setMemory(16384);
web1.setDisk(100);
web1.setNetwork("vpc-main");
web1.setSecurityGroup("sg-web");
web1.setMonitoring(true);
// ... 33 more setters

ServerConfig web2 = new ServerConfig();
web2.setOs("Ubuntu 22.04");       // same
web2.setCpu(4);                    // same
web2.setMemory(16384);             // same
web2.setDisk(100);                 // same
web2.setNetwork("vpc-main");       // same
web2.setSecurityGroup("sg-web");   // same
web2.setMonitoring(true);          // same
// ... 33 more identical setters
web2.setHostname("web-server-2");  // only this is different!
```

That is 40 lines of code duplicated for every server. If you need 50 servers, that is 2000 lines of nearly identical code.

The Solution: Prototype Pattern

The Prototype pattern says: create one well-configured object, then clone it.

```
+---------------------+
|    <<interface>>     |
|     Prototype        |
+---------------------+
| + clone(): Prototype |
+---------------------+
         ^
         |
+---------------------+
|  ConcretePrototype   |
+---------------------+
```

```
| - field1             |
| - field2             |
| - field3             |
+----------------------+
| + clone(): Prototype |
+----------------------+

Usage:
  original ---clone()---> copy
                            |
                      modify fields
                            |
                         result
```

Key participants:

- Prototype: declares the clone method
- ConcretePrototype: implements cloning by copying its own fields
- Client: creates new objects by asking a prototype to clone itself

---

Deep Copy vs Shallow Copy

Before writing code, you must understand this critical distinction.

```
SHALLOW COPY                          DEEP COPY
+----------+    +----------+         +----------+    +----------+
| Original |    |   Copy   |         | Original |    |   Copy   |
+----------+    +----------+         +----------+    +----------+
| name: A  |    | name: A  |         | name: A  |    | name: A  |
| tags: ---|------>[ list ] |         | tags: ---|->[ list1 ]    |
+----------+    +----------+         +----------+  | tags: ---|->[ list2 ]
                                                    +----------+
  Both point to the SAME list.         Each has its OWN list.
  Change one, both change.            Independent copies.
```

Shallow copy duplicates the object but shares references to nested objects. If the original has a list of tags, the copy points to the same list. Add a tag to the copy and it shows up in the original too.

Deep copy duplicates everything recursively. The copy gets its own independent list. Changes to one never affect the other.

Rule of thumb: If your object contains only primitive fields (int, String, boolean), shallow copy is fine. If it contains collections, arrays, or other objects, you almost certainly need deep copy.

Java Implementation

Basic Prototype with Cloneable

Java provides the `Cloneable` interface and `Object.clone()` method for this pattern.

```java
// ServerConfig.java
import java.util.ArrayList;
import java.util.List;

public class ServerConfig implements Cloneable {
    private String hostname;
    private String os;
    private int cpu;
    private int memoryMb;
    private int diskGb;
    private String network;
    private String securityGroup;
    private boolean monitoring;
    private List<String> tags;

    public ServerConfig(String os, int cpu, int memoryMb, int diskGb,
                        String network, String securityGroup,
                        boolean monitoring) {
        this.os = os;
        this.cpu = cpu;
        this.memoryMb = memoryMb;
        this.diskGb = diskGb;
        this.network = network;
        this.securityGroup = securityGroup;
        this.monitoring = monitoring;
        this.tags = new ArrayList<>();
    }

    // Deep clone
    @Override
    public ServerConfig clone() {
        try {
            ServerConfig copy = (ServerConfig) super.clone();
            // Deep copy the mutable list
            copy.tags = new ArrayList<>(this.tags);
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException("Clone failed", e);
        }
    }
```

```
    public void setHostname(String hostname) { this.hostname = hostname; }
    public void addTag(String tag) { this.tags.add(tag); }
    public List<String> getTags() { return tags; }
    public String getHostname() { return hostname; }

    @Override
    public String toString() {
        return String.format("Server[host=%s, os=%s, cpu=%d, mem=%dMB, " +
                "disk=%dGB, net=%s, sg=%s, mon=%s, tags=%s]",
                hostname, os, cpu, memoryMb, diskGb, network,
                securityGroup, monitoring, tags);
    }
}
```

```java
// PrototypeDemo.java
public class PrototypeDemo {
    public static void main(String[] args) {
        // Step 1: Create and configure the template once
        ServerConfig webTemplate = new ServerConfig(
            "Ubuntu 22.04", 4, 16384, 100,
            "vpc-main", "sg-web", true
        );
        webTemplate.addTag("production");
        webTemplate.addTag("web-tier");

        // Step 2: Clone and customize
        ServerConfig web1 = webTemplate.clone();
        web1.setHostname("web-server-1");

        ServerConfig web2 = webTemplate.clone();
        web2.setHostname("web-server-2");
        web2.addTag("primary");

        // Step 3: Verify independence (deep copy proof)
        System.out.println(web1);
        System.out.println(web2);
        System.out.println("web1 tags: " + web1.getTags());
        System.out.println("web2 tags: " + web2.getTags());
        System.out.println("Same object? " + (web1 == web2));
    }
}
```

Output:

```
Server[host=web-server-1, os=Ubuntu 22.04, cpu=4, mem=16384MB, disk=100GB, net=vpc-main...
Server[host=web-server-2, os=Ubuntu 22.04, cpu=4, mem=16384MB, disk=100GB, net=vpc-main...
web1 tags: [production, web-tier]
web2 tags: [production, web-tier, primary]
Same object? false
```

Notice that adding "primary" to web2's tags did not affect web1. That confirms our deep copy works correctly.

Copy Constructor Alternative

Many Java developers prefer copy constructors over `Cloneable` because they are more explicit and avoid the checked exception hassle.

```java
public class ServerConfig {
    // ... fields ...

    // Copy constructor
    public ServerConfig(ServerConfig source) {
        this.hostname = source.hostname;
        this.os = source.os;
        this.cpu = source.cpu;
        this.memoryMb = source.memoryMb;
        this.diskGb = source.diskGb;
        this.network = source.network;
        this.securityGroup = source.securityGroup;
        this.monitoring = source.monitoring;
        this.tags = new ArrayList<>(source.tags); // deep copy
    }
}

// Usage:
ServerConfig web1 = new ServerConfig(webTemplate);
web1.setHostname("web-server-1");
```

This approach is recommended by Joshua Bloch in Effective Java. It gives you full control over what gets copied and how.

Python Implementation

Using the copy Module

Python's `copy` module provides `copy()` for shallow copies and `deepcopy()` for deep copies.

```python
# server_config.py
import copy

class ServerConfig:
```

```
    def __init__(self, os_name, cpu, memory_mb, disk_gb,
                 network, security_group, monitoring):
        self.hostname = None
        self.os_name = os_name
        self.cpu = cpu
        self.memory_mb = memory_mb
        self.disk_gb = disk_gb
        self.network = network
        self.security_group = security_group
        self.monitoring = monitoring
        self.tags = []
        self.metadata = {}

    def clone(self):
        """Create a deep copy of this config."""
        return copy.deepcopy(self)

    def add_tag(self, tag):
        self.tags.append(tag)

    def __repr__(self):
        return (f"Server[host={self.hostname}, os={self.os_name}, "
                f"cpu={self.cpu}, mem={self.memory_mb}MB, "
                f"disk={self.disk_gb}GB, tags={self.tags}]")


# Create the template
web_template = ServerConfig(
    os_name="Ubuntu 22.04",
    cpu=4,
    memory_mb=16384,
    disk_gb=100,
    network="vpc-main",
    security_group="sg-web",
    monitoring=True
)
web_template.add_tag("production")
web_template.add_tag("web-tier")

# Clone and customize
web1 = web_template.clone()
web1.hostname = "web-server-1"

web2 = web_template.clone()
web2.hostname = "web-server-2"
web2.add_tag("primary")

print(web1)
print(web2)
print(f"web1 tags: {web1.tags}")
print(f"web2 tags: {web2.tags}")
print(f"Same object? {web1 is web2}")
```

Output:

```
Server[host=web-server-1, os=Ubuntu 22.04, cpu=4, mem=16384MB, disk=100GB, tags=['produ...
Server[host=web-server-2, os=Ubuntu 22.04, cpu=4, mem=16384MB, disk=100GB, tags=['produ...
web1 tags: ['production', 'web-tier']
web2 tags: ['production', 'web-tier', 'primary']
Same object? False
```

Demonstrating Shallow vs Deep Copy

```python
import copy

class Team:
    def __init__(self, name):
        self.name = name
        self.members = ["Alice", "Bob"]

original = Team("Backend")

# Shallow copy -- shares the members list
shallow = copy.copy(original)
shallow.name = "Frontend"
shallow.members.append("Charlie")

print(f"Original members: {original.members}")
print(f"Shallow members:  {shallow.members}")
# Both show Charlie! The list is shared.

# Deep copy -- independent members list
original2 = Team("Backend")
deep = copy.deepcopy(original2)
deep.name = "Frontend"
deep.members.append("Diana")

print(f"Original2 members: {original2.members}")
print(f"Deep members:      {deep.members}")
# Only deep copy has Diana. Lists are independent.
```

Output:

```
Original members: ['Alice', 'Bob', 'Charlie']
Shallow members:  ['Alice', 'Bob', 'Charlie']
Original2 members: ['Alice', 'Bob']
Deep members:      ['Alice', 'Bob', 'Diana']
```

Customizing Copy Behavior with copy and deepcopy

Python lets you control exactly how your objects are copied.

```python
import copy

class DatabaseConfig:
    """Some fields should never be copied (like active connections)."""

    def __init__(self, host, port, db_name):
        self.host = host
        self.port = port
        self.db_name = db_name
        self.connection = None  # should NOT be copied
        self.query_cache = {}   # should be reset on copy

    def __deepcopy__(self, memo):
        # Create new instance without calling __init__
        new_obj = DatabaseConfig.__new__(DatabaseConfig)
        memo[id(self)] = new_obj

        # Copy fields selectively
        new_obj.host = self.host
        new_obj.port = self.port
        new_obj.db_name = copy.deepcopy(self.db_name, memo)
        new_obj.connection = None       # reset, don't copy
        new_obj.query_cache = {}        # reset, don't copy

        return new_obj

    def __repr__(self):
        return (f"DBConfig[{self.host}:{self.port}/{self.db_name}, "
                f"connected={self.connection is not None}]")


config = DatabaseConfig("db.prod.internal", 5432, "users_db")
config.connection = "<active connection>"
config.query_cache = {"users": [1, 2, 3]}

clone = copy.deepcopy(config)
print(f"Original: {config}")
print(f"Clone:    {clone}")
print(f"Clone connection: {clone.connection}")
print(f"Clone cache: {clone.query_cache}")
```

Output:

```
Original: DBConfig[db.prod.internal:5432/users_db, connected=True]
Clone:    DBConfig[db.prod.internal:5432/users_db, connected=False]
Clone connection: None
Clone cache: {}
```

This is powerful: you decide which fields are cloned, which are reset, and which are shared.

Prototype Registry

A Prototype Registry stores pre-configured prototypes by name. Instead of remembering how to configure each type of object, you look up a template and clone it.

```
+-----------------------------+
|      PrototypeRegistry      |
+-----------------------------+
|  "web-small"  -> ServerConfig|
|  "web-large"  -> ServerConfig|
|  "db-primary" -> ServerConfig|
|  "cache"      -> ServerConfig|
+-----------------------------+
| + register(name, prototype)  |
| + get(name): ServerConfig    |
+-----------------------------+

  registry.get("web-small")
       |
       v
    clone()
       |
       v
  new independent copy
```

Java Prototype Registry

```java
import java.util.HashMap;
import java.util.Map;

public class ServerRegistry {
    private final Map<String, ServerConfig> prototypes = new HashMap<>();

    public void register(String name, ServerConfig prototype) {
        prototypes.put(name, prototype);
    }

    public ServerConfig create(String name) {
        ServerConfig prototype = prototypes.get(name);
        if (prototype == null) {
            throw new IllegalArgumentException(
                "No prototype registered: " + name);
        }
        return prototype.clone();
    }

    public boolean has(String name) {
        return prototypes.containsKey(name);
```

```
    }
}
```

```java
// RegistryDemo.java
public class RegistryDemo {
    public static void main(String[] args) {
        ServerRegistry registry = new ServerRegistry();

        // Register templates once at startup
        ServerConfig webSmall = new ServerConfig(
            "Ubuntu 22.04", 2, 4096, 50,
            "vpc-main", "sg-web", true);
        webSmall.addTag("web-tier");

        ServerConfig webLarge = new ServerConfig(
            "Ubuntu 22.04", 8, 32768, 200,
            "vpc-main", "sg-web", true);
        webLarge.addTag("web-tier");
        webLarge.addTag("high-traffic");

        ServerConfig dbPrimary = new ServerConfig(
            "Ubuntu 22.04", 16, 65536, 1000,
            "vpc-data", "sg-database", true);
        dbPrimary.addTag("database");
        dbPrimary.addTag("primary");

        registry.register("web-small", webSmall);
        registry.register("web-large", webLarge);
        registry.register("db-primary", dbPrimary);

        // Create servers from templates
        ServerConfig server1 = registry.create("web-small");
        server1.setHostname("web-1");

        ServerConfig server2 = registry.create("web-large");
        server2.setHostname("web-2");

        ServerConfig server3 = registry.create("db-primary");
        server3.setHostname("db-main");

        System.out.println(server1);
        System.out.println(server2);
        System.out.println(server3);
    }
}
```

Output:

```
Server[host=web-1, os=Ubuntu 22.04, cpu=2, mem=4096MB, disk=50GB, net=vpc-main, sg=sg-w...
Server[host=web-2, os=Ubuntu 22.04, cpu=8, mem=32768MB, disk=200GB, net=vpc-main, sg=sg...
Server[host=db-main, os=Ubuntu 22.04, cpu=16, mem=65536MB, disk=1000GB, net=vpc-data, s...
```

Python Prototype Registry

```python
import copy

class PrototypeRegistry:
    def __init__(self):
        self._prototypes = {}

    def register(self, name, prototype):
        self._prototypes[name] = prototype

    def create(self, name, **overrides):
        if name not in self._prototypes:
            raise KeyError(f"No prototype registered: {name}")

        clone = copy.deepcopy(self._prototypes[name])

        # Apply any field overrides
        for key, value in overrides.items():
            setattr(clone, key, value)

        return clone

    def list_templates(self):
        return list(self._prototypes.keys())


# Setup
registry = PrototypeRegistry()

web_small = ServerConfig("Ubuntu 22.04", 2, 4096, 50,
                         "vpc-main", "sg-web", True)
web_small.add_tag("web-tier")

db_template = ServerConfig("Ubuntu 22.04", 16, 65536, 1000,
                           "vpc-data", "sg-database", True)
db_template.add_tag("database")

registry.register("web-small", web_small)
registry.register("db-primary", db_template)

# Create with overrides
server = registry.create("web-small", hostname="web-1")
db = registry.create("db-primary", hostname="db-main")

print(server)
print(db)
print(f"Available templates: {registry.list_templates()}")
```

Output:

```
Server[host=web-1, os=Ubuntu 22.04, cpu=2, mem=4096MB, disk=50GB, tags=['web-tier']]
Server[host=db-main, os=Ubuntu 22.04, cpu=16, mem=65536MB, disk=1000GB, tags=['database']]
Available templates: ['web-small', 'db-primary']
```

Real-World Use Case: Test Data Factory

Prototype shines when generating test data. Tests need many similar objects with small variations.

```python
import copy
from datetime import datetime, timedelta

class User:
    def __init__(self):
        self.id = None
        self.name = "Test User"
        self.email = "test@example.com"
        self.role = "viewer"
        self.is_active = True
        self.created_at = datetime.now()
        self.preferences = {
            "theme": "light",
            "language": "en",
            "notifications": True
        }

    def __repr__(self):
        return (f"User(id={self.id}, name='{self.name}', "
                f"email='{self.email}', role='{self.role}')")


class TestDataFactory:
    """Factory that uses prototypes to generate test data quickly."""

    _counter = 0

    def __init__(self):
        self._prototypes = {}
        self._setup_defaults()

    def _setup_defaults(self):
        # Standard user prototype
        standard = User()
        self._prototypes["standard"] = standard

        # Admin user prototype
```

```
        admin = User()
        admin.name = "Admin User"
        admin.email = "admin@example.com"
        admin.role = "admin"
        admin.preferences["theme"] = "dark"
        self._prototypes["admin"] = admin

        # Inactive user prototype
        inactive = User()
        inactive.is_active = False
        inactive.name = "Inactive User"
        self._prototypes["inactive"] = inactive

    def create_user(self, template="standard", **overrides):
        TestDataFactory._counter += 1
        user = copy.deepcopy(self._prototypes[template])
        user.id = TestDataFactory._counter

        for key, value in overrides.items():
            setattr(user, key, value)

        return user

    def create_users(self, count, template="standard"):
        return [self.create_user(template) for _ in range(count)]


# Usage in tests
factory = TestDataFactory()

# Quick single users
alice = factory.create_user(name="Alice", email="alice@test.com")
admin = factory.create_user("admin", name="Bob")
inactive = factory.create_user("inactive")

print(alice)
print(admin)
print(inactive)

# Bulk creation
viewers = factory.create_users(3)
print(f"\nBulk viewers: {viewers}")

# Verify independence
alice.preferences["theme"] = "dark"
another = factory.create_user()
print(f"\nAlice theme: {alice.preferences['theme']}")
print(f"Another theme: {another.preferences['theme']}")
```

Output:

```
User(id=1, name='Alice', email='alice@test.com', role='viewer')
User(id=2, name='Bob', email='admin@example.com', role='admin')
```

```
User(id=3, name='Inactive User', email='test@example.com', role='viewer')

Bulk viewers: [User(id=4, name='Test User', email='test@example.com', role='viewer'), U...

Alice theme: dark
Another theme: light
```

Real-World Use Case: Template Configurations

```java
// EmailTemplate.java
import java.util.HashMap;
import java.util.Map;

public class EmailTemplate implements Cloneable {
    private String subject;
    private String body;
    private String sender;
    private Map<String, String> headers;
    private String contentType;

    public EmailTemplate(String subject, String body, String sender) {
        this.subject = subject;
        this.body = body;
        this.sender = sender;
        this.headers = new HashMap<>();
        this.contentType = "text/html";
    }

    @Override
    public EmailTemplate clone() {
        try {
            EmailTemplate copy = (EmailTemplate) super.clone();
            copy.headers = new HashMap<>(this.headers);
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }

    public void setSubject(String subject) { this.subject = subject; }
    public void setBody(String body) { this.body = body; }
    public void addHeader(String key, String value) {
        headers.put(key, value);
    }

    @Override
```

```
    public String toString() {
        return String.format("Email[to=%s, subject='%s', type=%s, " +
                "headers=%s]", sender, subject, contentType, headers);
    }
}

// Usage
public class EmailDemo {
    public static void main(String[] args) {
        // Create base template
        EmailTemplate welcomeTemplate = new EmailTemplate(
            "Welcome to Our Platform",
            "<h1>Welcome, {{name}}!</h1><p>Get started...</p>",
            "noreply@platform.com"
        );
        welcomeTemplate.addHeader("X-Priority", "1");
        welcomeTemplate.addHeader("X-Category", "onboarding");

        // Clone for different users
        EmailTemplate email1 = welcomeTemplate.clone();
        email1.setBody("<h1>Welcome, Alice!</h1><p>Get started...</p>");

        EmailTemplate email2 = welcomeTemplate.clone();
        email2.setBody("<h1>Welcome, Bob!</h1><p>Get started...</p>");

        // Clone and modify for a different template type
        EmailTemplate resetTemplate = welcomeTemplate.clone();
        resetTemplate.setSubject("Password Reset Request");
        resetTemplate.setBody("<h1>Reset your password</h1>");
        resetTemplate.addHeader("X-Category", "security");

        System.out.println(email1);
        System.out.println(email2);
        System.out.println(resetTemplate);
    }
}
```

Output:

```
Email[to=noreply@platform.com, subject='Welcome to Our Platform', type=text/html, heade...
Email[to=noreply@platform.com, subject='Welcome to Our Platform', type=text/html, heade...
Email[to=noreply@platform.com, subject='Password Reset Request', type=text/html, header...
```

Before vs After Comparison

Before: Without Prototype

```java
// Creating test users manually -- repetitive and error-prone
User user1 = new User();
user1.setName("Test User 1");
user1.setEmail("test1@example.com");
user1.setRole("viewer");
user1.setActive(true);
user1.setTheme("light");
user1.setLanguage("en");
user1.setNotifications(true);
user1.setCreatedAt(new Date());

User user2 = new User();
user2.setName("Test User 2");
user2.setEmail("test2@example.com");
user2.setRole("viewer");        // same
user2.setActive(true);           // same
user2.setTheme("light");         // same
user2.setLanguage("en");         // same
user2.setNotifications(true);    // same
user2.setCreatedAt(new Date());  // same

// Repeat 50 more times...
```

Problems:

- Massive code duplication
- Easy to miss a field
- Hard to update when a new field is added
- No single source of truth for "default" configuration

After: With Prototype

```java
User template = createDefaultUser();

User user1 = template.clone();
user1.setName("Test User 1");
user1.setEmail("test1@example.com");

User user2 = template.clone();
user2.setName("Test User 2");
user2.setEmail("test2@example.com");

// Clean, maintainable, consistent
```

Benefits:

- One template captures all defaults
- Changes to defaults propagate automatically

- Cloning is faster than constructing from scratch
- Less code, fewer bugs

When to Use / When NOT to Use

Use Prototype When

- Objects have many fields and most are the same across instances
- Object creation is expensive (database lookups, file parsing, network calls)
- You need many variations of a base configuration
- You want to avoid building complex class hierarchies of factories
- Runtime object creation from templates is needed
- Test data generation requires many similar objects

Do NOT Use Prototype When

- Objects are simple with few fields (just use a constructor)
- Every instance is completely different (nothing to share)
- Objects have no mutable state (use a shared reference instead)
- Circular references make deep copying complex and error-prone
- The object graph is very deep and cloning is actually slower than construction
- Objects hold non-cloneable resources (threads, file handles, sockets)

Common Mistakes

Mistake 1: Forgetting Deep Copy

```java
// WRONG: shallow copy shares the list
@Override
public Config clone() {
    try {
        return (Config) super.clone(); // tags list is shared!
    } catch (CloneNotSupportedException e) {
```

```
        throw new RuntimeException(e);
    }
}

// RIGHT: deep copy mutable fields
@Override
public Config clone() {
    try {
        Config copy = (Config) super.clone();
        copy.tags = new ArrayList<>(this.tags);    // new list
        copy.metadata = new HashMap<>(this.metadata); // new map
        return copy;
    } catch (CloneNotSupportedException e) {
        throw new RuntimeException(e);
    }
}
```

Mistake 2: Cloning Non-Cloneable Resources

```python
# WRONG: cloning a database connection
class Service:
    def __init__(self):
        self.db_connection = create_connection()  # cannot clone this
        self.config = {...}

    def clone(self):
        return copy.deepcopy(self)  # will fail or create invalid state

# RIGHT: reset non-cloneable fields
def __deepcopy__(self, memo):
    new_obj = Service.__new__(Service)
    new_obj.config = copy.deepcopy(self.config, memo)
    new_obj.db_connection = None  # must be initialized separately
    return new_obj
```

Mistake 3: Modifying the Prototype Itself

```python
# WRONG: accidentally modifying the template
template = registry.get("web-server")  # this IS the template
template.hostname = "oops"  # you just changed the template!

# RIGHT: always clone first
server = registry.create("web-server")  # returns a clone
server.hostname = "web-1"  # safe to modify
```

Best Practices

1. Always deep copy mutable fields. Lists, maps, sets, and nested objects must be independently copied. This is the number one source of Prototype bugs.
2. Use copy constructors in Java over Cloneable. The `Cloneable` interface is considered broken by many experts. Copy constructors are explicit and type-safe.
3. Document what gets copied. Not every field should be cloned (connections, caches, IDs). Make it clear in comments or documentation.
4. Make prototypes immutable after registration. Once a prototype is stored in a registry, treat it as read-only. Any modification corrupts all future clones.
5. Combine with Registry pattern. A prototype by itself is useful. A prototype registry makes it powerful for managing multiple templates.
6. Generate unique IDs on clone. Cloned objects should get new IDs, not copies of the original's ID. Handle this in the clone method or factory.
7. Test clone independence. Write tests that modify a clone and verify the original is unchanged. This catches shallow copy bugs early.

Quick Summary

The Prototype pattern creates new objects by cloning existing ones. It eliminates repetitive construction code and is especially useful when objects share most of their configuration. Deep copy ensures clones are fully independent. A Prototype Registry provides named access to pre-configured templates.

```
Problem:  Creating many similar objects is repetitive and expensive.
Solution: Clone a template object and customize the copy.
Key:      Always use deep copy for objects with mutable nested fields.
```

Key Points

- Prototype creates objects by cloning rather than constructing from scratch
- Shallow copy shares references to nested objects; deep copy creates independent copies
- Java offers `Cloneable` and `Object.clone()`, but copy constructors are often preferred
- Python's `copy.deepcopy()` handles deep cloning; customize with `__deepcopy__`
- A Prototype Registry maps names to pre-configured templates for easy lookup
- Always deep copy mutable fields (lists, maps, nested objects)
- Prototype shines for test data generation and configuration templates
- Never clone non-cloneable resources like connections or threads

---

Practice Questions

1. What is the difference between shallow copy and deep copy? Give an example where using shallow copy instead of deep copy would cause a bug.
2. Why does Joshua Bloch recommend copy constructors over `Cloneable` in Java? What are two specific advantages?
3. You have a `Report` object with a `List<Chart>` field, and each `Chart` has a `List<DataPoint>`. Describe step by step how you would implement deep cloning for this object in Java.
4. In what scenario would cloning an object actually be slower than constructing a new one? Describe a concrete example.
5. How does a Prototype Registry improve upon using the Prototype pattern alone?

Exercises

Exercise 1: Document Template System

Build a document template system with the following requirements:

- A `Document` class with fields: title, author, content, metadata (Map), tags (List), and creation date

- A `DocumentRegistry` that stores named templates ("invoice", "report", "memo")
- Cloning a document must produce a fully independent copy
- The creation date should reset to "now" on every clone, not copy the original's date
- Write a test that creates 3 invoices from the template, modifies each differently,

and verifies they are independent

Exercise 2: Game Character Prototype

Create a role-playing game character system:

- A `Character` class with name, health, attack, defense, inventory (List of Items),

and skills (Map of skill name to level)

- Pre-register three archetypes: "warrior", "mage", "rogue"
- Implement both shallow and deep clone methods
- Write code that demonstrates the bug caused by shallow cloning (modifying one

character's inventory affects another) and then fix it with deep cloning

Exercise 3: API Response Caching with Prototype

Design a caching layer that stores API response prototypes:

- Cache stores prototype `ApiResponse` objects (status, headers Map, body, timestamp)
- When a cache hit occurs, return a deep clone (so callers can modify without corrupting

the cache)

- The timestamp on the clone should reflect "now", not the cached time
- Implement cache expiration: entries older than N seconds return null
- Demonstrate that modifying a returned response does not affect the cached version

---

What Is Next?

In the next chapter, we move from creating objects to structuring them. The Adapter Pattern shows you how to make incompatible interfaces work together, like plugging a European appliance into an American outlet. Where Prototype is about efficient creation, Adapter is about seamless integration between systems that were never designed to collaborate.

Chapter 7: The Adapter Pattern

What You Will Learn

By the end of this chapter, you will be able to:

- Understand the Adapter pattern and why it exists
- Recognize when two interfaces are incompatible and need bridging
- Implement both class adapters and object adapters
- Build a `LegacyPaymentAdapter` in Java that wraps an old payment API behind a new interface
- Create an XML-to-JSON adapter in Python for data format translation
- Distinguish between class adapters and object adapters and choose the right one
- Apply the Adapter pattern to real-world scenarios like third-party API integration and legacy system migration
- Avoid common pitfalls when writing adapters

Why This Chapter Matters

Every backend system eventually faces this problem: you have code that works, and you have new code that expects a different interface. Maybe you are integrating a third-party payment gateway that uses XML while your system speaks JSON. Maybe you inherited a legacy billing system that cannot be rewritten, but your new microservice expects a clean, modern API. Maybe a vendor changed their SDK, and now nothing compiles.

The Adapter pattern solves all of these problems without modifying the existing code. It is one of the most practical design patterns you will use in your career.

Think of it this way. You travel from the United States to Europe. Your laptop charger has a US plug, but European outlets have a different shape. You do not rewire your charger or

modify the wall outlet. You buy a plug adapter that sits between the two and makes them compatible.

```
US Plug                Adapter              EU Outlet
┌─────┐            ┌──────────────┐        ┌──────────┐
│ | | │──────────>│  | |  ○○   │──────>│  ○   ○   │
│     │            │  US   EU     │        │    ○     │
└─────┘            └──────────────┘        └──────────┘
(Your code)    (Adapter class)      (Incompatible interface)
```

The Adapter pattern does exactly this for software interfaces. It wraps an existing class with a new interface so that incompatible classes can work together.

The Problem

Suppose you are building an e-commerce platform. Your payment processing module defines a clean interface:

```java
public interface PaymentProcessor {
    PaymentResult processPayment(String customerId, double amount, String currency);
    PaymentStatus checkStatus(String transactionId);
    RefundResult refund(String transactionId, double amount);
}
```

Your application code uses this interface everywhere. Controllers, services, tests -- everything depends on `PaymentProcessor`.

Now your company acquires a smaller business that uses a completely different payment system:

```java
public class LegacyPaymentGateway {
    public int makePayment(String xmlPayload) { /* ... */ }
    public String getPaymentInfo(int paymentCode) { /* ... */ }
    public int reverseTransaction(int paymentCode, String xmlPayload) { /* ... */ }
}
```

The legacy system:

- Uses XML strings instead of typed parameters
- Returns integer status codes instead of result objects
- Uses different method names

- Has a completely different calling convention

You cannot change `PaymentProcessor` because dozens of classes depend on it. You cannot rewrite `LegacyPaymentGateway` because it is a third-party library with no source code. You need something in between.

The Solution: Adapter Pattern

The Adapter pattern creates a wrapper class that translates calls from one interface into calls on another.

```
┌──────────────────────────────────────────────────────────┐
│                    Adapter Pattern Structure              │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────┐        ┌──────────────┐      ┌──────────────┐
│   │  Client  │──────>│   Target     │      │  Adaptee     │
│   │          │        │  Interface   │      │  (Legacy)    │
│   └──────────┘        └──────────────┘      └──────────────┘
│                              │                     │
│                              │   implements        │
│                              │                     │
│                        ┌──────────────┐            │
│                        │   Adapter    │────────────┘
│                        │              │   wraps / delegates
│                        └──────────────┘
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Four participants:

1. Target -- the interface the client expects (`PaymentProcessor`)
2. Adaptee -- the existing class with an incompatible interface (`LegacyPaymentGateway`)
3. Adapter -- the class that bridges the two (`LegacyPaymentAdapter`)
4. Client -- the code that uses the Target interface (your services, controllers)

Class Adapter vs Object Adapter

There are two ways to implement the Adapter pattern:

Object Adapter (Composition)

The adapter holds a reference to the adaptee and delegates calls to it.

```
┌──────────────────┐          ┌──────────────────┐
│  <<interface>>   │          │  LegacyGateway   │
│ PaymentProcessor │          │                  │
├──────────────────┤          ├──────────────────┤
│ + processPayment │          │ + makePayment()  │
│ + checkStatus()  │          │ + getPaymentInfo │
│ + refund()       │          │ + reverseTx()    │
└──────────────────┘          └──────────────────┘
         │ implements                  │
         │                             │ has-a (composition)
         │                             │
     ┌──────────────────┐──────────────┘
     │ LegacyPayment    │
     │ Adapter          │
     ├──────────────────┤
     │ - legacyGateway  │
     ├──────────────────┤
     │ + processPayment()│  --> calls legacyGateway.makePayment()
     │ + checkStatus()   │  --> calls legacyGateway.getPaymentInfo()
     │ + refund()        │  --> calls legacyGateway.reverseTx()
     └──────────────────┘
```

Class Adapter (Inheritance)

The adapter extends the adaptee and implements the target interface.

```
┌──────────────────┐          ┌──────────────────┐
│  <<interface>>   │          │  LegacyGateway   │
│ PaymentProcessor │          │                  │
└──────────────────┘          └──────────────────┘
         │ implements                  │ extends
         │                             │
     ┌──────────────────────────────────┐
     │   LegacyPaymentAdapter           │
     ├──────────────────────────────────┤
     │ + processPayment()│  --> calls this.makePayment()
     │ + checkStatus()   │  --> calls this.getPaymentInfo()
     │ + refund()        │  --> calls this.reverseTx()
     └──────────────────────────────────┘
```

Which one should you use?

Criteria	Object Adapter	Class Adapter
Flexibility	Can adapt multiple adaptees	Can only adapt one class

Language support	Works in any language	Requires multiple inheritance
Runtime swapping	Can swap adaptee at runtime	Fixed at compile time
Access to adaptee internals	Only public methods	Can override protected methods
Recommendation	Preferred in most cases	Rarely used in Java/Python

In Java (single inheritance) and Python (duck typing), the object adapter is almost always the right choice.

Java Example: LegacyPaymentAdapter

Let us build a complete, working example.

Step 1: Define the Target Interface

```java
// The interface your application expects
public interface PaymentProcessor {

    PaymentResult processPayment(String customerId, double amount, String currency);

    PaymentStatus checkStatus(String transactionId);

    RefundResult refund(String transactionId, double amount);
}
```

Step 2: Define the Result Types

```java
public class PaymentResult {
    private final boolean success;
    private final String transactionId;
    private final String message;

    public PaymentResult(boolean success, String transactionId, String message) {
        this.success = success;
        this.transactionId = transactionId;
        this.message = message;
    }

    public boolean isSuccess() { return success; }
```

```
    public String getTransactionId() { return transactionId; }
    public String getMessage() { return message; }

    @Override
    public String toString() {
        return "PaymentResult{success=" + success
            + ", transactionId='" + transactionId + "'"
            + ", message='" + message + "'}";
    }
}

public enum PaymentStatus {
    PENDING, COMPLETED, FAILED, REFUNDED, UNKNOWN
}

public class RefundResult {
    private final boolean success;
    private final String message;

    public RefundResult(boolean success, String message) {
        this.success = success;
        this.message = message;
    }

    public boolean isSuccess() { return success; }
    public String getMessage() { return message; }

    @Override
    public String toString() {
        return "RefundResult{success=" + success + ", message='" + message + "'}";
    }
}
```

Step 3: The Legacy System (Adaptee)

This is the old system you cannot modify. It uses XML, integer codes, and completely different method signatures.

```java
// Third-party legacy code -- you CANNOT modify this
public class LegacyPaymentGateway {

    public int makePayment(String xmlPayload) {
        // Parses XML, processes payment, returns status code
        System.out.println("  [Legacy] Processing XML payload: " + xmlPayload);

        // Simulate: 0 = success, 1 = insufficient funds, 2 = error
        if (xmlPayload.contains("amount=\"0\"")) {
            return 2; // error for zero amount
        }
```

```
        return 0; // success
    }

    public String getPaymentInfo(int paymentCode) {
        // Returns XML string with payment details
        System.out.println("  [Legacy] Looking up payment code: " + paymentCode);
        return "<payment><code>" + paymentCode
            + "</code><status>COMPLETED</status></payment>";
    }

    public int reverseTransaction(int paymentCode, String xmlPayload) {
        System.out.println("  [Legacy] Reversing payment code: " + paymentCode);
        return 0; // success
    }

    // Legacy method to generate a payment code from a transaction
    public int generatePaymentCode(String customerId) {
        return Math.abs(customerId.hashCode() % 100000);
    }
}
```

Step 4: The Adapter

This is where the magic happens. The adapter implements `PaymentProcessor` and delegates to `LegacyPaymentGateway`.

```java
public class LegacyPaymentAdapter implements PaymentProcessor {

    private final LegacyPaymentGateway legacyGateway;

    // Store mapping between our transaction IDs and legacy payment codes
    private final Map<String, Integer> transactionToCodeMap = new HashMap<>();

    public LegacyPaymentAdapter(LegacyPaymentGateway legacyGateway) {
        this.legacyGateway = legacyGateway;
    }

    @Override
    public PaymentResult processPayment(String customerId, double amount, String curren...
        // Step 1: Convert our parameters to XML (what the legacy system expects)
        String xmlPayload = buildPaymentXml(customerId, amount, currency);

        // Step 2: Call the legacy system
        int resultCode = legacyGateway.makePayment(xmlPayload);

        // Step 3: Convert the legacy result to our result type
        String transactionId = generateTransactionId();
        int paymentCode = legacyGateway.generatePaymentCode(customerId);
        transactionToCodeMap.put(transactionId, paymentCode);
```

```
        return convertToPaymentResult(resultCode, transactionId);
    }

    @Override
    public PaymentStatus checkStatus(String transactionId) {
        // Look up the legacy payment code from our transaction ID
        Integer paymentCode = transactionToCodeMap.get(transactionId);
        if (paymentCode == null) {
            return PaymentStatus.UNKNOWN;
        }

        // Call legacy system and parse the XML response
        String xmlResponse = legacyGateway.getPaymentInfo(paymentCode);
        return parseStatusFromXml(xmlResponse);
    }

    @Override
    public RefundResult refund(String transactionId, double amount) {
        Integer paymentCode = transactionToCodeMap.get(transactionId);
        if (paymentCode == null) {
            return new RefundResult(false, "Transaction not found");
        }

        String xmlPayload = "<refund><amount>" + amount + "</amount></refund>";
        int resultCode = legacyGateway.reverseTransaction(paymentCode, xmlPayload);

        if (resultCode == 0) {
            return new RefundResult(true, "Refund processed successfully");
        }
        return new RefundResult(false, "Refund failed with code: " + resultCode);
    }

    // --- Private helper methods for translation ---

    private String buildPaymentXml(String customerId, double amount, String currency) {
        return "<payment>"
            + "<customer>" + customerId + "</customer>"
            + "<amount=\"" + amount + "\"/>"
            + "<currency>" + currency + "</currency>"
            + "</payment>";
    }

    private String generateTransactionId() {
        return "TXN-" + System.currentTimeMillis();
    }

    private PaymentResult convertToPaymentResult(int legacyCode, String transactionId) {
        switch (legacyCode) {
            case 0:
                return new PaymentResult(true, transactionId, "Payment successful");
            case 1:
                return new PaymentResult(false, transactionId, "Insufficient funds");
            default:
                return new PaymentResult(false, transactionId,
```

```
                    "Payment failed with legacy code: " + legacyCode);
        }
    }

    private PaymentStatus parseStatusFromXml(String xml) {
        if (xml.contains("COMPLETED")) return PaymentStatus.COMPLETED;
        if (xml.contains("PENDING"))   return PaymentStatus.PENDING;
        if (xml.contains("FAILED"))    return PaymentStatus.FAILED;
        if (xml.contains("REFUNDED"))  return PaymentStatus.REFUNDED;
        return PaymentStatus.UNKNOWN;
    }
}
```

Step 5: Using the Adapter

```java
public class PaymentService {

    private final PaymentProcessor paymentProcessor;

    // The service only knows about PaymentProcessor -- it has no idea
    // whether it is talking to the new system or the legacy one
    public PaymentService(PaymentProcessor paymentProcessor) {
        this.paymentProcessor = paymentProcessor;
    }

    public void checkout(String customerId, double total) {
        System.out.println("Processing checkout for customer: " + customerId);

        PaymentResult result = paymentProcessor.processPayment(
            customerId, total, "USD"
        );
        System.out.println("Payment result: " + result);

        if (result.isSuccess()) {
            PaymentStatus status = paymentProcessor.checkStatus(
                result.getTransactionId()
            );
            System.out.println("Payment status: " + status);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // Create the legacy system (cannot modify this)
        LegacyPaymentGateway legacyGateway = new LegacyPaymentGateway();

        // Wrap it with our adapter
        PaymentProcessor adapter = new LegacyPaymentAdapter(legacyGateway);
```

```
        // Use it in our service -- the service does not know about the legacy system
        PaymentService service = new PaymentService(adapter);
        service.checkout("CUST-42", 99.99);
    }
}
```

Output:

```
Processing checkout for customer: CUST-42
  [Legacy] Processing XML payload: <payment><customer>CUST-42</customer>...
Payment result: PaymentResult{success=true, transactionId='TXN-1703001234567', message=...
  [Legacy] Looking up payment code: 28456
Payment status: COMPLETED
```

Before vs After

Before (without adapter) -- tight coupling to legacy system:

```java
// Every service that needs payments must deal with XML and integer codes
public class CheckoutService {
    private LegacyPaymentGateway gateway = new LegacyPaymentGateway();

    public void checkout(String customerId, double total) {
        // Build XML manually in every service method
        String xml = "<payment><customer>" + customerId + "</customer>"
            + "<amount>" + total + "</amount></payment>";
        int code = gateway.makePayment(xml);

        // Interpret integer codes in every service method
        if (code == 0) {
            System.out.println("Success");
        } else if (code == 1) {
            System.out.println("Insufficient funds");
        }
        // XML parsing scattered everywhere...
    }
}
```

After (with adapter) -- clean separation:

```java
public class CheckoutService {
    private PaymentProcessor processor; // clean interface

    public CheckoutService(PaymentProcessor processor) {
        this.processor = processor;
```

```
    }

    public void checkout(String customerId, double total) {
        PaymentResult result = processor.processPayment(customerId, total, "USD");
        if (result.isSuccess()) {
            System.out.println("Payment confirmed: " + result.getTransactionId());
        }
    }
}
```

Python Example: XML-to-JSON Adapter

Python's duck typing makes adapters even more natural. Here is a real-world scenario: you have an analytics service that consumes JSON, but a legacy reporting system produces XML.

The Problem

```python
# Your modern analytics service expects data like this:
class AnalyticsService:
    """Processes analytics data in JSON (dict) format."""

    def process_report(self, data: dict) -> None:
        """Expects a dict with keys: 'report_id', 'metrics', 'timestamp'."""
        print(f"Processing report: {data['report_id']}")
        for metric, value in data['metrics'].items():
            print(f"  {metric}: {value}")
        print(f"  Generated at: {data['timestamp']}")

    def aggregate_reports(self, reports: list[dict]) -> dict:
        """Combines multiple report dicts into a summary."""
        total_metrics = {}
        for report in reports:
            for metric, value in report['metrics'].items():
                total_metrics[metric] = total_metrics.get(metric, 0) + value
        return {"summary": total_metrics, "report_count": len(reports)}
```

```python
# But the legacy reporting system produces XML strings
class LegacyReportingSystem:
    """Old system that generates reports as XML strings."""

    def generate_daily_report(self) -> str:
        return """<?xml version="1.0"?>
        <report>
```

```
            <report_id>RPT-2024-001</report_id>
            <timestamp>2024-01-15T10:30:00Z</timestamp>
            <metrics>
                <metric name="page_views" value="15234"/>
                <metric name="unique_visitors" value="4521"/>
                <metric name="bounce_rate" value="34"/>
                <metric name="avg_session_minutes" value="7"/>
            </metrics>
        </report>"""

    def generate_weekly_report(self) -> str:
        return """<?xml version="1.0"?>
        <report>
            <report_id>RPT-2024-W03</report_id>
            <timestamp>2024-01-21T00:00:00Z</timestamp>
            <metrics>
                <metric name="page_views" value="98432"/>
                <metric name="unique_visitors" value="28910"/>
                <metric name="bounce_rate" value="31"/>
                <metric name="avg_session_minutes" value="8"/>
            </metrics>
        </report>"""
```

These two systems cannot talk to each other directly.

The Adapter

```python
import xml.etree.ElementTree as ET


class XMLToJSONReportAdapter:
    """Adapts the LegacyReportingSystem to produce JSON-compatible dicts."""

    def __init__(self, legacy_system: LegacyReportingSystem):
        self._legacy = legacy_system

    def get_daily_report(self) -> dict:
        """Gets the daily report as a dict (JSON-compatible)."""
        xml_string = self._legacy.generate_daily_report()
        return self._xml_to_dict(xml_string)

    def get_weekly_report(self) -> dict:
        """Gets the weekly report as a dict (JSON-compatible)."""
        xml_string = self._legacy.generate_weekly_report()
        return self._xml_to_dict(xml_string)

    def get_all_reports(self) -> list[dict]:
        """Gets all available reports as a list of dicts."""
        return [self.get_daily_report(), self.get_weekly_report()]
```

```
    def _xml_to_dict(self, xml_string: str) -> dict:
        """Converts an XML report string to a dict matching the expected format."""
        root = ET.fromstring(xml_string)

        # Extract report_id and timestamp
        report_id = root.find("report_id").text
        timestamp = root.find("timestamp").text

        # Extract metrics into a flat dict
        metrics = {}
        for metric_elem in root.findall(".//metric"):
            name = metric_elem.get("name")
            value = int(metric_elem.get("value"))
            metrics[name] = value

        return {
            "report_id": report_id,
            "timestamp": timestamp,
            "metrics": metrics,
        }
```

Using the Adapter

```python
def main():
    # The legacy system (cannot modify)
    legacy = LegacyReportingSystem()

    # The adapter (bridges the gap)
    adapter = XMLToJSONReportAdapter(legacy)

    # The modern service (expects dicts)
    analytics = AnalyticsService()

    # Now they work together seamlessly
    print("=== Daily Report ===")
    daily = adapter.get_daily_report()
    analytics.process_report(daily)

    print("\n=== Weekly Report ===")
    weekly = adapter.get_weekly_report()
    analytics.process_report(weekly)

    print("\n=== Aggregated Summary ===")
    all_reports = adapter.get_all_reports()
    summary = analytics.aggregate_reports(all_reports)
    print(f"Total reports: {summary['report_count']}")
    for metric, total in summary['summary'].items():
        print(f"  {metric}: {total}")
```

```
if __name__ == "__main__":
    main()
```

Output:

```
=== Daily Report ===
Processing report: RPT-2024-001
  page_views: 15234
  unique_visitors: 4521
  bounce_rate: 34
  avg_session_minutes: 7
  Generated at: 2024-01-15T10:30:00Z

=== Weekly Report ===
Processing report: RPT-2024-W03
  page_views: 98432
  unique_visitors: 28910
  bounce_rate: 31
  avg_session_minutes: 8
  Generated at: 2024-01-21T00:00:00Z

=== Aggregated Summary ===
Total reports: 2
  page_views: 113666
  unique_visitors: 33431
  bounce_rate: 65
  avg_session_minutes: 15
```

Real-World Backend Use Cases

Use Case 1: Third-Party API Integration

You build a notification service with a clean `NotificationSender` interface. Later, you need to integrate with Twilio (SMS), SendGrid (email), and Firebase (push notifications). Each has a completely different API. You write an adapter for each:

```
┌──────────────────┐
│ NotificationSender│
│   (interface)     │
└──────────────────┘
         │
         │
    ┌────────────────────────────────────┐
    │              │                     │
    │              │                     │
┌───┴──────────┐  ┌──────────────┐  ┌────────────────┐
```

```
│TwilioAdapter│ │SendGridAdapter│ │FirebaseAdapter │
│            │  │              │  │               │
│wraps Twilio│  │wraps SendGrid│  │wraps Firebase │
│   SDK      │  │    SDK       │  │     SDK       │
└────────────┘  └──────────────┘  └───────────────┘
```

Use Case 2: Legacy System Migration

You are migrating from a monolith to microservices. During the transition, some services still call the monolith's database directly. You write adapters that expose the monolith's data through the new microservice interfaces:

```
┌──────────────────┐          ┌──────────────────┐
│  New Microservice │          │  Legacy Monolith  │
│  UserService API  │          │  UserDAO          │
│                  │         │                  │
│  getUser(id)     │          │  findByPK(table,  │
│  createUser(dto) │          │    column, value) │
│  updateUser(dto) │          │  insertRow(table, │
└──────────────────┘          │    data)          │
         │                     └──────────────────┘
         │                                │
         │ implements                     │ delegates to
    ┌────┴──────────────────┐            │
    │ LegacyUserAdapter     │────────────┘
    │                       │
    │ Translates clean      │
    │ API calls to raw      │
    │ monolith queries      │
    └───────────────────────┘
```

Use Case 3: Database Driver Abstraction

Your application uses a `DataStore` interface. You need to support PostgreSQL, MongoDB, and Redis. Each driver has different APIs:

```java
public interface DataStore {
    void save(String key, Object value);
    Object find(String key);
    void delete(String key);
}

public class MongoAdapter implements DataStore {
    private MongoCollection<Document> collection;

    @Override
```

```
    public void save(String key, Object value) {
        // Translates to MongoDB's insertOne / replaceOne
        Document doc = new Document("_id", key).append("data", value);
        collection.replaceOne(eq("_id", key), doc, new ReplaceOptions().upsert(true));
    }

    @Override
    public Object find(String key) {
        // Translates to MongoDB's find
        Document doc = collection.find(eq("_id", key)).first();
        return doc != null ? doc.get("data") : null;
    }

    @Override
    public void delete(String key) {
        collection.deleteOne(eq("_id", key));
    }
}
```

When to Use / When NOT to Use

Use the Adapter Pattern When:

- You need to use an existing class whose interface does not match what you need
- You want to create a reusable class that cooperates with unrelated classes
- You need to integrate a third-party library or legacy system
- You are migrating from one API to another and need a transition layer
- You want to isolate your code from external dependencies for easier testing

Do NOT Use the Adapter Pattern When:

- You can simply modify the existing interface (and it is your code)
- The interfaces are already compatible -- do not add unnecessary layers
- You need to add new behavior, not just translate interfaces (use Decorator instead)
- The adaptation is trivial -- a simple method call does not need a full adapter class
- You are adapting too many things -- if everything needs an adapter, rethink your architecture

---

Common Mistakes

Mistake 1: Making the Adapter Too Smart

```java
// BAD: The adapter is doing business logic, not just translating
public class BadAdapter implements PaymentProcessor {
    @Override
    public PaymentResult processPayment(String customerId, double amount, String curren...
        // Adapter should NOT validate business rules
        if (amount > 10000) {
            throw new IllegalArgumentException("Amount too high");
        }
        // Adapter should NOT apply discounts
        double discountedAmount = amount * 0.95;
        return convertResult(legacy.makePayment(buildXml(customerId, discountedAmount, ...
    }
}

// GOOD: The adapter only translates between interfaces
public class GoodAdapter implements PaymentProcessor {
    @Override
    public PaymentResult processPayment(String customerId, double amount, String curren...
        String xml = buildXml(customerId, amount, currency);
        int legacyResult = legacy.makePayment(xml);
        return convertResult(legacyResult);
    }
}
```

An adapter translates. It does not add business logic.

Mistake 2: Leaking the Adaptee's Interface

```java
// BAD: Exposing legacy details through the adapter
public class LeakyAdapter implements PaymentProcessor {
    public LegacyPaymentGateway getLegacyGateway() {
        return this.legacyGateway; // Clients can bypass the adapter
    }

    public int getRawLegacyCode(String transactionId) {
        return this.legacyGateway.getPaymentInfo(/* ... */); // Legacy types leak out
    }
}
```

If clients can access the adaptee directly, the adapter is pointless.

Mistake 3: Creating One Adapter per Method Instead of per Interface

```java
// BAD: Separate adapter for each operation
public class PaymentAdapter { /* ... */ }
public class StatusAdapter { /* ... */ }
public class RefundAdapter { /* ... */ }

// GOOD: One adapter per incompatible interface
public class LegacyPaymentAdapter implements PaymentProcessor {
    // Handles all payment operations in one place
}
```

Best Practices

1. One adapter per incompatible interface. Group related translations together.
2. Keep adapters thin. Translation logic only. No business rules, no validation, no side effects beyond what the adaptee does.
3. Use composition over inheritance. Object adapters (wrapping via a field) are more flexible than class adapters (extending the adaptee).
4. Name adapters clearly. `LegacyPaymentAdapter`, `TwilioNotificationAdapter`, `MongoDataStoreAdapter`. The name should say what is being adapted.
5. Write integration tests for adapters. Adapters are boundary code. Test that the translation is correct, especially edge cases (null values, empty results, error codes).
6. Consider a two-way adapter when both systems need to communicate. Sometimes you need to translate in both directions.
7. Document the mapping. A comment or table showing which target method maps to which adaptee method helps the next developer understand the translation.

Quick Summary

The Adapter pattern wraps an existing class with a new interface, allowing incompatible classes to work together without modifying either one. It acts as a translator between two interfaces, just like a power plug adapter lets you use a US device in a European outlet.

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│   Client     │────>│   Adapter    │────>│   Adaptee    │
│ (your code)  │     │ (translator) │     │ (legacy/3rd  │
│              │     │              │     │   party)     │
│ Calls target │     │ Implements   │     │ Has its own  │
│ interface    │     │ target,      │     │ interface    │
│              │     │ wraps        │     │              │
│              │     │ adaptee      │     │              │
└──────────────┘     └──────────────┘     └──────────────┘
```

Key Points

- The Adapter pattern converts the interface of a class into another interface that clients expect
- Object adapters use composition (preferred); class adapters use inheritance
- Adapters should be thin -- they translate, they do not add business logic
- The pattern is essential for integrating third-party libraries and legacy systems
- Name your adapters clearly: describe what is being adapted
- Keep the adaptee hidden behind the adapter -- do not leak its interface
- Adapters make your code testable by allowing you to mock external dependencies

---

Practice Questions

1. You have a `FileStorage` interface with `upload(filename, bytes)` and `download(filename)` methods. You need to integrate with an AWS S3 SDK that uses `putObject(PutObjectRequest)` and `getObject(GetObjectRequest)`. Sketch the adapter class. What translations does it perform?
2. Your team is debating whether to use a class adapter or an object adapter. The adaptee has several protected methods that contain useful logic. Which approach would you recommend, and why?
3. A colleague suggests putting input validation inside the adapter (for example, checking that the payment amount is positive before calling the legacy system). Is this a good idea? Why or why not?
4. You have three different payment gateways, each with its own SDK. How would the Adapter pattern help you support all three? Draw the class diagram.

5. How does the Adapter pattern improve testability? Give a concrete example of how you would unit test a service that uses a `PaymentProcessor` without needing the actual legacy system.

Exercises

Exercise 1: Build a Temperature Sensor Adapter

You have a `TemperatureSensor` interface that returns temperature in Celsius. You receive a third-party sensor library that only returns Fahrenheit with a different method signature:

```java
// Your interface
public interface TemperatureSensor {
    double readCelsius();
    String getSensorId();
}

// Third-party library (cannot modify)
public class FahrenheitSensor {
    public float getTemperatureF() { return 72.5f; }
    public int getDeviceCode() { return 1042; }
}
```

Write a `FahrenheitSensorAdapter` that makes `FahrenheitSensor` work with your `TemperatureSensor` interface. Include the Fahrenheit-to-Celsius conversion.

Exercise 2: Python REST-to-GraphQL Adapter

You have a `DataFetcher` protocol that uses REST-style calls:

```python
class DataFetcher(Protocol):
    def get(self, endpoint: str, params: dict) -> dict: ...
    def post(self, endpoint: str, data: dict) -> dict: ...
```

Write a `GraphQLAdapter` that translates these REST-style calls into GraphQL queries against a GraphQL client. For `get("/users", {"id": 5})`, it should build and execute a GraphQL query.

Exercise 3: Multi-Adapter Registry

Design a system where you can register multiple adapters for different legacy systems and select the right one at runtime based on a configuration flag. Use the Factory pattern (from

Chapter 3) together with the Adapter pattern.

What Is Next?

You now know how to make incompatible interfaces work together. But what if you want to add new behavior to an existing object without modifying it? What if you want to layer logging on top of caching on top of authentication -- all without changing the original class?

In the next chapter, you will learn the Decorator pattern, which lets you wrap objects with new behavior like Russian nesting dolls. Where the Adapter changes an interface, the Decorator enhances it.

Chapter 8: The Decorator Pattern

What You Will Learn

By the end of this chapter, you will be able to:

- Understand the Decorator pattern and how it adds behavior dynamically
- Explain why composition is often better than inheritance for extending behavior
- Build decorator chains in Java using the InputStream pattern
- Use Python function decorators (`@retry`, `@cache`, `@log`) for cross-cutting concerns
- Recognize the Decorator pattern in real-world backend systems: logging, caching, auth middleware, response compression
- Compare the Decorator pattern with inheritance and know when each is appropriate
- Avoid common mistakes like overly deep decorator chains and ordering bugs

Why This Chapter Matters

Every backend system needs cross-cutting concerns. You need logging. You need caching. You need authentication checks. You need rate limiting. You need response compression. You need retry logic.

The naive approach is to shove all of this into every class. Your `UserService` does user logic and logging and caching and auth checks. Your `OrderService` does the same. Code is duplicated. Classes become bloated. Adding a new concern means modifying dozens of files.

The Decorator pattern solves this by letting you wrap an object with additional behavior, one layer at a time, without modifying the original class. Think of it like Russian nesting dolls (matryoshka dolls):

```
┌──────────────────────────────────────────────────┐
│  Logging Decorator                               │
│  ┌──────────────────────────────────────────┐    │
│  │  Caching Decorator                       │    │
│  │  ┌──────────────────────────────────┐    │    │
│  │  │  Auth Decorator                  │    │    │
│  │  │  ┌──────────────────────────┐    │    │    │
│  │  │  │                          │    │    │    │
│  │  │  │   Original Service       │    │    │    │
│  │  │  │   (core business logic)  │    │    │    │
│  │  │  │                          │    │    │    │
│  │  │  └──────────────────────────┘    │    │    │
│  │  └──────────────────────────────────┘    │    │
│  └──────────────────────────────────────────┘    │
└──────────────────────────────────────────────────┘
```

Each layer adds one behavior. The original service is untouched. You can mix and match layers as needed.

The Problem

You have a data service interface:

```java
public interface DataService {
    String getData(String key);
    void saveData(String key, String value);
}
```

And a basic implementation:

```java
public class DatabaseService implements DataService {
    @Override
    public String getData(String key) {
        // Simulate database query
        System.out.println("  [DB] Querying database for key: " + key);
        return "value_for_" + key;
    }

    @Override
    public void saveData(String key, String value) {
        System.out.println("  [DB] Saving to database: " + key + " = " + value);
    }
}
```

Now you need to add logging, caching, and authentication. The inheritance approach looks like this:

```
                    DataService
                        |
                DatabaseService
                        |
           LoggingDatabaseService
                        |
     CachingLoggingDatabaseService
                        |
AuthCachingLoggingDatabaseService
```

This is a nightmare. What if you want caching without logging? What if you want auth without caching? You end up with a class for every combination:

```
DatabaseService
LoggingDatabaseService
CachingDatabaseService
AuthDatabaseService
LoggingCachingDatabaseService
LoggingAuthDatabaseService
CachingAuthDatabaseService
LoggingCachingAuthDatabaseService
```

That is 8 classes for 3 features. With 4 features, you need 16 classes. With 5 features, 32 classes. This is called the class explosion problem.

The Solution: Decorator Pattern

Instead of inheritance, use composition. Each decorator wraps the original object and adds one behavior.

```
┌──────────────────────────────────────────────────────────┐
│                  Decorator Pattern Structure              │
├──────────────────────────────────────────────────────────┤
│                                                          │
│    ┌──────────────┐                                      │
│    │  <<interface>>│                                      │
│    │  DataService  │                                      │
│    └──────────────┘                                      │
│          │                                               │
│          │                                               │
│     ┌────────────────────────────┐                       │
```

```
│    │                                │                    │
│    │                                │                    │
│   ┌──────────────┐   ┌──────────────┴──────────┐         │
│   │DatabaseService│   │  <<abstract>>           │         │
│   │(concrete)     │   │  DataServiceDecorator   │         │
│   └──────────────┘   │  - wrapped: DataService │         │
│                      └─────────────────────────┘         │
│                                │                         │
│                                │                         │
│                ┌───────────────┴────────────┐            │
│                │                            │            │
│                │                            │            │
│      ┌─────────┴──────┐   ┌─────────┴──────┐   ┌─────────┴──────┐
│      │  Logging       │   │  Caching       │   │   Auth         │
│      │  Decorator     │   │  Decorator     │   │  Decorator     │
│      └────────────────┘   └────────────────┘   └────────────────┘
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Key insight: every decorator is a `DataService` and has a `DataService`. This lets you stack them.

Java Example: Building a Decorator Chain

Step 1: The Base Decorator

```java
public abstract class DataServiceDecorator implements DataService {

    protected final DataService wrapped;

    public DataServiceDecorator(DataService wrapped) {
        this.wrapped = wrapped;
    }

    @Override
    public String getData(String key) {
        return wrapped.getData(key);
    }

    @Override
    public void saveData(String key, String value) {
        wrapped.saveData(key, value);
    }
}
```

By default, the decorator simply delegates to the wrapped object. Subclasses override methods to add behavior.

Step 2: Logging Decorator

```java
public class LoggingDecorator extends DataServiceDecorator {

    private static final Logger logger = Logger.getLogger(LoggingDecorator.class.getNam...

    public LoggingDecorator(DataService wrapped) {
        super(wrapped);
    }

    @Override
    public String getData(String key) {
        long start = System.currentTimeMillis();
        logger.info("getData() called with key: " + key);

        String result = super.getData(key); // delegate to wrapped service

        long elapsed = System.currentTimeMillis() - start;
        logger.info("getData() returned in " + elapsed + "ms. Result length: "
            + (result != null ? result.length() : 0));
        return result;
    }

    @Override
    public void saveData(String key, String value) {
        logger.info("saveData() called with key: " + key
            + ", value length: " + value.length());

        super.saveData(key, value); // delegate to wrapped service

        logger.info("saveData() completed for key: " + key);
    }
}
```

Step 3: Caching Decorator

```java
public class CachingDecorator extends DataServiceDecorator {

    private final Map<String, String> cache = new ConcurrentHashMap<>();

    public CachingDecorator(DataService wrapped) {
        super(wrapped);
    }

    @Override
    public String getData(String key) {
        // Check cache first
        String cached = cache.get(key);
        if (cached != null) {
```

```
            System.out.println("  [Cache] HIT for key: " + key);
            return cached;
        }

        System.out.println("  [Cache] MISS for key: " + key);
        String result = super.getData(key); // delegate to wrapped service

        // Store in cache
        if (result != null) {
            cache.put(key, result);
        }
        return result;
    }

    @Override
    public void saveData(String key, String value) {
        super.saveData(key, value); // delegate to wrapped service
        cache.put(key, value); // update cache
        System.out.println("  [Cache] Updated cache for key: " + key);
    }
}
```

Step 4: Authentication Decorator

```java
public class AuthDecorator extends DataServiceDecorator {

    private final String requiredRole;
    private String currentUserRole;

    public AuthDecorator(DataService wrapped, String requiredRole) {
        super(wrapped);
        this.requiredRole = requiredRole;
    }

    public void setCurrentUserRole(String role) {
        this.currentUserRole = role;
    }

    @Override
    public String getData(String key) {
        checkAccess("getData");
        return super.getData(key);
    }

    @Override
    public void saveData(String key, String value) {
        checkAccess("saveData");
        super.saveData(key, value);
    }
```

```
    private void checkAccess(String operation) {
        if (currentUserRole == null || !currentUserRole.equals(requiredRole)) {
            throw new SecurityException(
                "Access denied for operation '" + operation
                + "'. Required role: " + requiredRole
                + ", current role: " + currentUserRole
            );
        }
        System.out.println("  [Auth] Access granted for role: " + currentUserRole);
    }
}
```

Step 5: Composing Decorators

```java
public class Main {
    public static void main(String[] args) {
        // Start with the base service
        DataService service = new DatabaseService();

        // Wrap with caching
        service = new CachingDecorator(service);

        // Wrap with logging
        service = new LoggingDecorator(service);

        // Now: Logging -> Caching -> Database
        // When getData() is called:
        //   1. LoggingDecorator logs the call
        //   2. CachingDecorator checks cache
        //   3. If cache miss, DatabaseService queries DB

        System.out.println("=== First call (cache miss) ===");
        String result1 = service.getData("user:42");
        System.out.println("Result: " + result1);

        System.out.println("\n=== Second call (cache hit) ===");
        String result2 = service.getData("user:42");
        System.out.println("Result: " + result2);

        System.out.println("\n=== Save data ===");
        service.saveData("user:99", "Alice");
    }
}
```

Output:

```
=== First call (cache miss) ===
INFO: getData() called with key: user:42
  [Cache] MISS for key: user:42
```

```
  [DB] Querying database for key: user:42
INFO: getData() returned in 5ms. Result length: 14
Result: value_for_user:42

=== Second call (cache hit) ===
INFO: getData() called with key: user:42
  [Cache] HIT for key: user:42
INFO: getData() returned in 0ms. Result length: 14
Result: value_for_user:42

=== Save data ===
INFO: saveData() called with key: user:99, value length: 5
  [DB] Saving to database: user:99 = Alice
  [Cache] Updated cache for key: user:99
INFO: saveData() completed for key: user:99
```

Notice the second call never hits the database. The caching decorator intercepted it.

The Java InputStream Chain: A Classic Decorator

Java's I/O library is one of the most famous examples of the Decorator pattern in the real world:

```java
// Each wrapper adds one capability
InputStream raw       = new FileInputStream("data.txt");       // reads bytes
InputStream buffered  = new BufferedInputStream(raw);           // adds buffering
InputStream data      = new DataInputStream(buffered);          // adds typed reads

// The chain: DataInputStream -> BufferedInputStream -> FileInputStream -> File
// Each layer IS an InputStream and HAS an InputStream
```

```
┌──────────────────────────────────────────────┐
│  DataInputStream                             │
│  (read Java primitives: readInt, readDouble) │
│  ┌──────────────────────────────────────┐    │
│  │  BufferedInputStream                 │    │
│  │  (read chunks into memory buffer)    │    │
│  │  ┌──────────────────────────────┐    │    │
│  │  │  FileInputStream             │    │    │
│  │  │  (read raw bytes from disk)  │    │    │
│  │  └──────────────────────────────┘    │    │
│  └──────────────────────────────────────┘    │
└──────────────────────────────────────────────┘
```

Before vs After

Before (without decorators) -- everything in one class:

```java
public class DoEverythingService implements DataService {
    private Map<String, String> cache = new HashMap<>();
    private Logger logger = Logger.getLogger("service");
    private String requiredRole = "ADMIN";
    private String currentRole;

    @Override
    public String getData(String key) {
        // Auth check
        if (!requiredRole.equals(currentRole)) throw new SecurityException("Denied");

        // Logging
        logger.info("getData called: " + key);
        long start = System.currentTimeMillis();

        // Caching
        if (cache.containsKey(key)) {
            logger.info("Cache hit");
            return cache.get(key);
        }

        // Actual work
        String result = queryDatabase(key);

        // More caching
        cache.put(key, result);

        // More logging
        logger.info("getData completed in " + (System.currentTimeMillis() - start) + "m...
        return result;
    }
    // Imagine saveData() with the same mess...
}
```

After (with decorators) -- each concern is separate:

```java
DataService service = new DatabaseService();              // core logic
service = new CachingDecorator(service);                  // + caching
service = new LoggingDecorator(service);                  // + logging
service = new AuthDecorator(service, "ADMIN");            // + auth
// Clean. Each class has one responsibility.
```

Python Example: Function Decorators

Python has built-in syntax for the Decorator pattern using the `@` symbol. This is so fundamental to Python that it has its own language feature.

The @retry Decorator

```python
import time
import functools
import random


def retry(max_attempts=3, delay_seconds=1.0, exceptions=(Exception,)):
    """Retry a function call if it raises specified exceptions."""

    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            last_exception = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    last_exception = e
                    print(f"  [Retry] Attempt {attempt}/{max_attempts} failed: {e}")
                    if attempt < max_attempts:
                        time.sleep(delay_seconds)
            raise last_exception

        return wrapper

    return decorator


# Usage
@retry(max_attempts=3, delay_seconds=0.5, exceptions=(ConnectionError,))
def fetch_user_data(user_id: int) -> dict:
    """Simulates an unreliable API call."""
    if random.random() < 0.6:  # 60% chance of failure
        raise ConnectionError(f"Failed to connect to user service")
    return {"user_id": user_id, "name": "Alice", "email": "alice@example.com"}


# When you call fetch_user_data(42), the retry decorator
# automatically retries up to 3 times if ConnectionError is raised
result = fetch_user_data(42)
print(f"Got user: {result}")
```

Output (example run):

```
  [Retry] Attempt 1/3 failed: Failed to connect to user service
  [Retry] Attempt 2/3 failed: Failed to connect to user service
Got user: {'user_id': 42, 'name': 'Alice', 'email': 'alice@example.com'}
```

The @cache Decorator

```python
import functools
import time


def cache(ttl_seconds=60):
    """Cache function results with a time-to-live."""

    def decorator(func):
        _cache = {}

        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            # Create a hashable cache key from the arguments
            cache_key = (args, tuple(sorted(kwargs.items())))

            # Check if cached and not expired
            if cache_key in _cache:
                result, cached_at = _cache[cache_key]
                age = time.time() - cached_at
                if age < ttl_seconds:
                    print(f"  [Cache] HIT (age: {age:.1f}s)")
                    return result
                else:
                    print(f"  [Cache] EXPIRED (age: {age:.1f}s)")

            print(f"  [Cache] MISS -- calling {func.__name__}")
            result = func(*args, **kwargs)
            _cache[cache_key] = (result, time.time())
            return result

        # Expose cache management methods
        wrapper.clear_cache = lambda: _cache.clear()
        wrapper.cache_size = lambda: len(_cache)
        return wrapper

    return decorator


@cache(ttl_seconds=30)
def get_product_catalog(category: str) -> list:
    """Expensive database query to fetch products."""
    print(f"  [DB] Querying products for category: {category}")
    time.sleep(0.5)  # simulate slow query
    return [
        {"id": 1, "name": "Widget A", "category": category},
        {"id": 2, "name": "Widget B", "category": category},
    ]


# First call: cache miss
products = get_product_catalog("electronics")
print(f"Products: {len(products)} items\n")
```

```
# Second call: cache hit
products = get_product_catalog("electronics")
print(f"Products: {len(products)} items")
```

Output:

```
  [Cache] MISS -- calling get_product_catalog
  [DB] Querying products for category: electronics
Products: 2 items

  [Cache] HIT (age: 0.5s)
Products: 2 items
```

The @log Decorator

```python
import functools
import logging
import time

logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(message)s")
logger = logging.getLogger(__name__)


def log(func):
    """Log function entry, exit, arguments, and duration."""

    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        func_name = func.__name__
        args_repr = [repr(a) for a in args]
        kwargs_repr = [f"{k}={v!r}" for k, v in kwargs.items()]
        signature = ", ".join(args_repr + kwargs_repr)

        logger.info(f"Calling {func_name}({signature})")
        start = time.time()

        try:
            result = func(*args, **kwargs)
            elapsed = time.time() - start
            logger.info(f"{func_name} returned {result!r} in {elapsed:.3f}s")
            return result
        except Exception as e:
            elapsed = time.time() - start
            logger.error(f"{func_name} raised {type(e).__name__}: {e} after {elapsed:.3...
            raise

    return wrapper
```

```
@log
def calculate_discount(price: float, discount_percent: float) -> float:
    """Calculate discounted price."""
    if discount_percent < 0 or discount_percent > 100:
        raise ValueError(f"Invalid discount: {discount_percent}")
    return price * (1 - discount_percent / 100)


# Normal call
result = calculate_discount(100.0, 15.0)
print(f"Discounted price: ${result:.2f}\n")

# Call that raises an exception
try:
    calculate_discount(100.0, -5.0)
except ValueError:
    print("Caught the error (but it was logged automatically)")
```

Output:

```
2024-01-15 10:30:00 - Calling calculate_discount(100.0, 15.0)
2024-01-15 10:30:00 - calculate_discount returned 85.0 in 0.000s
Discounted price: $85.00

2024-01-15 10:30:00 - Calling calculate_discount(100.0, -5.0)
2024-01-15 10:30:00 - calculate_discount raised ValueError: Invalid discount: -5.0 afte...
Caught the error (but it was logged automatically)
```

Stacking Python Decorators

The real power shows when you stack decorators. Decorators are applied bottom-up but execute top-down:

```python
@log                     # 3rd applied, 1st to execute
@retry(max_attempts=3)   # 2nd applied, 2nd to execute
@cache(ttl_seconds=300) # 1st applied, 3rd to execute
def fetch_exchange_rate(base: str, target: str) -> float:
    """Fetch currency exchange rate from external API."""
    print(f"  [API] Fetching {base}/{target} rate...")
    if random.random() < 0.3:
        raise ConnectionError("Rate service unavailable")
    return 1.08  # simulated EUR/USD rate
```

Execution flow:

```
    Call: fetch_exchange_rate("EUR", "USD")
         │
         ▼
    ┌──────────────┐
    │  @log        │  1. Log the call
    │              │
    │  ┌─────────┐ │
    │  │ @retry  │ │  2. Retry on failure
    │  │         │ │
    │  │ ┌─────┐ │ │
    │  │ │@cache│ │ │  3. Check cache first
    │  │ │     │ │ │
    │  │ │ func│ │ │  4. Call actual function
    │  │ │     │ │ │
    │  │ └─────┘ │ │
    │  └─────────┘ │
    └──────────────┘
```

Python Class-Based Decorators

For more complex scenarios, you can use class-based decorators that mirror the Java approach:

```python
from abc import ABC, abstractmethod


class DataService(ABC):
    """Target interface."""

    @abstractmethod
    def get_data(self, key: str) -> str:
        pass

    @abstractmethod
    def save_data(self, key: str, value: str) -> None:
        pass


class DatabaseService(DataService):
    """Concrete implementation."""

    def get_data(self, key: str) -> str:
        print(f"  [DB] Querying: {key}")
        return f"value_for_{key}"

    def save_data(self, key: str, value: str) -> None:
        print(f"  [DB] Saving: {key} = {value}")
```

```
class DataServiceDecorator(DataService):
    """Base decorator -- delegates everything to the wrapped service."""

    def __init__(self, wrapped: DataService):
        self._wrapped = wrapped

    def get_data(self, key: str) -> str:
        return self._wrapped.get_data(key)

    def save_data(self, key: str, value: str) -> None:
        self._wrapped.save_data(key, value)


class LoggingDecorator(DataServiceDecorator):
    def get_data(self, key: str) -> str:
        print(f"  [Log] get_data({key})")
        result = super().get_data(key)
        print(f"  [Log] get_data returned: {result}")
        return result

    def save_data(self, key: str, value: str) -> None:
        print(f"  [Log] save_data({key}, {value})")
        super().save_data(key, value)
        print(f"  [Log] save_data completed")


class CachingDecorator(DataServiceDecorator):
    def __init__(self, wrapped: DataService):
        super().__init__(wrapped)
        self._cache = {}

    def get_data(self, key: str) -> str:
        if key in self._cache:
            print(f"  [Cache] HIT: {key}")
            return self._cache[key]
        print(f"  [Cache] MISS: {key}")
        result = super().get_data(key)
        self._cache[key] = result
        return result


# Compose decorators
service = DatabaseService()
service = CachingDecorator(service)
service = LoggingDecorator(service)

service.get_data("user:1")
print()
service.get_data("user:1")  # cache hit
```

Output:

```
  [Log] get_data(user:1)
  [Cache] MISS: user:1
  [DB] Querying: user:1
  [Log] get_data returned: value_for_user:1

  [Log] get_data(user:1)
  [Cache] HIT: user:1
  [Log] get_data returned: value_for_user:1
```

Real-World Backend Use Cases

Use Case 1: HTTP Middleware Stack

Web frameworks like Express, Django, and Spring use the Decorator pattern for middleware:

```
Request ──> Auth ──> RateLimit ──> Logging ──> Compression ──> Handler
                                                                  │
Response <── Auth <── RateLimit <── Logging <── Compression <──────┘
```

Each middleware decorates the next one. It can modify the request before passing it along and modify the response on the way back.

Use Case 2: Response Compression

```java
public class CompressionDecorator extends DataServiceDecorator {

    public CompressionDecorator(DataService wrapped) {
        super(wrapped);
    }

    @Override
    public String getData(String key) {
        String rawData = super.getData(key);
        // In a real system, this would use GZIP or Brotli
        System.out.println("  [Compress] Compressing " + rawData.length() + " bytes");
        return compress(rawData);
    }

    private String compress(String data) {
        // Simulated compression
        return "[compressed:" + data + "]";
    }
}
```

Use Case 3: Metrics Collection

```python
import time


class MetricsDecorator(DataServiceDecorator):
    """Collects timing metrics for monitoring dashboards."""

    def __init__(self, wrapped, metrics_client):
        super().__init__(wrapped)
        self._metrics = metrics_client

    def get_data(self, key):
        start = time.time()
        try:
            result = super().get_data(key)
            self._metrics.record("get_data.success", time.time() - start)
            return result
        except Exception as e:
            self._metrics.record("get_data.error", time.time() - start)
            raise
```

Decorator vs Inheritance

Aspect	Decorator	Inheritance
Adding behavior	At runtime, flexible	At compile time, fixed
Combinations	Mix and match freely	One class per combination
Number of classes	One per behavior	Exponential (class explosion)
Complexity	Can be hard to debug deep chains	Simpler for 1-2 behaviors
Open/Closed Principle	Follows it perfectly	Often violates it
When to use	Cross-cutting concerns, optional behaviors	Is-a relationships, core type hierarchy

Rule of thumb: If you are adding behavior that is orthogonal to the core functionality (logging, caching, auth, metrics), use decorators. If you are defining a type hierarchy (Dog is an Animal), use inheritance.

When to Use / When NOT to Use

Use the Decorator Pattern When:

- You need to add responsibilities to objects dynamically
- You want to add cross-cutting concerns (logging, caching, auth, metrics)
- Extension by subclassing is impractical due to class explosion
- You want to combine behaviors flexibly at runtime
- You need to follow the Open/Closed Principle (open for extension, closed for modification)

Do NOT Use the Decorator Pattern When:

- You only need one fixed combination of behaviors -- just write that class
- The decorator chain would be deeply nested (more than 4-5 levels), making debugging hard
- The interface has many methods and most decorators only care about one -- consider aspect-oriented programming instead
- You need to access the specific type of the wrapped object -- decorators hide it

Common Mistakes

Mistake 1: Forgetting functools.wraps in Python

```python
# BAD: Loses the original function's name and docstring
def log(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log
def my_function():
    """Does something important."""
    pass
```

```
print(my_function.__name__)  # prints "wrapper" -- wrong!

# GOOD: Preserves metadata
def log(func):
    @functools.wraps(func)  # preserves __name__, __doc__, etc.
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper
```

Mistake 2: Order-Dependent Bugs

```java
// Order matters! These two produce different behavior:

// Option A: Cache first, then log
DataService serviceA = new LoggingDecorator(new CachingDecorator(new DatabaseService()));
// Logs every call, including cache hits

// Option B: Log first, then cache
DataService serviceB = new CachingDecorator(new LoggingDecorator(new DatabaseService()));
// Only logs cache misses (when the actual service is called)

// Think carefully about which behavior you want!
```

Mistake 3: Breaking the Interface Contract

```java
// BAD: Decorator changes the return type semantics
public class BadCachingDecorator extends DataServiceDecorator {
    @Override
    public String getData(String key) {
        String cached = cache.get(key);
        if (cached != null) return cached;
        return null; // BUG: should delegate to wrapped, not return null
    }
}
```

Always delegate to the wrapped object when the decorator cannot handle the call itself.

Best Practices

1. Keep decorators focused. Each decorator should add exactly one behavior. "LoggingAndCachingDecorator" is a sign that you should split it into two.
2. Order decorators intentionally. Think about what happens when each layer runs. Document the expected order if it matters.
3. Always use `functools.wraps` in Python function decorators to preserve metadata.
4. Make decorators transparent. A client should not be able to tell whether it is talking to a decorated or undecorated object.
5. Test decorators independently. Test each decorator with a mock wrapped object. Then test the composition.
6. Use a builder or factory to construct complex decorator chains, rather than nesting constructors manually.

Quick Summary

The Decorator pattern wraps an object with additional behavior without modifying its class. Like Russian nesting dolls, you can layer multiple decorators to combine behaviors. Each decorator implements the same interface as the wrapped object, so clients cannot tell the difference.

```
Client ——> Decorator A ——> Decorator B ——> Decorator C ——> Real Object
             (logging)       (caching)       (auth)       (core logic)
```

In Java, this appears as wrapper classes implementing a common interface. In Python, the `@decorator` syntax provides a concise way to wrap functions. Both approaches solve the same problem: adding behavior without modifying existing code.

Key Points

- The Decorator pattern adds behavior dynamically through composition, not inheritance
- It avoids the class explosion problem that arises from combining features via subclassing
- Each decorator implements the same interface as the object it wraps
- Decorators can be stacked in any order -- but order matters for behavior
- Java's InputStream hierarchy is a classic real-world example
- Python's `@decorator` syntax is syntactic sugar for the Decorator pattern

- Always use `functools.wraps` in Python decorators
- Keep decorators focused on one concern each

Practice Questions

1. You have a `MessageSender` interface with `send(to, message)`. You need logging, retry logic, and rate limiting. How many classes do you need with the Decorator pattern vs with inheritance for all combinations?
2. In the Java InputStream chain `new DataInputStream(new BufferedInputStream(new FileInputStream("f.txt")))`, which class is the "innermost doll" and which is the "outermost doll"? What happens when you call `readInt()` on the outer object?
3. Why does the order of decorators matter? Give an example where swapping two decorators produces different behavior.
4. How does the Decorator pattern relate to the Single Responsibility Principle?
5. A colleague writes a Python decorator that modifies a function's return value. When they try to access the function's docstring, they get `None`. What did they forget?

Exercises

Exercise 1: Build a Retry Decorator in Java

Create a `RetryDecorator` that wraps any `DataService` and retries failed calls up to N times with exponential backoff. The decorator should:

- Catch specified exception types
- Wait `delay * 2^attempt` between retries
- Log each retry attempt
- Re-throw the exception after all attempts are exhausted

Exercise 2: Python Rate-Limiting Decorator

Write a `@rate_limit(calls_per_second=5)` decorator that:

- Tracks how many times the function has been called in the current second
- Raises a `RateLimitExceeded` exception if the limit is exceeded

- Resets the counter each second
- Works with any function signature

Exercise 3: Compose a Full Middleware Stack

Using either Java or Python, build a complete HTTP middleware stack with these decorators:

1. `LoggingMiddleware` -- logs request/response
2. `AuthMiddleware` -- checks for an API key header
3. `RateLimitMiddleware` -- limits to 100 requests per minute
4. `CompressionMiddleware` -- compresses responses larger than 1KB

Wire them together and demonstrate a request flowing through all four layers.

What Is Next?

You now know how to add behavior to objects dynamically. But what if the problem is not adding behavior -- it is that the underlying system is too complex? What if a client needs to coordinate five different subsystems just to complete one operation?

In the next chapter, you will learn the Facade pattern, which provides a simple interface to a complex subsystem. Where the Decorator adds layers, the Facade hides them.

Chapter 9: The Facade Pattern

What You Will Learn

By the end of this chapter, you will be able to:

- Understand the Facade pattern and why it simplifies complex systems
- Recognize when a subsystem has too many moving parts for clients to manage
- Build an `OrderFacade` in Java that coordinates inventory, payment, shipping, and notification subsystems
- Create a `DataPipelineFacade` in Python that hides ETL complexity
- Distinguish between the Facade pattern and the Adapter pattern
- Apply the Facade pattern to SDK client design and complex business workflows
- Know when a facade helps and when it hides too much

Why This Chapter Matters

Backend systems grow. What starts as a simple "place an order" operation eventually involves checking inventory, validating the customer's payment, calculating shipping costs, reserving stock, charging the credit card, creating a shipment, sending a confirmation email, updating analytics, and notifying the warehouse. That is ten subsystems for one user action.

Without a facade, every controller, service, or API endpoint that needs to place an order must coordinate all ten subsystems in the correct sequence, handle errors at each step, and manage rollbacks if something fails halfway through.

Think of a hotel front desk. When you check in, you do not call housekeeping to prepare your room, contact the restaurant to reserve your dinner table, ask the valet to park your car, and notify the concierge about your preferences. You talk to one person at the front desk, and they coordinate everything behind the scenes.

```
   Without Facade                          With Facade
   (Guest does everything)                (Front desk handles it)

   ┌─────────┐                            ┌─────────┐
   │ Guest   │                            │ Guest   │
   └─────────┘                            └─────────┘
       │                                      │
       │                                      ▼
       ├──> Housekeeping                 ┌──────────────┐
       ├──> Restaurant                   │ Front Desk   │
       ├──> Valet                        │ (Facade)     │
       ├──> Concierge                    └──────────────┘
       ├──> Billing                            │
       └──> Security                           ├──> Housekeeping
                                               ├──> Restaurant
       6 interactions                          ├──> Valet
       Guest must know                         ├──> Concierge
       every department                        ├──> Billing
                                               └──> Security

                                         1 interaction
                                         Guest only knows
                                         the front desk
```

The Facade pattern provides a simplified interface to a complex subsystem. The subsystem still exists in all its complexity, but clients do not need to know about it.

The Problem

You are building an e-commerce backend. Placing an order involves four subsystems:

```java
// Subsystem 1: Inventory Management
public class InventoryService {
    public boolean checkStock(String productId, int quantity) { /* ... */ }
    public void reserveStock(String productId, int quantity) { /* ... */ }
    public void releaseStock(String productId, int quantity) { /* ... */ }
}

// Subsystem 2: Payment Processing
public class PaymentService {
    public boolean validatePaymentMethod(String customerId) { /* ... */ }
    public String chargeCustomer(String customerId, double amount) { /* ... */ }
    public void refundCharge(String chargeId) { /* ... */ }
}
```

```
// Subsystem 3: Shipping
public class ShippingService {
    public double calculateShippingCost(String address, double weight) { /* ... */ }
    public String createShipment(String orderId, String address) { /* ... */ }
    public String getTrackingNumber(String shipmentId) { /* ... */ }
}

// Subsystem 4: Notifications
public class NotificationService {
    public void sendOrderConfirmation(String email, String orderId) { /* ... */ }
    public void sendShippingNotification(String email, String trackingNumber) { /* ... ...
    public void sendFailureNotification(String email, String reason) { /* ... */ }
}
```

Without a facade, the controller looks like this:

```java
// BAD: Controller knows too much about every subsystem
@PostMapping("/orders")
public ResponseEntity<?> placeOrder(@RequestBody OrderRequest request) {
    // Step 1: Check inventory
    if (!inventoryService.checkStock(request.getProductId(), request.getQuantity())) {
        notificationService.sendFailureNotification(request.getEmail(), "Out of stock");
        return ResponseEntity.badRequest().body("Out of stock");
    }

    // Step 2: Reserve stock
    inventoryService.reserveStock(request.getProductId(), request.getQuantity());

    // Step 3: Validate payment
    if (!paymentService.validatePaymentMethod(request.getCustomerId())) {
        inventoryService.releaseStock(request.getProductId(), request.getQuantity());
        notificationService.sendFailureNotification(request.getEmail(), "Invalid paymen...
        return ResponseEntity.badRequest().body("Invalid payment method");
    }

    // Step 4: Calculate shipping
    double shippingCost = shippingService.calculateShippingCost(
        request.getAddress(), request.getWeight());
    double total = request.getAmount() + shippingCost;

    // Step 5: Charge customer
    String chargeId = paymentService.chargeCustomer(request.getCustomerId(), total);
    if (chargeId == null) {
        inventoryService.releaseStock(request.getProductId(), request.getQuantity());
        notificationService.sendFailureNotification(request.getEmail(), "Payment failed");
        return ResponseEntity.badRequest().body("Payment failed");
    }

    // Step 6: Create shipment
    String shipmentId = shippingService.createShipment(orderId, request.getAddress());
    String trackingNumber = shippingService.getTrackingNumber(shipmentId);
```

```
    // Step 7: Send notifications
    notificationService.sendOrderConfirmation(request.getEmail(), orderId);
    notificationService.sendShippingNotification(request.getEmail(), trackingNumber);

    return ResponseEntity.ok(new OrderResponse(orderId, trackingNumber));
}
```

This controller is doing too much. It knows the exact sequence of operations. It handles rollbacks. It coordinates four services. And if any other endpoint needs to place an order (an admin tool, a batch processor, a webhook handler), all this logic must be duplicated.

The Solution: Facade Pattern

Create a single class that exposes a simple interface and handles the coordination internally.

```
┌──────────────────────────────────────────────────────────────┐
│                     Facade Pattern Structure                 │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────┐        ┌──────────────────────────┐          │
│   │  Client  │──────>│      OrderFacade         │          │
│   │(Controller│        │                          │          │
│   │  or API) │        │  placeOrder(request)     │          │
│   └──────────┘        │  cancelOrder(orderId)    │          │
│                        │  getOrderStatus(orderId) │          │
│   Only calls           └──────────┬───────────────┘          │
│   the facade                      │                          │
│                          ┌────────┼────────┐                 │
│                          │        │        │                 │
│                     ┌────┴────┐ ┌─┴──────┐ ┌─┴──────┐       │
│                     │Inventory│ │Payment │ │Shipping│       │
│                     │Service  │ │Service │ │Service │       │
│                     └─────────┘ └────────┘ └────────┘       │
│                                              │               │
│                                          ┌───┴──────────┐    │
│                                          │Notification  │    │
│                                          │Service       │    │
│                                          └──────────────┘    │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Java Example: OrderFacade

The Subsystems

```java
public class InventoryService {

    public boolean checkStock(String productId, int quantity) {
        System.out.println("  [Inventory] Checking stock for " + productId
            + " (qty: " + quantity + ")");
        // Simulate: product "PROD-999" is out of stock
        return !productId.equals("PROD-999");
    }

    public void reserveStock(String productId, int quantity) {
        System.out.println("  [Inventory] Reserved " + quantity
            + " units of " + productId);
    }

    public void releaseStock(String productId, int quantity) {
        System.out.println("  [Inventory] Released " + quantity
            + " units of " + productId);
    }
}

public class PaymentService {

    public boolean validatePaymentMethod(String customerId) {
        System.out.println("  [Payment] Validating payment for customer: " + customerId);
        return true; // assume valid
    }

    public String chargeCustomer(String customerId, double amount) {
        System.out.println("  [Payment] Charging customer " + customerId
            + " amount: $" + String.format("%.2f", amount));
        return "CHG-" + System.currentTimeMillis();
    }

    public void refundCharge(String chargeId) {
        System.out.println("  [Payment] Refunding charge: " + chargeId);
    }
}

public class ShippingService {

    public double calculateShippingCost(String address, double weightKg) {
        System.out.println("  [Shipping] Calculating cost to: " + address);
        return weightKg * 2.50; // $2.50 per kg
    }

    public String createShipment(String orderId, String address) {
        System.out.println("  [Shipping] Creating shipment for order: " + orderId);
```

```
        return "SHIP-" + orderId;
    }

    public String getTrackingNumber(String shipmentId) {
        return "TRACK-" + shipmentId.hashCode();
    }
}

public class NotificationService {

    public void sendOrderConfirmation(String email, String orderId) {
        System.out.println("  [Notify] Order confirmation sent to " + email
            + " for order: " + orderId);
    }

    public void sendShippingNotification(String email, String trackingNumber) {
        System.out.println("  [Notify] Shipping notification sent to " + email
            + " tracking: " + trackingNumber);
    }

    public void sendFailureNotification(String email, String reason) {
        System.out.println("  [Notify] Failure notification sent to " + email
            + " reason: " + reason);
    }
}
```

The Order Request and Result

```java
public class OrderRequest {
    private final String customerId;
    private final String productId;
    private final int quantity;
    private final double unitPrice;
    private final String shippingAddress;
    private final double weightKg;
    private final String email;

    public OrderRequest(String customerId, String productId, int quantity,
                        double unitPrice, String shippingAddress,
                        double weightKg, String email) {
        this.customerId = customerId;
        this.productId = productId;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
        this.shippingAddress = shippingAddress;
        this.weightKg = weightKg;
        this.email = email;
    }

    // Getters omitted for brevity
```

```
    public String getCustomerId() { return customerId; }
    public String getProductId() { return productId; }
    public int getQuantity() { return quantity; }
    public double getUnitPrice() { return unitPrice; }
    public String getShippingAddress() { return shippingAddress; }
    public double getWeightKg() { return weightKg; }
    public String getEmail() { return email; }
}

public class OrderResult {
    private final boolean success;
    private final String orderId;
    private final String trackingNumber;
    private final String message;
    private final double totalCharged;

    public OrderResult(boolean success, String orderId, String trackingNumber,
                       String message, double totalCharged) {
        this.success = success;
        this.orderId = orderId;
        this.trackingNumber = trackingNumber;
        this.message = message;
        this.totalCharged = totalCharged;
    }

    public boolean isSuccess() { return success; }
    public String getOrderId() { return orderId; }

    @Override
    public String toString() {
        if (success) {
            return "OrderResult{SUCCESS, orderId='" + orderId
                + "', tracking='" + trackingNumber
                + "', total=$" + String.format("%.2f", totalCharged) + "}";
        }
        return "OrderResult{FAILED, message='" + message + "'}";
    }
}
```

The Facade

```java
public class OrderFacade {

    private final InventoryService inventory;
    private final PaymentService payment;
    private final ShippingService shipping;
    private final NotificationService notification;

    public OrderFacade(InventoryService inventory, PaymentService payment,
                       ShippingService shipping, NotificationService notification) {
```

```
        this.inventory = inventory;
        this.payment = payment;
        this.shipping = shipping;
        this.notification = notification;
    }

    /**
     * Places an order. This single method coordinates four subsystems:
     * inventory check, payment processing, shipment creation, and notifications.
     */
    public OrderResult placeOrder(OrderRequest request) {
        String orderId = "ORD-" + System.currentTimeMillis();
        System.out.println("=== Processing Order: " + orderId + " ===");

        // Step 1: Check and reserve inventory
        if (!inventory.checkStock(request.getProductId(), request.getQuantity())) {
            notification.sendFailureNotification(request.getEmail(), "Out of stock");
            return new OrderResult(false, null, null, "Product out of stock", 0);
        }
        inventory.reserveStock(request.getProductId(), request.getQuantity());

        // Step 2: Validate and process payment
        if (!payment.validatePaymentMethod(request.getCustomerId())) {
            inventory.releaseStock(request.getProductId(), request.getQuantity());
            notification.sendFailureNotification(request.getEmail(), "Invalid payment");
            return new OrderResult(false, null, null, "Invalid payment method", 0);
        }

        double subtotal = request.getUnitPrice() * request.getQuantity();
        double shippingCost = shipping.calculateShippingCost(
            request.getShippingAddress(), request.getWeightKg());
        double total = subtotal + shippingCost;

        String chargeId = payment.chargeCustomer(request.getCustomerId(), total);
        if (chargeId == null) {
            inventory.releaseStock(request.getProductId(), request.getQuantity());
            notification.sendFailureNotification(request.getEmail(), "Payment failed");
            return new OrderResult(false, null, null, "Payment processing failed", 0);
        }

        // Step 3: Create shipment
        String shipmentId = shipping.createShipment(orderId, request.getShippingAddress...
        String trackingNumber = shipping.getTrackingNumber(shipmentId);

        // Step 4: Send notifications
        notification.sendOrderConfirmation(request.getEmail(), orderId);
        notification.sendShippingNotification(request.getEmail(), trackingNumber);

        System.out.println("=== Order Complete ===\n");
        return new OrderResult(true, orderId, trackingNumber, "Order placed", total);
    }

    /**
     * Cancels an order. Another complex operation simplified to one call.
     */
```

```
    public boolean cancelOrder(String orderId, String customerId, String email) {
        System.out.println("=== Cancelling Order: " + orderId + " ===");
        // In a real system: look up order, refund payment, release stock, cancel shipment
        payment.refundCharge("CHG-" + orderId);
        notification.sendFailureNotification(email, "Order " + orderId + " cancelled");
        System.out.println("=== Cancellation Complete ===\n");
        return true;
    }
}
```

Using the Facade

```java
public class Main {
    public static void main(String[] args) {
        // Create subsystems
        InventoryService inventory = new InventoryService();
        PaymentService payment = new PaymentService();
        ShippingService shipping = new ShippingService();
        NotificationService notification = new NotificationService();

        // Create the facade -- this is the only thing the client knows about
        OrderFacade orderFacade = new OrderFacade(
            inventory, payment, shipping, notification
        );

        // Place an order -- one simple call
        OrderRequest request = new OrderRequest(
            "CUST-42",           // customerId
            "PROD-100",          // productId
            2,                   // quantity
            29.99,               // unitPrice
            "123 Main St, NYC",  // shippingAddress
            1.5,                 // weightKg
            "alice@example.com"  // email
        );

        OrderResult result = orderFacade.placeOrder(request);
        System.out.println("Result: " + result);
    }
}
```

Output:

```
=== Processing Order: ORD-1703001234567 ===
  [Inventory] Checking stock for PROD-100 (qty: 2)
  [Inventory] Reserved 2 units of PROD-100
  [Payment] Validating payment for customer: CUST-42
  [Shipping] Calculating cost to: 123 Main St, NYC
  [Payment] Charging customer CUST-42 amount: $63.73
```

```
  [Shipping] Creating shipment for order: ORD-1703001234567
  [Notify] Order confirmation sent to alice@example.com for order: ORD-1703001234567
  [Notify] Shipping notification sent to alice@example.com tracking: TRACK-987654321
=== Order Complete ===

Result: OrderResult{SUCCESS, orderId='ORD-1703001234567', tracking='TRACK-987654321', t...
```

Before vs After

Before (controller coordinates everything):

```java
@PostMapping("/orders")
public ResponseEntity<?> placeOrder(@RequestBody OrderRequest request) {
    // 40+ lines of coordination logic
    // Error handling at each step
    // Rollback logic scattered throughout
    // Duplicated in every endpoint that creates orders
}
```

After (controller uses facade):

```java
@PostMapping("/orders")
public ResponseEntity<?> placeOrder(@RequestBody OrderRequest request) {
    OrderResult result = orderFacade.placeOrder(request);
    if (result.isSuccess()) {
        return ResponseEntity.ok(result);
    }
    return ResponseEntity.badRequest().body(result);
}
```

The controller went from 40+ lines to 5 lines. The coordination logic lives in one place and is reusable.

Python Example: DataPipelineFacade

Data pipelines involve many steps: extracting data from sources, transforming it, validating it, loading it into a destination, and sending reports. A facade simplifies this.

The Subsystems

```python
import csv
import json
import io
from datetime import datetime


class DataExtractor:
    """Extracts data from various sources."""

    def extract_from_csv(self, file_path: str) -> list[dict]:
        print(f"  [Extractor] Reading CSV: {file_path}")
        # Simulated CSV data
        return [
            {"id": "1", "name": "Widget A", "price": "19.99", "quantity": "150"},
            {"id": "2", "name": "Widget B", "price": "29.99", "quantity": "75"},
            {"id": "3", "name": "Widget C", "price": "9.99", "quantity": "300"},
        ]

    def extract_from_api(self, endpoint: str) -> list[dict]:
        print(f"  [Extractor] Fetching API: {endpoint}")
        return [{"id": "4", "name": "Widget D", "price": "49.99", "quantity": "20"}]

    def extract_from_database(self, query: str) -> list[dict]:
        print(f"  [Extractor] Running query: {query}")
        return [{"id": "5", "name": "Widget E", "price": "14.99", "quantity": "200"}]


class DataTransformer:
    """Transforms and cleans data."""

    def normalize_fields(self, records: list[dict]) -> list[dict]:
        """Convert string numbers to proper types."""
        print(f"  [Transformer] Normalizing {len(records)} records")
        normalized = []
        for record in records:
            normalized.append({
                "id": int(record["id"]),
                "name": record["name"].strip().upper(),
                "price": float(record["price"]),
                "quantity": int(record["quantity"]),
            })
        return normalized

    def enrich_data(self, records: list[dict]) -> list[dict]:
        """Add computed fields."""
        print(f"  [Transformer] Enriching {len(records)} records")
        for record in records:
            record["total_value"] = record["price"] * record["quantity"]
            record["processed_at"] = datetime.now().isoformat()
        return records

    def filter_records(self, records: list[dict], min_quantity: int) -> list[dict]:
        """Remove records below a threshold."""
        filtered = [r for r in records if r["quantity"] >= min_quantity]
```

```
        print(f"  [Transformer] Filtered: {len(records)} -> {len(filtered)} records"
              f" (min_quantity={min_quantity})")
        return filtered


class DataValidator:
    """Validates data before loading."""

    def validate(self, records: list[dict]) -> tuple[list[dict], list[str]]:
        """Returns (valid_records, error_messages)."""
        print(f"  [Validator] Validating {len(records)} records")
        valid = []
        errors = []
        for record in records:
            if record["price"] <= 0:
                errors.append(f"Invalid price for {record['name']}: {record['price']}")
            elif record["quantity"] < 0:
                errors.append(f"Negative quantity for {record['name']}")
            else:
                valid.append(record)
        print(f"  [Validator] {len(valid)} valid, {len(errors)} errors")
        return valid, errors


class DataLoader:
    """Loads data into a destination."""

    def load_to_database(self, records: list[dict], table_name: str) -> int:
        print(f"  [Loader] Loading {len(records)} records into table: {table_name}")
        return len(records)

    def load_to_file(self, records: list[dict], file_path: str) -> int:
        print(f"  [Loader] Writing {len(records)} records to: {file_path}")
        return len(records)


class ReportGenerator:
    """Generates pipeline execution reports."""

    def generate_summary(self, stats: dict) -> str:
        print("  [Report] Generating pipeline summary")
        lines = ["Pipeline Execution Report", "=" * 40]
        for key, value in stats.items():
            lines.append(f"  {key}: {value}")
        report = "\n".join(lines)
        return report
```

The Facade

```python
class DataPipelineFacade:
    """Simple interface for running complex data pipelines."""

    def __init__(self):
        self._extractor = DataExtractor()
        self._transformer = DataTransformer()
        self._validator = DataValidator()
        self._loader = DataLoader()
        self._reporter = ReportGenerator()

    def run_csv_pipeline(
        self,
        csv_path: str,
        destination_table: str,
        min_quantity: int = 0,
    ) -> dict:
        """
        Complete ETL pipeline from CSV to database.

        One method call replaces what would otherwise be 6+ manual steps
        with error handling at each stage.
        """
        print(f"\n{'=' * 50}")
        print(f"Pipeline: CSV -> Database")
        print(f"{'=' * 50}")

        # Extract
        raw_data = self._extractor.extract_from_csv(csv_path)

        # Transform
        normalized = self._transformer.normalize_fields(raw_data)
        enriched = self._transformer.enrich_data(normalized)
        filtered = self._transformer.filter_records(enriched, min_quantity)

        # Validate
        valid_records, errors = self._validator.validate(filtered)

        # Load
        loaded_count = self._loader.load_to_database(valid_records, destination_table)

        # Report
        stats = {
            "source": csv_path,
            "destination": destination_table,
            "records_extracted": len(raw_data),
            "records_after_filter": len(filtered),
            "records_valid": len(valid_records),
            "records_loaded": loaded_count,
            "validation_errors": len(errors),
        }
        report = self._reporter.generate_summary(stats)
        print(f"\n{report}")

        return stats
```

```
    def run_multi_source_pipeline(
        self,
        csv_path: str,
        api_endpoint: str,
        db_query: str,
        output_file: str,
    ) -> dict:
        """
        Combines data from CSV, API, and database into a single output file.
        """
        print(f"\n{'=' * 50}")
        print(f"Pipeline: Multi-Source -> File")
        print(f"{'=' * 50}")

        # Extract from all sources
        csv_data = self._extractor.extract_from_csv(csv_path)
        api_data = self._extractor.extract_from_api(api_endpoint)
        db_data = self._extractor.extract_from_database(db_query)

        # Merge all sources
        all_data = csv_data + api_data + db_data
        print(f"  [Pipeline] Merged {len(all_data)} total records from 3 sources")

        # Transform
        normalized = self._transformer.normalize_fields(all_data)
        enriched = self._transformer.enrich_data(normalized)

        # Validate
        valid_records, errors = self._validator.validate(enriched)

        # Load
        loaded_count = self._loader.load_to_file(valid_records, output_file)

        stats = {
            "sources": 3,
            "total_extracted": len(all_data),
            "records_loaded": loaded_count,
            "output": output_file,
        }
        report = self._reporter.generate_summary(stats)
        print(f"\n{report}")

        return stats
```

Using the Facade

```python
def main():
    pipeline = DataPipelineFacade()
```

```
    # One call to run a complete ETL pipeline
    stats = pipeline.run_csv_pipeline(
        csv_path="sales_data.csv",
        destination_table="products",
        min_quantity=50,
    )
    print(f"\nLoaded {stats['records_loaded']} records successfully\n")

    # Another complex pipeline, still one call
    stats = pipeline.run_multi_source_pipeline(
        csv_path="inventory.csv",
        api_endpoint="https://api.supplier.com/products",
        db_query="SELECT * FROM legacy_products",
        output_file="merged_catalog.json",
    )


if __name__ == "__main__":
    main()
```

Output:

```
==================================================
Pipeline: CSV -> Database
==================================================
  [Extractor] Reading CSV: sales_data.csv
  [Transformer] Normalizing 3 records
  [Transformer] Enriching 3 records
  [Transformer] Filtered: 3 -> 3 records (min_quantity=50)
  [Validator] Validating 3 records
  [Validator] 3 valid, 0 errors
  [Loader] Loading 3 records into table: products
  [Report] Generating pipeline summary

Pipeline Execution Report
========================================
  source: sales_data.csv
  destination: products
  records_extracted: 3
  records_after_filter: 3
  records_valid: 3
  records_loaded: 3
  validation_errors: 0

Loaded 3 records successfully

==================================================
Pipeline: Multi-Source -> File
==================================================
  [Extractor] Reading CSV: inventory.csv
  [Extractor] Fetching API: https://api.supplier.com/products
  [Extractor] Running query: SELECT * FROM legacy_products
  [Pipeline] Merged 5 total records from 3 sources
```

```
  [Transformer] Normalizing 5 records
  [Transformer] Enriching 5 records
  [Validator] Validating 5 records
  [Validator] 5 valid, 0 errors
  [Loader] Writing 5 records to: merged_catalog.json
  [Report] Generating pipeline summary

Pipeline Execution Report
========================================
  sources: 3
  total_extracted: 5
  records_loaded: 5
  output: merged_catalog.json
```

Real-World Backend Use Cases

Use Case 1: SDK Client Libraries

Every cloud SDK is a facade. The AWS SDK for S3, for example, hides the complexity of HTTP requests, authentication, retries, region selection, and serialization behind simple methods:

```python
# Behind the scenes: HTTP client, auth signer, retry handler,
# region resolver, XML parser, multipart upload manager...
# The facade gives you this:
s3.upload_file("local.txt", "my-bucket", "remote.txt")
```

Use Case 2: Complex Business Workflows

```
┌──────────────────────────────────────────────────┐
│         UserRegistrationFacade                   │
│                                                  │
│  register(name, email, password)                 │
│    1. Validate email format                      │
│    2. Check if email already exists              │
│    3. Hash password                              │
│    4. Create user record in database             │
│    5. Create default preferences                 │
│    6. Assign default role                        │
│    7. Send welcome email                         │
│    8. Create audit log entry                     │
│    9. Sync with CRM system                       │
```

```
│   10. Update analytics                        │
│                                               │
│  Client just calls: facade.register(...)      │
└───────────────────────────────────────────────┘
```

Use Case 3: Testing Complex Systems

Facades make testing easier because you can mock one facade instead of five subsystems:

```java
// Without facade: must mock 4 services
@Test
void testOrderPlacement() {
    InventoryService mockInventory = mock(InventoryService.class);
    PaymentService mockPayment = mock(PaymentService.class);
    ShippingService mockShipping = mock(ShippingService.class);
    NotificationService mockNotification = mock(NotificationService.class);
    // Set up all mocks... tedious
}

// With facade: mock one thing
@Test
void testCheckout() {
    OrderFacade mockFacade = mock(OrderFacade.class);
    when(mockFacade.placeOrder(any())).thenReturn(successResult);
    // Much simpler
}
```

Facade vs Adapter

These two patterns are often confused. Here is how they differ:

Aspect	Facade	Adapter
Purpose	Simplify a complex subsystem	Make incompatible interfaces work together
Direction	Provides a new, simpler interface	Converts one existing interface to another
Subsystems	Works with multiple classes	Usually wraps one class
Client knowledge	Client does not know about subsystems	Client knows the target interface

Analogy	Hotel front desk	Power plug adapter

```
Facade:                                Adapter:
  Client ——> Facade ——> System A         Client ——> Adapter ——> Adaptee
                    ——> System B         (uses target       (has different
                    ——> System C          interface)          interface)
                    ——> System D

  Simplifies many into one               Translates one into another
```

When to Use / When NOT to Use

Use the Facade Pattern When:

- A subsystem has many classes and the client only needs a subset of functionality
- You want to layer your subsystem and provide an entry point to each layer
- You need to reduce coupling between clients and a complex subsystem
- You want to provide a simple default interface while still allowing advanced usage
- Multiple clients need to perform the same complex workflow

Do NOT Use the Facade Pattern When:

- The subsystem is already simple -- adding a facade adds unnecessary indirection
- Every client needs different customization -- the facade cannot serve them all
- You need the facade to enforce access control (use a different pattern for that)
- You are using it to hide poor design -- fix the subsystem instead

---

Common Mistakes

Mistake 1: God Facade

```java
// BAD: Facade does everything for the entire application
public class ApplicationFacade {
```

```
    public OrderResult placeOrder(...) { }
    public UserResult registerUser(...) { }
    public ReportResult generateReport(...) { }
    public InventoryResult updateInventory(...) { }
    public void sendNotification(...) { }
    public void processRefund(...) { }
    // 50 more methods...
}
```

A facade should cover one cohesive area, not the entire application. Create multiple facades for different domains: `OrderFacade`, `UserFacade`, `ReportFacade`.

Mistake 2: Facade Becomes the Only Access Point

```java
// BAD: Making subsystems package-private so only the facade can access them
// This turns the facade into a bottleneck

// GOOD: The facade provides a convenient interface, but advanced users
// can still access subsystems directly when they need fine-grained control
```

The facade should be an option, not a requirement. Clients that need more control should be able to bypass it.

Mistake 3: Business Logic in the Facade

```java
// BAD: Facade calculates discounts, validates data, makes decisions
public OrderResult placeOrder(OrderRequest request) {
    double discount = request.getQuantity() > 10 ? 0.15 : 0.05; // business rule!
    // ...
}

// GOOD: Facade coordinates, subsystems contain the logic
public OrderResult placeOrder(OrderRequest request) {
    double discount = pricingService.calculateDiscount(request); // delegated
    // ...
}
```

The facade orchestrates. It should not contain business logic -- that belongs in the subsystems.

Best Practices

1. Keep facades thin. A facade orchestrates calls to subsystems. It should not contain business logic, complex calculations, or data transformations.
2. Create domain-specific facades. One facade per bounded context or major feature area. `OrderFacade`, `UserFacade`, `ReportFacade` -- not one giant `ApplicationFacade`.
3. Do not force clients to use the facade. Make it convenient, not mandatory. Advanced clients might need direct subsystem access.
4. Handle errors in the facade. One of the facade's main jobs is coordinating error handling and rollbacks, so clients do not have to.
5. Use dependency injection. Pass subsystems into the facade's constructor. This makes testing easy and keeps the facade decoupled from specific implementations.
6. Document what the facade coordinates. A brief comment listing the subsystems and the sequence of operations helps the next developer.

Quick Summary

The Facade pattern provides a simplified interface to a complex subsystem. Like a hotel front desk that coordinates housekeeping, valet, and room service, a facade lets clients accomplish complex operations with a single method call while hiding the coordination logic.

```
Without Facade:                    With Facade:

Client ——> Service A               Client ——> Facade ——> Service A
       ——> Service B                                  ——> Service B
       ——> Service C                                  ——> Service C
       ——> Service D                                  ——> Service D

Client must coordinate             Facade coordinates
everything in the right order      everything internally
```

Key Points

- The Facade pattern simplifies a complex subsystem by providing a higher-level interface
- It does not add new functionality -- it makes existing functionality easier to use
- Facades reduce coupling between clients and subsystem classes
- Unlike the Adapter (which translates one interface to another), the Facade simplifies multiple interfaces into one

- Keep facades thin -- they orchestrate, they do not contain business logic
- Create one facade per domain, not one facade for everything
- Facades are optional conveniences, not gatekeepers

Practice Questions

1. You are building a user registration system that involves email validation, password hashing, database insertion, welcome email sending, and analytics tracking. Sketch the facade class. What methods would it expose?
2. A team has a `SystemFacade` class with 80 methods covering user management, order processing, reporting, and notifications. What is wrong with this design, and how would you fix it?
3. Your facade's `placeOrder()` method successfully charges the customer but fails when creating the shipment. How should the facade handle this situation? What about the notification to the customer?
4. When would you choose an Adapter over a Facade, and vice versa? Give a concrete example of each.
5. A junior developer suggests making all subsystem classes package-private so that only the facade can access them. What are the pros and cons of this approach?

Exercises

Exercise 1: Build a DeploymentFacade

Create a `DeploymentFacade` that coordinates these subsystems for deploying a web application:

- `BuildService`: compiles code and runs tests
- `ContainerService`: builds a Docker image and pushes it to a registry
- `KubernetesService`: updates the deployment with the new image
- `MonitoringService`: sets up health checks and alerts

The facade should expose a single `deploy(appName, version, environment)` method. Handle failures at each step with appropriate rollbacks.

Exercise 2: Python EmailCampaignFacade

Build a facade for sending marketing email campaigns:

- `TemplateEngine`: renders email templates with variables
- `RecipientService`: fetches recipient lists, handles unsubscribes
- `EmailSender`: sends individual emails with rate limiting
- `AnalyticsTracker`: tracks opens, clicks, and bounces

Expose `send_campaign(template_name, recipient_list_name, variables)` and `get_campaign_stats(campaign_id)`.

Exercise 3: Facade with Error Recovery

Extend the `OrderFacade` from this chapter to handle partial failures gracefully. If the payment succeeds but shipping fails, the facade should refund the payment and release the inventory. Write tests that verify the rollback behavior.

What Is Next?

You have learned how to simplify complex subsystems with a single interface. But what if you need to control access to an object rather than simplify it? What if you want to add lazy loading, access control, or rate limiting without the client knowing?

In the next chapter, you will learn the Proxy pattern, which controls access to an object by standing in front of it like a bodyguard. Where the Facade simplifies, the Proxy controls.

Chapter 10: The Proxy Pattern

What You Will Learn

By the end of this chapter, you will be able to:

- Understand the Proxy pattern and how it controls access to an object
- Implement four types of proxies: virtual (lazy loading), protection (access control), caching, and logging
- Recognize how JPA/Hibernate lazy loading uses proxies under the hood
- Build proxies in Python using property descriptors and custom classes
- Apply the Proxy pattern to real-world scenarios like API rate limiting and database connection pooling
- Know when a proxy adds value and when it adds unnecessary complexity

Why This Chapter Matters

Not every object should be accessed directly. Sometimes creating an object is expensive -- loading a 50MB image from disk or establishing a database connection takes time and memory, and you want to delay it until absolutely necessary. Sometimes access needs to be controlled -- only administrators should see certain data. Sometimes you want to track every access for auditing, or cache results to avoid redundant work.

The Proxy pattern handles all of these by placing a stand-in object in front of the real one. Think of a bodyguard. The bodyguard stands between the celebrity and the public. Every request to meet the celebrity goes through the bodyguard first. The bodyguard can:

- Check credentials before allowing access (protection proxy)
- Schedule the meeting for later when the celebrity is available (virtual proxy)

- Remember previous answers so the celebrity does not have to repeat them (caching proxy)
- Log every interaction for security records (logging proxy)

```
Without Proxy                        With Proxy

┌──────────┐     ┌──────────────┐    ┌──────────┐    ┌────────┐    ┌──────────────┐
│  Client  │────>│ Real Object│    │  Client  │──>│ Proxy │──>│ Real Object│
└──────────┘     └──────────────┘    └──────────┘    └────────┘    └──────────────┘
                                                            │
Direct access,                        Proxy controls:
no control                            - When to create
                                      - Who can access
                                      - What to cache
                                      - What to log
```

The key difference from other patterns: the proxy has the same interface as the real object. The client does not know it is talking to a proxy.

The Problem

You have a document management system. Loading documents from the database is expensive -- each document can be several megabytes. Your application loads lists of documents, but users usually only open one or two. Loading all documents eagerly wastes memory and time:

```java
// Without proxy: ALL documents loaded immediately
List<Document> docs = documentRepository.findAll(); // loads 1000 documents, 50MB each
// User only views one document...
// 50GB of memory wasted
```

You also need access control (only authorized users can view certain documents), caching (do not reload a document that was just viewed), and audit logging (track who accessed what).

Without the Proxy pattern, you would scatter all this logic across your codebase.

The Solution: Proxy Pattern

```
┌──────────────────────────────────────────────────────────────┐
│                    Proxy Pattern Structure                   │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ┌──────────────┐                                          │
│    │  <<interface>>│                                         │
│    │  Document     │                                         │
│    │  + getTitle() │                                         │
│    │  + getContent│                                          │
│    │  + getSize() │                                          │
│    └──────────────┘                                          │
│           │                                                  │
│     ┌─────┴──────────────────────────┐                       │
│     │                                │                       │
│   ┌─┴──────────────┐   ┌────────────┴──────────┐            │
│   │ RealDocument    │   │  DocumentProxy        │            │
│   │                 │   │                       │            │
│   │ - title         │   │ - realDocument        │────┐       │
│   │ - content       │   │ - documentId          │    │       │
│   │ - size          │   │ - loaded: boolean     │    │       │
│   │                 │   │                       │    │       │
│   │ + getTitle()    │   │ + getTitle()          │    │       │
│   │ + getContent()  │   │ + getContent()        │    │       │
│   │ + getSize()     │   │ + getSize()           │    │       │
│   └─────────────────┘   └───────────────────────┘    │       │
│         ^                                            │       │
│         └──────────────── creates lazily ────────────┘       │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Types of Proxies

1. Virtual Proxy (Lazy Loading)

Delays creation of an expensive object until it is actually needed.

```
┌──────────────────────────────────────────────────────┐
│  Virtual Proxy: Lazy Loading                         │
│                                                      │
│  Client calls getContent()                           │
│       │                                              │
│       ▼                                              │
│  ┌──────────────┐    Is real object    ┌──────────┐ │
│  │    Proxy     │──── loaded? ── NO ──>│ Create it │ │
│  │              │                      │ from DB  │ │
│  │              │──── loaded? ── YES   └──────────┘ │
```

```
│  │             │       │                   │       │
│  └─────────────┘       └───────────────────┘       │
│                              │                     │
│                              ▼                     │
│                     Delegate to real object         │
└─────────────────────────────────────────────────────┘
```

2. Protection Proxy (Access Control)

Checks whether the caller has permission to access the real object.

3. Caching Proxy

Stores results from the real object and returns cached values for repeated requests.

4. Logging Proxy

Records every access to the real object for auditing or debugging.

Java Example: Complete Proxy Implementation

Step 1: The Interface

```java
public interface DataService {
    String fetchData(String key);
    void updateData(String key, String value);
    List<String> listKeys();
}
```

Step 2: The Real Implementation

```java
public class DatabaseService implements DataService {

    private final Map<String, String> database = new HashMap<>();

    public DatabaseService() {
        // Simulate expensive initialization
```

```
        System.out.println("  [DB] Initializing database connection (slow)...");
        try { Thread.sleep(500); } catch (InterruptedException e) { }

        // Seed some data
        database.put("user:1", "Alice");
        database.put("user:2", "Bob");
        database.put("user:3", "Charlie");
        System.out.println("  [DB] Database ready with " + database.size() + " records");
    }

    @Override
    public String fetchData(String key) {
        System.out.println("  [DB] Fetching: " + key);
        try { Thread.sleep(100); } catch (InterruptedException e) { } // simulate latency
        return database.get(key);
    }

    @Override
    public void updateData(String key, String value) {
        System.out.println("  [DB] Updating: " + key + " = " + value);
        database.put(key, value);
    }

    @Override
    public List<String> listKeys() {
        System.out.println("  [DB] Listing all keys");
        return new ArrayList<>(database.keySet());
    }
}
```

Step 3: Virtual Proxy (Lazy Loading)

```java
public class LazyLoadingProxy implements DataService {

    private DataService realService; // null until first use
    private final Supplier<DataService> serviceFactory;

    public LazyLoadingProxy(Supplier<DataService> serviceFactory) {
        this.serviceFactory = serviceFactory;
        System.out.println("  [LazyProxy] Created (real service NOT initialized yet)");
    }

    private DataService getRealService() {
        if (realService == null) {
            System.out.println("  [LazyProxy] First access -- initializing real service...
            realService = serviceFactory.get();
        }
        return realService;
    }
```

```
    @Override
    public String fetchData(String key) {
        return getRealService().fetchData(key);
    }

    @Override
    public void updateData(String key, String value) {
        getRealService().updateData(key, value);
    }

    @Override
    public List<String> listKeys() {
        return getRealService().listKeys();
    }
}
```

Step 4: Protection Proxy (Access Control)

```java
public class ProtectionProxy implements DataService {

    private final DataService realService;
    private final String currentUserRole;

    // Define which roles can perform which operations
    private static final Set<String> READ_ROLES = Set.of("ADMIN", "USER", "VIEWER");
    private static final Set<String> WRITE_ROLES = Set.of("ADMIN", "USER");
    private static final Set<String> LIST_ROLES = Set.of("ADMIN");

    public ProtectionProxy(DataService realService, String currentUserRole) {
        this.realService = realService;
        this.currentUserRole = currentUserRole;
        System.out.println("  [AuthProxy] Created for role: " + currentUserRole);
    }

    @Override
    public String fetchData(String key) {
        checkAccess("fetchData", READ_ROLES);
        return realService.fetchData(key);
    }

    @Override
    public void updateData(String key, String value) {
        checkAccess("updateData", WRITE_ROLES);
        realService.updateData(key, value);
    }

    @Override
    public List<String> listKeys() {
        checkAccess("listKeys", LIST_ROLES);
        return realService.listKeys();
```

```
    }

    private void checkAccess(String operation, Set<String> allowedRoles) {
        if (!allowedRoles.contains(currentUserRole)) {
            System.out.println("  [AuthProxy] ACCESS DENIED: " + currentUserRole
                + " cannot perform " + operation);
            throw new SecurityException("Role '" + currentUserRole
                + "' is not authorized for operation: " + operation);
        }
        System.out.println("  [AuthProxy] Access granted for " + operation);
    }
}
```

Step 5: Caching Proxy

```java
public class CachingProxy implements DataService {

    private final DataService realService;
    private final Map<String, String> cache = new ConcurrentHashMap<>();
    private final Map<String, Long> cacheTimestamps = new ConcurrentHashMap<>();
    private final long ttlMillis;

    public CachingProxy(DataService realService, long ttlMillis) {
        this.realService = realService;
        this.ttlMillis = ttlMillis;
        System.out.println("  [CacheProxy] Created with TTL: " + ttlMillis + "ms");
    }

    @Override
    public String fetchData(String key) {
        // Check cache
        if (cache.containsKey(key)) {
            long age = System.currentTimeMillis() - cacheTimestamps.get(key);
            if (age < ttlMillis) {
                System.out.println("  [CacheProxy] HIT for " + key
                    + " (age: " + age + "ms)");
                return cache.get(key);
            }
            System.out.println("  [CacheProxy] EXPIRED for " + key);
            cache.remove(key);
            cacheTimestamps.remove(key);
        } else {
            System.out.println("  [CacheProxy] MISS for " + key);
        }

        // Fetch from real service and cache
        String value = realService.fetchData(key);
        if (value != null) {
            cache.put(key, value);
            cacheTimestamps.put(key, System.currentTimeMillis());
```

```
        }
        return value;
    }

    @Override
    public void updateData(String key, String value) {
        realService.updateData(key, value);
        // Invalidate cache for this key
        cache.remove(key);
        cacheTimestamps.remove(key);
        System.out.println("  [CacheProxy] Invalidated cache for " + key);
    }

    @Override
    public List<String> listKeys() {
        return realService.listKeys(); // lists are not cached
    }
}
```

Step 6: Logging Proxy

```java
public class LoggingProxy implements DataService {

    private final DataService realService;
    private final List<String> auditLog = new ArrayList<>();

    public LoggingProxy(DataService realService) {
        this.realService = realService;
    }

    @Override
    public String fetchData(String key) {
        String logEntry = timestamp() + " FETCH key=" + key;
        auditLog.add(logEntry);
        System.out.println("  [AuditProxy] " + logEntry);

        String result = realService.fetchData(key);

        String resultLog = timestamp() + " FETCH_RESULT key=" + key
            + " found=" + (result != null);
        auditLog.add(resultLog);
        return result;
    }

    @Override
    public void updateData(String key, String value) {
        String logEntry = timestamp() + " UPDATE key=" + key + " value=" + value;
        auditLog.add(logEntry);
        System.out.println("  [AuditProxy] " + logEntry);
        realService.updateData(key, value);
```

```
    }

    @Override
    public List<String> listKeys() {
        String logEntry = timestamp() + " LIST_KEYS";
        auditLog.add(logEntry);
        System.out.println("  [AuditProxy] " + logEntry);
        return realService.listKeys();
    }

    public List<String> getAuditLog() {
        return Collections.unmodifiableList(auditLog);
    }

    private String timestamp() {
        return java.time.LocalDateTime.now()
            .format(java.time.format.DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));
    }
}
```

Step 7: Composing Proxies

Proxies can be stacked just like decorators, because they share the same interface:

```java
public class Main {
    public static void main(String[] args) {
        // Layer 1: Lazy loading (delays DB initialization)
        DataService service = new LazyLoadingProxy(DatabaseService::new);

        // Layer 2: Caching (avoids redundant DB queries)
        service = new CachingProxy(service, 5000); // 5 second TTL

        // Layer 3: Access control
        service = new ProtectionProxy(service, "ADMIN");

        // Layer 4: Audit logging
        service = new LoggingProxy(service);

        // The chain: Logging -> Auth -> Caching -> LazyLoading -> Database
        System.out.println("=== Service created (DB not initialized yet) ===\n");

        System.out.println("=== First fetch (triggers lazy init + cache miss) ===");
        String result1 = service.fetchData("user:1");
        System.out.println("Result: " + result1 + "\n");

        System.out.println("=== Second fetch (cache hit, no DB access) ===");
        String result2 = service.fetchData("user:1");
        System.out.println("Result: " + result2 + "\n");

        System.out.println("=== Update (invalidates cache) ===");
```

```
        service.updateData("user:1", "Alice Updated");
        System.out.println();

        System.out.println("=== Fetch after update (cache miss again) ===");
        String result3 = service.fetchData("user:1");
        System.out.println("Result: " + result3);
    }
}
```

Output:

```
=== Service created (DB not initialized yet) ===

=== First fetch (triggers lazy init + cache miss) ===
  [AuditProxy] 10:30:01.234 FETCH key=user:1
  [AuthProxy] Access granted for fetchData
  [CacheProxy] MISS for user:1
  [LazyProxy] First access -- initializing real service now
  [DB] Initializing database connection (slow)...
  [DB] Database ready with 3 records
  [DB] Fetching: user:1
Result: Alice

=== Second fetch (cache hit, no DB access) ===
  [AuditProxy] 10:30:01.740 FETCH key=user:1
  [AuthProxy] Access granted for fetchData
  [CacheProxy] HIT for user:1 (age: 506ms)
Result: Alice

=== Update (invalidates cache) ===
  [AuditProxy] 10:30:01.741 UPDATE key=user:1 value=Alice Updated
  [AuthProxy] Access granted for updateData
  [DB] Updating: user:1 = Alice Updated
  [CacheProxy] Invalidated cache for user:1

=== Fetch after update (cache miss again) ===
  [AuditProxy] 10:30:01.742 FETCH key=user:1
  [AuthProxy] Access granted for fetchData
  [CacheProxy] MISS for user:1
  [DB] Fetching: user:1
Result: Alice Updated
```

JPA Lazy Loading Is a Proxy

Hibernate and JPA use the Proxy pattern extensively. When you define a `@ManyToOne(fetch = FetchType.LAZY)` relationship, JPA does not load the related entity immediately. Instead, it creates a proxy object:

```java
@Entity
public class Order {
    @Id
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    private Customer customer; // This is actually a PROXY, not a real Customer

    // When you call order.getCustomer().getName():
    // 1. getCustomer() returns the proxy object (no DB query)
    // 2. getName() triggers the proxy to load the real Customer from DB
    // 3. Subsequent calls to getName() return the cached value
}
```

```
┌─────────────────────────────────────────────────────┐
│  JPA Lazy Loading                                   │
│                                                     │
│  order.getCustomer()                                │
│       │                                             │
│       ▼                                             │
│  Returns CustomerProxy (not real Customer)          │
│                                                     │
│  order.getCustomer().getName()                      │
│       │                                             │
│       ▼                                             │
│  CustomerProxy.getName()                            │
│       │                                             │
│       ├── Is real Customer loaded? ── NO            │
│       │       │                                     │
│       │       ▼                                     │
│       │   SELECT * FROM customer WHERE id = ?       │
│       │       │                                     │
│       │       ▼                                     │
│       │   Store real Customer in proxy              │
│       │       │                                     │
│       ├── Is real Customer loaded? ── YES           │
│       │       │                                     │
│       ▼       ▼                                     │
│  Return realCustomer.getName()                      │
└─────────────────────────────────────────────────────┘
```

Before vs After

Before (without proxy) -- everything loaded eagerly:

```java
// Loading a user with their profile picture, orders, and reviews
User user = userRepository.findById(42); // Loads EVERYTHING
// user.profilePicture = 5MB blob (loaded even if not displayed)
```

```
// user.orders = 500 order objects (loaded even if not listed)
// user.reviews = 200 review objects (loaded even if not shown)
// Total: potentially 100MB of data for displaying a username
```

After (with proxy) -- loaded on demand:

```java
User user = userRepository.findById(42); // Loads just user fields
// user.profilePicture -> Proxy (loaded when user views their profile)
// user.orders -> Proxy (loaded when user clicks "My Orders")
// user.reviews -> Proxy (loaded when user visits reviews page)
// Total: a few KB until actually needed
```

Python Example: Proxy with Property Descriptors

Virtual Proxy with Lazy Properties

```python
class LazyProperty:
    """A descriptor that acts as a virtual proxy for expensive attributes."""

    def __init__(self, factory):
        self._factory = factory
        self._attr_name = None

    def __set_name__(self, owner, name):
        self._attr_name = f"_lazy_{name}"

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        if not hasattr(obj, self._attr_name):
            print(f"  [LazyProperty] Loading '{self._attr_name}' for first time")
            value = self._factory(obj)
            setattr(obj, self._attr_name, value)
        else:
            print(f"  [LazyProperty] Returning cached '{self._attr_name}'")
        return getattr(obj, self._attr_name)


class UserProfile:
    """User with lazily loaded expensive attributes."""

    def __init__(self, user_id: str, name: str):
        self.user_id = user_id
```

```
        self.name = name
        print(f"  [UserProfile] Created for {name} (no heavy data loaded)")

    @LazyProperty
    def order_history(self):
        """Expensive: loads all orders from database."""
        print(f"  [DB] Loading order history for {self.user_id}...")
        import time; time.sleep(0.3)
        return [
            {"order_id": "ORD-1", "total": 99.99},
            {"order_id": "ORD-2", "total": 149.50},
            {"order_id": "ORD-3", "total": 29.99},
        ]

    @LazyProperty
    def profile_image(self):
        """Expensive: loads large image from storage."""
        print(f"  [Storage] Loading profile image for {self.user_id}...")
        import time; time.sleep(0.5)
        return b"<fake image data -- imagine 5MB here>"


def main():
    print("=== Creating user (fast) ===")
    user = UserProfile("USR-42", "Alice")

    print("\n=== Accessing name (no proxy, instant) ===")
    print(f"Name: {user.name}")

    print("\n=== Accessing order_history (first time, slow) ===")
    orders = user.order_history
    print(f"Orders: {len(orders)} found")

    print("\n=== Accessing order_history (second time, cached) ===")
    orders = user.order_history
    print(f"Orders: {len(orders)} found")

    print("\n=== Profile image never accessed -- never loaded ===")
    print("Done!")


if __name__ == "__main__":
    main()
```

Output:

```
=== Creating user (fast) ===
  [UserProfile] Created for Alice (no heavy data loaded)

=== Accessing name (no proxy, instant) ===
Name: Alice

=== Accessing order_history (first time, slow) ===
```

```
  [LazyProperty] Loading '_lazy_order_history' for first time
  [DB] Loading order history for USR-42...
Orders: 3 found

=== Accessing order_history (second time, cached) ===
  [LazyProperty] Returning cached '_lazy_order_history'
Orders: 3 found

=== Profile image never accessed -- never loaded ===
Done!
```

Protection and Rate-Limiting Proxy

```python
import time
from collections import defaultdict


class APIService:
    """The real service that handles API requests."""

    def get_resource(self, resource_id: str) -> dict:
        print(f"  [API] Fetching resource: {resource_id}")
        return {"id": resource_id, "data": f"Content for {resource_id}"}

    def create_resource(self, data: dict) -> dict:
        print(f"  [API] Creating resource: {data}")
        return {"id": "new-123", **data}

    def delete_resource(self, resource_id: str) -> bool:
        print(f"  [API] Deleting resource: {resource_id}")
        return True


class RateLimitingProxy:
    """Proxy that enforces rate limits on the API service."""

    def __init__(self, real_service: APIService, max_calls_per_minute: int):
        self._service = real_service
        self._max_calls = max_calls_per_minute
        self._call_timestamps: list[float] = []

    def _check_rate_limit(self):
        """Remove old timestamps and check if limit is exceeded."""
        now = time.time()
        cutoff = now - 60  # one minute window
        self._call_timestamps = [t for t in self._call_timestamps if t > cutoff]

        if len(self._call_timestamps) >= self._max_calls:
            wait_time = 60 - (now - self._call_timestamps[0])
            raise RuntimeError(
```

```
                f"Rate limit exceeded ({self._max_calls}/min). "
                f"Try again in {wait_time:.1f} seconds."
            )
        self._call_timestamps.append(now)
        remaining = self._max_calls - len(self._call_timestamps)
        print(f"  [RateLimit] Request allowed. {remaining} calls remaining this minute.")

    def get_resource(self, resource_id: str) -> dict:
        self._check_rate_limit()
        return self._service.get_resource(resource_id)

    def create_resource(self, data: dict) -> dict:
        self._check_rate_limit()
        return self._service.create_resource(data)

    def delete_resource(self, resource_id: str) -> bool:
        self._check_rate_limit()
        return self._service.delete_resource(resource_id)


class ProtectionProxy:
    """Proxy that checks permissions before allowing operations."""

    PERMISSIONS = {
        "ADMIN": {"get_resource", "create_resource", "delete_resource"},
        "USER": {"get_resource", "create_resource"},
        "VIEWER": {"get_resource"},
    }

    def __init__(self, real_service, user_role: str):
        self._service = real_service
        self._role = user_role

    def _check_permission(self, operation: str):
        allowed = self.PERMISSIONS.get(self._role, set())
        if operation not in allowed:
            raise PermissionError(
                f"Role '{self._role}' cannot perform '{operation}'. "
                f"Allowed: {allowed}"
            )
        print(f"  [Auth] {self._role} authorized for {operation}")

    def get_resource(self, resource_id: str) -> dict:
        self._check_permission("get_resource")
        return self._service.get_resource(resource_id)

    def create_resource(self, data: dict) -> dict:
        self._check_permission("create_resource")
        return self._service.create_resource(data)

    def delete_resource(self, resource_id: str) -> bool:
        self._check_permission("delete_resource")
        return self._service.delete_resource(resource_id)
```

```
def main():
    # Build the proxy chain: Auth -> RateLimit -> Real Service
    real_service = APIService()
    rate_limited = RateLimitingProxy(real_service, max_calls_per_minute=5)
    protected = ProtectionProxy(rate_limited, user_role="USER")

    # Allowed: USER can get and create
    print("=== GET (allowed for USER) ===")
    result = protected.get_resource("item-42")
    print(f"Result: {result}\n")

    print("=== CREATE (allowed for USER) ===")
    result = protected.create_resource({"name": "New Item"})
    print(f"Result: {result}\n")

    # Denied: USER cannot delete
    print("=== DELETE (denied for USER) ===")
    try:
        protected.delete_resource("item-42")
    except PermissionError as e:
        print(f"Error: {e}\n")

    # Rate limiting demo
    print("=== Hitting rate limit ===")
    for i in range(5):
        try:
            protected.get_resource(f"item-{i}")
        except RuntimeError as e:
            print(f"Blocked: {e}")


if __name__ == "__main__":
    main()
```

Output:

```
=== GET (allowed for USER) ===
  [Auth] USER authorized for get_resource
  [RateLimit] Request allowed. 4 calls remaining this minute.
  [API] Fetching resource: item-42
Result: {'id': 'item-42', 'data': 'Content for item-42'}

=== CREATE (allowed for USER) ===
  [Auth] USER authorized for create_resource
  [RateLimit] Request allowed. 3 calls remaining this minute.
  [API] Creating resource: {'name': 'New Item'}
Result: {'id': 'new-123', 'name': 'New Item'}

=== DELETE (denied for USER) ===
Error: Role 'USER' cannot perform 'delete_resource'. Allowed: {'get_resource', 'create_...

=== Hitting rate limit ===
  [Auth] USER authorized for get_resource
```

```
  [RateLimit] Request allowed. 2 calls remaining this minute.
  [API] Fetching resource: item-0
  [Auth] USER authorized for get_resource
  [RateLimit] Request allowed. 1 calls remaining this minute.
  [API] Fetching resource: item-1
  [Auth] USER authorized for get_resource
  [RateLimit] Request allowed. 0 calls remaining this minute.
  [API] Fetching resource: item-2
  [Auth] USER authorized for get_resource
Blocked: Rate limit exceeded (5/min). Try again in 59.8 seconds.
```

Real-World Backend Use Cases

Use Case 1: Database Connection Pooling

Connection pools are essentially caching proxies. Instead of creating a new database connection for every query, the pool proxy returns an existing idle connection:

```
┌──────────────────────────────────────────────────────────┐
│  Connection Pool Proxy                                   │
│                                                          │
│  Client calls getConnection()                            │
│       │                                                  │
│       ▼                                                  │
│  ┌──────────────┐                                        │
│  │ Pool Proxy   │                                        │
│  │              │── Idle connection available? ──YES     │
│  │              │       │                      │         │
│  │              │       │              Return idle       │
│  │              │       │              connection        │
│  │              │       │                                │
│  │              │── NO ── Pool full? ── YES ── WAIT      │
│  │              │              │                         │
│  │              │              NO                        │
│  │              │              │                         │
│  │              │       Create new connection            │
│  └──────────────┘                                        │
└──────────────────────────────────────────────────────────┘
```

Use Case 2: API Rate Limiting

API gateways use protection/rate-limiting proxies to prevent abuse:

```java
public class RateLimitingProxy implements ApiEndpoint {
    private final ApiEndpoint realEndpoint;
    private final RateLimiter rateLimiter; // e.g., Guava RateLimiter

    @Override
    public Response handle(Request request) {
        String clientId = request.getHeader("X-API-Key");
        if (!rateLimiter.tryAcquire(clientId)) {
            return Response.status(429)
                .body("Rate limit exceeded. Try again later.")
                .build();
        }
        return realEndpoint.handle(request);
    }
}
```

Use Case 3: Remote Proxy

A remote proxy represents an object that lives on a different server. The client calls methods on the proxy as if it were local, but the proxy serializes the call and sends it over the network:

```
┌──────────┐    ┌──────────────┐     Network     ┌──────────────┐    ┌──────────┐
│  Client  │──>│ Remote Proxy│────────────────>│   Stub       │──>│  Real    │
│          │    │ (local)      │    HTTP/gRPC    │  (remote)    │    │ Object   │
│          │    │              │                 │              │    │ (remote)│
└──────────┘    └──────────────┘                 └──────────────┘    └──────────┘
```

Java RMI, gRPC stubs, and REST client libraries all follow this pattern.

When to Use / When NOT to Use

Use the Proxy Pattern When:

- You need lazy initialization of heavyweight objects (virtual proxy)
- You need access control based on caller identity or role (protection proxy)
- You want to cache results of expensive operations (caching proxy)
- You need audit logging of object access (logging proxy)
- You want to enforce rate limits on API calls
- You need to manage database connection pools
- You want to add remote communication transparency

Do NOT Use the Proxy Pattern When:

- Direct access is sufficient and the added indirection is not needed
- The proxy would add latency without providing value
- The object is cheap to create and does not need lazy loading
- Access control is better handled at a higher level (e.g., framework middleware)
- Caching is better handled by a dedicated cache layer (e.g., Redis)

Common Mistakes

Mistake 1: Proxy That Does Not Match the Real Object's Interface

```java
// BAD: Proxy adds methods that the real object does not have
public class BadProxy implements DataService {
    // DataService methods...

    public void clearCache() { } // Not part of DataService interface!
    public int getCacheSize() { } // Not part of DataService interface!
}

// GOOD: Proxy matches the interface exactly
// Put cache management in a separate interface or accessor
```

Mistake 2: Forgetting to Delegate

```java
// BAD: Proxy forgets to delegate updateData, breaks functionality
public class IncompleteProxy implements DataService {
    @Override
    public String fetchData(String key) {
        return realService.fetchData(key); // correct
    }

    @Override
    public void updateData(String key, String value) {
        // Forgot to delegate! Updates silently disappear
        System.out.println("Update received");
    }
}
```

Mistake 3: Thread-Unsafe Lazy Loading

```java
// BAD: Two threads can create the real object simultaneously
private DataService getRealService() {
    if (realService == null) { // Thread A checks, sees null
        // Thread B also checks, also sees null
        realService = new ExpensiveService(); // Both threads create it
    }
    return realService;
}

// GOOD: Thread-safe lazy initialization
private volatile DataService realService;

private synchronized DataService getRealService() {
    if (realService == null) {
        realService = new ExpensiveService();
    }
    return realService;
}
```

Best Practices

1. Match the interface exactly. A proxy must implement the same interface as the real object. Clients should not be able to tell the difference.
2. Use the right type of proxy. Do not combine lazy loading, caching, and auth into one proxy class. Separate concerns into separate proxy layers.
3. Be thread-safe. If the proxy is shared across threads, ensure lazy loading, caching, and rate limiting are synchronized correctly.
4. Document proxy behavior. Make it clear to other developers that the object they are using might be a proxy. Especially for lazy-loading proxies where accessing a field can trigger a database query.
5. Consider using framework proxies. Spring AOP, Java dynamic proxies, and Python descriptors provide proxy infrastructure. You do not always need to write proxies from scratch.
6. Cache invalidation is hard. If you build a caching proxy, think carefully about when cached values become stale and how they are evicted.

Quick Summary

The Proxy pattern places a stand-in object in front of the real object to control access. Like a bodyguard who screens visitors, a proxy can delay creation (virtual), check permissions (protection), store results (caching), or record access (logging). The proxy has the same interface as the real object, so clients use it transparently.

```
┌──────────┐       ┌──────────────────────────┐       ┌──────────────┐
│  Client  │──────>│         Proxy            │──────>│  Real Object │
│          │       │                          │       │              │
│ Calls the│       │  - Lazy load             │       │  Does the    │
│ interface│       │  - Check access          │       │  actual work │
│          │       │  - Cache results         │       │              │
│          │       │  - Log access            │       │              │
└──────────┘       └──────────────────────────┘       └──────────────┘
                    Same interface as real object
```

Key Points

- A proxy controls access to another object while maintaining the same interface
- Four main types: virtual (lazy), protection (auth), caching, and logging
- JPA/Hibernate lazy loading is the most common real-world proxy example in Java
- Proxies can be stacked like decorators since they share the same interface
- The client should not know (or care) whether it is talking to a proxy or the real object
- Thread safety is critical for lazy-loading and caching proxies
- Python descriptors and dynamic proxies reduce boilerplate

Practice Questions

1. You have a `ReportGenerator` that takes 30 seconds to produce a report. Most users request the same report multiple times. Which type of proxy would you use, and how would you handle cache invalidation when the underlying data changes?
2. How does the Proxy pattern differ from the Decorator pattern? They both wrap objects and forward calls -- what is the fundamental difference in intent?

3. Your JPA entity has a `@OneToMany(fetch = FetchType.LAZY)` relationship. You access the collection outside a transaction and get a `LazyInitializationException`. What happened, and how does this relate to the Proxy pattern?
4. A rate-limiting proxy keeps track of requests per minute. How would you handle this in a distributed system where multiple server instances share the rate limit?
5. You need to log every database query in production for compliance. Would you use a logging proxy or an aspect-oriented approach? What are the trade-offs?

Exercises

Exercise 1: Build a Connection Pool Proxy

Create a `ConnectionPoolProxy` that manages a fixed pool of database connections:

- Initialize with a maximum pool size
- `getConnection()` returns an available connection or blocks until one is free
- `releaseConnection()` returns a connection to the pool
- Track how many active and idle connections exist

Exercise 2: Python Smart Reference Proxy

Build a proxy class that wraps any object and:

- Counts how many times each method is called
- Tracks the last access time for each method
- Provides a `get_usage_stats()` method that returns all collected statistics
- Does not require the wrapped object's class to be modified

Exercise 3: Tiered Caching Proxy

Create a two-tier caching proxy:

- Level 1 (L1): In-memory cache with a 10-second TTL and max 100 entries
- Level 2 (L2): Simulated Redis cache with a 60-second TTL
- On cache miss: check L1, then L2, then the real service
- On cache hit in L2: populate L1 for faster subsequent access
- Implement cache eviction using LRU (Least Recently Used) for L1

What Is Next?

You now know how to control access to objects using proxies. But what happens when you have two independent dimensions of variation that multiply together? What if you have notifications (Email, SMS, Push) and urgency levels (Normal, Urgent) and you do not want to create a class for every combination?

In the next chapter, you will learn the Bridge pattern, which separates an abstraction from its implementation so they can vary independently. Where the Proxy controls access, the Bridge controls complexity.

Chapter 11: The Bridge Pattern

What You Will Learn

By the end of this chapter, you will be able to:

- Understand the Bridge pattern and the class explosion problem it solves
- Recognize when two independent dimensions of variation create a multiplication of classes
- Separate abstraction from implementation so they can vary independently
- Build a Notification system in Java with `Notification` (Email/SMS) x `Urgency` (Normal/Urgent)
- Create a DataExporter system in Python with `Format` (CSV/JSON) x `Destination` (File/S3/FTP)
- Know when the Bridge pattern helps most and when it is overkill
- Distinguish Bridge from Adapter, Strategy, and other structural patterns

Why This Chapter Matters

Imagine you are building a notification system for a backend application. You need to support different channels (Email, SMS, Push Notification) and different urgency levels (Normal, Urgent). The naive approach using inheritance looks like this:

```
                    Notification
                         │
          ┌──────────────┼──────────────┐
          │              │              │
    EmailNotification  SMSNotification  PushNotification
          │              │              │
    ┌─────┴─────┐    ┌─────┴─────┐    ┌─────┴─────┐
```

```
    |         |     |         |     |         |
  Normal   Urgent  Normal  Urgent  Normal  Urgent
  Email    Email   SMS     SMS     Push    Push
```

That is 6 classes for 2 dimensions (3 channels x 2 urgency levels). Now add a third urgency level (Critical). You need 9 classes. Add a fourth channel (Slack). You need 12 classes. Add a priority dimension (High/Medium/Low). You need 36 classes.

This is the class explosion problem: when two or more independent dimensions of variation are combined through inheritance, the number of classes grows multiplicatively.

The Bridge pattern solves this by separating the two dimensions into independent hierarchies connected by composition:

```
  Abstraction                    Implementation
  (Urgency)                      (Channel)

  ┌──────────────┐               ┌──────────────┐
  │ Notification │──────has-a──>│ MessageSender │
  └──────┬───────┘               └──────┬───────┘
         │                              │
    ┌────┴────┐                   ┌────┴────┐
    │         │                   │         │
  Normal   Urgent             Email   SMS    Push

  2 classes + 3 classes = 5 total (instead of 6)
  Add Critical: 3 + 3 = 6 (instead of 9)
  Add Slack: 3 + 4 = 7 (instead of 12)
```

Instead of multiplication, you get addition.

The Problem

You are building a notification service for an e-commerce platform. Notifications need to be sent through different channels, and the formatting and behavior changes based on urgency. Without the Bridge pattern:

```java
// Class explosion: one class per combination
public class NormalEmailNotification extends Notification { }
public class UrgentEmailNotification extends Notification { }
public class NormalSMSNotification extends Notification { }
```

```
public class UrgentSMSNotification extends Notification { }
public class NormalPushNotification extends Notification { }
public class UrgentPushNotification extends Notification { }
// Adding Slack channel: 2 more classes
// Adding Critical urgency: 3 more classes (now 4 channels)
// Adding scheduled delivery: doubles everything
```

Each class is nearly identical to its siblings, with only small differences. This violates the DRY principle and makes maintenance painful.

The Solution: Bridge Pattern

Separate the two dimensions into independent hierarchies and connect them with a bridge (composition).

```
┌──────────────────────────────────────────────────────────────────┐
│                      Bridge Pattern Structure                    │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    Abstraction                          Implementation           │
│    (What to do)                         (How to do it)           │
│                                                                  │
│   ┌──────────────────┐     bridge      ┌──────────────────┐      │
│   │   Notification   │──────────────>│  MessageSender   │      │
│   │                  │    (has-a)      │  <<interface>>   │      │
│   │ + send(msg)      │                 │                  │      │
│   │ + formatMessage()│                 │ + sendMessage()  │      │
│   └──────────────────┘                 └──────────────────┘      │
│            │                                    │                 │
│      ┌─────┴──────┐                  ┌──────────┴──────────┐     │
│      │            │                  │          │          │     │
│  ┌───┴────┐   ┌───┴────┐        ┌────┴────┐ ┌───┴────┐ ┌──┴──┐  │
│  │ Normal │   │ Urgent │        │ Email   │ │ SMS    │ │Push │  │
│  │Notif.  │   │ Notif. │        │ Sender  │ │Sender  │ │Send.│  │
│  └────────┘   └────────┘        └─────────┘ └────────┘ └─────┘  │
│                                                                  │
│   2 abstractions  +  3 implementations  =  5 classes             │
│   (instead of 2 x 3 = 6 with inheritance)                        │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Java Example: Notification System

Step 1: The Implementation Interface (Message Channel)

```java
/**
 * The "implementation" side of the bridge.
 * Defines HOW a message is actually delivered.
 */
public interface MessageSender {
    void sendMessage(String recipient, String subject, String body);
    String getChannelName();
}
```

Step 2: Concrete Implementations (Channels)

```java
public class EmailSender implements MessageSender {

    @Override
    public void sendMessage(String recipient, String subject, String body) {
        System.out.println("  [Email] To: " + recipient);
        System.out.println("  [Email] Subject: " + subject);
        System.out.println("  [Email] Body: " + body);
        System.out.println("  [Email] Sent via SMTP server\n");
    }

    @Override
    public String getChannelName() {
        return "EMAIL";
    }
}

public class SMSSender implements MessageSender {

    private static final int SMS_MAX_LENGTH = 160;

    @Override
    public void sendMessage(String recipient, String subject, String body) {
        // SMS has no subject; combine into one message
        String smsText = subject + ": " + body;
        if (smsText.length() > SMS_MAX_LENGTH) {
            smsText = smsText.substring(0, SMS_MAX_LENGTH - 3) + "...";
        }
        System.out.println("  [SMS] To: " + recipient);
        System.out.println("  [SMS] Message: " + smsText);
        System.out.println("  [SMS] Sent via SMS gateway\n");
    }
```

```
    @Override
    public String getChannelName() {
        return "SMS";
    }
}

public class PushNotificationSender implements MessageSender {

    @Override
    public void sendMessage(String recipient, String subject, String body) {
        System.out.println("  [Push] Device: " + recipient);
        System.out.println("  [Push] Title: " + subject);
        System.out.println("  [Push] Body: " + body.substring(0, Math.min(body.length()...
        System.out.println("  [Push] Sent via Firebase Cloud Messaging\n");
    }

    @Override
    public String getChannelName() {
        return "PUSH";
    }
}
```

Step 3: The Abstraction (Notification with Urgency)

```java
/**
 * The "abstraction" side of the bridge.
 * Defines WHAT the notification does (formatting, behavior).
 * Delegates the HOW to the MessageSender.
 */
public abstract class Notification {

    protected final MessageSender sender; // <-- the bridge

    public Notification(MessageSender sender) {
        this.sender = sender;
    }

    /**
     * Send a notification. Subclasses define the formatting
     * and behavior; the sender defines the delivery channel.
     */
    public abstract void send(String recipient, String message);

    /**
     * Each urgency level formats the message differently.
     */
    protected abstract String formatSubject(String message);
    protected abstract String formatBody(String message);
}
```

Step 4: Refined Abstractions (Urgency Levels)

```java
public class NormalNotification extends Notification {

    public NormalNotification(MessageSender sender) {
        super(sender);
    }

    @Override
    public void send(String recipient, String message) {
        System.out.println("--- Normal Notification via " + sender.getChannelName() + "...
        String subject = formatSubject(message);
        String body = formatBody(message);
        sender.sendMessage(recipient, subject, body);
    }

    @Override
    protected String formatSubject(String message) {
        return "Notification: " + message;
    }

    @Override
    protected String formatBody(String message) {
        return message + "\n\nThis is a standard notification. No action required.";
    }
}

public class UrgentNotification extends Notification {

    public UrgentNotification(MessageSender sender) {
        super(sender);
    }

    @Override
    public void send(String recipient, String message) {
        System.out.println("--- URGENT Notification via " + sender.getChannelName() + "...
        String subject = formatSubject(message);
        String body = formatBody(message);

        // Urgent: send immediately, no batching
        sender.sendMessage(recipient, subject, body);

        // Also log urgent notifications for monitoring
        System.out.println("  [Monitor] URGENT notification logged for recipient: "
            + recipient + "\n");
    }

    @Override
    protected String formatSubject(String message) {
        return "[URGENT] " + message;
    }

    @Override
```

```
    protected String formatBody(String message) {
        return "*** URGENT ACTION REQUIRED ***\n\n"
            + message
            + "\n\nPlease respond immediately.";
    }
}

public class CriticalNotification extends Notification {

    public CriticalNotification(MessageSender sender) {
        super(sender);
    }

    @Override
    public void send(String recipient, String message) {
        System.out.println("--- CRITICAL Notification via " + sender.getChannelName() +...
        String subject = formatSubject(message);
        String body = formatBody(message);

        // Critical: send multiple times to ensure delivery
        sender.sendMessage(recipient, subject, body);

        // Escalate: also notify the on-call team
        System.out.println("  [Escalation] Paging on-call team for: " + message);
        System.out.println("  [Escalation] Creating incident ticket\n");
    }

    @Override
    protected String formatSubject(String message) {
        return "[CRITICAL - IMMEDIATE ACTION] " + message;
    }

    @Override
    protected String formatBody(String message) {
        return "***** CRITICAL ALERT *****\n\n"
            + message
            + "\n\nThis requires immediate attention. "
            + "On-call team has been notified.";
    }
}
```

Step 5: Using the Bridge

```java
public class Main {
    public static void main(String[] args) {
        // Create senders (implementation dimension)
        MessageSender emailSender = new EmailSender();
        MessageSender smsSender = new SMSSender();
        MessageSender pushSender = new PushNotificationSender();
```

```
        // Combine any urgency with any channel
        Notification normalEmail = new NormalNotification(emailSender);
        normalEmail.send("alice@example.com", "Your order has shipped");

        Notification urgentSms = new UrgentNotification(smsSender);
        urgentSms.send("+1-555-0123", "Payment declined on order #4521");

        Notification criticalPush = new CriticalNotification(pushSender);
        criticalPush.send("device-token-abc", "Server cluster is down");

        // Easy to add new combinations without new classes
        Notification urgentEmail = new UrgentNotification(emailSender);
        urgentEmail.send("admin@example.com", "Database failover activated");
    }
}
```

Output:

```
--- Normal Notification via EMAIL ---
  [Email] To: alice@example.com
  [Email] Subject: Notification: Your order has shipped
  [Email] Body: Your order has shipped

This is a standard notification. No action required.
  [Email] Sent via SMTP server

--- URGENT Notification via SMS ---
  [SMS] To: +1-555-0123
  [SMS] Message: [URGENT] Payment declined on order #4521: *** URGENT ACTION REQUI...
  [SMS] Sent via SMS gateway

  [Monitor] URGENT notification logged for recipient: +1-555-0123

--- CRITICAL Notification via PUSH ---
  [Push] Device: device-token-abc
  [Push] Title: [CRITICAL - IMMEDIATE ACTION] Server cluster is down
  [Push] Body: ***** CRITICAL ALERT *****

Server cluster is down

This requires immediate attention
  [Push] Sent via Firebase Cloud Messaging

  [Escalation] Paging on-call team for: Server cluster is down
  [Escalation] Creating incident ticket

--- URGENT Notification via EMAIL ---
  [Email] To: admin@example.com
  [Email] Subject: [URGENT] Database failover activated
  [Email] Body: *** URGENT ACTION REQUIRED ***

Database failover activated
```

```
Please respond immediately.
  [Email] Sent via SMTP server

  [Monitor] URGENT notification logged for recipient: admin@example.com
```

Before vs After

Before (without Bridge) -- class explosion:

```
NormalEmailNotification
NormalSMSNotification
NormalPushNotification
UrgentEmailNotification
UrgentSMSNotification
UrgentPushNotification
CriticalEmailNotification
CriticalSMSNotification
CriticalPushNotification

Total: 9 classes (3 urgency x 3 channels)
Adding Slack: 12 classes
Adding WhatsApp: 15 classes
```

After (with Bridge) -- linear growth:

```
Abstractions: NormalNotification, UrgentNotification, CriticalNotification (3)
Implementations: EmailSender, SMSSender, PushSender (3)

Total: 6 classes (3 + 3)
Adding Slack: 7 classes (3 + 4)
Adding WhatsApp: 8 classes (3 + 5)
```

```
  Classes needed (Urgency levels x Channels)

  Without Bridge (multiplication):
  Channels:    3     4     5     6     7
  Urgency=3:   9    12    15    18    21
  Urgency=4:  12    16    20    24    28
  Urgency=5:  15    20    25    30    35

  With Bridge (addition):
  Channels:    3     4     5     6     7
  Urgency=3:   6     7     8     9    10
  Urgency=4:   7     8     9    10    11
  Urgency=5:   8     9    10    11    12
```

Python Example: DataExporter System

Two dimensions: Format (CSV, JSON, XML) and Destination (File, S3, FTP).

The Implementation Side (Destination)

```python
from abc import ABC, abstractmethod
import json
import os


class ExportDestination(ABC):
    """The implementation side: WHERE to export data."""

    @abstractmethod
    def write(self, data: str, filename: str) -> None:
        pass

    @abstractmethod
    def get_destination_name(self) -> str:
        pass


class FileDestination(ExportDestination):
    """Writes to the local filesystem."""

    def __init__(self, base_dir: str = "/tmp/exports"):
        self._base_dir = base_dir

    def write(self, data: str, filename: str) -> None:
        full_path = os.path.join(self._base_dir, filename)
        print(f"  [File] Writing {len(data)} bytes to: {full_path}")
        # In production: os.makedirs(self._base_dir, exist_ok=True)
        # with open(full_path, 'w') as f: f.write(data)

    def get_destination_name(self) -> str:
        return f"File({self._base_dir})"


class S3Destination(ExportDestination):
    """Uploads to AWS S3."""

    def __init__(self, bucket: str, prefix: str = "exports/"):
        self._bucket = bucket
        self._prefix = prefix

    def write(self, data: str, filename: str) -> None:
        s3_key = self._prefix + filename
```

```
        print(f"  [S3] Uploading {len(data)} bytes to s3://{self._bucket}/{s3_key}")
        # In production: boto3.client('s3').put_object(...)

    def get_destination_name(self) -> str:
        return f"S3({self._bucket})"


class FTPDestination(ExportDestination):
    """Uploads to an FTP server."""

    def __init__(self, host: str, port: int = 21, remote_dir: str = "/uploads"):
        self._host = host
        self._port = port
        self._remote_dir = remote_dir

    def write(self, data: str, filename: str) -> None:
        remote_path = f"{self._remote_dir}/{filename}"
        print(f"  [FTP] Uploading {len(data)} bytes to "
              f"ftp://{self._host}:{self._port}{remote_path}")
        # In production: ftplib.FTP(self._host).storbinary(...)

    def get_destination_name(self) -> str:
        return f"FTP({self._host})"
```

The Abstraction Side (Format)

```python
class DataExporter(ABC):
    """
    The abstraction side: HOW to format data.
    Uses a destination (bridge) to determine WHERE to write.
    """

    def __init__(self, destination: ExportDestination):
        self._destination = destination  # <-- the bridge

    @abstractmethod
    def export(self, records: list[dict], filename: str) -> None:
        pass

    @abstractmethod
    def get_format_name(self) -> str:
        pass

    def _write_output(self, formatted_data: str, filename: str) -> None:
        """Delegates writing to the destination (bridge)."""
        dest_name = self._destination.get_destination_name()
        fmt_name = self.get_format_name()
        print(f"Exporting {fmt_name} -> {dest_name}: {filename}")
        self._destination.write(formatted_data, filename)
```

```python
class CSVExporter(DataExporter):
    """Exports data as CSV."""

    def __init__(self, destination: ExportDestination, delimiter: str = ","):
        super().__init__(destination)
        self._delimiter = delimiter

    def export(self, records: list[dict], filename: str) -> None:
        if not records:
            print("  [CSV] No records to export")
            return

        # Build CSV string
        headers = list(records[0].keys())
        lines = [self._delimiter.join(headers)]
        for record in records:
            values = [str(record.get(h, "")) for h in headers]
            lines.append(self._delimiter.join(values))

        csv_data = "\n".join(lines)
        print(f"  [CSV] Formatted {len(records)} records ({len(lines)} lines)")
        self._write_output(csv_data, filename)

    def get_format_name(self) -> str:
        return "CSV"


class JSONExporter(DataExporter):
    """Exports data as JSON."""

    def __init__(self, destination: ExportDestination, pretty: bool = True):
        super().__init__(destination)
        self._pretty = pretty

    def export(self, records: list[dict], filename: str) -> None:
        indent = 2 if self._pretty else None
        json_data = json.dumps(records, indent=indent, default=str)
        print(f"  [JSON] Formatted {len(records)} records "
              f"({'pretty' if self._pretty else 'compact'})")
        self._write_output(json_data, filename)

    def get_format_name(self) -> str:
        return "JSON"


class XMLExporter(DataExporter):
    """Exports data as XML."""

    def export(self, records: list[dict], filename: str) -> None:
        lines = ['<?xml version="1.0" encoding="UTF-8"?>', "<records>"]
        for record in records:
            lines.append("  <record>")
            for key, value in record.items():
                lines.append(f"    <{key}>{value}</{key}>")
```

```
            lines.append("  </record>")
        lines.append("</records>")

        xml_data = "\n".join(lines)
        print(f"  [XML] Formatted {len(records)} records")
        self._write_output(xml_data, filename)

    def get_format_name(self) -> str:
        return "XML"
```

Using the Bridge

```python
def main():
    # Sample data
    records = [
        {"id": 1, "name": "Widget A", "price": 19.99, "stock": 150},
        {"id": 2, "name": "Widget B", "price": 29.99, "stock": 75},
        {"id": 3, "name": "Widget C", "price": 9.99, "stock": 300},
    ]

    # Create destinations
    local_file = FileDestination("/tmp/reports")
    s3_bucket = S3Destination("my-data-bucket", "reports/2024/")
    ftp_server = FTPDestination("ftp.partner.com")

    # Mix and match any format with any destination
    print("=== CSV to Local File ===")
    csv_to_file = CSVExporter(local_file)
    csv_to_file.export(records, "products.csv")

    print("\n=== JSON to S3 ===")
    json_to_s3 = JSONExporter(s3_bucket)
    json_to_s3.export(records, "products.json")

    print("\n=== XML to FTP ===")
    xml_to_ftp = XMLExporter(ftp_server)
    xml_to_ftp.export(records, "products.xml")

    print("\n=== JSON to FTP (new combination, no new class) ===")
    json_to_ftp = JSONExporter(ftp_server, pretty=False)
    json_to_ftp.export(records, "products_compact.json")

    # Adding a new destination (e.g., Azure Blob Storage) requires
    # only ONE new class, not 3 (one per format)

    # Adding a new format (e.g., Parquet) requires
    # only ONE new class, not 3 (one per destination)


if __name__ == "__main__":
    main()
```

Output:

```
=== CSV to Local File ===
  [CSV] Formatted 3 records (4 lines)
Exporting CSV -> File(/tmp/reports): products.csv
  [File] Writing 97 bytes to: /tmp/reports/products.csv

=== JSON to S3 ===
  [JSON] Formatted 3 records (pretty)
Exporting JSON -> S3(my-data-bucket): products.json
  [S3] Uploading 234 bytes to s3://my-data-bucket/reports/2024/products.json

=== XML to FTP ===
  [XML] Formatted 3 records
Exporting XML -> FTP(ftp.partner.com): products.xml
  [FTP] Uploading 312 bytes to ftp://ftp.partner.com:21/uploads/products.xml

=== JSON to FTP (new combination, no new class) ===
  [JSON] Formatted 3 records (compact)
Exporting JSON -> FTP(ftp.partner.com): products_compact.json
  [FTP] Uploading 178 bytes to ftp://ftp.partner.com:21/uploads/products_compact.json
```

When the Bridge Helps Most

The Bridge pattern provides the most value when:

1. Two dimensions of variation exist. If you only have one dimension (just formats, or just destinations), use simple polymorphism.
2. The dimensions change independently. Adding a new format should not require touching destination code, and vice versa.
3. The number of combinations is large. With 2 formats and 2 destinations, you have 4 combinations -- the Bridge saves little. With 5 formats and 5 destinations, you have 25 combinations vs 10 classes.
4. You want to swap implementations at runtime. The bridge (composition) lets you change the destination of an exporter without creating a new exporter.

```python
# Runtime swapping: same exporter, different destinations
exporter = JSONExporter(local_file)
exporter.export(records, "local_copy.json")

# Later, switch to S3 without creating a new class
exporter._destination = s3_bucket
exporter.export(records, "cloud_copy.json")
```

When to Use / When NOT to Use

Use the Bridge Pattern When:

- You have two or more independent dimensions of variation
- You want to avoid class explosion from combining dimensions via inheritance
- You need to switch implementations at runtime
- Both dimensions are likely to grow over time
- You want to change abstraction and implementation independently

Do NOT Use the Bridge Pattern When:

- You have only one dimension of variation -- use simple polymorphism
- The dimensions are not truly independent (one depends on the other)
- The total number of combinations is small (fewer than 6)
- Adding the pattern would introduce unnecessary abstraction for a simple problem
- The implementation side has only one class and will not grow

Common Mistakes

Mistake 1: Using Bridge When Strategy Suffices

```java
// If you only have ONE dimension of variation, you do not need Bridge.
// This is just Strategy:
public class ReportGenerator {
    private ReportFormat format; // Strategy, not Bridge
    public void generate(Data data) { format.render(data); }
}

// Bridge is for TWO dimensions:
public class ReportGenerator { // abstraction dimension: summary vs detailed
    private ReportFormat format; // implementation dimension: PDF vs HTML vs CSV
}
```

Mistake 2: Tight Coupling Between Abstraction and Implementation

```java
// BAD: The abstraction knows specific implementation details
public class UrgentNotification extends Notification {
    @Override
    public void send(String recipient, String message) {
        if (sender instanceof EmailSender) { // breaks the bridge!
            ((EmailSender) sender).setHighPriority(true);
        }
        sender.sendMessage(recipient, formatSubject(message), formatBody(message));
    }
}

// GOOD: The abstraction only uses the interface methods
public class UrgentNotification extends Notification {
    @Override
    public void send(String recipient, String message) {
        // No instanceof checks. Works with ANY sender.
        sender.sendMessage(recipient, formatSubject(message), formatBody(message));
    }
}
```

Mistake 3: Making the Implementation Too Thin

```java
// BAD: Implementation interface is too generic
public interface MessageSender {
    void send(Object data); // Too vague, abstraction must know details
}

// GOOD: Implementation interface is specific enough to be useful
public interface MessageSender {
    void sendMessage(String recipient, String subject, String body);
    String getChannelName();
}
```

Best Practices

1. Identify the two dimensions first. Before writing code, clearly name the two independent dimensions of variation. If you cannot find two, you probably do not need Bridge.
2. Keep the bridge interface stable. The implementation interface is the contract between the two dimensions. Changes to it ripple through both sides.

3. Do not leak implementation details into the abstraction. The abstraction should work with any implementation without type checks or casts.
4. Use with dependency injection. The implementation side is a natural fit for DI frameworks, making it easy to configure which implementation each abstraction uses.
5. Start without Bridge. If you start with only 2-3 combinations, simple classes are fine. Refactor to Bridge when you see class explosion happening.

Quick Summary

The Bridge pattern separates an abstraction from its implementation so the two can vary independently. Instead of creating a class for every combination of two dimensions (NxM classes), you create N + M classes connected by composition. This is especially valuable when both dimensions will grow over time.

```
Without Bridge:          With Bridge:

Abstraction              Abstraction ────bridge────> Implementation
     │                        │                           │
┌────┼────┐               ┌────┼────┐              ┌────┼────┐
│    │    │               │         │              │    │    │
A1   A2   A3          Normal   Urgent         Email   SMS   Push
│    │    │
├B1  ├B1  ├B1           2 classes + 3 classes = 5 total
├B2  ├B2  ├B2
└B3  └B3  └B3           Instead of 2 x 3 = 6

9 classes!
```

Key Points

- The Bridge pattern decouples abstraction from implementation, letting them vary independently
- It solves the class explosion problem caused by combining two dimensions via inheritance
- The "bridge" is a composition relationship: the abstraction has-a reference to the implementation
- Growth is additive (N + M) instead of multiplicative (N x M)

- Bridge is most valuable when both dimensions are expected to grow
- Do not confuse Bridge with Strategy (Strategy has one dimension, Bridge has two)
- Do not confuse Bridge with Adapter (Adapter makes things work together after design; Bridge plans for separation from the start)

Practice Questions

1. You are building a reporting system with report types (Summary, Detailed, Audit) and output formats (PDF, HTML, Excel). How many classes do you need with Bridge vs without? What are the two sides of the bridge?
2. How does the Bridge pattern differ from the Strategy pattern? Give a scenario where each would be appropriate.
3. Your team argues that the Bridge pattern is just "using interfaces." What would you say to explain why it is more specific than that?
4. You have a logging system with log levels (DEBUG, INFO, WARN, ERROR) and log destinations (Console, File, Remote). Should you use Bridge? Why or why not?
5. A colleague uses `instanceof` checks inside an abstraction class to handle specific implementation types. Why does this defeat the purpose of the Bridge pattern?

Exercises

Exercise 1: Shape Rendering Bridge

Create a Bridge pattern for drawing shapes:

- Abstraction dimension: Shape types (Circle, Rectangle, Triangle)
- Implementation dimension: Rendering engines (SVGRenderer, CanvasRenderer, ASCIIRenderer)

Each shape should delegate its rendering to the engine. Adding a new shape or renderer should require only one new class.

Exercise 2: Python Payment Processing Bridge

Build a payment system with two dimensions:

- Payment type (OneTimePayment, RecurringPayment, InstallmentPayment) -- each formats the request differently
- Payment gateway (StripeGateway, PayPalGateway, SquareGateway) -- each uses a different API

Demonstrate creating all 9 combinations using only 6 classes (3 + 3).

Exercise 3: Configurable Notification System

Extend the Java Notification example from this chapter:

- Add a `ScheduledNotification` abstraction that delays sending
- Add a `SlackSender` implementation
- Add a `WebhookSender` implementation
- Show how adding these 3 classes creates 5 new combinations (3 urgency levels x 2 new channels - but 5 new combinations, not 6 new classes)

What Is Next?

You now know how to handle two dimensions of variation without class explosion. But what about tree structures? What if your objects form hierarchies -- files inside folders, menu items inside submenus, employees inside departments -- and you want to treat individual objects and groups of objects uniformly?

In the next chapter, you will learn the Composite pattern, which lets you build tree structures where leaves and branches share the same interface. Where Bridge separates two flat dimensions, Composite handles recursive, nested structures.

Chapter 12: Composite Pattern -- Tree Structures with a Uniform Interface

What You Will Learn

- What the Composite pattern is and when to use it
- How to model tree structures where leaves and branches share the same interface
- The file system analogy that makes Composite click
- How to implement Composite in Java with menus and submenus
- How to implement Composite in Python with departments and employees
- The difference between Leaf and Composite nodes
- How to traverse, search, and aggregate across tree structures

Why This Chapter Matters

Backend systems are full of hierarchical data: organization charts, menu systems, file directories, permission trees, category taxonomies, and UI component trees. Without a clean pattern, you end up writing different code for "things that contain other things" versus "things that do not." That means if-else chains everywhere, constantly checking types before performing operations.

The Composite pattern eliminates this problem. It lets you treat individual objects and groups of objects through the same interface. Whether you are calculating the total cost of a single item or an entire shopping cart with nested bundles, the code looks the same.

The Problem

You are building an admin dashboard with a nested menu system. Some items are simple links (like "Dashboard" or "Settings"). Others are submenus that contain more items, and those items might contain even more submenus.

Without Composite, your rendering code looks like this:

```java
for (Object item : menuItems) {
    if (item instanceof MenuItem) {
        render((MenuItem) item);
    } else if (item instanceof SubMenu) {
        SubMenu sub = (SubMenu) item;
        renderSubMenuHeader(sub);
        for (Object child : sub.getChildren()) {
            if (child instanceof MenuItem) {
                render((MenuItem) child);
            } else if (child instanceof SubMenu) {
                // nested again... how deep does this go?
                SubMenu sub2 = (SubMenu) child;
                renderSubMenuHeader(sub2);
                // ... more nesting ...
            }
        }
    }
}
```

This code is fragile. Every new level of nesting requires more if-else blocks. Adding a new type of menu element means changing every place that handles menus.

The Solution: Composite Pattern

The Composite pattern defines a common interface for both individual items (leaves) and containers (composites). The container holds children that share the same interface, and operations cascade down the tree automatically.

```
+-------------------------+
|    <<interface>>        |
|    MenuComponent        |
+-------------------------+
| + render(indent)        |
| + getTitle(): String    |
| + search(query): List   |
```

```
+-------------------------+
        ^           ^
        |           |
+-------------+  +----------------+
|  MenuItem   |  |    SubMenu     |
|   (Leaf)    |  |  (Composite)   |
+-------------+  +----------------+
| - title     |  | - title        |
| - url       |  | - children[]   |
+-------------+  +----------------+
| + render()  |  | + render()     |
| + search()  |  | + add(comp)    |
+-------------+  | + remove(comp) |
                 | + search()     |
                 +----------------+
                       |
                 children can be
                 MenuItem OR SubMenu
```

Key insight: SubMenu's `children` list holds `MenuComponent` objects. Each child could be a simple MenuItem or another SubMenu. The SubMenu does not need to know which type it contains. It just calls `render()` on each child and lets polymorphism handle the rest.

The File System Analogy

The most intuitive example of Composite is a file system.

```
root/
+-- documents/
|   +-- resume.pdf        (Leaf)
|   +-- cover-letter.docx (Leaf)
|   +-- projects/          (Composite)
|       +-- project-a.md   (Leaf)
|       +-- project-b.md   (Leaf)
+-- photos/
|   +-- vacation.jpg       (Leaf)
+-- notes.txt              (Leaf)
```

When you ask for the total size of `root/`, it adds up:

- Size of `documents/` (which adds up its children, including `projects/`)
- Size of `photos/`
- Size of `notes.txt`

The operation is the same at every level: if you are a file, return your size. If you are a directory, sum the sizes of your children. That is Composite in action.

Java Implementation: Menu System

The Component Interface

```java
import java.util.List;

public interface MenuComponent {
    String getTitle();
    void render(int indent);
    List<MenuComponent> search(String query);
    int countItems();
}
```

The Leaf: MenuItem

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class MenuItem implements MenuComponent {
    private final String title;
    private final String url;
    private final String icon;

    public MenuItem(String title, String url, String icon) {
        this.title = title;
        this.url = url;
        this.icon = icon;
    }

    @Override
    public String getTitle() {
        return title;
    }

    @Override
    public void render(int indent) {
        String padding = " ".repeat(indent);
        System.out.printf("%s[%s] %s -> %s%n", padding, icon, title, url);
```

```
    }

    @Override
    public List<MenuComponent> search(String query) {
        if (title.toLowerCase().contains(query.toLowerCase())) {
            return List.of(this);
        }
        return Collections.emptyList();
    }

    @Override
    public int countItems() {
        return 1;
    }

    @Override
    public String toString() {
        return title;
    }
}
```

The Composite: SubMenu

```java
import java.util.ArrayList;
import java.util.List;

public class SubMenu implements MenuComponent {
    private final String title;
    private final String icon;
    private final List<MenuComponent> children;

    public SubMenu(String title, String icon) {
        this.title = title;
        this.icon = icon;
        this.children = new ArrayList<>();
    }

    public void add(MenuComponent component) {
        children.add(component);
    }

    public void remove(MenuComponent component) {
        children.remove(component);
    }

    @Override
    public String getTitle() {
        return title;
    }
```

```
    @Override
    public void render(int indent) {
        String padding = " ".repeat(indent);
        System.out.printf("%s[%s] %s%n", padding, icon, title);
        for (MenuComponent child : children) {
            child.render(indent + 4);
        }
    }

    @Override
    public List<MenuComponent> search(String query) {
        List<MenuComponent> results = new ArrayList<>();

        // Check if this submenu matches
        if (title.toLowerCase().contains(query.toLowerCase())) {
            results.add(this);
        }

        // Search children recursively
        for (MenuComponent child : children) {
            results.addAll(child.search(query));
        }

        return results;
    }

    @Override
    public int countItems() {
        int count = 0;
        for (MenuComponent child : children) {
            count += child.countItems();
        }
        return count;
    }

    @Override
    public String toString() {
        return title + " (" + children.size() + " children)";
    }
}
```

Building and Using the Menu

```java
public class MenuDemo {
    public static void main(String[] args) {
        // Build the menu tree
        SubMenu root = new SubMenu("Main Menu", "#");

        // Top-level items
        root.add(new MenuItem("Dashboard", "/dashboard", "D"));
```

```
        root.add(new MenuItem("Profile", "/profile", "P"));

        // Products submenu
        SubMenu products = new SubMenu("Products", ">");
        products.add(new MenuItem("All Products", "/products", "-"));
        products.add(new MenuItem("Add Product", "/products/new", "+"));

        // Nested: Product Categories
        SubMenu categories = new SubMenu("Categories", ">");
        categories.add(new MenuItem("Electronics", "/categories/electronics", "-"));
        categories.add(new MenuItem("Books", "/categories/books", "-"));
        categories.add(new MenuItem("Clothing", "/categories/clothing", "-"));
        products.add(categories);

        root.add(products);

        // Settings submenu
        SubMenu settings = new SubMenu("Settings", ">");
        settings.add(new MenuItem("General", "/settings/general", "-"));
        settings.add(new MenuItem("Security", "/settings/security", "-"));
        settings.add(new MenuItem("Notifications", "/settings/notif", "-"));
        root.add(settings);

        // Render entire menu tree
        System.out.println("=== Full Menu ===");
        root.render(0);

        // Count all items
        System.out.println("\nTotal menu items: " + root.countItems());

        // Search
        System.out.println("\n=== Search: 'product' ===");
        List<MenuComponent> results = root.search("product");
        for (MenuComponent result : results) {
            System.out.println("  Found: " + result);
        }
    }
}
```

Output:

```
=== Full Menu ===
[#] Main Menu
    [D] Dashboard -> /dashboard
    [P] Profile -> /profile
    [>] Products
        [-] All Products -> /products
        [+] Add Product -> /products/new
        [>] Categories
            [-] Electronics -> /categories/electronics
            [-] Books -> /categories/books
            [-] Clothing -> /categories/clothing
    [>] Settings
```

```
        [-] General -> /settings/general
        [-] Security -> /settings/security
        [-] Notifications -> /settings/notif

Total menu items: 10

=== Search: 'product' ===
  Found: Products (3 children)
  Found: All Products
  Found: Add Product
```

Notice how `render()`, `search()`, and `countItems()` work identically whether called on a leaf or a composite. The client code does not need to know the difference.

Python Implementation: Organization Tree

Department and Employee Hierarchy

```python
from abc import ABC, abstractmethod


class OrganizationComponent(ABC):
    """Common interface for both employees and departments."""

    @abstractmethod
    def get_name(self):
        pass

    @abstractmethod
    def get_salary_cost(self):
        pass

    @abstractmethod
    def display(self, indent=0):
        pass

    @abstractmethod
    def count_employees(self):
        pass

    @abstractmethod
    def search(self, query):
        pass
```

```
class Employee(OrganizationComponent):
    """Leaf node: an individual employee."""

    def __init__(self, name, role, salary):
        self.name = name
        self.role = role
        self.salary = salary

    def get_name(self):
        return self.name

    def get_salary_cost(self):
        return self.salary

    def display(self, indent=0):
        padding = "  " * indent
        print(f"{padding}- {self.name} ({self.role}) ${self.salary:,.0f}")

    def count_employees(self):
        return 1

    def search(self, query):
        query_lower = query.lower()
        if (query_lower in self.name.lower() or
                query_lower in self.role.lower()):
            return [self]
        return []

    def __repr__(self):
        return f"Employee({self.name})"


class Department(OrganizationComponent):
    """Composite node: a department containing employees or sub-departments."""

    def __init__(self, name):
        self.name = name
        self._children = []

    def add(self, component):
        self._children.append(component)
        return self  # enable chaining

    def remove(self, component):
        self._children.remove(component)

    def get_name(self):
        return self.name

    def get_salary_cost(self):
        return sum(child.get_salary_cost() for child in self._children)

    def display(self, indent=0):
        padding = "  " * indent
        cost = self.get_salary_cost()
```

```
        count = self.count_employees()
        print(f"{padding}[{self.name}] ({count} employees, "
              f"${cost:,.0f} total)")
        for child in self._children:
            child.display(indent + 1)

    def count_employees(self):
        return sum(child.count_employees() for child in self._children)

    def search(self, query):
        results = []
        if query.lower() in self.name.lower():
            results.append(self)
        for child in self._children:
            results.extend(child.search(query))
        return results

    def __repr__(self):
        return f"Department({self.name}, {len(self._children)} members)"
```

Building the Organization

```python
# Build organization tree
company = Department("Acme Corp")

# Engineering department with sub-departments
engineering = Department("Engineering")

backend = Department("Backend Team")
backend.add(Employee("Alice", "Senior Engineer", 150000))
backend.add(Employee("Bob", "Engineer", 120000))
backend.add(Employee("Charlie", "Junior Engineer", 90000))

frontend = Department("Frontend Team")
frontend.add(Employee("Diana", "Senior Engineer", 145000))
frontend.add(Employee("Eve", "Engineer", 115000))

engineering.add(backend)
engineering.add(frontend)
engineering.add(Employee("Frank", "Engineering Manager", 180000))

# Sales department
sales = Department("Sales")
sales.add(Employee("Grace", "Sales Director", 160000))
sales.add(Employee("Hank", "Account Executive", 110000))
sales.add(Employee("Ivy", "Account Executive", 105000))

company.add(engineering)
company.add(sales)
company.add(Employee("Jack", "CEO", 250000))
```

```
# Display the entire organization
print("=== Organization Chart ===")
company.display()

print(f"\nTotal employees: {company.count_employees()}")
print(f"Total salary cost: ${company.get_salary_cost():,.0f}")

# Display just engineering
print("\n=== Engineering Only ===")
engineering.display()
print(f"Engineering cost: ${engineering.get_salary_cost():,.0f}")

# Search
print("\n=== Search: 'engineer' ===")
results = company.search("engineer")
for r in results:
    print(f"  Found: {r}")
```

Output:

```
=== Organization Chart ===
[Acme Corp] (10 employees, $1,425,000 total)
  [Engineering] (6 employees, $800,000 total)
    [Backend Team] (3 employees, $360,000 total)
      - Alice (Senior Engineer) $150,000
      - Bob (Engineer) $120,000
      - Charlie (Junior Engineer) $90,000
    [Frontend Team] (2 employees, $260,000 total)
      - Diana (Senior Engineer) $145,000
      - Eve (Engineer) $115,000
    - Frank (Engineering Manager) $180,000
  [Sales] (3 employees, $375,000 total)
    - Grace (Sales Director) $160,000
    - Hank (Account Executive) $110,000
    - Ivy (Account Executive) $105,000
  - Jack (CEO) $250,000

Total employees: 10
Total salary cost: $1,425,000

=== Engineering Only ===
[Engineering] (6 employees, $800,000 total)
  [Backend Team] (3 employees, $360,000 total)
    - Alice (Senior Engineer) $150,000
    - Bob (Engineer) $120,000
    - Charlie (Junior Engineer) $90,000
  [Frontend Team] (2 employees, $260,000 total)
    - Diana (Senior Engineer) $145,000
    - Eve (Engineer) $115,000
  - Frank (Engineering Manager) $180,000
Engineering cost: $800,000
```

The same `display()`, `get_salary_cost()`, and `count_employees()` methods work at any level of the tree. The company, a department, and an individual employee all respond to the same interface.

Real-World Use Case: Permission System

```python
class Permission(ABC):
    @abstractmethod
    def has_access(self, resource):
        pass

    @abstractmethod
    def describe(self, indent=0):
        pass


class SinglePermission(Permission):
    """Leaf: access to one specific resource."""

    def __init__(self, resource, level="read"):
        self.resource = resource
        self.level = level

    def has_access(self, resource):
        return self.resource == resource

    def describe(self, indent=0):
        padding = "  " * indent
        print(f"{padding}[{self.level.upper()}] {self.resource}")


class PermissionGroup(Permission):
    """Composite: a named group of permissions."""

    def __init__(self, name):
        self.name = name
        self._permissions = []

    def add(self, permission):
        self._permissions.append(permission)

    def has_access(self, resource):
        return any(p.has_access(resource) for p in self._permissions)

    def describe(self, indent=0):
        padding = "  " * indent
        print(f"{padding}Group: {self.name}")
```

```
        for p in self._permissions:
            p.describe(indent + 1)


# Build permission tree
admin_perms = PermissionGroup("Admin")
admin_perms.add(SinglePermission("users", "write"))
admin_perms.add(SinglePermission("settings", "write"))
admin_perms.add(SinglePermission("logs", "read"))

editor_perms = PermissionGroup("Editor")
editor_perms.add(SinglePermission("articles", "write"))
editor_perms.add(SinglePermission("media", "write"))

super_editor = PermissionGroup("Super Editor")
super_editor.add(editor_perms)       # include all editor perms
super_editor.add(SinglePermission("comments", "write"))  # plus more

print("=== Super Editor Permissions ===")
super_editor.describe()
print(f"\nCan access 'articles'? {super_editor.has_access('articles')}")
print(f"Can access 'users'?    {super_editor.has_access('users')}")
```

Output:

```
=== Super Editor Permissions ===
Group: Super Editor
  Group: Editor
    [WRITE] articles
    [WRITE] media
  [WRITE] comments

Can access 'articles'? True
Can access 'users'?    False
```

Before vs After Comparison

Before: Without Composite

```python
def calculate_cost(item):
    if isinstance(item, Product):
        return item.price
    elif isinstance(item, Bundle):
```

```
        total = 0
        for sub_item in item.items:
            if isinstance(sub_item, Product):
                total += sub_item.price
            elif isinstance(sub_item, Bundle):
                # Recursion with type checking at every level
                for sub_sub in sub_item.items:
                    if isinstance(sub_sub, Product):
                        total += sub_sub.price
                    # How deep do we go?
        return total
```

After: With Composite

```python
def calculate_cost(component):
    return component.get_cost()  # works for Product AND Bundle
```

Both `Product.get_cost()` and `Bundle.get_cost()` implement the same interface. The Bundle version sums its children's costs. No type checking needed anywhere.

Leaf vs Composite: Key Differences

```
+------------------+----------------------------+----------------------------+
| Aspect           | Leaf                       | Composite                  |
+------------------+----------------------------+----------------------------+
| Children         | None                       | Holds a list of components |
| add/remove       | Not applicable             | Adds/removes children      |
| Operation        | Performs work directly      | Delegates to children      |
| Termination      | Base case of recursion     | Recursive case             |
| Example          | File, MenuItem, Employee   | Directory, SubMenu, Dept   |
+------------------+----------------------------+----------------------------+
```

Design decision: Should add/remove be in the base interface?

- Transparency approach: Put add/remove in the interface. Leaves throw UnsupportedOperationException. Client code is simpler but less safe.
- Safety approach: Only composites have add/remove. Client must cast to add children. Safer but requires type awareness.

Most modern implementations prefer the safety approach.

When to Use / When NOT to Use

Use Composite When

- You have tree or hierarchy structures (menus, org charts, file systems)
- You want clients to treat individual objects and groups uniformly
- Operations should cascade through the hierarchy (render, calculate, search)
- You need recursive data structures with varying depth
- New leaf or composite types should be addable without changing existing code

Do NOT Use Composite When

- The data is flat, not hierarchical (use a simple list instead)
- Leaves and composites have very different operations (forced uniformity hurts)
- The hierarchy has a fixed, known depth (simpler approaches work fine)
- Performance is critical and the tree is very deep (recursion overhead adds up)
- You need strict type safety about what can contain what

Common Mistakes

Mistake 1: Allowing Cycles

```python
# WRONG: this creates an infinite loop
dept_a = Department("A")
dept_b = Department("B")
dept_a.add(dept_b)
dept_b.add(dept_a)  # cycle!

dept_a.display()  # infinite recursion -> stack overflow
```

Fix: Check for cycles when adding children.

```python
def add(self, component):
    if self._creates_cycle(component):
        raise ValueError("Adding this component would create a cycle")
    self._children.append(component)

def _creates_cycle(self, component):
    if component is self:
        return True
    if isinstance(component, Department):
        for child in component._children:
            if self._creates_cycle(child):
                return True
    return False
```

Mistake 2: Inconsistent Interface

```java
// WRONG: treating leaves differently
if (component instanceof SubMenu) {
    ((SubMenu) component).add(newItem);
}
// This defeats the purpose of Composite!
```

Mistake 3: No Base Case

```java
// WRONG: composite that never terminates
public int countItems() {
    int count = 1; // counts itself AND children
    for (MenuComponent child : children) {
        count += child.countItems();
    }
    return count; // should it count itself? Be consistent!
}
```

Be clear about whether composites count themselves or only their leaves.

Best Practices

1. Keep the component interface focused. Only include operations that genuinely make sense for both leaves and composites.
2. Use the safety approach for add/remove. Putting child management only on the

composite class prevents runtime errors from trying to add children to leaves.

3. Prevent cycles. Validate on add that you are not creating circular references that would cause infinite recursion.
4. Consider caching. If operations like `count()` or `totalCost()` are called frequently, cache the result and invalidate when children change.
5. Use iterators for traversal. Instead of exposing the internal children list, provide an iterator. This allows different traversal orders (depth-first, breadth-first) without changing the composite.
6. Keep leaves simple. Leaves should not have unused methods. If your interface has many methods that leaves need to stub out, your interface is too broad.

Quick Summary

The Composite pattern models tree structures where individual objects (leaves) and containers (composites) share the same interface. Operations cascade through the tree via recursion. Clients interact with the tree uniformly without needing to know whether they are dealing with a leaf or a composite.

```
Problem:  Client code must distinguish between single items and groups,
          leading to complex type-checking logic.
Solution: Define a common interface for both, with composites delegating
          operations to their children recursively.
Key:      Leaves do the work; composites delegate to children.
```

Key Points

- Composite lets you treat individual objects and groups uniformly through a shared interface
- Leaves perform operations directly; composites delegate to their children
- The pattern naturally models tree/hierarchy structures like file systems, menus, and org charts

- Operations cascade through the tree via recursion: render, count, sum, search
- Prevent cycles when building the tree to avoid infinite recursion
- Choose between transparency (add/remove in base interface) and safety (add/remove only on composites)
- Composite works well with Iterator for traversal and Visitor for operations

---

Practice Questions

1. In the file system analogy, what is the leaf and what is the composite? How does calculating total directory size demonstrate the pattern?
2. What is the difference between the transparency approach and the safety approach for handling add/remove operations? Which would you choose for a production system and why?
3. How would you modify the menu example to support a `getUrl()` method on MenuItems but not on SubMenus? Does this break the Composite pattern?
4. Explain how Composite and Iterator patterns can work together. What are the benefits?
5. A company has a product catalog where a "Bundle" contains products and other bundles.

Describe how Composite would model this, including how to calculate the total price.

Exercises

Exercise 1: File System Implementation

Build a file system model with these requirements:

- `File` (leaf) with name, size in bytes, and extension
- `Directory` (composite) with name and children
- Operations: `get_total_size()`, `display(indent)`, `find_by_extension(ext)`
- Create a tree representing: `/home/user/` with `documents/`, `photos/`, nested subdirectories, and at least 8 files
- Demonstrate that `get_total_size()` on root equals the sum of all individual files

Exercise 2: E-Commerce Category Tree

Create a product category system:

- `Product` (leaf) with name, price, and stock count
- `Category` (composite) with name and children (products or subcategories)
- Operations: `total_products()`, `total_value()` (price * stock for all products), `find_out_of_stock()`
- Build: Electronics > Phones > (iPhone, Samsung), Laptops > (MacBook, ThinkPad)
- Each product should have realistic prices and stock values

Exercise 3: Expression Tree

Build a mathematical expression evaluator using Composite:

- `NumberNode` (leaf) holds a numeric value
- `OperationNode` (composite) holds an operator (+, -, *, /) and two children
- `evaluate()` method returns the computed result
- Build and evaluate: `(3 + 5) * (10 - 2)` which should equal 64
- Add a `to_string()` method that prints the expression with parentheses

---

What Is Next?

The next chapter introduces the Flyweight Pattern, which takes the opposite approach to Composite. Where Composite creates tree structures by combining objects, Flyweight reduces memory usage by sharing common data across many objects. Together, they give you tools for both structuring and optimizing your object hierarchies.

Chapter 13: Flyweight Pattern -- Share Common State, Save Memory

What You Will Learn

- What the Flyweight pattern is and why shared state saves memory
- The difference between intrinsic state (shared) and extrinsic state (unique)
- How to implement Flyweight in Java with a connection pool example
- How to implement Flyweight in Python with game entities
- When memory optimization matters and when it does not
- How to measure and verify memory savings

Why This Chapter Matters

Imagine a game server managing 100,000 trees in a forest. Each tree has a texture image (2MB), a mesh model (1MB), and a color palette (100KB). If every tree object stores its own copy of these assets, you need 300GB of memory. But there are only 5 types of trees. With Flyweight, you store 5 sets of shared data (15MB) and 100,000 tiny objects holding just position and size (a few megabytes). Total: under 20MB instead of 300GB.

Flyweight is not about clever algorithms. It is about a simple realization: when thousands of objects share the same data, store that data once and share it.

The Problem

You are building a text editor that represents every character on screen as an object. A typical document has 50,000 characters. Each character object stores:

```
Character {
    char value;          // 'A'           -- 2 bytes
    String fontFamily;   // "Arial"       -- ~40 bytes
    int fontSize;        // 12            -- 4 bytes
    Color color;         // RGB           -- 12 bytes
    boolean bold;        // true/false    -- 1 byte
    boolean italic;      // true/false    -- 1 byte
    int x, y;            // position      -- 8 bytes
}
// Object overhead:                      -- ~16 bytes
// Total per character:                  -- ~84 bytes
// 50,000 characters * 84 bytes = 4.2 MB
```

But most characters share the same font, size, color, and style. A document might use only 10 distinct formatting combinations. With Flyweight, you store those 10 formatting objects and each character just references one, plus its own position.

```
SharedFormat {                Character {
    fontFamily: "Arial"           format: -> SharedFormat  // 8 bytes (reference)
    fontSize: 12                  value: 'A'               // 2 bytes
    color: BLACK                  x: 150                   // 4 bytes
    bold: false                   y: 200                   // 4 bytes
    italic: false             }                            // + 16 bytes overhead
}                             // Total per character: ~34 bytes
                              // 50,000 * 34 = 1.7 MB (60% less)
```

The Solution: Flyweight Pattern

```
+---------------------------+
|    FlyweightFactory       |
+---------------------------+
| - cache: Map<Key, Fly>    |
+---------------------------+
| + get(key): Flyweight     |
+---------------------------+
         |
         | creates/returns
         v
+---------------------------+
|      Flyweight            |
+---------------------------+
| Intrinsic State           |
| (shared, immutable)       |
+---------------------------+
```

```
| + operation(extrinsic)    |
+---------------------------+

  Client holds:
  +-----------+     +---------------------------+
  | Context   |---->|      Flyweight            |
  +-----------+     | (shared among many        |
  | extrinsic |     |  context objects)          |
  | state     |     +---------------------------+
  +-----------+
```

Key concepts:

- Intrinsic state: Data that is shared across many objects. It is stored inside the flyweight and never changes. Examples: font family, texture, mesh model, icon image.
- Extrinsic state: Data unique to each instance. It is stored outside the flyweight, typically by the client or context. Examples: position, timestamp, user ID.
- Flyweight Factory: Creates and caches flyweight objects. When asked for a flyweight, it returns an existing one if possible, or creates a new one.

```
INTRINSIC vs EXTRINSIC STATE

+-------------------------------------------------------+
|                     Tree Object                       |
+-------------------------------------------------------+
| INTRINSIC (shared)        | EXTRINSIC (unique)        |
|---------------------------|-------------------------|
| texture: oak_texture.png  | x: 150                    |
| mesh: oak_model.obj       | y: 340                    |
| color_palette: autumn     | scale: 1.2                |
| species: "Oak"            | health: 85                |
| base_height: 15           | age: 30                   |
+-------------------------------------------------------+
    Stored ONCE per type         Stored per instance
    (5 tree types = 5 copies)    (100,000 instances)
```

Java Implementation: Connection Pool

A database connection pool is a practical Flyweight. The "intrinsic state" is the connection configuration and the pooled connection itself. The "extrinsic state" is which query is being executed and by whom.

The Flyweight: Connection Configuration

```java
import java.util.Objects;

public class ConnectionConfig {
    private final String host;
    private final int port;
    private final String database;
    private final String driver;
    private final int timeoutSeconds;

    public ConnectionConfig(String host, int port, String database,
                            String driver, int timeoutSeconds) {
        this.host = host;
        this.port = port;
        this.database = database;
        this.driver = driver;
        this.timeoutSeconds = timeoutSeconds;
    }

    public String getConnectionString() {
        return String.format("%s://%s:%d/%s?timeout=%d",
                driver, host, port, database, timeoutSeconds);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof ConnectionConfig)) return false;
        ConnectionConfig that = (ConnectionConfig) o;
        return port == that.port &&
               timeoutSeconds == that.timeoutSeconds &&
               host.equals(that.host) &&
               database.equals(that.database) &&
               driver.equals(that.driver);
    }

    @Override
    public int hashCode() {
        return Objects.hash(host, port, database, driver, timeoutSeconds);
    }

    @Override
    public String toString() {
        return getConnectionString();
    }
}
```

The Flyweight Factory: Connection Pool

```java
import java.util.HashMap;
import java.util.Map;
import java.util.Queue;
import java.util.LinkedList;

public class ConnectionPool {
    private final Map<String, Queue<PooledConnection>> pool;
    private final Map<String, ConnectionConfig> configs;
    private final int maxPerKey;
    private int totalCreated = 0;
    private int totalReused = 0;

    public ConnectionPool(int maxPerKey) {
        this.pool = new HashMap<>();
        this.configs = new HashMap<>();
        this.maxPerKey = maxPerKey;
    }

    public void registerConfig(String name, ConnectionConfig config) {
        configs.put(name, config);
        pool.putIfAbsent(name, new LinkedList<>());
    }

    public PooledConnection acquire(String configName) {
        Queue<PooledConnection> available = pool.get(configName);
        if (available == null) {
            throw new IllegalArgumentException(
                "Unknown config: " + configName);
        }

        if (!available.isEmpty()) {
            totalReused++;
            PooledConnection conn = available.poll();
            conn.markAcquired();
            System.out.printf("  [REUSE] Connection for '%s' "
                    + "(pool size: %d)%n", configName, available.size());
            return conn;
        }

        totalCreated++;
        ConnectionConfig config = configs.get(configName);
        PooledConnection conn = new PooledConnection(config, configName);
        conn.markAcquired();
        System.out.printf("  [NEW]   Connection for '%s' created%n",
                configName);
        return conn;
    }

    public void release(PooledConnection connection) {
        String name = connection.getConfigName();
        Queue<PooledConnection> available = pool.get(name);

        if (available.size() < maxPerKey) {
            connection.markReleased();
```

```
            available.offer(connection);
            System.out.printf("  [RETURN] Connection for '%s' "
                    + "returned to pool (pool size: %d)%n",
                    name, available.size());
        } else {
            connection.close();
            System.out.printf("  [CLOSE] Connection for '%s' "
                    + "closed (pool full)%n", name);
        }
    }

    public void printStats() {
        System.out.printf("%nPool Stats: %d created, %d reused "
                + "(%.0f%% reuse rate)%n",
                totalCreated, totalReused,
                totalCreated + totalReused == 0 ? 0 :
                (100.0 * totalReused / (totalCreated + totalReused)));
    }
}
```

```java
public class PooledConnection {
    private final ConnectionConfig config;  // intrinsic (shared)
    private final String configName;
    private boolean inUse;                  // extrinsic (per-use)
    private long acquiredAt;                // extrinsic (per-use)

    public PooledConnection(ConnectionConfig config, String configName) {
        this.config = config;
        this.configName = configName;
        this.inUse = false;
    }

    public void markAcquired() {
        this.inUse = true;
        this.acquiredAt = System.currentTimeMillis();
    }

    public void markReleased() {
        this.inUse = false;
    }

    public void close() {
        this.inUse = false;
    }

    public String getConfigName() { return configName; }
    public boolean isInUse() { return inUse; }

    // Execute uses shared config (intrinsic) with per-call query (extrinsic)
    public String execute(String query) {
        return String.format("Executing '%s' on %s", query, config);
    }
}
```

Demo

```java
public class ConnectionPoolDemo {
    public static void main(String[] args) {
        ConnectionPool pool = new ConnectionPool(3);

        // Register shared configurations (intrinsic state)
        pool.registerConfig("users-db", new ConnectionConfig(
                "db1.internal", 5432, "users", "postgresql", 30));
        pool.registerConfig("orders-db", new ConnectionConfig(
                "db2.internal", 5432, "orders", "postgresql", 30));

        System.out.println("=== Round 1: First requests ===");
        PooledConnection c1 = pool.acquire("users-db");
        PooledConnection c2 = pool.acquire("users-db");
        PooledConnection c3 = pool.acquire("orders-db");

        System.out.println("\n=== Executing queries (extrinsic state) ===");
        System.out.println(c1.execute("SELECT * FROM users WHERE id=1"));
        System.out.println(c2.execute("SELECT * FROM users WHERE id=2"));
        System.out.println(c3.execute("SELECT * FROM orders LIMIT 10"));

        System.out.println("\n=== Releasing connections ===");
        pool.release(c1);
        pool.release(c2);
        pool.release(c3);

        System.out.println("\n=== Round 2: Connections reused ===");
        PooledConnection c4 = pool.acquire("users-db");
        PooledConnection c5 = pool.acquire("orders-db");
        System.out.println(c4.execute("SELECT COUNT(*) FROM users"));
        System.out.println(c5.execute("INSERT INTO orders VALUES(...)"));

        pool.release(c4);
        pool.release(c5);

        pool.printStats();
    }
}
```

Output:

```
=== Round 1: First requests ===
  [NEW]   Connection for 'users-db' created
  [NEW]   Connection for 'users-db' created
  [NEW]   Connection for 'orders-db' created

=== Executing queries (extrinsic state) ===
Executing 'SELECT * FROM users WHERE id=1' on postgresql://db1.internal:5432/users?time...
Executing 'SELECT * FROM users WHERE id=2' on postgresql://db1.internal:5432/users?time...
Executing 'SELECT * FROM orders LIMIT 10' on postgresql://db2.internal:5432/orders?time...
```

```
=== Releasing connections ===
  [RETURN] Connection for 'users-db' returned to pool (pool size: 1)
  [RETURN] Connection for 'users-db' returned to pool (pool size: 2)
  [RETURN] Connection for 'orders-db' returned to pool (pool size: 1)

=== Round 2: Connections reused ===
  [REUSE] Connection for 'users-db' (pool size: 1)
  [REUSE] Connection for 'orders-db' (pool size: 0)
Executing 'SELECT COUNT(*) FROM users' on postgresql://db1.internal:5432/users?timeout=30
Executing 'INSERT INTO orders VALUES(...)' on postgresql://db2.internal:5432/orders?tim...
  [RETURN] Connection for 'users-db' returned to pool (pool size: 2)
  [RETURN] Connection for 'orders-db' returned to pool (pool size: 1)

Pool Stats: 3 created, 2 reused (40% reuse rate)
```

The `ConnectionConfig` is intrinsic state, shared among all connections to the same database. The query string and timing are extrinsic state, different every time.

Python Implementation: Game Entities

Sharing Sprite Data Across Thousands of Entities

```python
import sys


class SpriteData:
    """Intrinsic state: shared graphical data for an entity type.

    In a real game, this would hold actual texture bytes, mesh data, etc.
    Here we simulate it with a large byte array.
    """

    def __init__(self, entity_type, texture_size_kb, animation_frames):
        self.entity_type = entity_type
        self.animation_frames = animation_frames
        # Simulate large texture data
        self.texture_data = bytearray(texture_size_kb * 1024)
        self.frame_count = len(animation_frames)

    def get_frame(self, frame_index):
        return self.animation_frames[frame_index % self.frame_count]


class SpriteFactory:
    """Flyweight factory: creates and caches shared sprite data."""
```

```
    _sprites = {}
    _stats = {"hits": 0, "misses": 0}

    @classmethod
    def get_sprite(cls, entity_type, texture_size_kb=512,
                   animation_frames=None):
        if entity_type in cls._sprites:
            cls._stats["hits"] += 1
            return cls._sprites[entity_type]

        if animation_frames is None:
            animation_frames = ["idle_0", "idle_1", "idle_2"]

        cls._stats["misses"] += 1
        sprite = SpriteData(entity_type, texture_size_kb,
                            animation_frames)
        cls._sprites[entity_type] = sprite
        return sprite

    @classmethod
    def get_stats(cls):
        total = cls._stats["hits"] + cls._stats["misses"]
        reuse = (cls._stats["hits"] / total * 100) if total > 0 else 0
        return {
            "unique_sprites": len(cls._sprites),
            "cache_hits": cls._stats["hits"],
            "cache_misses": cls._stats["misses"],
            "reuse_rate": f"{reuse:.1f}%"
        }


class GameEntity:
    """Context: individual entity with unique position and state."""

    def __init__(self, entity_type, x, y):
        # Extrinsic state (unique per entity)
        self.x = x
        self.y = y
        self.health = 100
        self.current_frame = 0

        # Intrinsic state (shared via flyweight)
        self.sprite = SpriteFactory.get_sprite(entity_type)

    def update(self):
        self.current_frame += 1

    def render(self):
        frame = self.sprite.get_frame(self.current_frame)
        return (f"{self.sprite.entity_type} at ({self.x},{self.y}) "
                f"frame={frame} hp={self.health}")


# Create many entities sharing few sprite types
```

```
import random

entities = []
entity_types = ["goblin", "skeleton", "orc", "bat", "spider"]

print("Creating 10,000 game entities...")
for i in range(10_000):
    etype = random.choice(entity_types)
    x = random.randint(0, 1000)
    y = random.randint(0, 1000)
    entities.append(GameEntity(etype, x, y))

# Show some entities
print("\nSample entities:")
for entity in entities[:5]:
    print(f"  {entity.render()}")

# Memory analysis
sprite_memory = sum(
    sys.getsizeof(s.texture_data)
    for s in SpriteFactory._sprites.values()
)
entity_memory = len(entities) * sys.getsizeof(entities[0])

print(f"\n=== Memory Analysis ===")
print(f"Unique sprite types: {len(SpriteFactory._sprites)}")
print(f"Shared sprite data:  {sprite_memory / 1024:.0f} KB")
print(f"Entity references:   {entity_memory / 1024:.0f} KB")
print(f"Total with sharing:  {(sprite_memory + entity_memory) / 1024:.0f} KB")

without_sharing = len(entities) * (512 * 1024)  # each with own texture
print(f"Without sharing:     {without_sharing / 1024 / 1024:.0f} MB")
print(f"Memory saved:        {(1 - (sprite_memory + entity_memory) / without_sharing) *...

print(f"\nFactory stats: {SpriteFactory.get_stats()}")
```

Output:

```
Creating 10,000 game entities...

Sample entities:
  skeleton at (423,891) frame=idle_0 hp=100
  goblin at (156,302) frame=idle_0 hp=100
  orc at (789,445) frame=idle_0 hp=100
  bat at (234,667) frame=idle_0 hp=100
  spider at (912,123) frame=idle_0 hp=100

=== Memory Analysis ===
Unique sprite types: 5
Shared sprite data:  2560 KB
Entity references:   547 KB
Total with sharing:  3107 KB
Without sharing:     5000 MB
```

```
Memory saved:        99.9%

Factory stats: {'unique_sprites': 5, 'cache_hits': 9995, 'cache_misses': 5, 'reuse_rate...
```

Instead of 5GB of texture data (10,000 entities x 512KB each), we use only 2.5MB (5 unique textures). That is a 99.9% reduction.

Real-World Use Case: Text Formatting

```python
class CharacterFormat:
    """Flyweight: shared text formatting."""

    def __init__(self, font, size, color, bold, italic):
        self.font = font
        self.size = size
        self.color = color
        self.bold = bold
        self.italic = italic

    def __repr__(self):
        style = ""
        if self.bold:
            style += "B"
        if self.italic:
            style += "I"
        return f"[{self.font} {self.size}pt {self.color} {style}]"


class FormatFactory:
    """Flyweight factory for text formats."""

    _cache = {}

    @classmethod
    def get_format(cls, font="Arial", size=12, color="black",
                   bold=False, italic=False):
        key = (font, size, color, bold, italic)
        if key not in cls._cache:
            cls._cache[key] = CharacterFormat(
                font, size, color, bold, italic)
        return cls._cache[key]

    @classmethod
    def format_count(cls):
        return len(cls._cache)
```

```
class FormattedChar:
    """Context: one character with its format and position."""

    def __init__(self, char, row, col, fmt):
        self.char = char        # extrinsic
        self.row = row          # extrinsic
        self.col = col          # extrinsic
        self.format = fmt       # intrinsic (shared flyweight)


class Document:
    def __init__(self):
        self.characters = []

    def add_text(self, text, row, start_col, **fmt_kwargs):
        fmt = FormatFactory.get_format(**fmt_kwargs)
        for i, char in enumerate(text):
            self.characters.append(
                FormattedChar(char, row, start_col + i, fmt))

    def render(self):
        current_fmt = None
        output = []
        for ch in self.characters:
            if ch.format != current_fmt:
                current_fmt = ch.format
                output.append(f"\n  {current_fmt}: ")
            output.append(ch.char)
        return "".join(output)


# Build a document
doc = Document()
doc.add_text("Hello World", row=0, start_col=0,
             font="Arial", size=24, bold=True)
doc.add_text("This is normal text. ", row=1, start_col=0)
doc.add_text("This is italic.", row=1, start_col=21,
             italic=True)
doc.add_text("Back to normal text again.", row=2, start_col=0)

print("=== Document ===")
print(doc.render())
print(f"\nTotal characters: {len(doc.characters)}")
print(f"Unique formats:   {FormatFactory.format_count()}")
print(f"Formats shared across {len(doc.characters)} characters")
```

Output:

```
=== Document ===
  [Arial 24pt black B]: Hello World
  [Arial 12pt black ]: This is normal text.
  [Arial 12pt black I]: This is italic.
```

```
  [Arial 12pt black ]: Back to normal text again.

Total characters: 72
Unique formats:   3
Formats shared across 72 characters
```

72 characters share only 3 format objects. In a real document with thousands of characters, this saving becomes significant.

Before vs After Comparison

Before: Every Object Owns Its Data

```python
class TreeWithoutFlyweight:
    def __init__(self, species, x, y, texture_path):
        self.species = species
        self.x = x
        self.y = y
        # Each tree loads its own copy of the texture
        self.texture = load_texture(texture_path)  # 2MB each!
        self.mesh = load_mesh(species)              # 1MB each!

# 10,000 oak trees = 10,000 * 3MB = 30GB
trees = [TreeWithoutFlyweight("oak", i, i, "oak.png")
         for i in range(10_000)]
```

After: Shared Intrinsic State

```python
class TreeType:  # Flyweight
    def __init__(self, species, texture_path):
        self.species = species
        self.texture = load_texture(texture_path)  # 2MB, loaded ONCE
        self.mesh = load_mesh(species)              # 1MB, loaded ONCE

class Tree:  # Context
    def __init__(self, tree_type, x, y):
        self.type = tree_type  # reference to shared flyweight
        self.x = x             # unique per tree
        self.y = y             # unique per tree

# 5 tree types * 3MB = 15MB shared data
```

```
# 10,000 trees * ~32 bytes = ~312KB for positions
# Total: ~15.3MB instead of 30GB
```

When Memory Matters

Flyweight is worth the complexity only when:

```
+-------------------------------------------------------------------+
| Memory savings = (N - U) * S                                      |
|                                                                   |
| N = number of objects                                             |
| U = number of unique intrinsic state combinations                 |
| S = size of intrinsic state per object                            |
|                                                                   |
| Example:                                                          |
| 10,000 trees - 5 types = 9,995 duplicates avoided                |
| 9,995 * 3MB = ~29.9 GB saved                                     |
+-------------------------------------------------------------------+
```

Flyweight is worth it when:

- N is very large (thousands or millions of objects)
- U is much smaller than N (few unique types)
- S is significant (kilobytes or more per shared state)

Flyweight is NOT worth it when:

- N is small (hundreds or fewer)
- Every object is unique (U equals N)
- S is tiny (a few bytes)
- The complexity of managing shared state outweighs memory savings

---

When to Use / When NOT to Use

Use Flyweight When

- Your application creates huge numbers of similar objects
- Memory consumption is a real, measured problem

- Objects can be split into shared (intrinsic) and unique (extrinsic) state
- The intrinsic state is large relative to the extrinsic state
- Most objects share a small number of distinct intrinsic state combinations
- Examples: game sprites, text formatting, connection pools, cached configs

Do NOT Use Flyweight When

- You have few objects (premature optimization)
- Memory is not a bottleneck (profile first, optimize second)
- Objects cannot be meaningfully split into shared/unique state
- The shared state is mutable (flyweights must be immutable)
- Thread safety of shared state is too complex for your use case
- The indirection makes code significantly harder to understand

---

Common Mistakes

Mistake 1: Mutable Flyweights

```python
# WRONG: flyweight state can be changed
class SpriteData:
    def __init__(self, entity_type):
        self.entity_type = entity_type
        self.color = "red"  # mutable!

sprite = SpriteFactory.get("goblin")
sprite.color = "blue"  # Changes it for ALL goblins!

# RIGHT: flyweights must be immutable
class SpriteData:
    def __init__(self, entity_type, color):
        self._entity_type = entity_type
        self._color = color

    @property
    def entity_type(self):
        return self._entity_type

    @property
    def color(self):
        return self._color
```

Mistake 2: Putting Extrinsic State in the Flyweight

```java
// WRONG: position is unique per entity, not shared
public class TreeType {
    String species;
    byte[] texture;
    int x, y;  // This should NOT be here!
}

// RIGHT: only shared data in the flyweight
public class TreeType {
    final String species;
    final byte[] texture;
    // x, y stays in the Tree context object
}
```

Mistake 3: No Factory Control

```python
# WRONG: creating flyweights directly
sprite1 = SpriteData("goblin", 512, ["idle"])
sprite2 = SpriteData("goblin", 512, ["idle"])
# sprite1 is not sprite2 -- no sharing!

# RIGHT: always go through the factory
sprite1 = SpriteFactory.get_sprite("goblin")
sprite2 = SpriteFactory.get_sprite("goblin")
# sprite1 is sprite2 -- same object, shared
```

Best Practices

1. Make flyweights immutable. Since they are shared, any mutation affects all users. Use final fields in Java, properties in Python, and no setters.
2. Always use a factory. Direct construction bypasses the cache and defeats the pattern. Make the flyweight constructor private or protected.
3. Profile before applying. Flyweight adds indirection. Only use it when you have measured that memory is actually a problem. Do not guess.
4. Clearly separate intrinsic and extrinsic state. Document which fields are shared and which are unique. This is the core design decision of Flyweight.

5. Consider thread safety. Since flyweights are shared across threads, they must be thread-safe. Immutability is the easiest way to achieve this.
6. Use identity comparison for cache hits. After getting a flyweight from the factory, you can use `==` (Java) or `is` (Python) to check if two objects share the same flyweight.
7. Combine with other patterns. Flyweight often works with Composite (shared leaf data in trees) and Factory Method (the flyweight factory).

Quick Summary

The Flyweight pattern reduces memory by sharing common (intrinsic) state across many objects while keeping unique (extrinsic) state external. A factory ensures that shared objects are created once and reused. The pattern is essential when creating thousands or millions of similar objects where most of the data is identical.

```
Problem:  Thousands of objects duplicate the same large data.
Solution: Extract shared data into flyweight objects, referenced by many.
Key:      Intrinsic state is shared and immutable.
          Extrinsic state is unique and stored externally.
```

Key Points

- Flyweight shares intrinsic (common, immutable) state to reduce memory
- Intrinsic state is stored inside the flyweight; extrinsic state stays with the context object
- A Flyweight Factory creates and caches flyweight instances
- Flyweights must be immutable because they are shared
- The pattern saves memory when N objects share U << N distinct states of size S
- Always profile first -- only apply Flyweight when memory is a measured problem
- Common examples: game sprites, text formatting, connection pools, icon caches
- Java's `String.intern()` and `Integer.valueOf()` (for -128 to 127) are built-in flyweight implementations

Practice Questions

1. Explain the difference between intrinsic and extrinsic state. For a "Button" UI component used across 500 forms, what would be intrinsic and what would be extrinsic?
2. Why must flyweight objects be immutable? What bug would occur if a flyweight's intrinsic state were mutable?
3. Java caches Integer objects for values -128 to 127. Explain why this is an example of the Flyweight pattern and what the intrinsic/extrinsic states are.
4. When would applying Flyweight actually make performance worse? Describe a scenario where the overhead of the factory and lookups exceeds the memory savings.
5. How does Flyweight differ from a simple cache? What is the key distinction?

Exercises

Exercise 1: Icon Cache System

Build an icon management system for a dashboard:

- An `Icon` flyweight class with: name, SVG path data (simulate with a large string), default size, and color
- An `IconFactory` that caches icons by name
- An `IconInstance` context with position (x, y), actual size, and rotation
- Create 1,000 icon instances from 10 unique icons
- Measure and print memory usage with and without sharing

Exercise 2: Map Tile System

Implement a game map with shared tiles:

- A `TileType` flyweight with: terrain name, movement cost, texture data (simulated), and walkable flag

- A `MapTile` context with: row, column, elevation, and damage modifier
- A `TileFactory` that manages the shared tile types
- Create a 100x100 grid (10,000 tiles) using only 6 terrain types
- Implement `render_map()` that prints a small section using terrain abbreviations

Exercise 3: Log Entry Formatter

Build a logging system that shares format templates:

- A `LogFormat` flyweight with: pattern string, date format, separator, and colors
- A `LogEntry` context with: message, timestamp, level, and source
- A `LogFormatFactory` with preset formats: "simple", "detailed", "json", "csv"
- Generate 10,000 log entries across 4 format types
- Compare memory usage: each entry owning its format vs sharing via flyweight

---

What Is Next?

Having learned how to reduce memory with shared state, the next chapter explores the Proxy Pattern, which controls access to objects. Where Flyweight optimizes resource usage by sharing data, Proxy optimizes resource usage by deferring creation, caching results, or restricting access. Both patterns put a layer between the client and the real object, but for very different reasons.

Chapter 14: Strategy Pattern -- Swap Algorithms at Runtime

What You Will Learn

- How to replace tangled `if/else` chains with interchangeable algorithm objects
- Implementing the Strategy pattern in Java and Python
- Using Spring's `@Qualifier` to inject strategies automatically
- Applying strategies to real-world problems like payment processing, compression, and authentication
- Deciding when Strategy is the right tool and when it is overkill

Why This Chapter Matters

Every backend developer eventually writes code like this: a growing pile of `if/else` or `switch` statements that pick different behaviors based on some condition. Add a new payment method? Another branch. Support a new compression algorithm? Another branch. Every change risks breaking existing logic, and the method grows until nobody wants to touch it.

The Strategy pattern solves this by pulling each algorithm into its own object. You pick the one you need at runtime, plug it in, and the rest of your code never changes. It is one of the most practical patterns you will use in backend development.

The Problem: The Growing If/Else Chain

Imagine you are building a pricing service. Different customer types get different discounts.

The Painful Way (Before Strategy)

```java
// Java -- PricingService.java (BEFORE)
public class PricingService {

    public double calculatePrice(String customerType, double basePrice) {
        if ("REGULAR".equals(customerType)) {
            return basePrice;
        } else if ("PREMIUM".equals(customerType)) {
            return basePrice * 0.9;  // 10% off
        } else if ("VIP".equals(customerType)) {
            return basePrice * 0.8;  // 20% off
        } else if ("EMPLOYEE".equals(customerType)) {
            return basePrice * 0.7;  // 30% off
        } else {
            throw new IllegalArgumentException("Unknown type: " + customerType);
        }
        // New customer type? Add ANOTHER branch here.
        // Testing? You must test the ENTIRE method every time.
    }
}
```

```
Problems with this approach:
+-------------------------------------------+
| 1. Every new type modifies this class     |
| 2. Violates Open/Closed Principle         |
| 3. Hard to test individual strategies     |
| 4. String-based type checking is fragile  |
| 5. Logic for ALL types lives in ONE place |
+-------------------------------------------+
```

The Solution: Strategy Pattern

The Strategy pattern says: define a family of algorithms, put each in its own class, and make them interchangeable.

```
+-------------------------------------------------------+
|                   HOW STRATEGY WORKS                  |
+-------------------------------------------------------+

  Client (PricingService)
      |
      | uses
      v
```

```
+-----------------------+
| PricingStrategy       |  <--- Interface
| + calculatePrice()    |
+-----------------------+
    ^         ^         ^
    |         |         |
+--------+ +--------+ +--------+
|Regular | |Premium | | VIP    |  <--- Concrete Strategies
|Strategy| |Strategy| |Strategy|
+--------+ +--------+ +--------+

The client does NOT know which strategy it holds.
You can swap strategies at runtime without changing
the client code.
```

Java Implementation: PricingStrategy

Step 1: Define the Strategy Interface

```java
// PricingStrategy.java
public interface PricingStrategy {
    double calculatePrice(double basePrice);
    String getStrategyName();
}
```

Step 2: Implement Concrete Strategies

```java
// RegularPricingStrategy.java
public class RegularPricingStrategy implements PricingStrategy {

    @Override
    public double calculatePrice(double basePrice) {
        return basePrice;  // No discount
    }

    @Override
    public String getStrategyName() {
        return "REGULAR";
    }
}

// PremiumPricingStrategy.java
```

```
public class PremiumPricingStrategy implements PricingStrategy {

    @Override
    public double calculatePrice(double basePrice) {
        return basePrice * 0.9;  // 10% discount
    }

    @Override
    public String getStrategyName() {
        return "PREMIUM";
    }
}

// VIPPricingStrategy.java
public class VIPPricingStrategy implements PricingStrategy {

    @Override
    public double calculatePrice(double basePrice) {
        return basePrice * 0.8;  // 20% discount
    }

    @Override
    public String getStrategyName() {
        return "VIP";
    }
}
```

Step 3: Use the Strategy in a Service

```java
// PricingService.java (AFTER -- Strategy Pattern)
public class PricingService {
    private PricingStrategy strategy;

    public PricingService(PricingStrategy strategy) {
        this.strategy = strategy;
    }

    public void setStrategy(PricingStrategy strategy) {
        this.strategy = strategy;
    }

    public double calculatePrice(double basePrice) {
        return strategy.calculatePrice(basePrice);
    }
}
```

Step 4: Client Code

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        PricingService service = new PricingService(new RegularPricingStrategy());

        double price = 100.0;
        System.out.println("Regular: $" + service.calculatePrice(price));

        // Swap strategy at runtime -- no if/else needed
        service.setStrategy(new PremiumPricingStrategy());
        System.out.println("Premium: $" + service.calculatePrice(price));

        service.setStrategy(new VIPPricingStrategy());
        System.out.println("VIP:     $" + service.calculatePrice(price));
    }
}
```

Output:

```
Regular: $100.0
Premium: $90.0
VIP:     $80.0
```

Before vs After Comparison

```
BEFORE (if/else):                    AFTER (Strategy):
+---------------------------+        +---------------------------+
| PricingService            |        | PricingService            |
|                           |        |   - strategy: Strategy    |
| if (type == "REGULAR")    |        |   + calculatePrice()      |
|   return basePrice;       |        +---------------------------+
| else if (type == "PREMIUM"|                   |
|   return basePrice * 0.9; |                   v
| else if (type == "VIP")   |        +---------------------------+
|   return basePrice * 0.8; |        | <<interface>>             |
| else if ...               |        | PricingStrategy           |
| // KEEPS GROWING          |        |   + calculatePrice()      |
+---------------------------+        +---------------------------+
                                        ^       ^       ^
  Adding a new type:                    |       |       |
  - Modify existing class          Regular  Premium   VIP
  - Risk breaking other types
  - Retest everything               Adding a new type:
                                     - Add a new class
                                     - Existing code untouched
```

```
                                        - Test ONLY the new class
```

Java Implementation: SortingStrategy

Here is another common backend scenario -- choosing a sorting algorithm based on data characteristics.

```java
// SortingStrategy.java
public interface SortingStrategy<T extends Comparable<T>> {
    void sort(List<T> data);
    String getName();
}

// QuickSortStrategy.java
public class QuickSortStrategy<T extends Comparable<T>> implements SortingStrategy<T> {

    @Override
    public void sort(List<T> data) {
        Collections.sort(data);  // Java uses TimSort internally
        System.out.println("  [QuickSort applied to " + data.size() + " elements]");
    }

    @Override
    public String getName() {
        return "QuickSort";
    }
}

// BubbleSortStrategy.java
public class BubbleSortStrategy<T extends Comparable<T>> implements SortingStrategy<T> {

    @Override
    public void sort(List<T> data) {
        for (int i = 0; i < data.size() - 1; i++) {
            for (int j = 0; j < data.size() - i - 1; j++) {
                if (data.get(j).compareTo(data.get(j + 1)) > 0) {
                    T temp = data.get(j);
                    data.set(j, data.get(j + 1));
                    data.set(j + 1, temp);
                }
            }
        }
        System.out.println("  [BubbleSort applied to " + data.size() + " elements]");
    }

    @Override
```

```
    public String getName() {
        return "BubbleSort";
    }
}

// DataProcessor.java
public class DataProcessor<T extends Comparable<T>> {
    private SortingStrategy<T> sortingStrategy;

    public DataProcessor(SortingStrategy<T> sortingStrategy) {
        this.sortingStrategy = sortingStrategy;
    }

    public void setSortingStrategy(SortingStrategy<T> strategy) {
        this.sortingStrategy = strategy;
    }

    public List<T> process(List<T> data) {
        List<T> copy = new ArrayList<>(data);
        System.out.println("Processing with " + sortingStrategy.getName() + ":");
        sortingStrategy.sort(copy);
        return copy;
    }
}
```

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        List<Integer> data = Arrays.asList(42, 17, 93, 5, 28, 61);

        DataProcessor<Integer> processor = new DataProcessor<>(new QuickSortStrategy<>());
        List<Integer> result1 = processor.process(data);
        System.out.println("Result: " + result1);

        // Switch to BubbleSort for small datasets
        processor.setSortingStrategy(new BubbleSortStrategy<>());
        List<Integer> smallData = Arrays.asList(3, 1, 2);
        List<Integer> result2 = processor.process(smallData);
        System.out.println("Result: " + result2);
    }
}
```

Output:

```
Processing with QuickSort:
  [QuickSort applied to 6 elements]
Result: [5, 17, 28, 42, 61, 93]
Processing with BubbleSort:
  [BubbleSort applied to 3 elements]
Result: [1, 2, 3]
```

Python Implementation: AuthStrategy

Authentication is a perfect use case. Different endpoints may need different authentication methods.

```python
# auth_strategy.py
from abc import ABC, abstractmethod
import hashlib
import hmac
import base64
import time


class AuthStrategy(ABC):
    """Base strategy for authentication."""

    @abstractmethod
    def authenticate(self, request: dict) -> bool:
        """Return True if the request is authenticated."""
        pass

    @abstractmethod
    def get_name(self) -> str:
        pass


class APIKeyAuthStrategy(AuthStrategy):
    """Authenticate using an API key in the header."""

    VALID_KEYS = {"sk-abc123", "sk-def456", "sk-ghi789"}

    def authenticate(self, request: dict) -> bool:
        api_key = request.get("headers", {}).get("X-API-Key", "")
        return api_key in self.VALID_KEYS

    def get_name(self) -> str:
        return "API Key"


class JWTAuthStrategy(AuthStrategy):
    """Authenticate using a JWT token."""

    SECRET = "my-secret-key"

    def authenticate(self, request: dict) -> bool:
        auth_header = request.get("headers", {}).get("Authorization", "")
        if not auth_header.startswith("Bearer "):
            return False
```

```
        token = auth_header[7:]  # Remove "Bearer "
        # Simplified JWT validation (real apps use a JWT library)
        try:
            parts = token.split(".")
            return len(parts) == 3 and len(token) > 20
        except Exception:
            return False

    def get_name(self) -> str:
        return "JWT"


class HMACAuthStrategy(AuthStrategy):
    """Authenticate using HMAC signature."""

    SECRET = b"shared-secret-key"

    def authenticate(self, request: dict) -> bool:
        signature = request.get("headers", {}).get("X-Signature", "")
        body = request.get("body", "")

        expected = hmac.new(
            self.SECRET, body.encode(), hashlib.sha256
        ).hexdigest()
        return hmac.compare_digest(signature, expected)

    def get_name(self) -> str:
        return "HMAC"


class AuthService:
    """Service that delegates authentication to a strategy."""

    def __init__(self, strategy: AuthStrategy):
        self._strategy = strategy

    @property
    def strategy(self) -> AuthStrategy:
        return self._strategy

    @strategy.setter
    def strategy(self, new_strategy: AuthStrategy):
        self._strategy = new_strategy

    def authenticate(self, request: dict) -> bool:
        print(f"  Authenticating with {self._strategy.get_name()}...", end=" ")
        result = self._strategy.authenticate(request)
        print("OK" if result else "FAILED")
        return result


# --- Usage ---
if __name__ == "__main__":
    # API Key authentication
```

```
    service = AuthService(APIKeyAuthStrategy())

    request_with_key = {
        "headers": {"X-API-Key": "sk-abc123"},
        "body": '{"action": "list_users"}'
    }
    service.authenticate(request_with_key)

    # Switch to JWT authentication at runtime
    service.strategy = JWTAuthStrategy()

    request_with_jwt = {
        "headers": {"Authorization": "Bearer eyJhbGciOiJIUzI1NiJ9.payload.signature"},
        "body": '{"action": "get_profile"}'
    }
    service.authenticate(request_with_jwt)

    # Switch to HMAC for webhook verification
    service.strategy = HMACAuthStrategy()

    body = '{"event": "payment.completed"}'
    signature = hmac.new(
        b"shared-secret-key", body.encode(), hashlib.sha256
    ).hexdigest()

    request_with_hmac = {
        "headers": {"X-Signature": signature},
        "body": body
    }
    service.authenticate(request_with_hmac)
```

Output:

```
  Authenticating with API Key... OK
  Authenticating with JWT... OK
  Authenticating with HMAC... OK
```

Spring Boot: Strategy with @Qualifier

Spring makes the Strategy pattern even more powerful with dependency injection.

```java
// PricingStrategy.java
public interface PricingStrategy {
    double calculatePrice(double basePrice);
```

```
    String getType();
}

// RegularPricing.java
@Component
@Qualifier("regular")
public class RegularPricing implements PricingStrategy {

    @Override
    public double calculatePrice(double basePrice) {
        return basePrice;
    }

    @Override
    public String getType() {
        return "REGULAR";
    }
}

// PremiumPricing.java
@Component
@Qualifier("premium")
public class PremiumPricing implements PricingStrategy {

    @Override
    public double calculatePrice(double basePrice) {
        return basePrice * 0.9;
    }

    @Override
    public String getType() {
        return "PREMIUM";
    }
}

// VIPPricing.java
@Component
@Qualifier("vip")
public class VIPPricing implements PricingStrategy {

    @Override
    public double calculatePrice(double basePrice) {
        return basePrice * 0.8;
    }

    @Override
    public String getType() {
        return "VIP";
    }
}
```

Strategy Registry Pattern (Spring-Powered)

```java
// PricingStrategyRegistry.java
@Component
public class PricingStrategyRegistry {

    private final Map<String, PricingStrategy> strategies;

    // Spring injects ALL PricingStrategy beans automatically
    @Autowired
    public PricingStrategyRegistry(List<PricingStrategy> strategyList) {
        strategies = strategyList.stream()
            .collect(Collectors.toMap(
                PricingStrategy::getType,
                Function.identity()
            ));
        System.out.println("Registered strategies: " + strategies.keySet());
    }

    public PricingStrategy getStrategy(String customerType) {
        PricingStrategy strategy = strategies.get(customerType);
        if (strategy == null) {
            throw new IllegalArgumentException(
                "No strategy for type: " + customerType
                + ". Available: " + strategies.keySet()
            );
        }
        return strategy;
    }
}

// PricingController.java
@RestController
@RequestMapping("/api/pricing")
public class PricingController {

    private final PricingStrategyRegistry registry;

    public PricingController(PricingStrategyRegistry registry) {
        this.registry = registry;
    }

    @GetMapping("/calculate")
    public Map<String, Object> calculate(
            @RequestParam String customerType,
            @RequestParam double basePrice) {

        PricingStrategy strategy = registry.getStrategy(customerType);
        double finalPrice = strategy.calculatePrice(basePrice);

        return Map.of(
            "customerType", customerType,
            "basePrice", basePrice,
            "finalPrice", finalPrice,
            "discount", (1 - finalPrice / basePrice) * 100 + "%"
        );
```

```
    }
}
```

```
How Spring Strategy Registry Works:

  Spring Container
  +-----------------------------------------------+
  |                                               |
  |  @Component         @Component     @Component  |
  |  RegularPricing     PremiumPricing VIPPricing  |
  |       |                  |             |        |
  |       +------------------+-------------+        |
  |                     |                           |
  |                     v                           |
  |        PricingStrategyRegistry                  |
  |        Map<String, PricingStrategy>             |
  |        {                                        |
  |          "REGULAR" -> RegularPricing,            |
  |          "PREMIUM" -> PremiumPricing,            |
  |          "VIP"     -> VIPPricing                 |
  |        }                                        |
  |                     |                           |
  |                     v                           |
  |           PricingController                     |
  |           GET /api/pricing/calculate            |
  +-----------------------------------------------+

  To add a new strategy:
  1. Create a new @Component class
  2. Implement PricingStrategy
  3. Done! Spring auto-discovers it.
```

Real-World Use Case: Payment Processing

```java
// PaymentStrategy.java
public interface PaymentStrategy {
    PaymentResult process(PaymentRequest request);
    boolean supports(String paymentMethod);
    int getRetryLimit();
}

// CreditCardPayment.java
@Component
public class CreditCardPayment implements PaymentStrategy {
```

```
    @Override
    public PaymentResult process(PaymentRequest request) {
        // Connect to credit card gateway
        System.out.println("Processing credit card: **** " +
            request.getCardNumber().substring(12));
        // Charge the card...
        return new PaymentResult(true, "CC-" + System.currentTimeMillis());
    }

    @Override
    public boolean supports(String paymentMethod) {
        return "CREDIT_CARD".equals(paymentMethod);
    }

    @Override
    public int getRetryLimit() {
        return 3;  // Allow retries for network issues
    }
}

// PayPalPayment.java
@Component
public class PayPalPayment implements PaymentStrategy {

    @Override
    public PaymentResult process(PaymentRequest request) {
        System.out.println("Redirecting to PayPal for: $" + request.getAmount());
        // Initiate PayPal flow...
        return new PaymentResult(true, "PP-" + System.currentTimeMillis());
    }

    @Override
    public boolean supports(String paymentMethod) {
        return "PAYPAL".equals(paymentMethod);
    }

    @Override
    public int getRetryLimit() {
        return 1;  // No retries -- user must restart
    }
}

// BankTransferPayment.java
@Component
public class BankTransferPayment implements PaymentStrategy {

    @Override
    public PaymentResult process(PaymentRequest request) {
        System.out.println("Initiating bank transfer: $" + request.getAmount());
        // Queue bank transfer...
        return new PaymentResult(true, "BT-" + System.currentTimeMillis());
    }

    @Override
    public boolean supports(String paymentMethod) {
```

```
        return "BANK_TRANSFER".equals(paymentMethod);
    }

    @Override
    public int getRetryLimit() {
        return 0;  // Never retry bank transfers
    }
}

// PaymentService.java
@Service
public class PaymentService {

    private final List<PaymentStrategy> strategies;

    @Autowired
    public PaymentService(List<PaymentStrategy> strategies) {
        this.strategies = strategies;
    }

    public PaymentResult processPayment(PaymentRequest request) {
        PaymentStrategy strategy = strategies.stream()
            .filter(s -> s.supports(request.getPaymentMethod()))
            .findFirst()
            .orElseThrow(() -> new UnsupportedOperationException(
                "No strategy for: " + request.getPaymentMethod()
            ));

        int attempts = 0;
        while (attempts <= strategy.getRetryLimit()) {
            try {
                return strategy.process(request);
            } catch (Exception e) {
                attempts++;
                System.out.println("Attempt " + attempts + " failed: " + e.getMessage());
            }
        }
        return new PaymentResult(false, "FAILED after " + attempts + " attempts");
    }
}
```

Real-World Use Case: Compression Strategy

```python
# compression_strategy.py
from abc import ABC, abstractmethod
import gzip
```

```
import zlib
import json


class CompressionStrategy(ABC):

    @abstractmethod
    def compress(self, data: bytes) -> bytes:
        pass

    @abstractmethod
    def decompress(self, data: bytes) -> bytes:
        pass

    @abstractmethod
    def get_name(self) -> str:
        pass


class GzipCompression(CompressionStrategy):

    def compress(self, data: bytes) -> bytes:
        return gzip.compress(data)

    def decompress(self, data: bytes) -> bytes:
        return gzip.decompress(data)

    def get_name(self) -> str:
        return "gzip"


class ZlibCompression(CompressionStrategy):

    def compress(self, data: bytes) -> bytes:
        return zlib.compress(data, level=9)

    def decompress(self, data: bytes) -> bytes:
        return zlib.decompress(data)

    def get_name(self) -> str:
        return "zlib"


class NoCompression(CompressionStrategy):

    def compress(self, data: bytes) -> bytes:
        return data  # Pass-through

    def decompress(self, data: bytes) -> bytes:
        return data

    def get_name(self) -> str:
        return "none"
```

```
class DataStore:
    """Stores data using a pluggable compression strategy."""

    def __init__(self, strategy: CompressionStrategy):
        self._strategy = strategy
        self._storage: dict[str, bytes] = {}

    @property
    def strategy(self):
        return self._strategy

    @strategy.setter
    def strategy(self, new_strategy: CompressionStrategy):
        self._strategy = new_strategy

    def store(self, key: str, data: str) -> None:
        raw = data.encode("utf-8")
        compressed = self._strategy.compress(raw)
        self._storage[key] = compressed
        ratio = len(compressed) / len(raw) * 100
        print(f"  Stored '{key}': {len(raw)}B -> {len(compressed)}B "
              f"({ratio:.1f}%) using {self._strategy.get_name()}")

    def retrieve(self, key: str) -> str:
        compressed = self._storage[key]
        raw = self._strategy.decompress(compressed)
        return raw.decode("utf-8")


# --- Usage ---
if __name__ == "__main__":
    store = DataStore(GzipCompression())

    large_data = json.dumps({"users": [{"name": f"User {i}"} for i in range(100)]})
    store.store("users", large_data)

    # Switch to zlib for different data
    store.strategy = ZlibCompression()
    store.store("config", large_data)

    # Retrieve data
    retrieved = store.retrieve("users")
    print(f"  Retrieved 'users': {len(retrieved)} bytes")
```

Output:

```
  Stored 'users': 2090B -> bytes 138B (6.6%) using gzip
  Stored 'config': 2090B -> 126B (6.0%) using zlib
  Retrieved 'users': 2090 bytes
```

When to Use / When NOT to Use

Use Strategy When

Scenario	Why Strategy Helps
Multiple algorithms for the same task	Each algorithm is isolated and testable
Behavior selected at runtime	Swap strategies without code changes
Complex conditional logic	Replace if/else chains with objects
Algorithms evolve independently	Change one without affecting others
Different clients need different behaviors	Inject the right strategy per client

Do NOT Use Strategy When

Scenario	Why Not
Only two simple options	A simple `if/else` is clearer
Algorithm never changes at runtime	Direct implementation is simpler
Strategies share lots of state	They become tightly coupled anyway
You have only one algorithm	Do not over-engineer a non-problem

---

Common Mistakes

Mistake 1: Strategies That Know About Each Other

```java
// BAD -- Strategy depends on another strategy
public class PremiumPricing implements PricingStrategy {
    @Override
    public double calculatePrice(double basePrice) {
        // This couples Premium to Regular -- WRONG
        RegularPricing regular = new RegularPricing();
```

```
        return regular.calculatePrice(basePrice) * 0.9;
    }
}

// GOOD -- Each strategy is self-contained
public class PremiumPricing implements PricingStrategy {
    @Override
    public double calculatePrice(double basePrice) {
        return basePrice * 0.9;
    }
}
```

Mistake 2: Passing Too Much Context to Strategies

```java
// BAD -- Strategy knows about the entire service
public interface PricingStrategy {
    double calculatePrice(PricingService service, Order order,
                          Customer customer, Inventory inventory);
}

// GOOD -- Strategy receives only what it needs
public interface PricingStrategy {
    double calculatePrice(double basePrice);
}
```

Mistake 3: Forgetting a Default Strategy

```java
// BAD -- Crashes when strategy is null
public class PricingService {
    private PricingStrategy strategy;  // Could be null!

    public double calculate(double price) {
        return strategy.calculatePrice(price);  // NullPointerException
    }
}

// GOOD -- Always has a fallback
public class PricingService {
    private PricingStrategy strategy = new RegularPricingStrategy();

    public double calculate(double price) {
        return strategy.calculatePrice(price);  // Always safe
    }
}
```

Best Practices

1. Name strategies after what they do, not how they are selected. `GzipCompression` is better than `Option3`.
2. Use a registry or factory to map identifiers to strategies instead of hard-coding the mapping.
3. Keep the strategy interface small. One or two methods. If you need more, you might need a different pattern.
4. Make strategies stateless when possible. Stateless strategies can be shared across threads safely.
5. Provide a default strategy so the system works even without explicit configuration.
6. Let Spring auto-discover strategies by implementing a common interface and using `List<Strategy>` injection.
7. Document which strategy to use when. The pattern makes code flexible, but someone still has to pick the right one.

Quick Summary

```
+-----------------------------------------------------------------+
|                     STRATEGY PATTERN SUMMARY                    |
+-----------------------------------------------------------------+
| Intent:     Define a family of algorithms, encapsulate each     |
|             one, and make them interchangeable.                 |
+-----------------------------------------------------------------+
| Problem:    if/else chains that grow with every new algorithm   |
| Solution:   One interface, many implementations, swap freely   |
+-----------------------------------------------------------------+
| Key parts:                                                      |
|   - Strategy interface (the contract)                           |
|   - Concrete strategies (the algorithms)                        |
|   - Context (the class that uses a strategy)                    |
+-----------------------------------------------------------------+
| Runtime behavior: Change strategy = change behavior             |
| Compile-time safety: Interface guarantees compatibility         |
+-----------------------------------------------------------------+
```

Key Points

- The Strategy pattern replaces conditional logic with polymorphism.
- Each algorithm lives in its own class, making it easy to test and modify independently.
- The context class holds a reference to a strategy and delegates work to it.
- Strategies can be swapped at runtime without modifying client code.
- Spring's dependency injection and `@Qualifier` make strategy selection automatic.
- Strategy is one of the most frequently used patterns in backend services.

Practice Questions

1. You have a notification service that sends alerts via email, SMS, and push notifications. How would you apply the Strategy pattern? What would the strategy interface look like?
2. A colleague argues that an `enum` with a method per value is simpler than Strategy. When is the colleague right, and when does Strategy become the better choice?
3. In the Spring Strategy Registry example, what happens if two strategies return the same value from `getType()`? How would you prevent this?
4. How does the Strategy pattern relate to the Open/Closed Principle? Give a concrete example from the payment processing use case.
5. If strategies need access to shared resources (like a database connection), how should you provide that access without coupling strategies to infrastructure?

Exercises

Exercise 1: Shipping Cost Calculator

Build a shipping cost calculator with three strategies: `StandardShipping` (flat $5), `ExpressShipping` ($5 + $2 per kg), and `FreeShipping` (returns 0). Write a `ShippingService` that accepts a strategy and calculates the cost. Write tests that verify swapping strategies at runtime changes the output.

Exercise 2: Text Formatter

Create a `TextFormatterStrategy` with implementations for `MarkdownFormatter`, `HTMLFormatter`, and `PlainTextFormatter`. Each takes a title and a list of bullet points and formats them differently. Build a report generator that uses these strategies to produce output in any format.

Exercise 3: Rate Limiter (Advanced)

Implement a rate-limiting system with strategies: `FixedWindowLimiter` (N requests per minute), `SlidingWindowLimiter` (smooth rate limiting), and `TokenBucketLimiter` (burst-friendly). Use the Strategy pattern so the API gateway can switch rate-limiting algorithms per endpoint.

What Is Next?

You have seen how Strategy lets you swap algorithms at runtime. But what if you need to notify multiple parts of your system when something changes? In the next chapter, we will explore the Observer pattern -- a way to broadcast events so that many listeners can react without the sender knowing who they are. This pattern powers everything from order notifications to cache invalidation in modern backends.

Chapter 15: Observer Pattern -- Notify Dependents of State Changes

What You Will Learn

- How the Observer pattern decouples event producers from event consumers
- Implementing Observer in Java with custom `EventPublisher` and `EventListener`
- Building a Python signal/slot system for loose coupling
- Using Spring's `ApplicationEventPublisher` and `@EventListener` for production systems
- Understanding the differences between Observer and Pub/Sub
- Applying Observer to order events, audit logging, and cache invalidation

Why This Chapter Matters

Backend systems rarely do just one thing when something happens. When a user places an order, you need to send a confirmation email, update inventory, log the event for analytics, notify the warehouse, and maybe invalidate a cache. If the order service calls all these systems directly, it becomes a tangled mess of dependencies that breaks every time you add a new requirement.

The Observer pattern solves this by letting the order service say "something happened" without knowing or caring who is listening. Each listener independently decides how to react. This is the foundation of event-driven architecture, and understanding it will change how you design backend systems.

The Problem: Direct Coupling Between Services

```java
// OrderService.java (BEFORE -- tightly coupled)
public class OrderService {
    private final EmailService emailService;
    private final InventoryService inventoryService;
    private final AnalyticsService analyticsService;
    private final WarehouseService warehouseService;
    private final CacheService cacheService;

    // Every new requirement = new dependency + new code here
    public Order placeOrder(OrderRequest request) {
        Order order = createOrder(request);

        emailService.sendConfirmation(order);        // Coupling #1
        inventoryService.updateStock(order);          // Coupling #2
        analyticsService.trackPurchase(order);        // Coupling #3
        warehouseService.notifyNewOrder(order);       // Coupling #4
        cacheService.invalidateProductCache(order);   // Coupling #5
        // Next month: loyaltyService.addPoints(order);  // Coupling #6
        // Month after: fraudService.checkOrder(order);  // Coupling #7

        return order;
    }
}
```

```
The Problem Visualized:

              OrderService
            /   |   |   \   \
           /    |   |    \   \
          v     v   v     v   v
        Email Inventory Analytics Warehouse Cache
        Service Service  Service  Service  Service

  OrderService KNOWS about every consumer.
  Adding a new consumer requires modifying OrderService.
  If EmailService is slow, the entire order is slow.
  If AnalyticsService throws, does the order fail?
```

The Solution: Observer Pattern

```
Observer Pattern Structure:

  +-------------------+         +-------------------+
  |    Subject        |         |    Observer       |
  | (Event Publisher) |  fires  | (Event Listener)  |
  |                   | ------> |                   |
```

```
  | + subscribe()      |         | + onEvent(event)   |
  | + unsubscribe()    |         +--------------------+
  | + notify()         |              ^   ^   ^
  +--------------------+              |   |   |
                                      |   |   |
                              Email  Inv  Analytics
                              Lstnr  Lstnr Listener

  The Subject does NOT know what the Observers do.
  Observers register themselves and react independently.
```

Java Implementation: Custom EventPublisher

Step 1: Define the Event

```java
// OrderEvent.java
public class OrderEvent {
    private final String orderId;
    private final String customerEmail;
    private final double totalAmount;
    private final List<String> items;
    private final LocalDateTime timestamp;

    public OrderEvent(String orderId, String customerEmail,
                      double totalAmount, List<String> items) {
        this.orderId = orderId;
        this.customerEmail = customerEmail;
        this.totalAmount = totalAmount;
        this.items = List.copyOf(items);
        this.timestamp = LocalDateTime.now();
    }

    // Getters
    public String getOrderId() { return orderId; }
    public String getCustomerEmail() { return customerEmail; }
    public double getTotalAmount() { return totalAmount; }
    public List<String> getItems() { return items; }
    public LocalDateTime getTimestamp() { return timestamp; }

    @Override
    public String toString() {
        return "OrderEvent{id=" + orderId + ", total=$" + totalAmount + "}";
    }
}
```

Step 2: Define the Observer Interface

```java
// EventListener.java
public interface EventListener<T> {
    void onEvent(T event);
    String getName();
}
```

Step 3: Build the EventPublisher (Subject)

```java
// EventPublisher.java
public class EventPublisher<T> {
    private final List<EventListener<T>> listeners = new ArrayList<>();
    private final String topic;

    public EventPublisher(String topic) {
        this.topic = topic;
    }

    public void subscribe(EventListener<T> listener) {
        listeners.add(listener);
        System.out.println("  [" + topic + "] Subscribed: " + listener.getName());
    }

    public void unsubscribe(EventListener<T> listener) {
        listeners.remove(listener);
        System.out.println("  [" + topic + "] Unsubscribed: " + listener.getName());
    }

    public void publish(T event) {
        System.out.println("  [" + topic + "] Publishing to "
            + listeners.size() + " listeners: " + event);
        for (EventListener<T> listener : listeners) {
            try {
                listener.onEvent(event);
            } catch (Exception e) {
                System.err.println("  [" + topic + "] Error in "
                    + listener.getName() + ": " + e.getMessage());
                // One listener failing should NOT stop others
            }
        }
    }
}
```

Step 4: Implement Concrete Observers

```java
// EmailNotificationListener.java
public class EmailNotificationListener implements EventListener<OrderEvent> {

    @Override
    public void onEvent(OrderEvent event) {
        System.out.println("    [Email] Sending confirmation to "
            + event.getCustomerEmail()
            + " for order " + event.getOrderId());
    }

    @Override
    public String getName() {
        return "EmailNotification";
    }
}

// InventoryUpdateListener.java
public class InventoryUpdateListener implements EventListener<OrderEvent> {

    @Override
    public void onEvent(OrderEvent event) {
        System.out.println("    [Inventory] Reserving "
            + event.getItems().size() + " items for order "
            + event.getOrderId());
    }

    @Override
    public String getName() {
        return "InventoryUpdate";
    }
}

// AuditLogListener.java
public class AuditLogListener implements EventListener<OrderEvent> {

    @Override
    public void onEvent(OrderEvent event) {
        System.out.println("    [Audit] Logged: Order "
            + event.getOrderId() + " - $"
            + event.getTotalAmount() + " at " + event.getTimestamp());
    }

    @Override
    public String getName() {
        return "AuditLog";
    }
}
```

Step 5: Refactored OrderService

```java
// OrderService.java (AFTER -- Observer Pattern)
public class OrderService {
    private final EventPublisher<OrderEvent> orderPublisher;

    public OrderService(EventPublisher<OrderEvent> orderPublisher) {
        this.orderPublisher = orderPublisher;
    }

    public String placeOrder(String customerEmail, double total,
                             List<String> items) {
        String orderId = "ORD-" + System.currentTimeMillis();
        System.out.println("\nPlacing order: " + orderId);

        // Business logic here...

        // Notify all listeners -- OrderService does NOT know who they are
        OrderEvent event = new OrderEvent(orderId, customerEmail, total, items);
        orderPublisher.publish(event);

        return orderId;
    }
}
```

Step 6: Wire It Together

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        // Create the publisher
        EventPublisher<OrderEvent> publisher = new EventPublisher<>("orders");

        // Register listeners -- each is independent
        publisher.subscribe(new EmailNotificationListener());
        publisher.subscribe(new InventoryUpdateListener());
        publisher.subscribe(new AuditLogListener());

        // Create the service
        OrderService orderService = new OrderService(publisher);

        // Place an order
        orderService.placeOrder(
            "alice@example.com", 149.99,
            List.of("Laptop Stand", "USB-C Hub")
        );

        // Add a NEW listener without changing OrderService
        publisher.subscribe(new EventListener<OrderEvent>() {
            @Override
            public void onEvent(OrderEvent event) {
                System.out.println("    [Analytics] Tracked purchase: $"
```

```
                    + event.getTotalAmount());
            }
            @Override
            public String getName() { return "Analytics"; }
        });

        // Next order goes to ALL listeners including Analytics
        orderService.placeOrder(
            "bob@example.com", 299.99,
            List.of("Mechanical Keyboard")
        );
    }
}
```

Output:

```
  [orders] Subscribed: EmailNotification
  [orders] Subscribed: InventoryUpdate
  [orders] Subscribed: AuditLog

Placing order: ORD-1700000001
  [orders] Publishing to 3 listeners: OrderEvent{id=ORD-1700000001, total=$149.99}
    [Email] Sending confirmation to alice@example.com for order ORD-1700000001
    [Inventory] Reserving 2 items for order ORD-1700000001
    [Audit] Logged: Order ORD-1700000001 - $149.99 at 2024-11-15T10:30:00

  [orders] Subscribed: Analytics

Placing order: ORD-1700000002
  [orders] Publishing to 4 listeners: OrderEvent{id=ORD-1700000002, total=$299.99}
    [Email] Sending confirmation to bob@example.com for order ORD-1700000002
    [Inventory] Reserving 1 items for order ORD-1700000002
    [Audit] Logged: Order ORD-1700000002 - $299.99 at 2024-11-15T10:30:01
    [Analytics] Tracked purchase: $299.99
```

Before vs After

```
BEFORE:                              AFTER:
+-------------------+                +-------------------+
| OrderService      |                | OrderService      |
|                   |                |                   |
| - emailService    |                | - publisher       |
| - inventoryService|                |   (EventPublisher)|
| - analyticsService|                |                   |
| - warehouseService|                | placeOrder() {    |
```

```
| - cacheService    |                |   publisher       |
|                   |                |     .publish(evt) |
| placeOrder() {    |                | }                 |
|   email.send()    |                +-------------------+
|   inventory.update|                         |
|   analytics.track |                    publishes to
|   warehouse.notify|                         |
|   cache.invalidate|                +--------+--------+
| }                 |                |        |        |
+-------------------+                v        v        v
                                   Email    Inv.    Audit
  5 dependencies                   Lstnr    Lstnr   Lstnr
  Change = modify OrderService
                                   0 dependencies in OrderService
                                   Change = add/remove a listener
```

Python Implementation: Signal/Slot System

```python
# signal_slot.py
from typing import Callable, Any
from dataclasses import dataclass, field
from datetime import datetime


class Signal:
    """A signal that can be connected to multiple slots (callbacks)."""

    def __init__(self, name: str):
        self.name = name
        self._slots: list[Callable] = []

    def connect(self, slot: Callable) -> None:
        """Connect a slot (callback) to this signal."""
        self._slots.append(slot)
        print(f"  [{self.name}] Connected: {slot.__name__}")

    def disconnect(self, slot: Callable) -> None:
        """Disconnect a slot from this signal."""
        self._slots.remove(slot)
        print(f"  [{self.name}] Disconnected: {slot.__name__}")

    def emit(self, *args, **kwargs) -> None:
        """Emit the signal, calling all connected slots."""
        print(f"  [{self.name}] Emitting to {len(self._slots)} slots")
        for slot in self._slots:
            try:
```

```
                slot(*args, **kwargs)
            except Exception as e:
                print(f"  [{self.name}] Error in {slot.__name__}: {e}")


@dataclass
class OrderEvent:
    order_id: str
    customer: str
    total: float
    items: list[str]
    timestamp: str = field(default_factory=lambda: datetime.now().isoformat())


class OrderService:
    """Service that emits signals instead of calling listeners directly."""

    def __init__(self):
        self.order_placed = Signal("order_placed")
        self.order_cancelled = Signal("order_cancelled")

    def place_order(self, customer: str, total: float, items: list[str]) -> str:
        order_id = f"ORD-{id(self) % 10000:04d}"
        event = OrderEvent(order_id, customer, total, items)
        print(f"\nPlacing order: {order_id}")
        self.order_placed.emit(event)
        return order_id

    def cancel_order(self, order_id: str, reason: str) -> None:
        print(f"\nCancelling order: {order_id}")
        self.order_cancelled.emit(order_id, reason)


# --- Slot functions (listeners) ---

def send_confirmation_email(event: OrderEvent) -> None:
    print(f"    [Email] Confirmation sent to {event.customer}")

def update_inventory(event: OrderEvent) -> None:
    print(f"    [Inventory] Reserved {len(event.items)} items")

def log_audit_trail(event: OrderEvent) -> None:
    print(f"    [Audit] Order {event.order_id}: ${event.total:.2f}")

def handle_cancellation(order_id: str, reason: str) -> None:
    print(f"    [Cancel] Processing refund for {order_id}: {reason}")

def notify_warehouse_cancel(order_id: str, reason: str) -> None:
    print(f"    [Warehouse] Stopping fulfillment for {order_id}")


# --- Usage ---
if __name__ == "__main__":
    service = OrderService()
```

```
    # Connect slots to signals
    service.order_placed.connect(send_confirmation_email)
    service.order_placed.connect(update_inventory)
    service.order_placed.connect(log_audit_trail)

    service.order_cancelled.connect(handle_cancellation)
    service.order_cancelled.connect(notify_warehouse_cancel)

    # Place an order
    service.place_order("alice@example.com", 149.99, ["Laptop Stand", "USB Hub"])

    # Cancel it
    service.cancel_order("ORD-0001", "Customer changed mind")
```

Output:

```
  [order_placed] Connected: send_confirmation_email
  [order_placed] Connected: update_inventory
  [order_placed] Connected: log_audit_trail
  [order_cancelled] Connected: handle_cancellation
  [order_cancelled] Connected: notify_warehouse_cancel

Placing order: ORD-4821
  [order_placed] Emitting to 3 slots
    [Email] Confirmation sent to alice@example.com
    [Inventory] Reserved 2 items
    [Audit] Order ORD-4821: $149.99

Cancelling order: ORD-0001
  [order_cancelled] Emitting to 2 slots
    [Cancel] Processing refund for ORD-0001: Customer changed mind
    [Warehouse] Stopping fulfillment for ORD-0001
```

Spring Boot: ApplicationEventPublisher and @EventListener

Spring has the Observer pattern built into the framework.

Step 1: Define Domain Events

```java
// OrderPlacedEvent.java
public class OrderPlacedEvent {
```

```
    private final String orderId;
    private final String customerEmail;
    private final double total;
    private final List<String> items;
    private final Instant timestamp;

    public OrderPlacedEvent(String orderId, String customerEmail,
                            double total, List<String> items) {
        this.orderId = orderId;
        this.customerEmail = customerEmail;
        this.total = total;
        this.items = items;
        this.timestamp = Instant.now();
    }

    // Getters...
    public String getOrderId() { return orderId; }
    public String getCustomerEmail() { return customerEmail; }
    public double getTotal() { return total; }
    public List<String> getItems() { return items; }
    public Instant getTimestamp() { return timestamp; }
}

// OrderCancelledEvent.java
public class OrderCancelledEvent {
    private final String orderId;
    private final String reason;
    private final Instant timestamp;

    public OrderCancelledEvent(String orderId, String reason) {
        this.orderId = orderId;
        this.reason = reason;
        this.timestamp = Instant.now();
    }

    public String getOrderId() { return orderId; }
    public String getReason() { return reason; }
    public Instant getTimestamp() { return timestamp; }
}
```

Step 2: Publish Events from the Service

```java
// OrderService.java
@Service
public class OrderService {

    private final ApplicationEventPublisher eventPublisher;

    public OrderService(ApplicationEventPublisher eventPublisher) {
        this.eventPublisher = eventPublisher;
```

```
    }

    @Transactional
    public String placeOrder(String email, double total, List<String> items) {
        String orderId = "ORD-" + UUID.randomUUID().toString().substring(0, 8);

        // Save order to database...

        // Publish event -- Spring delivers it to all @EventListener methods
        eventPublisher.publishEvent(
            new OrderPlacedEvent(orderId, email, total, items)
        );

        return orderId;
    }

    @Transactional
    public void cancelOrder(String orderId, String reason) {
        // Update order status in database...

        eventPublisher.publishEvent(
            new OrderCancelledEvent(orderId, reason)
        );
    }
}
```

Step 3: Listen for Events with @EventListener

```java
// EmailEventListener.java
@Component
public class EmailEventListener {

    @EventListener
    public void handleOrderPlaced(OrderPlacedEvent event) {
        System.out.println("[Email] Sending confirmation to "
            + event.getCustomerEmail()
            + " for order " + event.getOrderId());
        // emailService.sendOrderConfirmation(event);
    }

    @EventListener
    public void handleOrderCancelled(OrderCancelledEvent event) {
        System.out.println("[Email] Sending cancellation notice for "
            + event.getOrderId());
    }
}

// InventoryEventListener.java
@Component
public class InventoryEventListener {
```

```
    @EventListener
    public void handleOrderPlaced(OrderPlacedEvent event) {
        System.out.println("[Inventory] Reserving "
            + event.getItems().size() + " items");
        // inventoryRepository.reserveItems(event.getItems());
    }

    @EventListener
    public void handleOrderCancelled(OrderCancelledEvent event) {
        System.out.println("[Inventory] Releasing stock for "
            + event.getOrderId());
    }
}

// AuditEventListener.java
@Component
public class AuditEventListener {

    @EventListener
    public void handleOrderPlaced(OrderPlacedEvent event) {
        System.out.println("[Audit] Order placed: " + event.getOrderId()
            + " at " + event.getTimestamp());
    }

    @EventListener
    public void handleOrderCancelled(OrderCancelledEvent event) {
        System.out.println("[Audit] Order cancelled: " + event.getOrderId()
            + " reason: " + event.getReason());
    }
}

// Async listener for non-critical work
@Component
public class AnalyticsEventListener {

    @Async
    @EventListener
    public void handleOrderPlaced(OrderPlacedEvent event) {
        System.out.println("[Analytics] Tracking purchase: $" + event.getTotal());
        // analyticsClient.trackEvent("purchase", event);
    }
}
```

```
Spring Event Flow:

  OrderService
      |
      | publishEvent(OrderPlacedEvent)
      v
  ApplicationEventPublisher (Spring)
      |
      | Spring finds all @EventListener methods
```

```
      | that accept OrderPlacedEvent
      |
      +---> EmailEventListener.handleOrderPlaced()
      |         (synchronous -- runs in same thread)
      |
      +---> InventoryEventListener.handleOrderPlaced()
      |         (synchronous)
      |
      +---> AuditEventListener.handleOrderPlaced()
      |         (synchronous)
      |
      +---> AnalyticsEventListener.handleOrderPlaced()
                (@Async -- runs in separate thread)
```

Real-World Use Case: Cache Invalidation

```java
// CacheInvalidationListener.java
@Component
public class CacheInvalidationListener {

    private final CacheManager cacheManager;

    public CacheInvalidationListener(CacheManager cacheManager) {
        this.cacheManager = cacheManager;
    }

    @EventListener
    public void onProductUpdated(ProductUpdatedEvent event) {
        Cache productCache = cacheManager.getCache("products");
        if (productCache != null) {
            productCache.evict(event.getProductId());
            System.out.println("[Cache] Evicted product: " + event.getProductId());
        }
    }

    @EventListener
    public void onOrderPlaced(OrderPlacedEvent event) {
        // Inventory changed, invalidate related caches
        Cache inventoryCache = cacheManager.getCache("inventory");
        if (inventoryCache != null) {
            for (String itemId : event.getItems()) {
                inventoryCache.evict(itemId);
            }
            System.out.println("[Cache] Evicted " + event.getItems().size()
                + " inventory entries");
        }
    }
}
```

Observer vs Pub/Sub

These two patterns are related but differ in important ways.

```
Observer Pattern:
+----------+    direct reference    +----------+
| Subject  | ---------------------> | Observer |
| (knows   |    calls onEvent()     | (known   |
|  its     |                        |  to the  |
| observers|                        |  subject)|
+----------+                        +----------+

  - Subject holds references to observers
  - Tight coupling (same process, same JVM)
  - Synchronous by default
  - Subject knows observer interface


Pub/Sub Pattern:
+----------+                        +----------+
|Publisher  |   message    +------+  |Subscriber|
|(does NOT  | -----------> |Broker|->|(does NOT |
| know      |              |/Bus  |  | know     |
| subscribers)             +------+  | publisher)
+----------+                          +----------+

  - Publisher and subscriber do NOT know each other
  - Fully decoupled (can be different processes/machines)
  - Asynchronous (message broker handles delivery)
  - Communication via topics/channels
```

Comparison Table

Feature	Observer	Pub/Sub
Coupling	Subject knows observers	Fully decoupled via broker
Location	Same process	Can cross processes/networks
Delivery	Synchronous (usually)	Asynchronous (usually)
Reliability	No guarantees	Broker can guarantee delivery

Complexity	Simple	Needs infrastructure (Kafka, RabbitMQ)
Use when	In-process events	Cross-service events

Spring Bridges Both

```java
// In-process Observer (same JVM)
eventPublisher.publishEvent(new OrderPlacedEvent(...));

// Pub/Sub (cross-service, using Spring Cloud Stream + Kafka)
streamBridge.send("orders-out", new OrderPlacedEvent(...));
```

When to Use / When NOT to Use

Use Observer When

Scenario	Why Observer Helps
Multiple reactions to one event	Each listener handles its own concern
Adding reactions without modifying the source	Open/Closed Principle in action
In-process event notification	Simple, no infrastructure needed
Audit logging and tracking	Listeners observe without affecting flow
Cache invalidation on data change	Automatic, decoupled cache management

Do NOT Use Observer When

Scenario	Why Not
Cross-service communication	Use Pub/Sub with a message broker
Order of execution matters	Observer does not guarantee order
Listeners need to modify the event	Events should be immutable

Only one listener will ever exist	Direct method call is simpler
Performance-critical hot path	Observer adds overhead per listener

Common Mistakes

Mistake 1: One Listener Failure Breaks Everything

```java
// BAD -- No error handling
public void publish(T event) {
    for (EventListener<T> listener : listeners) {
        listener.onEvent(event);  // If this throws, remaining listeners are skipped
    }
}

// GOOD -- Isolate listener failures
public void publish(T event) {
    for (EventListener<T> listener : listeners) {
        try {
            listener.onEvent(event);
        } catch (Exception e) {
            log.error("Listener {} failed: {}", listener.getName(), e.getMessage());
        }
    }
}
```

Mistake 2: Modifying Listener List During Notification

```java
// BAD -- ConcurrentModificationException
public void publish(T event) {
    for (EventListener<T> listener : listeners) {
        listener.onEvent(event);  // What if onEvent() calls subscribe()?
    }
}

// GOOD -- Iterate over a copy
public void publish(T event) {
    List<EventListener<T>> snapshot = new ArrayList<>(listeners);
    for (EventListener<T> listener : snapshot) {
        listener.onEvent(event);
    }
}
```

Mistake 3: Memory Leaks from Forgotten Subscriptions

```java
// BAD -- Listener never unsubscribed, holds reference forever
publisher.subscribe(new HeavyResourceListener());
// The publisher now holds a reference, preventing garbage collection

// GOOD -- Use weak references or explicit lifecycle management
public void shutdown() {
    publisher.unsubscribe(this);
}
```

Best Practices

1. Make events immutable. Once published, an event should not change. Use `final` fields and `List.copyOf()`.
2. Handle listener exceptions. One failing listener must not prevent other listeners from receiving the event.
3. Use `@Async` for slow listeners. Email sending should not slow down order placement.
4. Name events in past tense. `OrderPlacedEvent`, not `PlaceOrderEvent`. Events describe something that already happened.
5. Keep events focused. An event should carry only the data its listeners need. Do not dump the entire domain model into an event.
6. Unsubscribe when done. Especially in long-running applications, forgotten subscriptions cause memory leaks.
7. Log event publishing. When debugging, knowing which events fired and which listeners ran is invaluable.

Quick Summary

```
+-----------------------------------------------------------------+
|                    OBSERVER PATTERN SUMMARY                     |
+-----------------------------------------------------------------+
| Intent:     Define a one-to-many dependency so that when one    |
|             object changes state, all dependents are notified.  |
+-----------------------------------------------------------------+
| Problem:    Source must call N services directly                |
```

```
| Solution:   Source publishes events, listeners react           |
+----------------------------------------------------------------+
| Key parts:                                                     |
|   - Subject / Publisher (fires events)                         |
|   - Observer / Listener (reacts to events)                     |
|   - Event object (carries data)                                |
+----------------------------------------------------------------+
| Spring:     ApplicationEventPublisher + @EventListener         |
| Python:     Signal/slot pattern or callback lists              |
+----------------------------------------------------------------+
```

Key Points

- Observer decouples the event source from the event consumers.
- The publisher does not know what listeners do or how many exist.
- Adding a new reaction means adding a new listener, not modifying the publisher.
- Spring's `@EventListener` makes Observer nearly invisible -- just annotate a method.
- Use `@Async` for listeners that do slow work like sending emails or calling APIs.
- Observer is in-process; Pub/Sub extends the concept across services using a message broker.
- Always handle exceptions in listeners to prevent cascading failures.

Practice Questions

1. In the Spring example, what happens if `InventoryEventListener` throws an exception? Does the email still get sent? How would you change this behavior?
2. Why should events be immutable? What could go wrong if a listener modifies the event object before the next listener receives it?
3. You need to guarantee that the email confirmation is sent after the inventory is reserved. Can Observer guarantee this ordering? What alternatives exist?
4. How does `@Async` on an `@EventListener` change the transactional behavior? If the order transaction rolls back, does the async listener still run?
5. When would you choose Pub/Sub over Observer? Give two specific backend scenarios where Observer is insufficient.

Exercises

Exercise 1: User Registration Events

Build a user registration system where `UserService.register()` publishes a `UserRegisteredEvent`. Create three listeners: one sends a welcome email, one creates a default profile, and one logs the registration for analytics. Verify that adding a fourth listener (free trial activation) requires zero changes to `UserService`.

Exercise 2: Stock Price Monitor

Create a `StockTicker` that publishes price updates. Implement observers for `PriceAlertObserver` (triggers when price crosses a threshold), `MovingAverageObserver` (calculates running average), and `DashboardObserver` (prints current price). Allow observers to subscribe and unsubscribe dynamically.

Exercise 3: Event Replay System

Extend the Observer pattern to store all published events in a list. Add a `replay()` method that re-publishes all stored events to any newly subscribed listener. This is how event sourcing systems allow new services to "catch up" on history.

What Is Next?

Observer lets you broadcast events to multiple listeners. But what if you need to treat a request itself as an object -- something you can queue, undo, log, or replay? In the next chapter, we explore the Command pattern, which wraps requests into objects, enabling undo/redo operations, task queues, and transaction logging.

Chapter 16: Command Pattern -- Encapsulate Requests as Objects

What You Will Learn

- How to turn method calls into objects that can be stored, queued, and undone
- Implementing the Command pattern in Java with `execute()` and `undo()`
- Building a Python command queue for batch processing
- Real-world applications: task queues, transaction logging, macro recording
- An introduction to CQRS (Command Query Responsibility Segregation)
- When Command adds value and when it adds unnecessary complexity

Why This Chapter Matters

In backend systems, you often need to do more than just execute a request. You might need to undo it, replay it, schedule it for later, log it for auditing, or queue it for batch processing. A plain method call cannot do any of these things -- once it runs, it is gone.

The Command pattern solves this by wrapping each request in an object. That object can be passed around, stored in a list, serialized to a database, or placed in a queue. It is the pattern behind undo/redo systems, job queues, transaction logs, and even the CQRS architecture used in large-scale backends.

The Problem: Requests That Cannot Be Managed

```java
// Without Command -- you cannot undo, queue, or replay
public class AccountService {

    public void transfer(String from, String to, double amount) {
        debit(from, amount);   // Done. Cannot undo.
        credit(to, amount);    // Done. Cannot replay.
        // No record of what happened.
        // No way to queue this for later.
        // No way to batch multiple operations.
    }
}
```

```
What we WANT to do with requests:

  +---> Execute now
  |
  Request --+---> Execute later (queue)
  |
  +---> Undo after execution
  |
  +---> Log for auditing
  |
  +---> Replay from history
  |
  +---> Batch multiple requests

  A plain method call gives us ONLY "Execute now."
  The Command pattern gives us ALL of these.
```

The Solution: Command Pattern

```
+-----------------------------------------------------------+
|                  COMMAND PATTERN STRUCTURE                |
+-----------------------------------------------------------+

  +----------+         +-------------------+
  |  Client  | creates |     Command       |
  |          |-------->| + execute()       |
  +----------+         | + undo()          |
                        +-------------------+
                            ^         ^
                            |         |
                  +-----------+   +-----------+
                  | Transfer  |   | Deposit   |
                  | Command   |   | Command   |
                  +-----------+   +-----------+
```

```
                        |               |
                        v               v
                  +----------------------------+
                  |        Receiver            |
                  |    (Account, Database)     |
                  |  The object that does the  |
                  |  actual work               |
                  +----------------------------+

  +-------------------+
  |     Invoker       |  Stores and manages commands
  | (CommandQueue,    |  Can execute, undo, replay
  |  TransactionLog)  |
  +------------------+
```

Java Implementation: Command with Execute and Undo

Step 1: Define the Command Interface

```java
// Command.java
public interface Command {
    void execute();
    void undo();
    String describe();
}
```

Step 2: Create the Receiver

```java
// BankAccount.java
public class BankAccount {
    private final String accountId;
    private double balance;

    public BankAccount(String accountId, double initialBalance) {
        this.accountId = accountId;
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
```

```
        balance += amount;
        System.out.println("    [" + accountId + "] Deposited $"
            + amount + " -> Balance: $" + balance);
    }

    public void withdraw(double amount) {
        if (balance < amount) {
            throw new IllegalStateException(
                accountId + ": Insufficient funds. Balance: $"
                + balance + ", Requested: $" + amount);
        }
        balance -= amount;
        System.out.println("    [" + accountId + "] Withdrew $"
            + amount + " -> Balance: $" + balance);
    }

    public double getBalance() {
        return balance;
    }

    public String getAccountId() {
        return accountId;
    }
}
```

Step 3: Implement Concrete Commands

```java
// DepositCommand.java
public class DepositCommand implements Command {
    private final BankAccount account;
    private final double amount;

    public DepositCommand(BankAccount account, double amount) {
        this.account = account;
        this.amount = amount;
    }

    @Override
    public void execute() {
        account.deposit(amount);
    }

    @Override
    public void undo() {
        account.withdraw(amount);  // Reverse: withdraw what was deposited
    }

    @Override
    public String describe() {
        return "Deposit $" + amount + " to " + account.getAccountId();
```

```
    }
}

// WithdrawCommand.java
public class WithdrawCommand implements Command {
    private final BankAccount account;
    private final double amount;

    public WithdrawCommand(BankAccount account, double amount) {
        this.account = account;
        this.amount = amount;
    }

    @Override
    public void execute() {
        account.withdraw(amount);
    }

    @Override
    public void undo() {
        account.deposit(amount);  // Reverse: deposit what was withdrawn
    }

    @Override
    public String describe() {
        return "Withdraw $" + amount + " from " + account.getAccountId();
    }
}

// TransferCommand.java
public class TransferCommand implements Command {
    private final BankAccount from;
    private final BankAccount to;
    private final double amount;

    public TransferCommand(BankAccount from, BankAccount to, double amount) {
        this.from = from;
        this.to = to;
        this.amount = amount;
    }

    @Override
    public void execute() {
        from.withdraw(amount);
        to.deposit(amount);
    }

    @Override
    public void undo() {
        to.withdraw(amount);
        from.deposit(amount);
    }

    @Override
    public String describe() {
```

```
        return "Transfer $" + amount + " from "
            + from.getAccountId() + " to " + to.getAccountId();
    }
}
```

Step 4: Build the Invoker with Undo/Redo

```java
// TransactionManager.java
public class TransactionManager {
    private final Deque<Command> undoStack = new ArrayDeque<>();
    private final Deque<Command> redoStack = new ArrayDeque<>();
    private final List<Command> history = new ArrayList<>();

    public void execute(Command command) {
        System.out.println("  Executing: " + command.describe());
        command.execute();
        undoStack.push(command);
        redoStack.clear();  // New action invalidates redo history
        history.add(command);
    }

    public void undo() {
        if (undoStack.isEmpty()) {
            System.out.println("  Nothing to undo.");
            return;
        }
        Command command = undoStack.pop();
        System.out.println("  Undoing: " + command.describe());
        command.undo();
        redoStack.push(command);
    }

    public void redo() {
        if (redoStack.isEmpty()) {
            System.out.println("  Nothing to redo.");
            return;
        }
        Command command = redoStack.pop();
        System.out.println("  Redoing: " + command.describe());
        command.execute();
        undoStack.push(command);
    }

    public void printHistory() {
        System.out.println("\n  Transaction History:");
        for (int i = 0; i < history.size(); i++) {
            System.out.println("    " + (i + 1) + ". " + history.get(i).describe());
        }
    }
}
```

Step 5: Put It All Together

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        BankAccount checking = new BankAccount("CHK-001", 1000.0);
        BankAccount savings = new BankAccount("SAV-001", 5000.0);

        TransactionManager txManager = new TransactionManager();

        // Execute commands
        txManager.execute(new DepositCommand(checking, 500.0));
        txManager.execute(new WithdrawCommand(checking, 200.0));
        txManager.execute(new TransferCommand(checking, savings, 300.0));

        System.out.println("\nBalances after transactions:");
        System.out.println("  Checking: $" + checking.getBalance());
        System.out.println("  Savings:  $" + savings.getBalance());

        // Undo the transfer
        System.out.println("\nUndoing last transaction:");
        txManager.undo();

        System.out.println("\nBalances after undo:");
        System.out.println("  Checking: $" + checking.getBalance());
        System.out.println("  Savings:  $" + savings.getBalance());

        // Redo it
        System.out.println("\nRedoing:");
        txManager.redo();

        System.out.println("\nBalances after redo:");
        System.out.println("  Checking: $" + checking.getBalance());
        System.out.println("  Savings:  $" + savings.getBalance());

        txManager.printHistory();
    }
}
```

Output:

```
  Executing: Deposit $500.0 to CHK-001
    [CHK-001] Deposited $500.0 -> Balance: $1500.0
  Executing: Withdraw $200.0 from CHK-001
    [CHK-001] Withdrew $200.0 -> Balance: $1300.0
  Executing: Transfer $300.0 from CHK-001 to SAV-001
    [CHK-001] Withdrew $300.0 -> Balance: $1000.0
    [SAV-001] Deposited $300.0 -> Balance: $5300.0

Balances after transactions:
  Checking: $1000.0
  Savings:  $5300.0
```

```
Undoing last transaction:
  Undoing: Transfer $300.0 from CHK-001 to SAV-001
    [SAV-001] Withdrew $300.0 -> Balance: $5000.0
    [CHK-001] Deposited $300.0 -> Balance: $1300.0

Balances after undo:
  Checking: $1300.0
  Savings:  $5000.0

Redoing:
  Redoing: Transfer $300.0 from CHK-001 to SAV-001
    [CHK-001] Withdrew $300.0 -> Balance: $1000.0
    [SAV-001] Deposited $300.0 -> Balance: $5300.0

Balances after redo:
  Checking: $1000.0
  Savings:  $5300.0

  Transaction History:
    1. Deposit $500.0 to CHK-001
    2. Withdraw $200.0 from CHK-001
    3. Transfer $300.0 from CHK-001 to SAV-001
```

Before vs After

```
BEFORE (direct calls):              AFTER (Command objects):
+----------------------------+      +----------------------------+
| service.transfer(a, b, $)  |      | Command cmd = new Transfer |
| // Executed immediately    |      |   Command(a, b, $);       |
| // Cannot undo             |      |                            |
| // No history              |      | manager.execute(cmd);      |
| // Cannot queue            |      | // Can undo: manager.undo()|
| // Cannot replay           |      | // Has history             |
+----------------------------+      | // Can queue for later     |
                                    | // Can replay from log     |
                                    +----------------------------+
```

Python Implementation: Command Queue

```python
# command_queue.py
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime
from collections import deque
from typing import Optional
import json


class Command(ABC):
    """Base command with execute, undo, and serialization."""

    @abstractmethod
    def execute(self) -> bool:
        """Execute the command. Returns True on success."""
        pass

    @abstractmethod
    def undo(self) -> bool:
        """Reverse the command. Returns True on success."""
        pass

    @abstractmethod
    def describe(self) -> str:
        pass


@dataclass
class Task:
    name: str
    priority: int = 0
    completed: bool = False


class TaskManager:
    """Receiver -- the object commands act upon."""

    def __init__(self):
        self.tasks: dict[str, Task] = {}

    def add_task(self, name: str, priority: int = 0) -> None:
        self.tasks[name] = Task(name=name, priority=priority)
        print(f"    [TaskManager] Added: '{name}' (priority={priority})")

    def remove_task(self, name: str) -> Optional[Task]:
        task = self.tasks.pop(name, None)
        if task:
            print(f"    [TaskManager] Removed: '{name}'")
        return task

    def complete_task(self, name: str) -> bool:
        task = self.tasks.get(name)
        if task:
            task.completed = True
```

```
            print(f"    [TaskManager] Completed: '{name}'")
            return True
        return False

    def uncomplete_task(self, name: str) -> bool:
        task = self.tasks.get(name)
        if task:
            task.completed = False
            print(f"    [TaskManager] Uncompleted: '{name}'")
            return True
        return False

    def list_tasks(self) -> None:
        print("    Current tasks:")
        for name, task in sorted(self.tasks.items(),
                                 key=lambda x: -x[1].priority):
            status = "DONE" if task.completed else "TODO"
            print(f"      [{status}] {name} (priority={task.priority})")


class AddTaskCommand(Command):
    def __init__(self, manager: TaskManager, name: str, priority: int = 0):
        self.manager = manager
        self.name = name
        self.priority = priority

    def execute(self) -> bool:
        self.manager.add_task(self.name, self.priority)
        return True

    def undo(self) -> bool:
        self.manager.remove_task(self.name)
        return True

    def describe(self) -> str:
        return f"Add task '{self.name}'"


class CompleteTaskCommand(Command):
    def __init__(self, manager: TaskManager, name: str):
        self.manager = manager
        self.name = name

    def execute(self) -> bool:
        return self.manager.complete_task(self.name)

    def undo(self) -> bool:
        return self.manager.uncomplete_task(self.name)

    def describe(self) -> str:
        return f"Complete task '{self.name}'"


class RemoveTaskCommand(Command):
    def __init__(self, manager: TaskManager, name: str):
```

```
        self.manager = manager
        self.name = name
        self._removed_task: Optional[Task] = None

    def execute(self) -> bool:
        self._removed_task = self.manager.remove_task(self.name)
        return self._removed_task is not None

    def undo(self) -> bool:
        if self._removed_task:
            self.manager.add_task(
                self._removed_task.name,
                self._removed_task.priority
            )
            return True
        return False

    def describe(self) -> str:
        return f"Remove task '{self.name}'"


class MacroCommand(Command):
    """A command that groups multiple commands into one."""

    def __init__(self, name: str, commands: list[Command]):
        self._name = name
        self._commands = commands

    def execute(self) -> bool:
        print(f"  [Macro] Executing '{self._name}' "
              f"({len(self._commands)} commands)")
        for cmd in self._commands:
            if not cmd.execute():
                return False
        return True

    def undo(self) -> bool:
        print(f"  [Macro] Undoing '{self._name}'")
        for cmd in reversed(self._commands):
            cmd.undo()
        return True

    def describe(self) -> str:
        return f"Macro '{self._name}' ({len(self._commands)} commands)"


class CommandQueue:
    """Invoker -- queues commands and manages undo history."""

    def __init__(self):
        self._queue: deque[Command] = deque()
        self._history: list[Command] = []
        self._undo_stack: list[Command] = []

    def enqueue(self, command: Command) -> None:
```

```
        self._queue.append(command)
        print(f"  Queued: {command.describe()}")

    def process_all(self) -> None:
        print(f"\n  Processing {len(self._queue)} queued commands:")
        while self._queue:
            command = self._queue.popleft()
            print(f"  Executing: {command.describe()}")
            if command.execute():
                self._history.append(command)
                self._undo_stack.append(command)

    def undo_last(self) -> None:
        if not self._undo_stack:
            print("  Nothing to undo.")
            return
        command = self._undo_stack.pop()
        print(f"  Undoing: {command.describe()}")
        command.undo()

    def print_history(self) -> None:
        print("\n  Command History:")
        for i, cmd in enumerate(self._history, 1):
            print(f"    {i}. {cmd.describe()}")


# --- Usage ---
if __name__ == "__main__":
    manager = TaskManager()
    queue = CommandQueue()

    # Queue up commands (not executed yet)
    queue.enqueue(AddTaskCommand(manager, "Design API", priority=3))
    queue.enqueue(AddTaskCommand(manager, "Write tests", priority=2))
    queue.enqueue(AddTaskCommand(manager, "Deploy to staging", priority=1))
    queue.enqueue(CompleteTaskCommand(manager, "Design API"))

    # Process all queued commands at once
    queue.process_all()
    manager.list_tasks()

    # Undo the last command
    print("\n  --- Undo last command ---")
    queue.undo_last()
    manager.list_tasks()

    # Macro command
    print("\n  --- Macro: Sprint Setup ---")
    macro = MacroCommand("Sprint Setup", [
        AddTaskCommand(manager, "Sprint planning", priority=3),
        AddTaskCommand(manager, "Daily standup setup", priority=2),
        AddTaskCommand(manager, "Create board", priority=1),
    ])
    queue.enqueue(macro)
    queue.process_all()
```

```
    manager.list_tasks()

    queue.print_history()
```

Output:

```
Queued: Add task 'Design API'
Queued: Add task 'Write tests'
Queued: Add task 'Deploy to staging'
Queued: Complete task 'Design API'

Processing 4 queued commands:
Executing: Add task 'Design API'
  [TaskManager] Added: 'Design API' (priority=3)
Executing: Add task 'Write tests'
  [TaskManager] Added: 'Write tests' (priority=2)
Executing: Add task 'Deploy to staging'
  [TaskManager] Added: 'Deploy to staging' (priority=1)
Executing: Complete task 'Design API'
  [TaskManager] Completed: 'Design API'
  Current tasks:
    [DONE] Design API (priority=3)
    [TODO] Write tests (priority=2)
    [TODO] Deploy to staging (priority=1)

--- Undo last command ---
Undoing: Complete task 'Design API'
  [TaskManager] Uncompleted: 'Design API'
  Current tasks:
    [TODO] Design API (priority=3)
    [TODO] Write tests (priority=2)
    [TODO] Deploy to staging (priority=1)

--- Macro: Sprint Setup ---
Queued: Macro 'Sprint Setup' (3 commands)

Processing 1 queued commands:
Executing: Macro 'Sprint Setup' (3 commands)
[Macro] Executing 'Sprint Setup' (3 commands)
  [TaskManager] Added: 'Sprint planning' (priority=3)
  [TaskManager] Added: 'Daily standup setup' (priority=2)
  [TaskManager] Added: 'Create board' (priority=1)
  Current tasks:
    [TODO] Design API (priority=3)
    [TODO] Sprint planning (priority=3)
    [TODO] Daily standup setup (priority=2)
    [TODO] Write tests (priority=2)
    [TODO] Create board (priority=1)
    [TODO] Deploy to staging (priority=1)

Command History:
  1. Add task 'Design API'
  2. Add task 'Write tests'
```

```
    3. Add task 'Deploy to staging'
    4. Complete task 'Design API'
    5. Macro 'Sprint Setup' (3 commands)
```

Real-World Use Case: Transaction Logging

```java
// TransactionLog.java -- Persist commands for replay
public class TransactionLog {
    private final List<String> entries = new ArrayList<>();

    public void log(Command command, String status) {
        String entry = String.format("[%s] %s: %s",
            LocalDateTime.now().format(DateTimeFormatter.ISO_LOCAL_DATE_TIME),
            status, command.describe());
        entries.add(entry);
    }

    public void printLog() {
        System.out.println("\nTransaction Log:");
        entries.forEach(e -> System.out.println("  " + e));
    }

    // In a real system, you would serialize commands to a database
    // and replay them to reconstruct state (Event Sourcing)
}

// AuditedTransactionManager.java
public class AuditedTransactionManager {
    private final TransactionManager txManager = new TransactionManager();
    private final TransactionLog log = new TransactionLog();

    public void execute(Command command) {
        try {
            txManager.execute(command);
            log.log(command, "EXECUTED");
        } catch (Exception e) {
            log.log(command, "FAILED: " + e.getMessage());
            throw e;
        }
    }

    public void undo() {
        // The last executed command is logged as undone
        txManager.undo();
    }
```

```
    public TransactionLog getLog() {
        return log;
    }
}
```

Real-World Use Case: CQRS Introduction

CQRS separates read operations from write operations. Commands (writes) and Queries (reads) use different models.

```
Traditional CRUD:                    CQRS:
+--------------------+               +----------+     +-----------+
|   Single Model     |               | Command |     |  Query    |
|                    |               | Model   |     |  Model    |
| Create             |               | (Write) |     | (Read)    |
| Read               |               +----------+     +-----------+
| Update             |                    |               ^
| Delete             |                    v               |
|                    |               +-----------+    +-----------+
+--------------------+               | Write DB |    | Read DB  |
        |                            +-----------+    +-----------+
        v                                 |               ^
  +-----------+                           +--- sync ------+
  | Database |
  +-----------+

CQRS uses the Command pattern at its core:
- Each write operation is a Command object
- Commands are validated, executed, and stored
- The read side is optimized separately
```

```java
// CQRS-style command
public class CreateOrderCommand {
    private final String customerId;
    private final List<OrderItem> items;
    private final String shippingAddress;

    // Constructor, getters...
}

// Command Handler
@Service
public class CreateOrderCommandHandler {
```

```
    private final OrderRepository orderRepository;
    private final ApplicationEventPublisher eventPublisher;

    public OrderId handle(CreateOrderCommand command) {
        // Validate
        validateCustomer(command.getCustomerId());
        validateItems(command.getItems());

        // Execute
        Order order = Order.create(
            command.getCustomerId(),
            command.getItems(),
            command.getShippingAddress()
        );
        orderRepository.save(order);

        // Publish event for the read side to update
        eventPublisher.publishEvent(new OrderCreatedEvent(order));

        return order.getId();
    }
}

// Query side -- separate, optimized read model
@Service
public class OrderQueryService {

    private final OrderReadRepository readRepository;

    public OrderSummaryDTO getOrderSummary(String orderId) {
        // Reads from a denormalized, read-optimized store
        return readRepository.findSummaryById(orderId);
    }
}
```

When to Use / When NOT to Use

Use Command When

Scenario	Why Command Helps
Undo/redo functionality	Commands store state needed to reverse
Task/job queues	Commands can be serialized and queued
Transaction logging	Commands describe exactly what happened

Macro recording	Group commands into composite macros
Deferred execution	Create now, execute later
Audit trails	Every action is a named, logged object

Do NOT Use Command When

Scenario	Why Not
Simple one-shot operations	Direct method calls are simpler
No need for undo or queuing	The overhead is not justified
Operations that cannot be undone	Delete-from-external-API has no undo
Very high throughput paths	Object creation overhead may matter

---

Common Mistakes

Mistake 1: Commands That Cannot Be Undone

```java
// BAD -- No way to undo
public class SendEmailCommand implements Command {
    @Override
    public void execute() {
        emailService.send(email);  // Sent. Cannot unsend.
    }

    @Override
    public void undo() {
        // What do we do here? Nothing useful.
        throw new UnsupportedOperationException("Cannot unsend email");
    }
}

// BETTER -- Be honest about it
public class SendEmailCommand implements Command {
    private boolean executed = false;

    @Override
    public void undo() {
```

```
        if (executed) {
            // Log that a compensating action is needed
            auditLog.warn("Email sent to " + recipient
                + " -- manual follow-up required for reversal");
        }
    }
}
```

Mistake 2: Mutable Commands

```java
// BAD -- Command state can be corrupted
public class DepositCommand implements Command {
    public double amount;  // Public and mutable!

    @Override
    public void execute() {
        account.deposit(amount);
    }
}

// GOOD -- Immutable commands
public class DepositCommand implements Command {
    private final BankAccount account;
    private final double amount;  // Private and final

    public DepositCommand(BankAccount account, double amount) {
        this.account = account;
        this.amount = amount;
    }
}
```

Mistake 3: Commands That Do Too Much

```java
// BAD -- One command does everything
public class ProcessOrderCommand implements Command {
    @Override
    public void execute() {
        validateOrder();
        chargePayment();
        updateInventory();
        sendConfirmation();
        notifyWarehouse();
    }
}

// GOOD -- Compose small commands
```

```
MacroCommand processOrder = new MacroCommand("ProcessOrder",
    List.of(
        new ValidateOrderCommand(order),
        new ChargePaymentCommand(order),
        new UpdateInventoryCommand(order),
        new SendConfirmationCommand(order)
    )
);
```

Best Practices

1. Keep commands small and focused. One command, one action. Use `MacroCommand` to compose them.
2. Make commands immutable. Set all fields in the constructor and make them `final`.
3. Store undo state at execution time. The command should capture whatever it needs to reverse itself when `execute()` runs.
4. Handle partial failures in macros. If the third command in a macro fails, undo the first two.
5. Separate command creation from execution. The client creates commands; the invoker decides when to run them.
6. Log commands for auditing. Every command is a clear record of what action was requested and by whom.
7. Consider serialization. If commands need to survive restarts (job queues), they must be serializable.

Quick Summary

```
+-----------------------------------------------------------------+
|                     COMMAND PATTERN SUMMARY                      |
+-----------------------------------------------------------------+
| Intent:     Encapsulate a request as an object, allowing you    |
|             to parameterize, queue, log, and undo operations.   |
+-----------------------------------------------------------------+
| Problem:    Method calls cannot be undone, queued, or replayed |
| Solution:   Wrap each action in a Command object                |
+-----------------------------------------------------------------+
| Key parts:                                                      |
```

```
|   - Command interface (execute + undo)                        |
|   - Concrete commands (each wraps one action)                 |
|   - Receiver (the object being acted upon)                    |
|   - Invoker (manages command lifecycle)                       |
+---------------------------------------------------------------+
| Enables:    Undo/redo, queuing, logging, macros, CQRS         |
+---------------------------------------------------------------+
```

Key Points

- The Command pattern turns requests into objects with `execute()` and `undo()` methods.
- Commands can be stored in stacks (undo/redo), queues (task processing), or logs (audit trails).
- `MacroCommand` groups multiple commands into a single composite operation.
- CQRS uses commands to separate the write path from the read path.
- Commands should be immutable -- store all parameters at construction time.
- Not every operation can be undone. Be honest about it in your `undo()` implementation.

Practice Questions

1. In the bank account example, what happens if the `TransferCommand.execute()` succeeds on `withdraw` but fails on `deposit`? How should the command handle this partial failure?
2. How would you serialize a `Command` to store it in a database for later replay? What challenges arise with commands that reference in-memory objects?
3. A macro command has five sub-commands. The fourth one fails. Should you undo the first three? Always? What factors influence this decision?
4. How does the Command pattern differ from simply using a `Runnable` or a lambda in Java? What does Command add that `Runnable` does not?
5. In CQRS, why is the read model eventually consistent? What problems can this cause, and how do real systems handle them?

Exercises

Exercise 1: Text Editor Undo/Redo

Build a simple text editor that supports `InsertTextCommand`, `DeleteTextCommand`, and `ReplaceTextCommand`. Each command must support `undo()`. Implement a `TextEditorHistory` that lets the user undo and redo multiple steps.

Exercise 2: Job Queue with Priority

Create a command-based job queue where commands have priorities. Implement `HighPriorityCommand`, `NormalCommand`, and `LowPriorityCommand`. The queue should process high-priority commands first. Add the ability to cancel queued (not yet executed) commands.

Exercise 3: Event-Sourced Shopping Cart

Build a shopping cart using event sourcing. Every change (add item, remove item, change quantity) is a command that produces an event. Store all events in a list. Rebuild the cart state by replaying events from the beginning. Verify that replaying produces the same final state.

What Is Next?

Commands let you treat individual requests as objects. But what if you have a multi-step process where the overall algorithm is fixed, but individual steps vary? In the next chapter, we explore the Template Method pattern -- a way to define the skeleton of an algorithm in a base class while letting subclasses fill in the specific steps.

Chapter 17: Template Method Pattern -- Define Algorithm Skeleton, Let Subclasses Fill in Steps

What You Will Learn

- How to define a fixed algorithm structure while allowing steps to vary
- Implementing Template Method in Java with an `AbstractDataProcessor` for ETL
- Building a Python `ReportGenerator` with customizable steps
- Understanding hook methods and when to use them
- Real-world applications: ETL pipelines, document generators, test frameworks
- Template Method vs Strategy: when to use each

Why This Chapter Matters

Many backend processes follow the same overall structure but differ in the details. Every ETL pipeline extracts data, transforms it, and loads it. Every report generator fetches data, formats it, and outputs it. Every test runs setup, the test itself, and teardown. The structure is identical -- only the individual steps change.

If you copy-paste the structure and modify the details each time, you end up with duplicated code that drifts apart over time. The Template Method pattern solves this by putting the algorithm skeleton in a base class and letting subclasses fill in the specific steps. The structure stays consistent, and new variants only need to implement what is different.

The Problem: Duplicated Algorithm Structure

```java
// CSVDataProcessor.java
public class CSVDataProcessor {
    public void process() {
        System.out.println("Opening CSV file...");
        System.out.println("Reading CSV rows...");        // Step 1: Extract
        System.out.println("Converting strings to ints..."); // Step 2: Transform
        System.out.println("Validating data...");           // Shared step
        System.out.println("Inserting into MySQL...");      // Step 3: Load
        System.out.println("Closing connections...");       // Shared step
    }
}

// JSONDataProcessor.java
public class JSONDataProcessor {
    public void process() {
        System.out.println("Opening JSON file...");
        System.out.println("Parsing JSON objects...");     // Step 1: Extract
        System.out.println("Flattening nested objects..."); // Step 2: Transform
        System.out.println("Validating data...");           // Shared step (DUPLICATED)
        System.out.println("Inserting into MySQL...");      // Step 3: Load (DUPLICATED)
        System.out.println("Closing connections...");       // Shared step (DUPLICATED)
    }
}

// The shared steps are copy-pasted. If validation logic changes,
// you must update EVERY processor. This does not scale.
```

```
The Duplication Problem:

  CSVProcessor       JSONProcessor      XMLProcessor
  +------------+     +------------+     +------------+
  | extract()  |     | extract()  |     | extract()  |
  | transform()|     | transform()|     | transform()|
  | validate() |     | validate() |     | validate() |  <-- Same in all three
  | load()     |     | load()     |     | load()     |  <-- Same in all three
  | cleanup()  |     | cleanup()  |     | cleanup()  |  <-- Same in all three
  +------------+     +------------+     +------------+

  3 classes x 3 duplicated methods = maintenance nightmare
```

The Solution: Template Method Pattern

```
Template Method Structure:

  +-----------------------------------------+
```

```
|       AbstractDataProcessor              |
|                                          |
|  process()  <-- TEMPLATE METHOD (final)  |
|  {                                       |
|      extract();     // abstract          |
|      transform();   // abstract          |
|      validate();    // concrete (shared)  |
|      load();        // abstract          |
|      cleanup();     // concrete (shared)  |
|  }                                       |
|                                          |
|  abstract extract();                     |
|  abstract transform();                   |
|  abstract load();                        |
|  validate() { ... }  // shared logic     |
|  cleanup()  { ... }  // shared logic     |
+------------------------------------------+
         ^                ^
         |                |
+----------------+ +----------------+
| CSVProcessor   | | JSONProcessor  |
|                | |                |
| extract() {...}| | extract() {...}|
| transform(){..}| | transform(){..}|
| load() {...}   | | load() {...}   |
+----------------+ +----------------+

Subclasses ONLY implement what differs.
The algorithm structure is defined ONCE.
```

Java Implementation: AbstractDataProcessor (ETL)

Step 1: Define the Abstract Base Class

```java
// AbstractDataProcessor.java
public abstract class AbstractDataProcessor {

    // The TEMPLATE METHOD -- defines the algorithm skeleton
    // 'final' prevents subclasses from changing the structure
    public final void process() {
        System.out.println("=== Starting " + getProcessorName() + " ===");

        List<Map<String, Object>> rawData = extract();
```

```
        System.out.println("  Extracted " + rawData.size() + " records");

        List<Map<String, Object>> transformedData = transform(rawData);
        System.out.println("  Transformed " + transformedData.size() + " records");

        // Hook method -- subclasses CAN override, but do not have to
        if (shouldValidate()) {
            validate(transformedData);
            System.out.println("  Validation passed");
        }

        int loaded = load(transformedData);
        System.out.println("  Loaded " + loaded + " records");

        cleanup();
        System.out.println("=== Completed " + getProcessorName() + " ===\n");
    }

    // Abstract methods -- subclasses MUST implement these
    protected abstract List<Map<String, Object>> extract();
    protected abstract List<Map<String, Object>> transform(
        List<Map<String, Object>> data);
    protected abstract int load(List<Map<String, Object>> data);
    protected abstract String getProcessorName();

    // Hook method -- default behavior that subclasses CAN override
    protected boolean shouldValidate() {
        return true;  // Validate by default
    }

    // Concrete methods -- shared logic, same for all subclasses
    protected void validate(List<Map<String, Object>> data) {
        for (Map<String, Object> record : data) {
            if (record.isEmpty()) {
                throw new IllegalStateException("Empty record found");
            }
        }
    }

    protected void cleanup() {
        System.out.println("  Releasing resources...");
    }
}
```

Step 2: Implement Concrete Processors

```java
// CSVDataProcessor.java
public class CSVDataProcessor extends AbstractDataProcessor {

    @Override
```

```
    protected List<Map<String, Object>> extract() {
        System.out.println("  Reading CSV file: data.csv");
        // Simulate CSV parsing
        return List.of(
            Map.of("name", "Alice", "age", "30", "city", "NYC"),
            Map.of("name", "Bob", "age", "25", "city", "LA"),
            Map.of("name", "Charlie", "age", "35", "city", "Chicago")
        );
    }

    @Override
    protected List<Map<String, Object>> transform(
            List<Map<String, Object>> data) {
        System.out.println("  Converting string ages to integers");
        List<Map<String, Object>> result = new ArrayList<>();
        for (Map<String, Object> record : data) {
            Map<String, Object> transformed = new HashMap<>(record);
            transformed.put("age", Integer.parseInt((String) record.get("age")));
            result.add(transformed);
        }
        return result;
    }

    @Override
    protected int load(List<Map<String, Object>> data) {
        System.out.println("  Inserting into 'users' table");
        // Simulate database insert
        return data.size();
    }

    @Override
    protected String getProcessorName() {
        return "CSV Data Processor";
    }
}

// JSONDataProcessor.java
public class JSONDataProcessor extends AbstractDataProcessor {

    @Override
    protected List<Map<String, Object>> extract() {
        System.out.println("  Parsing JSON file: data.json");
        return List.of(
            Map.of("product", "Laptop", "price", 999.99,
                   "specs", Map.of("ram", "16GB", "storage", "512GB")),
            Map.of("product", "Phone", "price", 699.99,
                   "specs", Map.of("ram", "8GB", "storage", "256GB"))
        );
    }

    @Override
    protected List<Map<String, Object>> transform(
            List<Map<String, Object>> data) {
        System.out.println("  Flattening nested JSON objects");
        List<Map<String, Object>> result = new ArrayList<>();
```

```
        for (Map<String, Object> record : data) {
            Map<String, Object> flat = new HashMap<>();
            flat.put("product", record.get("product"));
            flat.put("price", record.get("price"));
            @SuppressWarnings("unchecked")
            Map<String, String> specs = (Map<String, String>) record.get("specs");
            flat.put("ram", specs.get("ram"));
            flat.put("storage", specs.get("storage"));
            result.add(flat);
        }
        return result;
    }

    @Override
    protected int load(List<Map<String, Object>> data) {
        System.out.println("  Inserting into 'products' table");
        return data.size();
    }

    @Override
    protected String getProcessorName() {
        return "JSON Data Processor";
    }

    // Override hook -- skip validation for trusted JSON sources
    @Override
    protected boolean shouldValidate() {
        return false;
    }
}
```

Step 3: Run the Processors

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        AbstractDataProcessor csvProcessor = new CSVDataProcessor();
        csvProcessor.process();

        AbstractDataProcessor jsonProcessor = new JSONDataProcessor();
        jsonProcessor.process();
    }
}
```

Output:

```
=== Starting CSV Data Processor ===
  Reading CSV file: data.csv
  Extracted 3 records
```

```
  Converting string ages to integers
  Transformed 3 records
  Validation passed
  Inserting into 'users' table
  Loaded 3 records
  Releasing resources...
=== Completed CSV Data Processor ===

=== Starting JSON Data Processor ===
  Parsing JSON file: data.json
  Extracted 2 records
  Flattening nested JSON objects
  Transformed 2 records
  Inserting into 'products' table
  Loaded 2 records
  Releasing resources...
=== Completed JSON Data Processor ===
```

Notice how the JSON processor skipped validation because it overrode the `shouldValidate()` hook.

Hook Methods Explained

```
Types of Methods in Template Method:

  +-------------------------------------------------------------+
  | Abstract Methods (MUST override)                            |
  |   extract(), transform(), load()                            |
  |   Subclass MUST provide implementation.                     |
  +-------------------------------------------------------------+
  | Hook Methods (CAN override)                                 |
  |   shouldValidate(), beforeLoad(), afterExtract()            |
  |   Have a default implementation. Override only if needed.   |
  +-------------------------------------------------------------+
  | Concrete Methods (SHOULD NOT override)                      |
  |   validate(), cleanup()                                     |
  |   Shared logic. Same for all subclasses.                    |
  +-------------------------------------------------------------+
  | Template Method (CANNOT override)                           |
  |   process()  -- marked 'final'                              |
  |   Defines the algorithm structure. Never changed.           |
  +-------------------------------------------------------------+

  Hook methods give subclasses OPTIONAL control points
  without forcing them to implement anything.
```

Python Implementation: ReportGenerator

```python
# report_generator.py
from abc import ABC, abstractmethod
from datetime import datetime
from typing import Any


class ReportGenerator(ABC):
    """Template Method pattern for generating reports."""

    def generate(self) -> str:
        """The template method -- defines the report generation algorithm."""
        print(f"=== Generating {self.get_report_name()} ===")

        # Step 1: Fetch data
        data = self.fetch_data()
        print(f"  Fetched {len(data)} records")

        # Step 2: Hook -- filter data (optional)
        data = self.filter_data(data)
        print(f"  After filtering: {len(data)} records")

        # Step 3: Format the report
        formatted = self.format_report(data)

        # Step 4: Hook -- add header/footer (optional)
        header = self.get_header()
        footer = self.get_footer()
        report = f"{header}\n{formatted}\n{footer}"

        # Step 5: Output the report
        output_path = self.output_report(report)
        print(f"  Report saved to: {output_path}")

        print(f"=== Completed {self.get_report_name()} ===\n")
        return report

    # Abstract methods -- MUST implement
    @abstractmethod
    def fetch_data(self) -> list[dict[str, Any]]:
        pass

    @abstractmethod
    def format_report(self, data: list[dict[str, Any]]) -> str:
        pass

    @abstractmethod
    def get_report_name(self) -> str:
```

```
        pass

    # Hook methods -- CAN override
    def filter_data(self, data: list[dict[str, Any]]) -> list[dict[str, Any]]:
        """Override to filter data. Default: no filtering."""
        return data

    def get_header(self) -> str:
        return f"--- {self.get_report_name()} ---"

    def get_footer(self) -> str:
        return f"--- Generated: {datetime.now().strftime('%Y-%m-%d %H:%M')} ---"

    def output_report(self, report: str) -> str:
        """Override to change output destination."""
        filename = f"{self.get_report_name().lower().replace(' ', '_')}.txt"
        # In a real system, write to file
        return filename


class SalesReport(ReportGenerator):
    """Generates a plain text sales report."""

    def fetch_data(self) -> list[dict[str, Any]]:
        print("  Querying sales database...")
        return [
            {"product": "Laptop", "quantity": 45, "revenue": 44955.00},
            {"product": "Phone", "quantity": 120, "revenue": 83880.00},
            {"product": "Tablet", "quantity": 30, "revenue": 14970.00},
            {"product": "Watch", "quantity": 200, "revenue": 59800.00},
        ]

    def format_report(self, data: list[dict[str, Any]]) -> str:
        lines = ["Product       | Qty  | Revenue"]
        lines.append("-" * 40)
        total_revenue = 0
        for row in data:
            lines.append(
                f"{row['product']:<14}| {row['quantity']:<5}| "
                f"${row['revenue']:>10,.2f}"
            )
            total_revenue += row['revenue']
        lines.append("-" * 40)
        lines.append(f"{'TOTAL':<14}|      | ${total_revenue:>10,.2f}")
        return "\n".join(lines)

    def get_report_name(self) -> str:
        return "Sales Report"


class TopProductsReport(ReportGenerator):
    """Generates a report of top products only."""

    def __init__(self, min_revenue: float = 50000):
        self.min_revenue = min_revenue
```

```
    def fetch_data(self) -> list[dict[str, Any]]:
        print("  Querying sales database...")
        return [
            {"product": "Laptop", "quantity": 45, "revenue": 44955.00},
            {"product": "Phone", "quantity": 120, "revenue": 83880.00},
            {"product": "Tablet", "quantity": 30, "revenue": 14970.00},
            {"product": "Watch", "quantity": 200, "revenue": 59800.00},
        ]

    # Override hook to filter data
    def filter_data(self, data: list[dict[str, Any]]) -> list[dict[str, Any]]:
        print(f"  Filtering: revenue >= ${self.min_revenue:,.2f}")
        return [row for row in data if row["revenue"] >= self.min_revenue]

    def format_report(self, data: list[dict[str, Any]]) -> str:
        lines = ["TOP PRODUCTS"]
        for i, row in enumerate(data, 1):
            lines.append(
                f"  {i}. {row['product']}: ${row['revenue']:,.2f} "
                f"({row['quantity']} units)"
            )
        return "\n".join(lines)

    def get_report_name(self) -> str:
        return "Top Products Report"

    # Override hook for custom header
    def get_header(self) -> str:
        return f"*** {self.get_report_name()} (min: ${self.min_revenue:,.2f}) ***"


# --- Usage ---
if __name__ == "__main__":
    sales = SalesReport()
    report1 = sales.generate()
    print(report1)

    top = TopProductsReport(min_revenue=50000)
    report2 = top.generate()
    print(report2)
```

Output:

```
=== Generating Sales Report ===
  Querying sales database...
  Fetched 4 records
  After filtering: 4 records
  Report saved to: sales_report.txt
=== Completed Sales Report ===

--- Sales Report ---
Product       | Qty  | Revenue
```

```
----------------------------------------
Laptop        | 45   | $ 44,955.00
Phone         | 120  | $ 83,880.00
Tablet        | 30   | $ 14,970.00
Watch         | 200  | $ 59,800.00
----------------------------------------
TOTAL         |      | $203,605.00
--- Generated: 2024-11-15 10:30 ---

=== Generating Top Products Report ===
  Querying sales database...
  Fetched 4 records
  Filtering: revenue >= $50,000.00
  After filtering: 2 records
  Report saved to: top_products_report.txt
=== Completed Top Products Report ===

*** Top Products Report (min: $50,000.00) ***
TOP PRODUCTS
  1. Phone: $83,880.00 (120 units)
  2. Watch: $59,800.00 (200 units)
--- Generated: 2024-11-15 10:30 ---
```

Template Method vs Strategy

These two patterns are often confused. Here is when to use each.

```
Template Method:                    Strategy:
+----------------------------+      +----------------------------+
| Base class defines         |      | Interface defines          |
| the OVERALL algorithm.     |      | ONE interchangeable step.  |
|                            |      |                            |
| Subclasses fill in steps.  |      | Implementations provide    |
|                            |      | different algorithms.      |
| Uses INHERITANCE.          |      | Uses COMPOSITION.          |
+----------------------------+      +----------------------------+

  Template Method:                  Strategy:

  AbstractProcessor                 Processor
  |                                 |
  +-- process() {                   +-- process(strategy) {
  |     extract();   // abstract    |     strategy.execute();
  |     transform(); // abstract    |   }
  |     load();      // abstract    |
  |   }                             Strategy strategy
  |                                      ^       ^
  CSVProcessor   JSONProcessor      QuickSort  BubbleSort
```

Aspect	Template Method	Strategy
Mechanism	Inheritance	Composition
What varies	Steps within a fixed algorithm	The entire algorithm
When chosen	At compile time (class hierarchy)	At runtime (inject/swap)
Flexibility	Less (locked to class hierarchy)	More (swap any time)
Use when	Structure is fixed, steps vary	Algorithm changes at runtime

When to Use / When NOT to Use

Use Template Method When

Scenario	Why Template Method Helps
Multiple classes share the same algorithm structure	Define it once in the base class
Steps differ but order is fixed	Base class controls the order
You want to enforce a workflow	`final` template method prevents changes
Hook methods add optional customization	Subclasses choose what to customize

Do NOT Use Template Method When

Scenario	Why Not
Algorithm structure changes at runtime	Use Strategy instead
Too many abstract methods	Subclasses become painful to implement
Only one step varies	A simple callback or Strategy is lighter
Deep inheritance hierarchies	Leads to fragile base class problem

Common Mistakes

Mistake 1: Template Method Not Marked Final

```java
// BAD -- Subclass can override the template method itself
public class AbstractProcessor {
    public void process() {  // NOT final -- can be overridden
        extract();
        transform();
        load();
    }
}

// Subclass breaks the algorithm
public class BadProcessor extends AbstractProcessor {
    @Override
    public void process() {
        load();  // Oops -- loaded before extracting
    }
}

// GOOD -- Template method is final
public abstract class AbstractProcessor {
    public final void process() {  // Cannot be overridden
        extract();
        transform();
        load();
    }
}
```

Mistake 2: Too Many Abstract Methods

```java
// BAD -- 8 abstract methods, painful to implement
public abstract class AbstractProcessor {
    public final void process() {
        step1(); step2(); step3(); step4();
        step5(); step6(); step7(); step8();
    }
    protected abstract void step1();
    protected abstract void step2();
    // ... Subclasses must implement ALL 8 methods
}

// GOOD -- Few abstract methods, hooks for optional customization
public abstract class AbstractProcessor {
    public final void process() {
        beforeProcess();  // hook (optional)
```

```
        extract();        // abstract (required)
        transform();      // abstract (required)
        afterTransform(); // hook (optional)
        load();           // abstract (required)
        afterProcess();   // hook (optional)
    }
    // Only 3 required methods
}
```

Mistake 3: Calling Super Without Understanding Order

```python
# BAD -- Overriding a hook and forgetting it changes behavior
class MyProcessor(AbstractProcessor):
    def filter_data(self, data):
        # Forgot to call super() -- other filtering logic lost
        return [d for d in data if d["active"]]

# GOOD -- Be explicit about whether you extend or replace
class MyProcessor(AbstractProcessor):
    def filter_data(self, data):
        # First apply base filtering, then add our own
        data = super().filter_data(data)
        return [d for d in data if d["active"]]
```

Best Practices

1. Mark the template method `final` (Java) or document it as not-to-be-overridden (Python). The whole point is that the structure does not change.
2. Use hooks sparingly. Each hook is a decision point a subclass might use. Too many hooks make the base class hard to understand.
3. Provide sensible defaults in hooks. Hooks should do something reasonable if not overridden. Do not force subclasses to override hooks.
4. Keep the number of abstract methods small. Three to five is typical. More than that, and subclasses become burdensome.
5. Name abstract methods clearly. `extract()`, `transform()`, `load()` are much better than `step1()`, `step2()`, `step3()`.
6. Prefer Strategy when algorithms change at runtime. Template Method is a compile-time decision; Strategy is a runtime decision.

Quick Summary

```
+-----------------------------------------------------------------+
|                  TEMPLATE METHOD PATTERN SUMMARY                |
+-----------------------------------------------------------------+
| Intent:     Define the skeleton of an algorithm in a base       |
|             class, deferring some steps to subclasses.          |
+-----------------------------------------------------------------+
| Problem:    Same algorithm structure duplicated across classes |
| Solution:   Base class owns the structure, subclasses vary      |
+-----------------------------------------------------------------+
| Key parts:                                                      |
|   - Abstract base class (template method + abstract steps)      |
|   - Concrete subclasses (implement abstract steps)              |
|   - Hook methods (optional override points)                     |
+-----------------------------------------------------------------+
| Structure: Fixed by base class (marked final)                   |
| Steps:     Vary by subclass                                     |
+-----------------------------------------------------------------+
```

Key Points

- Template Method defines an algorithm skeleton in a base class, with abstract steps filled in by subclasses.
- The template method itself should be `final` to prevent subclasses from altering the algorithm structure.
- Hook methods provide optional customization points with sensible defaults.
- This pattern eliminates duplicated algorithm structures across similar classes.
- Template Method uses inheritance; Strategy uses composition. Prefer Strategy when the algorithm needs to change at runtime.
- Keep the number of abstract methods small to avoid burdening subclasses.

Practice Questions

1. In the ETL example, the JSON processor skips validation by overriding `shouldValidate()`. What are the risks of letting subclasses skip safety checks? How would you make validation mandatory for certain data sources?

2. You have a test framework where every test runs `setup()`, `runTest()`, and `teardown()`. This is a classic Template Method. JUnit uses `@Before`, `@Test`, and `@After` instead. What advantage does the annotation-based approach have over Template Method?
3. If the template method calls five abstract methods, and a subclass only needs custom behavior for two of them, what should you do with the other three? How do hooks help here?
4. Can you combine Template Method and Strategy? Give an example where the template method delegates one of its steps to a Strategy object.
5. Why does Template Method violate the "prefer composition over inheritance" principle? When is this trade-off acceptable?

Exercises

Exercise 1: HTTP Request Handler

Create an `AbstractRequestHandler` with a template method `handleRequest()` that follows this structure: authenticate, validate input, process, format response. Implement `RestHandler` (returns JSON) and `GraphQLHandler` (returns GraphQL response). Add a hook for rate limiting.

Exercise 2: Notification Pipeline

Build a notification system with `AbstractNotificationSender`. The template method should: build message, validate recipient, send notification, log result. Implement `EmailSender`, `SMSSender`, and `PushNotificationSender`. Each formats the message differently but follows the same pipeline.

Exercise 3: Data Migration Tool

Create an `AbstractMigrator` that defines: connect to source, extract batch, transform batch, load to destination, verify count, disconnect. Implement `MySQLToPostgresMigrator` and `MongoToElasticMigrator`. Add hooks for `onBatchComplete()` and `onError()`.

What Is Next?

Template Method defines a fixed algorithm with variable steps. But what if a request needs to pass through a series of independent handlers, where each handler decides whether to process it or pass it along? In the next chapter, we explore the Chain of Responsibility pattern -- the pattern behind HTTP middleware, validation pipelines, and exception handling chains.

Chapter 18: Chain of Responsibility -- Pass Requests Along a Handler Chain

What You Will Learn

- How to build a chain of handlers that each decide whether to process or pass a request
- Implementing middleware chains in Java (AuthHandler, ValidationHandler, LoggingHandler)
- Building Python approval chains for business workflows
- Using Spring interceptors and servlet filters as real-world chains
- Applying Chain of Responsibility to HTTP middleware, validation pipelines, and exception handling

Why This Chapter Matters

In every backend system, requests pass through multiple processing stages before reaching their destination. An HTTP request might need authentication, then rate limiting, then input validation, then logging -- all before the actual business logic runs. You need a way to compose these stages flexibly, add or remove stages without changing others, and let each stage decide independently whether the request should continue.

The Chain of Responsibility pattern does exactly this. It links handlers into a chain where each handler either processes the request and stops, or passes it to the next handler. This is the pattern behind servlet filters, Express middleware, Django middleware, and Spring interceptors.

The Problem: Hardcoded Processing Pipeline

```java
// RequestProcessor.java (BEFORE -- monolithic)
public class RequestProcessor {

    public Response process(Request request) {
        // Authentication
        if (!isAuthenticated(request)) {
            return new Response(401, "Unauthorized");
        }

        // Rate limiting
        if (isRateLimited(request)) {
            return new Response(429, "Too Many Requests");
        }

        // Input validation
        if (!isValid(request)) {
            return new Response(400, "Bad Request");
        }

        // Logging
        logRequest(request);

        // Business logic
        return handleBusinessLogic(request);
    }
    // Adding a new check = modifying this method
    // Removing a check = modifying this method
    // Reordering checks = modifying this method
    // Testing one check in isolation = impossible
}
```

```
The Problem:

  +-----------------------------------------+
  |          RequestProcessor                |
  |                                          |
  |  if (!authenticated) return 401          |
  |  if (rateLimited) return 429             |
  |  if (!valid) return 400                  |
  |  logRequest()                            |
  |  return businessLogic()                  |
  |                                          |
  |  ALL logic in ONE class.                 |
  |  Cannot reorder, add, or remove steps    |
  |  without touching this code.             |
  +-----------------------------------------+
```

The Solution: Chain of Responsibility

```
Chain of Responsibility Structure:

  Request
    |
    v
  +--------+     +--------+     +--------+     +----------+
  |  Auth   |---->| Rate   |---->|Validate|---->| Business |
  | Handler |     | Limiter|     | Handler|     |  Logic   |
  +--------+     +--------+     +--------+     +----------+
    |               |               |
    v               v               v
  (reject)       (reject)       (reject)
  401            429             400

  Each handler:
  1. Inspects the request
  2. Either handles it (stops the chain) or passes it to the next handler
  3. Knows NOTHING about other handlers in the chain
```

Java Implementation: Middleware Chain

Step 1: Define the Handler Interface

```java
// Handler.java
public abstract class Handler {
    private Handler next;

    public Handler setNext(Handler next) {
        this.next = next;
        return next;  // Return next for fluent chaining
    }

    public Response handle(Request request) {
        if (next != null) {
            return next.handle(request);
        }
        return new Response(200, "OK -- reached end of chain");
    }

    protected Handler getNext() {
        return next;
    }
```

```
}

// Request.java
public class Request {
    private final String path;
    private final String method;
    private final Map<String, String> headers;
    private final String body;
    private final String clientIp;

    public Request(String path, String method,
                   Map<String, String> headers, String body,
                   String clientIp) {
        this.path = path;
        this.method = method;
        this.headers = headers;
        this.body = body;
        this.clientIp = clientIp;
    }

    // Getters
    public String getPath() { return path; }
    public String getMethod() { return method; }
    public Map<String, String> getHeaders() { return headers; }
    public String getBody() { return body; }
    public String getClientIp() { return clientIp; }
}

// Response.java
public class Response {
    private final int statusCode;
    private final String body;

    public Response(int statusCode, String body) {
        this.statusCode = statusCode;
        this.body = body;
    }

    public int getStatusCode() { return statusCode; }
    public String getBody() { return body; }

    @Override
    public String toString() {
        return statusCode + ": " + body;
    }
}
```

Step 2: Implement Concrete Handlers

```java
// AuthHandler.java
public class AuthHandler extends Handler {

    private static final Set<String> VALID_TOKENS = Set.of(
        "token-abc123", "token-def456"
    );

    @Override
    public Response handle(Request request) {
        System.out.println("  [Auth] Checking authentication...");

        String authHeader = request.getHeaders().get("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            System.out.println("  [Auth] REJECTED -- No token provided");
            return new Response(401, "Unauthorized: Missing token");
        }

        String token = authHeader.substring(7);
        if (!VALID_TOKENS.contains(token)) {
            System.out.println("  [Auth] REJECTED -- Invalid token");
            return new Response(401, "Unauthorized: Invalid token");
        }

        System.out.println("  [Auth] PASSED");
        return super.handle(request);  // Pass to next handler
    }
}

// RateLimitHandler.java
public class RateLimitHandler extends Handler {

    private final Map<String, Integer> requestCounts = new HashMap<>();
    private final int maxRequests;

    public RateLimitHandler(int maxRequestsPerMinute) {
        this.maxRequests = maxRequestsPerMinute;
    }

    @Override
    public Response handle(Request request) {
        System.out.println("  [RateLimit] Checking rate limit...");

        String clientIp = request.getClientIp();
        int count = requestCounts.getOrDefault(clientIp, 0) + 1;
        requestCounts.put(clientIp, count);

        if (count > maxRequests) {
            System.out.println("  [RateLimit] REJECTED -- "
                + count + "/" + maxRequests + " requests");
            return new Response(429, "Too Many Requests");
        }

        System.out.println("  [RateLimit] PASSED (" + count
            + "/" + maxRequests + ")");
```

```
        return super.handle(request);
    }
}

// ValidationHandler.java
public class ValidationHandler extends Handler {

    @Override
    public Response handle(Request request) {
        System.out.println("  [Validation] Validating request...");

        if ("POST".equals(request.getMethod())
                || "PUT".equals(request.getMethod())) {
            if (request.getBody() == null || request.getBody().isEmpty()) {
                System.out.println("  [Validation] REJECTED -- Empty body");
                return new Response(400, "Bad Request: Body required");
            }
        }

        if (request.getPath() == null || request.getPath().isEmpty()) {
            System.out.println("  [Validation] REJECTED -- No path");
            return new Response(400, "Bad Request: Path required");
        }

        System.out.println("  [Validation] PASSED");
        return super.handle(request);
    }
}

// LoggingHandler.java
public class LoggingHandler extends Handler {

    @Override
    public Response handle(Request request) {
        System.out.println("  [Logging] " + request.getMethod()
            + " " + request.getPath()
            + " from " + request.getClientIp());

        long start = System.currentTimeMillis();
        Response response = super.handle(request);  // Continue chain
        long duration = System.currentTimeMillis() - start;

        System.out.println("  [Logging] Response: "
            + response.getStatusCode() + " (" + duration + "ms)");
        return response;
    }
}

// BusinessLogicHandler.java
public class BusinessLogicHandler extends Handler {

    @Override
    public Response handle(Request request) {
        System.out.println("  [Business] Processing business logic...");
```

```
        // Simulate actual work
        String result = "Processed: " + request.getMethod()
            + " " + request.getPath();

        return new Response(200, result);
    }
}
```

Step 3: Build and Use the Chain

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        // Build the chain
        Handler auth = new AuthHandler();
        Handler rateLimit = new RateLimitHandler(5);
        Handler validation = new ValidationHandler();
        Handler logging = new LoggingHandler();
        Handler business = new BusinessLogicHandler();

        // Link: Auth -> RateLimit -> Validation -> Logging -> Business
        auth.setNext(rateLimit)
            .setNext(validation)
            .setNext(logging)
            .setNext(business);

        // Test 1: Valid request
        System.out.println("--- Test 1: Valid Request ---");
        Request validRequest = new Request(
            "/api/users", "GET",
            Map.of("Authorization", "Bearer token-abc123"),
            "", "192.168.1.1"
        );
        Response r1 = auth.handle(validRequest);
        System.out.println("Final: " + r1 + "\n");

        // Test 2: No auth token
        System.out.println("--- Test 2: Missing Auth ---");
        Request noAuth = new Request(
            "/api/users", "GET",
            Map.of(), "", "192.168.1.2"
        );
        Response r2 = auth.handle(noAuth);
        System.out.println("Final: " + r2 + "\n");

        // Test 3: POST with empty body
        System.out.println("--- Test 3: Empty POST Body ---");
        Request emptyPost = new Request(
            "/api/users", "POST",
            Map.of("Authorization", "Bearer token-abc123"),
```

```
            "", "192.168.1.1"
        );
        Response r3 = auth.handle(emptyPost);
        System.out.println("Final: " + r3 + "\n");
    }
}
```

Output:

```
--- Test 1: Valid Request ---
  [Auth] Checking authentication...
  [Auth] PASSED
  [RateLimit] Checking rate limit...
  [RateLimit] PASSED (1/5)
  [Validation] Validating request...
  [Validation] PASSED
  [Logging] GET /api/users from 192.168.1.1
  [Business] Processing business logic...
  [Logging] Response: 200 (1ms)
Final: 200: Processed: GET /api/users

--- Test 2: Missing Auth ---
  [Auth] Checking authentication...
  [Auth] REJECTED -- No token provided
Final: 401: Unauthorized: Missing token

--- Test 3: Empty POST Body ---
  [Auth] Checking authentication...
  [Auth] PASSED
  [RateLimit] Checking rate limit...
  [RateLimit] PASSED (2/5)
  [Validation] Validating request...
  [Validation] REJECTED -- Empty body
Final: 400: Bad Request: Body required
```

Before vs After

```
BEFORE (monolithic):                 AFTER (chain):
+---------------------------+       +------+  +------+  +------+
| if (!auth) return 401     |       | Auth |->| Rate |->|Valid. |
| if (rateLimit) return 429 |       +------+  +------+  +------+
| if (!valid) return 400    |           |         |         |
| log()                     |           v         v         v
| return business()         |        reject    reject    reject
+---------------------------+
```

```
                                      +------+  +--------+
  Adding CORS handler:                | Log  |->|Business|
  Modify the monolithic class         +------+  +--------+

                                      Adding CORS handler:
                                      Insert CorsHandler between
                                      Auth and RateLimit. No other
                                      handler changes.
```

Python Implementation: Approval Chain

```python
# approval_chain.py
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional


@dataclass
class PurchaseRequest:
    description: str
    amount: float
    requester: str
    department: str


@dataclass
class ApprovalResult:
    approved: bool
    approver: str
    message: str


class Approver(ABC):
    """Base handler in the approval chain."""

    def __init__(self, name: str, limit: float):
        self.name = name
        self.limit = limit
        self._next: Optional[Approver] = None

    def set_next(self, approver: "Approver") -> "Approver":
        self._next = approver
        return approver

    def handle(self, request: PurchaseRequest) -> ApprovalResult:
        if request.amount <= self.limit:
```

```
            return self.approve(request)

        if self._next:
            print(f"  [{self.name}] ${request.amount:,.2f} exceeds "
                  f"my limit (${self.limit:,.2f}). Escalating...")
            return self._next.handle(request)

        return ApprovalResult(
            approved=False,
            approver=self.name,
            message=f"No one can approve ${request.amount:,.2f}"
        )

    def approve(self, request: PurchaseRequest) -> ApprovalResult:
        print(f"  [{self.name}] APPROVED: {request.description} "
              f"(${request.amount:,.2f})")
        return ApprovalResult(
            approved=True,
            approver=self.name,
            message=f"Approved by {self.name}"
        )


class TeamLead(Approver):
    def __init__(self):
        super().__init__("Team Lead", limit=1000.0)


class Manager(Approver):
    def __init__(self):
        super().__init__("Manager", limit=10000.0)


class Director(Approver):
    def __init__(self):
        super().__init__("Director", limit=50000.0)


class VP(Approver):
    def __init__(self):
        super().__init__("VP", limit=100000.0)

    def approve(self, request: PurchaseRequest) -> ApprovalResult:
        # VP adds extra scrutiny
        print(f"  [{self.name}] Reviewing budget impact for "
              f"${request.amount:,.2f}...")
        if request.department == "R&D" and request.amount > 75000:
            print(f"  [{self.name}] R&D budget limit reached. "
                  f"Needs board approval.")
            return ApprovalResult(
                approved=False,
                approver=self.name,
                message="Needs board approval for R&D expenses > $75k"
            )
        return super().approve(request)
```

```
# --- Usage ---
if __name__ == "__main__":
    # Build the chain
    team_lead = TeamLead()
    manager = Manager()
    director = Director()
    vp = VP()

    team_lead.set_next(manager).set_next(director).set_next(vp)

    # Test different amounts
    requests = [
        PurchaseRequest("Office supplies", 500, "Alice", "Engineering"),
        PurchaseRequest("New monitors", 5000, "Bob", "Engineering"),
        PurchaseRequest("Server upgrade", 25000, "Charlie", "Ops"),
        PurchaseRequest("ML training cluster", 80000, "Dave", "R&D"),
        PurchaseRequest("Office renovation", 150000, "Eve", "Facilities"),
    ]

    for req in requests:
        print(f"\n--- Purchase Request: {req.description} "
              f"(${req.amount:,.2f}) ---")
        result = team_lead.handle(req)
        status = "APPROVED" if result.approved else "DENIED"
        print(f"  Result: {status} -- {result.message}")
```

Output:

```
--- Purchase Request: Office supplies ($500.00) ---
  [Team Lead] APPROVED: Office supplies ($500.00)
  Result: APPROVED -- Approved by Team Lead

--- Purchase Request: New monitors ($5,000.00) ---
  [Team Lead] $5,000.00 exceeds my limit ($1,000.00). Escalating...
  [Manager] APPROVED: New monitors ($5,000.00)
  Result: APPROVED -- Approved by Manager

--- Purchase Request: Server upgrade ($25,000.00) ---
  [Team Lead] $25,000.00 exceeds my limit ($1,000.00). Escalating...
  [Manager] $25,000.00 exceeds my limit ($10,000.00). Escalating...
  [Director] APPROVED: Server upgrade ($25,000.00)
  Result: APPROVED -- Approved by Director

--- Purchase Request: ML training cluster ($80,000.00) ---
  [Team Lead] $80,000.00 exceeds my limit ($1,000.00). Escalating...
  [Manager] $80,000.00 exceeds my limit ($10,000.00). Escalating...
  [Director] $80,000.00 exceeds my limit ($50,000.00). Escalating...
  [VP] Reviewing budget impact for $80,000.00...
  [VP] R&D budget limit reached. Needs board approval.
  Result: DENIED -- Needs board approval for R&D expenses > $75k
```

```
--- Purchase Request: Office renovation ($150,000.00) ---
  [Team Lead] $150,000.00 exceeds my limit ($1,000.00). Escalating...
  [Manager] $150,000.00 exceeds my limit ($10,000.00). Escalating...
  [Director] $150,000.00 exceeds my limit ($50,000.00). Escalating...
  [VP] $150,000.00 exceeds my limit ($100,000.00). Escalating...
  Result: DENIED -- No one can approve $150,000.00
```

Spring Interceptors

Spring uses Chain of Responsibility via `HandlerInterceptor`.

```java
// AuthInterceptor.java
@Component
public class AuthInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response,
                             Object handler) throws Exception {
        String token = request.getHeader("Authorization");
        if (token == null || !token.startsWith("Bearer ")) {
            response.setStatus(401);
            response.getWriter().write("Unauthorized");
            return false;  // STOP the chain
        }
        return true;  // CONTINUE the chain
    }
}

// LoggingInterceptor.java
@Component
public class LoggingInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response,
                             Object handler) throws Exception {
        request.setAttribute("startTime", System.currentTimeMillis());
        System.out.println("[Log] " + request.getMethod()
            + " " + request.getRequestURI());
        return true;  // Always continue
    }

    @Override
    public void afterCompletion(HttpServletRequest request,
                                HttpServletResponse response,
```

```
                                Object handler, Exception ex) {
        long start = (Long) request.getAttribute("startTime");
        long duration = System.currentTimeMillis() - start;
        System.out.println("[Log] Completed in " + duration + "ms"
            + " -- Status: " + response.getStatus());
    }
}

// Register interceptors
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Autowired private AuthInterceptor authInterceptor;
    @Autowired private LoggingInterceptor loggingInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        // Order matters -- auth runs first
        registry.addInterceptor(loggingInterceptor)
                .addPathPatterns("/**");

        registry.addInterceptor(authInterceptor)
                .addPathPatterns("/api/**")
                .excludePathPatterns("/api/public/**");
    }
}
```

```
Spring Interceptor Chain:

  HTTP Request
       |
       v
  +------------------+
  | LoggingInterceptor|  preHandle()
  +------------------+
       |
       v
  +------------------+
  | AuthInterceptor   |  preHandle()
  +------------------+       |
       |                 return false -> 401 (chain stops)
       v                 return true  -> chain continues
  +------------------+
  | Controller        |  handleRequest()
  +------------------+
       |
       v
  +------------------+
  | AuthInterceptor   |  afterCompletion()
  +------------------+
       |
       v
  +------------------+
```

```
  | LoggingInterceptor|  afterCompletion()
  +------------------+
       |
       v
  HTTP Response
```

Real-World Use Case: Validation Pipeline

```java
// ValidationHandler.java
public abstract class ValidationHandler extends Handler {
    private final String ruleName;

    protected ValidationHandler(String ruleName) {
        this.ruleName = ruleName;
    }

    @Override
    public Response handle(Request request) {
        List<String> errors = validate(request);
        if (!errors.isEmpty()) {
            System.out.println("  [" + ruleName + "] FAILED: " + errors);
            return new Response(400, "Validation failed: " + errors);
        }
        System.out.println("  [" + ruleName + "] PASSED");
        return super.handle(request);
    }

    protected abstract List<String> validate(Request request);
}

// ContentTypeValidator.java
public class ContentTypeValidator extends ValidationHandler {
    public ContentTypeValidator() { super("ContentType"); }

    @Override
    protected List<String> validate(Request request) {
        List<String> errors = new ArrayList<>();
        String contentType = request.getHeaders().get("Content-Type");
        if ("POST".equals(request.getMethod())
                && !"application/json".equals(contentType)) {
            errors.add("Content-Type must be application/json");
        }
        return errors;
    }
}
```

```java
// BodySizeValidator.java
public class BodySizeValidator extends ValidationHandler {
    private final int maxSize;

    public BodySizeValidator(int maxSizeBytes) {
        super("BodySize");
        this.maxSize = maxSizeBytes;
    }

    @Override
    protected List<String> validate(Request request) {
        List<String> errors = new ArrayList<>();
        if (request.getBody() != null && request.getBody().length() > maxSize) {
            errors.add("Body exceeds " + maxSize + " bytes");
        }
        return errors;
    }
}

// RequiredFieldsValidator.java
public class RequiredFieldsValidator extends ValidationHandler {
    private final Set<String> requiredFields;

    public RequiredFieldsValidator(String... fields) {
        super("RequiredFields");
        this.requiredFields = Set.of(fields);
    }

    @Override
    protected List<String> validate(Request request) {
        List<String> errors = new ArrayList<>();
        // In a real system, parse JSON body and check fields
        for (String field : requiredFields) {
            if (!request.getBody().contains("\"" + field + "\"")) {
                errors.add("Missing required field: " + field);
            }
        }
        return errors;
    }
}
```

When to Use / When NOT to Use

Use Chain of Responsibility When

Scenario	Why Chain Helps

HTTP middleware pipeline	Each middleware is independent
Validation with multiple rules	Each rule is a separate handler
Approval workflows	Different approval levels
Exception handling	Try catch chains with fallbacks
Logging/monitoring pipeline	Non-blocking pass-through handlers

Do NOT Use Chain of Responsibility When

Scenario	Why Not
Guaranteed processing by all handlers	Chain can stop early -- use Observer
Order does not matter	A simple list of validators is simpler
Only one handler ever handles a request	Direct dispatch is clearer
Performance-critical path	Chain traversal adds overhead

Common Mistakes

Mistake 1: Broken Chain (Forgetting to Call Next)

```java
// BAD -- Handler processes but never passes to next
public class LoggingHandler extends Handler {
    @Override
    public Response handle(Request request) {
        log(request);
        // Forgot super.handle(request) -- chain stops here!
        return new Response(200, "Logged");
    }
}

// GOOD -- Always call super.handle() to continue the chain
public class LoggingHandler extends Handler {
    @Override
    public Response handle(Request request) {
        log(request);
```

```
        return super.handle(request);  // Continue chain
    }
}
```

Mistake 2: Handlers That Depend on Order

```java
// BAD -- Validation assumes auth already ran
public class ValidationHandler extends Handler {
    @Override
    public Response handle(Request request) {
        User user = request.getAuthenticatedUser();  // Assumes auth set this
        // NullPointerException if auth handler was removed or reordered
    }
}

// GOOD -- Each handler is self-contained
public class ValidationHandler extends Handler {
    @Override
    public Response handle(Request request) {
        // Validate independently, without assumptions about other handlers
        if (request.getBody() == null) {
            return new Response(400, "Body required");
        }
        return super.handle(request);
    }
}
```

Mistake 3: Circular Chain

```java
// BAD -- Creates infinite loop
Handler a = new AuthHandler();
Handler b = new LoggingHandler();
a.setNext(b);
b.setNext(a);  // Circular! Infinite loop when handling requests

// GOOD -- Chain always terminates
// Either reaches end (next is null) or a handler stops it
```

Best Practices

1. Each handler should be independent. It should not assume what handlers ran before it or will run after it.

2. Always provide a chain terminator. The last handler should produce a final response or action.
3. Make handler order configurable. Use a builder or configuration file to define the chain, not hardcoded wiring.
4. Log at each handler. When debugging, you need to know which handler in the chain processed or rejected the request.
5. Handle exceptions in each handler. One handler throwing should not crash the entire chain.
6. Keep handlers focused. One handler, one responsibility. Authentication, validation, and logging should be separate handlers.

Quick Summary

```
+-----------------------------------------------------------------+
|             CHAIN OF RESPONSIBILITY SUMMARY                      |
+-----------------------------------------------------------------+
| Intent:     Pass a request along a chain of handlers. Each      |
|             handler processes or forwards the request.           |
+-----------------------------------------------------------------+
| Problem:    Monolithic request processing with tangled logic    |
| Solution:   Independent handlers linked in a configurable      |
|             chain                                                |
+-----------------------------------------------------------------+
| Key parts:                                                       |
|   - Handler (abstract, knows next handler)                       |
|   - Concrete handlers (auth, validation, logging, etc.)          |
|   - Client (builds chain, sends initial request)                 |
+-----------------------------------------------------------------+
| Real-world: HTTP middleware, servlet filters, Spring             |
|             interceptors, approval workflows                     |
+-----------------------------------------------------------------+
```

Key Points

- Chain of Responsibility links handlers in a sequence where each handler decides to process or pass the request.
- Each handler is independent and can be added, removed, or reordered without affecting others.

- The pattern is the foundation of HTTP middleware in every major web framework.
- Spring uses `HandlerInterceptor` with `preHandle()` (before controller) and `afterCompletion()` (after response).
- Always ensure the chain terminates -- either a handler stops it or it reaches the end.
- Handlers should not depend on the order of other handlers in the chain.

Practice Questions

1. In the middleware chain example, what happens if the `LoggingHandler` is placed after the `AuthHandler` and a request fails authentication? Does the request get logged? How would you ensure all requests are logged regardless of outcome?
2. How does Chain of Responsibility differ from a simple `List<Validator>` where you iterate and check each one? When does the chain pattern add value over a list?
3. You need to add a CORS handler to the middleware chain. Where should it go? What happens if it is placed after the AuthHandler?
4. In the approval chain, the VP rejects R&D requests over $75K. How would you make this rule configurable without modifying the VP class?
5. Spring's `FilterChain.doFilter()` uses Chain of Responsibility. What happens if a filter calls `doFilter()` twice? How does Spring prevent this?

Exercises

Exercise 1: Exception Handler Chain

Build a chain of exception handlers: `ValidationExceptionHandler` (returns 400), `AuthenticationExceptionHandler` (returns 401), `NotFoundExceptionHandler` (returns 404), and `GenericExceptionHandler` (returns 500 as fallback). Each handler checks if it can handle the exception type, and if not, passes it to the next.

Exercise 2: Content Negotiation

Create a handler chain for content negotiation. `JSONHandler` checks if the client accepts JSON, `XMLHandler` checks for XML, and `HTMLHandler` checks for HTML. The first handler that matches the client's `Accept` header produces the response. Add a `DefaultHandler` that returns JSON if no match is found.

Exercise 3: Multi-Stage Data Pipeline

Build a data processing pipeline as a chain: `DeduplicationHandler` (removes duplicates), `NormalizationHandler` (standardizes formats), `EnrichmentHandler` (adds computed fields), and `PersistenceHandler` (saves to database). Each handler transforms the data and passes the modified version to the next handler.

What Is Next?

Chain of Responsibility lets requests flow through a series of independent handlers. But what happens when an object's behavior needs to change based on its internal state, and you want to avoid a mess of `if/else` conditions? In the next chapter, we explore the State pattern -- a clean way to model state machines where objects behave differently depending on what state they are in.

Chapter 19: State Pattern -- Behavior Changes Based on Internal State

What You Will Learn

- How to eliminate state-based `if/else` chains by using state objects
- Implementing a state machine for order processing in Java (Pending, Confirmed, Shipped, Delivered)
- Building a Python document workflow (Draft, Review, Published)
- Managing state transitions cleanly and preventing invalid transitions
- Real-world applications: order processing, user sessions, payment states
- When the State pattern is the right tool and when a simple enum is enough

Why This Chapter Matters

Almost every backend system manages objects that change behavior based on their state. An order behaves differently when it is pending versus shipped. A user session behaves differently when active versus expired. A document goes through draft, review, and published states with different rules at each stage.

The naive approach -- giant `if/else` or `switch` blocks checking the current state -- quickly becomes unreadable and error-prone. Adding a new state means modifying every method that checks state. The State pattern solves this by putting each state's behavior into its own class, making state transitions explicit and state-specific logic isolated.

The Problem: State Checking Everywhere

```java
// Order.java (BEFORE -- state checks everywhere)
public class Order {
    private String state = "PENDING";

    public void confirm() {
        if ("PENDING".equals(state)) {
            state = "CONFIRMED";
        } else if ("CONFIRMED".equals(state)) {
            throw new IllegalStateException("Already confirmed");
        } else if ("SHIPPED".equals(state)) {
            throw new IllegalStateException("Already shipped");
        } else if ("DELIVERED".equals(state)) {
            throw new IllegalStateException("Already delivered");
        } else if ("CANCELLED".equals(state)) {
            throw new IllegalStateException("Order is cancelled");
        }
    }

    public void ship() {
        if ("CONFIRMED".equals(state)) {
            state = "SHIPPED";
        } else if ("PENDING".equals(state)) {
            throw new IllegalStateException("Must confirm first");
        } else if ("SHIPPED".equals(state)) {
            throw new IllegalStateException("Already shipped");
        }
        // ... more checks for every method, every state
    }

    // cancel(), deliver(), getStatus() -- ALL have similar if/else blocks
    // Adding a new state (e.g., PROCESSING) means modifying EVERY method.
}
```

```
The State Explosion Problem:

  States:   PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED
  Methods:  confirm(), ship(), deliver(), cancel(), getStatus()

  Every method checks every state:
  5 states x 5 methods = 25 conditional branches

  Add one state (PROCESSING):
  6 states x 5 methods = 30 conditional branches
  You must modify ALL 5 methods.
```

The Solution: State Pattern

```
State Pattern Structure:

  +-------------------+          +-------------------+
  |     Order         |          |   OrderState      |
  | (Context)         |  has-a   |   (Interface)     |
  |                   |--------->|                   |
  | - state: State    |          | + confirm(order)  |
  | + confirm()       |          | + ship(order)     |
  | + ship()          |          | + deliver(order)  |
  | + cancel()        |          | + cancel(order)   |
  +-------------------+          | + getStatus()     |
                                 +-------------------+
                                   ^    ^    ^    ^
                                   |    |    |    |
                         Pending  Confirmed Shipped Delivered
                          State    State    State    State

  Each state class handles ONLY its own behavior.
  The Order delegates to its current state.
  State transitions happen by replacing the state object.
```

Java Implementation: Order State Machine

Step 1: Define the State Interface

```java
// OrderState.java
public interface OrderState {
    void confirm(Order order);
    void ship(Order order);
    void deliver(Order order);
    void cancel(Order order);
    String getStatus();
}
```

Step 2: Implement Each State

```java
// PendingState.java
public class PendingState implements OrderState {
```

```
    @Override
    public void confirm(Order order) {
        System.out.println("  Order confirmed. Preparing for fulfillment.");
        order.setState(new ConfirmedState());
    }

    @Override
    public void ship(Order order) {
        throw new IllegalStateException(
            "Cannot ship -- order must be confirmed first.");
    }

    @Override
    public void deliver(Order order) {
        throw new IllegalStateException(
            "Cannot deliver -- order is still pending.");
    }

    @Override
    public void cancel(Order order) {
        System.out.println("  Order cancelled. No charges applied.");
        order.setState(new CancelledState());
    }

    @Override
    public String getStatus() {
        return "PENDING";
    }
}

// ConfirmedState.java
public class ConfirmedState implements OrderState {

    @Override
    public void confirm(Order order) {
        throw new IllegalStateException("Order is already confirmed.");
    }

    @Override
    public void ship(Order order) {
        System.out.println("  Order shipped. Tracking number generated.");
        order.setState(new ShippedState());
    }

    @Override
    public void deliver(Order order) {
        throw new IllegalStateException(
            "Cannot deliver -- order has not been shipped.");
    }

    @Override
    public void cancel(Order order) {
        System.out.println("  Order cancelled. Refund initiated.");
        order.setState(new CancelledState());
```

```
    }

    @Override
    public String getStatus() {
        return "CONFIRMED";
    }
}

// ShippedState.java
public class ShippedState implements OrderState {

    @Override
    public void confirm(Order order) {
        throw new IllegalStateException("Order is already past confirmation.");
    }

    @Override
    public void ship(Order order) {
        throw new IllegalStateException("Order is already shipped.");
    }

    @Override
    public void deliver(Order order) {
        System.out.println("  Order delivered. Customer notified.");
        order.setState(new DeliveredState());
    }

    @Override
    public void cancel(Order order) {
        throw new IllegalStateException(
            "Cannot cancel -- order is already in transit.");
    }

    @Override
    public String getStatus() {
        return "SHIPPED";
    }
}

// DeliveredState.java
public class DeliveredState implements OrderState {

    @Override
    public void confirm(Order order) {
        throw new IllegalStateException("Order already delivered.");
    }

    @Override
    public void ship(Order order) {
        throw new IllegalStateException("Order already delivered.");
    }

    @Override
    public void deliver(Order order) {
        throw new IllegalStateException("Order already delivered.");
```

```
    }

    @Override
    public void cancel(Order order) {
        throw new IllegalStateException(
            "Cannot cancel a delivered order. Use returns.");
    }

    @Override
    public String getStatus() {
        return "DELIVERED";
    }
}

// CancelledState.java
public class CancelledState implements OrderState {

    @Override
    public void confirm(Order order) {
        throw new IllegalStateException("Cannot confirm a cancelled order.");
    }

    @Override
    public void ship(Order order) {
        throw new IllegalStateException("Cannot ship a cancelled order.");
    }

    @Override
    public void deliver(Order order) {
        throw new IllegalStateException("Cannot deliver a cancelled order.");
    }

    @Override
    public void cancel(Order order) {
        throw new IllegalStateException("Order is already cancelled.");
    }

    @Override
    public String getStatus() {
        return "CANCELLED";
    }
}
```

Step 3: Build the Context (Order)

```java
// Order.java (AFTER -- State Pattern)
public class Order {
    private final String orderId;
    private OrderState state;
    private final List<String> history = new ArrayList<>();
```

```
    public Order(String orderId) {
        this.orderId = orderId;
        this.state = new PendingState();
        recordTransition("Order created");
    }

    public void setState(OrderState newState) {
        String transition = state.getStatus() + " -> " + newState.getStatus();
        this.state = newState;
        recordTransition(transition);
    }

    public void confirm()  { state.confirm(this); }
    public void ship()     { state.ship(this); }
    public void deliver()  { state.deliver(this); }
    public void cancel()   { state.cancel(this); }
    public String getStatus() { return state.getStatus(); }

    private void recordTransition(String description) {
        history.add("[" + java.time.LocalTime.now()
            .format(java.time.format.DateTimeFormatter.ofPattern("HH:mm:ss"))
            + "] " + description);
    }

    public void printHistory() {
        System.out.println("  History for " + orderId + ":");
        history.forEach(h -> System.out.println("    " + h));
    }
}
```

Step 4: Use It

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        Order order = new Order("ORD-001");
        System.out.println("Status: " + order.getStatus());

        // Happy path: Pending -> Confirmed -> Shipped -> Delivered
        order.confirm();
        System.out.println("Status: " + order.getStatus());

        order.ship();
        System.out.println("Status: " + order.getStatus());

        order.deliver();
        System.out.println("Status: " + order.getStatus());

        order.printHistory();
```

```
        // Try invalid transition
        System.out.println("\nAttempting to cancel delivered order:");
        try {
            order.cancel();
        } catch (IllegalStateException e) {
            System.out.println("  Error: " + e.getMessage());
        }

        // Cancellation path
        System.out.println("\n--- Cancellation Scenario ---");
        Order order2 = new Order("ORD-002");
        order2.confirm();
        order2.cancel();
        System.out.println("Status: " + order2.getStatus());
        order2.printHistory();
    }
}
```

Output:

```
Status: PENDING
  Order confirmed. Preparing for fulfillment.
Status: CONFIRMED
  Order shipped. Tracking number generated.
Status: SHIPPED
  Order delivered. Customer notified.
Status: DELIVERED
  History for ORD-001:
    [10:30:01] Order created
    [10:30:01] PENDING -> CONFIRMED
    [10:30:01] CONFIRMED -> SHIPPED
    [10:30:01] SHIPPED -> DELIVERED

Attempting to cancel delivered order:
  Error: Cannot cancel a delivered order. Use returns.

--- Cancellation Scenario ---
  Order confirmed. Preparing for fulfillment.
  Order cancelled. Refund initiated.
Status: CANCELLED
  History for ORD-002:
    [10:30:01] Order created
    [10:30:01] PENDING -> CONFIRMED
    [10:30:01] CONFIRMED -> CANCELLED
```

State Transition Diagram

```
+-------------------------------------------------------------+
|                 ORDER STATE TRANSITIONS                     |
+-------------------------------------------------------------+

                    +----------+
                    | PENDING  |
                    +----------+
                   /            \
            confirm()          cancel()
                /                \
        +----------+        +----------+
        |CONFIRMED |        |CANCELLED |
        +----------+        +----------+
       /            \            (final)
    ship()        cancel()
      /              \
 +----------+        |
 | SHIPPED  |        |
 +----------+   (goes to CANCELLED)
      |
   deliver()
      |
 +----------+
 |DELIVERED |
 +----------+
    (final)

  Valid transitions are defined by each state class.
  Invalid transitions throw IllegalStateException.
  No if/else needed -- just call the method.
+-------------------------------------------------------------+
```

Python Implementation: Document Workflow

```python
# document_workflow.py
from abc import ABC, abstractmethod
from datetime import datetime
from typing import Optional


class DocumentState(ABC):
    """Base state for document workflow."""

    @abstractmethod
    def edit(self, document: "Document", content: str) -> None:
        pass
```

```
    @abstractmethod
    def submit_for_review(self, document: "Document") -> None:
        pass

    @abstractmethod
    def approve(self, document: "Document", reviewer: str) -> None:
        pass

    @abstractmethod
    def reject(self, document: "Document", reason: str) -> None:
        pass

    @abstractmethod
    def publish(self, document: "Document") -> None:
        pass

    @abstractmethod
    def archive(self, document: "Document") -> None:
        pass

    @abstractmethod
    def get_status(self) -> str:
        pass


class DraftState(DocumentState):

    def edit(self, document, content):
        document.content = content
        print(f"  [Draft] Content updated ({len(content)} chars)")

    def submit_for_review(self, document):
        if not document.content:
            raise ValueError("Cannot submit empty document for review")
        print("  [Draft] Submitted for review")
        document.set_state(ReviewState())

    def approve(self, document, reviewer):
        raise IllegalTransition("Cannot approve a draft. Submit for review first.")

    def reject(self, document, reason):
        raise IllegalTransition("Cannot reject a draft.")

    def publish(self, document):
        raise IllegalTransition("Cannot publish a draft. Must go through review.")

    def archive(self, document):
        print("  [Draft] Draft archived")
        document.set_state(ArchivedState())

    def get_status(self):
        return "DRAFT"
```

```
class ReviewState(DocumentState):

    def edit(self, document, content):
        raise IllegalTransition("Cannot edit during review. Reject first.")

    def submit_for_review(self, document):
        raise IllegalTransition("Already in review.")

    def approve(self, document, reviewer):
        document.approved_by = reviewer
        document.approved_at = datetime.now()
        print(f"  [Review] Approved by {reviewer}")
        document.set_state(ApprovedState())

    def reject(self, document, reason):
        document.rejection_reason = reason
        print(f"  [Review] Rejected: {reason}")
        document.set_state(DraftState())  # Back to draft for edits

    def publish(self, document):
        raise IllegalTransition("Cannot publish without approval.")

    def archive(self, document):
        raise IllegalTransition("Cannot archive during review.")

    def get_status(self):
        return "IN_REVIEW"


class ApprovedState(DocumentState):

    def edit(self, document, content):
        raise IllegalTransition("Cannot edit approved document.")

    def submit_for_review(self, document):
        raise IllegalTransition("Already approved.")

    def approve(self, document, reviewer):
        raise IllegalTransition("Already approved.")

    def reject(self, document, reason):
        print(f"  [Approved] Approval revoked: {reason}")
        document.set_state(DraftState())

    def publish(self, document):
        document.published_at = datetime.now()
        print(f"  [Approved] Document published!")
        document.set_state(PublishedState())

    def archive(self, document):
        raise IllegalTransition("Publish or reject before archiving.")

    def get_status(self):
        return "APPROVED"
```

```
class PublishedState(DocumentState):

    def edit(self, document, content):
        raise IllegalTransition("Cannot edit published document. Create a new version.")

    def submit_for_review(self, document):
        raise IllegalTransition("Already published.")

    def approve(self, document, reviewer):
        raise IllegalTransition("Already published.")

    def reject(self, document, reason):
        raise IllegalTransition("Cannot reject published document.")

    def publish(self, document):
        raise IllegalTransition("Already published.")

    def archive(self, document):
        print("  [Published] Document archived")
        document.set_state(ArchivedState())

    def get_status(self):
        return "PUBLISHED"


class ArchivedState(DocumentState):

    def edit(self, document, content):
        raise IllegalTransition("Cannot edit archived document.")

    def submit_for_review(self, document):
        raise IllegalTransition("Cannot submit archived document.")

    def approve(self, document, reviewer):
        raise IllegalTransition("Cannot approve archived document.")

    def reject(self, document, reason):
        raise IllegalTransition("Cannot reject archived document.")

    def publish(self, document):
        raise IllegalTransition("Cannot publish archived document.")

    def archive(self, document):
        raise IllegalTransition("Already archived.")

    def get_status(self):
        return "ARCHIVED"


class IllegalTransition(Exception):
    pass


class Document:
```

```
    """Context -- the object whose behavior changes with state."""

    def __init__(self, title: str, author: str):
        self.title = title
        self.author = author
        self.content: str = ""
        self.approved_by: Optional[str] = None
        self.approved_at: Optional[datetime] = None
        self.published_at: Optional[datetime] = None
        self.rejection_reason: Optional[str] = None
        self._state: DocumentState = DraftState()
        self._history: list[str] = [f"Created by {author}"]

    def set_state(self, new_state: DocumentState) -> None:
        old = self._state.get_status()
        self._state = new_state
        self._history.append(f"{old} -> {new_state.get_status()}")

    @property
    def status(self) -> str:
        return self._state.get_status()

    def edit(self, content: str) -> None:
        self._state.edit(self, content)

    def submit_for_review(self) -> None:
        self._state.submit_for_review(self)

    def approve(self, reviewer: str) -> None:
        self._state.approve(self, reviewer)

    def reject(self, reason: str) -> None:
        self._state.reject(self, reason)

    def publish(self) -> None:
        self._state.publish(self)

    def archive(self) -> None:
        self._state.archive(self)

    def print_history(self) -> None:
        print(f"  History for '{self.title}':")
        for entry in self._history:
            print(f"    - {entry}")


# --- Usage ---
if __name__ == "__main__":
    doc = Document("API Design Guide", "Alice")

    # Happy path: Draft -> Review -> Approved -> Published
    print("--- Happy Path ---")
    doc.edit("REST API best practices and conventions...")
    print(f"  Status: {doc.status}")
```

```
    doc.submit_for_review()
    print(f"  Status: {doc.status}")

    doc.approve("Bob")
    print(f"  Status: {doc.status}")

    doc.publish()
    print(f"  Status: {doc.status}")

    doc.print_history()

    # Rejection path
    print("\n--- Rejection Path ---")
    doc2 = Document("Security Policy", "Charlie")
    doc2.edit("Initial security guidelines...")
    doc2.submit_for_review()
    doc2.reject("Needs more detail on authentication")
    print(f"  Status: {doc2.status}")  # Back to DRAFT

    doc2.edit("Updated with authentication details...")
    doc2.submit_for_review()
    doc2.approve("Dave")
    doc2.publish()
    print(f"  Status: {doc2.status}")

    doc2.print_history()

    # Invalid transition
    print("\n--- Invalid Transition ---")
    try:
        doc2.edit("Try to edit published doc")
    except IllegalTransition as e:
        print(f"  Error: {e}")
```

Output:

```
--- Happy Path ---
  [Draft] Content updated (43 chars)
  Status: DRAFT
  [Draft] Submitted for review
  Status: IN_REVIEW
  [Review] Approved by Bob
  Status: APPROVED
  [Approved] Document published!
  Status: PUBLISHED
  History for 'API Design Guide':
    - Created by Alice
    - DRAFT -> IN_REVIEW
    - IN_REVIEW -> APPROVED
    - APPROVED -> PUBLISHED

--- Rejection Path ---
  [Draft] Content updated (32 chars)
```

```
  [Draft] Submitted for review
  [Review] Rejected: Needs more detail on authentication
  Status: DRAFT
  [Draft] Content updated (42 chars)
  [Draft] Submitted for review
  [Review] Approved by Dave
  [Approved] Document published!
  Status: PUBLISHED
  History for 'Security Policy':
    - Created by Charlie
    - DRAFT -> IN_REVIEW
    - IN_REVIEW -> DRAFT
    - DRAFT -> IN_REVIEW
    - IN_REVIEW -> APPROVED
    - APPROVED -> PUBLISHED

--- Invalid Transition ---
  Error: Cannot edit published document. Create a new version.
```

Before vs After

```
BEFORE (if/else state checks):       AFTER (State Pattern):

  Order.confirm():                    Order.confirm():
  if (state == "PENDING")               state.confirm(this);
    state = "CONFIRMED";              // That is it. No if/else.
  else if (state == "CONFIRMED")
    throw ...;                        PendingState.confirm(order):
  else if (state == "SHIPPED")          order.setState(new ConfirmedState());
    throw ...;
  else if (state == "DELIVERED")      ConfirmedState.confirm(order):
    throw ...;                          throw "Already confirmed";
  else if (state == "CANCELLED")
    throw ...;                        Each state handles ONLY its behavior.

  Adding a new state:                 Adding a new state:
  Modify EVERY method in Order        Add ONE new state class
  (5 methods x 1 new branch = 5)     (implements the interface)
```

When to Use / When NOT to Use

Use State When

Scenario	Why State Helps
Object behavior changes with state	Each state encapsulates its rules
Many state-dependent conditionals	Eliminates if/else chains
State transitions are complex	Transitions are explicit in each state
States will grow over time	Adding a state = adding a class
You need transition history	Easy to log state changes

Do NOT Use State When

Scenario	Why Not
Only 2-3 simple states	A boolean or enum is simpler
States do not change behavior	State is just data, not behavior
All states behave the same	No variation = no need for pattern
Simple flag-based logic	`isActive` boolean suffices

Common Mistakes

Mistake 1: State Classes That Know About Each Other

```java
// BAD -- PendingState creates ConfirmedState directly
public class PendingState implements OrderState {
    public void confirm(Order order) {
        ConfirmedState next = new ConfirmedState();
        next.setPreviousState(this);  // Circular dependency
        order.setState(next);
    }
}

// GOOD -- States are independent, context manages transitions
```

```
public class PendingState implements OrderState {
    public void confirm(Order order) {
        order.setState(new ConfirmedState());  // Simple and clean
    }
}
```

Mistake 2: Putting Business Logic in the Context

```java
// BAD -- Order has state-specific logic
public class Order {
    public void confirm() {
        if (state.getStatus().equals("PENDING")) {
            // Business logic here defeats the purpose
            sendConfirmationEmail();
        }
        state.confirm(this);
    }
}

// GOOD -- State handles all state-specific behavior
public class PendingState implements OrderState {
    public void confirm(Order order) {
        // State-specific logic lives HERE
        order.setState(new ConfirmedState());
    }
}
```

Mistake 3: Forgetting Terminal States

```java
// BAD -- Delivered order can transition to anything
public class DeliveredState implements OrderState {
    public void confirm(Order order) {
        order.setState(new ConfirmedState());  // This makes no sense
    }
}

// GOOD -- Terminal states reject all transitions
public class DeliveredState implements OrderState {
    public void confirm(Order order) {
        throw new IllegalStateException("Order already delivered.");
    }
    // ALL methods throw -- this is a final state
}
```

Best Practices

1. Make invalid transitions throw explicit exceptions. The caller should know exactly why a transition failed.
2. Log state transitions. Maintain a history of state changes for debugging and audit trails.
3. Keep state classes stateless when possible. If a state does not store data, it can be shared (Flyweight).
4. Define transitions in the state, not the context. The context delegates; the state decides.
5. Draw the state diagram first. Before writing code, sketch all states and valid transitions. This prevents missed edge cases.
6. Consider using an enum for simple cases. If states do not have complex behavior, an enum with methods may be sufficient.

Quick Summary

```
+-----------------------------------------------------------------+
|                     STATE PATTERN SUMMARY                        |
+-----------------------------------------------------------------+
| Intent:     Allow an object to change its behavior when its    |
|             internal state changes.                             |
+-----------------------------------------------------------------+
| Problem:    if/else chains checking state in every method      |
| Solution:   Each state is a class that handles its own         |
|             behavior and transitions                            |
+-----------------------------------------------------------------+
| Key parts:                                                      |
|   - Context (the object with state, e.g., Order)                |
|   - State interface (defines all state-dependent methods)       |
|   - Concrete states (one class per state)                       |
+-----------------------------------------------------------------+
| Transitions: State objects replace themselves in the context   |
+-----------------------------------------------------------------+
```

Key Points

- The State pattern replaces state-conditional logic with polymorphic state objects.
- Each state is a separate class that defines behavior for that state only.

- State transitions happen by replacing the state object in the context.
- Invalid transitions are explicitly rejected with descriptive exceptions.
- Always draw the state diagram before coding to catch invalid transitions early.
- For simple cases with 2-3 states and minimal behavior, an enum is sufficient.

Practice Questions

1. In the order example, how would you add a `PROCESSING` state between `CONFIRMED` and `SHIPPED`? How many existing classes would you need to modify?
2. Can two states share common behavior? For example, both `PendingState` and `ConfirmedState` allow cancellation. How would you avoid duplicating the cancellation logic?
3. The document workflow sends a document back to `DraftState` after rejection. But the original `DraftState` has no memory of previous reviews. How would you track that a document has been rejected before?
4. How does the State pattern relate to finite state machines (FSMs)? Could you generate state classes from an FSM definition?
5. In a multithreaded environment, what happens if two threads try to transition an order's state simultaneously? How would you make state transitions thread-safe?

Exercises

Exercise 1: Traffic Light State Machine

Build a traffic light with states: `Red`, `Green`, `Yellow`. Each state transitions to the next in sequence (Red -> Green -> Yellow -> Red). Each state has a `getDuration()` method returning how long it lasts. Add a `FlashingYellow` emergency state that can be entered from any state.

Exercise 2: User Account States

Implement a user account with states: `Unverified`, `Active`, `Suspended`, `Closed`. Define valid transitions (e.g., Unverified -> Active via email verification, Active -> Suspended via admin action). Each state determines what actions the user can perform (login, post, edit profile).

Exercise 3: Payment State Machine

Build a payment processor with states: `Initiated`, `Authorized`, `Captured`, `Refunded`, `Failed`. Model the transitions for a typical payment flow. Include partial refunds (Captured -> PartiallyRefunded -> Refunded). Log every transition with timestamps and amounts.

What Is Next?

The State pattern manages how one object transitions through different behaviors. But what happens when you have many objects that need to communicate with each other, creating a tangled web of dependencies? In the next chapter, we explore the Mediator pattern -- a central coordinator that simplifies many-to-many communication down to manageable one-to-many relationships.

Chapter 20: Mediator Pattern -- Reduce Many-to-Many Complexity

What You Will Learn

- What the Mediator pattern is and why direct dependencies create chaos
- The air traffic controller analogy that makes Mediator intuitive
- How to implement a chat room mediator in Java
- How to build an event-driven service mediator in Python
- The difference between Mediator and Observer
- When centralized coordination helps and when it becomes a bottleneck

Why This Chapter Matters

As systems grow, components start talking to each other directly. Service A calls Service B, which notifies Service C, which updates Service A. Before long, every component depends on every other component. Adding a new component means modifying ten existing ones. Removing one breaks five others.

The Mediator pattern breaks this web of dependencies by introducing a central coordinator. Instead of components talking to each other, they talk to the mediator. The mediator knows the rules and routes messages accordingly. Components become independent and reusable.

The Problem

You are building a trading platform. The order system needs to communicate with:

- The inventory service (check stock)

- The payment service (charge the customer)
- The notification service (send confirmation)
- The analytics service (track the sale)
- The shipping service (create shipment)

Without a mediator, every service has direct references to the others:

```
WITHOUT MEDIATOR: Every service knows about every other service

    +----------+     +----------+     +---------+
    |  Order   |<--->| Inventory|<--->| Payment |
    +----------+     +----------+     +---------+
        ^   ^            ^              ^
        |   |            |              |
        |   +--------+   |    +----------+
        |            |   |    |
    +----------+  +----------+  +---------+
    | Shipping |  | Notify   |  |Analytics|
    +----------+  +----------+  +---------+

    15 direct connections between 6 services.
    Adding service #7 requires modifying all 6 existing services.
```

Each service has 5 dependencies. Adding a seventh service means updating 6 existing services. This is the many-to-many problem.

The Solution: Mediator Pattern

```
WITH MEDIATOR: Every service only knows the mediator

    +----------+     +----------+     +---------+
    |  Order   |     | Inventory|     | Payment |
    +----------+     +----------+     +---------+
         \               |               /
          \              |              /
           v             v             v
         +----------------------------+
         |         MEDIATOR           |
         |   (coordinates all         |
         |    communication)          |
         +----------------------------+
           ^             ^             ^
          /              |              \
         /               |               \
    +----------+  +----------+  +---------+
```

```
    | Shipping |  | Notify   |  |Analytics|
    +----------+  +----------+  +---------+

    6 connections (one per service to mediator).
    Adding service #7 means updating ONLY the mediator.
```

Key participants:

- Mediator: defines the interface for communication between components
- ConcreteMediator: implements the coordination logic
- Colleague: each component that communicates through the mediator

---

The Air Traffic Controller Analogy

An airport without a control tower would be chaos. Every pilot would need to communicate with every other pilot to avoid collisions.

```
WITHOUT TOWER (Direct Communication)
Plane A <---> Plane B
Plane A <---> Plane C
Plane B <---> Plane C
Plane A <---> Plane D
...
10 planes = 45 direct communication channels

WITH TOWER (Mediator)
Plane A ---> Tower ---> Plane B
Plane C ---> Tower ---> Plane A
10 planes = 10 communication channels (each to tower)
```

The tower (mediator) knows all the rules:

- Only one plane lands at a time
- Planes queue in order of fuel level
- Emergency landings preempt everything

No plane needs to know these rules. They just report their status to the tower and follow instructions. The tower coordinates.

Java Implementation: Chat Room

The Mediator Interface and Colleagues

```java
import java.util.List;

public interface ChatMediator {
    void sendMessage(String message, User sender);
    void sendPrivateMessage(String message, User sender, String recipientName);
    void addUser(User user);
    void removeUser(User user);
    List<String> getOnlineUsers();
}
```

```java
public class User {
    private final String name;
    private ChatMediator mediator;
    private final List<String> messageLog;

    public User(String name) {
        this.name = name;
        this.messageLog = new java.util.ArrayList<>();
    }

    public void setMediator(ChatMediator mediator) {
        this.mediator = mediator;
    }

    public void send(String message) {
        System.out.printf("[%s] sends: %s%n", name, message);
        mediator.sendMessage(message, this);
    }

    public void sendPrivate(String message, String recipientName) {
        System.out.printf("[%s] whispers to %s: %s%n",
                name, recipientName, message);
        mediator.sendPrivateMessage(message, this, recipientName);
    }

    public void receive(String message, String senderName) {
        String formatted = String.format("  [%s] received from %s: %s",
                name, senderName, message);
        messageLog.add(formatted);
        System.out.println(formatted);
    }

    public String getName() { return name; }

    public List<String> getMessageLog() {
```

```
        return java.util.Collections.unmodifiableList(messageLog);
    }
}
```

The Concrete Mediator

```java
import java.util.*;

public class ChatRoom implements ChatMediator {
    private final String roomName;
    private final List<User> users;
    private final List<String> bannedWords;

    public ChatRoom(String roomName) {
        this.roomName = roomName;
        this.users = new ArrayList<>();
        this.bannedWords = Arrays.asList("spam", "scam");
    }

    @Override
    public void addUser(User user) {
        users.add(user);
        user.setMediator(this);
        System.out.printf(">>> %s joined '%s' (online: %d)%n",
                user.getName(), roomName, users.size());

        // Notify others
        for (User u : users) {
            if (u != user) {
                u.receive(user.getName() + " has joined the chat",
                         "System");
            }
        }
    }

    @Override
    public void removeUser(User user) {
        users.remove(user);
        System.out.printf(">>> %s left '%s' (online: %d)%n",
                user.getName(), roomName, users.size());

        for (User u : users) {
            u.receive(user.getName() + " has left the chat", "System");
        }
    }

    @Override
    public void sendMessage(String message, User sender) {
        // Mediator applies rules: filter banned words
        if (containsBannedWord(message)) {
```

```
            sender.receive("Your message was blocked (policy violation)",
                           "System");
            return;
        }

        // Broadcast to all EXCEPT sender
        for (User user : users) {
            if (user != sender) {
                user.receive(message, sender.getName());
            }
        }
    }

    @Override
    public void sendPrivateMessage(String message, User sender,
                                   String recipientName) {
        for (User user : users) {
            if (user.getName().equals(recipientName)) {
                user.receive("[PRIVATE] " + message, sender.getName());
                return;
            }
        }
        sender.receive("User '" + recipientName + "' not found", "System");
    }

    @Override
    public List<String> getOnlineUsers() {
        List<String> names = new ArrayList<>();
        for (User u : users) {
            names.add(u.getName());
        }
        return names;
    }

    private boolean containsBannedWord(String message) {
        String lower = message.toLowerCase();
        return bannedWords.stream().anyMatch(lower::contains);
    }
}
```

Demo

```java
public class ChatDemo {
    public static void main(String[] args) {
        ChatRoom room = new ChatRoom("Backend Developers");

        User alice = new User("Alice");
        User bob = new User("Bob");
        User charlie = new User("Charlie");
```

```
        // Users join through the mediator
        room.addUser(alice);
        room.addUser(bob);
        room.addUser(charlie);

        System.out.println("\n--- Messaging ---");
        alice.send("Hey everyone, standup in 5 minutes!");
        bob.send("Got it, thanks!");

        System.out.println("\n--- Private Message ---");
        alice.sendPrivate("Can you review my PR?", "Bob");

        System.out.println("\n--- Content Filtering ---");
        charlie.send("Check out this spam link");

        System.out.println("\n--- User Leaves ---");
        room.removeUser(bob);
        alice.send("Bob just left, meeting canceled.");

        System.out.println("\n--- Online Users ---");
        System.out.println("Online: " + room.getOnlineUsers());
    }
}
```

Output:

```
>>> Alice joined 'Backend Developers' (online: 1)
>>> Bob joined 'Backend Developers' (online: 2)
  [Alice] received from System: Bob has joined the chat
>>> Charlie joined 'Backend Developers' (online: 3)
  [Alice] received from System: Charlie has joined the chat
  [Bob] received from System: Charlie has joined the chat

--- Messaging ---
[Alice] sends: Hey everyone, standup in 5 minutes!
  [Bob] received from Alice: Hey everyone, standup in 5 minutes!
  [Charlie] received from Alice: Hey everyone, standup in 5 minutes!
[Bob] sends: Got it, thanks!
  [Alice] received from Bob: Got it, thanks!
  [Charlie] received from Bob: Got it, thanks!

--- Private Message ---
[Alice] whispers to Bob: Can you review my PR?
  [Bob] received from Alice: [PRIVATE] Can you review my PR?

--- Content Filtering ---
[Charlie] sends: Check out this spam link
  [Charlie] received from System: Your message was blocked (policy violation)

--- User Leaves ---
>>> Bob left 'Backend Developers' (online: 2)
  [Alice] received from System: Bob has left the chat
  [Charlie] received from System: Bob has left the chat
```

```
[Alice] sends: Bob just left, meeting canceled.
  [Charlie] received from Alice: Bob just left, meeting canceled.

--- Online Users ---
Online: [Alice, Charlie]
```

Notice that no User object references another User. All communication goes through the ChatRoom mediator. The mediator handles routing, filtering, and notifications.

Python Implementation: Event Mediator for Microservices

Event-Driven Service Coordination

```python
from datetime import datetime
from typing import Dict, List, Callable, Any


class EventMediator:
    """Central mediator that coordinates events between services."""

    def __init__(self):
        self._handlers: Dict[str, List[Callable]] = {}
        self._services: Dict[str, object] = {}
        self._event_log: List[dict] = []

    def register_service(self, name, service):
        self._services[name] = service
        service.mediator = self
        print(f"[Mediator] Registered service: {name}")

    def subscribe(self, event_type, handler):
        if event_type not in self._handlers:
            self._handlers[event_type] = []
        self._handlers[event_type].append(handler)

    def publish(self, event_type, data, source="unknown"):
        self._event_log.append({
            "event": event_type,
            "source": source,
            "timestamp": datetime.now().isoformat(),
            "data": str(data)
        })
```

```
        handlers = self._handlers.get(event_type, [])
        print(f"\n[Mediator] Event '{event_type}' from {source} "
              f"-> {len(handlers)} handler(s)")

        for handler in handlers:
            try:
                handler(data)
            except Exception as e:
                print(f"[Mediator] Handler error: {e}")

    def get_service(self, name):
        return self._services.get(name)

    def get_event_log(self):
        return self._event_log


class OrderService:
    """Handles order creation and management."""

    def __init__(self):
        self.mediator = None
        self.orders = {}
        self._next_id = 1000

    def setup(self):
        self.mediator.subscribe("payment.completed", self._on_payment)
        self.mediator.subscribe("inventory.reserved", self._on_reserved)

    def create_order(self, customer, items, total):
        order_id = self._next_id
        self._next_id += 1

        self.orders[order_id] = {
            "id": order_id,
            "customer": customer,
            "items": items,
            "total": total,
            "status": "created"
        }

        print(f"  [Order] Created order #{order_id} for {customer}")
        self.mediator.publish("order.created", {
            "order_id": order_id,
            "items": items,
            "total": total,
            "customer": customer
        }, source="OrderService")

        return order_id

    def _on_payment(self, data):
        order_id = data["order_id"]
        if order_id in self.orders:
```

```
            self.orders[order_id]["status"] = "paid"
            print(f"  [Order] Order #{order_id} marked as paid")

    def _on_reserved(self, data):
        order_id = data["order_id"]
        if order_id in self.orders:
            self.orders[order_id]["status"] = "reserved"
            print(f"  [Order] Order #{order_id} inventory reserved")


class InventoryService:
    """Manages product stock levels."""

    def __init__(self):
        self.mediator = None
        self.stock = {
            "WIDGET-A": 50,
            "WIDGET-B": 30,
            "GADGET-X": 10
        }

    def setup(self):
        self.mediator.subscribe("order.created", self._on_order_created)

    def _on_order_created(self, data):
        order_id = data["order_id"]
        items = data["items"]
        all_available = True

        for item, qty in items.items():
            available = self.stock.get(item, 0)
            if available >= qty:
                self.stock[item] -= qty
                print(f"  [Inventory] Reserved {qty}x {item} "
                      f"(remaining: {self.stock[item]})")
            else:
                print(f"  [Inventory] INSUFFICIENT {item}: "
                      f"need {qty}, have {available}")
                all_available = False

        if all_available:
            self.mediator.publish("inventory.reserved", {
                "order_id": order_id,
                "status": "reserved"
            }, source="InventoryService")
        else:
            self.mediator.publish("inventory.failed", {
                "order_id": order_id,
                "reason": "insufficient stock"
            }, source="InventoryService")


class PaymentService:
    """Processes payments."""
```

```
    def __init__(self):
        self.mediator = None
        self.payments = {}

    def setup(self):
        self.mediator.subscribe("inventory.reserved",
                                self._on_inventory_reserved)

    def _on_inventory_reserved(self, data):
        order_id = data["order_id"]
        print(f"  [Payment] Processing payment for order #{order_id}")

        # Simulate payment processing
        self.payments[order_id] = {"status": "completed"}

        self.mediator.publish("payment.completed", {
            "order_id": order_id,
            "status": "completed"
        }, source="PaymentService")


class NotificationService:
    """Sends notifications to customers."""

    def __init__(self):
        self.mediator = None
        self.sent = []

    def setup(self):
        self.mediator.subscribe("order.created", self._on_order)
        self.mediator.subscribe("payment.completed", self._on_payment)
        self.mediator.subscribe("inventory.failed", self._on_inv_fail)

    def _on_order(self, data):
        msg = f"Order #{data['order_id']} received for {data['customer']}"
        self.sent.append(msg)
        print(f"  [Notify] EMAIL: {msg}")

    def _on_payment(self, data):
        msg = f"Payment confirmed for order #{data['order_id']}"
        self.sent.append(msg)
        print(f"  [Notify] EMAIL: {msg}")

    def _on_inv_fail(self, data):
        msg = (f"Order #{data['order_id']} cannot be fulfilled: "
               f"{data['reason']}")
        self.sent.append(msg)
        print(f"  [Notify] EMAIL: {msg}")


class AnalyticsService:
    """Tracks business events."""

    def __init__(self):
        self.mediator = None
```

```
        self.events = []

    def setup(self):
        self.mediator.subscribe("order.created", self._track)
        self.mediator.subscribe("payment.completed", self._track)
        self.mediator.subscribe("inventory.failed", self._track)

    def _track(self, data):
        self.events.append(data)
        print(f"  [Analytics] Tracked event (total: {len(self.events)})")
```

Wiring Everything Together

```python
# Create the mediator
mediator = EventMediator()

# Create services
orders = OrderService()
inventory = InventoryService()
payments = PaymentService()
notifications = NotificationService()
analytics = AnalyticsService()

# Register all services with the mediator
mediator.register_service("orders", orders)
mediator.register_service("inventory", inventory)
mediator.register_service("payments", payments)
mediator.register_service("notifications", notifications)
mediator.register_service("analytics", analytics)

# Subscribe to events
orders.setup()
inventory.setup()
payments.setup()
notifications.setup()
analytics.setup()

# Trigger a successful order flow
print("\n" + "=" * 60)
print("SCENARIO 1: Successful Order")
print("=" * 60)
order_id = orders.create_order(
    customer="Alice",
    items={"WIDGET-A": 2, "GADGET-X": 1},
    total=149.99
)

# Trigger an order that fails due to insufficient stock
print("\n" + "=" * 60)
print("SCENARIO 2: Failed Order (insufficient stock)")
```

```
print("=" * 60)
orders.create_order(
    customer="Bob",
    items={"GADGET-X": 100},  # only 9 left
    total=999.99
)

# Show event log
print("\n" + "=" * 60)
print("EVENT LOG")
print("=" * 60)
for entry in mediator.get_event_log():
    print(f"  {entry['event']:25s} from {entry['source']}")
```

Output:

```
[Mediator] Registered service: orders
[Mediator] Registered service: inventory
[Mediator] Registered service: payments
[Mediator] Registered service: notifications
[Mediator] Registered service: analytics

============================================================
SCENARIO 1: Successful Order
============================================================
  [Order] Created order #1000 for Alice

[Mediator] Event 'order.created' from OrderService -> 3 handler(s)
  [Inventory] Reserved 2x WIDGET-A (remaining: 48)
  [Inventory] Reserved 1x GADGET-X (remaining: 9)

[Mediator] Event 'inventory.reserved' from InventoryService -> 2 handler(s)
  [Order] Order #1000 inventory reserved
  [Payment] Processing payment for order #1000

[Mediator] Event 'payment.completed' from PaymentService -> 3 handler(s)
  [Order] Order #1000 marked as paid
  [Notify] EMAIL: Payment confirmed for order #1000
  [Analytics] Tracked event (total: 2)
  [Notify] EMAIL: Order #1000 received for Alice
  [Analytics] Tracked event (total: 1)

============================================================
SCENARIO 2: Failed Order (insufficient stock)
============================================================
  [Order] Created order #1001 for Bob

[Mediator] Event 'order.created' from OrderService -> 3 handler(s)
  [Inventory] INSUFFICIENT GADGET-X: need 100, have 9

[Mediator] Event 'inventory.failed' from InventoryService -> 2 handler(s)
  [Notify] EMAIL: Order #1001 cannot be fulfilled: insufficient stock
  [Analytics] Tracked event (total: 3)
```

```
  [Notify] EMAIL: Order #1001 received for Bob
  [Analytics] Tracked event (total: 2)

============================================================
EVENT LOG
============================================================
  order.created             from OrderService
  inventory.reserved        from InventoryService
  payment.completed         from PaymentService
  order.created             from OrderService
  inventory.failed          from InventoryService
```

No service references another service directly. OrderService does not import InventoryService. PaymentService does not know about NotificationService. They all communicate through the mediator.

Mediator vs Observer

These patterns are frequently confused. Here is the difference:

```
+-------------------+-----------------------------------+-----------------------------...
| Aspect            | Mediator                          | Observer                    ...
+-------------------+-----------------------------------+-----------------------------...
| Direction         | Bidirectional (components talk    | Unidirectional (subject noti...
|                   | to mediator, mediator talks back) | observers)                  ...
+-------------------+-----------------------------------+-----------------------------...
| Knowledge         | Mediator knows all components     | Subject does not know observ...
|                   |                                   | details                     ...
+-------------------+-----------------------------------+-----------------------------...
| Logic location    | Coordination logic in mediator    | No coordination logic; obser...
|                   |                                   | react independently         ...
+-------------------+-----------------------------------+-----------------------------...
| Purpose           | Reduce coupling between peers     | Notify interested parties of...
|                   |                                   | state changes               ...
+-------------------+-----------------------------------+-----------------------------...
| Components        | Peers (equal participants)        | Subject and subscribers     ...
|                   |                                   | (hierarchy)                 ...
+-------------------+-----------------------------------+-----------------------------...
```

Observer says: "When X changes, notify everyone who cares." There is no logic about what to do with the notification.

Mediator says: "When X happens, here is exactly what Y and Z should do." The mediator contains business logic about coordination.

In practice, a Mediator often uses Observer internally (publish/subscribe for routing) but adds coordination logic on top.

Before vs After Comparison

Before: Direct Dependencies

```python
class OrderService:
    def __init__(self, inventory, payment, notifications, analytics):
        self.inventory = inventory      # direct dependency
        self.payment = payment          # direct dependency
        self.notifications = notifications  # direct dependency
        self.analytics = analytics      # direct dependency

    def create_order(self, data):
        self.inventory.reserve(data)
        self.payment.charge(data)
        self.notifications.send(data)
        self.analytics.track(data)
        # Adding shipping means changing this class
```

After: Mediator

```python
class OrderService:
    def __init__(self, mediator):
        self.mediator = mediator  # single dependency

    def create_order(self, data):
        self.mediator.publish("order.created", data)
        # Adding shipping means adding a handler to the mediator
        # This class does not change
```

When to Use / When NOT to Use

Use Mediator When

- Multiple components communicate in complex ways (many-to-many)

- You want to reuse components independently without their communication partners
- Adding new components should not require modifying existing ones
- Communication logic should be centralized and easy to change
- You need audit logging of all inter-component communication

Do NOT Use Mediator When

- Components have simple, one-to-one relationships (direct calls are fine)
- The mediator would become a massive "God Object" doing too much
- Real-time performance is critical (the indirection adds latency)
- The coordination logic is trivial (adding a mediator is over-engineering)
- You only have 2-3 components (not enough complexity to justify it)

Common Mistakes

Mistake 1: God Mediator

```python
# WRONG: mediator does ALL the business logic
class GodMediator:
    def handle_order(self, data):
        self.check_inventory(data)      # business logic here
        self.calculate_tax(data)         # and here
        self.process_payment(data)       # and here
        self.generate_invoice(data)      # and here
        self.send_notification(data)     # and here
        # Mediator becomes a 5000-line monolith

# RIGHT: mediator only routes, services contain logic
class OrderMediator:
    def handle_order(self, data):
        self.publish("order.created", data)
        # Each service handles its own logic
```

Mistake 2: Components Bypassing the Mediator

```java
// WRONG: direct communication defeats the pattern
public class UserService {
    private NotificationService notifier; // direct reference!
```

```
    public void createUser(User user) {
        // ... create user ...
        notifier.sendWelcome(user); // bypassing mediator!
    }
}
```

Mistake 3: Circular Event Chains

```python
# WRONG: A triggers B, B triggers A -> infinite loop
mediator.subscribe("order.updated", payment_service.handle)
mediator.subscribe("payment.updated", order_service.handle)
# order.updated -> payment.updated -> order.updated -> ...
```

Fix: Use event naming that distinguishes commands from responses, or add loop detection to the mediator.

Best Practices

1. Keep the mediator thin. It should coordinate, not compute. Business logic belongs in the services, not the mediator.
2. Use event naming conventions. Adopt a pattern like `entity.action` (order.created, payment.completed) to make events discoverable.
3. Log all mediated communication. Since everything goes through the mediator, it is the perfect place for audit trails and debugging.
4. Prevent circular events. Track event chains and break cycles. Consider maximum chain depth limits.
5. Consider splitting large mediators. If one mediator handles 50 event types, split it into domain-specific mediators (OrderMediator, UserMediator).
6. Make the mediator replaceable. Define the mediator as an interface so you can swap implementations (in-memory for testing, message queue for production).

Quick Summary

The Mediator pattern replaces many-to-many direct dependencies with many-to-one relationships through a central coordinator. Components communicate exclusively through the mediator, making them independent and reusable. The mediator encapsulates the coordination logic that would otherwise be scattered across components.

```
Problem:  N components with direct dependencies create N*(N-1)/2 connections.
Solution: Route all communication through a central mediator.
Key:      Components know only the mediator, never each other.
```

Key Points

- Mediator centralizes complex communication between multiple objects
- It converts many-to-many relationships into many-to-one (each to mediator)
- Components (colleagues) are decoupled from each other and only know the mediator
- The air traffic controller is the classic analogy: planes talk to the tower, not each other
- Mediator vs Observer: Mediator adds coordination logic; Observer just broadcasts
- Watch out for the God Mediator anti-pattern: keep coordination logic thin
- Use event naming conventions and logging for debuggability
- Adding new components requires changing only the mediator, not existing components

---

Practice Questions

1. In the air traffic controller analogy, what happens when the tower (mediator) goes down? How does this translate to software systems, and what can you do about it?
2. How would you prevent the God Mediator anti-pattern in a system with 20+ event types? Describe a concrete approach.
3. Explain the difference between Mediator and Observer with a concrete example of each solving the same problem differently.

4. You have 4 microservices that currently communicate directly. Adding a 5th service requires changing all 4. Describe how a Mediator would solve this, step by step.

5. In the Python EventMediator example, what would happen if the InventoryService published "order.created" instead of "inventory.reserved"? How would you prevent such bugs?

Exercises

Exercise 1: Smart Home Mediator

Build a smart home controller:

- Components: Thermostat, Lights, SecuritySystem, MusicPlayer, CoffeeMaker
- Rules coordinated by the mediator:
  - When SecuritySystem detects "away mode", turn off Lights and Thermostat, stop Music
  - When SecuritySystem detects "home mode", turn on Lights, set Thermostat to 72F
  - When Thermostat goes above 80F, turn off CoffeeMaker (heat reduction)
- No component should reference another directly
- Demonstrate adding a new component (Blinds) by only modifying the mediator

Exercise 2: Form Validation Mediator

Create a form validation system:

- Fields: UsernameField, EmailField, PasswordField, ConfirmPasswordField, SubmitButton
- Rules:
  - SubmitButton is disabled until all fields are valid
  - ConfirmPasswordField validates against PasswordField (must match)
  - EmailField shows a warning if username part of email does not match UsernameField
- All validation logic lives in the mediator, not in the fields
- Fields only emit "changed" events and receive "setError" or "clearError" commands

Exercise 3: Microservice Orchestrator

Build an order fulfillment orchestrator:

- Services: OrderService, WarehouseService, PaymentService, ShippingService, NotificationService
- Flow: Order -> Warehouse reserves -> Payment charges -> Shipping creates label -> Notification sends confirmation
- If any step fails, previous steps must be compensated (reverse order)
- The mediator tracks the state of each order through the pipeline
- Add logging that shows the complete event chain for debugging

---

What Is Next?

The next chapter introduces the Iterator Pattern, which provides a way to access elements of a collection sequentially without exposing the underlying structure. Where Mediator centralizes communication between components, Iterator standardizes how you traverse them. Both patterns reduce coupling, but in different dimensions.

Chapter 21: Iterator Pattern -- Sequential Access Without Exposing Internals

What You Will Learn

- What the Iterator pattern is and why it matters for clean collection traversal
- How to implement custom iterators in Java with `Iterable` and `Iterator`
- How to leverage Java's Stream API for modern iteration
- How to implement Python's `__iter__` and `__next__` protocols
- How to use Python generators for elegant, memory-efficient iteration
- Real-world use case: paginated database results

Why This Chapter Matters

Every backend system processes collections of data: users from a database, messages from a queue, files from a directory, events from a log. The Iterator pattern defines how to traverse these collections without exposing their internal structure.

You use iterators every day. Every `for` loop in Java and Python relies on the Iterator pattern under the hood. Understanding how iterators work lets you build custom collections that integrate seamlessly with language features, handle datasets too large for memory, and create lazy evaluation pipelines.

The Problem

You have a `UserRepository` backed by a database. You need to process all 100,000 users, but loading them all into a list at once consumes too much memory.

Without Iterator, your options are ugly:

```java
// Option 1: Load everything into memory (dangerous for large datasets)
List<User> allUsers = repo.findAll(); // 100,000 users in memory!
for (User user : allUsers) {
    process(user);
}

// Option 2: Manual pagination (exposes internals)
int page = 0;
int pageSize = 100;
while (true) {
    List<User> batch = repo.findPage(page, pageSize);
    if (batch.isEmpty()) break;
    for (User user : batch) {
        process(user);
    }
    page++;
}
// Every consumer must know about pagination!
```

Option 1 can crash with OutOfMemoryError. Option 2 forces every caller to handle pagination logic. The caller needs to know about pages, sizes, and when to stop.

With Iterator, the caller just writes:

```java
for (User user : repo) {
    process(user);
}
// Pagination is hidden inside the iterator.
```

The Solution: Iterator Pattern

```
+--------------------+          +--------------------+
|   <<interface>>    |          |   <<interface>>    |
|    Iterable<T>     |          |    Iterator<T>     |
+--------------------+          +--------------------+
| + iterator():      |--------->| + hasNext(): bool  |
|   Iterator<T>      |          | + next(): T        |
+--------------------+          +--------------------+
        ^                               ^
```

```
        |                                |
+--------------------+          +--------------------+
| ConcreteCollection|          | ConcreteIterator  |
+--------------------+          +--------------------+
| - elements[]      |          | - position: int   |
| + iterator()      |          | - collection      |
+--------------------+          | + hasNext()       |
                                | + next()          |
                                +--------------------+
```

Key participants:

- Iterator: interface with `hasNext()` and `next()` methods
- Iterable: interface that returns an Iterator (enables for-each loops)
- ConcreteIterator: implements traversal logic for a specific collection
- ConcreteCollection: stores elements and creates iterators

Java Implementation: Custom Collection

A Filtered, Sorted Task List

```java
import java.util.Iterator;
import java.util.List;
import java.util.ArrayList;
import java.util.NoSuchElementException;

public class Task {
    private final String name;
    private final String priority;  // "high", "medium", "low"
    private final boolean completed;

    public Task(String name, String priority, boolean completed) {
        this.name = name;
        this.priority = priority;
        this.completed = completed;
    }

    public String getName() { return name; }
    public String getPriority() { return priority; }
    public boolean isCompleted() { return completed; }

    @Override
    public String toString() {
        String status = completed ? "done" : "todo";
```

```
        return String.format("[%s] %s (%s)", priority.toUpperCase(),
                name, status);
    }
}
```

The Custom Iterator

```java
public class PriorityTaskIterator implements Iterator<Task> {
    private final List<Task> tasks;
    private final String priorityFilter;
    private final boolean skipCompleted;
    private int position;

    public PriorityTaskIterator(List<Task> tasks, String priorityFilter,
                                boolean skipCompleted) {
        this.tasks = tasks;
        this.priorityFilter = priorityFilter;
        this.skipCompleted = skipCompleted;
        this.position = 0;
        advanceToNext(); // position at first valid element
    }

    @Override
    public boolean hasNext() {
        return position < tasks.size();
    }

    @Override
    public Task next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        Task task = tasks.get(position);
        position++;
        advanceToNext(); // move to next valid element
        return task;
    }

    private void advanceToNext() {
        while (position < tasks.size()) {
            Task task = tasks.get(position);
            boolean matchesPriority = priorityFilter == null ||
                    task.getPriority().equals(priorityFilter);
            boolean matchesCompleted = !skipCompleted ||
                    !task.isCompleted();

            if (matchesPriority && matchesCompleted) {
                return; // found a valid element
            }
            position++;
```

```
        }
    }
}
```

The Iterable Collection

```java
public class TaskList implements Iterable<Task> {
    private final List<Task> tasks = new ArrayList<>();

    public void add(Task task) {
        tasks.add(task);
    }

    // Default iterator: all tasks
    @Override
    public Iterator<Task> iterator() {
        return tasks.iterator();
    }

    // Custom iterator: filter by priority
    public Iterable<Task> byPriority(String priority) {
        return () -> new PriorityTaskIterator(tasks, priority, false);
    }

    // Custom iterator: only pending tasks
    public Iterable<Task> pending() {
        return () -> new PriorityTaskIterator(tasks, null, true);
    }

    // Custom iterator: pending tasks of a specific priority
    public Iterable<Task> pendingByPriority(String priority) {
        return () -> new PriorityTaskIterator(tasks, priority, true);
    }

    public int size() {
        return tasks.size();
    }
}
```

Demo

```java
public class IteratorDemo {
    public static void main(String[] args) {
        TaskList tasks = new TaskList();
        tasks.add(new Task("Deploy to production", "high", false));
        tasks.add(new Task("Write unit tests", "high", true));
```

```
        tasks.add(new Task("Update documentation", "medium", false));
        tasks.add(new Task("Fix login bug", "high", false));
        tasks.add(new Task("Refactor database layer", "low", false));
        tasks.add(new Task("Code review PR #42", "medium", true));
        tasks.add(new Task("Setup CI pipeline", "medium", false));

        // Default: iterate all tasks
        System.out.println("=== All Tasks ===");
        for (Task task : tasks) {
            System.out.println("  " + task);
        }

        // Filter: high priority only
        System.out.println("\n=== High Priority ===");
        for (Task task : tasks.byPriority("high")) {
            System.out.println("  " + task);
        }

        // Filter: pending tasks only
        System.out.println("\n=== Pending Only ===");
        for (Task task : tasks.pending()) {
            System.out.println("  " + task);
        }

        // Filter: pending high priority
        System.out.println("\n=== Pending High Priority ===");
        for (Task task : tasks.pendingByPriority("high")) {
            System.out.println("  " + task);
        }
    }
}
```

Output:

```
=== All Tasks ===
  [HIGH] Deploy to production (todo)
  [HIGH] Write unit tests (done)
  [MEDIUM] Update documentation (todo)
  [HIGH] Fix login bug (todo)
  [LOW] Refactor database layer (todo)
  [MEDIUM] Code review PR #42 (done)
  [MEDIUM] Setup CI pipeline (todo)

=== High Priority ===
  [HIGH] Deploy to production (todo)
  [HIGH] Write unit tests (done)
  [HIGH] Fix login bug (todo)

=== Pending Only ===
  [HIGH] Deploy to production (todo)
  [MEDIUM] Update documentation (todo)
  [HIGH] Fix login bug (todo)
  [LOW] Refactor database layer (todo)
```

```
  [MEDIUM] Setup CI pipeline (todo)

=== Pending High Priority ===
  [HIGH] Deploy to production (todo)
  [HIGH] Fix login bug (todo)
```

Notice that the for-each loop works with every custom iterator because they implement `Iterable`. The caller does not know or care about the filtering logic.

Java Stream API: Modern Iteration

Java 8+ Streams provide a functional approach to iteration with built-in filtering, mapping, and reducing.

```java
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamDemo {
    public static void main(String[] args) {
        List<Task> tasks = Arrays.asList(
            new Task("Deploy to production", "high", false),
            new Task("Write unit tests", "high", true),
            new Task("Update documentation", "medium", false),
            new Task("Fix login bug", "high", false),
            new Task("Refactor database layer", "low", false),
            new Task("Code review PR #42", "medium", true),
            new Task("Setup CI pipeline", "medium", false)
        );

        // Filter and collect
        System.out.println("=== Pending High Priority (Stream) ===");
        List<Task> urgent = tasks.stream()
                .filter(t -> !t.isCompleted())
                .filter(t -> t.getPriority().equals("high"))
                .collect(Collectors.toList());

        urgent.forEach(t -> System.out.println("  " + t));

        // Count by priority
        System.out.println("\n=== Task Counts ===");
        tasks.stream()
                .collect(Collectors.groupingBy(
                        Task::getPriority, Collectors.counting()))
                .forEach((priority, count) ->
```

```
                        System.out.printf("  %s: %d%n", priority, count));

        // Map to names
        System.out.println("\n=== Pending Task Names ===");
        String names = tasks.stream()
                .filter(t -> !t.isCompleted())
                .map(Task::getName)
                .collect(Collectors.joining(", "));
        System.out.println("  " + names);

        // Find first urgent task
        tasks.stream()
                .filter(t -> !t.isCompleted())
                .filter(t -> t.getPriority().equals("high"))
                .findFirst()
                .ifPresent(t -> System.out.println(
                        "\nFirst urgent: " + t.getName()));
    }
}
```

Output:

```
=== Pending High Priority (Stream) ===
  [HIGH] Deploy to production (todo)
  [HIGH] Fix login bug (todo)

=== Task Counts ===
  high: 3
  medium: 3
  low: 1

=== Pending Task Names ===
  Deploy to production, Update documentation, Fix login bug, Refactor database layer, S...

First urgent: Deploy to production
```

Streams are lazy: they do not process elements until a terminal operation (like `collect` or `forEach`) is called. This makes them efficient for large datasets.

Python Implementation: iter and next

Custom Iterator Protocol

```python
class Task:
    def __init__(self, name, priority, completed=False):
        self.name = name
        self.priority = priority
        self.completed = completed

    def __repr__(self):
        status = "done" if self.completed else "todo"
        return f"[{self.priority.upper()}] {self.name} ({status})"


class TaskIterator:
    """Custom iterator with filtering support."""

    def __init__(self, tasks, priority_filter=None, skip_completed=False):
        self._tasks = tasks
        self._priority_filter = priority_filter
        self._skip_completed = skip_completed
        self._index = 0

    def __iter__(self):
        return self

    def __next__(self):
        while self._index < len(self._tasks):
            task = self._tasks[self._index]
            self._index += 1

            # Apply filters
            if self._priority_filter and \
               task.priority != self._priority_filter:
                continue
            if self._skip_completed and task.completed:
                continue

            return task

        raise StopIteration


class TaskList:
    """Collection that provides multiple iteration strategies."""

    def __init__(self):
        self._tasks = []

    def add(self, task):
        self._tasks.append(task)

    def __iter__(self):
        return iter(self._tasks)

    def __len__(self):
        return len(self._tasks)
```

```
    def by_priority(self, priority):
        return IterableView(self._tasks, priority_filter=priority)

    def pending(self):
        return IterableView(self._tasks, skip_completed=True)

    def pending_by_priority(self, priority):
        return IterableView(self._tasks, priority_filter=priority,
                            skip_completed=True)


class IterableView:
    """A filtered view of a task list that is itself iterable."""

    def __init__(self, tasks, priority_filter=None, skip_completed=False):
        self._tasks = tasks
        self._priority_filter = priority_filter
        self._skip_completed = skip_completed

    def __iter__(self):
        return TaskIterator(self._tasks, self._priority_filter,
                            self._skip_completed)


# Usage
tasks = TaskList()
tasks.add(Task("Deploy to production", "high"))
tasks.add(Task("Write unit tests", "high", completed=True))
tasks.add(Task("Update documentation", "medium"))
tasks.add(Task("Fix login bug", "high"))
tasks.add(Task("Refactor database layer", "low"))
tasks.add(Task("Code review PR #42", "medium", completed=True))

print("=== All Tasks ===")
for task in tasks:
    print(f"  {task}")

print("\n=== High Priority ===")
for task in tasks.by_priority("high"):
    print(f"  {task}")

print("\n=== Pending High Priority ===")
for task in tasks.pending_by_priority("high"):
    print(f"  {task}")
```

Output:

```
=== All Tasks ===
  [HIGH] Deploy to production (todo)
  [HIGH] Write unit tests (done)
  [MEDIUM] Update documentation (todo)
  [HIGH] Fix login bug (todo)
```

```
  [LOW] Refactor database layer (todo)
  [MEDIUM] Code review PR #42 (done)

=== High Priority ===
  [HIGH] Deploy to production (todo)
  [HIGH] Write unit tests (done)
  [HIGH] Fix login bug (todo)

=== Pending High Priority ===
  [HIGH] Deploy to production (todo)
  [HIGH] Fix login bug (todo)
```

Python Generators: Elegant Iteration

Generators are Python's most powerful iteration feature. A generator function uses `yield` instead of `return`. It pauses execution after each yield and resumes when the next value is requested.

```python
def fibonacci(limit):
    """Generate Fibonacci numbers up to limit."""
    a, b = 0, 1
    while a < limit:
        yield a
        a, b = b, a + b

# Use in a for loop
print("Fibonacci up to 100:")
for num in fibonacci(100):
    print(f"  {num}", end="")
print()

# Use with list comprehension
fibs = [n for n in fibonacci(50)]
print(f"As list: {fibs}")
```

Output:

```
Fibonacci up to 100:
  0  1  1  2  3  5  8  13  21  34  55  89
As list: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Generator-Based Task Filtering

```python
class TaskList:
    def __init__(self):
        self._tasks = []

    def add(self, task):
        self._tasks.append(task)

    def __iter__(self):
        return iter(self._tasks)

    def by_priority(self, priority):
        """Generator that yields tasks matching the priority."""
        for task in self._tasks:
            if task.priority == priority:
                yield task

    def pending(self):
        """Generator that yields only incomplete tasks."""
        for task in self._tasks:
            if not task.completed:
                yield task

    def pending_by_priority(self, priority):
        """Compose generators: pending AND matching priority."""
        for task in self.pending():
            if task.priority == priority:
                yield task

    def summary(self):
        """Generator that yields summary strings."""
        from collections import Counter
        counts = Counter(t.priority for t in self._tasks if not t.completed)
        for priority, count in counts.most_common():
            yield f"{priority}: {count} pending"


tasks = TaskList()
tasks.add(Task("Deploy", "high"))
tasks.add(Task("Tests", "high", completed=True))
tasks.add(Task("Docs", "medium"))
tasks.add(Task("Bug fix", "high"))
tasks.add(Task("Refactor", "low"))

print("=== Summary ===")
for line in tasks.summary():
    print(f"  {line}")

print("\n=== Pending (generator) ===")
pending = tasks.pending()  # no work done yet
print(f"  Type: {type(pending)}")
print(f"  First: {next(pending)}")
print(f"  Second: {next(pending)}")
# Remaining items are never generated -- lazy!
```

Output:

```
=== Summary ===
  high: 1 pending
  medium: 1 pending
  low: 1 pending

=== Pending (generator) ===
  Type: <class 'generator'>
  First: [HIGH] Deploy (todo)
  Second: [MEDIUM] Docs (todo)
```

Generators are lazy: they compute values on demand. This is critical for large datasets.

Real-World Use Case: Paginated Database Results

This is the most practical application of iterators in backend development.

Java: Paginated Iterator

```java
import java.util.*;

// Simulated database
class UserDatabase {
    private final List<String> users = new ArrayList<>();

    public UserDatabase() {
        for (int i = 1; i <= 47; i++) {
            users.add("User-" + i);
        }
    }

    public List<String> fetchPage(int offset, int limit) {
        int start = Math.min(offset, users.size());
        int end = Math.min(offset + limit, users.size());
        System.out.printf("    [DB] Fetching offset=%d limit=%d "
                + "(returned %d rows)%n", offset, limit, end - start);
        return users.subList(start, end);
    }

    public int count() {
        return users.size();
```

```
    }
}

public class PaginatedIterator implements Iterator<String> {
    private final UserDatabase db;
    private final int pageSize;
    private List<String> currentPage;
    private int pageIndex;     // position within current page
    private int globalOffset;  // position in the full dataset
    private boolean exhausted;

    public PaginatedIterator(UserDatabase db, int pageSize) {
        this.db = db;
        this.pageSize = pageSize;
        this.globalOffset = 0;
        this.pageIndex = 0;
        this.exhausted = false;
        fetchNextPage();
    }

    private void fetchNextPage() {
        currentPage = db.fetchPage(globalOffset, pageSize);
        pageIndex = 0;
        if (currentPage.isEmpty()) {
            exhausted = true;
        }
    }

    @Override
    public boolean hasNext() {
        if (exhausted) return false;
        if (pageIndex < currentPage.size()) return true;

        // Current page exhausted, try next page
        globalOffset += pageSize;
        fetchNextPage();
        return !exhausted;
    }

    @Override
    public String next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        return currentPage.get(pageIndex++);
    }
}

// Iterable wrapper
class PaginatedUsers implements Iterable<String> {
    private final UserDatabase db;
    private final int pageSize;

    public PaginatedUsers(UserDatabase db, int pageSize) {
        this.db = db;
```

```
        this.pageSize = pageSize;
    }

    @Override
    public Iterator<String> iterator() {
        return new PaginatedIterator(db, pageSize);
    }
}
```

```java
public class PaginationDemo {
    public static void main(String[] args) {
        UserDatabase db = new UserDatabase();

        System.out.println("=== Paginated Iteration (page size: 10) ===");
        PaginatedUsers users = new PaginatedUsers(db, 10);

        int count = 0;
        for (String user : users) {
            count++;
        }
        System.out.println("Processed " + count + " users");

        System.out.println("\n=== Small pages (page size: 15) ===");
        PaginatedUsers smallPages = new PaginatedUsers(db, 15);
        count = 0;
        for (String user : smallPages) {
            count++;
        }
        System.out.println("Processed " + count + " users");
    }
}
```

Output:

```
=== Paginated Iteration (page size: 10) ===
    [DB] Fetching offset=0 limit=10 (returned 10 rows)
    [DB] Fetching offset=10 limit=10 (returned 10 rows)
    [DB] Fetching offset=20 limit=10 (returned 10 rows)
    [DB] Fetching offset=30 limit=10 (returned 10 rows)
    [DB] Fetching offset=40 limit=10 (returned 7 rows)
    [DB] Fetching offset=50 limit=10 (returned 0 rows)
Processed 47 users

=== Small pages (page size: 15) ===
    [DB] Fetching offset=0 limit=15 (returned 15 rows)
    [DB] Fetching offset=15 limit=15 (returned 15 rows)
    [DB] Fetching offset=30 limit=15 (returned 15 rows)
    [DB] Fetching offset=45 limit=15 (returned 2 rows)
    [DB] Fetching offset=60 limit=15 (returned 0 rows)
Processed 47 users
```

The caller just writes `for (String user : users)`. The pagination is completely hidden.

Python: Generator-Based Pagination

```python
class UserDatabase:
    """Simulated database with 47 users."""

    def __init__(self):
        self._users = [f"User-{i}" for i in range(1, 48)]

    def fetch_page(self, offset, limit):
        page = self._users[offset:offset + limit]
        print(f"    [DB] Fetching offset={offset} limit={limit} "
              f"(returned {len(page)} rows)")
        return page

    def count(self):
        return len(self._users)


def paginated_users(db, page_size=10):
    """Generator that yields users one at a time, fetching in pages."""
    offset = 0
    while True:
        page = db.fetch_page(offset, page_size)
        if not page:
            return  # no more data

        for user in page:
            yield user

        offset += page_size


# Usage -- caller sees a simple loop
db = UserDatabase()

print("=== Paginated Iteration (page size: 10) ===")
count = 0
for user in paginated_users(db, page_size=10):
    count += 1
print(f"Processed {count} users")

print("\n=== Early termination (stop after 25) ===")
count = 0
for user in paginated_users(db, page_size=10):
    count += 1
    if count >= 25:
        break  # generator stops fetching pages!
print(f"Processed {count} users (remaining pages never fetched)")
```

Output:

```
=== Paginated Iteration (page size: 10) ===
    [DB] Fetching offset=0 limit=10 (returned 10 rows)
    [DB] Fetching offset=10 limit=10 (returned 10 rows)
    [DB] Fetching offset=20 limit=10 (returned 10 rows)
    [DB] Fetching offset=30 limit=10 (returned 10 rows)
    [DB] Fetching offset=40 limit=10 (returned 7 rows)
    [DB] Fetching offset=50 limit=10 (returned 0 rows)
Processed 47 users

=== Early termination (stop after 25) ===
    [DB] Fetching offset=0 limit=10 (returned 10 rows)
    [DB] Fetching offset=10 limit=10 (returned 10 rows)
    [DB] Fetching offset=20 limit=10 (returned 10 rows)
Processed 25 users (remaining pages never fetched)
```

Notice in the early termination case: the generator only fetched 3 pages (30 rows). It never fetched the remaining 17 users. Lazy evaluation means no wasted work.

Before vs After Comparison

Before: Client Handles Pagination

```python
# Every caller must implement pagination
offset = 0
page_size = 100
while True:
    page = db.fetch_page(offset, page_size)
    if not page:
        break
    for user in page:
        send_email(user)
    offset += page_size

# Duplicated everywhere users are processed
offset = 0
while True:
    page = db.fetch_page(offset, page_size)
    if not page:
        break
    for user in page:
        generate_report(user)
    offset += page_size
```

After: Iterator Hides Pagination

```python
# Simple and reusable
for user in paginated_users(db):
    send_email(user)

for user in paginated_users(db):
    generate_report(user)
```

When to Use / When NOT to Use

Use Iterator When

- You need to traverse a collection without exposing its internal structure
- The collection is too large to load entirely into memory
- You want multiple traversal strategies over the same collection
- You need lazy evaluation (compute values only when requested)
- You want to integrate custom collections with language for-each loops
- Paginated API or database access should be transparent to callers

Do NOT Use Iterator When

- A simple list or array suffices (do not over-engineer)
- You need random access by index (iterators are sequential)
- The collection is small and fits in memory easily
- You need to traverse backwards frequently (most iterators are forward-only)
- The overhead of iterator objects exceeds the benefit (very tight inner loops)

Common Mistakes

Mistake 1: Modifying Collection During Iteration

```java
// WRONG: ConcurrentModificationException
for (Task task : tasks) {
```

```
    if (task.isCompleted()) {
        tasks.remove(task); // modifying during iteration!
    }
}

// RIGHT: use Iterator.remove() or collect then remove
Iterator<Task> it = tasks.iterator();
while (it.hasNext()) {
    if (it.next().isCompleted()) {
        it.remove(); // safe removal through iterator
    }
}
```

Mistake 2: Exhausted Iterator Reuse

```python
# WRONG: generators are single-use
gen = paginated_users(db)
list1 = list(gen)  # consumes the generator
list2 = list(gen)  # empty! generator is exhausted

# RIGHT: create a new generator each time
list1 = list(paginated_users(db))
list2 = list(paginated_users(db))

# OR: make the container iterable (creates new iterator each time)
class Users:
    def __iter__(self):
        return paginated_users(db)  # fresh generator each time
```

Mistake 3: Not Handling StopIteration

```python
# WRONG: calling next() without checking
it = iter([1, 2])
print(next(it))  # 1
print(next(it))  # 2
print(next(it))  # StopIteration exception!

# RIGHT: use default value
print(next(it, None))  # None instead of exception

# Or use a for loop (handles StopIteration automatically)
for item in iter([1, 2]):
    print(item)
```

Best Practices

1. Implement Iterable, not just Iterator. In Java, implement `Iterable<T>` on your collection so it works with for-each loops. In Python, define `__iter__` on the collection and return a fresh iterator each time.
2. Use generators in Python. They are simpler than manual `__iter__`/`__next__` implementations and handle state management automatically.
3. Make iterators lazy. Fetch data only when `next()` is called, not during construction. This enables processing of datasets larger than memory.
4. Support early termination. Lazy iterators should stop fetching when the consumer stops consuming. Generators in Python handle this automatically.
5. Use Java Streams for functional operations. When you need filter-map-reduce operations, Streams are more readable than manual iterators.
6. Do not modify collections during iteration. Use `Iterator.remove()` in Java or build a new collection instead.
7. Document iterator guarantees. State whether the iterator is lazy or eager, single-use or reusable, and whether it supports concurrent modification.

Quick Summary

The Iterator pattern provides a standard way to traverse collections without exposing their internals. In Java, `Iterable` and `Iterator` interfaces enable for-each loops. In Python, `__iter__`/`__next__` and generators provide the same capability. The pattern is essential for paginated data access, lazy evaluation, and clean separation between collections and their traversal logic.

```
Problem:  Clients need to traverse collections but should not know internals.
Solution: Provide an iterator that yields elements one at a time.
Key:      Lazy evaluation enables processing datasets larger than memory.
```

Key Points

- Iterator decouples traversal logic from the collection's internal structure
- Java uses `Iterable<T>` and `Iterator<T>` interfaces (enabling for-each loops)
- Java Streams provide functional iteration: filter, map, reduce, collect
- Python uses `__iter__()` and `__next__()` methods (enabling for loops)
- Python generators (`yield`) are the most elegant way to implement iterators
- Generators are lazy: they compute values on demand, saving memory
- Paginated database access is the most common backend use case
- Generators are single-use: create a new one each time you need to iterate
- Never modify a collection while iterating over it

Practice Questions

1. What is the difference between `Iterable` and `Iterator` in Java? Why are they separate interfaces?
2. Explain why generators in Python are single-use. How would you make a class that can be iterated multiple times using a generator internally?
3. How does lazy evaluation in iterators help when processing a 10-million-row database table? What would happen without lazy evaluation?
4. Compare Java Streams with Python generators. What are two advantages of each?
5. In the paginated database example, what happens if a new row is inserted into the database while pagination is in progress? How would you handle this?

Exercises

Exercise 1: CSV File Iterator

Build a lazy CSV file reader:

- A `CsvIterator` that reads a CSV file line by line (not loading the entire file)

- Each iteration yields a dictionary mapping column headers to values
- Support filtering rows by a column value
- Support limiting the number of rows returned
- Test with a CSV file of at least 100 rows
- Demonstrate that memory usage stays constant regardless of file size

Exercise 2: Binary Tree Iterator

Implement three traversal iterators for a binary tree:

- `InOrderIterator`: left, root, right
- `PreOrderIterator`: root, left, right
- `PostOrderIterator`: left, right, root
- The tree should implement `Iterable` with a default order
- Provide methods to get each specific traversal: `tree.in_order()`, `tree.pre_order()`, `tree.post_order()`
- Build a tree with at least 7 nodes and show the output of each traversal

Exercise 3: API Cursor Pagination

Build an iterator for cursor-based API pagination:

- Simulate an API that returns `{"data": [...], "next_cursor": "abc123"}` or `{"data": [...], "next_cursor": null}` when done
- The iterator should automatically follow cursors until no more data
- Support a maximum number of API calls (to prevent infinite loops)
- Add a delay parameter between API calls (rate limiting)
- The caller should see a simple `for item in api_results` loop

What Is Next?

The next chapter introduces the Visitor Pattern, which lets you add new operations to existing class hierarchies without modifying them. Where Iterator standardizes how you traverse a structure, Visitor standardizes what you do with each element during traversal. The two patterns are natural partners: Iterator tells you what to visit next, and Visitor tells you what to do when you get there.

Chapter 22: Visitor Pattern -- Add Operations Without Modifying Classes

What You Will Learn

- What the Visitor pattern is and the problem it solves
- How double dispatch works and why it matters
- How to implement Visitor in Java with a tax calculation example
- How to implement Visitor in Python with AST (Abstract Syntax Tree) processing
- When the added complexity of Visitor is justified
- The trade-offs between Visitor and other approaches

Why This Chapter Matters

You have a stable class hierarchy: different types of products, AST nodes, document elements, or file system entries. Now you need to add new operations: calculate tax, export to JSON, validate, generate code, pretty-print. Each operation needs different logic for each type.

Without Visitor, you either add methods to every class (modifying many files) or write switch/if-else chains on type (violating the Open/Closed Principle). The Visitor pattern lets you add new operations by creating new classes, without touching existing ones.

The Problem

You are building an e-commerce system with different product types. Each type needs different tax calculation logic, different discount rules, and different shipping cost formulas.

Without Visitor, you have two bad options:

Option 1: Methods in each class (violates Open/Closed Principle)

```java
// Every time you add a new operation, you modify ALL classes
class Electronics {
    double calculateTax() { ... }
    double calculateShipping() { ... }
    String exportToJson() { ... }
    String generateReport() { ... }
    // New operation = modify this class AND Food AND Clothing AND ...
}

class Food {
    double calculateTax() { ... }     // different logic
    double calculateShipping() { ... } // different logic
    String exportToJson() { ... }
    String generateReport() { ... }
}

// Adding "calculateInsurance()" means modifying 5+ classes
```

Option 2: Type-checking chains (violates Open/Closed Principle differently)

```java
double calculateTax(Product p) {
    if (p instanceof Electronics) { return p.getPrice() * 0.15; }
    else if (p instanceof Food) { return p.getPrice() * 0.05; }
    else if (p instanceof Clothing) { return p.getPrice() * 0.08; }
    // Adding a new product type means updating ALL these chains
}
```

The Solution: Visitor Pattern

The Visitor pattern separates algorithms from the objects they operate on. Each new operation is a new Visitor class. Each element accepts a visitor and tells it what type it is (double dispatch).

```
+----------------------+         +-----------------------+
|   <<interface>>      |         |    <<interface>>      |
|     Element          |         |      Visitor          |
```

```
+----------------------+         +-----------------------+
| + accept(Visitor v)  |<------->| + visitElectronics()  |
+----------------------+         | + visitFood()         |
    ^       ^       ^            | + visitClothing()     |
    |       |       |            +-----------------------+
+-----+ +-----+ +------+             ^            ^
|Elec.| |Food | |Cloth.|        +--------+   +--------+
+-----+ +-----+ +------+        |TaxVisit|   |ShipVis.|
                                +--------+   +--------+

  element.accept(visitor)
    -> visitor.visitElectronics(this)   // double dispatch
```

How double dispatch works:

1. Client calls `element.accept(visitor)`
2. Element calls `visitor.visitElectronics(this)` (or whichever type it is)
3. The correct method runs based on BOTH the visitor type AND the element type

This is called "double dispatch" because the operation depends on two types: the visitor (which operation) and the element (which type of data).

```
Single Dispatch (normal method call):
  element.calculateTax()
  Dispatch based on: element type only

Double Dispatch (visitor):
  element.accept(taxVisitor)
    -> taxVisitor.visitElectronics(this)
  Dispatch based on: element type AND visitor type
```

Java Implementation: Tax Calculator

Define the Element and Visitor Interfaces

```java
// Visitor interface: one method per element type
public interface ProductVisitor<T> {
    T visitElectronics(Electronics electronics);
    T visitFood(Food food);
    T visitClothing(Clothing clothing);
    T visitDigitalService(DigitalService service);
}
```

```
// Element interface: accepts a visitor
public interface Product {
    String getName();
    double getPrice();
    <T> T accept(ProductVisitor<T> visitor);
}
```

Define the Element Classes

```java
public class Electronics implements Product {
    private final String name;
    private final double price;
    private final double weight;  // for shipping
    private final boolean imported;

    public Electronics(String name, double price, double weight,
                       boolean imported) {
        this.name = name;
        this.price = price;
        this.weight = weight;
        this.imported = imported;
    }

    @Override
    public String getName() { return name; }
    @Override
    public double getPrice() { return price; }
    public double getWeight() { return weight; }
    public boolean isImported() { return imported; }

    @Override
    public <T> T accept(ProductVisitor<T> visitor) {
        return visitor.visitElectronics(this);
    }
}

public class Food implements Product {
    private final String name;
    private final double price;
    private final boolean organic;
    private final boolean perishable;

    public Food(String name, double price, boolean organic,
                boolean perishable) {
        this.name = name;
        this.price = price;
        this.organic = organic;
        this.perishable = perishable;
    }
```

```
    @Override
    public String getName() { return name; }
    @Override
    public double getPrice() { return price; }
    public boolean isOrganic() { return organic; }
    public boolean isPerishable() { return perishable; }

    @Override
    public <T> T accept(ProductVisitor<T> visitor) {
        return visitor.visitFood(this);
    }
}

public class Clothing implements Product {
    private final String name;
    private final double price;
    private final String size;

    public Clothing(String name, double price, String size) {
        this.name = name;
        this.price = price;
        this.size = size;
    }

    @Override
    public String getName() { return name; }
    @Override
    public double getPrice() { return price; }
    public String getSize() { return size; }

    @Override
    public <T> T accept(ProductVisitor<T> visitor) {
        return visitor.visitClothing(this);
    }
}

public class DigitalService implements Product {
    private final String name;
    private final double price;
    private final String region;

    public DigitalService(String name, double price, String region) {
        this.name = name;
        this.price = price;
        this.region = region;
    }

    @Override
    public String getName() { return name; }
    @Override
    public double getPrice() { return price; }
    public String getRegion() { return region; }

    @Override
```

```
    public <T> T accept(ProductVisitor<T> visitor) {
        return visitor.visitDigitalService(this);
    }
}
```

Visitor 1: Tax Calculator

```java
public class TaxVisitor implements ProductVisitor<Double> {

    @Override
    public Double visitElectronics(Electronics e) {
        double tax = e.getPrice() * 0.15;  // 15% electronics tax
        if (e.isImported()) {
            tax += e.getPrice() * 0.05;    // 5% import duty
        }
        return tax;
    }

    @Override
    public Double visitFood(Food f) {
        if (f.isOrganic()) {
            return 0.0;  // organic food is tax-exempt
        }
        return f.getPrice() * 0.05;  // 5% food tax
    }

    @Override
    public Double visitClothing(Clothing c) {
        if (c.getPrice() < 50.0) {
            return 0.0;  // cheap clothing is tax-exempt
        }
        return c.getPrice() * 0.08;  // 8% clothing tax
    }

    @Override
    public Double visitDigitalService(DigitalService s) {
        // Digital services taxed by region
        return switch (s.getRegion()) {
            case "EU" -> s.getPrice() * 0.20;  // 20% VAT
            case "US" -> s.getPrice() * 0.07;  // 7% sales tax
            default -> s.getPrice() * 0.10;    // 10% default
        };
    }
}
```

Visitor 2: Shipping Calculator

```java
public class ShippingVisitor implements ProductVisitor<Double> {

    @Override
    public Double visitElectronics(Electronics e) {
        return e.getWeight() * 2.50;  // $2.50 per kg
    }

    @Override
    public Double visitFood(Food f) {
        if (f.isPerishable()) {
            return 15.00;  // flat rate for cold chain
        }
        return 5.00;  // flat rate for non-perishable
    }

    @Override
    public Double visitClothing(Clothing c) {
        return 3.00;  // flat rate for clothing
    }

    @Override
    public Double visitDigitalService(DigitalService s) {
        return 0.0;  // no shipping for digital services
    }
}
```

Visitor 3: Summary Report

```java
public class SummaryVisitor implements ProductVisitor<String> {

    @Override
    public String visitElectronics(Electronics e) {
        return String.format("ELECTRONICS: %s ($%.2f, %.1fkg%s)",
                e.getName(), e.getPrice(), e.getWeight(),
                e.isImported() ? ", imported" : "");
    }

    @Override
    public String visitFood(Food f) {
        return String.format("FOOD: %s ($%.2f%s%s)",
                f.getName(), f.getPrice(),
                f.isOrganic() ? ", organic" : "",
                f.isPerishable() ? ", perishable" : "");
    }

    @Override
    public String visitClothing(Clothing c) {
        return String.format("CLOTHING: %s ($%.2f, size %s)",
                c.getName(), c.getPrice(), c.getSize());
    }
```

```
    @Override
    public String visitDigitalService(DigitalService s) {
        return String.format("DIGITAL: %s ($%.2f, region: %s)",
                s.getName(), s.getPrice(), s.getRegion());
    }
}
```

Using the Visitors

```java
import java.util.List;

public class VisitorDemo {
    public static void main(String[] args) {
        List<Product> cart = List.of(
            new Electronics("Laptop", 999.99, 2.5, true),
            new Food("Organic Apples", 6.99, true, true),
            new Food("Rice (5kg)", 12.50, false, false),
            new Clothing("T-Shirt", 29.99, "M"),
            new Clothing("Winter Jacket", 189.99, "L"),
            new DigitalService("Cloud Storage", 9.99, "EU")
        );

        TaxVisitor taxCalc = new TaxVisitor();
        ShippingVisitor shipCalc = new ShippingVisitor();
        SummaryVisitor summary = new SummaryVisitor();

        double totalTax = 0;
        double totalShipping = 0;
        double totalPrice = 0;

        System.out.println("=== Shopping Cart ===");
        for (Product product : cart) {
            String desc = product.accept(summary);
            double tax = product.accept(taxCalc);
            double shipping = product.accept(shipCalc);

            System.out.printf("  %s%n", desc);
            System.out.printf("    Tax: $%.2f | Shipping: $%.2f%n",
                    tax, shipping);

            totalPrice += product.getPrice();
            totalTax += tax;
            totalShipping += shipping;
        }

        System.out.println("\n=== Totals ===");
        System.out.printf("  Subtotal:  $%.2f%n", totalPrice);
        System.out.printf("  Tax:       $%.2f%n", totalTax);
        System.out.printf("  Shipping:  $%.2f%n", totalShipping);
```

```
        System.out.printf("  Total:     $%.2f%n",
                totalPrice + totalTax + totalShipping);
    }
}
```

Output:

```
=== Shopping Cart ===
  ELECTRONICS: Laptop ($999.99, 2.5kg, imported)
    Tax: $200.00 | Shipping: $6.25
  FOOD: Organic Apples ($6.99, organic, perishable)
    Tax: $0.00 | Shipping: $15.00
  FOOD: Rice (5kg) ($12.50)
    Tax: $0.63 | Shipping: $5.00
  CLOTHING: T-Shirt ($29.99, size M)
    Tax: $0.00 | Shipping: $3.00
  CLOTHING: Winter Jacket ($189.99, size L)
    Tax: $15.20 | Shipping: $3.00
  DIGITAL: Cloud Storage ($9.99, region: EU)
    Tax: $2.00 | Shipping: $0.00

=== Totals ===
  Subtotal:  $1249.45
  Tax:       $217.83
  Shipping:  $32.25
  Total:     $1499.53
```

Adding a new operation (like insurance calculation) means creating a new `InsuranceVisitor` class. No existing product classes change. That is the power of Visitor.

Python Implementation: AST Visitor

Abstract Syntax Tree Processing

Compilers and interpreters are the classic use case for Visitor. An AST represents parsed code, and you need many operations: evaluate, type-check, optimize, pretty-print, compile.

```python
from abc import ABC, abstractmethod


# --- AST Node Types (Elements) ---
```

```
class ASTNode(ABC):
    @abstractmethod
    def accept(self, visitor):
        pass


class NumberNode(ASTNode):
    def __init__(self, value):
        self.value = value

    def accept(self, visitor):
        return visitor.visit_number(self)

    def __repr__(self):
        return f"Number({self.value})"


class BinaryOpNode(ASTNode):
    def __init__(self, operator, left, right):
        self.operator = operator
        self.left = left
        self.right = right

    def accept(self, visitor):
        return visitor.visit_binary_op(self)

    def __repr__(self):
        return f"BinaryOp({self.operator})"


class UnaryOpNode(ASTNode):
    def __init__(self, operator, operand):
        self.operator = operator
        self.operand = operand

    def accept(self, visitor):
        return visitor.visit_unary_op(self)


class VariableNode(ASTNode):
    def __init__(self, name):
        self.name = name

    def accept(self, visitor):
        return visitor.visit_variable(self)


class AssignmentNode(ASTNode):
    def __init__(self, name, expression):
        self.name = name
        self.expression = expression

    def accept(self, visitor):
        return visitor.visit_assignment(self)
```

```
class FunctionCallNode(ASTNode):
    def __init__(self, name, arguments):
        self.name = name
        self.arguments = arguments

    def accept(self, visitor):
        return visitor.visit_function_call(self)


# --- Visitor Interface ---

class ASTVisitor(ABC):
    @abstractmethod
    def visit_number(self, node): pass

    @abstractmethod
    def visit_binary_op(self, node): pass

    @abstractmethod
    def visit_unary_op(self, node): pass

    @abstractmethod
    def visit_variable(self, node): pass

    @abstractmethod
    def visit_assignment(self, node): pass

    @abstractmethod
    def visit_function_call(self, node): pass
```

Visitor 1: Evaluator

```python
import math


class EvaluatorVisitor(ASTVisitor):
    """Evaluates the AST and returns a numeric result."""

    def __init__(self):
        self.variables = {}
        self.functions = {
            "sqrt": math.sqrt,
            "abs": abs,
            "max": max,
            "min": min,
        }

    def visit_number(self, node):
        return node.value
```

```
    def visit_binary_op(self, node):
        left = node.left.accept(self)
        right = node.right.accept(self)

        ops = {
            "+": lambda a, b: a + b,
            "-": lambda a, b: a - b,
            "*": lambda a, b: a * b,
            "/": lambda a, b: a / b,
            "**": lambda a, b: a ** b,
        }

        if node.operator not in ops:
            raise ValueError(f"Unknown operator: {node.operator}")
        return ops[node.operator](left, right)

    def visit_unary_op(self, node):
        value = node.operand.accept(self)
        if node.operator == "-":
            return -value
        if node.operator == "+":
            return value
        raise ValueError(f"Unknown unary operator: {node.operator}")

    def visit_variable(self, node):
        if node.name not in self.variables:
            raise NameError(f"Undefined variable: {node.name}")
        return self.variables[node.name]

    def visit_assignment(self, node):
        value = node.expression.accept(self)
        self.variables[node.name] = value
        return value

    def visit_function_call(self, node):
        if node.name not in self.functions:
            raise NameError(f"Undefined function: {node.name}")
        args = [arg.accept(self) for arg in node.arguments]
        return self.functions[node.name](*args)
```

Visitor 2: Pretty Printer

```python
class PrettyPrintVisitor(ASTVisitor):
    """Converts AST back to a readable string."""

    def visit_number(self, node):
        return str(node.value)

    def visit_binary_op(self, node):
```

```
        left = node.left.accept(self)
        right = node.right.accept(self)
        return f"({left} {node.operator} {right})"

    def visit_unary_op(self, node):
        operand = node.operand.accept(self)
        return f"({node.operator}{operand})"

    def visit_variable(self, node):
        return node.name

    def visit_assignment(self, node):
        expr = node.expression.accept(self)
        return f"{node.name} = {expr}"

    def visit_function_call(self, node):
        args = ", ".join(arg.accept(self) for arg in node.arguments)
        return f"{node.name}({args})"
```

Visitor 3: Complexity Analyzer

```python
class ComplexityVisitor(ASTVisitor):
    """Counts the number of operations in an expression."""

    def visit_number(self, node):
        return 0  # no operation

    def visit_binary_op(self, node):
        left_ops = node.left.accept(self)
        right_ops = node.right.accept(self)
        return 1 + left_ops + right_ops

    def visit_unary_op(self, node):
        return 1 + node.operand.accept(self)

    def visit_variable(self, node):
        return 0  # lookup, not computation

    def visit_assignment(self, node):
        return node.expression.accept(self)

    def visit_function_call(self, node):
        arg_ops = sum(arg.accept(self) for arg in node.arguments)
        return 1 + arg_ops  # function call is one operation
```

Using the Visitors

```python
# Build AST for: x = (3 + 5) * sqrt(16) - 2
ast = AssignmentNode("x",
    BinaryOpNode("-",
        BinaryOpNode("*",
            BinaryOpNode("+",
                NumberNode(3),
                NumberNode(5)
            ),
            FunctionCallNode("sqrt", [NumberNode(16)])
        ),
        NumberNode(2)
    )
)

evaluator = EvaluatorVisitor()
printer = PrettyPrintVisitor()
complexity = ComplexityVisitor()

print("=== AST Processing ===")
print(f"Expression: {ast.accept(printer)}")
print(f"Result:     {ast.accept(evaluator)}")
print(f"Operations: {ast.accept(complexity)}")

# Process more expressions
exprs = [
    ("Simple add", BinaryOpNode("+", NumberNode(2), NumberNode(3))),
    ("Nested", BinaryOpNode("*",
        BinaryOpNode("+", NumberNode(1), NumberNode(2)),
        BinaryOpNode("-", NumberNode(10), NumberNode(4))
    )),
    ("With negation", UnaryOpNode("-", NumberNode(42))),
    ("Function", FunctionCallNode("max",
        [NumberNode(10), NumberNode(20), NumberNode(15)])),
]

print("\n=== Multiple Expressions ===")
for label, expr in exprs:
    result = expr.accept(evaluator)
    pretty = expr.accept(printer)
    ops = expr.accept(complexity)
    print(f"  {label}: {pretty} = {result} ({ops} operations)")

# Variable usage
print("\n=== Variables ===")
assign = AssignmentNode("y", BinaryOpNode("+", NumberNode(10), NumberNode(20)))
print(f"  {assign.accept(printer)}")
assign.accept(evaluator)  # sets y = 30

use_var = BinaryOpNode("*", VariableNode("y"), NumberNode(2))
print(f"  {use_var.accept(printer)} = {use_var.accept(evaluator)}")
```

Output:

```
=== AST Processing ===
Expression: x = (((3 + 5) * sqrt(16)) - 2)
Result:     30.0
Operations: 4

=== Multiple Expressions ===
  Simple add: (2 + 3) = 5 (1 operations)
  Nested: ((1 + 2) * (10 - 4)) = 18 (3 operations)
  With negation: (-42) = -42 (1 operations)
  Function: max(10, 20, 15) = 20 (1 operations)

=== Variables ===
  y = (10 + 20)
  (y * 2) = 60
```

Three completely different operations (evaluate, print, count) work on the same AST without any AST node class being modified. Each operation is a separate Visitor class.

Before vs After Comparison

Before: Operations Embedded in Classes

```python
class NumberNode:
    def evaluate(self): return self.value
    def pretty_print(self): return str(self.value)
    def count_ops(self): return 0
    def type_check(self): return "number"
    def compile(self): return f"PUSH {self.value}"
    # Adding a new operation means modifying this class
    # AND BinaryOpNode AND UnaryOpNode AND ...

class BinaryOpNode:
    def evaluate(self): ...     # different logic
    def pretty_print(self): ... # different logic
    def count_ops(self): ...    # different logic
    def type_check(self): ...   # different logic
    def compile(self): ...      # different logic
    # Same problem: modifying every node class
```

After: Operations as Visitors

```python
class NumberNode:
    def accept(self, visitor):
        return visitor.visit_number(self)
    # Never changes again!

# New operations = new classes, no existing code modified
class EvaluatorVisitor: ...
class PrettyPrintVisitor: ...
class ComplexityVisitor: ...
class TypeCheckVisitor: ...      # new, no changes to nodes
class CompilerVisitor: ...       # new, no changes to nodes
```

When Is Visitor Worth the Complexity?

Visitor adds significant complexity. It is justified only in specific situations.

```
+-----------------------------------------------------+
| Visitor is WORTH IT when:                           |
+-----------------------------------------------------+
| - Element hierarchy is stable (rarely add types)    |
| - Operations change frequently (add new visitors)   |
| - Many operations across many element types         |
| - Operations need access to element-specific data    |
| - You want to keep element classes clean             |
+-----------------------------------------------------+

+-----------------------------------------------------+
| Visitor is NOT WORTH IT when:                       |
+-----------------------------------------------------+
| - Element types change frequently (add new nodes)   |
| - Few operations (1-2 operations, just use methods)|
| - Simple operations (instanceof checks are simpler)|
| - Elements have no type-specific data to access     |
+-----------------------------------------------------+
```

The key trade-off: Visitor makes adding operations easy but adding element types hard. Every new element type requires updating every Visitor. Every new Visitor requires implementing a method for every element type.

When to Use / When NOT to Use

Use Visitor When

- The class hierarchy is stable but operations change frequently
- You need many different operations across a hierarchy of types
- Operations need access to type-specific data (not just the common interface)
- You want to keep data classes clean and focused on data
- You are building compilers, interpreters, or document processors
- You need to perform operations on a Composite structure (Visitor + Composite)

Do NOT Use Visitor When

- New element types are added frequently (every visitor must be updated)
- You have only 1-2 operations (direct methods are simpler)
- The element hierarchy is small (2-3 types, not worth the overhead)
- Operations only use the common interface (no need for type-specific dispatch)
- The team is unfamiliar with the pattern (readability trumps elegance)

Common Mistakes

Mistake 1: Forgetting to Implement All Visit Methods

```java
// WRONG: missing visitDigitalService
public class DiscountVisitor implements ProductVisitor<Double> {
    public Double visitElectronics(Electronics e) { return 0.1; }
    public Double visitFood(Food f) { return 0.05; }
    public Double visitClothing(Clothing c) { return 0.15; }
    // compile error: visitDigitalService not implemented!
}
```

The compiler catches this in Java. In Python, use `@abstractmethod` to get runtime errors instead of silent bugs.

Mistake 2: Putting Logic in accept()

```java
// WRONG: logic in the element, not the visitor
public <T> T accept(ProductVisitor<T> visitor) {
```

```
    if (this.isOnSale()) {
        return visitor.visitElectronics(this);
    }
    return null;  // skipping visit -- BAD
}

// RIGHT: accept just dispatches, visitor decides
public <T> T accept(ProductVisitor<T> visitor) {
    return visitor.visitElectronics(this);
}
```

Mistake 3: Using Visitor When Types Change Often

```python
# If you add ProductType every month, Visitor is wrong:
class NewProductType(Product):
    def accept(self, visitor):
        return visitor.visit_new_product(self)  # added

# Now you must update EVERY visitor:
class TaxVisitor:
    def visit_new_product(self, p): ...  # must add

class ShippingVisitor:
    def visit_new_product(self, p): ...  # must add

class SummaryVisitor:
    def visit_new_product(self, p): ...  # must add
# 10 visitors * 1 new type = 10 methods to write!
```

Best Practices

1. Use Visitor when operations change more than types. If you add new product types monthly but new operations yearly, Visitor is wrong. If types are stable but operations grow, Visitor is right.
2. Use generics for return types. Java's `ProductVisitor<T>` lets each visitor return a different type (Double for tax, String for reports). This is much better than Object.
3. Provide a default visitor. Create a base visitor class with default implementations so that new visitors only override the methods they care about.
4. Combine with Composite. Visitor is a natural fit for traversing Composite

structures. The Composite handles traversal; the Visitor handles per-element logic.

5. Keep visit methods focused. Each visit method should do one thing. If your visitor grows to hundreds of lines, split it into multiple visitors.

6. In Python, consider simpler alternatives. Python's dynamic nature means you can use `functools.singledispatch` or dictionary-based dispatch instead of the full Visitor pattern.

Quick Summary

The Visitor pattern lets you add new operations to a class hierarchy without modifying existing classes. Each operation is a separate Visitor class with one method per element type. Elements accept visitors through double dispatch, enabling type-specific behavior without type-checking chains.

```
Problem:  Adding new operations requires modifying every class in the hierarchy.
Solution: Define operations in separate Visitor classes; elements accept visitors.
Key:      Double dispatch resolves both the operation AND the element type.
Trade-off: Easy to add operations, hard to add new element types.
```

Key Points

- Visitor separates algorithms from the objects they operate on
- Double dispatch resolves the correct method based on both visitor and element type
- Adding a new operation = adding a new Visitor class (no existing code changes)
- Adding a new element type = updating every existing Visitor (expensive)
- Use Visitor when types are stable but operations change frequently
- Classic use cases: compilers (AST processing), tax calculators, document exporters
- Java generics (`Visitor<T>`) allow type-safe return values per visitor
- Python alternatives: `singledispatch`, dictionary dispatch, or `match` statements
- Visitor pairs well with Composite for tree structure operations

---

Practice Questions

1. Explain double dispatch in your own words. Why can't single dispatch (normal method calls) achieve the same result?
2. You have an AST with 5 node types and 8 visitor operations. You need to add a 6th node type. How many methods must you write or modify? What if you need to add a 9th visitor instead?
3. Compare Visitor with the Strategy pattern. Both separate algorithms from data. What is the key difference?
4. In Python, `functools.singledispatch` can dispatch based on argument type. How does this compare to the Visitor pattern? When would you choose one over the other?
5. A developer argues that Visitor violates encapsulation because visitors access element internals. Is this a valid concern? How would you address it?

Exercises

Exercise 1: Document Export System

Build a document model with visitors:

- Element types: Heading, Paragraph, CodeBlock, Image, Table
- Visitor 1: `HtmlExporter` -- converts to HTML tags
- Visitor 2: `MarkdownExporter` -- converts to Markdown syntax
- Visitor 3: `WordCountVisitor` -- counts words across all text elements
- Build a document with at least one of each element type
- Export the same document in both HTML and Markdown formats

Exercise 2: Shape Area and Perimeter

Create a geometry system:

- Shapes: Circle, Rectangle, Triangle, Polygon
- Visitor 1: `AreaVisitor` -- calculates area for each shape type
- Visitor 2: `PerimeterVisitor` -- calculates perimeter

- Visitor 3: `DrawVisitor` -- generates ASCII art description of each shape
- Create a list of 5 shapes and process them with each visitor
- Demonstrate adding a new `ScaleVisitor` (doubles dimensions) without modifying shapes

Exercise 3: Permission Audit System

Build a permission auditing system using Visitor:

- Element types: User, Group, Role, Resource
- Visitor 1: `AuditVisitor` -- generates a security audit report showing who has access to what
- Visitor 2: `ComplianceVisitor` -- checks for compliance violations (e.g., no user should have both "approve" and "execute" permissions)
- Visitor 3: `VisualizationVisitor` -- generates an ASCII tree showing the permission hierarchy
- Build a realistic permission structure with 3+ users, 2+ groups, and 5+ resources

---

What Is Next?

The next chapter moves from object-oriented design patterns to broader architectural patterns. We begin with the Repository Pattern, which abstracts data access behind a clean interface. Where Visitor adds operations without modifying classes, Repository hides storage details without modifying business logic. Both patterns keep your code clean by separating concerns.

Chapter 23: Repository Pattern

What You Will Learn

- What the Repository pattern is and why it exists
- How to create a collection-like interface for data access
- How to implement repositories in Java using Spring Data JPA
- How to implement repositories in Python using SQLAlchemy
- The difference between Repository and DAO (Data Access Object)
- How to build in-memory repositories for testing
- How to apply the Repository pattern in clean architecture

Why This Chapter Matters

Every backend application needs to store and retrieve data. The moment you scatter SQL queries or ORM calls throughout your business logic, you create a tangled mess that is painful to test, hard to change, and impossible to migrate to a different database.

The Repository pattern solves this by placing a clean boundary between your domain logic and your data access code. Your business layer talks to a simple, collection-like interface: "give me all active users," "save this order," "find the product with this ID." It never knows whether the data comes from PostgreSQL, MongoDB, an in-memory list, or a remote API.

This separation is not academic theory. It is a practical technique used in production systems worldwide. When your team decides to switch from MySQL to DynamoDB, only the repository implementation changes. When you write unit tests, you swap in a fake repository that runs in memory. Your business logic stays untouched.

The Problem

Imagine you are building an order management system. Your service class needs to find orders, save new ones, and update existing ones. Without any abstraction, your code looks like this:

Before: Data Access Mixed with Business Logic

Java (JDBC scattered everywhere):

```java
public class OrderService {

    private final DataSource dataSource;

    public OrderService(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public Order findOrderById(Long id) {
        String sql = "SELECT id, customer_id, total, status FROM orders WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setLong(1, id);
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) {
                Order order = new Order();
                order.setId(rs.getLong("id"));
                order.setCustomerId(rs.getLong("customer_id"));
                order.setTotal(rs.getBigDecimal("total"));
                order.setStatus(rs.getString("status"));
                return order;
            }
            return null;
        } catch (SQLException e) {
            throw new RuntimeException("Failed to find order", e);
        }
    }

    public void applyDiscount(Long orderId, double discountPercent) {
        // Business logic tangled with data access
        String sql = "SELECT id, customer_id, total, status FROM orders WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setLong(1, orderId);
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) {
                BigDecimal total = rs.getBigDecimal("total");
                BigDecimal discount = total.multiply(
                    BigDecimal.valueOf(discountPercent / 100));
                BigDecimal newTotal = total.subtract(discount);
```

```
                String updateSql = "UPDATE orders SET total = ? WHERE id = ?";
                try (PreparedStatement updateStmt =
                         conn.prepareStatement(updateSql)) {
                    updateStmt.setBigDecimal(1, newTotal);
                    updateStmt.setLong(2, orderId);
                    updateStmt.executeUpdate();
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException("Failed to apply discount", e);
        }
    }
}
```

Python (raw SQL everywhere):

```python
import sqlite3

class OrderService:
    def __init__(self, db_path):
        self.db_path = db_path

    def find_order_by_id(self, order_id):
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
        cursor.execute(
            "SELECT id, customer_id, total, status FROM orders WHERE id = ?",
            (order_id,)
        )
        row = cursor.fetchone()
        conn.close()
        if row:
            return {"id": row[0], "customer_id": row[1],
                    "total": row[2], "status": row[3]}
        return None

    def apply_discount(self, order_id, discount_percent):
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
        cursor.execute(
            "SELECT total FROM orders WHERE id = ?", (order_id,)
        )
        row = cursor.fetchone()
        if row:
            new_total = row[0] * (1 - discount_percent / 100)
            cursor.execute(
                "UPDATE orders SET total = ? WHERE id = ?",
                (new_total, order_id)
            )
            conn.commit()
        conn.close()
```

What is wrong with this?

1. Business logic (calculating discounts) is mixed with data access (SQL queries)
2. You cannot test `applyDiscount` without a real database
3. Changing the database means rewriting every method
4. SQL is duplicated across multiple service classes
5. No consistent error handling strategy

The Solution: Repository Pattern

The Repository pattern introduces an abstraction that looks like an in-memory collection of domain objects. Your service code calls methods like `findById()`, `save()`, and `findAll()` without knowing anything about the underlying storage.

```
+------------------+       +--------------------+       +------------+
|                  |       |                    |       |            |
|  Business Logic  | ----> |    Repository      | ----> |  Database  |
|  (OrderService)  |       |  (Interface)       |       |            |
|                  |       |                    |       |            |
+------------------+       +--------------------+       +------------+
                                     ^
                                     |
                           +--------+--------+
                           |                 |
                     +------+------+   +------+------+
                     |   JPA Impl  |   | In-Memory   |
                     | (Production)|   |   (Testing)  |
                     +-------------+   +-------------+
```

After: Clean Separation with Repository

Java:

```java
// Step 1: Define the repository interface
public interface OrderRepository {
    Order findById(Long id);
    List<Order> findAll();
    List<Order> findByStatus(String status);
    Order save(Order order);
    void delete(Order order);
}
```

```
// Step 2: Service uses only the interface
public class OrderService {

    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public void applyDiscount(Long orderId, double discountPercent) {
        Order order = orderRepository.findById(orderId);
        if (order == null) {
            throw new IllegalArgumentException("Order not found: " + orderId);
        }

        BigDecimal discount = order.getTotal()
            .multiply(BigDecimal.valueOf(discountPercent / 100));
        order.setTotal(order.getTotal().subtract(discount));

        orderRepository.save(order);
    }

    public List<Order> getPendingOrders() {
        return orderRepository.findByStatus("PENDING");
    }
}
```

Python:

```python
from abc import ABC, abstractmethod
from typing import List, Optional

class OrderRepository(ABC):
    """Abstract repository - looks like a collection of orders."""

    @abstractmethod
    def find_by_id(self, order_id: int) -> Optional[dict]:
        pass

    @abstractmethod
    def find_all(self) -> List[dict]:
        pass

    @abstractmethod
    def find_by_status(self, status: str) -> List[dict]:
        pass

    @abstractmethod
    def save(self, order: dict) -> dict:
        pass
```

```
    @abstractmethod
    def delete(self, order_id: int) -> None:
        pass


class OrderService:
    """Service uses only the repository interface."""

    def __init__(self, order_repository: OrderRepository):
        self.order_repository = order_repository

    def apply_discount(self, order_id: int, discount_percent: float):
        order = self.order_repository.find_by_id(order_id)
        if order is None:
            raise ValueError(f"Order not found: {order_id}")

        discount = order["total"] * (discount_percent / 100)
        order["total"] -= discount

        self.order_repository.save(order)

    def get_pending_orders(self) -> list:
        return self.order_repository.find_by_status("PENDING")
```

Notice how `OrderService` has zero knowledge of SQL, databases, or ORMs. It simply calls repository methods that behave like collection operations.

How the Repository Pattern Works

```
    Client Code (Service Layer)
            |
            |  findById(42)
            v
    +-------------------+
    |    Repository      |     <-- Interface: Collection-like API
    |    Interface       |
    +-------------------+
            |
            |  (Implementation chosen at runtime)
            v
    +-------------------+         +-------------------+
    |  JpaOrderRepo     |   OR   | InMemoryOrderRepo  |
    |  (Production)     |         | (Testing)          |
    +-------------------+         +-------------------+
            |                               |
            v                               v
    +-------------------+         +-------------------+
```

```
    |   PostgreSQL      |         |   HashMap / Dict   |
    |   Database        |         |   (in memory)      |
    +-------------------+         +--------------------+
```

The key components:

1. Repository Interface - Defines collection-like operations (find, save, delete)
2. Concrete Implementation - Connects to the actual data store
3. Domain Entity - The object being stored and retrieved
4. Client Code - Uses only the interface, never the implementation directly

---

Java Implementation: Spring Data JPA Repository

Spring Data JPA makes the Repository pattern almost effortless. You define an interface, and Spring generates the implementation at runtime.

Step 1: Define the Entity

```java
import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "customer_id", nullable = false)
    private Long customerId;

    @Column(nullable = false)
    private BigDecimal total;

    @Column(nullable = false)
    private String status;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    // Constructors
```

```
    public Order() {}

    public Order(Long customerId, BigDecimal total, String status) {
        this.customerId = customerId;
        this.total = total;
        this.status = status;
        this.createdAt = LocalDateTime.now();
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Long getCustomerId() { return customerId; }
    public void setCustomerId(Long customerId) {
        this.customerId = customerId;
    }
    public BigDecimal getTotal() { return total; }
    public void setTotal(BigDecimal total) { this.total = total; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    @Override
    public String toString() {
        return "Order{id=" + id + ", customerId=" + customerId +
               ", total=" + total + ", status='" + status + "'}";
    }
}
```

Step 2: Define the Repository Interface

```java
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.math.BigDecimal;
import java.util.List;

@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Spring generates SQL from method name automatically
    List<Order> findByStatus(String status);

    List<Order> findByCustomerId(Long customerId);

    List<Order> findByStatusAndCustomerId(String status, Long customerId);
```

```
    // Custom query when method naming is not enough
    @Query("SELECT o FROM Order o WHERE o.total > :minTotal ORDER BY o.total DESC")
    List<Order> findExpensiveOrders(BigDecimal minTotal);

    @Query("SELECT COUNT(o) FROM Order o WHERE o.status = :status")
    long countByStatus(String status);
}
```

Step 3: Use the Repository in a Service

```java
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.math.BigDecimal;
import java.util.List;

@Service
public class OrderService {

    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    @Transactional
    public Order createOrder(Long customerId, BigDecimal total) {
        Order order = new Order(customerId, total, "PENDING");
        return orderRepository.save(order);
    }

    @Transactional
    public Order confirmOrder(Long orderId) {
        Order order = orderRepository.findById(orderId)
            .orElseThrow(() -> new RuntimeException("Order not found"));
        order.setStatus("CONFIRMED");
        return orderRepository.save(order);
    }

    public List<Order> getPendingOrders() {
        return orderRepository.findByStatus("PENDING");
    }

    public List<Order> getCustomerOrders(Long customerId) {
        return orderRepository.findByCustomerId(customerId);
    }
}
```

Output when running:

```
Created: Order{id=1, customerId=100, total=250.00, status='PENDING'}
Created: Order{id=2, customerId=100, total=75.50, status='PENDING'}
Created: Order{id=3, customerId=200, total=500.00, status='PENDING'}

Confirmed: Order{id=1, customerId=100, total=250.00, status='CONFIRMED'}

Pending orders: 2
Customer 100 orders: 2
```

Python Implementation: SQLAlchemy Repository

In Python, we build the Repository pattern explicitly since SQLAlchemy does not generate implementations for us.

Step 1: Define the Model

```python
from sqlalchemy import Column, Integer, String, Float, DateTime, create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
from datetime import datetime

Base = declarative_base()

class Order(Base):
    __tablename__ = "orders"

    id = Column(Integer, primary_key=True, autoincrement=True)
    customer_id = Column(Integer, nullable=False)
    total = Column(Float, nullable=False)
    status = Column(String(50), nullable=False, default="PENDING")
    created_at = Column(DateTime, default=datetime.utcnow)

    def __repr__(self):
        return (f"Order(id={self.id}, customer_id={self.customer_id}, "
                f"total={self.total}, status='{self.status}')")

    def to_dict(self):
        return {
            "id": self.id,
            "customer_id": self.customer_id,
            "total": self.total,
            "status": self.status,
            "created_at": str(self.created_at),
        }
```

Step 2: Define the Abstract Repository

```python
from abc import ABC, abstractmethod
from typing import List, Optional

class AbstractOrderRepository(ABC):
    """Collection-like interface for Order entities."""

    @abstractmethod
    def find_by_id(self, order_id: int) -> Optional[Order]:
        pass

    @abstractmethod
    def find_all(self) -> List[Order]:
        pass

    @abstractmethod
    def find_by_status(self, status: str) -> List[Order]:
        pass

    @abstractmethod
    def find_by_customer_id(self, customer_id: int) -> List[Order]:
        pass

    @abstractmethod
    def save(self, order: Order) -> Order:
        pass

    @abstractmethod
    def delete(self, order: Order) -> None:
        pass
```

Step 3: SQLAlchemy Implementation

```python
from sqlalchemy.orm import Session

class SqlAlchemyOrderRepository(AbstractOrderRepository):
    """Repository implementation using SQLAlchemy."""

    def __init__(self, session: Session):
        self.session = session

    def find_by_id(self, order_id: int) -> Optional[Order]:
        return self.session.query(Order).filter(
            Order.id == order_id
        ).first()

    def find_all(self) -> List[Order]:
        return self.session.query(Order).all()
```

```
    def find_by_status(self, status: str) -> List[Order]:
        return self.session.query(Order).filter(
            Order.status == status
        ).all()

    def find_by_customer_id(self, customer_id: int) -> List[Order]:
        return self.session.query(Order).filter(
            Order.customer_id == customer_id
        ).all()

    def save(self, order: Order) -> Order:
        if order.id is None:
            self.session.add(order)
        self.session.flush()  # Get the generated ID
        return order

    def delete(self, order: Order) -> None:
        self.session.delete(order)
        self.session.flush()
```

Step 4: Service Layer

```python
class OrderService:
    """Business logic that depends only on the repository interface."""

    def __init__(self, repository: AbstractOrderRepository):
        self.repository = repository

    def create_order(self, customer_id: int, total: float) -> Order:
        order = Order(customer_id=customer_id, total=total, status="PENDING")
        return self.repository.save(order)

    def confirm_order(self, order_id: int) -> Order:
        order = self.repository.find_by_id(order_id)
        if order is None:
            raise ValueError(f"Order not found: {order_id}")
        order.status = "CONFIRMED"
        return self.repository.save(order)

    def get_pending_orders(self) -> list:
        return self.repository.find_by_status("PENDING")

    def apply_discount(self, order_id: int, percent: float) -> Order:
        order = self.repository.find_by_id(order_id)
        if order is None:
            raise ValueError(f"Order not found: {order_id}")
        order.total = order.total * (1 - percent / 100)
        return self.repository.save(order)
```

Step 5: Wire It Together

```python
# Setup database
engine = create_engine("sqlite:///orders.db", echo=False)
Base.metadata.create_all(engine)
SessionLocal = sessionmaker(bind=engine)

# Create session and repository
session = SessionLocal()
repository = SqlAlchemyOrderRepository(session)
service = OrderService(repository)

# Use the service
order1 = service.create_order(customer_id=100, total=250.00)
order2 = service.create_order(customer_id=100, total=75.50)
order3 = service.create_order(customer_id=200, total=500.00)

print(f"Created: {order1}")
print(f"Created: {order2}")
print(f"Created: {order3}")

# Confirm an order
confirmed = service.confirm_order(order1.id)
print(f"\nConfirmed: {confirmed}")

# Query pending orders
pending = service.get_pending_orders()
print(f"\nPending orders: {len(pending)}")
for o in pending:
    print(f"  {o}")

session.commit()
session.close()
```

Output:

```
Created: Order(id=1, customer_id=100, total=250.0, status='PENDING')
Created: Order(id=2, customer_id=100, total=75.5, status='PENDING')
Created: Order(id=3, customer_id=200, total=500.0, status='PENDING')

Confirmed: Order(id=1, customer_id=100, total=250.0, status='CONFIRMED')

Pending orders: 2
  Order(id=2, customer_id=100, total=75.5, status='PENDING')
  Order(id=3, customer_id=200, total=500.0, status='PENDING')
```

In-Memory Repository for Testing

One of the biggest benefits of the Repository pattern is testability. You can create an in-memory implementation that requires no database at all.

Java: In-Memory Repository

```java
import java.util.*;
import java.util.concurrent.atomic.AtomicLong;
import java.util.stream.Collectors;

public class InMemoryOrderRepository implements OrderRepository {

    private final Map<Long, Order> store = new HashMap<>();
    private final AtomicLong idGenerator = new AtomicLong(1);

    @Override
    public Order findById(Long id) {
        return store.get(id);
    }

    @Override
    public List<Order> findAll() {
        return new ArrayList<>(store.values());
    }

    @Override
    public List<Order> findByStatus(String status) {
        return store.values().stream()
            .filter(o -> o.getStatus().equals(status))
            .collect(Collectors.toList());
    }

    @Override
    public Order save(Order order) {
        if (order.getId() == null) {
            order.setId(idGenerator.getAndIncrement());
        }
        store.put(order.getId(), order);
        return order;
    }

    @Override
    public void delete(Order order) {
        store.remove(order.getId());
    }

    // Helper for tests
    public void clear() {
        store.clear();
        idGenerator.set(1);
    }
```

```
    public int size() {
        return store.size();
    }
}
```

Java: Unit Test Using In-Memory Repository

```java
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import java.math.BigDecimal;

class OrderServiceTest {

    private InMemoryOrderRepository repository;
    private OrderService service;

    @BeforeEach
    void setUp() {
        repository = new InMemoryOrderRepository();
        service = new OrderService(repository);
    }

    @Test
    void shouldCreateOrder() {
        Order order = service.createOrder(100L, new BigDecimal("250.00"));

        assertNotNull(order.getId());
        assertEquals("PENDING", order.getStatus());
        assertEquals(1, repository.size());
    }

    @Test
    void shouldConfirmOrder() {
        Order order = service.createOrder(100L, new BigDecimal("250.00"));
        Order confirmed = service.confirmOrder(order.getId());

        assertEquals("CONFIRMED", confirmed.getStatus());
    }

    @Test
    void shouldApplyDiscount() {
        Order order = service.createOrder(100L, new BigDecimal("100.00"));
        service.applyDiscount(order.getId(), 10.0);

        Order updated = repository.findById(order.getId());
        assertEquals(new BigDecimal("90.00"), updated.getTotal());
    }

    @Test
```

```
    void shouldThrowWhenOrderNotFound() {
        assertThrows(RuntimeException.class,
            () -> service.confirmOrder(999L));
    }
}
```

Test Output:

```
OrderServiceTest
  shouldCreateOrder           PASSED
  shouldConfirmOrder          PASSED
  shouldApplyDiscount         PASSED
  shouldThrowWhenOrderNotFound PASSED

4 tests passed, 0 failed
```

Python: In-Memory Repository

```python
class InMemoryOrderRepository(AbstractOrderRepository):
    """In-memory repository for fast unit tests."""

    def __init__(self):
        self._store: dict[int, Order] = {}
        self._next_id = 1

    def find_by_id(self, order_id: int) -> Optional[Order]:
        return self._store.get(order_id)

    def find_all(self) -> List[Order]:
        return list(self._store.values())

    def find_by_status(self, status: str) -> List[Order]:
        return [o for o in self._store.values() if o.status == status]

    def find_by_customer_id(self, customer_id: int) -> List[Order]:
        return [o for o in self._store.values()
                if o.customer_id == customer_id]

    def save(self, order: Order) -> Order:
        if order.id is None:
            order.id = self._next_id
            self._next_id += 1
        self._store[order.id] = order
        return order

    def delete(self, order: Order) -> None:
        self._store.pop(order.id, None)
```

```
    def clear(self):
        self._store.clear()
        self._next_id = 1
```

Python: Unit Test

```python
import pytest

class TestOrderService:

    def setup_method(self):
        self.repository = InMemoryOrderRepository()
        self.service = OrderService(self.repository)

    def test_create_order(self):
        order = self.service.create_order(customer_id=100, total=250.00)
        assert order.id is not None
        assert order.status == "PENDING"
        assert len(self.repository.find_all()) == 1

    def test_confirm_order(self):
        order = self.service.create_order(customer_id=100, total=250.00)
        confirmed = self.service.confirm_order(order.id)
        assert confirmed.status == "CONFIRMED"

    def test_apply_discount(self):
        order = self.service.create_order(customer_id=100, total=100.00)
        updated = self.service.apply_discount(order.id, 10.0)
        assert updated.total == 90.0

    def test_order_not_found(self):
        with pytest.raises(ValueError):
            self.service.confirm_order(999)
```

Output:

```
test_order_service.py
  test_create_order      PASSED
  test_confirm_order     PASSED
  test_apply_discount    PASSED
  test_order_not_found   PASSED

4 passed in 0.02s
```

Repository vs DAO (Data Access Object)

These two patterns are often confused. Here is how they differ:

```
+-------------------+-----------------------------------+-------------------------------...
|    Aspect         |       Repository                  |          DAO                  ...
+-------------------+-----------------------------------+-------------------------------...
| Abstraction Level | Domain-oriented (collection of    | Data-oriented (table-level    ...
|                   | domain objects)                   | operations)                   ...
+-------------------+-----------------------------------+-------------------------------...
| Interface Style   | findByStatus("ACTIVE")            | executeQuery("SELECT ... WHERE...
|                   | save(order)                       | insert(row)                   ...
+-------------------+-----------------------------------+-------------------------------...
| Return Type       | Domain entities                   | DTOs, Maps, or ResultSets     ...
+-------------------+-----------------------------------+-------------------------------...
| Awareness         | Knows nothing about SQL or        | May expose database-specific  ...
|                   | database structure                | concepts                      ...
+-------------------+-----------------------------------+-------------------------------...
| Belongs To        | Domain layer                      | Infrastructure / persistence  ...
|                   |                                   | layer                         ...
+-------------------+-----------------------------------+-------------------------------...
| Composability     | Can aggregate multiple data       | Usually maps 1:1 to a table   ...
|                   | sources behind one interface      |                               ...
+-------------------+-----------------------------------+-------------------------------...
```

DAO Example (Data-Oriented)

```java
// DAO - exposes data access details
public class OrderDao {
    public ResultSet executeQuery(String sql) { ... }
    public void insert(String table, Map<String, Object> values) { ... }
    public void update(String table, Map<String, Object> values,
                       String whereClause) { ... }
}
```

Repository Example (Domain-Oriented)

```java
// Repository - domain language, no SQL exposure
public interface OrderRepository {
    Order findById(Long id);
    List<Order> findActiveOrdersForCustomer(Long customerId);
    Order save(Order order);
}
```

In practice, the distinction has blurred. Spring Data's `JpaRepository` is called a repository but sometimes behaves like a DAO. The important thing is to keep your interface domain-focused rather than data-focused.

Real-World Use Case: Clean Architecture Data Layer

In clean architecture, the Repository pattern forms the boundary between the domain layer and the infrastructure layer.

```
+-------------------------------------------------------------------+
|                        Presentation Layer                         |
|                    (Controllers, REST APIs)                       |
+-------------------------------------------------------------------+
                              |
                              v
+-------------------------------------------------------------------+
|                        Application Layer                          |
|                     (Use Cases / Services)                        |
|                                                                   |
|   OrderService uses OrderRepository (interface)                   |
+-------------------------------------------------------------------+
                              |
                              | depends on interface only
                              v
+-------------------------------------------------------------------+
|                         Domain Layer                              |
|                  (Entities, Repository Interfaces)                |
|                                                                   |
|   interface OrderRepository { findById(), save(), ... }           |
+-------------------------------------------------------------------+
                              ^
                              | implements
                              |
+-------------------------------------------------------------------+
|                       Infrastructure Layer                        |
|              (Database, ORM, External Services)                   |
|                                                                   |
|   JpaOrderRepository implements OrderRepository                  |
|   MongoOrderRepository implements OrderRepository                |
+-------------------------------------------------------------------+
```

Generic Repository Base Class

For larger projects, create a generic repository to avoid repeating common operations:

Java:

```java
public interface GenericRepository<T, ID> {
    Optional<T> findById(ID id);
    List<T> findAll();
    T save(T entity);
    void delete(T entity);
    long count();
    boolean existsById(ID id);
}

// Specific repository adds domain methods
public interface ProductRepository extends GenericRepository<Product, Long> {
    List<Product> findByCategory(String category);
    List<Product> findByPriceRange(BigDecimal min, BigDecimal max);
    List<Product> findInStock();
}
```

Python:

```python
from typing import TypeVar, Generic, List, Optional

T = TypeVar("T")
ID = TypeVar("ID")

class GenericRepository(ABC, Generic[T, ID]):
    """Base repository with common CRUD operations."""

    @abstractmethod
    def find_by_id(self, entity_id: ID) -> Optional[T]:
        pass

    @abstractmethod
    def find_all(self) -> List[T]:
        pass

    @abstractmethod
    def save(self, entity: T) -> T:
        pass

    @abstractmethod
    def delete(self, entity: T) -> None:
        pass

    @abstractmethod
    def count(self) -> int:
        pass
```

```
class ProductRepository(GenericRepository[Product, int]):
    """Product-specific repository with domain methods."""

    @abstractmethod
    def find_by_category(self, category: str) -> List[Product]:
        pass

    @abstractmethod
    def find_by_price_range(self, min_price: float,
                            max_price: float) -> List[Product]:
        pass

    @abstractmethod
    def find_in_stock(self) -> List[Product]:
        pass
```

When to Use / When NOT to Use

Use the Repository Pattern When

- You want to isolate business logic from data access details
- You need to support multiple data stores (SQL, NoSQL, APIs)
- You want fast, database-free unit tests
- Your domain model is complex and does not map one-to-one to tables
- You are following clean architecture or hexagonal architecture
- Your team works on different layers independently

Do NOT Use the Repository Pattern When

- Your application is a simple CRUD with no business logic
- You are building a quick prototype or proof of concept
- The framework already provides sufficient abstraction (simple Django views)
- Adding a repository layer would just be a pass-through with no value
- Your application has only one data source and will never change

---

Common Mistakes

1. Leaking persistence details through the interface. Repository methods should use domain language, not SQL or ORM concepts. Avoid methods like `executeQuery()` or `flush()` on the interface.
2. Creating repositories that are too generic. A repository with only `save()` and `findById()` is too thin. Add domain-specific finder methods that express business concepts.
3. Putting business logic in the repository. The repository should only handle data access. Calculations, validations, and rules belong in the service or domain layer.
4. Not using the interface for dependency injection. If your service class depends on `JpaOrderRepository` instead of `OrderRepository`, you lose the ability to swap implementations.
5. One repository per table instead of per aggregate. In domain-driven design, repositories should correspond to aggregate roots, not individual tables. An `OrderRepository` might internally manage `Order` and `OrderItem` together.

Best Practices

1. Name repository methods using domain language. Use `findActiveCustomers()` rather than `findByStatusEquals("ACTIVE")`.
2. Return domain objects, not database rows. The repository translates between the persistence format and domain entities.
3. Keep the repository interface in the domain layer. The implementation lives in the infrastructure layer. This ensures the domain never depends on infrastructure.
4. Create in-memory implementations for testing. This makes your tests fast and independent of any database.
5. Use the Specification pattern for complex queries. Instead of adding a new method for every query combination, pass specification objects that describe the criteria.
6. Limit the number of methods. A repository with 30 methods is a sign that you need to rethink your design. Focus on the queries your domain actually needs.

Quick Summary

The Repository pattern provides a collection-like interface for accessing domain entities. It hides all persistence details behind a clean abstraction, allowing your business logic to remain independent of the data store. In Java, Spring Data JPA generates repository

implementations automatically. In Python, you define an abstract base class and implement it with SQLAlchemy. The pattern enables fast testing through in-memory implementations and supports clean architecture by keeping the domain layer free from infrastructure concerns.

Key Points

- The Repository pattern abstracts data access behind a collection-like interface
- Business logic depends on the repository interface, not the implementation
- Spring Data JPA generates repository implementations from interface definitions
- In Python, define an abstract repository class and implement it with SQLAlchemy
- In-memory repositories enable fast, database-free unit tests
- Repository is domain-oriented; DAO is data-oriented
- Repository interfaces belong in the domain layer; implementations in infrastructure
- Generic repositories reduce boilerplate for common CRUD operations
- Methods should use domain language, not database terminology

---

Practice Questions

1. What is the main difference between a Repository and a DAO? In what situations would you choose one over the other?
2. Why is it important that the service layer depends on the repository interface rather than a concrete implementation? What design principle does this follow?
3. You have an `Order` entity that contains a list of `OrderItem` entities. Should you create separate repositories for `Order` and `OrderItem`, or a single `OrderRepository` that manages both? Explain your reasoning.
4. How does Spring Data JPA derive SQL queries from method names like `findByStatusAndCustomerId`? What happens when the method name becomes too complex?
5. A teammate argues that the Repository pattern adds unnecessary complexity to a simple CRUD application. How would you respond?

Exercises

Exercise 1: Build a User Repository

Create a `UserRepository` interface with methods: `findById`, `findByEmail`, `findActiveUsers`, `save`, and `delete`. Implement both a SQLAlchemy version and an in-memory version in Python. Write unit tests using the in-memory version to verify that all operations work correctly.

Exercise 2: Generic Repository in Java

Create a generic `CrudRepository<T, ID>` interface with basic CRUD operations. Then create a `ProductRepository` that extends it with methods like `findByCategory(String category)` and `findByPriceBelow(BigDecimal price)`. Implement an in-memory version and write tests.

Exercise 3: Repository with Specification Pattern

Extend your repository to support dynamic queries using a Specification pattern. Create a `Specification<T>` interface with a `boolean isSatisfiedBy(T entity)` method. Add a `findAll(Specification<T> spec)` method to your repository. Build specifications like `OrderAboveTotal`, `OrderByCustomer`, and `PendingOrders`. Demonstrate how specifications can be combined with AND/OR logic.

What Is Next?

Now that you understand how the Repository pattern isolates data access behind a clean interface, the next chapter explores the Unit of Work pattern. While the repository handles individual entity operations, the Unit of Work tracks all changes made during a business transaction and commits them as a single atomic operation. Together, these two patterns form the foundation of a robust data access layer.

Chapter 24: Unit of Work Pattern

What You Will Learn

- What the Unit of Work pattern is and what problem it solves
- How to track changes to multiple entities and commit them as one transaction
- How JPA EntityManager and Hibernate Session act as a Unit of Work in Java
- How SQLAlchemy Session implements the Unit of Work pattern in Python
- How to handle partial commits and ensure transactional consistency
- How to apply Unit of Work in batch operations and real-world scenarios

Why This Chapter Matters

In any non-trivial backend system, a single business operation often touches multiple entities. When a customer places an order, you need to create the order record, update inventory, record a payment, and send a confirmation. If the payment step fails after inventory has already been deducted, your data is inconsistent. The customer sees a charge but no order, or worse, inventory disappears without a sale.

The Unit of Work pattern solves this by keeping track of every change you make during a business operation and committing them all at once as a single transaction. If anything fails, everything rolls back. Nothing is saved halfway.

This is not a niche pattern. Every ORM you use -- JPA, Hibernate, SQLAlchemy, Entity Framework -- implements it under the hood. Understanding how it works helps you use these tools correctly and avoid subtle bugs that appear only in production.

The Problem

Imagine processing an order that involves multiple database operations:

Before: Multiple Independent Saves

Java:

```java
public class OrderProcessor {

    private final OrderRepository orderRepository;
    private final InventoryRepository inventoryRepository;
    private final PaymentRepository paymentRepository;

    public void processOrder(OrderRequest request) {
        // Save the order
        Order order = new Order(request.getCustomerId(), request.getTotal());
        orderRepository.save(order);  // COMMIT 1

        // Deduct inventory
        for (OrderItem item : request.getItems()) {
            Inventory inv = inventoryRepository.findByProductId(
                item.getProductId());
            inv.setQuantity(inv.getQuantity() - item.getQuantity());
            inventoryRepository.save(inv);  // COMMIT 2, 3, 4...
        }

        // Record payment -- THIS FAILS!
        Payment payment = new Payment(order.getId(), request.getTotal());
        paymentRepository.save(payment);  // THROWS EXCEPTION

        // Problem: Order is saved, inventory is deducted,
        // but payment failed. Data is now inconsistent!
    }
}
```

Python:

```python
class OrderProcessor:
    def process_order(self, request):
        # Save the order
        order = Order(customer_id=request.customer_id, total=request.total)
        db.session.add(order)
        db.session.commit()  # COMMIT 1

        # Deduct inventory
        for item in request.items:
            inv = Inventory.query.filter_by(
                product_id=item.product_id).first()
            inv.quantity -= item.quantity
            db.session.commit()  # COMMIT 2, 3, 4...
```

```
        # Record payment -- THIS FAILS!
        payment = Payment(order_id=order.id, amount=request.total)
        db.session.add(payment)
        db.session.commit()  # THROWS EXCEPTION

        # Order and inventory changes are already committed.
        # Cannot undo them!
```

```
Timeline of failure:

  COMMIT 1      COMMIT 2      COMMIT 3       BOOM!
  Save Order    Update Inv    Update Inv     Payment fails
     |              |             |              |
     v              v             v              v
  [Saved]       [Saved]       [Saved]       [Exception]

  Result: Order exists, inventory is wrong, no payment
          Data is INCONSISTENT
```

The Solution: Unit of Work Pattern

The Unit of Work pattern tracks all changes made during a business operation and commits them as a single atomic transaction.

```
+-------------------------------------------------------------+
|                      Unit of Work                           |
|                                                             |
|   +-- New Objects ----+  +-- Dirty Objects --+  +-- Deleted -+
|   | Order #1          |  | Inventory A       |  |            |
|   | Payment #1        |  | Inventory B       |  |            |
|   +-------------------+  +-------------------+  +------------+
|                                                             |
|   commit() --> BEGIN TRANSACTION                            |
|                INSERT order                                 |
|                UPDATE inventory_a                           |
|                UPDATE inventory_b                           |
|                INSERT payment                               |
|                COMMIT   (all or nothing)                    |
|                                                             |
|   rollback() --> ROLLBACK  (undo everything)                |
+-------------------------------------------------------------+
```

After: Single Transaction with Unit of Work

Java (with JPA/Spring):

```java
@Service
public class OrderProcessor {

    private final OrderRepository orderRepository;
    private final InventoryRepository inventoryRepository;
    private final PaymentRepository paymentRepository;

    @Transactional  // <-- This makes the entire method one Unit of Work
    public void processOrder(OrderRequest request) {
        // Create the order
        Order order = new Order(request.getCustomerId(), request.getTotal());
        orderRepository.save(order);

        // Deduct inventory
        for (OrderItem item : request.getItems()) {
            Inventory inv = inventoryRepository.findByProductId(
                item.getProductId());
            inv.setQuantity(inv.getQuantity() - item.getQuantity());
            inventoryRepository.save(inv);
        }

        // Record payment
        Payment payment = new Payment(order.getId(), request.getTotal());
        paymentRepository.save(payment);

        // If ANY step fails, ALL changes are rolled back automatically
    }
}
```

Python (with SQLAlchemy):

```python
class OrderProcessor:
    def __init__(self, session):
        self.session = session  # SQLAlchemy Session IS the Unit of Work

    def process_order(self, request):
        try:
            # Create the order
            order = Order(customer_id=request.customer_id,
                          total=request.total)
            self.session.add(order)

            # Deduct inventory
            for item in request.items:
                inv = self.session.query(Inventory).filter_by(
                    product_id=item.product_id).first()
                inv.quantity -= item.quantity

            # Record payment
```

```
            payment = Payment(order_id=order.id, amount=request.total)
            self.session.add(payment)

            # Commit everything as ONE transaction
            self.session.commit()

        except Exception:
            # If anything fails, undo ALL changes
            self.session.rollback()
            raise
```

```
Timeline with Unit of Work:

  Track        Track         Track        COMMIT ALL
  Order        Inventory     Payment      or ROLLBACK
    |              |             |             |
    v              v             v             v
  [Pending]    [Pending]    [Pending]    [All Saved]
                                          or
                                         [All Reverted]

  Result: Data is ALWAYS consistent
```

How the Unit of Work Pattern Works

The Unit of Work maintains three internal lists:

```
+-------------------------------------------------------------+
|                    Unit of Work                             |
|                                                             |
|  1. NEW objects     -- entities to be INSERTed              |
|     [Order #1, Payment #1]                                  |
|                                                             |
|  2. DIRTY objects   -- entities to be UPDATEd               |
|     [Inventory A (qty: 100 -> 95)]                          |
|     [Inventory B (qty: 50 -> 48)]                           |
|                                                             |
|  3. REMOVED objects -- entities to be DELETEd               |
|     [OldPromotion #7]                                       |
|                                                             |
|  commit():                                                  |
|     BEGIN TRANSACTION                                       |
|     for each new    -> INSERT INTO ...                      |
|     for each dirty  -> UPDATE ... SET ...                   |
|     for each removed -> DELETE FROM ...                     |
|     COMMIT                                                  |
```

```
|                                                          |
|  rollback():                                             |
|     Discard all tracked changes                          |
|     Reset to clean state                                 |
+----------------------------------------------------------+
```

Java: JPA EntityManager as Unit of Work

The JPA `EntityManager` (and Hibernate `Session`) already implements the Unit of Work pattern. Understanding this helps you use it correctly.

How EntityManager Tracks Changes

```java
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

public class JpaUnitOfWorkDemo {

    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.createEntityManagerFactory(
            "demo-unit");
        EntityManager em = emf.createEntityManager();

        // Begin the unit of work
        em.getTransaction().begin();

        // 1. NEW: Create a new entity (tracked as "new")
        Order order = new Order(100L, new BigDecimal("250.00"), "PENDING");
        em.persist(order);
        System.out.println("Order persisted (not yet in DB)");

        // 2. DIRTY: Modify an existing entity (tracked as "dirty")
        Product product = em.find(Product.class, 1L);
        product.setStock(product.getStock() - 5);
        // No explicit save needed! EntityManager detects the change
        System.out.println("Product stock modified (tracked automatically)");

        // 3. REMOVED: Delete an entity (tracked as "removed")
        Coupon expiredCoupon = em.find(Coupon.class, 42L);
        em.remove(expiredCoupon);
        System.out.println("Coupon marked for removal");

        // Commit: All three operations happen in ONE transaction
        em.getTransaction().commit();
```

```
        System.out.println("All changes committed as one transaction");

        em.close();
        emf.close();
    }
}
```

Output:

```
Order persisted (not yet in DB)
Product stock modified (tracked automatically)
Coupon marked for removal
All changes committed as one transaction
```

Spring @Transactional as Unit of Work Boundary

In Spring, the `@Transactional` annotation defines the boundary of a Unit of Work:

```java
@Service
public class TransferService {

    private final AccountRepository accountRepository;
    private final TransferLogRepository transferLogRepository;

    public TransferService(AccountRepository accountRepository,
                           TransferLogRepository transferLogRepository) {
        this.accountRepository = accountRepository;
        this.transferLogRepository = transferLogRepository;
    }

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        // Load accounts (they become "managed" by the EntityManager)
        Account from = accountRepository.findById(fromId)
            .orElseThrow(() -> new RuntimeException("Source not found"));
        Account to = accountRepository.findById(toId)
            .orElseThrow(() -> new RuntimeException("Target not found"));

        // Validate
        if (from.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException("Not enough balance");
        }

        // Modify entities (EntityManager tracks these changes)
        from.setBalance(from.getBalance().subtract(amount));
        to.setBalance(to.getBalance().add(amount));

        // Log the transfer
```

```
        TransferLog log = new TransferLog(fromId, toId, amount);
        transferLogRepository.save(log);

        // When this method exits normally, Spring commits the transaction
        // If an exception is thrown, Spring rolls back everything
    }
}
```

```
@Transactional method execution:

  Method starts
      |
      v
  Spring creates EntityManager + begins transaction
      |
      v
  Your code runs (find, modify, save entities)
      |
      v
  Method returns normally?
      |          |
     YES         NO (exception thrown)
      |          |
      v          v
   COMMIT     ROLLBACK
  (all saved) (nothing saved)
```

Python: SQLAlchemy Session as Unit of Work

SQLAlchemy's `Session` is an explicit implementation of the Unit of Work pattern.

Basic Session Usage

```python
from sqlalchemy import create_engine, Column, Integer, String, Float
from sqlalchemy.orm import declarative_base, sessionmaker, Session

Base = declarative_base()

class Account(Base):
    __tablename__ = "accounts"
    id = Column(Integer, primary_key=True)
    owner = Column(String(100), nullable=False)
    balance = Column(Float, nullable=False, default=0.0)
```

```
    def __repr__(self):
        return f"Account(id={self.id}, owner='{self.owner}', balance={self.balance})"


class TransferLog(Base):
    __tablename__ = "transfer_logs"
    id = Column(Integer, primary_key=True, autoincrement=True)
    from_id = Column(Integer, nullable=False)
    to_id = Column(Integer, nullable=False)
    amount = Column(Float, nullable=False)


# Setup
engine = create_engine("sqlite:///bank.db")
Base.metadata.create_all(engine)
SessionFactory = sessionmaker(bind=engine)
```

Tracking Changes

```python
def demonstrate_unit_of_work():
    session = SessionFactory()

    try:
        # 1. NEW: Add new entities
        alice = Account(id=1, owner="Alice", balance=1000.0)
        bob = Account(id=2, owner="Bob", balance=500.0)
        session.add(alice)
        session.add(bob)
        print(f"New objects: {session.new}")
        # Output: New objects: IdentitySet([Account(...), Account(...)])

        session.commit()
        print("Initial accounts committed")

        # 2. DIRTY: Modify existing entities
        alice = session.query(Account).get(1)
        alice.balance -= 200.0
        bob = session.query(Account).get(2)
        bob.balance += 200.0
        print(f"Dirty objects: {session.dirty}")
        # Output: Dirty objects: IdentitySet([Account(...), Account(...)])

        # 3. NEW: Log the transfer
        log = TransferLog(from_id=1, to_id=2, amount=200.0)
        session.add(log)

        # Commit all changes as one transaction
        session.commit()
        print("Transfer committed successfully")
```

```
        # Verify
        alice = session.query(Account).get(1)
        bob = session.query(Account).get(2)
        print(f"Alice: {alice.balance}")  # 800.0
        print(f"Bob: {bob.balance}")      # 700.0

    except Exception as e:
        session.rollback()
        print(f"Rolled back: {e}")
    finally:
        session.close()

demonstrate_unit_of_work()
```

Output:

```
New objects: IdentitySet([Account(id=1, owner='Alice', balance=1000.0), Account(id=2, o...
Initial accounts committed
Dirty objects: IdentitySet([Account(id=1, owner='Alice', balance=800.0), Account(id=2, ...
Transfer committed successfully
Alice: 800.0
Bob: 700.0
```

Context Manager Pattern

```python
from contextlib import contextmanager

@contextmanager
def unit_of_work():
    """Context manager that provides a transactional scope."""
    session = SessionFactory()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()


# Usage
def transfer_money(from_id: int, to_id: int, amount: float):
    with unit_of_work() as session:
        source = session.query(Account).get(from_id)
        target = session.query(Account).get(to_id)
```

```
        if source.balance < amount:
            raise ValueError("Insufficient funds")

        source.balance -= amount
        target.balance += amount

        log = TransferLog(from_id=from_id, to_id=to_id, amount=amount)
        session.add(log)

        # commit happens automatically when the with block exits
        # rollback happens automatically if an exception is raised
```

Building a Custom Unit of Work

Understanding the internals helps you appreciate what ORMs do for you. Here is a simplified custom implementation.

Python: Custom Unit of Work

```python
class UnitOfWork:
    """Tracks changes and commits them as a single transaction."""

    def __init__(self, connection):
        self.connection = connection
        self._new = []       # Entities to INSERT
        self._dirty = []     # Entities to UPDATE
        self._removed = []   # Entities to DELETE

    def register_new(self, entity):
        """Mark an entity for insertion."""
        if entity not in self._new:
            self._new.append(entity)
            print(f"  Registered NEW: {entity}")

    def register_dirty(self, entity):
        """Mark an entity for update."""
        if entity not in self._dirty and entity not in self._new:
            self._dirty.append(entity)
            print(f"  Registered DIRTY: {entity}")

    def register_removed(self, entity):
        """Mark an entity for deletion."""
        if entity in self._new:
            self._new.remove(entity)
        else:
```

```
            if entity not in self._removed:
                self._removed.append(entity)
                print(f"  Registered REMOVED: {entity}")

    def commit(self):
        """Persist all tracked changes in one transaction."""
        cursor = self.connection.cursor()
        try:
            # Process inserts
            for entity in self._new:
                sql, params = entity.to_insert_sql()
                cursor.execute(sql, params)
                print(f"  INSERT: {entity}")

            # Process updates
            for entity in self._dirty:
                sql, params = entity.to_update_sql()
                cursor.execute(sql, params)
                print(f"  UPDATE: {entity}")

            # Process deletes
            for entity in self._removed:
                sql, params = entity.to_delete_sql()
                cursor.execute(sql, params)
                print(f"  DELETE: {entity}")

            self.connection.commit()
            print("  COMMITTED all changes")

        except Exception as e:
            self.connection.rollback()
            print(f"  ROLLED BACK: {e}")
            raise
        finally:
            self._new.clear()
            self._dirty.clear()
            self._removed.clear()

    def rollback(self):
        """Discard all tracked changes."""
        self._new.clear()
        self._dirty.clear()
        self._removed.clear()
        self.connection.rollback()
        print("  ROLLED BACK all pending changes")
```

Using the Custom Unit of Work

```python
import sqlite3
```

```
# Simple entity class
class Product:
    def __init__(self, id, name, price, stock):
        self.id = id
        self.name = name
        self.price = price
        self.stock = stock

    def to_insert_sql(self):
        return (
            "INSERT INTO products (id, name, price, stock) VALUES (?,?,?,?)",
            (self.id, self.name, self.price, self.stock)
        )

    def to_update_sql(self):
        return (
            "UPDATE products SET name=?, price=?, stock=? WHERE id=?",
            (self.name, self.price, self.stock, self.id)
        )

    def to_delete_sql(self):
        return ("DELETE FROM products WHERE id=?", (self.id,))

    def __repr__(self):
        return f"Product(id={self.id}, name='{self.name}', stock={self.stock})"


# Demo
conn = sqlite3.connect(":memory:")
conn.execute("""CREATE TABLE products (
    id INTEGER PRIMARY KEY, name TEXT, price REAL, stock INTEGER
)""")

uow = UnitOfWork(conn)

# Register changes
laptop = Product(1, "Laptop", 999.99, 50)
mouse = Product(2, "Mouse", 29.99, 200)

uow.register_new(laptop)
uow.register_new(mouse)

# Simulate a stock update
laptop.stock = 45
uow.register_dirty(laptop)

# Commit everything
print("\nCommitting:")
uow.commit()
```

Output:

```
  Registered NEW: Product(id=1, name='Laptop', stock=50)
  Registered NEW: Product(id=2, name='Mouse', stock=200)
  Registered DIRTY: Product(id=1, name='Laptop', stock=45)

Committing:
  INSERT: Product(id=1, name='Laptop', stock=50)
  INSERT: Product(id=2, name='Mouse', stock=200)
  UPDATE: Product(id=1, name='Laptop', stock=45)
  COMMITTED all changes
```

Real-World Use Case: Batch Operations

The Unit of Work pattern shines in batch processing where you need to process many records efficiently.

Java: Batch Import with Unit of Work

```java
@Service
public class ProductImportService {

    private final EntityManager entityManager;

    @Transactional
    public ImportResult importProducts(List<ProductCsvRow> rows) {
        int created = 0;
        int updated = 0;
        int batchSize = 50;

        for (int i = 0; i < rows.size(); i++) {
            ProductCsvRow row = rows.get(i);

            Product existing = entityManager
                .createQuery("SELECT p FROM Product p WHERE p.sku = :sku",
                             Product.class)
                .setParameter("sku", row.getSku())
                .getResultStream()
                .findFirst()
                .orElse(null);

            if (existing != null) {
                // Dirty tracking: EntityManager detects these changes
                existing.setName(row.getName());
                existing.setPrice(row.getPrice());
                existing.setStock(row.getStock());
                updated++;
```

```
            } else {
                // New entity
                Product product = new Product(
                    row.getSku(), row.getName(),
                    row.getPrice(), row.getStock());
                entityManager.persist(product);
                created++;
            }

            // Flush and clear periodically to avoid memory issues
            if (i % batchSize == 0 && i > 0) {
                entityManager.flush();
                entityManager.clear();
                System.out.println("Flushed batch at index " + i);
            }
        }

        // Final commit happens when @Transactional method exits
        return new ImportResult(created, updated);
    }
}
```

Python: Batch Processing with Session

```python
def import_products(csv_rows: list, batch_size: int = 50):
    """Import products in batches using Unit of Work."""
    with unit_of_work() as session:
        created = 0
        updated = 0

        for i, row in enumerate(csv_rows):
            existing = session.query(Product).filter_by(
                sku=row["sku"]
            ).first()

            if existing:
                existing.name = row["name"]
                existing.price = row["price"]
                existing.stock = row["stock"]
                updated += 1
            else:
                product = Product(
                    sku=row["sku"],
                    name=row["name"],
                    price=row["price"],
                    stock=row["stock"]
                )
                session.add(product)
                created += 1
```

```
            # Flush periodically for memory efficiency
            if i % batch_size == 0 and i > 0:
                session.flush()
                print(f"Flushed batch at index {i}")

        # Commit happens automatically via context manager
        print(f"Import complete: {created} created, {updated} updated")

# Usage
csv_data = [
    {"sku": "LAP001", "name": "Laptop Pro", "price": 1299.99, "stock": 30},
    {"sku": "MOU001", "name": "Wireless Mouse", "price": 49.99, "stock": 150},
    {"sku": "KEY001", "name": "Mech Keyboard", "price": 129.99, "stock": 75},
]
import_products(csv_data)
```

Output:

```
Import complete: 3 created, 0 updated
```

Unit of Work with Repository Pattern

The Unit of Work and Repository patterns work together naturally. The repository handles entity retrieval and registration, while the Unit of Work manages the transaction.

```
+-----------------------------------------------------+
|                   Service Layer                     |
|                                                     |
|   orderService.processOrder(request)                |
+-----------------------------------------------------+
          |                    |
          v                    v
+------------------+  +------------------+
| OrderRepository  |  | InventoryRepo    |
| .save(order)     |  | .findById(id)    |
| .findById(id)    |  | .save(inv)       |
+------------------+  +------------------+
          |                    |
          +-------- + ---------+
                    |
                    v
          +------------------+
          |  Unit of Work    |
          |  (Session /      |
          |   EntityManager) |
```

```
          |                  |
          |  commit()        |
          |  rollback()      |
          +------------------+
                    |
                    v
          +------------------+
          |    Database      |
          +------------------+
```

Python: Repositories Sharing a Unit of Work

```python
class SqlAlchemyUnitOfWork:
    """Unit of Work that provides repositories sharing the same session."""

    def __init__(self, session_factory):
        self.session_factory = session_factory
        self.session = None
        self.orders = None
        self.products = None
        self.payments = None

    def __enter__(self):
        self.session = self.session_factory()
        self.orders = SqlAlchemyOrderRepository(self.session)
        self.products = SqlAlchemyProductRepository(self.session)
        self.payments = SqlAlchemyPaymentRepository(self.session)
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            self.rollback()
        self.session.close()

    def commit(self):
        self.session.commit()

    def rollback(self):
        self.session.rollback()


# Usage in service
class CheckoutService:
    def __init__(self, uow_factory):
        self.uow_factory = uow_factory

    def checkout(self, cart):
        with self.uow_factory() as uow:
            # Create order
            order = Order(customer_id=cart.customer_id,
```

```
                    total=cart.total)
        uow.orders.save(order)

        # Update inventory
        for item in cart.items:
            product = uow.products.find_by_id(item.product_id)
            product.stock -= item.quantity
            uow.products.save(product)

        # Record payment
        payment = Payment(order_id=order.id, amount=cart.total)
        uow.payments.save(payment)

        # One commit for everything
        uow.commit()

        return order
```

When to Use / When NOT to Use

Use the Unit of Work Pattern When

- A business operation modifies multiple entities that must be consistent
- You need transactional guarantees across multiple repository calls
- You are doing batch imports or updates
- You want to minimize database round-trips by batching changes
- You are building a clean architecture with explicit transaction boundaries

Do NOT Use the Unit of Work Pattern When

- Each operation is independent and does not need to be atomic
- You are working with a single entity at a time (simple CRUD)
- Your data store does not support transactions (some NoSQL databases)
- You are building a read-only application (queries only)
- The overhead of tracking changes is not justified by the complexity

---

Common Mistakes

1. Committing too early. Calling `commit()` or `flush()` in the middle of a business operation defeats the purpose. Wait until the entire operation is complete.
2. Forgetting to rollback on failure. If you catch an exception but do not rollback, the session may be in an inconsistent state. Always rollback in your error handler.
3. Long-running Units of Work. Keeping a transaction open for minutes or hours locks database rows and hurts concurrency. Keep your Unit of Work short and focused.
4. Mixing auto-commit with Unit of Work. If your database connection is in auto-commit mode, every statement is its own transaction. Make sure auto-commit is off when using Unit of Work.
5. Not flushing in batch operations. When processing thousands of entities, the Unit of Work accumulates all changes in memory. Flush periodically and clear the session to avoid out-of-memory errors.

Best Practices

1. Use framework-provided Unit of Work. Do not build your own unless you have a specific reason. JPA EntityManager, Hibernate Session, and SQLAlchemy Session are battle-tested implementations.
2. Define clear transaction boundaries. In Spring, use `@Transactional` on service methods. In Python, use context managers. The service layer is the right place for transaction boundaries.
3. Keep transactions short. Do all your non-database work (validation, calculation) before starting the Unit of Work. Only include database operations inside the transaction.
4. Pair with the Repository pattern. Let repositories handle data access and the Unit of Work handle transaction management. This gives you clean separation of concerns.
5. Flush in batches during bulk operations. For large imports, flush every 50-100 entities and clear the session to keep memory usage constant.
6. Handle nested transactions carefully. In Spring, understand the propagation settings (`REQUIRED`, `REQUIRES_NEW`, etc.). In SQLAlchemy, use savepoints for nested transactions.

Quick Summary

The Unit of Work pattern tracks all changes made to entities during a business operation and commits them as a single database transaction. This ensures data consistency -- either all

changes are saved or none are. In Java, the JPA EntityManager and Spring's `@Transactional` annotation provide built-in Unit of Work behavior. In Python, SQLAlchemy's Session tracks new, dirty, and removed objects and commits them together. The pattern is essential for any operation that touches multiple entities and must remain atomic.

Key Points

- Unit of Work tracks new, modified, and deleted entities during a business operation
- All tracked changes are committed as a single database transaction
- If any part fails, all changes are rolled back to maintain consistency
- JPA EntityManager and Hibernate Session are built-in Java implementations
- SQLAlchemy Session is the standard Python implementation
- Spring's @Transactional annotation defines the Unit of Work boundary
- Batch operations should flush periodically to manage memory
- Unit of Work pairs naturally with the Repository pattern
- Keep transactions short to avoid locking issues

Practice Questions

1. What three types of changes does a Unit of Work track? How does each type map to a SQL operation?
2. In a Spring application, what happens when an exception is thrown inside a method annotated with `@Transactional`? What if the exception is a checked exception versus an unchecked exception?
3. Explain the difference between `flush()` and `commit()` in the context of JPA and SQLAlchemy. When would you use `flush()` without committing?
4. You are importing 100,000 products from a CSV file. Without any optimizations, what problem would you encounter with the Unit of Work pattern? How would you solve it?
5. Why is it important that repositories sharing the same Unit of Work use the same database session? What would happen if they used different sessions?

Exercises

Exercise 1: Bank Transfer System

Build a bank transfer system in Python with `Account` and `TransferLog` entities. Create a `transfer()` function that deducts from one account and adds to another, logging the transfer. Use SQLAlchemy's Session as the Unit of Work. Write tests that verify: (a) successful transfers update both accounts and create a log, (b) transfers with insufficient funds roll back completely.

Exercise 2: Batch Order Processor

Create a Java service that processes a list of orders in a single transaction. Each order should: create an Order entity, deduct inventory for each item, and record a payment. Use `@Transactional` and flush every 20 orders. Write a test that processes 100 orders and verify they all commit together.

Exercise 3: Custom Unit of Work

Build a custom Unit of Work class in Python (without SQLAlchemy) that tracks new, dirty, and removed objects. Implement `register_new()`, `register_dirty()`, `register_removed()`, `commit()`, and `rollback()` methods. Use it with a simple in-memory data store and write tests demonstrating atomic commits and rollbacks.

What Is Next?

You now understand how the Repository pattern provides a clean data access interface and how the Unit of Work pattern ensures transactional consistency. The next chapter introduces CQRS (Command Query Responsibility Segregation), a pattern that takes data access to the next level by using separate models for reading and writing data. This separation allows you to optimize reads and writes independently, which is critical for high-traffic backend systems.

Chapter 25: CQRS (Command Query Responsibility Segregation)

What You Will Learn

- What CQRS is and the problem it solves
- How to separate read models from write models
- How to implement separate command and query services in Java
- How to implement CQRS in Python
- When to use different databases for reads and writes
- How CQRS applies to real-world e-commerce systems
- When CQRS is overkill and simpler patterns suffice

Why This Chapter Matters

Most backend applications start with a single data model for both reading and writing. The `Product` table stores product data, and the same model serves both the admin editing products and the customer browsing the catalog. This works fine until it does not.

As your application grows, reads and writes develop conflicting requirements. Customers want fast, denormalized product listings with ratings, reviews, and pricing tiers joined together. Admins want a normalized write model that enforces business rules. The single model becomes a compromise that serves neither audience well. Read queries slow down because of the normalization needed for writes. Write operations become complex because of the denormalized fields needed for reads.

CQRS solves this by splitting your application into two sides: a Command side that handles writes with a model optimized for enforcing business rules, and a Query side that handles reads with a model optimized for fast retrieval. Each side can be designed, scaled, and optimized independently.

The Problem

Consider an e-commerce product catalog. The write side needs to enforce rules: prices must be positive, stock cannot go negative, categories must exist. The read side needs to display products with their category names, average ratings, review counts, and calculated discount prices -- all in a single fast query.

Before: One Model for Everything

```java
// Single model tries to serve both reads and writes
@Entity
public class Product {
    @Id private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private int stock;
    private Long categoryId;
    // ... 20 more fields
}

@Service
public class ProductService {

    // WRITE: complex validation and business rules
    public void updatePrice(Long productId, BigDecimal newPrice) {
        Product product = repository.findById(productId).orElseThrow();
        if (newPrice.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Price must be positive");
        }
        product.setPrice(newPrice);
        repository.save(product);
        // Also update search index, cache, price history...
    }

    // READ: needs joins across 5 tables for a catalog page
    public ProductCatalogView getProductForCatalog(Long productId) {
        // This query joins products, categories, reviews, images, promotions
        // It is slow and complex
        return entityManager.createQuery(
            "SELECT new ProductCatalogView(p.id, p.name, p.price, " +
            "c.name, AVG(r.rating), COUNT(r.id), ...) " +
            "FROM Product p " +
            "JOIN Category c ON p.categoryId = c.id " +
            "LEFT JOIN Review r ON r.productId = p.id " +
```

```
            "LEFT JOIN ProductImage i ON i.productId = p.id " +
            "LEFT JOIN Promotion pr ON pr.productId = p.id " +
            "WHERE p.id = :id " +
            "GROUP BY p.id, p.name, p.price, c.name",
            ProductCatalogView.class
        ).setParameter("id", productId).getSingleResult();
    }
}
```

Problems:

1. The catalog query joins 5 tables and is slow
2. Every product write triggers cache invalidation for reads
3. You cannot scale reads independently from writes
4. The Product entity is bloated with fields for both purposes
5. Read-optimized indexes slow down writes, and vice versa

The Solution: CQRS

CQRS splits your system into two distinct paths:

```
                        +-------------+
                        |   Client    |
                        +------+------+
                               |
                 +-------------+-------------+
                 |                           |
                 v                           v
        +--------+--------+        +--------+--------+
        |    COMMAND       |        |     QUERY        |
        |    (Write Side)  |        |    (Read Side)   |
        +--------+--------+        +--------+---------+
                 |                           |
                 v                           v
        +--------+--------+        +---------+--------+
        | Write Model      |        | Read Model        |
        | (Normalized)     |        | (Denormalized)    |
        |                  |        |                   |
        | products         |        | product_catalog   |
        | categories       |  --->  | (pre-joined view  |
        | reviews          | sync   |  with all data    |
        | promotions       |        |  flattened)       |
        +------------------+        +-------------------+
        | PostgreSQL       |        | PostgreSQL / Redis |
        | (optimized for   |        | (optimized for    |
```

```
        |  consistency)    |        |  fast reads)      |
        +------------------+        +--------------------+
```

How CQRS Works

The Command Side (Writes)

Commands represent intentions to change state. They are imperative: "Create Product," "Update Price," "Cancel Order."

```
Command Flow:

  Client sends command
         |
         v
  +------------------+
  | Command Handler  |  <-- Validates and processes
  +------------------+
         |
         v
  +------------------+
  | Domain Model     |  <-- Business rules enforced
  | (Write Model)    |
  +------------------+
         |
         v
  +------------------+
  | Write Database   |  <-- Normalized tables
  +------------------+
         |
         v
  +------------------+
  | Publish Event    |  <-- "ProductPriceChanged"
  +------------------+
```

The Query Side (Reads)

Queries return data without modifying state. They are questions: "Get product details," "List products in category."

```
Query Flow:
```

```
Client sends query
       |
       v
+------------------+
| Query Handler    |  <-- Simple data retrieval
+------------------+
       |
       v
+------------------+
| Read Model       |  <-- Denormalized, pre-joined
| (View/Projection)|
+------------------+
       |
       v
+------------------+
| Read Database    |  <-- Optimized for fast reads
+------------------+
```

Java Implementation: Separate Command and Query Services

Step 1: Define Commands and Queries

```java
// ===== COMMANDS (Write operations) =====

public class CreateProductCommand {
    private final String name;
    private final String description;
    private final BigDecimal price;
    private final int stock;
    private final Long categoryId;

    public CreateProductCommand(String name, String description,
                                BigDecimal price, int stock,
                                Long categoryId) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.stock = stock;
        this.categoryId = categoryId;
    }

    // Getters
    public String getName() { return name; }
```

```
    public String getDescription() { return description; }
    public BigDecimal getPrice() { return price; }
    public int getStock() { return stock; }
    public Long getCategoryId() { return categoryId; }
}

public class UpdatePriceCommand {
    private final Long productId;
    private final BigDecimal newPrice;

    public UpdatePriceCommand(Long productId, BigDecimal newPrice) {
        this.productId = productId;
        this.newPrice = newPrice;
    }

    public Long getProductId() { return productId; }
    public BigDecimal getNewPrice() { return newPrice; }
}

// ===== QUERIES (Read operations) =====

public class GetProductQuery {
    private final Long productId;

    public GetProductQuery(Long productId) {
        this.productId = productId;
    }

    public Long getProductId() { return productId; }
}

public class SearchProductsQuery {
    private final String keyword;
    private final String category;
    private final int page;
    private final int size;

    public SearchProductsQuery(String keyword, String category,
                               int page, int size) {
        this.keyword = keyword;
        this.category = category;
        this.page = page;
        this.size = size;
    }

    // Getters...
    public String getKeyword() { return keyword; }
    public String getCategory() { return category; }
    public int getPage() { return page; }
    public int getSize() { return size; }
}
```

Step 2: Write Model (Command Side)

```java
// Write-side entity: normalized, enforces business rules
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String description;

    @Column(nullable = false)
    private BigDecimal price;

    @Column(nullable = false)
    private int stock;

    @Column(name = "category_id", nullable = false)
    private Long categoryId;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    // Business rule: price must be positive
    public void updatePrice(BigDecimal newPrice) {
        if (newPrice.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Price must be positive");
        }
        this.price = newPrice;
        this.updatedAt = LocalDateTime.now();
    }

    // Business rule: stock cannot go negative
    public void deductStock(int quantity) {
        if (quantity > this.stock) {
            throw new IllegalStateException("Insufficient stock");
        }
        this.stock -= quantity;
        this.updatedAt = LocalDateTime.now();
    }

    // Getters, setters, constructor omitted for brevity
}
```

Step 3: Read Model (Query Side)

```java
// Read-side view: denormalized, optimized for display
public class ProductCatalogView {
    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private BigDecimal discountedPrice;
    private int stock;
    private boolean inStock;
    private String categoryName;
    private double averageRating;
    private int reviewCount;
    private String thumbnailUrl;

    // This class is a flat, read-optimized projection
    // No business logic, just data for display

    public ProductCatalogView() {}

    public ProductCatalogView(Long id, String name, BigDecimal price,
                              String categoryName, double averageRating,
                              int reviewCount, int stock) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.categoryName = categoryName;
        this.averageRating = averageRating;
        this.reviewCount = reviewCount;
        this.stock = stock;
        this.inStock = stock > 0;
    }

    // Getters and setters...
    public Long getId() { return id; }
    public String getName() { return name; }
    public BigDecimal getPrice() { return price; }
    public String getCategoryName() { return categoryName; }
    public double getAverageRating() { return averageRating; }
    public int getReviewCount() { return reviewCount; }
    public boolean isInStock() { return inStock; }

    @Override
    public String toString() {
        return String.format("%-20s $%-8s %-12s Rating: %.1f (%d reviews) %s",
            name, price, categoryName, averageRating, reviewCount,
            inStock ? "In Stock" : "Out of Stock");
    }
}
```

Step 4: Command Handler (Write Service)

```java
@Service
public class ProductCommandService {

    private final ProductRepository productRepository;
    private final ApplicationEventPublisher eventPublisher;

    public ProductCommandService(ProductRepository productRepository,
                                 ApplicationEventPublisher eventPublisher) {
        this.productRepository = productRepository;
        this.eventPublisher = eventPublisher;
    }

    @Transactional
    public Long handle(CreateProductCommand command) {
        Product product = new Product();
        product.setName(command.getName());
        product.setDescription(command.getDescription());
        product.setPrice(command.getPrice());
        product.setStock(command.getStock());
        product.setCategoryId(command.getCategoryId());
        product.setCreatedAt(LocalDateTime.now());

        Product saved = productRepository.save(product);

        // Publish event to sync the read model
        eventPublisher.publishEvent(
            new ProductCreatedEvent(saved.getId(), saved.getName(),
                                    saved.getPrice(), saved.getStock()));

        System.out.println("Command: Created product " + saved.getId());
        return saved.getId();
    }

    @Transactional
    public void handle(UpdatePriceCommand command) {
        Product product = productRepository.findById(command.getProductId())
            .orElseThrow(() -> new RuntimeException("Product not found"));

        product.updatePrice(command.getNewPrice());
        productRepository.save(product);

        // Publish event to sync the read model
        eventPublisher.publishEvent(
            new ProductPriceChangedEvent(product.getId(),
                                         command.getNewPrice()));

        System.out.println("Command: Updated price for product " +
                           product.getId());
    }
}
```

Step 5: Query Handler (Read Service)

```java
@Service
public class ProductQueryService {

    private final JdbcTemplate jdbcTemplate;

    public ProductQueryService(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public ProductCatalogView handle(GetProductQuery query) {
        // Read directly from the denormalized read model
        String sql = """
            SELECT id, name, price, category_name, average_rating,
                   review_count, stock
            FROM product_catalog_view
            WHERE id = ?
            """;

        return jdbcTemplate.queryForObject(sql,
            (rs, rowNum) -> new ProductCatalogView(
                rs.getLong("id"),
                rs.getString("name"),
                rs.getBigDecimal("price"),
                rs.getString("category_name"),
                rs.getDouble("average_rating"),
                rs.getInt("review_count"),
                rs.getInt("stock")
            ),
            query.getProductId()
        );
    }

    public List<ProductCatalogView> handle(SearchProductsQuery query) {
        String sql = """
            SELECT id, name, price, category_name, average_rating,
                   review_count, stock
            FROM product_catalog_view
            WHERE (? IS NULL OR name ILIKE ?)
              AND (? IS NULL OR category_name = ?)
            ORDER BY average_rating DESC
            LIMIT ? OFFSET ?
            """;

        String keyword = query.getKeyword() != null
            ? "%" + query.getKeyword() + "%" : null;

        return jdbcTemplate.query(sql,
            (rs, rowNum) -> new ProductCatalogView(
                rs.getLong("id"),
                rs.getString("name"),
                rs.getBigDecimal("price"),
                rs.getString("category_name"),
                rs.getDouble("average_rating"),
                rs.getInt("review_count"),
```

```
                rs.getInt("stock")
            ),
            keyword, keyword,
            query.getCategory(), query.getCategory(),
            query.getSize(), query.getPage() * query.getSize()
        );
    }
}
```

Step 6: Event Handler (Syncs Read Model)

```java
@Component
public class ProductReadModelUpdater {

    private final JdbcTemplate jdbcTemplate;

    public ProductReadModelUpdater(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    @EventListener
    public void on(ProductCreatedEvent event) {
        jdbcTemplate.update("""
            INSERT INTO product_catalog_view
                (id, name, price, category_name, average_rating,
                 review_count, stock)
            VALUES (?, ?, ?, ?, 0.0, 0, ?)
            """,
            event.getProductId(),
            event.getName(),
            event.getPrice(),
            "Uncategorized",  // Would look up category name
            event.getStock()
        );
        System.out.println("Read model: Added product " +
                           event.getProductId());
    }

    @EventListener
    public void on(ProductPriceChangedEvent event) {
        jdbcTemplate.update(
            "UPDATE product_catalog_view SET price = ? WHERE id = ?",
            event.getNewPrice(),
            event.getProductId()
        );
        System.out.println("Read model: Updated price for product " +
                           event.getProductId());
    }
}
```

Step 7: REST Controller

```java
@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductCommandService commandService;
    private final ProductQueryService queryService;

    public ProductController(ProductCommandService commandService,
                             ProductQueryService queryService) {
        this.commandService = commandService;
        this.queryService = queryService;
    }

    // COMMAND endpoints (writes)
    @PostMapping
    public ResponseEntity<Long> createProduct(
            @RequestBody CreateProductCommand command) {
        Long id = commandService.handle(command);
        return ResponseEntity.status(HttpStatus.CREATED).body(id);
    }

    @PutMapping("/{id}/price")
    public ResponseEntity<Void> updatePrice(
            @PathVariable Long id,
            @RequestBody BigDecimal newPrice) {
        commandService.handle(new UpdatePriceCommand(id, newPrice));
        return ResponseEntity.ok().build();
    }

    // QUERY endpoints (reads)
    @GetMapping("/{id}")
    public ProductCatalogView getProduct(@PathVariable Long id) {
        return queryService.handle(new GetProductQuery(id));
    }

    @GetMapping
    public List<ProductCatalogView> searchProducts(
            @RequestParam(required = false) String keyword,
            @RequestParam(required = false) String category,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size) {
        return queryService.handle(
            new SearchProductsQuery(keyword, category, page, size));
    }
}
```

Output when running:

```
Command: Created product 1
Read model: Added product 1
```

```
Command: Created product 2
Read model: Added product 2

Command: Updated price for product 1
Read model: Updated price for product 1

Query result:
Laptop Pro           $1299.99  Electronics  Rating: 4.5 (120 reviews) In Stock
Wireless Mouse       $29.99    Accessories  Rating: 4.2 (85 reviews)  In Stock
```

Python Implementation

```python
from dataclasses import dataclass, field
from typing import List, Optional
from datetime import datetime
from abc import ABC, abstractmethod

# ===== COMMANDS =====

@dataclass
class CreateProductCommand:
    name: str
    description: str
    price: float
    stock: int
    category_id: int

@dataclass
class UpdatePriceCommand:
    product_id: int
    new_price: float

# ===== QUERIES =====

@dataclass
class GetProductQuery:
    product_id: int

@dataclass
class SearchProductsQuery:
    keyword: Optional[str] = None
    category: Optional[str] = None
    page: int = 0
    size: int = 20
```

```
# ===== WRITE MODEL =====

class Product:
    """Write model: enforces business rules."""

    def __init__(self, id, name, description, price, stock, category_id):
        self.id = id
        self.name = name
        self.description = description
        self.price = price
        self.stock = stock
        self.category_id = category_id
        self.created_at = datetime.utcnow()
        self.updated_at = datetime.utcnow()

    def update_price(self, new_price: float):
        if new_price <= 0:
            raise ValueError("Price must be positive")
        self.price = new_price
        self.updated_at = datetime.utcnow()

    def deduct_stock(self, quantity: int):
        if quantity > self.stock:
            raise ValueError("Insufficient stock")
        self.stock -= quantity
        self.updated_at = datetime.utcnow()

# ===== READ MODEL =====

@dataclass
class ProductCatalogView:
    """Read model: optimized for display."""
    id: int
    name: str
    price: float
    category_name: str
    average_rating: float
    review_count: int
    stock: int
    in_stock: bool = True

    def __post_init__(self):
        self.in_stock = self.stock > 0

    def __str__(self):
        status = "In Stock" if self.in_stock else "Out of Stock"
        return (f"{self.name:<20} ${self.price:<8.2f} "
                f"{self.category_name:<12} "
                f"Rating: {self.average_rating:.1f} "
                f"({self.review_count} reviews) {status}")


# ===== COMMAND HANDLER =====

class ProductCommandHandler:
```

```
    """Handles write operations."""

    def __init__(self, write_repo, event_bus):
        self.write_repo = write_repo
        self.event_bus = event_bus

    def handle_create(self, command: CreateProductCommand) -> int:
        product = Product(
            id=None,
            name=command.name,
            description=command.description,
            price=command.price,
            stock=command.stock,
            category_id=command.category_id
        )
        saved = self.write_repo.save(product)
        print(f"Command: Created product {saved.id}")

        # Publish event to sync read model
        self.event_bus.publish("product_created", {
            "id": saved.id,
            "name": saved.name,
            "price": saved.price,
            "stock": saved.stock,
            "category_id": saved.category_id,
        })
        return saved.id

    def handle_update_price(self, command: UpdatePriceCommand):
        product = self.write_repo.find_by_id(command.product_id)
        if product is None:
            raise ValueError(f"Product not found: {command.product_id}")

        product.update_price(command.new_price)
        self.write_repo.save(product)
        print(f"Command: Updated price for product {product.id}")

        self.event_bus.publish("product_price_changed", {
            "id": product.id,
            "new_price": command.new_price,
        })


# ===== QUERY HANDLER =====

class ProductQueryHandler:
    """Handles read operations from the read model."""

    def __init__(self, read_repo):
        self.read_repo = read_repo

    def handle_get(self, query: GetProductQuery) -> Optional[ProductCatalogView]:
        return self.read_repo.find_by_id(query.product_id)

    def handle_search(self, query: SearchProductsQuery) -> List[ProductCatalogView]:
```

```
        results = self.read_repo.find_all()

        if query.keyword:
            results = [p for p in results
                       if query.keyword.lower() in p.name.lower()]
        if query.category:
            results = [p for p in results
                       if p.category_name == query.category]

        start = query.page * query.size
        return results[start:start + query.size]


# ===== EVENT BUS (syncs read model) =====

class SimpleEventBus:
    def __init__(self):
        self._handlers = {}

    def subscribe(self, event_type, handler):
        self._handlers.setdefault(event_type, []).append(handler)

    def publish(self, event_type, data):
        for handler in self._handlers.get(event_type, []):
            handler(data)


# ===== IN-MEMORY REPOSITORIES =====

class InMemoryWriteRepo:
    def __init__(self):
        self._store = {}
        self._next_id = 1

    def save(self, product):
        if product.id is None:
            product.id = self._next_id
            self._next_id += 1
        self._store[product.id] = product
        return product

    def find_by_id(self, product_id):
        return self._store.get(product_id)


class InMemoryReadRepo:
    def __init__(self):
        self._store = {}

    def save(self, view: ProductCatalogView):
        self._store[view.id] = view

    def find_by_id(self, product_id):
        return self._store.get(product_id)
```

```
    def find_all(self):
        return list(self._store.values())

    def update_price(self, product_id, new_price):
        if product_id in self._store:
            self._store[product_id].price = new_price


# ===== WIRE IT TOGETHER =====

# Categories lookup (simplified)
categories = {1: "Electronics", 2: "Accessories", 3: "Books"}

# Setup
write_repo = InMemoryWriteRepo()
read_repo = InMemoryReadRepo()
event_bus = SimpleEventBus()

# Read model updater (subscribes to events)
def on_product_created(data):
    view = ProductCatalogView(
        id=data["id"],
        name=data["name"],
        price=data["price"],
        category_name=categories.get(data["category_id"], "Unknown"),
        average_rating=0.0,
        review_count=0,
        stock=data["stock"],
    )
    read_repo.save(view)
    print(f"Read model: Added product {data['id']}")

def on_price_changed(data):
    read_repo.update_price(data["id"], data["new_price"])
    print(f"Read model: Updated price for product {data['id']}")

event_bus.subscribe("product_created", on_product_created)
event_bus.subscribe("product_price_changed", on_price_changed)

# Handlers
command_handler = ProductCommandHandler(write_repo, event_bus)
query_handler = ProductQueryHandler(read_repo)

# Use the system
command_handler.handle_create(
    CreateProductCommand("Laptop Pro", "High-end laptop", 1299.99, 50, 1))
command_handler.handle_create(
    CreateProductCommand("Wireless Mouse", "Ergonomic mouse", 29.99, 200, 2))

command_handler.handle_update_price(UpdatePriceCommand(1, 1199.99))

print("\nSearch results:")
results = query_handler.handle_search(SearchProductsQuery())
for product in results:
    print(f"  {product}")
```

Output:

```
Command: Created product 1
Read model: Added product 1
Command: Created product 2
Read model: Added product 2
Command: Updated price for product 1
Read model: Updated price for product 1

Search results:
  Laptop Pro           $1199.99  Electronics  Rating: 0.0 (0 reviews) In Stock
  Wireless Mouse       $29.99    Accessories  Rating: 0.0 (0 reviews) In Stock
```

Using Different Databases for Read and Write

In advanced CQRS, you can use entirely different databases:

```
+--------------------+          +--------------------+
|   Command Side     |          |    Query Side      |
+--------------------+          +--------------------+
|                    |          |                    |
| PostgreSQL         |  Events  | Elasticsearch      |
| (Normalized)       | -------> | (Full-text search)|
|                    |          |                    |
| - products         |          | - product_catalog |
| - categories       |   or     |   (denormalized    |
| - reviews          |          |    documents)      |
| - inventory        |          |                    |
+--------------------+  Redis   +--------------------+
                        Cache
                          |
                          v
                     +----------+
                     |  Redis   |
                     | (Hot     |
                     |  data    |
                     |  cache)  |
                     +----------+
```

Synchronization Strategies

```
Strategy 1: Synchronous (Simple, Consistent)
  Write --> Update Read Model --> Return Response
  Pro: Read model always up to date
  Con: Slower writes

Strategy 2: Asynchronous via Events (Scalable)
  Write --> Return Response
            \--> Event Queue --> Update Read Model (eventually)
  Pro: Fast writes, scalable
  Con: Read model may be stale briefly (eventual consistency)

Strategy 3: Periodic Sync (Simple, Batch)
  Write --> Return Response
  Cron job (every 5 min) --> Rebuild Read Model
  Pro: Very simple
  Con: Data can be up to 5 minutes stale
```

Real-World: E-Commerce Product Catalog

```
WRITE SIDE (Admin Panel)                READ SIDE (Customer Storefront)
+---------------------------+          +---------------------------+
| Normalized PostgreSQL     |          | Denormalized Read Store   |
|                           |          |                           |
| products                  |          | product_catalog_view      |
|   id, name, price,        |          |   id, name, price,        |
|   description, category_id|  Events  |   discounted_price,       |
|                           | -------> |   category_name,          |
| categories                |          |   avg_rating,             |
|   id, name, parent_id     |          |   review_count,           |
|                           |          |   thumbnail_url,          |
| reviews                   |          |   in_stock,               |
|   id, product_id, rating, |          |   tags                    |
|   comment                 |          |                           |
|                           |          | (Single table, no joins,  |
| promotions                |          |  indexed for search)      |
|   id, product_id,         |          |                           |
|   discount_percent        |          +---------------------------+
+---------------------------+
```

The write side has 4 normalized tables with referential integrity. The read side has 1 flat table with all data pre-joined. A customer catalog query hits one table with no joins, returning results in milliseconds.

When CQRS Is Overkill

CQRS adds complexity. Do not use it when:

```
+-------------------------------+-----------------------------------+
| Scenario                      | Better Alternative                |
+-------------------------------+-----------------------------------+
| Simple CRUD application       | Standard service + repository     |
| Read and write models are     | Single model works fine           |
|   nearly identical            |                                   |
| Low traffic (< 100 req/sec)   | Single database with indexes      |
| Small team (1-3 developers)   | Simpler architecture, less        |
|                               |   cognitive overhead              |
| Prototype / MVP               | Ship fast, refactor later         |
| Data is always read right     | Single source of truth is         |
|   after write                 |   simpler                         |
+-------------------------------+-----------------------------------+
```

When to Use / When NOT to Use

Use CQRS When

- Read and write workloads have very different performance requirements
- You need to scale reads and writes independently
- Your read queries require complex joins across many tables
- You are building an event-driven system
- Different teams own the read and write sides
- You need different data stores for different access patterns

Do NOT Use CQRS When

- Your read and write models look the same
- You have a simple CRUD application
- Eventual consistency is not acceptable for your use case
- Your team is small and the added complexity is not justified
- You do not have the infrastructure to manage separate data stores

---

Common Mistakes

1. Applying CQRS everywhere. CQRS is a pattern for specific parts of your system, not the entire application. Use it only where read and write models genuinely diverge.
2. Ignoring eventual consistency. When the read model is updated asynchronously, it may be slightly behind the write model. Your UI must handle this gracefully.
3. Making the read model too complex. The read model should be simple and flat. If you are writing complex query logic in the read service, you are doing it wrong.
4. Not handling sync failures. If the event that updates the read model fails, the data becomes permanently stale. Build retry mechanisms and monitoring.
5. Using CQRS without events. You can use CQRS without event sourcing, but you need a synchronization mechanism. Without one, the read model drifts.

Best Practices

1. Start with a single database. Use database views or materialized views for the read model before introducing separate databases.
2. Use events for synchronization. Domain events are the natural way to keep the read model in sync with the write model.
3. Design read models for specific use cases. Create one read model per screen or API endpoint. A product list view and a product detail view can be different read models.
4. Monitor sync lag. Track the delay between a write and when the read model reflects it. Alert if it exceeds your SLA.
5. Make commands explicit. Name your commands after business actions: `PlaceOrder`, `CancelSubscription`, `ApproveRefund`. Not `UpdateOrderStatus`.

Quick Summary

CQRS separates your application into a command side (writes) and a query side (reads), each with its own model optimized for its purpose. The write model is normalized and enforces business rules. The read model is denormalized and optimized for fast retrieval. Events synchronize the two sides. CQRS enables independent scaling and optimization of reads and writes but adds complexity through eventual consistency and data synchronization. Use it when read and write patterns genuinely differ; avoid it for simple CRUD applications.

Key Points

- CQRS splits the application into separate command (write) and query (read) paths
- The write model is normalized and enforces business rules
- The read model is denormalized and optimized for fast retrieval
- Events synchronize the write model with the read model
- You can use different databases for reads and writes
- Eventual consistency is a trade-off you must design around
- CQRS is not all-or-nothing; apply it to specific bounded contexts
- Start simple with database views before introducing separate stores
- CQRS pairs naturally with Event Sourcing (covered in the next chapter)

Practice Questions

1. Explain the difference between the command model and the query model in CQRS. Why would you want them to be different?
2. What is eventual consistency, and how does it affect the user experience in a CQRS system? How would you handle a scenario where a user creates a product and immediately tries to search for it?
3. You have an e-commerce system where 95% of traffic is product browsing (reads) and 5% is admin updates (writes). How would CQRS help you scale this system?
4. Compare synchronous and asynchronous synchronization strategies for keeping the read model updated. What are the trade-offs of each?
5. A colleague wants to apply CQRS to a simple blog application with 100 daily users. What would you advise?

Exercises

Exercise 1: Blog with CQRS

Build a simple blog system using CQRS. The write side handles `CreatePost`, `UpdatePost`, and `PublishPost` commands. The read side provides `GetPost`, `ListPublishedPosts`, and `SearchPosts` queries. Use events to sync the read model. The read model should include the author name (joined from a users table on the write side).

Exercise 2: Inventory CQRS

Create an inventory system where the write side tracks stock levels with business rules (no negative stock, maximum stock limits). The read side provides a dashboard view with product name, current stock, stock status (Low/Medium/High), and last restocked date -- all in a single flat query.

Exercise 3: Multi-View Read Models

Extend the e-commerce example to have three different read models: (1) a product list view with minimal data for catalog pages, (2) a product detail view with full information, and (3) a product analytics view with sales counts and revenue. Demonstrate how all three are updated from the same write events.

What Is Next?

CQRS separates how you read and write data, but both sides still store the current state. The next chapter introduces Event Sourcing, where instead of storing the current state, you store the sequence of events that led to that state. Event Sourcing and CQRS are natural partners: events from the write side rebuild the read model. Together, they provide a complete audit trail, the ability to replay history, and powerful debugging capabilities.

Chapter 26: Event Sourcing

What You Will Learn

- What Event Sourcing is and how it differs from traditional state storage
- How to store events instead of current state
- How to rebuild current state from a sequence of events
- How to use snapshots for performance optimization
- How to implement event sourcing for a bank account in Java
- How Event Sourcing combines with CQRS
- Real-world applications in financial systems and audit trails
- The complexity trade-offs of adopting Event Sourcing

Why This Chapter Matters

Traditional databases store only the current state of your data. When a bank account has a balance of $500, that is all you know. You do not know if the customer deposited $1000 and then withdrew $500, or if they received five $100 deposits. The history is lost.

Event Sourcing flips this approach. Instead of storing the current state, you store every event that happened: "Account Created," "Money Deposited $1000," "Money Withdrawn $500." The current state is derived by replaying these events. You never lose history. You can answer questions like "what was the balance on March 15th?" or "how many deposits did this customer make last quarter?"

This is not just about history. Event Sourcing enables powerful capabilities: complete audit trails, temporal queries, debugging by replaying events, and undo operations. Financial systems, healthcare platforms, and compliance-heavy applications use Event Sourcing because the law often requires a complete record of every change.

The Problem

Traditional State Storage

```
Traditional approach: Only the CURRENT state is stored

  Time 0:  Account created with $0        DB: balance = 0
  Time 1:  Deposit $1000                  DB: balance = 1000
  Time 2:  Withdraw $200                  DB: balance = 800
  Time 3:  Deposit $500                   DB: balance = 1300
  Time 4:  Withdraw $100                  DB: balance = 1200

  What the database knows:  balance = 1200
  What is lost:  HOW we got to 1200

  Questions you CANNOT answer:
    - What was the balance at Time 2?
    - How many deposits were made?
    - What was the largest withdrawal?
    - Did anyone modify the balance directly?
```

Before: State-Based Updates

```java
// Traditional approach: update state directly
@Entity
public class BankAccount {
    @Id private Long id;
    private String owner;
    private BigDecimal balance;

    public void deposit(BigDecimal amount) {
        this.balance = this.balance.add(amount);
        // Previous balance is GONE
    }

    public void withdraw(BigDecimal amount) {
        if (amount.compareTo(this.balance) > 0) {
            throw new InsufficientFundsException();
        }
        this.balance = this.balance.subtract(amount);
        // No record of this withdrawal
    }
}
```

The Solution: Event Sourcing

Instead of storing state, store the events that produce that state.

```
Event Sourcing approach: Store EVERY event

  Event Store:
  +-----+-------------------+-------------------+----------+
  | Seq |   Event Type      |   Data            | Timestamp|
  +-----+-------------------+-------------------+----------+
  |  1  | AccountCreated    | owner: "Alice"    | 10:00:00 |
  |  2  | MoneyDeposited    | amount: 1000      | 10:05:00 |
  |  3  | MoneyWithdrawn    | amount: 200       | 10:10:00 |
  |  4  | MoneyDeposited    | amount: 500       | 11:00:00 |
  |  5  | MoneyWithdrawn    | amount: 100       | 11:30:00 |
  +-----+-------------------+-------------------+----------+

  To get current state: REPLAY all events
    Start:   balance = 0
    Event 1: AccountCreated    -> balance = 0
    Event 2: MoneyDeposited    -> balance = 0 + 1000 = 1000
    Event 3: MoneyWithdrawn    -> balance = 1000 - 200 = 800
    Event 4: MoneyDeposited    -> balance = 800 + 500 = 1300
    Event 5: MoneyWithdrawn    -> balance = 1300 - 100 = 1200

  Current balance: 1200 (same result, but with FULL history)
```

How Event Sourcing Works

```
+-------------------+       +-------------------+       +-----------------+
|                   |       |                   |       |                 |
|  Command          | ----> |  Domain Logic     | ----> |  Event Store    |
|  "Deposit $500"   |       |  (Validate +      |       |  (Append Only)  |
|                   |       |   Generate Event)  |       |                 |
+-------------------+       +-------------------+       +-----------------+
                                                                |
                                                                v
                                                        +-------+-------+
                                                        |  Event 1      |
                                                        |  Event 2      |
                                                        |  Event 3      |
                                                        |  ...          |
                                                        |  Event N      |
                                                        +---------------+
                                                                |
                                                        Replay  |
```

```
                                                                    v
                                                              +---------------+
                                                              | Current State |
                                                              | balance: 1200 |
                                                              +---------------+
```

Key principles:

1. Events are immutable -- once stored, they never change
2. Events are append-only -- you only add new events, never delete old ones
3. State is derived -- current state is computed by replaying events
4. Events capture intent -- "MoneyDeposited" not "BalanceUpdated"

Java Implementation: Bank Account Event Sourcing

Step 1: Define Events

```java
import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

// Base event
public abstract class DomainEvent {
    private final String eventId;
    private final String aggregateId;
    private final Instant timestamp;
    private final int version;

    protected DomainEvent(String aggregateId, int version) {
        this.eventId = UUID.randomUUID().toString();
        this.aggregateId = aggregateId;
        this.timestamp = Instant.now();
        this.version = version;
    }

    public String getEventId() { return eventId; }
    public String getAggregateId() { return aggregateId; }
    public Instant getTimestamp() { return timestamp; }
    public int getVersion() { return version; }
}
```

```
// Concrete events
public class AccountCreatedEvent extends DomainEvent {
    private final String owner;
    private final BigDecimal initialBalance;

    public AccountCreatedEvent(String aggregateId, int version,
                               String owner, BigDecimal initialBalance) {
        super(aggregateId, version);
        this.owner = owner;
        this.initialBalance = initialBalance;
    }

    public String getOwner() { return owner; }
    public BigDecimal getInitialBalance() { return initialBalance; }

    @Override
    public String toString() {
        return String.format("AccountCreated{owner='%s', balance=%s}",
                             owner, initialBalance);
    }
}

public class MoneyDepositedEvent extends DomainEvent {
    private final BigDecimal amount;
    private final String description;

    public MoneyDepositedEvent(String aggregateId, int version,
                               BigDecimal amount, String description) {
        super(aggregateId, version);
        this.amount = amount;
        this.description = description;
    }

    public BigDecimal getAmount() { return amount; }
    public String getDescription() { return description; }

    @Override
    public String toString() {
        return String.format("MoneyDeposited{amount=%s, desc='%s'}",
                             amount, description);
    }
}

public class MoneyWithdrawnEvent extends DomainEvent {
    private final BigDecimal amount;
    private final String description;

    public MoneyWithdrawnEvent(String aggregateId, int version,
                               BigDecimal amount, String description) {
        super(aggregateId, version);
        this.amount = amount;
        this.description = description;
    }

    public BigDecimal getAmount() { return amount; }
```

```
    public String getDescription() { return description; }

    @Override
    public String toString() {
        return String.format("MoneyWithdrawn{amount=%s, desc='%s'}",
                             amount, description);
    }
}
```

Step 2: Event-Sourced Aggregate

```java
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

public class BankAccount {

    private String id;
    private String owner;
    private BigDecimal balance;
    private int version;

    // Uncommitted events generated by commands
    private final List<DomainEvent> uncommittedEvents = new ArrayList<>();

    // Private constructor -- use factory methods
    private BankAccount() {
        this.balance = BigDecimal.ZERO;
        this.version = 0;
    }

    // ===== FACTORY: Create from events (rebuilding state) =====
    public static BankAccount fromEvents(List<DomainEvent> events) {
        BankAccount account = new BankAccount();
        for (DomainEvent event : events) {
            account.apply(event);
        }
        return account;
    }

    // ===== COMMANDS: Business operations that generate events =====

    public static BankAccount create(String id, String owner,
                                     BigDecimal initialBalance) {
        BankAccount account = new BankAccount();
        AccountCreatedEvent event = new AccountCreatedEvent(
            id, account.version + 1, owner, initialBalance);
        account.apply(event);
        account.uncommittedEvents.add(event);
        return account;
```

```
    }

    public void deposit(BigDecimal amount, String description) {
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive");
        }
        MoneyDepositedEvent event = new MoneyDepositedEvent(
            this.id, this.version + 1, amount, description);
        apply(event);
        uncommittedEvents.add(event);
    }

    public void withdraw(BigDecimal amount, String description) {
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException(
                "Withdrawal amount must be positive");
        }
        if (amount.compareTo(this.balance) > 0) {
            throw new IllegalStateException(
                "Insufficient funds. Balance: " + this.balance +
                ", Requested: " + amount);
        }
        MoneyWithdrawnEvent event = new MoneyWithdrawnEvent(
            this.id, this.version + 1, amount, description);
        apply(event);
        uncommittedEvents.add(event);
    }

    // ===== EVENT APPLICATION: Update state based on event type =====

    private void apply(DomainEvent event) {
        if (event instanceof AccountCreatedEvent e) {
            this.id = e.getAggregateId();
            this.owner = e.getOwner();
            this.balance = e.getInitialBalance();
        } else if (event instanceof MoneyDepositedEvent e) {
            this.balance = this.balance.add(e.getAmount());
        } else if (event instanceof MoneyWithdrawnEvent e) {
            this.balance = this.balance.subtract(e.getAmount());
        }
        this.version = event.getVersion();
    }

    // ===== GETTERS =====

    public String getId() { return id; }
    public String getOwner() { return owner; }
    public BigDecimal getBalance() { return balance; }
    public int getVersion() { return version; }

    public List<DomainEvent> getUncommittedEvents() {
        return new ArrayList<>(uncommittedEvents);
    }

    public void clearUncommittedEvents() {
```

```
        uncommittedEvents.clear();
    }

    @Override
    public String toString() {
        return String.format("BankAccount{id='%s', owner='%s', balance=%s, version=%d}",
                             id, owner, balance, version);
    }
}
```

Step 3: Event Store

```java
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.stream.Collectors;

public class InMemoryEventStore {

    // aggregateId -> list of events
    private final Map<String, List<DomainEvent>> store =
        new ConcurrentHashMap<>();

    public void save(String aggregateId, List<DomainEvent> events,
                     int expectedVersion) {
        List<DomainEvent> existingEvents =
            store.getOrDefault(aggregateId, new ArrayList<>());

        // Optimistic concurrency check
        int currentVersion = existingEvents.size();
        if (currentVersion != expectedVersion) {
            throw new ConcurrentModificationException(
                "Expected version " + expectedVersion +
                " but found " + currentVersion);
        }

        existingEvents.addAll(events);
        store.put(aggregateId, existingEvents);
    }

    public List<DomainEvent> getEvents(String aggregateId) {
        return store.getOrDefault(aggregateId, Collections.emptyList());
    }

    public List<DomainEvent> getEventsUpToVersion(String aggregateId,
                                                  int maxVersion) {
        return getEvents(aggregateId).stream()
            .filter(e -> e.getVersion() <= maxVersion)
            .collect(Collectors.toList());
    }
```

```
    public List<DomainEvent> getAllEvents() {
        return store.values().stream()
            .flatMap(Collection::stream)
            .sorted(Comparator.comparing(DomainEvent::getTimestamp))
            .collect(Collectors.toList());
    }
}
```

Step 4: Account Service

```java
public class BankAccountService {

    private final InMemoryEventStore eventStore;

    public BankAccountService(InMemoryEventStore eventStore) {
        this.eventStore = eventStore;
    }

    public BankAccount createAccount(String id, String owner,
                                     BigDecimal initialBalance) {
        BankAccount account = BankAccount.create(id, owner, initialBalance);

        eventStore.save(id, account.getUncommittedEvents(), 0);
        account.clearUncommittedEvents();

        System.out.println("Created: " + account);
        return account;
    }

    public void deposit(String accountId, BigDecimal amount,
                        String description) {
        // Rebuild state from events
        List<DomainEvent> events = eventStore.getEvents(accountId);
        BankAccount account = BankAccount.fromEvents(events);

        // Execute command
        account.deposit(amount, description);

        // Save new events
        eventStore.save(accountId, account.getUncommittedEvents(),
                        events.size());
        account.clearUncommittedEvents();

        System.out.println("Deposited " + amount + " -> " + account);
    }

    public void withdraw(String accountId, BigDecimal amount,
                         String description) {
        List<DomainEvent> events = eventStore.getEvents(accountId);
        BankAccount account = BankAccount.fromEvents(events);
```

```
        account.withdraw(amount, description);

        eventStore.save(accountId, account.getUncommittedEvents(),
                        events.size());
        account.clearUncommittedEvents();

        System.out.println("Withdrew " + amount + " -> " + account);
    }

    public BankAccount getAccount(String accountId) {
        List<DomainEvent> events = eventStore.getEvents(accountId);
        if (events.isEmpty()) {
            throw new RuntimeException("Account not found: " + accountId);
        }
        return BankAccount.fromEvents(events);
    }

    // Temporal query: get balance at a specific point in time
    public BankAccount getAccountAtVersion(String accountId, int version) {
        List<DomainEvent> events = eventStore.getEventsUpToVersion(
            accountId, version);
        return BankAccount.fromEvents(events);
    }

    public void printEventHistory(String accountId) {
        List<DomainEvent> events = eventStore.getEvents(accountId);
        System.out.println("\nEvent history for account " + accountId + ":");
        for (DomainEvent event : events) {
            System.out.println("  v" + event.getVersion() + ": " + event);
        }
    }
}
```

Step 5: Running the System

```java
public class EventSourcingDemo {

    public static void main(String[] args) {
        InMemoryEventStore eventStore = new InMemoryEventStore();
        BankAccountService service = new BankAccountService(eventStore);

        // Create account
        service.createAccount("ACC-001", "Alice", BigDecimal.ZERO);

        // Perform transactions
        service.deposit("ACC-001", new BigDecimal("1000"), "Initial deposit");
        service.deposit("ACC-001", new BigDecimal("500"), "Salary");
        service.withdraw("ACC-001", new BigDecimal("200"), "Groceries");
        service.deposit("ACC-001", new BigDecimal("50"), "Refund");
```

```
        service.withdraw("ACC-001", new BigDecimal("100"), "Utilities");

        // Print event history
        service.printEventHistory("ACC-001");

        // Temporal query: what was the balance after the salary deposit?
        BankAccount atVersion3 = service.getAccountAtVersion("ACC-001", 3);
        System.out.println("\nBalance at version 3: " + atVersion3.getBalance());

        // Current state
        BankAccount current = service.getAccount("ACC-001");
        System.out.println("Current balance: " + current.getBalance());
    }
}
```

Output:

```
Created: BankAccount{id='ACC-001', owner='Alice', balance=0, version=1}
Deposited 1000 -> BankAccount{id='ACC-001', owner='Alice', balance=1000, version=2}
Deposited 500 -> BankAccount{id='ACC-001', owner='Alice', balance=1500, version=3}
Withdrew 200 -> BankAccount{id='ACC-001', owner='Alice', balance=1300, version=4}
Deposited 50 -> BankAccount{id='ACC-001', owner='Alice', balance=1350, version=5}
Withdrew 100 -> BankAccount{id='ACC-001', owner='Alice', balance=1250, version=6}

Event history for account ACC-001:
  v1: AccountCreated{owner='Alice', balance=0}
  v2: MoneyDeposited{amount=1000, desc='Initial deposit'}
  v3: MoneyDeposited{amount=500, desc='Salary'}
  v4: MoneyWithdrawn{amount=200, desc='Groceries'}
  v5: MoneyDeposited{amount=50, desc='Refund'}
  v6: MoneyWithdrawn{amount=100, desc='Utilities'}

Balance at version 3: 1500
Current balance: 1250
```

Snapshots for Performance

As events accumulate, replaying them all becomes slow. Snapshots capture the state at a point in time so you only need to replay events after the snapshot.

```
Without snapshots (1000 events):
  Replay event 1, 2, 3, ... 998, 999, 1000  --> Current State
  Time: O(n) where n = total events

With snapshot at event 900:
```

```
  Load snapshot (state at event 900)
  Replay event 901, 902, ... 999, 1000  --> Current State
  Time: O(k) where k = events since snapshot
```

Java: Snapshot Implementation

```java
public class Snapshot {
    private final String aggregateId;
    private final int version;
    private final BigDecimal balance;
    private final String owner;
    private final Instant createdAt;

    public Snapshot(String aggregateId, int version,
                    BigDecimal balance, String owner) {
        this.aggregateId = aggregateId;
        this.version = version;
        this.balance = balance;
        this.owner = owner;
        this.createdAt = Instant.now();
    }

    // Getters
    public String getAggregateId() { return aggregateId; }
    public int getVersion() { return version; }
    public BigDecimal getBalance() { return balance; }
    public String getOwner() { return owner; }
}

public class SnapshotStore {
    private final Map<String, Snapshot> snapshots = new HashMap<>();

    public void save(Snapshot snapshot) {
        snapshots.put(snapshot.getAggregateId(), snapshot);
        System.out.println("  Snapshot saved at version " +
                           snapshot.getVersion());
    }

    public Snapshot getLatest(String aggregateId) {
        return snapshots.get(aggregateId);
    }
}

// Enhanced service with snapshots
public class BankAccountServiceWithSnapshots {

    private final InMemoryEventStore eventStore;
    private final SnapshotStore snapshotStore;
    private final int snapshotInterval;
```

```
    public BankAccountServiceWithSnapshots(InMemoryEventStore eventStore,
                                           SnapshotStore snapshotStore,
                                           int snapshotInterval) {
        this.eventStore = eventStore;
        this.snapshotStore = snapshotStore;
        this.snapshotInterval = snapshotInterval;
    }

    public BankAccount getAccount(String accountId) {
        Snapshot snapshot = snapshotStore.getLatest(accountId);
        List<DomainEvent> events;

        BankAccount account;
        if (snapshot != null) {
            // Load from snapshot + replay only recent events
            events = eventStore.getEvents(accountId).stream()
                .filter(e -> e.getVersion() > snapshot.getVersion())
                .toList();

            account = BankAccount.fromSnapshot(snapshot);
            for (DomainEvent event : events) {
                account.replayEvent(event);
            }

            System.out.println("  Loaded from snapshot v" +
                               snapshot.getVersion() +
                               " + replayed " + events.size() + " events");
        } else {
            // No snapshot, replay all events
            events = eventStore.getEvents(accountId);
            account = BankAccount.fromEvents(events);
            System.out.println("  Loaded by replaying " +
                               events.size() + " events");
        }

        // Create snapshot if needed
        if (account.getVersion() % snapshotInterval == 0) {
            snapshotStore.save(new Snapshot(
                accountId, account.getVersion(),
                account.getBalance(), account.getOwner()));
        }

        return account;
    }
}
```

Output with snapshots (interval = 5):

```
After 7 transactions:
  Loaded from snapshot v5 + replayed 2 events
  Balance: 1250

Without snapshots:
```

```
  Loaded by replaying 7 events
  Balance: 1250
```

Python Implementation

```python
from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Optional, Dict
import uuid

# ===== EVENTS =====

@dataclass
class DomainEvent:
    event_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    aggregate_id: str = ""
    version: int = 0
    timestamp: datetime = field(default_factory=datetime.utcnow)

@dataclass
class AccountCreated(DomainEvent):
    owner: str = ""
    initial_balance: float = 0.0

@dataclass
class MoneyDeposited(DomainEvent):
    amount: float = 0.0
    description: str = ""

@dataclass
class MoneyWithdrawn(DomainEvent):
    amount: float = 0.0
    description: str = ""


# ===== EVENT-SOURCED AGGREGATE =====

class BankAccount:
    def __init__(self):
        self.id = ""
        self.owner = ""
        self.balance = 0.0
        self.version = 0
        self._uncommitted_events: List[DomainEvent] = []

    @classmethod
```

```
    def create(cls, account_id: str, owner: str,
               initial_balance: float = 0.0) -> "BankAccount":
        account = cls()
        event = AccountCreated(
            aggregate_id=account_id,
            version=1,
            owner=owner,
            initial_balance=initial_balance
        )
        account._apply(event)
        account._uncommitted_events.append(event)
        return account

    @classmethod
    def from_events(cls, events: List[DomainEvent]) -> "BankAccount":
        account = cls()
        for event in events:
            account._apply(event)
        return account

    def deposit(self, amount: float, description: str = ""):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        event = MoneyDeposited(
            aggregate_id=self.id,
            version=self.version + 1,
            amount=amount,
            description=description
        )
        self._apply(event)
        self._uncommitted_events.append(event)

    def withdraw(self, amount: float, description: str = ""):
        if amount <= 0:
            raise ValueError("Withdrawal amount must be positive")
        if amount > self.balance:
            raise ValueError(
                f"Insufficient funds. Balance: {self.balance}, "
                f"Requested: {amount}")
        event = MoneyWithdrawn(
            aggregate_id=self.id,
            version=self.version + 1,
            amount=amount,
            description=description
        )
        self._apply(event)
        self._uncommitted_events.append(event)

    def _apply(self, event: DomainEvent):
        if isinstance(event, AccountCreated):
            self.id = event.aggregate_id
            self.owner = event.owner
            self.balance = event.initial_balance
        elif isinstance(event, MoneyDeposited):
            self.balance += event.amount
```

```
        elif isinstance(event, MoneyWithdrawn):
            self.balance -= event.amount
        self.version = event.version

    def get_uncommitted_events(self) -> List[DomainEvent]:
        return list(self._uncommitted_events)

    def clear_uncommitted_events(self):
        self._uncommitted_events.clear()

    def __repr__(self):
        return (f"BankAccount(id='{self.id}', owner='{self.owner}', "
                f"balance={self.balance:.2f}, version={self.version})")


# ===== EVENT STORE =====

class EventStore:
    def __init__(self):
        self._store: Dict[str, List[DomainEvent]] = {}

    def save(self, aggregate_id: str, events: List[DomainEvent],
             expected_version: int):
        existing = self._store.get(aggregate_id, [])
        if len(existing) != expected_version:
            raise Exception(
                f"Concurrency conflict: expected version "
                f"{expected_version}, found {len(existing)}")
        existing.extend(events)
        self._store[aggregate_id] = existing

    def get_events(self, aggregate_id: str) -> List[DomainEvent]:
        return self._store.get(aggregate_id, [])

    def get_events_up_to(self, aggregate_id: str,
                         max_version: int) -> List[DomainEvent]:
        return [e for e in self.get_events(aggregate_id)
                if e.version <= max_version]


# ===== SERVICE =====

class BankAccountService:
    def __init__(self, event_store: EventStore):
        self.event_store = event_store

    def create_account(self, account_id: str, owner: str,
                       initial_balance: float = 0.0) -> BankAccount:
        account = BankAccount.create(account_id, owner, initial_balance)
        self.event_store.save(
            account_id, account.get_uncommitted_events(), 0)
        account.clear_uncommitted_events()
        print(f"Created: {account}")
        return account
```

```
    def deposit(self, account_id: str, amount: float,
                description: str = ""):
        events = self.event_store.get_events(account_id)
        account = BankAccount.from_events(events)
        account.deposit(amount, description)
        self.event_store.save(
            account_id, account.get_uncommitted_events(), len(events))
        account.clear_uncommitted_events()
        print(f"Deposited {amount:.2f} -> {account}")

    def withdraw(self, account_id: str, amount: float,
                 description: str = ""):
        events = self.event_store.get_events(account_id)
        account = BankAccount.from_events(events)
        account.withdraw(amount, description)
        self.event_store.save(
            account_id, account.get_uncommitted_events(), len(events))
        account.clear_uncommitted_events()
        print(f"Withdrew {amount:.2f} -> {account}")

    def get_account(self, account_id: str) -> BankAccount:
        events = self.event_store.get_events(account_id)
        return BankAccount.from_events(events)

    def get_balance_at_version(self, account_id: str,
                               version: int) -> float:
        events = self.event_store.get_events_up_to(account_id, version)
        account = BankAccount.from_events(events)
        return account.balance

    def print_history(self, account_id: str):
        events = self.event_store.get_events(account_id)
        print(f"\nEvent history for {account_id}:")
        for event in events:
            event_type = type(event).__name__
            if isinstance(event, AccountCreated):
                detail = f"owner={event.owner}"
            elif isinstance(event, MoneyDeposited):
                detail = f"amount={event.amount:.2f}, {event.description}"
            elif isinstance(event, MoneyWithdrawn):
                detail = f"amount={event.amount:.2f}, {event.description}"
            else:
                detail = ""
            print(f"  v{event.version}: {event_type} [{detail}]")


# ===== DEMO =====

store = EventStore()
service = BankAccountService(store)

service.create_account("ACC-001", "Alice")
service.deposit("ACC-001", 1000.00, "Initial deposit")
service.deposit("ACC-001", 500.00, "Salary")
service.withdraw("ACC-001", 200.00, "Groceries")
```

```
service.deposit("ACC-001", 50.00, "Refund")
service.withdraw("ACC-001", 100.00, "Utilities")

service.print_history("ACC-001")

# Temporal query
balance_v3 = service.get_balance_at_version("ACC-001", 3)
print(f"\nBalance at version 3: {balance_v3:.2f}")

current = service.get_account("ACC-001")
print(f"Current balance: {current.balance:.2f}")
```

Output:

```
Created: BankAccount(id='ACC-001', owner='Alice', balance=0.00, version=1)
Deposited 1000.00 -> BankAccount(id='ACC-001', owner='Alice', balance=1000.00, version=2)
Deposited 500.00 -> BankAccount(id='ACC-001', owner='Alice', balance=1500.00, version=3)
Withdrew 200.00 -> BankAccount(id='ACC-001', owner='Alice', balance=1300.00, version=4)
Deposited 50.00 -> BankAccount(id='ACC-001', owner='Alice', balance=1350.00, version=5)
Withdrew 100.00 -> BankAccount(id='ACC-001', owner='Alice', balance=1250.00, version=6)

Event history for ACC-001:
  v1: AccountCreated [owner=Alice]
  v2: MoneyDeposited [amount=1000.00, Initial deposit]
  v3: MoneyDeposited [amount=500.00, Salary]
  v4: MoneyWithdrawn [amount=200.00, Groceries]
  v5: MoneyDeposited [amount=50.00, Refund]
  v6: MoneyWithdrawn [amount=100.00, Utilities]

Balance at version 3: 1500.00
Current balance: 1250.00
```

Event Sourcing + CQRS Combination

Event Sourcing and CQRS are natural partners:

```
+-------------------+       +-------------------+       +------------------+
|   Command Side    |       |   Event Store     |       |   Query Side     |
|                   |       |                   |       |                  |
| Validate command  | ----> | Append events     | ----> | Project events   |
| Apply to          |       |                   |       | into read models |
| aggregate         |       | AccountCreated    |       |                  |
|                   |       | MoneyDeposited    |       | account_summary  |
|                   |       | MoneyWithdrawn    |       | (denormalized)   |
+-------------------+       +-------------------+       +------------------+
```

```
                                                                  |
                                                                  v
                                                        +------------------+
                                                        | Read Database    |
                                                        | - Balance: 1250  |
                                                        | - Tx Count: 6    |
                                                        | - Last Tx: ...   |
                                                        +------------------+
```

Events from the event store feed projections that build read-optimized views. The event store is the single source of truth. Read models are disposable -- you can delete and rebuild them by replaying events.

Real-World Applications

Financial Systems

```
Banking transactions: Every deposit, withdrawal, transfer is an event
Audit trail:          Regulators can see every change and who made it
Reconciliation:       Replay events to verify account balances
Debugging:            Reproduce exact state at any point in time
```

E-Commerce Order Tracking

```
OrderPlaced -> PaymentReceived -> ItemsPicked -> OrderShipped -> Delivered
     |               |                |              |             |
   Event 1         Event 2          Event 3        Event 4      Event 5

Customer can see full order history
Support can replay to see what happened
Analytics can count events by type
```

Healthcare Records

```
PatientAdmitted -> DiagnosisRecorded -> TreatmentStarted -> MedicationGiven
Complete history required by law
Cannot delete records (append only matches regulation)
Temporal queries: "What medications was the patient on at date X?"
```

Complexity Trade-Offs

```
+---------------------------+-----------------------------------+
| Benefit                   | Cost                              |
+---------------------------+-----------------------------------+
| Complete audit trail      | More storage (events accumulate) |
| Temporal queries          | Complex event schema evolution   |
| Debug by replaying events | Eventual consistency with CQRS   |
| Undo / compensation       | Learning curve for the team      |
| Event-driven integration  | Snapshot management overhead     |
| Rebuild read models       | Complex conflict resolution      |
+---------------------------+-----------------------------------+
```

When to Use / When NOT to Use

Use Event Sourcing When

- You need a complete audit trail (finance, healthcare, compliance)
- You need to answer "what was the state at time X?"
- You are building an event-driven architecture
- Domain experts think in terms of events ("order placed," "payment received")
- You need to support undo operations
- You are using CQRS and want events as the sync mechanism

Do NOT Use Event Sourcing When

- You have a simple CRUD application
- You do not need audit trails or temporal queries
- Your team is unfamiliar with the pattern and delivery timelines are tight
- The domain is not naturally event-driven
- You need to delete data (GDPR right to be forgotten requires special handling)
- Read-after-write consistency is critical and eventual consistency is not acceptable

Common Mistakes

1. Storing too much data in events. Events should capture the fact and its parameters, not the entire entity state. "PriceChanged(productId, newPrice)" not "ProductUpdated(all fields)."
2. Not planning for event schema evolution. Events are immutable. When the schema changes, you need upcasters to transform old events into the new format during replay.
3. Forgetting snapshots. Without snapshots, replaying 100,000 events to load an aggregate becomes unusably slow.
4. Making events too fine-grained. "FieldXUpdated" events are not meaningful business events. Use domain language: "OrderShipped," "PaymentReceived."
5. Treating the event store as a message queue. The event store is for persistence and rebuilding state. Use a separate message broker for event distribution.

Best Practices

1. Name events in past tense. Events describe what happened: "OrderPlaced," not "PlaceOrder" (that is a command).
2. Include all necessary data in the event. Events should be self-contained. Do not require external lookups to process an event.
3. Plan for schema evolution from day one. Use versioned event schemas and write upcasters early.
4. Take snapshots at regular intervals. Every 100 or 1000 events, depending on replay time requirements.
5. Test by replaying events. Write tests that create events, replay them, and verify the resulting state.
6. Keep aggregates small. An aggregate with millions of events is a design problem. Consider splitting it.

Quick Summary

Event Sourcing stores every state change as an immutable event instead of overwriting the current state. Current state is derived by replaying events from the beginning. Snapshots improve performance by capturing state at intervals so only recent events need replay. Event

Sourcing provides complete audit trails, temporal queries, and the ability to rebuild state from history. It pairs naturally with CQRS, where events from the write side feed projections that build read models. The pattern adds significant complexity and is best suited for domains where history and auditability are critical, such as financial and healthcare systems.

Key Points

- Event Sourcing stores events (facts), not state
- Events are immutable and append-only
- Current state is derived by replaying events
- Snapshots prevent performance degradation as events accumulate
- Event Sourcing enables temporal queries ("what was the state at time X?")
- Events should use past tense and domain language
- CQRS and Event Sourcing are complementary patterns
- Schema evolution requires careful planning (upcasters, versioning)
- The event store is not a message queue
- GDPR "right to erasure" requires special handling (crypto shredding)

Practice Questions

1. What is the fundamental difference between traditional state storage and Event Sourcing? What information do you gain and what complexity do you introduce?
2. Explain how snapshots improve performance in an Event Sourced system. How would you decide the snapshot interval?
3. A bank account has been active for 5 years with 50,000 events. A customer disputes a transaction from 3 years ago. How does Event Sourcing help resolve this dispute compared to traditional state storage?
4. What happens when you need to change the schema of an event? For example, you originally stored "MoneyDeposited(amount)" and now need "MoneyDeposited(amount, currency)". How do you handle existing events?
5. Explain the relationship between Event Sourcing and CQRS. Can you use one without the other? When would you use both together?

Exercises

Exercise 1: Shopping Cart Event Sourcing

Build an event-sourced shopping cart with events: `CartCreated`, `ItemAdded`, `ItemRemoved`, `QuantityChanged`, `CartCheckedOut`. Implement the aggregate, event store, and a service that supports temporal queries ("what was in the cart before the last item was removed?").

Exercise 2: Snapshots

Extend the bank account example to include automatic snapshot creation every 100 events. Write a benchmark that measures loading time with and without snapshots for accounts with 1000, 5000, and 10000 events.

Exercise 3: Event Sourcing + CQRS

Combine Event Sourcing with CQRS for a simple order system. Events like `OrderPlaced`, `OrderPaid`, `OrderShipped` feed into two read models: (1) an order status view for customers and (2) an order analytics view that counts orders by status and calculates total revenue.

What Is Next?

Event Sourcing and CQRS handle data complexity within your services. But what happens when a downstream service fails? The next chapter introduces the Circuit Breaker pattern, which prevents cascading failures in distributed systems by detecting when a service is down and failing fast instead of hanging indefinitely.

Chapter 27: Circuit Breaker Pattern

What You Will Learn

- What the Circuit Breaker pattern is and how it prevents cascading failures
- How the three states work: Closed, Open, and Half-Open
- How to implement a Circuit Breaker in Java using Resilience4j
- How to build a custom Circuit Breaker in Python
- How to design fallback strategies for degraded service
- How Circuit Breaker compares to simple retry logic
- Real-world application in microservice communication

Why This Chapter Matters

In a microservice architecture, your service depends on other services. Your order service calls the payment service, which calls the fraud detection service, which calls an external credit check API. When that external API goes down, the fraud service starts timing out. The payment service is waiting for the fraud service. The order service is waiting for the payment service. Every request piles up, threads exhaust, and your entire system grinds to a halt -- even though your order service itself is perfectly healthy.

This is a cascading failure, and it is one of the most dangerous problems in distributed systems. The Circuit Breaker pattern prevents it by detecting when a downstream service is failing and short-circuiting requests to it. Instead of waiting 30 seconds for a timeout, the Circuit Breaker fails immediately and returns a fallback response. This protects your service from being dragged down by someone else's outage.

The name comes from electrical circuit breakers. When a short circuit occurs, the breaker trips to prevent the wiring from catching fire. In software, when a service fails repeatedly, the circuit breaker trips to prevent your system from overloading.

The Problem

Before: No Circuit Breaker

```
Order Service --> Payment Service --> Fraud Service --> External API (DOWN)
     |                  |                  |
     |  Waiting...      |  Waiting...      |  Timeout: 30s
     |  Waiting...      |  Waiting...      |
     |  Waiting...      |  Waiting...      |
     |                  |                  |
     v                  v                  v
  Thread pool          Thread pool        Connection pool
  exhausted            exhausted          exhausted

  Result: ALL services are DOWN because ONE external API is down
```

```java
// Without circuit breaker: every request waits for timeout
@Service
public class PaymentService {

    private final RestTemplate restTemplate;

    public PaymentResult processPayment(PaymentRequest request) {
        try {
            // This call hangs for 30 seconds when fraud service is down
            FraudCheckResult fraudCheck = restTemplate.postForObject(
                "http://fraud-service/check",
                request,
                FraudCheckResult.class
            );
            // Process payment...
            return new PaymentResult("SUCCESS");
        } catch (Exception e) {
            // After 30 seconds of waiting, we get here
            // But by then, 100 other requests are also waiting
            throw new PaymentException("Payment failed: " + e.getMessage());
        }
    }
}
```

What happens:

1. Request 1 waits 30 seconds for the fraud service -- timeout
2. Request 2 waits 30 seconds -- timeout
3. Requests 3-100 pile up, all waiting 30 seconds each
4. Thread pool exhausted -- order service stops responding

5. Services calling order service also start timing out
6. Entire system cascades into failure

The Solution: Circuit Breaker

The Circuit Breaker monitors failures and prevents calls to a failing service.

```
                +------------------+
        +------>|     CLOSED       |<------+
        |       | (Normal: calls   |       |
        |       |  pass through)   |       |
        |       +--------+---------+       |
        |                |                 |
        |     Failure threshold            |
        |     reached (e.g., 5             |
        |     failures in 10 calls)        |
        |                |                 |
        |                v                 |
        |       +------------------+       |
        |       |      OPEN        |       |
        |       | (Tripped: calls  |       |
        |       |  rejected fast)  |       |
        |       +--------+---------+       |
        |                |                 |
        |     Wait timeout expires         |
        |     (e.g., 60 seconds)           |  Success:
        |                |                 |  reset to
        |                v                 |  CLOSED
        |       +------------------+       |
        +-------+   HALF-OPEN      +-------+
          Fail  | (Testing: allow  |  Pass
                |  a few calls     |
                |  through)        |
                +------------------+
```

The Three States

```
+-------------+-------------------------------------------------+
| State       | Behavior                                        |
+-------------+-------------------------------------------------+
| CLOSED      | Normal operation. All calls pass through.       |
|             | Failures are counted. If failures exceed the    |
|             | threshold, transition to OPEN.                  |
+-------------+-------------------------------------------------+
| OPEN        | Circuit is tripped. All calls fail immediately |
```

```
|             | without reaching the downstream service.       |
|             | After a timeout period, transition to          |
|             | HALF-OPEN.                                     |
+-------------+------------------------------------------------+
| HALF-OPEN   | Testing the waters. A limited number of calls  |
|             | are allowed through. If they succeed,          |
|             | transition to CLOSED. If they fail, transition |
|             | back to OPEN.                                  |
+-------------+------------------------------------------------+
```

Java Implementation: Resilience4j CircuitBreaker

Resilience4j is the standard Java library for resilience patterns. Here is how to use its Circuit Breaker.

Step 1: Add Dependencies

```xml
<!-- Maven -->
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-circuitbreaker</artifactId>
    <version>2.1.0</version>
</dependency>
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
    <version>2.1.0</version>
</dependency>
```

Step 2: Configure the Circuit Breaker

```java
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.circuitbreaker.CircuitBreakerConfig;
import io.github.resilience4j.circuitbreaker.CircuitBreakerRegistry;

import java.time.Duration;

public class CircuitBreakerSetup {

    public static CircuitBreaker createPaymentCircuitBreaker() {
        CircuitBreakerConfig config = CircuitBreakerConfig.custom()
```

```
            // Percentage of failures to trip the breaker
            .failureRateThreshold(50)
            // Minimum number of calls before evaluating failure rate
            .minimumNumberOfCalls(5)
            // How long to wait in OPEN state before trying HALF-OPEN
            .waitDurationInOpenState(Duration.ofSeconds(30))
            // Number of calls allowed in HALF-OPEN state
            .permittedNumberOfCallsInHalfOpenState(3)
            // Use a sliding window to track call outcomes
            .slidingWindowType(
                CircuitBreakerConfig.SlidingWindowType.COUNT_BASED)
            .slidingWindowSize(10)
            .build();

        CircuitBreakerRegistry registry =
            CircuitBreakerRegistry.of(config);

        return registry.circuitBreaker("paymentService");
    }
}
```

Step 3: Use the Circuit Breaker

```java
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.circuitbreaker.CallNotPermittedException;

import java.util.function.Supplier;

@Service
public class PaymentService {

    private final RestTemplate restTemplate;
    private final CircuitBreaker circuitBreaker;

    public PaymentService(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
        this.circuitBreaker = CircuitBreakerSetup
            .createPaymentCircuitBreaker();
    }

    public PaymentResult processPayment(PaymentRequest request) {
        // Wrap the call with the circuit breaker
        Supplier<PaymentResult> decoratedSupplier =
            CircuitBreaker.decorateSupplier(circuitBreaker, () -> {
                FraudCheckResult fraudCheck = restTemplate.postForObject(
                    "http://fraud-service/check",
                    request,
                    FraudCheckResult.class
                );
                // Process payment logic...
```

```
                return new PaymentResult("SUCCESS", request.getAmount());
            });

        try {
            return decoratedSupplier.get();
        } catch (CallNotPermittedException e) {
            // Circuit is OPEN -- fail fast with fallback
            System.out.println("Circuit OPEN: returning fallback");
            return fallbackPayment(request);
        } catch (Exception e) {
            // Call was attempted but failed
            System.out.println("Call failed: " + e.getMessage());
            return fallbackPayment(request);
        }
    }

    private PaymentResult fallbackPayment(PaymentRequest request) {
        // Queue for later processing instead of failing completely
        System.out.println("Fallback: queueing payment for retry");
        return new PaymentResult("PENDING", request.getAmount());
    }
}
```

Step 4: Spring Boot Annotation-Based Approach

```java
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import org.springframework.stereotype.Service;

@Service
public class FraudCheckService {

    private final RestTemplate restTemplate;

    public FraudCheckService(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    @CircuitBreaker(name = "fraudService",
                    fallbackMethod = "fraudCheckFallback")
    public FraudCheckResult checkFraud(PaymentRequest request) {
        return restTemplate.postForObject(
            "http://fraud-service/check",
            request,
            FraudCheckResult.class
        );
    }

    // Fallback method -- same signature plus Throwable
    private FraudCheckResult fraudCheckFallback(PaymentRequest request,
                                                Throwable t) {
```

```
        System.out.println("Fraud check circuit open, using fallback");
        // Default to allowing the payment with manual review flag
        return new FraudCheckResult(false, "MANUAL_REVIEW_REQUIRED");
    }
}
```

application.yml Configuration

```yaml
resilience4j:
  circuitbreaker:
    instances:
      fraudService:
        failureRateThreshold: 50
        minimumNumberOfCalls: 5
        waitDurationInOpenState: 30s
        permittedNumberOfCallsInHalfOpenState: 3
        slidingWindowSize: 10
        slidingWindowType: COUNT_BASED
        registerHealthIndicator: true
```

Step 5: Monitoring Circuit Breaker State

```java
public class CircuitBreakerMonitor {

    public static void printStatus(CircuitBreaker circuitBreaker) {
        CircuitBreaker.Metrics metrics = circuitBreaker.getMetrics();

        System.out.println("=== Circuit Breaker Status ===");
        System.out.println("State: " + circuitBreaker.getState());
        System.out.println("Failure rate: " +
                           metrics.getFailureRate() + "%");
        System.out.println("Total calls: " +
                           metrics.getNumberOfBufferedCalls());
        System.out.println("Failed calls: " +
                           metrics.getNumberOfFailedCalls());
        System.out.println("Successful calls: " +
                           metrics.getNumberOfSuccessfulCalls());
        System.out.println("Not permitted: " +
                           metrics.getNumberOfNotPermittedCalls());
        System.out.println("==============================");
    }

    // Register event listeners
    public static void registerListeners(CircuitBreaker circuitBreaker) {
        circuitBreaker.getEventPublisher()
            .onStateTransition(event ->
```

```
                System.out.println("STATE CHANGE: " +
                    event.getStateTransition()))
            .onFailureRateExceeded(event ->
                System.out.println("FAILURE RATE EXCEEDED: " +
                    event.getFailureRate() + "%"))
            .onCallNotPermitted(event ->
                System.out.println("CALL NOT PERMITTED (circuit open)"));
    }
}
```

Output during a failure scenario:

```
Call 1: SUCCESS (fraud service responding)
Call 2: SUCCESS
Call 3: FAILED (fraud service error)
Call 4: FAILED
Call 5: FAILED

=== Circuit Breaker Status ===
State: CLOSED
Failure rate: 60.0%

STATE CHANGE: CLOSED -> OPEN
FAILURE RATE EXCEEDED: 60.0%

Call 6: CALL NOT PERMITTED (circuit open) -> Fallback
Call 7: CALL NOT PERMITTED (circuit open) -> Fallback
Call 8: CALL NOT PERMITTED (circuit open) -> Fallback

[30 seconds pass]

STATE CHANGE: OPEN -> HALF_OPEN

Call 9: SUCCESS (fraud service recovered)
Call 10: SUCCESS
Call 11: SUCCESS

STATE CHANGE: HALF_OPEN -> CLOSED

Call 12: SUCCESS (back to normal)
```

Python Implementation: Custom Circuit Breaker

```python
import time
from enum import Enum
from threading import Lock
from datetime import datetime, timedelta
from typing import Callable, Optional, Any

class CircuitState(Enum):
    CLOSED = "CLOSED"
    OPEN = "OPEN"
    HALF_OPEN = "HALF_OPEN"


class CircuitBreaker:
    """
    Circuit Breaker implementation.

    States:
      CLOSED    -> Normal operation, tracking failures
      OPEN      -> Rejecting all calls, waiting for timeout
      HALF_OPEN -> Allowing limited calls to test recovery
    """

    def __init__(
        self,
        name: str,
        failure_threshold: int = 5,
        recovery_timeout: int = 30,
        half_open_max_calls: int = 3,
    ):
        self.name = name
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.half_open_max_calls = half_open_max_calls

        self._state = CircuitState.CLOSED
        self._failure_count = 0
        self._success_count = 0
        self._half_open_calls = 0
        self._last_failure_time: Optional[datetime] = None
        self._lock = Lock()

        # Metrics
        self.total_calls = 0
        self.total_failures = 0
        self.total_rejected = 0

    @property
    def state(self) -> CircuitState:
        with self._lock:
            if self._state == CircuitState.OPEN:
                if self._should_try_recovery():
                    self._transition_to(CircuitState.HALF_OPEN)
            return self._state
```

```
    def call(self, func: Callable, *args,
             fallback: Optional[Callable] = None, **kwargs) -> Any:
        """Execute a function through the circuit breaker."""

        self.total_calls += 1
        current_state = self.state

        if current_state == CircuitState.OPEN:
            self.total_rejected += 1
            print(f"  [{self.name}] OPEN: Call rejected")
            if fallback:
                return fallback(*args, **kwargs)
            raise CircuitBreakerOpenException(
                f"Circuit breaker '{self.name}' is OPEN")

        if current_state == CircuitState.HALF_OPEN:
            with self._lock:
                if self._half_open_calls >= self.half_open_max_calls:
                    self.total_rejected += 1
                    if fallback:
                        return fallback(*args, **kwargs)
                    raise CircuitBreakerOpenException(
                        f"Circuit breaker '{self.name}' HALF_OPEN limit reached")
                self._half_open_calls += 1

        # Attempt the call
        try:
            result = func(*args, **kwargs)
            self._on_success()
            return result
        except Exception as e:
            self._on_failure()
            if fallback:
                return fallback(*args, **kwargs)
            raise

    def _on_success(self):
        with self._lock:
            if self._state == CircuitState.HALF_OPEN:
                self._success_count += 1
                if self._success_count >= self.half_open_max_calls:
                    self._transition_to(CircuitState.CLOSED)
            else:
                self._failure_count = max(0, self._failure_count - 1)

    def _on_failure(self):
        with self._lock:
            self.total_failures += 1
            self._failure_count += 1
            self._last_failure_time = datetime.now()

            if self._state == CircuitState.HALF_OPEN:
                self._transition_to(CircuitState.OPEN)
            elif self._failure_count >= self.failure_threshold:
                self._transition_to(CircuitState.OPEN)
```

```
    def _should_try_recovery(self) -> bool:
        if self._last_failure_time is None:
            return False
        elapsed = (datetime.now() - self._last_failure_time).total_seconds()
        return elapsed >= self.recovery_timeout

    def _transition_to(self, new_state: CircuitState):
        old_state = self._state
        self._state = new_state
        print(f"  [{self.name}] State: {old_state.value} -> {new_state.value}")

        if new_state == CircuitState.CLOSED:
            self._failure_count = 0
            self._success_count = 0
        elif new_state == CircuitState.HALF_OPEN:
            self._half_open_calls = 0
            self._success_count = 0

    def get_metrics(self) -> dict:
        return {
            "state": self._state.value,
            "total_calls": self.total_calls,
            "total_failures": self.total_failures,
            "total_rejected": self.total_rejected,
            "failure_count": self._failure_count,
        }


class CircuitBreakerOpenException(Exception):
    pass
```

Using the Python Circuit Breaker

```python
import random

# Simulate an unreliable external service
class FraudService:
    def __init__(self, failure_rate=0.0):
        self.failure_rate = failure_rate
        self.call_count = 0

    def check(self, transaction_id: str) -> dict:
        self.call_count += 1
        if random.random() < self.failure_rate:
            raise ConnectionError(
                f"Fraud service unavailable (call #{self.call_count})")
        return {"transaction_id": transaction_id, "fraud": False}
```

```
# Create circuit breaker and service
cb = CircuitBreaker(
    name="fraud-service",
    failure_threshold=3,
    recovery_timeout=5,   # Short for demo
    half_open_max_calls=2,
)

fraud_service = FraudService(failure_rate=0.0)  # Start healthy


def fallback_fraud_check(transaction_id: str) -> dict:
    """Fallback: allow transaction but flag for manual review."""
    return {"transaction_id": transaction_id,
            "fraud": False, "manual_review": True}


# Demo: normal operation
print("=== Phase 1: Normal operation ===")
for i in range(5):
    result = cb.call(fraud_service.check, f"TX-{i}",
                     fallback=fallback_fraud_check)
    print(f"  TX-{i}: {result}")

# Demo: service starts failing
print("\n=== Phase 2: Service failing ===")
fraud_service.failure_rate = 1.0  # 100% failure
for i in range(5, 12):
    result = cb.call(fraud_service.check, f"TX-{i}",
                     fallback=fallback_fraud_check)
    print(f"  TX-{i}: {result}")

# Demo: wait for recovery timeout
print("\n=== Phase 3: Waiting for recovery ===")
time.sleep(6)

# Demo: service recovers
print("\n=== Phase 4: Service recovered ===")
fraud_service.failure_rate = 0.0  # Healthy again
for i in range(12, 18):
    result = cb.call(fraud_service.check, f"TX-{i}",
                     fallback=fallback_fraud_check)
    print(f"  TX-{i}: {result}")

# Print metrics
print(f"\nMetrics: {cb.get_metrics()}")
```

Output:

```
=== Phase 1: Normal operation ===
  TX-0: {'transaction_id': 'TX-0', 'fraud': False}
  TX-1: {'transaction_id': 'TX-1', 'fraud': False}
  TX-2: {'transaction_id': 'TX-2', 'fraud': False}
```

```
  TX-3: {'transaction_id': 'TX-3', 'fraud': False}
  TX-4: {'transaction_id': 'TX-4', 'fraud': False}

=== Phase 2: Service failing ===
  TX-5: Call failed (ConnectionError)
  TX-6: Call failed (ConnectionError)
  TX-7: Call failed (ConnectionError)
  [fraud-service] State: CLOSED -> OPEN
  [fraud-service] OPEN: Call rejected
  TX-8: {'transaction_id': 'TX-8', 'fraud': False, 'manual_review': True}
  [fraud-service] OPEN: Call rejected
  TX-9: {'transaction_id': 'TX-9', 'fraud': False, 'manual_review': True}
  [fraud-service] OPEN: Call rejected
  TX-10: {'transaction_id': 'TX-10', 'fraud': False, 'manual_review': True}
  [fraud-service] OPEN: Call rejected
  TX-11: {'transaction_id': 'TX-11', 'fraud': False, 'manual_review': True}

=== Phase 3: Waiting for recovery ===

=== Phase 4: Service recovered ===
  [fraud-service] State: OPEN -> HALF_OPEN
  TX-12: {'transaction_id': 'TX-12', 'fraud': False}
  TX-13: {'transaction_id': 'TX-13', 'fraud': False}
  [fraud-service] State: HALF_OPEN -> CLOSED
  TX-14: {'transaction_id': 'TX-14', 'fraud': False}
  TX-15: {'transaction_id': 'TX-15', 'fraud': False}
  TX-16: {'transaction_id': 'TX-16', 'fraud': False}
  TX-17: {'transaction_id': 'TX-17', 'fraud': False}

Metrics: {'state': 'CLOSED', 'total_calls': 18, 'total_failures': 3, 'total_rejected': ...
```

Fallback Strategies

When the circuit is open, you need a fallback. Here are common strategies:

```
+-------------------+-----------------------------------------+------------------+
| Strategy          | Description                             | Example          |
+-------------------+-----------------------------------------+------------------+
| Default Value     | Return a safe default                   | Empty list,      |
|                   |                                         | cached value     |
+-------------------+-----------------------------------------+------------------+
| Cached Response   | Return the last successful response     | Product prices   |
|                   |                                         | from cache       |
+-------------------+-----------------------------------------+------------------+
| Queue for Retry   | Accept the request and process later    | Payment queued   |
|                   |                                         | for retry        |
+-------------------+-----------------------------------------+------------------+
| Degraded Service  | Provide limited functionality            | Skip fraud check |
```

```
|                   |                                          | flag for review  |
+-------------------+------------------------------------------+------------------+
| Alternative       | Call a backup service                    | Secondary payment|
| Service           |                                          | provider         |
+-------------------+------------------------------------------+------------------+
| Fail Fast         | Return error immediately                 | "Service         |
|                   |                                          | unavailable"     |
+-------------------+------------------------------------------+------------------+
```

Java: Multiple Fallback Strategies

```java
@Service
public class ProductService {

    private final RestTemplate restTemplate;
    private final CacheManager cacheManager;

    @CircuitBreaker(name = "recommendationService",
                    fallbackMethod = "recommendationFallback")
    public List<Product> getRecommendations(Long userId) {
        return restTemplate.exchange(
            "http://recommendation-service/users/" + userId,
            HttpMethod.GET,
            null,
            new ParameterizedTypeReference<List<Product>>() {}
        ).getBody();
    }

    // Fallback: return cached recommendations
    private List<Product> recommendationFallback(Long userId,
                                                 Throwable t) {
        System.out.println("Recommendation service down, using cache");

        // Strategy 1: Try cache
        Cache cache = cacheManager.getCache("recommendations");
        if (cache != null) {
            List<Product> cached = cache.get(userId, List.class);
            if (cached != null) {
                return cached;
            }
        }

        // Strategy 2: Return popular products as default
        return getPopularProducts();
    }

    private List<Product> getPopularProducts() {
        // Return a static or pre-computed list
        return List.of(
            new Product(1L, "Bestseller Book", new BigDecimal("19.99")),
```

```
            new Product(2L, "Popular Gadget", new BigDecimal("49.99"))
        );
    }
}
```

Circuit Breaker vs Retry

These patterns are complementary, not alternatives:

```
+------------------+-------------------------------------+
| Retry            | Circuit Breaker                     |
+------------------+-------------------------------------+
| Tries again      | Stops trying                        |
| Hope it works    | Assumes it will not work            |
| Good for         | Good for sustained failures         |
| transient errors |                                     |
| Adds latency     | Reduces latency (fail fast)         |
| per retry        |                                     |
+------------------+-------------------------------------+

Best practice: Use BOTH together

  Request --> Retry (2-3 attempts) --> Circuit Breaker --> Service
                                                |
                                        If open: Fallback
```

Java: Retry + Circuit Breaker Together

```java
@Service
public class PaymentGateway {

    // Retry wraps Circuit Breaker
    // First: retry up to 3 times
    // If retries exhaust: circuit breaker tracks the failure
    // If circuit opens: retries are skipped entirely

    @Retry(name = "paymentRetry", fallbackMethod = "paymentFallback")
    @CircuitBreaker(name = "paymentCircuit",
                    fallbackMethod = "paymentFallback")
    public PaymentResult charge(String cardToken, BigDecimal amount) {
        return restTemplate.postForObject(
            "http://payment-provider/charge",
            new ChargeRequest(cardToken, amount),
```

```
            PaymentResult.class
        );
    }

    private PaymentResult paymentFallback(String cardToken,
                                          BigDecimal amount,
                                          Throwable t) {
        return new PaymentResult("PENDING",
            "Payment queued for processing");
    }
}
```

Real-World: Microservice Communication

```
                        API Gateway
                            |
              +-------------+-------------+
              |             |             |
         Order Service  Product Svc   User Service
              |
    +---------+---------+
    |         |         |
  Payment   Inventory  Shipping
  Service   Service    Service
    |
    |  [Circuit Breaker]
    |
  External
  Payment
  Provider
  (Stripe, PayPal)
```

Each service-to-service call should have its own Circuit Breaker with configuration tuned to the specific dependency:

```yaml
resilience4j:
  circuitbreaker:
    instances:
      # External payment provider: slow, unreliable
      paymentProvider:
        failureRateThreshold: 30
        waitDurationInOpenState: 60s
        slidingWindowSize: 20
```

```
      # Internal inventory service: fast, usually reliable
      inventoryService:
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
        slidingWindowSize: 10

      # External shipping API: moderately reliable
      shippingApi:
        failureRateThreshold: 40
        waitDurationInOpenState: 30s
        slidingWindowSize: 15
```

When to Use / When NOT to Use

Use the Circuit Breaker Pattern When

- Your service depends on external or remote services
- Downstream services can fail or become slow
- You want to fail fast instead of waiting for timeouts
- You need to prevent cascading failures across services
- You want to provide degraded functionality during outages

Do NOT Use the Circuit Breaker Pattern When

- Calls are to local, in-process components
- The downstream service is always available (embedded database)
- Failures are always transient and a simple retry suffices
- Your system is a monolith with no remote dependencies
- The cost of implementing fallbacks exceeds the benefit

Common Mistakes

1. Setting thresholds too high. If the failure threshold is 100, your system makes 100 failing calls before the breaker trips. That is 100 slow requests. Start with lower thresholds (5-10) and tune.

2. No fallback strategy. A Circuit Breaker without a fallback just throws a different exception faster. Always provide meaningful fallback behavior.
3. One Circuit Breaker for all services. Each dependency should have its own Circuit Breaker with its own configuration. A failing payment service should not affect the recommendation Circuit Breaker.
4. Ignoring the HALF-OPEN state. The HALF-OPEN state is critical for recovery. Allow enough test calls to accurately determine if the service has recovered.
5. Not monitoring Circuit Breaker state. If you do not know when breakers are tripping, you cannot respond to outages. Export metrics and set up alerts.

Best Practices

1. Configure per dependency. Different services have different failure characteristics. Tune thresholds, timeouts, and window sizes individually.
2. Design meaningful fallbacks. Cached data, default values, or queued processing are better than error messages.
3. Combine with retry. Use retry for transient errors and Circuit Breaker for sustained failures. Retry first, circuit break second.
4. Monitor and alert. Track the state of every Circuit Breaker. Alert when a breaker opens. Dashboard the failure rates.
5. Test your fallbacks. Inject failures in staging environments to verify that fallbacks work correctly.
6. Log state transitions. When a Circuit Breaker changes state, log it with details about why. This is invaluable for debugging.

Quick Summary

The Circuit Breaker pattern prevents cascading failures by detecting when a downstream service is failing and short-circuiting requests to it. It operates in three states: Closed (normal, tracking failures), Open (tripped, rejecting calls fast), and Half-Open (testing if the service has recovered). In Java, Resilience4j provides production-ready Circuit Breakers with annotation-based configuration. In Python, you can implement the pattern with a custom class that tracks failures and manages state transitions. Always pair Circuit Breakers with fallback strategies and monitoring.

Key Points

- Circuit Breaker prevents cascading failures in distributed systems
- Three states: Closed (normal), Open (rejecting), Half-Open (testing recovery)
- Fails fast instead of waiting for timeouts when a service is down
- Each dependency should have its own Circuit Breaker
- Fallback strategies: cached data, default values, queued processing, degraded service
- Combine with retry: retry handles transient errors, Circuit Breaker handles sustained failures
- Resilience4j is the standard Java library for Circuit Breaker
- Monitor and alert on Circuit Breaker state changes
- Test fallbacks by injecting failures in staging environments

Practice Questions

1. Explain the three states of a Circuit Breaker and the conditions that trigger transitions between them.
2. What is the difference between a Circuit Breaker and a retry mechanism? When would you use both together, and in what order?
3. Your payment service depends on three external providers: Stripe, PayPal, and a bank API. How would you configure Circuit Breakers for each? What factors influence your threshold and timeout choices?
4. Describe three different fallback strategies and when you would use each one. Which strategy would you use for a payment service? For a recommendation service?
5. How would you test that your Circuit Breaker works correctly in a staging environment? What scenarios would you simulate?

Exercises

Exercise 1: Python Circuit Breaker with Decorator

Create a Python decorator `@circuit_breaker(threshold=5, timeout=30)` that wraps any function with Circuit Breaker logic. The decorator should support a `fallback` parameter. Test it with a simulated unreliable HTTP service.

Exercise 2: Multi-Service Circuit Breakers

Build a Java service that calls three downstream services (user, product, order), each with its own Circuit Breaker configuration. Simulate failures in one service and verify that only its Circuit Breaker opens while the others continue normally.

Exercise 3: Circuit Breaker Dashboard

Create a monitoring endpoint that exposes the state and metrics of all Circuit Breakers in your application. Include: state, failure rate, total calls, rejected calls, and last state transition time. Display this as a simple text-based dashboard.

What Is Next?

The Circuit Breaker protects individual service calls from cascading failures. But what happens when a business operation spans multiple services and one of them fails? You cannot simply roll back a database transaction across microservices. The next chapter introduces the Saga pattern, which coordinates distributed transactions by defining a sequence of local transactions with compensating actions for rollback.

Chapter 28: Saga Pattern -- Managing Distributed Transactions

What You Will Learn

- What the Saga pattern is and why distributed transactions are hard
- The difference between Choreography and Orchestration approaches
- How compensating transactions undo partial work on failure
- How to implement an order saga in Java
- A complete e-commerce checkout saga with failure handling
- When to use Sagas vs traditional database transactions

Why This Chapter Matters

In a monolithic application, a database transaction guarantees that either all operations succeed or none do. Transfer money from Account A to Account B? Both the debit and credit happen, or neither does. The database handles it.

In a microservices architecture, a single business operation spans multiple services, each with its own database. The order service creates an order, the payment service charges the card, the inventory service reserves stock, and the shipping service creates a label. There is no single database transaction that can wrap all of these.

If payment succeeds but inventory fails, you have charged the customer for products you cannot ship. The Saga pattern solves this by breaking the distributed transaction into a sequence of local transactions, each with a compensating action to undo its work.

The Problem

You are building an e-commerce checkout that involves four services:

```
Customer clicks "Place Order"
    |
    v
+-- OrderService: Create order record
    |
    v
+-- InventoryService: Reserve items
    |
    v
+-- PaymentService: Charge credit card
    |
    v
+-- ShippingService: Create shipping label
    |
    v
Order confirmed!
```

What if PaymentService fails after InventoryService has already reserved the items?

```
OrderService:     Created order #1001        SUCCESS
InventoryService: Reserved 3 items           SUCCESS
PaymentService:   Card declined              FAILURE!
ShippingService:  Never reached              ---

Result: Items are reserved but never paid for.
        Other customers cannot buy those items.
        The order exists but is invalid.
```

You need to undo the inventory reservation and cancel the order. That is what Sagas do.

The Solution: Saga Pattern

A Saga is a sequence of local transactions. Each transaction has a compensating transaction that can undo its effects.

```
FORWARD FLOW (happy path):
  T1 ---------> T2 ---------> T3 ---------> T4
  Create        Reserve        Charge         Ship
  Order         Inventory      Payment        Items
```

```
COMPENSATION FLOW (on failure at T3):
  T1 ---------> T2 ---------> T3 (FAIL!)
  Create        Reserve        Charge
  Order         Inventory      Payment
                   |              |
                   v              v
                  C2             C3
                  Release        (nothing to
                  Inventory       undo, it failed)
                   |
                   v
                  C1
                  Cancel
                  Order
```

Key concepts:

- Local transaction (T): A single step that modifies one service's data
- Compensating transaction (C): The reverse of a local transaction, undoing its effects
- Saga Execution Coordinator (SEC): The component that drives the saga forward and triggers compensations on failure

---

Choreography vs Orchestration

There are two ways to coordinate a saga.

Choreography: Decentralized, Event-Driven

Each service listens for events and reacts. No central coordinator.

```
+--------+  order.created  +-----------+  inventory.reserved  +---------+
| Order  |---------------->| Inventory |--------------------->| Payment |
+--------+                 +-----------+                      +---------+
    ^                           |                                  |
    |                           | inventory.failed                 |
    |    order.cancelled        v                                  |
    +--<--------------------[cancel]                               |
    |                                                              |
    |                      payment.completed                       |
    +--<-------------------------------------------------------------+
```

```
    |                      payment.failed
    +--<-------------------------------------------------------+
```

Pros:

- Simple for small sagas (2-3 steps)
- No single point of failure
- Services are loosely coupled

Cons:

- Hard to understand the full flow (logic scattered across services)
- Difficult to debug (follow events across multiple logs)
- Risk of circular event dependencies
- Adding a step means modifying multiple services

Orchestration: Centralized Coordinator

A saga orchestrator tells each service what to do and when.

```
                    +-----------------------+
                    |   Saga Orchestrator    |
                    |                       |
                    | 1. Create Order  -----|---> OrderService
                    | 2. Reserve Stock -----|---> InventoryService
                    | 3. Charge Payment ----|---> PaymentService
                    | 4. Create Shipment ---|---> ShippingService
                    |                       |
                    | On failure:           |
                    | 3. Refund Payment ----|---> PaymentService
                    | 2. Release Stock -----|---> InventoryService
                    | 1. Cancel Order  -----|---> OrderService
                    +-----------------------+
```

Pros:

- Easy to understand (entire flow in one place)
- Easy to add, remove, or reorder steps
- Central place for logging and monitoring
- Simpler to test

Cons:

- Single point of failure (mitigate with redundancy)
- Orchestrator can become complex for large sagas
- Risk of tight coupling to the orchestrator

Recommendation: Use Choreography for simple, 2-3 step sagas between closely related services. Use Orchestration for complex, multi-step sagas where visibility and control are important. Most production systems use orchestration.

Java Implementation: Order Saga Orchestrator

Step Definition

```java
import java.util.function.Consumer;

public class SagaStep {
    private final String name;
    private final Runnable action;
    private final Runnable compensation;
    private StepStatus status;

    public enum StepStatus {
        PENDING, RUNNING, COMPLETED, FAILED, COMPENSATED
    }

    public SagaStep(String name, Runnable action, Runnable compensation) {
        this.name = name;
        this.action = action;
        this.compensation = compensation;
        this.status = StepStatus.PENDING;
    }

    public boolean execute() {
        status = StepStatus.RUNNING;
        try {
            System.out.printf("  [STEP] Executing: %s%n", name);
            action.run();
            status = StepStatus.COMPLETED;
            System.out.printf("  [STEP] Completed: %s%n", name);
            return true;
        } catch (Exception e) {
            status = StepStatus.FAILED;
            System.out.printf("  [STEP] Failed: %s -- %s%n",
                    name, e.getMessage());
            return false;
        }
    }

    public void compensate() {
        if (status == StepStatus.COMPLETED) {
            System.out.printf("  [COMP] Compensating: %s%n", name);
```

```
            try {
                compensation.run();
                status = StepStatus.COMPENSATED;
                System.out.printf("  [COMP] Compensated: %s%n", name);
            } catch (Exception e) {
                System.out.printf("  [COMP] Compensation FAILED: %s -- %s%n",
                        name, e.getMessage());
                // In production: alert, retry queue, manual intervention
            }
        }
    }

    public String getName() { return name; }
    public StepStatus getStatus() { return status; }
}
```

Saga Orchestrator

```java
import java.util.ArrayList;
import java.util.List;

public class SagaOrchestrator {
    private final String sagaId;
    private final List<SagaStep> steps;
    private SagaStatus status;

    public enum SagaStatus {
        NOT_STARTED, RUNNING, COMPLETED, COMPENSATING, FAILED
    }

    public SagaOrchestrator(String sagaId) {
        this.sagaId = sagaId;
        this.steps = new ArrayList<>();
        this.status = SagaStatus.NOT_STARTED;
    }

    public SagaOrchestrator addStep(String name, Runnable action,
                                    Runnable compensation) {
        steps.add(new SagaStep(name, action, compensation));
        return this;
    }

    public boolean execute() {
        System.out.printf("%n=== Saga '%s' Starting ===%n", sagaId);
        status = SagaStatus.RUNNING;
        int completedIndex = -1;

        // Execute steps forward
        for (int i = 0; i < steps.size(); i++) {
            boolean success = steps.get(i).execute();
```

```
            if (success) {
                completedIndex = i;
            } else {
                // Step failed -- compensate all completed steps
                System.out.printf("%n  [SAGA] Step %d failed. "
                        + "Starting compensation...%n", i + 1);
                status = SagaStatus.COMPENSATING;
                compensate(completedIndex);
                status = SagaStatus.FAILED;
                System.out.printf("=== Saga '%s' FAILED (compensated) ===%n",
                        sagaId);
                return false;
            }
        }

        status = SagaStatus.COMPLETED;
        System.out.printf("=== Saga '%s' COMPLETED ===%n", sagaId);
        return true;
    }

    private void compensate(int fromIndex) {
        // Compensate in reverse order
        for (int i = fromIndex; i >= 0; i--) {
            steps.get(i).compensate();
        }
    }

    public void printStatus() {
        System.out.printf("%nSaga '%s' Status: %s%n", sagaId, status);
        for (int i = 0; i < steps.size(); i++) {
            SagaStep step = steps.get(i);
            System.out.printf("  Step %d: %-25s [%s]%n",
                    i + 1, step.getName(), step.getStatus());
        }
    }
}
```

Service Simulators

```java
public class OrderService {
    private String currentOrderId = null;

    public void createOrder(String orderId) {
        System.out.printf("    OrderService: Creating order %s%n", orderId);
        this.currentOrderId = orderId;
        // In production: INSERT INTO orders ...
    }

    public void cancelOrder(String orderId) {
        System.out.printf("    OrderService: Cancelling order %s%n", orderId);
```

```
        this.currentOrderId = null;
        // In production: UPDATE orders SET status='cancelled' ...
    }
}

public class InventoryService {
    private final java.util.Map<String, Integer> stock;
    private final List<String> reservations;

    public InventoryService() {
        stock = new java.util.HashMap<>();
        stock.put("LAPTOP", 5);
        stock.put("MOUSE", 20);
        stock.put("KEYBOARD", 0);  // out of stock!
        reservations = new ArrayList<>();
    }

    public void reserveItems(String orderId, String item, int qty) {
        int available = stock.getOrDefault(item, 0);
        if (available < qty) {
            throw new RuntimeException(
                String.format("Insufficient stock for %s: need %d, have %d",
                    item, qty, available));
        }
        stock.put(item, available - qty);
        reservations.add(orderId + ":" + item + ":" + qty);
        System.out.printf("    InventoryService: Reserved %dx %s "
                + "(remaining: %d)%n", qty, item, stock.get(item));
    }

    public void releaseItems(String orderId, String item, int qty) {
        stock.put(item, stock.getOrDefault(item, 0) + qty);
        reservations.removeIf(r -> r.startsWith(orderId));
        System.out.printf("    InventoryService: Released %dx %s "
                + "(remaining: %d)%n", qty, item, stock.get(item));
    }
}

public class PaymentService {
    private boolean simulateFailure = false;

    public void setSimulateFailure(boolean fail) {
        this.simulateFailure = fail;
    }

    public void charge(String orderId, double amount) {
        if (simulateFailure) {
            throw new RuntimeException("Payment declined: card expired");
        }
        System.out.printf("    PaymentService: Charged $%.2f for order %s%n",
                amount, orderId);
    }

    public void refund(String orderId, double amount) {
        System.out.printf("    PaymentService: Refunded $%.2f for order %s%n",
```

```
                amount, orderId);
    }
}

public class ShippingService {
    public void createShipment(String orderId) {
        System.out.printf("    ShippingService: Created shipment for %s%n",
                orderId);
    }

    public void cancelShipment(String orderId) {
        System.out.printf("    ShippingService: Cancelled shipment for %s%n",
                orderId);
    }
}
```

Running the Saga

```java
public class SagaDemo {
    public static void main(String[] args) {
        OrderService orders = new OrderService();
        InventoryService inventory = new InventoryService();
        PaymentService payments = new PaymentService();
        ShippingService shipping = new ShippingService();

        // Scenario 1: Successful order
        System.out.println("*".repeat(60));
        System.out.println("SCENARIO 1: Successful Order");
        System.out.println("*".repeat(60));

        SagaOrchestrator saga1 = new SagaOrchestrator("order-1001")
            .addStep("Create Order",
                () -> orders.createOrder("1001"),
                () -> orders.cancelOrder("1001"))
            .addStep("Reserve Inventory",
                () -> inventory.reserveItems("1001", "LAPTOP", 1),
                () -> inventory.releaseItems("1001", "LAPTOP", 1))
            .addStep("Process Payment",
                () -> payments.charge("1001", 999.99),
                () -> payments.refund("1001", 999.99))
            .addStep("Create Shipment",
                () -> shipping.createShipment("1001"),
                () -> shipping.cancelShipment("1001"));

        saga1.execute();
        saga1.printStatus();

        // Scenario 2: Payment failure (triggers compensation)
        System.out.println("\n" + "*".repeat(60));
        System.out.println("SCENARIO 2: Payment Failure");
```

```
        System.out.println("*".repeat(60));

        payments.setSimulateFailure(true);

        SagaOrchestrator saga2 = new SagaOrchestrator("order-1002")
            .addStep("Create Order",
                () -> orders.createOrder("1002"),
                () -> orders.cancelOrder("1002"))
            .addStep("Reserve Inventory",
                () -> inventory.reserveItems("1002", "MOUSE", 2),
                () -> inventory.releaseItems("1002", "MOUSE", 2))
            .addStep("Process Payment",
                () -> payments.charge("1002", 49.98),
                () -> payments.refund("1002", 49.98))
            .addStep("Create Shipment",
                () -> shipping.createShipment("1002"),
                () -> shipping.cancelShipment("1002"));

        saga2.execute();
        saga2.printStatus();

        // Scenario 3: Inventory failure
        payments.setSimulateFailure(false);

        System.out.println("\n" + "*".repeat(60));
        System.out.println("SCENARIO 3: Out of Stock");
        System.out.println("*".repeat(60));

        SagaOrchestrator saga3 = new SagaOrchestrator("order-1003")
            .addStep("Create Order",
                () -> orders.createOrder("1003"),
                () -> orders.cancelOrder("1003"))
            .addStep("Reserve Inventory",
                () -> inventory.reserveItems("1003", "KEYBOARD", 1),
                () -> inventory.releaseItems("1003", "KEYBOARD", 1))
            .addStep("Process Payment",
                () -> payments.charge("1003", 79.99),
                () -> payments.refund("1003", 79.99));

        saga3.execute();
        saga3.printStatus();
    }
}
```

Output:

```
************************************************************
SCENARIO 1: Successful Order
************************************************************

=== Saga 'order-1001' Starting ===
  [STEP] Executing: Create Order
    OrderService: Creating order 1001
```

```
  [STEP] Completed: Create Order
  [STEP] Executing: Reserve Inventory
    InventoryService: Reserved 1x LAPTOP (remaining: 4)
  [STEP] Completed: Reserve Inventory
  [STEP] Executing: Process Payment
    PaymentService: Charged $999.99 for order 1001
  [STEP] Completed: Process Payment
  [STEP] Executing: Create Shipment
    ShippingService: Created shipment for 1001
  [STEP] Completed: Create Shipment
=== Saga 'order-1001' COMPLETED ===

Saga 'order-1001' Status: COMPLETED
  Step 1: Create Order              [COMPLETED]
  Step 2: Reserve Inventory         [COMPLETED]
  Step 3: Process Payment           [COMPLETED]
  Step 4: Create Shipment           [COMPLETED]

************************************************************
SCENARIO 2: Payment Failure
************************************************************

=== Saga 'order-1002' Starting ===
  [STEP] Executing: Create Order
    OrderService: Creating order 1002
  [STEP] Completed: Create Order
  [STEP] Executing: Reserve Inventory
    InventoryService: Reserved 2x MOUSE (remaining: 18)
  [STEP] Completed: Reserve Inventory
  [STEP] Executing: Process Payment
  [STEP] Failed: Process Payment -- Payment declined: card expired

  [SAGA] Step 3 failed. Starting compensation...
  [COMP] Compensating: Reserve Inventory
    InventoryService: Released 2x MOUSE (remaining: 20)
  [COMP] Compensated: Reserve Inventory
  [COMP] Compensating: Create Order
    OrderService: Cancelling order 1002
  [COMP] Compensated: Create Order
=== Saga 'order-1002' FAILED (compensated) ===

Saga 'order-1002' Status: FAILED
  Step 1: Create Order              [COMPENSATED]
  Step 2: Reserve Inventory         [COMPENSATED]
  Step 3: Process Payment           [FAILED]
  Step 4: Create Shipment           [PENDING]

************************************************************
SCENARIO 3: Out of Stock
************************************************************

=== Saga 'order-1003' Starting ===
  [STEP] Executing: Create Order
    OrderService: Creating order 1003
  [STEP] Completed: Create Order
```

```
  [STEP] Executing: Reserve Inventory
  [STEP] Failed: Reserve Inventory -- Insufficient stock for KEYBOARD: need 1, have 0

  [SAGA] Step 2 failed. Starting compensation...
  [COMP] Compensating: Create Order
    OrderService: Cancelling order 1003
  [COMP] Compensated: Create Order
=== Saga 'order-1003' FAILED (compensated) ===

Saga 'order-1003' Status: FAILED
  Step 1: Create Order              [COMPENSATED]
  Step 2: Reserve Inventory         [FAILED]
  Step 3: Process Payment           [PENDING]
```

Compensating Transactions: The Hard Part

Compensating transactions are not always simple "undo" operations. They must account for the real-world effects that have already occurred.

```
+--------------------+----------------------------+---------------------------+
| Action             | Compensation               | Complication              |
+--------------------+----------------------------+---------------------------+
| Create order       | Cancel order               | Simple status update      |
| Reserve inventory  | Release inventory          | Simple: add quantity back |
| Charge credit card | Refund credit card         | Refund takes 3-5 days!    |
| Send email         | Send correction email      | Cannot unsend             |
| Ship package       | Initiate return shipment   | Package may be in transit |
| Update external API| Revert API call            | API may not support it    |
+--------------------+----------------------------+---------------------------+
```

Key rules for compensating transactions:

1. Compensations must be idempotent. They might be called multiple times (retries). Running a compensation twice should have the same effect as running it once.
2. Compensations must always succeed. If a compensation fails, you need a retry mechanism, dead letter queue, or manual intervention process.
3. Compensations are semantic, not technical. You cannot "undo" a credit card charge. You issue a refund, which is a separate forward transaction.
4. Order matters. Compensate in reverse order of execution. You cannot release inventory before cancelling the shipment that depends on it.

Python Implementation: E-Commerce Checkout Saga

```python
from enum import Enum
from datetime import datetime
from typing import Callable, Optional, List
import traceback


class StepStatus(Enum):
    PENDING = "pending"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    COMPENSATING = "compensating"
    COMPENSATED = "compensated"


class SagaStep:
    def __init__(self, name: str, action: Callable,
                 compensation: Callable):
        self.name = name
        self.action = action
        self.compensation = compensation
        self.status = StepStatus.PENDING
        self.error: Optional[str] = None
        self.started_at: Optional[datetime] = None
        self.completed_at: Optional[datetime] = None

    def execute(self, context: dict) -> bool:
        self.status = StepStatus.RUNNING
        self.started_at = datetime.now()
        try:
            self.action(context)
            self.status = StepStatus.COMPLETED
            self.completed_at = datetime.now()
            return True
        except Exception as e:
            self.status = StepStatus.FAILED
            self.error = str(e)
            self.completed_at = datetime.now()
            return False

    def compensate(self, context: dict) -> bool:
        if self.status != StepStatus.COMPLETED:
            return True  # nothing to compensate
```

```
        self.status = StepStatus.COMPENSATING
        try:
            self.compensation(context)
            self.status = StepStatus.COMPENSATED
            return True
        except Exception as e:
            self.error = f"Compensation failed: {e}"
            return False


class SagaOrchestrator:
    def __init__(self, saga_id: str):
        self.saga_id = saga_id
        self.steps: List[SagaStep] = []
        self.context: dict = {"saga_id": saga_id}
        self.status = "not_started"

    def add_step(self, name, action, compensation):
        self.steps.append(SagaStep(name, action, compensation))
        return self

    def execute(self) -> bool:
        print(f"\n{'=' * 55}")
        print(f"  Saga '{self.saga_id}' STARTING")
        print(f"{'=' * 55}")
        self.status = "running"
        completed_steps = []

        for i, step in enumerate(self.steps):
            print(f"\n  Step {i + 1}/{len(self.steps)}: {step.name}")

            if step.execute(self.context):
                completed_steps.append(step)
                print(f"  --> OK")
            else:
                print(f"  --> FAILED: {step.error}")
                print(f"\n  Starting compensation "
                      f"({len(completed_steps)} steps to undo)...")

                self.status = "compensating"
                for comp_step in reversed(completed_steps):
                    print(f"  Undoing: {comp_step.name}")
                    if comp_step.compensate(self.context):
                        print(f"  --> Undone")
                    else:
                        print(f"  --> UNDO FAILED: {comp_step.error}")

                self.status = "failed"
                print(f"\n{'=' * 55}")
                print(f"  Saga '{self.saga_id}' FAILED (compensated)")
                print(f"{'=' * 55}")
                self._print_status()
                return False
```

```
        self.status = "completed"
        print(f"\n{'=' * 55}")
        print(f"  Saga '{self.saga_id}' COMPLETED")
        print(f"{'=' * 55}")
        self._print_status()
        return True

    def _print_status(self):
        print(f"\n  Step Status:")
        for i, step in enumerate(self.steps):
            marker = {
                StepStatus.COMPLETED: "+",
                StepStatus.COMPENSATED: "~",
                StepStatus.FAILED: "X",
                StepStatus.PENDING: ".",
            }.get(step.status, "?")
            print(f"    [{marker}] {step.name}: {step.status.value}")


# --- Service Simulators ---

class CheckoutServices:
    def __init__(self):
        self.orders = {}
        self.inventory = {"SKU-001": 10, "SKU-002": 5, "SKU-003": 0}
        self.payments = {}
        self.shipments = {}
        self.emails_sent = []
        self.fail_payment = False

    # Order Service
    def create_order(self, ctx):
        order_id = ctx["saga_id"]
        self.orders[order_id] = {
            "status": "created",
            "items": ctx["items"],
            "total": ctx["total"]
        }
        print(f"    [Order] Created order {order_id}")

    def cancel_order(self, ctx):
        order_id = ctx["saga_id"]
        self.orders[order_id]["status"] = "cancelled"
        print(f"    [Order] Cancelled order {order_id}")

    # Inventory Service
    def reserve_inventory(self, ctx):
        for sku, qty in ctx["items"].items():
            available = self.inventory.get(sku, 0)
            if available < qty:
                raise Exception(
                    f"Out of stock: {sku} (need {qty}, have {available})")
            self.inventory[sku] -= qty
            print(f"    [Inventory] Reserved {qty}x {sku} "
                  f"(remaining: {self.inventory[sku]})")
```

```
    def release_inventory(self, ctx):
        for sku, qty in ctx["items"].items():
            self.inventory[sku] = self.inventory.get(sku, 0) + qty
            print(f"    [Inventory] Released {qty}x {sku} "
                  f"(remaining: {self.inventory[sku]})")

    # Payment Service
    def process_payment(self, ctx):
        if self.fail_payment:
            raise Exception("Payment declined: insufficient funds")
        order_id = ctx["saga_id"]
        self.payments[order_id] = {
            "amount": ctx["total"],
            "status": "charged"
        }
        print(f"    [Payment] Charged ${ctx['total']:.2f}")

    def refund_payment(self, ctx):
        order_id = ctx["saga_id"]
        if order_id in self.payments:
            self.payments[order_id]["status"] = "refunded"
            print(f"    [Payment] Refunded ${ctx['total']:.2f}")

    # Shipping Service
    def create_shipment(self, ctx):
        order_id = ctx["saga_id"]
        self.shipments[order_id] = {"status": "label_created"}
        print(f"    [Shipping] Created shipping label")

    def cancel_shipment(self, ctx):
        order_id = ctx["saga_id"]
        if order_id in self.shipments:
            self.shipments[order_id]["status"] = "cancelled"
            print(f"    [Shipping] Cancelled shipment")

    # Notification Service
    def send_confirmation(self, ctx):
        order_id = ctx["saga_id"]
        self.emails_sent.append(f"confirmation-{order_id}")
        print(f"    [Email] Sent order confirmation")

    def send_cancellation(self, ctx):
        order_id = ctx["saga_id"]
        self.emails_sent.append(f"cancellation-{order_id}")
        print(f"    [Email] Sent cancellation notice")


# --- Run Scenarios ---

services = CheckoutServices()

# Scenario 1: Successful checkout
saga1 = SagaOrchestrator("ORD-2001")
saga1.context.update({
```

```
    "items": {"SKU-001": 2, "SKU-002": 1},
    "total": 149.97,
    "customer": "alice@example.com"
})

(saga1
    .add_step("Create Order",
              services.create_order, services.cancel_order)
    .add_step("Reserve Inventory",
              services.reserve_inventory, services.release_inventory)
    .add_step("Process Payment",
              services.process_payment, services.refund_payment)
    .add_step("Create Shipment",
              services.create_shipment, services.cancel_shipment)
    .add_step("Send Confirmation",
              services.send_confirmation, services.send_cancellation))

saga1.execute()

# Scenario 2: Payment failure
print("\n" + "#" * 55)
print("  SCENARIO 2: Payment Failure")
print("#" * 55)

services.fail_payment = True

saga2 = SagaOrchestrator("ORD-2002")
saga2.context.update({
    "items": {"SKU-001": 1},
    "total": 49.99,
    "customer": "bob@example.com"
})

(saga2
    .add_step("Create Order",
              services.create_order, services.cancel_order)
    .add_step("Reserve Inventory",
              services.reserve_inventory, services.release_inventory)
    .add_step("Process Payment",
              services.process_payment, services.refund_payment)
    .add_step("Create Shipment",
              services.create_shipment, services.cancel_shipment))

saga2.execute()

# Verify inventory was restored
print(f"\n  Inventory after compensation: {services.inventory}")
```

Output:

```
=======================================================
  Saga 'ORD-2001' STARTING
=======================================================
```

```
  Step 1/5: Create Order
    [Order] Created order ORD-2001
  --> OK

  Step 2/5: Reserve Inventory
    [Inventory] Reserved 2x SKU-001 (remaining: 8)
    [Inventory] Reserved 1x SKU-002 (remaining: 4)
  --> OK

  Step 3/5: Process Payment
    [Payment] Charged $149.97
  --> OK

  Step 4/5: Create Shipment
    [Shipping] Created shipping label
  --> OK

  Step 5/5: Send Confirmation
    [Email] Sent order confirmation
  --> OK

========================================================
  Saga 'ORD-2001' COMPLETED
========================================================

  Step Status:
    [+] Create Order: completed
    [+] Reserve Inventory: completed
    [+] Process Payment: completed
    [+] Create Shipment: completed
    [+] Send Confirmation: completed

########################################################
  SCENARIO 2: Payment Failure
########################################################

========================================================
  Saga 'ORD-2002' STARTING
========================================================

  Step 1/4: Create Order
    [Order] Created order ORD-2002
  --> OK

  Step 2/4: Reserve Inventory
    [Inventory] Reserved 1x SKU-001 (remaining: 7)
  --> OK

  Step 3/4: Process Payment
  --> FAILED: Payment declined: insufficient funds

  Starting compensation (2 steps to undo)...
  Undoing: Reserve Inventory
    [Inventory] Released 1x SKU-001 (remaining: 8)
```

```
  --> Undone
  Undoing: Create Order
    [Order] Cancelled order ORD-2002
  --> Undone

========================================================
  Saga 'ORD-2002' FAILED (compensated)
========================================================

  Step Status:
    [~] Create Order: compensated
    [~] Reserve Inventory: compensated
    [X] Process Payment: failed
    [.] Create Shipment: pending

  Inventory after compensation: {'SKU-001': 8, 'SKU-002': 4, 'SKU-003': 0}
```

The inventory was correctly restored after the payment failure. The order was cancelled. The system is back to a consistent state.

Before vs After Comparison

Before: No Saga (Inconsistent State on Failure)

```python
def checkout(order_data):
    create_order(order_data)        # succeeds
    reserve_inventory(order_data)   # succeeds
    charge_payment(order_data)      # FAILS!
    # Inventory is reserved but payment failed
    # Order exists but is invalid
    # No automatic cleanup
```

After: Saga (Automatic Compensation)

```python
saga = SagaOrchestrator("order-123")
saga.add_step("Create Order", create_order, cancel_order)
saga.add_step("Reserve Inventory", reserve, release)
saga.add_step("Charge Payment", charge, refund)
saga.execute()
# If charge fails: inventory released, order cancelled automatically
```

When to Use / When NOT to Use

Use Saga When

- Business operations span multiple services or databases
- You need eventual consistency across service boundaries
- Operations have natural compensating actions
- The system must remain available even during partial failures
- You are building microservices that cannot share a database

Do NOT Use Saga When

- All operations are within a single database (use ACID transactions)
- You need strict immediate consistency (Sagas provide eventual consistency)
- Compensating actions are impossible or meaningless
- The operation is simple (2 services, one call each)
- You can use a distributed transaction coordinator (like XA) and accept the performance cost

---

Common Mistakes

Mistake 1: Non-Idempotent Compensations

```python
# WRONG: calling refund twice charges the refund twice
def refund_payment(ctx):
    payment_service.refund(ctx["amount"])  # no idempotency check

# RIGHT: use idempotency keys
def refund_payment(ctx):
    refund_id = f"refund-{ctx['saga_id']}"
    if not payment_service.refund_exists(refund_id):
        payment_service.refund(ctx["amount"], idempotency_key=refund_id)
```

Mistake 2: Ignoring Compensation Failures

```python
# WRONG: silently swallowing compensation errors
def compensate(self):
    try:
        self.compensation()
    except:
        pass  # hope for the best

# RIGHT: log, alert, and queue for retry
def compensate(self):
    try:
        self.compensation()
    except Exception as e:
        logger.critical(f"Compensation failed: {self.name}: {e}")
        alert_ops_team(self.name, e)
        dead_letter_queue.add(self)
```

Mistake 3: Forward Compensation Order

```python
# WRONG: compensating in forward order
# Release inventory BEFORE cancelling shipment
# Shipment references the inventory reservation!

# RIGHT: compensate in REVERSE order
# 1. Cancel shipment (depends on inventory reservation)
# 2. Release inventory (now safe, no shipment depends on it)
# 3. Refund payment
# 4. Cancel order
```

Best Practices

1. Make compensations idempotent. Network retries might trigger the same compensation multiple times. Use idempotency keys to prevent duplicate effects.
2. Compensate in reverse order. Later steps may depend on earlier steps. Undo the latest step first.
3. Persist saga state. Store the saga's progress in a database so it survives crashes. On restart, resume from where it left off.
4. Use timeouts. If a step does not respond within a timeout, treat it as failed and begin compensation.

5. Log everything. Sagas cross service boundaries, making debugging hard. Log every step, every compensation, and every context value.
6. Handle compensation failures. When a compensation fails, queue it for retry or alert for manual intervention. Never silently ignore failures.
7. Start with orchestration. Orchestration is easier to understand, test, and debug. Only use choreography when you need extreme decoupling.

Quick Summary

The Saga pattern manages distributed transactions by breaking them into local transactions with compensating actions. When a step fails, all previously completed steps are undone in reverse order. Orchestration uses a central coordinator; choreography uses events. Compensating transactions must be idempotent and always succeed.

```
Problem:  Distributed operations lack ACID transactions.
Solution: Execute steps in sequence; compensate in reverse on failure.
Key:      Every forward action must have a compensating action.
```

Key Points

- Saga manages distributed transactions across multiple services
- Each step has a compensating transaction to undo its effects on failure
- Orchestration: central coordinator drives the saga (recommended for most cases)
- Choreography: services react to events (simpler for 2-3 step sagas)
- Compensations execute in reverse order of the forward steps
- Compensations must be idempotent (safe to retry)
- Saga provides eventual consistency, not immediate consistency
- Persist saga state to survive service crashes and restarts
- Handle compensation failures with retries, dead letter queues, or manual intervention

---

Practice Questions

1. Explain the difference between Choreography and Orchestration with a concrete example. When would you choose each?
2. A saga has 5 steps. Step 4 fails. Which steps need compensation and in what order? What about Step 4 itself?
3. Why must compensating transactions be idempotent? Give a concrete example of what goes wrong if they are not.
4. You have an e-commerce saga where step 3 (charge credit card) succeeds but step 4 (reserve inventory) fails. The refund takes 3-5 business days. How do you handle the customer experience during this time?
5. How would you handle a scenario where the compensation itself fails? Describe three different strategies.

Exercises

Exercise 1: Travel Booking Saga

Build a travel booking saga with these steps:

- Book flight (compensation: cancel flight)
- Book hotel (compensation: cancel hotel)
- Book car rental (compensation: cancel rental)
- Charge payment (compensation: refund)
- Send confirmation email (compensation: send cancellation email)

Implement scenarios: all succeed, hotel unavailable, payment fails. Show that each failure correctly compensates all previous steps.

Exercise 2: Saga with Retries

Extend the SagaOrchestrator to support:

- Configurable retry count per step (e.g., retry payment up to 3 times)
- Exponential backoff between retries (1s, 2s, 4s)

- Only trigger compensation after all retries are exhausted
- Log each retry attempt
- Demonstrate with a step that fails twice then succeeds on the third try

Exercise 3: Choreography-Based Saga

Implement the e-commerce checkout using choreography (event-driven) instead of orchestration:

- Each service publishes events when it completes its work
- Each service listens for events and reacts
- Each service also listens for failure events and compensates
- No central orchestrator
- Track the order status across all services using a shared event log
- Compare the code complexity with the orchestration approach

---

What Is Next?

The next chapter introduces the API Gateway Pattern, which provides a single entry point for all client requests to your microservices. Where Saga coordinates transactions across services, the API Gateway coordinates request routing, authentication, and rate limiting. Both patterns are essential for building robust microservice architectures.

Chapter 29: API Gateway Pattern -- Single Entry Point for Microservices

What You Will Learn

- What the API Gateway pattern is and why microservices need a single entry point
- How routing, authentication, rate limiting, and response aggregation work
- How to implement a gateway in Java with Spring Cloud Gateway concepts
- How to build a lightweight gateway in Python with FastAPI
- The Backend-for-Frontend (BFF) pattern and when to use it
- Trade-offs: single point of failure vs operational simplicity

Why This Chapter Matters

In a microservices architecture, a mobile app might need data from five different services to render a single screen: user profile from the User Service, recent orders from the Order Service, recommendations from the ML Service, notifications from the Notification Service, and loyalty points from the Points Service.

Without an API Gateway, the client must know the address of every service, handle authentication with each one, and make five separate HTTP calls. When you add a new service, every client must be updated. When you change a service URL, every client breaks.

The API Gateway solves this by providing a single, stable entry point. Clients talk to one URL. The gateway handles routing, authentication, rate limiting, and response aggregation behind the scenes.

The Problem

```
WITHOUT API GATEWAY:

Mobile App -----> User Service (users.internal:8001)
     |
     +----------> Order Service (orders.internal:8002)
     |
     +----------> Product Service (products.internal:8003)
     |
     +----------> Payment Service (payments.internal:8004)
     |
     +----------> Notification Service (notif.internal:8005)

Problems:
- Client knows 5 different URLs
- Each service needs its own auth logic
- No centralized rate limiting
- No request logging
- Adding service #6 means updating every client
- Services are exposed directly to the internet
```

The Solution: API Gateway

```
WITH API GATEWAY:

Mobile App -----> API Gateway (api.myapp.com)
                        |
                        +----> User Service
                        +----> Order Service
                        +----> Product Service
                        +----> Payment Service
                        +----> Notification Service

Benefits:
- Client knows ONE URL
- Auth handled once at the gateway
- Centralized rate limiting
- Unified logging and monitoring
- Add services without client changes
- Internal services are never exposed
```

What the Gateway Does

```
+-----------------------------------------------------------------+
|                       API GATEWAY                               |
|                                                                 |
|  Incoming     +----------+  +----------+  +-----------+       |
|  Request ---->| Auth     |->| Rate     |->| Route     |--+    |
|               | Check    |  | Limit    |  | Resolver  |  |    |
|               +----------+  +----------+  +-----------+  |    |
|                                                          |    |
|               +----------+  +----------+  +-----------+  |    |
|  Response <---| Response |<-| Cache    |<-| Load      |<-+    |
|               | Transform|  | Check    |  | Balance   |       |
|               +----------+  +----------+  +-----------+       |
+-----------------------------------------------------------------+
```

Core responsibilities:

1. Routing: Maps external URLs to internal service endpoints
2. Authentication: Validates tokens/API keys before forwarding requests
3. Rate Limiting: Prevents abuse by limiting requests per client
4. Load Balancing: Distributes requests across service instances
5. Response Aggregation: Combines responses from multiple services
6. Caching: Caches responses to reduce backend load
7. Protocol Translation: Converts between protocols (REST to gRPC, etc.)
8. Logging/Monitoring: Centralizes request tracing and metrics

---

Java Implementation: Spring Cloud Gateway Concepts

Route Configuration and Gateway Core

```java
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.function.Function;

public class Route {
    private final String id;
    private final String pathPattern;
    private final String targetUrl;
    private final boolean authRequired;
    private final int rateLimitPerMinute;
```

```
    public Route(String id, String pathPattern, String targetUrl,
                 boolean authRequired, int rateLimitPerMinute) {
        this.id = id;
        this.pathPattern = pathPattern;
        this.targetUrl = targetUrl;
        this.authRequired = authRequired;
        this.rateLimitPerMinute = rateLimitPerMinute;
    }

    public boolean matches(String path) {
        // Simple prefix matching (production would use regex/ant patterns)
        String prefix = pathPattern.replace("/**", "");
        return path.startsWith(prefix);
    }

    public String resolveTarget(String path) {
        String prefix = pathPattern.replace("/**", "");
        String remainder = path.substring(prefix.length());
        return targetUrl + remainder;
    }

    public String getId() { return id; }
    public boolean isAuthRequired() { return authRequired; }
    public int getRateLimitPerMinute() { return rateLimitPerMinute; }
}
```

Authentication Filter

```java
public class AuthFilter {
    private final Map<String, String> validTokens;

    public AuthFilter() {
        validTokens = new HashMap<>();
        validTokens.put("token-alice-123", "alice");
        validTokens.put("token-bob-456", "bob");
        validTokens.put("api-key-service-1", "service-1");
    }

    public AuthResult authenticate(String authHeader) {
        if (authHeader == null || authHeader.isEmpty()) {
            return new AuthResult(false, null, "Missing auth header");
        }

        String token = authHeader.replace("Bearer ", "");
        String user = validTokens.get(token);

        if (user != null) {
            return new AuthResult(true, user, null);
        }
```

```
        return new AuthResult(false, null, "Invalid token");
    }

    public static class AuthResult {
        public final boolean authenticated;
        public final String userId;
        public final String error;

        public AuthResult(boolean authenticated, String userId,
                          String error) {
            this.authenticated = authenticated;
            this.userId = userId;
            this.error = error;
        }
    }
}
```

Rate Limiter (Token Bucket)

```java
public class RateLimiter {
    private final Map<String, TokenBucket> buckets;

    public RateLimiter() {
        this.buckets = new ConcurrentHashMap<>();
    }

    public boolean allowRequest(String clientId, int maxPerMinute) {
        TokenBucket bucket = buckets.computeIfAbsent(
            clientId, k -> new TokenBucket(maxPerMinute));
        return bucket.tryConsume();
    }

    private static class TokenBucket {
        private int tokens;
        private final int maxTokens;
        private long lastRefill;

        TokenBucket(int maxTokens) {
            this.maxTokens = maxTokens;
            this.tokens = maxTokens;
            this.lastRefill = System.currentTimeMillis();
        }

        synchronized boolean tryConsume() {
            refill();
            if (tokens > 0) {
                tokens--;
                return true;
            }
```

```
            return false;
        }

        private void refill() {
            long now = System.currentTimeMillis();
            long elapsed = now - lastRefill;
            int newTokens = (int) (elapsed * maxTokens / 60000);
            if (newTokens > 0) {
                tokens = Math.min(maxTokens, tokens + newTokens);
                lastRefill = now;
            }
        }
    }
}
```

The Gateway

```java
public class ApiGateway {
    private final List<Route> routes;
    private final AuthFilter authFilter;
    private final RateLimiter rateLimiter;
    private final List<String> requestLog;

    public ApiGateway() {
        this.routes = new ArrayList<>();
        this.authFilter = new AuthFilter();
        this.rateLimiter = new RateLimiter();
        this.requestLog = new ArrayList<>();
    }

    public void addRoute(Route route) {
        routes.add(route);
    }

    public GatewayResponse handleRequest(String method, String path,
                                         Map<String, String> headers) {
        long startTime = System.currentTimeMillis();
        String requestId = UUID.randomUUID().toString().substring(0, 8);

        System.out.printf("[GW-%s] %s %s%n", requestId, method, path);

        // Step 1: Find matching route
        Route route = findRoute(path);
        if (route == null) {
            System.out.printf("[GW-%s] No route found%n", requestId);
            return new GatewayResponse(404, "Not Found",
                    Map.of("error", "No route matches: " + path));
        }

        // Step 2: Authentication
```

```
        if (route.isAuthRequired()) {
            AuthFilter.AuthResult auth = authFilter.authenticate(
                    headers.get("Authorization"));
            if (!auth.authenticated) {
                System.out.printf("[GW-%s] Auth failed: %s%n",
                        requestId, auth.error);
                return new GatewayResponse(401, "Unauthorized",
                        Map.of("error", auth.error));
            }
            System.out.printf("[GW-%s] Authenticated: %s%n",
                    requestId, auth.userId);
        }

        // Step 3: Rate limiting
        String clientId = headers.getOrDefault("X-Client-Id", "anonymous");
        if (!rateLimiter.allowRequest(clientId,
                route.getRateLimitPerMinute())) {
            System.out.printf("[GW-%s] Rate limited: %s%n",
                    requestId, clientId);
            return new GatewayResponse(429, "Too Many Requests",
                    Map.of("error", "Rate limit exceeded",
                           "retry_after", "60"));
        }

        // Step 4: Route to backend
        String targetUrl = route.resolveTarget(path);
        System.out.printf("[GW-%s] Routing to: %s%n", requestId, targetUrl);

        // Simulate backend response
        Map<String, Object> backendResponse = simulateBackend(
                route.getId(), method, path);

        long duration = System.currentTimeMillis() - startTime;
        System.out.printf("[GW-%s] Response: 200 (%dms)%n",
                requestId, duration);

        requestLog.add(String.format("%s %s -> %s (%dms)",
                method, path, route.getId(), duration));

        return new GatewayResponse(200, "OK", backendResponse);
    }

    private Route findRoute(String path) {
        return routes.stream()
                .filter(r -> r.matches(path))
                .findFirst()
                .orElse(null);
    }

    private Map<String, Object> simulateBackend(String serviceId,
                                                String method, String path) {
        Map<String, Object> response = new HashMap<>();
        response.put("service", serviceId);
        response.put("path", path);
        response.put("status", "success");
```

```
        switch (serviceId) {
            case "user-service":
                response.put("data", Map.of(
                    "id", 1, "name", "Alice", "email", "alice@example.com"));
                break;
            case "order-service":
                response.put("data", List.of(
                    Map.of("id", 101, "total", 49.99),
                    Map.of("id", 102, "total", 129.50)));
                break;
            case "product-service":
                response.put("data", List.of(
                    Map.of("id", "P1", "name", "Widget", "price", 19.99)));
                break;
        }
        return response;
    }

    public List<String> getRequestLog() { return requestLog; }
}

public class GatewayResponse {
    public final int statusCode;
    public final String statusMessage;
    public final Map<String, Object> body;

    public GatewayResponse(int statusCode, String statusMessage,
                           Map<String, Object> body) {
        this.statusCode = statusCode;
        this.statusMessage = statusMessage;
        this.body = body;
    }

    @Override
    public String toString() {
        return String.format("HTTP %d %s: %s",
                statusCode, statusMessage, body);
    }
}
```

Demo

```java
public class GatewayDemo {
    public static void main(String[] args) {
        ApiGateway gateway = new ApiGateway();

        // Configure routes
        gateway.addRoute(new Route("user-service",
                "/api/users/**", "http://users.internal:8001",
```

```
                true, 100));
        gateway.addRoute(new Route("order-service",
                "/api/orders/**", "http://orders.internal:8002",
                true, 50));
        gateway.addRoute(new Route("product-service",
                "/api/products/**", "http://products.internal:8003",
                false, 200));  // public, no auth

        Map<String, String> authHeaders = Map.of(
                "Authorization", "Bearer token-alice-123",
                "X-Client-Id", "mobile-app");

        Map<String, String> noAuthHeaders = Map.of(
                "X-Client-Id", "mobile-app");

        // Successful requests
        System.out.println("=== Successful Requests ===");
        System.out.println(gateway.handleRequest(
                "GET", "/api/users/1", authHeaders));
        System.out.println(gateway.handleRequest(
                "GET", "/api/orders", authHeaders));
        System.out.println(gateway.handleRequest(
                "GET", "/api/products", noAuthHeaders));

        // Auth failure
        System.out.println("\n=== Auth Failure ===");
        System.out.println(gateway.handleRequest(
                "GET", "/api/users/1", noAuthHeaders));

        // Invalid token
        System.out.println("\n=== Invalid Token ===");
        System.out.println(gateway.handleRequest(
                "GET", "/api/orders",
                Map.of("Authorization", "Bearer invalid-token",
                       "X-Client-Id", "hacker")));

        // No route
        System.out.println("\n=== No Route ===");
        System.out.println(gateway.handleRequest(
                "GET", "/api/unknown", authHeaders));

        // Request log
        System.out.println("\n=== Request Log ===");
        for (String entry : gateway.getRequestLog()) {
            System.out.println("  " + entry);
        }
    }
}
```

Output:

```
=== Successful Requests ===
[GW-a1b2c3d4] GET /api/users/1
```

```
[GW-a1b2c3d4] Authenticated: alice
[GW-a1b2c3d4] Routing to: http://users.internal:8001/1
[GW-a1b2c3d4] Response: 200 (2ms)
HTTP 200 OK: {service=user-service, path=/api/users/1, status=success, data={id=1, name...

[GW-e5f6g7h8] GET /api/orders
[GW-e5f6g7h8] Authenticated: alice
[GW-e5f6g7h8] Routing to: http://orders.internal:8002
[GW-e5f6g7h8] Response: 200 (1ms)
HTTP 200 OK: {service=order-service, path=/api/orders, status=success, data=[{id=101, t...

[GW-i9j0k1l2] GET /api/products
[GW-i9j0k1l2] Routing to: http://products.internal:8003
[GW-i9j0k1l2] Response: 200 (1ms)
HTTP 200 OK: {service=product-service, path=/api/products, status=success, data=[{id=P1...

=== Auth Failure ===
[GW-m3n4o5p6] GET /api/users/1
[GW-m3n4o5p6] Auth failed: Missing auth header
HTTP 401 Unauthorized: {error=Missing auth header}

=== Invalid Token ===
[GW-q7r8s9t0] GET /api/orders
[GW-q7r8s9t0] Auth failed: Invalid token
HTTP 401 Unauthorized: {error=Invalid token}

=== No Route ===
[GW-u1v2w3x4] GET /api/unknown
[GW-u1v2w3x4] No route found
HTTP 404 Not Found: {error=No route matches: /api/unknown}

=== Request Log ===
  GET /api/users/1 -> user-service (2ms)
  GET /api/orders -> order-service (1ms)
  GET /api/products -> product-service (1ms)
```

Python Implementation: FastAPI Gateway

Lightweight Gateway with FastAPI Concepts

```python
import time
import uuid
from collections import defaultdict
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Any
```

```
@dataclass
class RouteConfig:
    route_id: str
    path_prefix: str
    target_url: str
    auth_required: bool = True
    rate_limit_per_minute: int = 100
    cache_ttl_seconds: int = 0  # 0 = no cache
    methods: List[str] = field(
        default_factory=lambda: ["GET", "POST", "PUT", "DELETE"])


@dataclass
class Request:
    method: str
    path: str
    headers: Dict[str, str] = field(default_factory=dict)
    body: Optional[dict] = None
    query_params: Dict[str, str] = field(default_factory=dict)


@dataclass
class Response:
    status_code: int
    body: Any
    headers: Dict[str, str] = field(default_factory=dict)

    def __repr__(self):
        return f"Response({self.status_code}, {self.body})"


class TokenBucketRateLimiter:
    """Rate limiter using token bucket algorithm."""

    def __init__(self):
        self._buckets: Dict[str, dict] = {}

    def allow(self, client_id: str, max_per_minute: int) -> bool:
        now = time.time()

        if client_id not in self._buckets:
            self._buckets[client_id] = {
                "tokens": max_per_minute,
                "max": max_per_minute,
                "last_refill": now
            }

        bucket = self._buckets[client_id]

        # Refill tokens based on elapsed time
        elapsed = now - bucket["last_refill"]
        new_tokens = int(elapsed * bucket["max"] / 60)
        if new_tokens > 0:
```

```
            bucket["tokens"] = min(bucket["max"],
                                   bucket["tokens"] + new_tokens)
            bucket["last_refill"] = now

        if bucket["tokens"] > 0:
            bucket["tokens"] -= 1
            return True
        return False


class ResponseCache:
    """Simple TTL-based response cache."""

    def __init__(self):
        self._cache: Dict[str, dict] = {}

    def get(self, key: str) -> Optional[Response]:
        if key in self._cache:
            entry = self._cache[key]
            if time.time() < entry["expires_at"]:
                print(f"    [Cache] HIT: {key}")
                return entry["response"]
            else:
                del self._cache[key]
        return None

    def set(self, key: str, response: Response, ttl: int):
        if ttl > 0:
            self._cache[key] = {
                "response": response,
                "expires_at": time.time() + ttl
            }
            print(f"    [Cache] STORED: {key} (ttl={ttl}s)")


class ApiGateway:
    """Lightweight API Gateway implementation."""

    def __init__(self):
        self.routes: List[RouteConfig] = []
        self.auth_tokens = {
            "token-alice": {"user_id": "alice", "roles": ["user", "admin"]},
            "token-bob": {"user_id": "bob", "roles": ["user"]},
        }
        self.rate_limiter = TokenBucketRateLimiter()
        self.cache = ResponseCache()
        self.request_log: List[dict] = []
        self.middleware: List[callable] = []

    def add_route(self, config: RouteConfig):
        self.routes.append(config)
        print(f"[Gateway] Route added: {config.path_prefix} "
              f"-> {config.target_url}")

    def add_middleware(self, middleware_fn):
```

```
        self.middleware.append(middleware_fn)

    def handle_request(self, request: Request) -> Response:
        request_id = str(uuid.uuid4())[:8]
        start = time.time()

        print(f"\n[GW-{request_id}] {request.method} {request.path}")

        # Step 1: Route matching
        route = self._find_route(request.path)
        if not route:
            return Response(404, {"error": f"No route: {request.path}"})

        # Step 2: Method check
        if request.method not in route.methods:
            return Response(405, {"error": "Method not allowed"})

        # Step 3: Authentication
        if route.auth_required:
            auth_result = self._authenticate(request)
            if not auth_result:
                return Response(401, {"error": "Unauthorized"})
            print(f"[GW-{request_id}] Auth: {auth_result['user_id']}")

        # Step 4: Rate limiting
        client_id = request.headers.get("X-Client-Id", "anonymous")
        if not self.rate_limiter.allow(client_id,
                                       route.rate_limit_per_minute):
            print(f"[GW-{request_id}] Rate limited: {client_id}")
            return Response(429, {
                "error": "Rate limit exceeded",
                "retry_after_seconds": 60
            })

        # Step 5: Cache check (GET only)
        if request.method == "GET" and route.cache_ttl_seconds > 0:
            cache_key = f"{request.method}:{request.path}"
            cached = self.cache.get(cache_key)
            if cached:
                return cached

        # Step 6: Forward to backend
        target_url = route.target_url + request.path[
            len(route.path_prefix.rstrip("*").rstrip("/")):]
        print(f"[GW-{request_id}] -> {target_url}")

        response = self._call_backend(route.route_id, request)

        # Step 7: Cache response
        if request.method == "GET" and route.cache_ttl_seconds > 0:
            cache_key = f"{request.method}:{request.path}"
            self.cache.set(cache_key, response, route.cache_ttl_seconds)

        duration_ms = (time.time() - start) * 1000
        print(f"[GW-{request_id}] <- {response.status_code} "
```

```
              f"({duration_ms:.0f}ms)")

        self.request_log.append({
            "request_id": request_id,
            "method": request.method,
            "path": request.path,
            "route": route.route_id,
            "status": response.status_code,
            "duration_ms": duration_ms
        })

        return response

    def _find_route(self, path):
        for route in self.routes:
            prefix = route.path_prefix.rstrip("*").rstrip("/")
            if path.startswith(prefix):
                return route
        return None

    def _authenticate(self, request):
        auth = request.headers.get("Authorization", "")
        token = auth.replace("Bearer ", "")
        return self.auth_tokens.get(token)

    def _call_backend(self, route_id, request):
        """Simulate backend service responses."""
        backends = {
            "user-service": {
                "data": {"id": 1, "name": "Alice",
                         "email": "alice@example.com"}
            },
            "order-service": {
                "data": [
                    {"id": 101, "total": 49.99, "status": "shipped"},
                    {"id": 102, "total": 129.50, "status": "processing"}
                ]
            },
            "product-service": {
                "data": [
                    {"id": "P1", "name": "Widget", "price": 19.99},
                    {"id": "P2", "name": "Gadget", "price": 34.99}
                ]
            }
        }
        data = backends.get(route_id,
                            {"data": None, "message": "Service not found"})
        return Response(200, data)

    def aggregate(self, requests: List[Request]) -> Response:
        """Handle multiple requests and aggregate responses."""
        print("\n[Gateway] Aggregating multiple requests...")
        results = {}
        for req in requests:
            key = req.path.split("/")[2]  # e.g., "users" from "/api/users/1"
```

```
            response = self.handle_request(req)
            results[key] = {
                "status": response.status_code,
                "data": response.body
            }
        return Response(200, results)


# --- Demo ---

gateway = ApiGateway()

# Configure routes
gateway.add_route(RouteConfig(
    route_id="user-service",
    path_prefix="/api/users",
    target_url="http://users.internal:8001",
    auth_required=True,
    rate_limit_per_minute=100
))

gateway.add_route(RouteConfig(
    route_id="order-service",
    path_prefix="/api/orders",
    target_url="http://orders.internal:8002",
    auth_required=True,
    rate_limit_per_minute=50
))

gateway.add_route(RouteConfig(
    route_id="product-service",
    path_prefix="/api/products",
    target_url="http://products.internal:8003",
    auth_required=False,
    rate_limit_per_minute=200,
    cache_ttl_seconds=300  # cache for 5 minutes
))

# Successful authenticated request
print("\n" + "=" * 50)
print("SCENARIO 1: Authenticated Request")
print("=" * 50)
resp = gateway.handle_request(Request(
    method="GET",
    path="/api/users/1",
    headers={"Authorization": "Bearer token-alice",
             "X-Client-Id": "mobile-app"}
))
print(f"Result: {resp}")

# Public request (no auth needed)
print("\n" + "=" * 50)
print("SCENARIO 2: Public Request with Caching")
print("=" * 50)
resp1 = gateway.handle_request(Request(
```

```
    method="GET",
    path="/api/products",
    headers={"X-Client-Id": "web-app"}
))
print(f"First call: {resp1}")

resp2 = gateway.handle_request(Request(
    method="GET",
    path="/api/products",
    headers={"X-Client-Id": "web-app"}
))
print(f"Second call (cached): {resp2}")

# Auth failure
print("\n" + "=" * 50)
print("SCENARIO 3: Auth Failure")
print("=" * 50)
resp = gateway.handle_request(Request(
    method="GET",
    path="/api/orders",
    headers={"X-Client-Id": "unknown"}
))
print(f"Result: {resp}")

# Response aggregation
print("\n" + "=" * 50)
print("SCENARIO 4: Response Aggregation")
print("=" * 50)
headers = {"Authorization": "Bearer token-alice",
           "X-Client-Id": "mobile-app"}
agg = gateway.aggregate([
    Request("GET", "/api/users/1", headers),
    Request("GET", "/api/orders", headers),
    Request("GET", "/api/products", {"X-Client-Id": "mobile-app"})
])
print(f"\nAggregated response keys: {list(agg.body.keys())}")

# Request log
print("\n" + "=" * 50)
print("REQUEST LOG")
print("=" * 50)
for entry in gateway.request_log:
    print(f"  [{entry['request_id']}] {entry['method']:6s} "
          f"{entry['path']:20s} -> {entry['route']:15s} "
          f"({entry['status']}, {entry['duration_ms']:.0f}ms)")
```

Output:

```
[Gateway] Route added: /api/users -> http://users.internal:8001
[Gateway] Route added: /api/orders -> http://orders.internal:8002
[Gateway] Route added: /api/products -> http://products.internal:8003

==================================================
```

```
SCENARIO 1: Authenticated Request
==================================================

[GW-a1b2c3d4] GET /api/users/1
[GW-a1b2c3d4] Auth: alice
[GW-a1b2c3d4] -> http://users.internal:8001/1
[GW-a1b2c3d4] <- 200 (1ms)
Result: Response(200, {'data': {'id': 1, 'name': 'Alice', 'email': 'alice@example.com'}})

==================================================
SCENARIO 2: Public Request with Caching
==================================================

[GW-e5f6g7h8] GET /api/products
[GW-e5f6g7h8] -> http://products.internal:8003
    [Cache] STORED: GET:/api/products (ttl=300s)
[GW-e5f6g7h8] <- 200 (1ms)
First call: Response(200, {'data': [...]})

[GW-i9j0k1l2] GET /api/products
    [Cache] HIT: GET:/api/products
Second call (cached): Response(200, {'data': [...]})

==================================================
SCENARIO 3: Auth Failure
==================================================

[GW-m3n4o5p6] GET /api/orders
Result: Response(401, {'error': 'Unauthorized'})

==================================================
SCENARIO 4: Response Aggregation
==================================================

[Gateway] Aggregating multiple requests...
[GW-...] GET /api/users/1 ...
[GW-...] GET /api/orders ...
[GW-...] GET /api/products ...

Aggregated response keys: ['users', 'orders', 'products']
```

The BFF Pattern: Backend for Frontend

Different clients need different data. A mobile app needs a compact response. A web dashboard needs detailed data. An admin panel needs everything.

```
WITHOUT BFF:

Mobile App  ----+
Web App     ----+--> Single API Gateway --> Services
Admin Panel ----+

Problem: One gateway tries to serve all clients.
Mobile gets too much data. Admin needs special endpoints.

WITH BFF:

Mobile App  ----> Mobile BFF Gateway ---+
                                        +--> Services
Web App     ----> Web BFF Gateway ------+
                                        |
Admin Panel ----> Admin BFF Gateway ----+

Each BFF is tailored to its client's needs.
```

```python
class MobileBFF:
    """Gateway optimized for mobile clients."""

    def __init__(self, gateway):
        self.gateway = gateway

    def get_home_screen(self, user_token):
        """Single call returns everything the mobile home screen needs."""
        headers = {"Authorization": f"Bearer {user_token}",
                   "X-Client-Id": "mobile-bff"}

        # Fetch from multiple services
        user = self.gateway.handle_request(
            Request("GET", "/api/users/me", headers))
        orders = self.gateway.handle_request(
            Request("GET", "/api/orders?limit=3", headers))
        products = self.gateway.handle_request(
            Request("GET", "/api/products?featured=true", headers))

        # Return compact mobile-optimized response
        return {
            "user_name": user.body.get("data", {}).get("name"),
            "recent_orders": len(orders.body.get("data", [])),
            "featured_count": len(products.body.get("data", [])),
        }


class WebBFF:
    """Gateway optimized for web dashboard."""

    def __init__(self, gateway):
        self.gateway = gateway
```

```
    def get_dashboard(self, user_token):
        """Returns full dashboard data for web."""
        headers = {"Authorization": f"Bearer {user_token}",
                   "X-Client-Id": "web-bff"}

        user = self.gateway.handle_request(
            Request("GET", "/api/users/me", headers))
        orders = self.gateway.handle_request(
            Request("GET", "/api/orders", headers))

        # Return full data for web
        return {
            "user": user.body.get("data"),
            "orders": orders.body.get("data"),
            "order_stats": {
                "total": len(orders.body.get("data", [])),
            }
        }
```

Before vs After Comparison

Before: Clients Talk Directly to Services

```
Mobile app makes 5 HTTP calls:
  GET http://users.prod:8001/api/users/me
  GET http://orders.prod:8002/api/orders
  GET http://products.prod:8003/api/products
  GET http://notif.prod:8004/api/notifications
  GET http://points.prod:8005/api/points

Each call needs:
  - Service URL (client must know all 5)
  - Authentication (each service validates separately)
  - Error handling (5 possible failure points)
  - Client-side response merging
```

After: Gateway Aggregates

```
Mobile app makes 1 HTTP call:
  GET http://api.myapp.com/mobile/home

Gateway internally:
  - Authenticates once
```

```
  - Calls 5 services in parallel
  - Merges responses
  - Returns one optimized response
```

When to Use / When NOT to Use

Use API Gateway When

- Multiple clients (mobile, web, third-party) consume your APIs
- You have 3+ microservices that clients need to interact with
- You need centralized authentication and authorization
- Rate limiting is required across all services
- You want to hide internal service topology from clients
- Response aggregation reduces client-side complexity

Do NOT Use API Gateway When

- You have a monolithic application (no services to route to)
- You have only 1-2 backend services (a reverse proxy like Nginx suffices)
- All clients need the exact same data format (no aggregation needed)
- Ultra-low latency is required (every hop adds latency)
- You cannot afford the operational complexity of another service

---

Common Mistakes

Mistake 1: Fat Gateway

```python
# WRONG: business logic in the gateway
def handle_order(self, request):
    order = request.body
    order["tax"] = calculate_tax(order)        # business logic!
    order["discount"] = apply_discount(order)   # business logic!
    order["total"] = compute_total(order)        # business logic!
```

```
    return self.forward(request)
# Gateway becomes a monolith

# RIGHT: gateway only routes
def handle_order(self, request):
    return self.forward_to("order-service", request)
```

Mistake 2: No Timeout on Backend Calls

```python
# WRONG: waiting forever for a slow service
response = call_backend(target_url)  # blocks indefinitely

# RIGHT: set timeouts and handle failures
try:
    response = call_backend(target_url, timeout=5)
except TimeoutError:
    return Response(504, {"error": "Backend timeout"})
```

Mistake 3: Single Point of Failure

```
# WRONG: one gateway instance
Client -> Gateway (single instance) -> Services
         If it dies, EVERYTHING is down.

# RIGHT: multiple instances behind a load balancer
Client -> Load Balancer -> Gateway Instance 1 -> Services
                        -> Gateway Instance 2
                        -> Gateway Instance 3
```

Best Practices

1. Keep the gateway thin. Route, authenticate, rate limit, log. No business logic.

The gateway is infrastructure, not application code.

2. Add timeouts to all backend calls. A slow backend should not make the gateway

unresponsive. Use circuit breakers for frequently failing services.

3. Deploy multiple gateway instances. The gateway is a single point of failure.

Run at least 2-3 instances behind a load balancer.

4. Use correlation IDs. Generate a unique ID for each request and pass it to all backend services. This enables end-to-end request tracing.
5. Cache aggressively for read-heavy endpoints. Products, categories, and static content rarely change. Cache at the gateway to reduce backend load.
6. Consider BFF for diverse clients. If mobile and web need very different data, a Backend-for-Frontend gateway simplifies both client and backend code.
7. Monitor gateway metrics. Track request count, latency percentiles (p50, p95, p99), error rates, and rate limit hits. The gateway sees all traffic.

Quick Summary

The API Gateway pattern provides a single entry point for client requests to a microservice architecture. It handles cross-cutting concerns like authentication, rate limiting, routing, and response aggregation. The BFF variant creates specialized gateways for different client types.

```
Problem:  Clients must know about and communicate with many services.
Solution: Put a gateway in front that routes, authenticates, and aggregates.
Key:      Keep the gateway thin -- infrastructure only, no business logic.
```

Key Points

- API Gateway provides a single entry point for all client-to-service communication
- Core responsibilities: routing, authentication, rate limiting, logging, caching
- Response aggregation reduces multiple backend calls to one client call
- BFF (Backend for Frontend) creates specialized gateways per client type
- Keep the gateway thin: no business logic, only infrastructure concerns
- Deploy multiple instances to avoid single point of failure
- Use correlation IDs for end-to-end request tracing
- Add timeouts and circuit breakers for backend calls
- Popular implementations: Kong, AWS API Gateway, Spring Cloud Gateway, Nginx

---

Practice Questions

1. What are the three most important responsibilities of an API Gateway? Explain why each belongs at the gateway level rather than in individual services.
2. Explain the BFF pattern. When would you use separate BFF gateways instead of one shared gateway?
3. How would you prevent the API Gateway from becoming a single point of failure? Describe at least two strategies.
4. A gateway adds latency to every request. How would you minimize this overhead while keeping the benefits of centralized routing?
5. Compare an API Gateway with a reverse proxy (like Nginx). What can a gateway do that a simple reverse proxy cannot?

Exercises

Exercise 1: API Gateway with Circuit Breaker

Extend the gateway to include circuit breaker functionality:

- Track failure rates per backend service
- When a service fails 5 times in 30 seconds, "open" the circuit
- While open, return a fallback response immediately (no backend call)
- After 10 seconds, allow one "probe" request through
- If the probe succeeds, close the circuit; if it fails, keep it open
- Demonstrate with a service that fails intermittently

Exercise 2: Request Transformation Gateway

Build a gateway that transforms requests and responses:

- Convert REST requests to simulated gRPC calls (change format)
- Add request headers (correlation ID, timestamp, client version)

- Transform snake_case response keys to camelCase for JavaScript clients
- Strip sensitive fields (passwords, internal IDs) from responses
- Log the before/after of each transformation

Exercise 3: Multi-Tenant API Gateway

Build a gateway that supports multiple tenants:

- Each tenant has its own API key, rate limits, and allowed routes
- Tenant "free" gets 10 requests/minute and access to /api/products only
- Tenant "pro" gets 100 requests/minute and access to all routes
- Tenant "enterprise" gets 1000 requests/minute, all routes, plus /api/admin
- Track usage per tenant and display a summary
- Block requests from unknown tenants

What Is Next?

The next chapter introduces Service Discovery, which solves the question the API Gateway raises: how does the gateway know where services are running? In dynamic environments with auto-scaling, service instances come and go. Service Discovery provides the mechanism for finding them automatically.

Chapter 30: Service Discovery -- Finding Services in Dynamic Environments

What You Will Learn

- What Service Discovery is and why static configuration breaks at scale
- The difference between client-side and server-side discovery
- How Eureka, Consul, and similar registries work
- How Kubernetes handles service discovery natively
- How auto-scaling and dynamic environments depend on discovery
- Practical implementations in Java and Python

Why This Chapter Matters

In the previous chapter, the API Gateway routed requests to backend services. But we hardcoded the service URLs: `http://users.internal:8001`. In production, services are not at fixed addresses. They run in containers that start and stop. Auto-scaling adds new instances when traffic spikes and removes them when traffic drops. Deployments replace old instances with new ones.

If your gateway has a hardcoded list of service addresses, it breaks every time an instance moves, scales, or restarts. Service Discovery solves this by providing a dynamic registry where services register themselves and clients look them up.

The Problem

You have 5 microservices, each running 3 instances for reliability. That is 15 addresses to track.

```
WITHOUT SERVICE DISCOVERY:

application.yml:
  user-service:
    - http://10.0.1.10:8001
    - http://10.0.1.11:8001
    - http://10.0.1.12:8001
  order-service:
    - http://10.0.2.20:8002
    - http://10.0.2.21:8002
    - http://10.0.2.22:8002
  product-service:
    - http://10.0.3.30:8003
    - ...

Problems:
- Auto-scaling adds instance at 10.0.1.13 -- config is stale
- Instance 10.0.1.10 crashes -- requests still route there
- Deploy moves service to new IP -- manual config update
- 50 services x 5 instances = 250 addresses to maintain
```

Every change requires a configuration update and often a restart. This does not work in dynamic environments.

The Solution: Service Discovery

```
WITH SERVICE DISCOVERY:

+----------------+                    +-----------------+
| Service A      |--- register ------>|   Service       |
| (startup)      |    name: "users"   |   Registry      |
|                |    host: 10.0.1.10 |                 |
|                |    port: 8001      | users:          |
+----------------+                    |   - 10.0.1.10   |
                                      |   - 10.0.1.11   |
+----------------+                    |   - 10.0.1.12   |
| Service B      |--- lookup -------->|                 |
| (needs users)  |    name: "users"   | orders:         |
|                |<-- [10.0.1.10,     |   - 10.0.2.20   |
|                |     10.0.1.11,     |   - 10.0.2.21   |
|                |     10.0.1.12] ----|                 |
+----------------+                    +-----------------+
```

```
On crash:
  10.0.1.10 stops sending heartbeat
  Registry removes it after timeout
  Clients get updated list: [10.0.1.11, 10.0.1.12]

On scale-up:
  10.0.1.13 starts and registers
  Clients get updated list: [10.0.1.11, 10.0.1.12, 10.0.1.13]
```

Key concepts:

- Service Registration: When a service starts, it registers its name, address, and port with the registry
- Health Checks: Services send periodic heartbeats. If a service stops responding, the registry removes it
- Service Lookup: Clients query the registry by service name to get available instances
- Load Balancing: Clients choose among multiple instances (round-robin, random, least connections)

Client-Side vs Server-Side Discovery

Client-Side Discovery

The client queries the registry directly and chooses which instance to call.

```
+--------+    1. lookup("orders")    +----------+
| Client |---------------------------->| Registry |
|        |<---[inst1, inst2, inst3]---|          |
+--------+                            +----------+
    |
    | 2. client picks inst2 (load balancing)
    v
+---------+
| inst2   |
| (orders)|
+---------+
```

How it works:

1. Client asks registry: "Where is the order service?"
2. Registry returns: [instance1, instance2, instance3]
3. Client picks one (round-robin, random, etc.)
4. Client calls the chosen instance directly

Examples: Netflix Eureka, client-side load balancing

Pros: No extra network hop, client controls load balancing strategy Cons: Client must implement discovery logic, tightly couples client to registry

Server-Side Discovery

The client calls a load balancer or router that handles discovery internally.

```
+--------+    1. call /api/orders    +----------+    2. lookup    +----------+
| Client |--------------------------->|   Load   |--------------->| Registry |
|        |                            | Balancer |<--[inst1,2,3]--|          |
+--------+                            +----------+                +----------+
                                           |
                                           | 3. forward to inst2
                                           v
                                      +---------+
                                      | inst2   |
                                      | (orders)|
                                      +---------+
```

How it works:

1. Client calls a well-known load balancer address
2. Load balancer queries registry for available instances
3. Load balancer picks an instance and forwards the request

Examples: AWS ALB, Kubernetes Services, Nginx

Pros: Client is simple (just knows one URL), discovery logic is centralized Cons: Extra network hop, load balancer is a potential bottleneck

```
+---------------------+---------------------------+---------------------------+
| Aspect              | Client-Side               | Server-Side               |
+---------------------+---------------------------+---------------------------+
| Discovery logic     | In each client            | In load balancer/proxy    |
| Network hops        | Client -> Service         | Client -> LB -> Service   |
| Client complexity   | High (discovery + LB)     | Low (just call one URL)   |
| Language support    | Need library per language | Language agnostic         |
| Example             | Eureka + Ribbon           | Kubernetes Service        |
+---------------------+---------------------------+---------------------------+
```

Java Implementation: Service Registry

The Registry Core

```java
import java.util.*;
import java.util.concurrent.*;

public class ServiceInstance {
    private final String serviceId;
    private final String instanceId;
    private final String host;
    private final int port;
    private final Map<String, String> metadata;
    private long lastHeartbeat;
    private InstanceStatus status;

    public enum InstanceStatus {
        UP, DOWN, STARTING, OUT_OF_SERVICE
    }

    public ServiceInstance(String serviceId, String host, int port) {
        this.serviceId = serviceId;
        this.instanceId = serviceId + "-" + host + ":" + port;
        this.host = host;
        this.port = port;
        this.metadata = new HashMap<>();
        this.lastHeartbeat = System.currentTimeMillis();
        this.status = InstanceStatus.STARTING;
    }

    public void heartbeat() {
        this.lastHeartbeat = System.currentTimeMillis();
        this.status = InstanceStatus.UP;
    }

    public boolean isExpired(long timeoutMs) {
        return System.currentTimeMillis() - lastHeartbeat > timeoutMs;
    }

    public String getUrl() {
        return String.format("http://%s:%d", host, port);
    }

    // Getters
    public String getServiceId() { return serviceId; }
    public String getInstanceId() { return instanceId; }
    public String getHost() { return host; }
    public int getPort() { return port; }
```

```
    public InstanceStatus getStatus() { return status; }
    public void setStatus(InstanceStatus s) { this.status = s; }

    @Override
    public String toString() {
        return String.format("%s [%s] %s", instanceId, status, getUrl());
    }
}
```

```java
public class ServiceRegistry {
    private final ConcurrentHashMap<String, List<ServiceInstance>> services;
    private final long heartbeatTimeoutMs;
    private final ScheduledExecutorService cleaner;

    public ServiceRegistry(long heartbeatTimeoutMs) {
        this.services = new ConcurrentHashMap<>();
        this.heartbeatTimeoutMs = heartbeatTimeoutMs;

        // Background thread to evict expired instances
        this.cleaner = Executors.newSingleThreadScheduledExecutor();
        this.cleaner.scheduleAtFixedRate(
            this::evictExpired, 5, 5, TimeUnit.SECONDS);
    }

    public ServiceInstance register(String serviceId, String host,
                                    int port) {
        ServiceInstance instance = new ServiceInstance(
                serviceId, host, port);
        instance.heartbeat();

        services.computeIfAbsent(serviceId, k -> new CopyOnWriteArrayList<>())
                .add(instance);

        System.out.printf("[Registry] Registered: %s%n", instance);
        return instance;
    }

    public void deregister(String instanceId) {
        services.values().forEach(instances ->
            instances.removeIf(i -> {
                if (i.getInstanceId().equals(instanceId)) {
                    System.out.printf("[Registry] Deregistered: %s%n",
                            instanceId);
                    return true;
                }
                return false;
            })
        );
    }

    public void heartbeat(String instanceId) {
        services.values().forEach(instances ->
```

```
            instances.stream()
                .filter(i -> i.getInstanceId().equals(instanceId))
                .forEach(ServiceInstance::heartbeat)
        );
    }

    public List<ServiceInstance> getInstances(String serviceId) {
        List<ServiceInstance> instances = services.getOrDefault(
                serviceId, Collections.emptyList());

        return instances.stream()
                .filter(i -> i.getStatus() ==
                        ServiceInstance.InstanceStatus.UP)
                .toList();
    }

    public Map<String, List<ServiceInstance>> getAllServices() {
        return Collections.unmodifiableMap(services);
    }

    private void evictExpired() {
        services.values().forEach(instances ->
            instances.removeIf(i -> {
                if (i.isExpired(heartbeatTimeoutMs)) {
                    System.out.printf("[Registry] Evicted (no heartbeat): "
                            + "%s%n", i.getInstanceId());
                    return true;
                }
                return false;
            })
        );
    }

    public void shutdown() {
        cleaner.shutdown();
    }

    public void printStatus() {
        System.out.println("\n=== Registry Status ===");
        if (services.isEmpty()) {
            System.out.println("  (empty)");
            return;
        }
        services.forEach((serviceId, instances) -> {
            System.out.printf("  %s: %d instance(s)%n",
                    serviceId, instances.size());
            instances.forEach(i -> System.out.printf("    %s%n", i));
        });
    }
}
```

Client-Side Load Balancer

```java
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class LoadBalancer {
    public enum Strategy {
        ROUND_ROBIN, RANDOM
    }

    private final ServiceRegistry registry;
    private final Strategy strategy;
    private final AtomicInteger counter = new AtomicInteger(0);

    public LoadBalancer(ServiceRegistry registry, Strategy strategy) {
        this.registry = registry;
        this.strategy = strategy;
    }

    public ServiceInstance choose(String serviceId) {
        List<ServiceInstance> instances = registry.getInstances(serviceId);

        if (instances.isEmpty()) {
            throw new RuntimeException(
                "No instances available for: " + serviceId);
        }

        ServiceInstance chosen;
        switch (strategy) {
            case ROUND_ROBIN:
                int idx = counter.getAndIncrement() % instances.size();
                chosen = instances.get(idx);
                break;
            case RANDOM:
                chosen = instances.get(
                    (int) (Math.random() * instances.size()));
                break;
            default:
                chosen = instances.get(0);
        }

        System.out.printf("[LB] %s -> %s (from %d instances)%n",
                serviceId, chosen.getUrl(), instances.size());
        return chosen;
    }
}
```

Discovery-Aware Service Client

```java
public class DiscoveryClient {
    private final LoadBalancer loadBalancer;
```

```
    public DiscoveryClient(LoadBalancer loadBalancer) {
        this.loadBalancer = loadBalancer;
    }

    public String call(String serviceId, String path) {
        ServiceInstance instance = loadBalancer.choose(serviceId);
        String url = instance.getUrl() + path;

        // Simulate HTTP call
        System.out.printf("[Client] Calling: %s%n", url);
        return String.format("Response from %s at %s",
                serviceId, instance.getUrl());
    }
}
```

Demo

```java
public class DiscoveryDemo {
    public static void main(String[] args) throws InterruptedException {
        ServiceRegistry registry = new ServiceRegistry(10_000);

        // Services register at startup
        System.out.println("=== Service Registration ===");
        ServiceInstance user1 = registry.register(
                "user-service", "10.0.1.10", 8001);
        ServiceInstance user2 = registry.register(
                "user-service", "10.0.1.11", 8001);
        ServiceInstance user3 = registry.register(
                "user-service", "10.0.1.12", 8001);

        ServiceInstance order1 = registry.register(
                "order-service", "10.0.2.20", 8002);
        ServiceInstance order2 = registry.register(
                "order-service", "10.0.2.21", 8002);

        registry.printStatus();

        // Client-side discovery with load balancing
        LoadBalancer lb = new LoadBalancer(
                registry, LoadBalancer.Strategy.ROUND_ROBIN);
        DiscoveryClient client = new DiscoveryClient(lb);

        System.out.println("\n=== Client Calls (Round Robin) ===");
        for (int i = 0; i < 6; i++) {
            String response = client.call("user-service", "/api/users/1");
            System.out.println("  " + response);
        }

        // Simulate instance failure (no more heartbeats)
        System.out.println("\n=== Simulating Instance Failure ===");
```

```
        System.out.println("Instance user1 stops sending heartbeats...");
        user1.setStatus(ServiceInstance.InstanceStatus.DOWN);

        // Now calls only go to healthy instances
        System.out.println("\n=== Calls After Failure ===");
        for (int i = 0; i < 4; i++) {
            client.call("user-service", "/api/users/1");
        }

        // Simulate scale-up
        System.out.println("\n=== Auto-Scale: Adding Instance ===");
        registry.register("user-service", "10.0.1.13", 8001);

        registry.printStatus();

        // Graceful deregister
        System.out.println("\n=== Graceful Shutdown ===");
        registry.deregister(user2.getInstanceId());
        registry.printStatus();

        registry.shutdown();
    }
}
```

Output:

```
=== Service Registration ===
[Registry] Registered: user-service-10.0.1.10:8001 [UP] http://10.0.1.10:8001
[Registry] Registered: user-service-10.0.1.11:8001 [UP] http://10.0.1.11:8001
[Registry] Registered: user-service-10.0.1.12:8001 [UP] http://10.0.1.12:8001
[Registry] Registered: order-service-10.0.2.20:8002 [UP] http://10.0.2.20:8002
[Registry] Registered: order-service-10.0.2.21:8002 [UP] http://10.0.2.21:8002

=== Registry Status ===
  user-service: 3 instance(s)
    user-service-10.0.1.10:8001 [UP] http://10.0.1.10:8001
    user-service-10.0.1.11:8001 [UP] http://10.0.1.11:8001
    user-service-10.0.1.12:8001 [UP] http://10.0.1.12:8001
  order-service: 2 instance(s)
    order-service-10.0.2.20:8002 [UP] http://10.0.2.20:8002
    order-service-10.0.2.21:8002 [UP] http://10.0.2.21:8002

=== Client Calls (Round Robin) ===
[LB] user-service -> http://10.0.1.10:8001 (from 3 instances)
[Client] Calling: http://10.0.1.10:8001/api/users/1
  Response from user-service at http://10.0.1.10:8001
[LB] user-service -> http://10.0.1.11:8001 (from 3 instances)
[Client] Calling: http://10.0.1.11:8001/api/users/1
  Response from user-service at http://10.0.1.11:8001
[LB] user-service -> http://10.0.1.12:8001 (from 3 instances)
[Client] Calling: http://10.0.1.12:8001/api/users/1
  Response from user-service at http://10.0.1.12:8001
[LB] user-service -> http://10.0.1.10:8001 (from 3 instances)
```

```
...

=== Simulating Instance Failure ===
Instance user1 stops sending heartbeats...

=== Calls After Failure ===
[LB] user-service -> http://10.0.1.11:8001 (from 2 instances)
[Client] Calling: http://10.0.1.11:8001/api/users/1
[LB] user-service -> http://10.0.1.12:8001 (from 2 instances)
[Client] Calling: http://10.0.1.12:8001/api/users/1
...

=== Auto-Scale: Adding Instance ===
[Registry] Registered: user-service-10.0.1.13:8001 [UP] http://10.0.1.13:8001

=== Registry Status ===
  user-service: 4 instance(s)
    user-service-10.0.1.10:8001 [DOWN] http://10.0.1.10:8001
    user-service-10.0.1.11:8001 [UP] http://10.0.1.11:8001
    user-service-10.0.1.12:8001 [UP] http://10.0.1.12:8001
    user-service-10.0.1.13:8001 [UP] http://10.0.1.13:8001
  order-service: 2 instance(s)
    ...

=== Graceful Shutdown ===
[Registry] Deregistered: user-service-10.0.1.11:8001

=== Registry Status ===
  user-service: 3 instance(s)
    user-service-10.0.1.10:8001 [DOWN] http://10.0.1.10:8001
    user-service-10.0.1.12:8001 [UP] http://10.0.1.12:8001
    user-service-10.0.1.13:8001 [UP] http://10.0.1.13:8001
  order-service: 2 instance(s)
    ...
```

Python Implementation: Service Discovery with Health Checks

```python
import time
import random
import threading
from dataclasses import dataclass, field
from typing import Dict, List, Optional
from enum import Enum
```

```
class InstanceStatus(Enum):
    UP = "UP"
    DOWN = "DOWN"
    STARTING = "STARTING"


@dataclass
class ServiceInstance:
    service_id: str
    host: str
    port: int
    status: InstanceStatus = InstanceStatus.UP
    metadata: Dict[str, str] = field(default_factory=dict)
    last_heartbeat: float = field(default_factory=time.time)
    registered_at: float = field(default_factory=time.time)

    @property
    def instance_id(self):
        return f"{self.service_id}-{self.host}:{self.port}"

    @property
    def url(self):
        return f"http://{self.host}:{self.port}"

    def heartbeat(self):
        self.last_heartbeat = time.time()
        self.status = InstanceStatus.UP

    def is_expired(self, timeout_seconds):
        return time.time() - self.last_heartbeat > timeout_seconds

    def __repr__(self):
        return f"{self.instance_id} [{self.status.value}] {self.url}"


class ServiceRegistry:
    """Central service registry with health checking."""

    def __init__(self, heartbeat_timeout=30):
        self._services: Dict[str, List[ServiceInstance]] = {}
        self._heartbeat_timeout = heartbeat_timeout
        self._lock = threading.Lock()

    def register(self, service_id, host, port, **metadata):
        instance = ServiceInstance(
            service_id=service_id,
            host=host,
            port=port,
            metadata=metadata
        )

        with self._lock:
            if service_id not in self._services:
                self._services[service_id] = []
```

```
            self._services[service_id].append(instance)

        print(f"[Registry] Registered: {instance}")
        return instance

    def deregister(self, instance_id):
        with self._lock:
            for instances in self._services.values():
                before = len(instances)
                instances[:] = [i for i in instances
                                if i.instance_id != instance_id]
                if len(instances) < before:
                    print(f"[Registry] Deregistered: {instance_id}")
                    return True
        return False

    def heartbeat(self, instance_id):
        with self._lock:
            for instances in self._services.values():
                for inst in instances:
                    if inst.instance_id == instance_id:
                        inst.heartbeat()
                        return True
        return False

    def get_instances(self, service_id):
        with self._lock:
            instances = self._services.get(service_id, [])
            return [i for i in instances
                    if i.status == InstanceStatus.UP]

    def get_all_services(self):
        with self._lock:
            return {
                sid: list(instances)
                for sid, instances in self._services.items()
            }

    def evict_expired(self):
        with self._lock:
            for sid, instances in self._services.items():
                expired = [i for i in instances
                           if i.is_expired(self._heartbeat_timeout)]
                for inst in expired:
                    inst.status = InstanceStatus.DOWN
                    print(f"[Registry] Expired: {inst.instance_id}")

    def print_status(self):
        print("\n--- Registry Status ---")
        for sid, instances in self._services.items():
            up_count = sum(1 for i in instances
                           if i.status == InstanceStatus.UP)
            print(f"  {sid}: {up_count}/{len(instances)} UP")
            for inst in instances:
                print(f"    {inst}")
```

```
        print()


class LoadBalancer:
    """Client-side load balancer."""

    def __init__(self, registry, strategy="round_robin"):
        self.registry = registry
        self.strategy = strategy
        self._counters: Dict[str, int] = {}

    def choose(self, service_id) -> ServiceInstance:
        instances = self.registry.get_instances(service_id)

        if not instances:
            raise RuntimeError(
                f"No healthy instances for: {service_id}")

        if self.strategy == "round_robin":
            idx = self._counters.get(service_id, 0)
            chosen = instances[idx % len(instances)]
            self._counters[service_id] = idx + 1
        elif self.strategy == "random":
            chosen = random.choice(instances)
        else:
            chosen = instances[0]

        return chosen


class ServiceClient:
    """Discovery-aware HTTP client."""

    def __init__(self, registry, strategy="round_robin"):
        self.lb = LoadBalancer(registry, strategy)

    def call(self, service_id, path, method="GET"):
        instance = self.lb.choose(service_id)
        url = f"{instance.url}{path}"
        print(f"[Client] {method} {url}")
        # In production: actual HTTP call here
        return {"url": url, "status": 200, "instance": instance.instance_id}


# --- Demo ---

registry = ServiceRegistry(heartbeat_timeout=30)

# Register service instances
print("=== Registration ===")
u1 = registry.register("user-service", "10.0.1.10", 8001, version="2.1")
u2 = registry.register("user-service", "10.0.1.11", 8001, version="2.1")
u3 = registry.register("user-service", "10.0.1.12", 8001, version="2.1")

o1 = registry.register("order-service", "10.0.2.20", 8002)
```

```
o2 = registry.register("order-service", "10.0.2.21", 8002)

p1 = registry.register("product-service", "10.0.3.30", 8003)

registry.print_status()

# Client calls with load balancing
client = ServiceClient(registry)

print("=== Round Robin Calls ===")
for i in range(6):
    result = client.call("user-service", "/api/users/1")
    print(f"  -> {result['instance']}")

# Simulate failure
print("\n=== Simulate Failure ===")
u1.status = InstanceStatus.DOWN
print(f"Instance {u1.instance_id} is DOWN")

print("\nCalls after failure:")
for i in range(4):
    result = client.call("user-service", "/api/users/1")
    print(f"  -> {result['instance']}")

# Scale up
print("\n=== Scale Up ===")
u4 = registry.register("user-service", "10.0.1.13", 8001, version="2.1")

print("\nCalls after scale-up:")
for i in range(4):
    result = client.call("user-service", "/api/users/1")
    print(f"  -> {result['instance']}")

# Graceful shutdown
print("\n=== Graceful Shutdown ===")
registry.deregister(u2.instance_id)
registry.print_status()
```

Output:

```
=== Registration ===
[Registry] Registered: user-service-10.0.1.10:8001 [UP] http://10.0.1.10:8001
[Registry] Registered: user-service-10.0.1.11:8001 [UP] http://10.0.1.11:8001
[Registry] Registered: user-service-10.0.1.12:8001 [UP] http://10.0.1.12:8001
[Registry] Registered: order-service-10.0.2.20:8002 [UP] http://10.0.2.20:8002
[Registry] Registered: order-service-10.0.2.21:8002 [UP] http://10.0.2.21:8002
[Registry] Registered: product-service-10.0.3.30:8003 [UP] http://10.0.3.30:8003

--- Registry Status ---
  user-service: 3/3 UP
    user-service-10.0.1.10:8001 [UP] http://10.0.1.10:8001
    user-service-10.0.1.11:8001 [UP] http://10.0.1.11:8001
    user-service-10.0.1.12:8001 [UP] http://10.0.1.12:8001
```

```
  order-service: 2/2 UP
    order-service-10.0.2.20:8002 [UP] http://10.0.2.20:8002
    order-service-10.0.2.21:8002 [UP] http://10.0.2.21:8002
  product-service: 1/1 UP
    product-service-10.0.3.30:8003 [UP] http://10.0.3.30:8003

=== Round Robin Calls ===
[Client] GET http://10.0.1.10:8001/api/users/1
  -> user-service-10.0.1.10:8001
[Client] GET http://10.0.1.11:8001/api/users/1
  -> user-service-10.0.1.11:8001
[Client] GET http://10.0.1.12:8001/api/users/1
  -> user-service-10.0.1.12:8001
[Client] GET http://10.0.1.10:8001/api/users/1
  -> user-service-10.0.1.10:8001
...

=== Simulate Failure ===
Instance user-service-10.0.1.10:8001 is DOWN

Calls after failure:
[Client] GET http://10.0.1.11:8001/api/users/1
  -> user-service-10.0.1.11:8001
[Client] GET http://10.0.1.12:8001/api/users/1
  -> user-service-10.0.1.12:8001
...

=== Scale Up ===
[Registry] Registered: user-service-10.0.1.13:8001 [UP] http://10.0.1.13:8001

Calls after scale-up:
[Client] GET http://10.0.1.11:8001/api/users/1
  -> user-service-10.0.1.11:8001
[Client] GET http://10.0.1.12:8001/api/users/1
  -> user-service-10.0.1.12:8001
[Client] GET http://10.0.1.13:8001/api/users/1
  -> user-service-10.0.1.13:8001
...
```

Kubernetes Service Discovery

Kubernetes has built-in service discovery. No external registry needed.

```
+-----------------------------------------+
|            Kubernetes Cluster            |
|                                          |
|  Service: user-service                   |
|  ClusterIP: 10.96.0.1                   |
```

```
|  DNS: user-service.default.svc.cluster    |
|       |                                   |
|       +---> Pod 1 (10.244.0.10)          |
|       +---> Pod 2 (10.244.0.11)          |
|       +---> Pod 3 (10.244.0.12)          |
|                                           |
|  Any pod can call:                        |
|    http://user-service:8001/api/users     |
|  Kubernetes routes to a healthy pod.      |
+-------------------------------------------+
```

How Kubernetes discovery works:

1. You deploy a Service object (YAML manifest)
2. Kubernetes creates a stable DNS name: `user-service.default.svc.cluster.local`
3. Kubernetes maintains a list of healthy Pods behind that DNS name
4. When a Pod starts, it is added; when it fails health checks, it is removed
5. Any Pod in the cluster can call `http://user-service:8001` -- Kubernetes handles

the rest

```yaml
# Kubernetes Service manifest
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  selector:
    app: user-service
  ports:
    - port: 8001
      targetPort: 8001
  type: ClusterIP
```

Kubernetes vs External Registry:

```
+-------------------+---------------------------+---------------------------+
| Feature           | Eureka/Consul             | Kubernetes Service        |
+-------------------+---------------------------+---------------------------+
| Setup             | Install and configure     | Built-in, zero setup      |
| Health checks     | Service sends heartbeats  | Kubernetes probes Pods    |
| DNS               | Registry API calls        | Automatic DNS names       |
| Load balancing    | Client-side (need lib)    | Built-in (iptables/IPVS) |
| Works outside K8s | Yes                       | No                        |
| Extra component   | Yes (registry server)     | No                        |
+-------------------+---------------------------+---------------------------+
```

Real-World: Eureka and Consul

Eureka (Netflix)

Eureka is the most widely used service discovery for Java/Spring applications.

```
How Eureka Works:

1. Service starts -> registers with Eureka Server
2. Service sends heartbeat every 30 seconds
3. Eureka Server removes instance if no heartbeat for 90 seconds
4. Clients fetch registry every 30 seconds (cached locally)
5. If Eureka Server dies, clients use cached registry

Key features:
- Self-preservation mode (does not evict during network partitions)
- Zone-aware load balancing
- Dashboard UI for monitoring
```

Consul (HashiCorp)

Consul provides service discovery plus health checking, key-value storage, and service mesh.

```
How Consul Works:

1. Consul Agent runs on every node (as sidecar or daemon)
2. Service registers with local agent
3. Agent performs health checks (HTTP, TCP, script)
4. Agents gossip with each other to share state
5. DNS or HTTP API for lookups

Key features:
- Multi-datacenter support
- Built-in health checking (HTTP, TCP, gRPC, script)
- Key-value store for configuration
- Service mesh with automatic TLS
```

Auto-Scaling and Service Discovery

Service discovery enables auto-scaling by making new instances automatically available.

```
Normal Load:
  user-service: 3 instances

Traffic Spike Detected (CPU > 80%):
  Auto-scaler creates 2 new instances
  New instances register with registry
  Load balancer immediately includes them
  user-service: 5 instances

Traffic Returns to Normal:
  Auto-scaler terminates 2 instances
  Instances deregister (or heartbeat times out)
  Load balancer removes them
  user-service: 3 instances

No configuration changes needed at any step!
```

Before vs After Comparison

Before: Static Configuration

```yaml
# application.yml -- hardcoded addresses
services:
  user-service: http://10.0.1.10:8001
  order-service: http://10.0.2.20:8002

# When 10.0.1.10 crashes:
#   1. Ops team notices (maybe)
#   2. Update config file
#   3. Restart all dependent services
#   4. Hope nothing else broke
```

After: Dynamic Discovery

```python
# Service registration at startup
registry.register("user-service", my_host, my_port)

# Client lookup at call time
client.call("user-service", "/api/users/1")

# When an instance crashes:
```

```
#   1. Registry detects missing heartbeat (automatic)
#   2. Instance removed from registry (automatic)
#   3. Clients get updated list (automatic)
#   4. No config changes, no restarts
```

When to Use / When NOT to Use

Use Service Discovery When

- Services have multiple instances (horizontal scaling)
- Instances are ephemeral (containers, cloud VMs)
- Auto-scaling adds/removes instances dynamically
- You have more than 3-4 microservices
- Service locations change across deployments
- You need health-aware routing (skip unhealthy instances)

Do NOT Use Service Discovery When

- You have a monolithic application (one process, one address)
- Services run on fixed, known infrastructure (bare metal, static VMs)
- You have only 1-2 services with 1 instance each
- A simple load balancer (Nginx, HAProxy) handles your needs
- You are already using Kubernetes (it has built-in discovery)

Common Mistakes

Mistake 1: Not Handling Registry Unavailability

```python
# WRONG: crash if registry is down
instances = registry.get_instances("user-service")
# If registry is unreachable -> exception -> service fails

# RIGHT: cache last known good instances
```

```
try:
    instances = registry.get_instances("user-service")
    self._cache["user-service"] = instances
except RegistryUnavailable:
    instances = self._cache.get("user-service", [])
    if not instances:
        raise
```

Mistake 2: No Graceful Deregistration

```python
# WRONG: just kill the process
os.exit(1)  # instance disappears without deregistering
# Registry waits 90 seconds before evicting -- 90 seconds of errors!

# RIGHT: deregister before shutdown
import signal

def shutdown_handler(signum, frame):
    registry.deregister(my_instance_id)
    sys.exit(0)

signal.signal(signal.SIGTERM, shutdown_handler)
```

Mistake 3: Health Check Returns 200 When Unhealthy

```python
# WRONG: always returns healthy
@app.get("/health")
def health():
    return {"status": "UP"}  # even when DB is down!

# RIGHT: check actual dependencies
@app.get("/health")
def health():
    db_ok = check_database_connection()
    cache_ok = check_cache_connection()
    if db_ok and cache_ok:
        return {"status": "UP"}
    return Response(status_code=503,
                    content={"status": "DOWN",
                             "db": db_ok, "cache": cache_ok})
```

Best Practices

1. Implement graceful shutdown. When a service stops, deregister from the registry before exiting. This prevents routing to dead instances.
2. Cache the registry locally. If the registry goes down, clients should use their last known good copy. Eureka does this by default.
3. Use meaningful health checks. Do not just return 200. Check database connections, downstream services, and disk space. Only report healthy when actually ready.
4. Set appropriate timeouts. Heartbeat intervals (30s) and eviction timeouts (90s) should balance between quick failure detection and false positives.
5. Use Kubernetes discovery when possible. If you are running on Kubernetes, its built-in service discovery eliminates the need for Eureka or Consul.
6. Monitor registry health. The registry itself is a critical component. Monitor its availability and set up alerts for anomalies (sudden drops in registered instances).
7. Test failure scenarios. Regularly test: What happens when an instance crashes? When the registry goes down? When a network partition occurs?

Quick Summary

Service Discovery provides a dynamic registry where services register themselves and clients look them up. It replaces static configuration with automatic, health-aware routing. Client-side discovery puts the logic in each client; server-side discovery uses a load balancer. Kubernetes provides built-in discovery. External registries like Eureka and Consul serve non-Kubernetes environments.

```
Problem:  Service locations change dynamically in cloud/container environments.
Solution: Services register with a central registry; clients look up addresses.
Key:      Health checks automatically remove failed instances.
```

Key Points

- Service Discovery replaces static configuration with dynamic service registration

- Service Registration: services announce themselves at startup, deregister at shutdown
- Health Checks: heartbeats detect failed instances and remove them automatically
- Client-side discovery (Eureka): client queries registry, picks an instance
- Server-side discovery (Kubernetes, AWS ALB): load balancer handles discovery
- Kubernetes has built-in discovery via DNS and Services -- no external registry needed
- Eureka (Netflix) and Consul (HashiCorp) are popular external registries
- Cache registry data locally to survive registry outages
- Graceful shutdown (deregister before exit) prevents routing to dead instances

---

Practice Questions

1. Explain the difference between client-side and server-side service discovery. When would you choose each approach?
2. What happens when the service registry itself goes down? How do systems like Eureka handle this?
3. A service instance crashes without deregistering. How long before clients stop routing to it? What determines this delay?
4. You are migrating from VMs to Kubernetes. How does your service discovery strategy change? What can you remove?
5. Your service has a health check endpoint that returns 200 even when its database is down. What are the consequences? How would you fix it?

Exercises

Exercise 1: Service Registry with Health Dashboard

Build a service registry with a dashboard:

- Services register with name, host, port, and version
- Health checker pings services every 10 seconds
- Dashboard displays: service name, instance count (healthy/total), uptime
- Simulate: start 5 instances of 3 services, kill 2, add 3 more
- Print dashboard after each change showing real-time state

Exercise 2: Multi-Zone Discovery

Implement zone-aware service discovery:

- Each instance registers with a zone (us-east-1a, us-east-1b, us-east-1c)
- Load balancer prefers instances in the same zone as the caller
- If no same-zone instances are healthy, fall back to other zones
- If a zone has fewer than 2 healthy instances, stop preferring it
- Demonstrate zone failover with 3 zones and selective instance failures

Exercise 3: Service Discovery with API Gateway

Integrate service discovery with the API Gateway from Chapter 29:

- The gateway resolves service URLs dynamically from the registry
- When routing `/api/users/**`, look up "user-service" in the registry
- Use round-robin load balancing across discovered instances
- Handle the case where a backend call fails and retry with the next instance
- Demonstrate: register 3 instances, route requests, kill one instance, show automatic failover

---

What Is Next?

The next chapter ties everything together in a comprehensive project. We build a complete E-Commerce Application applying 15+ patterns from this book, including Singleton, Factory, Builder, Strategy, Observer, Command, Repository, CQRS, Circuit Breaker, Saga, API Gateway, and Service Discovery. It is the culmination of everything you have learned.

Chapter 31: Project -- Building a Complete E-Commerce System

What You Will Learn

- How to combine 15+ design patterns into a cohesive system
- Architecture decisions for a real e-commerce platform
- How patterns interact and reinforce each other
- Complete working code for core e-commerce functionality
- How to think architecturally about pattern selection

Why This Chapter Matters

Individual patterns are tools. This chapter teaches you to use them together. Building a real system requires knowing not just what each pattern does, but when to apply which pattern, how they compose, and where they overlap. You will see how Singleton, Factory, Builder, Strategy, Observer, Command, Chain of Responsibility, Repository, CQRS, Circuit Breaker, and Saga work together to create a production-quality e-commerce backend.

System Architecture Overview

```
+-----------------------------------------------------------------+
|                        API Gateway                              |
|  (Routing, Auth, Rate Limiting)                                 |
+-----------------------------------------------------------------+
         |              |              |              |
         v              v              v              v
```

```
+------------+  +------------+  +------------+  +------------+
|  Product   |  |   Order    |  |  Payment   |  | Notification|
|  Service   |  |  Service   |  |  Service   |  |  Service   |
+------------+  +------------+  +------------+  +------------+
     |               |              |               |
     v               v              v               v
+------------+  +------------+  +------------+  +------------+
| Product DB |  | Order DB   |  | Payment DB |  | Email/SMS  |
+------------+  +------------+  +------------+  +------------+

Cross-cutting:
  - Service Discovery (all services register)
  - Circuit Breaker (all inter-service calls)
  - Observer/Events (async communication)
  - Saga (distributed transactions)
```

Patterns Applied

Here is every pattern we use in this project and why:

```
+----------------------------+-------------------------------------------+
| Pattern                    | Where We Use It                           |
+----------------------------+-------------------------------------------+
| Singleton                  | Configuration manager, DB connection pool |
| Factory Method             | Create payment processors by type         |
| Builder                    | Construct complex Order objects            |
| Strategy                   | Pricing, discount, and shipping rules     |
| Observer                   | Event bus for async notifications         |
| Command                    | Order operations (place, cancel, refund)  |
| Chain of Responsibility    | Order validation pipeline                 |
| Repository                 | Data access abstraction                   |
| CQRS                       | Separate read/write models for products  |
| Circuit Breaker            | Resilient inter-service calls             |
| Saga                       | Distributed checkout transaction          |
+----------------------------+-------------------------------------------+
```

Part 1: Foundation Layer

Singleton -- Configuration Manager

Every service needs access to configuration. A Singleton ensures one configuration instance across the application.

```python
import threading


class Config:
    """Singleton configuration manager."""

    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
                    cls._instance._initialized = False
        return cls._instance

    def __init__(self):
        if self._initialized:
            return
        self._initialized = True
        self._settings = {
            "db_host": "localhost",
            "db_port": 5432,
            "db_name": "ecommerce",
            "payment_timeout": 30,
            "max_retry": 3,
            "tax_rate": 0.08,
            "free_shipping_threshold": 50.00,
            "currency": "USD",
        }

    def get(self, key, default=None):
        return self._settings.get(key, default)

    def set(self, key, value):
        self._settings[key] = value


# Usage across the entire application
config = Config()
print(f"Tax rate: {config.get('tax_rate')}")
print(f"Same instance? {Config() is Config()}")  # True
```

Output:

```
Tax rate: 0.08
Same instance? True
```

Repository -- Data Access Layer

The Repository pattern abstracts away data storage, making business logic independent of the database.

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import List, Optional, Dict
from datetime import datetime
import uuid


@dataclass
class Product:
    id: str
    name: str
    price: float
    category: str
    stock: int
    description: str = ""
    created_at: datetime = field(default_factory=datetime.now)


@dataclass
class OrderItem:
    product_id: str
    product_name: str
    quantity: int
    unit_price: float

    @property
    def subtotal(self):
        return self.quantity * self.unit_price


@dataclass
class Order:
    id: str
    customer_id: str
    items: List[OrderItem]
    status: str = "pending"
    total: float = 0.0
    tax: float = 0.0
    shipping_cost: float = 0.0
    discount: float = 0.0
    payment_method: str = ""
    shipping_address: str = ""
    created_at: datetime = field(default_factory=datetime.now)
    updated_at: datetime = field(default_factory=datetime.now)


class ProductRepository(ABC):
```

```
    @abstractmethod
    def find_by_id(self, product_id: str) -> Optional[Product]:
        pass

    @abstractmethod
    def find_by_category(self, category: str) -> List[Product]:
        pass

    @abstractmethod
    def save(self, product: Product) -> Product:
        pass

    @abstractmethod
    def update_stock(self, product_id: str, quantity_change: int) -> bool:
        pass

    @abstractmethod
    def find_all(self) -> List[Product]:
        pass


class OrderRepository(ABC):
    @abstractmethod
    def find_by_id(self, order_id: str) -> Optional[Order]:
        pass

    @abstractmethod
    def find_by_customer(self, customer_id: str) -> List[Order]:
        pass

    @abstractmethod
    def save(self, order: Order) -> Order:
        pass

    @abstractmethod
    def update_status(self, order_id: str, status: str) -> bool:
        pass


class InMemoryProductRepository(ProductRepository):
    """In-memory implementation for demonstration."""

    def __init__(self):
        self._products: Dict[str, Product] = {}

    def find_by_id(self, product_id):
        return self._products.get(product_id)

    def find_by_category(self, category):
        return [p for p in self._products.values()
                if p.category == category]

    def save(self, product):
        self._products[product.id] = product
        return product
```

```
    def update_stock(self, product_id, quantity_change):
        product = self._products.get(product_id)
        if product and product.stock + quantity_change >= 0:
            product.stock += quantity_change
            return True
        return False

    def find_all(self):
        return list(self._products.values())


class InMemoryOrderRepository(OrderRepository):
    def __init__(self):
        self._orders: Dict[str, Order] = {}

    def find_by_id(self, order_id):
        return self._orders.get(order_id)

    def find_by_customer(self, customer_id):
        return [o for o in self._orders.values()
                if o.customer_id == customer_id]

    def save(self, order):
        self._orders[order.id] = order
        return order

    def update_status(self, order_id, status):
        order = self._orders.get(order_id)
        if order:
            order.status = status
            order.updated_at = datetime.now()
            return True
        return False
```

Part 2: Business Logic Layer

Builder -- Order Construction

Orders are complex objects with many optional fields. The Builder pattern makes construction clean and readable.

```python
class OrderBuilder:
    """Builds Order objects step by step."""
```

```
    def __init__(self, customer_id: str):
        self._customer_id = customer_id
        self._items: List[OrderItem] = []
        self._shipping_address = ""
        self._payment_method = ""
        self._discount = 0.0

    def add_item(self, product: Product, quantity: int):
        if product.stock < quantity:
            raise ValueError(
                f"Insufficient stock for {product.name}: "
                f"need {quantity}, have {product.stock}")
        self._items.append(OrderItem(
            product_id=product.id,
            product_name=product.name,
            quantity=quantity,
            unit_price=product.price
        ))
        return self

    def shipping_address(self, address: str):
        self._shipping_address = address
        return self

    def payment_method(self, method: str):
        self._payment_method = method
        return self

    def apply_discount(self, discount: float):
        self._discount = discount
        return self

    def build(self) -> Order:
        if not self._items:
            raise ValueError("Order must have at least one item")
        if not self._shipping_address:
            raise ValueError("Shipping address is required")
        if not self._payment_method:
            raise ValueError("Payment method is required")

        order = Order(
            id=f"ORD-{uuid.uuid4().hex[:8].upper()}",
            customer_id=self._customer_id,
            items=list(self._items),
            shipping_address=self._shipping_address,
            payment_method=self._payment_method,
            discount=self._discount
        )
        return order
```

Strategy -- Pricing, Discounts, and Shipping

Different business rules for calculating prices, discounts, and shipping costs are encapsulated as interchangeable strategies.

```python
class PricingStrategy(ABC):
    @abstractmethod
    def calculate(self, items: List[OrderItem]) -> float:
        pass


class StandardPricing(PricingStrategy):
    def calculate(self, items):
        return sum(item.subtotal for item in items)


class BulkPricing(PricingStrategy):
    """10% off when ordering 5+ of any single item."""

    def calculate(self, items):
        total = 0.0
        for item in items:
            if item.quantity >= 5:
                total += item.subtotal * 0.90  # 10% off
            else:
                total += item.subtotal
        return total


class DiscountStrategy(ABC):
    @abstractmethod
    def apply(self, subtotal: float, customer_id: str) -> float:
        pass


class NoDiscount(DiscountStrategy):
    def apply(self, subtotal, customer_id):
        return 0.0


class PercentageDiscount(DiscountStrategy):
    def __init__(self, percentage: float):
        self.percentage = percentage

    def apply(self, subtotal, customer_id):
        return subtotal * (self.percentage / 100)


class LoyaltyDiscount(DiscountStrategy):
    """VIP customers get 15% off orders over $100."""

    def __init__(self, vip_customers: set):
        self.vip_customers = vip_customers

    def apply(self, subtotal, customer_id):
```

```
        if customer_id in self.vip_customers and subtotal > 100:
            return subtotal * 0.15
        return 0.0


class ShippingStrategy(ABC):
    @abstractmethod
    def calculate(self, subtotal: float, items: List[OrderItem]) -> float:
        pass


class StandardShipping(ShippingStrategy):
    def calculate(self, subtotal, items):
        config = Config()
        if subtotal >= config.get("free_shipping_threshold", 50):
            return 0.0
        return 5.99


class ExpressShipping(ShippingStrategy):
    def calculate(self, subtotal, items):
        total_items = sum(item.quantity for item in items)
        return 9.99 + (total_items * 1.50)


class FreeShipping(ShippingStrategy):
    def calculate(self, subtotal, items):
        return 0.0
```

Factory Method -- Payment Processors

Different payment methods require different processing logic. The Factory pattern creates the right processor based on the payment method.

```python
class PaymentProcessor(ABC):
    @abstractmethod
    def charge(self, amount: float, order_id: str) -> dict:
        pass

    @abstractmethod
    def refund(self, amount: float, order_id: str) -> dict:
        pass


class CreditCardProcessor(PaymentProcessor):
    def charge(self, amount, order_id):
        print(f"    [CreditCard] Charging ${amount:.2f} "
              f"for order {order_id}")
        return {"status": "charged", "method": "credit_card",
```

```
                "transaction_id": f"CC-{uuid.uuid4().hex[:8]}"}

    def refund(self, amount, order_id):
        print(f"    [CreditCard] Refunding ${amount:.2f} "
              f"for order {order_id}")
        return {"status": "refunded", "method": "credit_card"}


class PayPalProcessor(PaymentProcessor):
    def charge(self, amount, order_id):
        print(f"    [PayPal] Charging ${amount:.2f} for order {order_id}")
        return {"status": "charged", "method": "paypal",
                "transaction_id": f"PP-{uuid.uuid4().hex[:8]}"}

    def refund(self, amount, order_id):
        print(f"    [PayPal] Refunding ${amount:.2f} for order {order_id}")
        return {"status": "refunded", "method": "paypal"}


class CryptoProcessor(PaymentProcessor):
    def charge(self, amount, order_id):
        print(f"    [Crypto] Charging ${amount:.2f} for order {order_id}")
        return {"status": "charged", "method": "crypto",
                "transaction_id": f"BTC-{uuid.uuid4().hex[:8]}"}

    def refund(self, amount, order_id):
        print(f"    [Crypto] Refunding ${amount:.2f} for order {order_id}")
        return {"status": "refunded", "method": "crypto"}


class PaymentProcessorFactory:
    """Factory that creates the right processor by payment method."""

    _processors = {
        "credit_card": CreditCardProcessor,
        "paypal": PayPalProcessor,
        "crypto": CryptoProcessor,
    }

    @classmethod
    def create(cls, method: str) -> PaymentProcessor:
        processor_class = cls._processors.get(method)
        if not processor_class:
            raise ValueError(f"Unsupported payment method: {method}")
        return processor_class()

    @classmethod
    def register(cls, method: str, processor_class):
        cls._processors[method] = processor_class
```

Part 3: Chain of Responsibility -- Order Validation

Before processing an order, it must pass through a validation pipeline. Each validator checks one thing and passes to the next.

```python
class OrderValidator(ABC):
    def __init__(self):
        self._next: Optional['OrderValidator'] = None

    def set_next(self, validator: 'OrderValidator') -> 'OrderValidator':
        self._next = validator
        return validator

    def validate(self, order: Order, product_repo: ProductRepository) \
            -> List[str]:
        errors = self._check(order, product_repo)
        if self._next:
            errors.extend(self._next.validate(order, product_repo))
        return errors

    @abstractmethod
    def _check(self, order: Order,
               product_repo: ProductRepository) -> List[str]:
        pass


class ItemsValidator(OrderValidator):
    def _check(self, order, product_repo):
        if not order.items:
            return ["Order must have at least one item"]
        return []


class StockValidator(OrderValidator):
    def _check(self, order, product_repo):
        errors = []
        for item in order.items:
            product = product_repo.find_by_id(item.product_id)
            if not product:
                errors.append(f"Product not found: {item.product_id}")
            elif product.stock < item.quantity:
                errors.append(
                    f"Insufficient stock for {product.name}: "
                    f"need {item.quantity}, have {product.stock}")
        return errors


class PriceValidator(OrderValidator):
    def _check(self, order, product_repo):
        errors = []
        for item in order.items:
```

```
            product = product_repo.find_by_id(item.product_id)
            if product and item.unit_price != product.price:
                errors.append(
                    f"Price mismatch for {item.product_name}: "
                    f"expected ${product.price}, got ${item.unit_price}")
        return errors


class MinimumOrderValidator(OrderValidator):
    def __init__(self, minimum: float = 1.00):
        super().__init__()
        self.minimum = minimum

    def _check(self, order, product_repo):
        subtotal = sum(item.subtotal for item in order.items)
        if subtotal < self.minimum:
            return [f"Minimum order amount is ${self.minimum:.2f}"]
        return []


def build_validation_chain() -> OrderValidator:
    """Build the validation pipeline."""
    items = ItemsValidator()
    stock = StockValidator()
    price = PriceValidator()
    minimum = MinimumOrderValidator(5.00)

    items.set_next(stock).set_next(price).set_next(minimum)
    return items
```

Part 4: Observer -- Event System

Services communicate through events. The Observer pattern enables loose coupling between components.

```python
class EventBus:
    """Central event bus using Observer pattern."""

    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._handlers = {}
            cls._instance._event_log = []
        return cls._instance
```

```
    def subscribe(self, event_type: str, handler):
        if event_type not in self._handlers:
            self._handlers[event_type] = []
        self._handlers[event_type].append(handler)

    def publish(self, event_type: str, data: dict):
        self._event_log.append({
            "event": event_type,
            "data": data,
            "timestamp": datetime.now().isoformat()
        })

        handlers = self._handlers.get(event_type, [])
        for handler in handlers:
            try:
                handler(data)
            except Exception as e:
                print(f"  [EventBus] Handler error for {event_type}: {e}")

    def get_log(self):
        return self._event_log


# Event handlers
class NotificationHandler:
    def on_order_placed(self, data):
        print(f"  [Notification] Order {data['order_id']} confirmation "
              f"sent to customer {data['customer_id']}")

    def on_order_cancelled(self, data):
        print(f"  [Notification] Order {data['order_id']} cancellation "
              f"notice sent")

    def on_payment_completed(self, data):
        print(f"  [Notification] Payment receipt sent for "
              f"order {data['order_id']}")


class AnalyticsHandler:
    def __init__(self):
        self.events = []

    def on_order_placed(self, data):
        self.events.append({"type": "order_placed", **data})
        print(f"  [Analytics] Tracked order {data['order_id']} "
              f"(total tracked: {len(self.events)})")

    def on_payment_completed(self, data):
        self.events.append({"type": "payment", **data})


class InventoryHandler:
    def __init__(self, product_repo: ProductRepository):
        self.product_repo = product_repo
```

```
    def on_order_placed(self, data):
        for item in data.get("items", []):
            success = self.product_repo.update_stock(
                item["product_id"], -item["quantity"])
            if success:
                print(f"  [Inventory] Reserved {item['quantity']}x "
                      f"{item['product_name']}")

    def on_order_cancelled(self, data):
        for item in data.get("items", []):
            self.product_repo.update_stock(
                item["product_id"], item["quantity"])
            print(f"  [Inventory] Released {item['quantity']}x "
                  f"{item['product_name']}")
```

Part 5: Command -- Order Operations

The Command pattern encapsulates order operations, enabling undo, logging, and queuing.

```python
class OrderCommand(ABC):
    @abstractmethod
    def execute(self) -> dict:
        pass

    @abstractmethod
    def undo(self) -> dict:
        pass

    @abstractmethod
    def describe(self) -> str:
        pass


class PlaceOrderCommand(OrderCommand):
    def __init__(self, order: Order, order_repo: OrderRepository,
                 pricing: PricingStrategy, discount: DiscountStrategy,
                 shipping: ShippingStrategy):
        self.order = order
        self.order_repo = order_repo
        self.pricing = pricing
        self.discount = discount
        self.shipping = shipping
        self._executed = False

    def execute(self) -> dict:
```

```
        # Calculate totals using strategies
        subtotal = self.pricing.calculate(self.order.items)
        discount_amount = self.discount.apply(
            subtotal, self.order.customer_id)
        shipping_cost = self.shipping.calculate(
            subtotal - discount_amount, self.order.items)
        tax = (subtotal - discount_amount) * Config().get("tax_rate", 0.08)
        total = subtotal - discount_amount + shipping_cost + tax

        self.order.total = round(total, 2)
        self.order.tax = round(tax, 2)
        self.order.shipping_cost = round(shipping_cost, 2)
        self.order.discount = round(discount_amount, 2)
        self.order.status = "placed"

        self.order_repo.save(self.order)
        self._executed = True

        # Publish event
        EventBus().publish("order.placed", {
            "order_id": self.order.id,
            "customer_id": self.order.customer_id,
            "total": self.order.total,
            "items": [
                {"product_id": i.product_id,
                 "product_name": i.product_name,
                 "quantity": i.quantity}
                for i in self.order.items
            ]
        })

        return {
            "order_id": self.order.id,
            "subtotal": round(subtotal, 2),
            "discount": self.order.discount,
            "shipping": self.order.shipping_cost,
            "tax": self.order.tax,
            "total": self.order.total,
            "status": self.order.status
        }

    def undo(self) -> dict:
        if not self._executed:
            return {"error": "Order was not placed"}

        self.order_repo.update_status(self.order.id, "cancelled")
        self.order.status = "cancelled"

        EventBus().publish("order.cancelled", {
            "order_id": self.order.id,
            "customer_id": self.order.customer_id,
            "items": [
                {"product_id": i.product_id,
                 "product_name": i.product_name,
                 "quantity": i.quantity}
```

```
                for i in self.order.items
            ]
        })

        return {"order_id": self.order.id, "status": "cancelled"}

    def describe(self):
        return f"Place order {self.order.id} for {self.order.customer_id}"
```

Part 6: Circuit Breaker -- Resilient Service Calls

When calling external services (payment, shipping), the Circuit Breaker prevents cascading failures.

```python
import time
from enum import Enum


class CircuitState(Enum):
    CLOSED = "closed"       # normal operation
    OPEN = "open"           # failing, reject calls
    HALF_OPEN = "half_open" # testing recovery


class CircuitBreaker:
    def __init__(self, name, failure_threshold=3,
                 recovery_timeout=10):
        self.name = name
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.state = CircuitState.CLOSED
        self.failure_count = 0
        self.last_failure_time = 0
        self.success_count = 0

    def call(self, func, *args, **kwargs):
        if self.state == CircuitState.OPEN:
            if time.time() - self.last_failure_time > self.recovery_timeout:
                self.state = CircuitState.HALF_OPEN
                print(f"  [CB:{self.name}] HALF_OPEN: testing...")
            else:
                raise RuntimeError(
                    f"Circuit breaker {self.name} is OPEN")

        try:
            result = func(*args, **kwargs)
```

```
            if self.state == CircuitState.HALF_OPEN:
                self.state = CircuitState.CLOSED
                self.failure_count = 0
                print(f"  [CB:{self.name}] CLOSED: recovered")

            return result

        except Exception as e:
            self.failure_count += 1
            self.last_failure_time = time.time()

            if self.failure_count >= self.failure_threshold:
                self.state = CircuitState.OPEN
                print(f"  [CB:{self.name}] OPEN: too many failures "
                      f"({self.failure_count})")

            raise
```

Part 7: Saga -- Checkout Transaction

The complete checkout flow as a Saga with compensating transactions.

```python
class CheckoutSaga:
    """Orchestrates the checkout process across multiple services."""

    def __init__(self, order_repo, product_repo, payment_factory):
        self.order_repo = order_repo
        self.product_repo = product_repo
        self.payment_factory = payment_factory
        self.circuit_breaker = CircuitBreaker("payment", 3, 30)

    def execute(self, order: Order) -> dict:
        print(f"\n{'=' * 50}")
        print(f"  CHECKOUT SAGA: {order.id}")
        print(f"{'=' * 50}")

        completed_steps = []

        try:
            # Step 1: Validate order
            print(f"\n  Step 1: Validate Order")
            validator = build_validation_chain()
            errors = validator.validate(order, self.product_repo)
            if errors:
                return {"status": "failed", "errors": errors}
```

```
            print(f"  --> Validation passed")

            # Step 2: Reserve inventory
            print(f"\n  Step 2: Reserve Inventory")
            reserved_items = []
            for item in order.items:
                success = self.product_repo.update_stock(
                    item.product_id, -item.quantity)
                if not success:
                    # Compensate already-reserved items
                    for ri in reserved_items:
                        self.product_repo.update_stock(
                            ri["product_id"], ri["quantity"])
                    return {"status": "failed",
                            "errors": [f"Failed to reserve {item.product_name}"]}
                reserved_items.append({
                    "product_id": item.product_id,
                    "quantity": item.quantity
                })
                product = self.product_repo.find_by_id(item.product_id)
                print(f"    Reserved {item.quantity}x {item.product_name} "
                      f"(remaining: {product.stock})")
            completed_steps.append("inventory")
            print(f"  --> Inventory reserved")

            # Step 3: Process payment
            print(f"\n  Step 3: Process Payment")
            processor = self.payment_factory.create(order.payment_method)
            try:
                payment_result = self.circuit_breaker.call(
                    processor.charge, order.total, order.id)
            except RuntimeError as e:
                # Circuit breaker open or payment failed
                self._compensate(completed_steps, order, reserved_items)
                return {"status": "failed", "errors": [str(e)]}
            completed_steps.append("payment")
            print(f"  --> Payment processed: {payment_result['transaction_id']}")

            # Step 4: Confirm order
            print(f"\n  Step 4: Confirm Order")
            self.order_repo.update_status(order.id, "confirmed")
            order.status = "confirmed"
            completed_steps.append("order_confirmed")

            # Publish success event
            EventBus().publish("payment.completed", {
                "order_id": order.id,
                "customer_id": order.customer_id,
                "total": order.total,
                "transaction_id": payment_result["transaction_id"]
            })

            print(f"\n{'=' * 50}")
            print(f"  CHECKOUT COMPLETE: {order.id}")
            print(f"  Total: ${order.total:.2f}")
```

```
            print(f"{'=' * 50}")

            return {
                "status": "confirmed",
                "order_id": order.id,
                "total": order.total,
                "transaction_id": payment_result["transaction_id"]
            }

        except Exception as e:
            print(f"\n  SAGA FAILED: {e}")
            self._compensate(completed_steps, order, reserved_items
                             if 'reserved_items' in dir() else [])
            return {"status": "failed", "errors": [str(e)]}

    def _compensate(self, completed_steps, order, reserved_items):
        print(f"\n  Compensating {len(completed_steps)} steps...")

        if "payment" in completed_steps:
            processor = self.payment_factory.create(order.payment_method)
            processor.refund(order.total, order.id)
            print(f"    Refunded payment")

        if "inventory" in completed_steps:
            for ri in reserved_items:
                self.product_repo.update_stock(
                    ri["product_id"], ri["quantity"])
            print(f"    Released inventory")

        self.order_repo.update_status(order.id, "cancelled")
        print(f"    Order cancelled")
```

Part 8: CQRS -- Product Read/Write Separation

High-traffic product pages need fast reads. CQRS separates the read and write paths.

```python
class ProductCommandService:
    """Write side: handles product mutations."""

    def __init__(self, product_repo: ProductRepository):
        self.repo = product_repo
        self.event_bus = EventBus()

    def create_product(self, name, price, category, stock, description=""):
        product = Product(
            id=f"PROD-{uuid.uuid4().hex[:6].upper()}",
```

```
            name=name,
            price=price,
            category=category,
            stock=stock,
            description=description
        )
        self.repo.save(product)
        self.event_bus.publish("product.created", {
            "product_id": product.id,
            "name": product.name,
            "price": product.price,
            "category": product.category
        })
        return product

    def update_price(self, product_id, new_price):
        product = self.repo.find_by_id(product_id)
        if product:
            old_price = product.price
            product.price = new_price
            self.repo.save(product)
            self.event_bus.publish("product.price_updated", {
                "product_id": product_id,
                "old_price": old_price,
                "new_price": new_price
            })
            return product
        return None


class ProductQueryService:
    """Read side: optimized for fast product lookups."""

    def __init__(self, product_repo: ProductRepository):
        self.repo = product_repo
        self._cache: Dict[str, dict] = {}
        self._category_cache: Dict[str, List[dict]] = {}

    def get_product(self, product_id):
        if product_id in self._cache:
            return self._cache[product_id]

        product = self.repo.find_by_id(product_id)
        if product:
            view = self._to_view(product)
            self._cache[product_id] = view
            return view
        return None

    def get_by_category(self, category):
        if category in self._category_cache:
            return self._category_cache[category]

        products = self.repo.find_by_category(category)
        views = [self._to_view(p) for p in products]
```

```
        self._category_cache[category] = views
        return views

    def get_catalog(self):
        """Full product catalog for browsing."""
        return [self._to_view(p) for p in self.repo.find_all()]

    def invalidate_cache(self, data):
        """Called when products change (event handler)."""
        product_id = data.get("product_id")
        if product_id in self._cache:
            del self._cache[product_id]
        self._category_cache.clear()

    def _to_view(self, product):
        return {
            "id": product.id,
            "name": product.name,
            "price": product.price,
            "category": product.category,
            "in_stock": product.stock > 0,
            "stock_level": "high" if product.stock > 10
                           else "low" if product.stock > 0
                           else "out_of_stock"
        }
```

Part 9: Putting It All Together

```python
def run_ecommerce_demo():
    """Complete e-commerce flow using 15+ patterns."""

    print("=" * 60)
    print("  E-COMMERCE SYSTEM DEMO")
    print("  Patterns: Singleton, Factory, Builder, Strategy,")
    print("  Observer, Command, Chain of Responsibility,")
    print("  Repository, CQRS, Circuit Breaker, Saga")
    print("=" * 60)

    # --- Setup (Singleton: Config, Repository: Data Access) ---
    config = Config()
    product_repo = InMemoryProductRepository()
    order_repo = InMemoryOrderRepository()

    # --- CQRS: Separate read/write services ---
    product_cmd = ProductCommandService(product_repo)
    product_query = ProductQueryService(product_repo)
```

```
# --- Observer: Wire up event handlers ---
event_bus = EventBus()
notification = NotificationHandler()
analytics = AnalyticsHandler()
inventory = InventoryHandler(product_repo)

event_bus.subscribe("order.placed", notification.on_order_placed)
event_bus.subscribe("order.placed", analytics.on_order_placed)
event_bus.subscribe("order.cancelled", notification.on_order_cancelled)
event_bus.subscribe("order.cancelled", inventory.on_order_cancelled)
event_bus.subscribe("payment.completed", notification.on_payment_completed)
event_bus.subscribe("payment.completed", analytics.on_payment_completed)
event_bus.subscribe("product.price_updated",
                    product_query.invalidate_cache)

# --- Create products (CQRS write side) ---
print("\n--- Creating Products ---")
laptop = product_cmd.create_product(
    "Gaming Laptop", 999.99, "electronics", 15,
    "High-performance gaming laptop")
mouse = product_cmd.create_product(
    "Wireless Mouse", 29.99, "electronics", 50)
book = product_cmd.create_product(
    "Design Patterns Book", 44.99, "books", 100)
shirt = product_cmd.create_product(
    "Python T-Shirt", 24.99, "clothing", 200)

print(f"  Created: {laptop.name} (${laptop.price})")
print(f"  Created: {mouse.name} (${mouse.price})")
print(f"  Created: {book.name} (${book.price})")
print(f"  Created: {shirt.name} (${shirt.price})")

# --- Browse products (CQRS read side) ---
print("\n--- Product Catalog (CQRS Read) ---")
catalog = product_query.get_catalog()
for p in catalog:
    print(f"  {p['name']:25s} ${p['price']:>8.2f}  "
          f"[{p['stock_level']}]")

# --- Build an order (Builder Pattern) ---
print("\n--- Building Order (Builder) ---")
order = (OrderBuilder("customer-alice")
    .add_item(laptop, 1)
    .add_item(mouse, 2)
    .add_item(book, 1)
    .shipping_address("123 Main St, Springfield")
    .payment_method("credit_card")
    .build())

print(f"  Order {order.id} built with {len(order.items)} items")

# --- Calculate totals (Strategy Pattern) ---
print("\n--- Calculating Totals (Strategy) ---")
pricing = StandardPricing()
```

```
    vip_set = {"customer-alice"}
    discount = LoyaltyDiscount(vip_set)
    shipping = StandardShipping()

    # --- Place order (Command Pattern) ---
    place_cmd = PlaceOrderCommand(
        order, order_repo, pricing, discount, shipping)
    print(f"\n--- Placing Order (Command) ---")
    result = place_cmd.execute()

    print(f"\n  Order Summary:")
    print(f"    Subtotal:  ${result['subtotal']:>8.2f}")
    print(f"    Discount:  -${result['discount']:>7.2f}")
    print(f"    Shipping:  ${result['shipping']:>8.2f}")
    print(f"    Tax:       ${result['tax']:>8.2f}")
    print(f"    Total:     ${result['total']:>8.2f}")

    # --- Checkout (Saga + Circuit Breaker + Factory) ---
    saga = CheckoutSaga(order_repo, product_repo,
                        PaymentProcessorFactory)
    checkout_result = saga.execute(order)

    # --- Verify stock updated ---
    print("\n--- Stock After Order ---")
    for product in product_repo.find_all():
        print(f"  {product.name:25s} stock: {product.stock}")

    # --- Order history (Repository) ---
    print("\n--- Order History (Repository) ---")
    alice_orders = order_repo.find_by_customer("customer-alice")
    for o in alice_orders:
        print(f"  {o.id}: ${o.total:.2f} [{o.status}]")

    # --- Event log (Observer) ---
    print("\n--- Event Log ---")
    for event in event_bus.get_log()[-6:]:
        print(f"  {event['event']:25s} at {event['timestamp'][:19]}")

    # --- Second order: with cancellation (Command undo) ---
    print("\n--- Building Second Order (will be cancelled) ---")
    order2 = (OrderBuilder("customer-bob")
        .add_item(shirt, 3)
        .shipping_address("456 Oak Ave, Shelbyville")
        .payment_method("paypal")
        .build())

    place_cmd2 = PlaceOrderCommand(
        order2, order_repo, StandardPricing(),
        NoDiscount(), ExpressShipping())
    result2 = place_cmd2.execute()
    print(f"  Placed: {order2.id} total=${result2['total']}")

    print("\n--- Cancelling Order (Command Undo) ---")
    cancel_result = place_cmd2.undo()
    print(f"  Cancelled: {cancel_result}")
```

```
    print("\n--- Final Stock ---")
    for product in product_repo.find_all():
        print(f"  {product.name:25s} stock: {product.stock}")

    print(f"\n{'=' * 60}")
    print(f"  DEMO COMPLETE")
    print(f"  Patterns demonstrated: Singleton, Factory, Builder,")
    print(f"  Strategy, Observer, Command, Chain of Responsibility,")
    print(f"  Repository, CQRS, Circuit Breaker, Saga")
    print(f"{'=' * 60}")


# Run the demo
run_ecommerce_demo()
```

Output:

```
============================================================
  E-COMMERCE SYSTEM DEMO
  Patterns: Singleton, Factory, Builder, Strategy,
  Observer, Command, Chain of Responsibility,
  Repository, CQRS, Circuit Breaker, Saga
============================================================

--- Creating Products ---
  Created: Gaming Laptop ($999.99)
  Created: Wireless Mouse ($29.99)
  Created: Design Patterns Book ($44.99)
  Created: Python T-Shirt ($24.99)

--- Product Catalog (CQRS Read) ---
  Gaming Laptop             $  999.99  [high]
  Wireless Mouse            $   29.99  [high]
  Design Patterns Book      $   44.99  [high]
  Python T-Shirt            $   24.99  [high]

--- Building Order (Builder) ---
  Order ORD-A1B2C3D4 built with 3 items

--- Calculating Totals (Strategy) ---

--- Placing Order (Command) ---
  [Notification] Order ORD-A1B2C3D4 confirmation sent to customer customer-alice
  [Analytics] Tracked order ORD-A1B2C3D4 (total tracked: 1)

  Order Summary:
    Subtotal:  $ 1104.96
    Discount:  -$ 165.74
    Shipping:  $    0.00
    Tax:       $   75.14
    Total:     $ 1014.36
```

```
==================================================
  CHECKOUT SAGA: ORD-A1B2C3D4
==================================================

  Step 1: Validate Order
  --> Validation passed

  Step 2: Reserve Inventory
    Reserved 1x Gaming Laptop (remaining: 14)
    Reserved 2x Wireless Mouse (remaining: 48)
    Reserved 1x Design Patterns Book (remaining: 99)
  --> Inventory reserved

  Step 3: Process Payment
    [CreditCard] Charging $1014.36 for order ORD-A1B2C3D4
  --> Payment processed: CC-e5f6g7h8

  Step 4: Confirm Order
  [Notification] Payment receipt sent for order ORD-A1B2C3D4
  [Analytics] Tracked order ... (total tracked: 2)

==================================================
  CHECKOUT COMPLETE: ORD-A1B2C3D4
  Total: $1014.36
==================================================

--- Stock After Order ---
  Gaming Laptop             stock: 14
  Wireless Mouse            stock: 48
  Design Patterns Book      stock: 99
  Python T-Shirt            stock: 200

--- Order History (Repository) ---
  ORD-A1B2C3D4: $1014.36 [confirmed]

--- Event Log ---
  order.placed              at 2026-03-29T10:30
  payment.completed         at 2026-03-29T10:30
  ...

--- Building Second Order (will be cancelled) ---
  Placed: ORD-E5F6G7H8 total=$89.46

--- Cancelling Order (Command Undo) ---
  [Notification] Order ORD-E5F6G7H8 cancellation notice sent
  Cancelled: {'order_id': 'ORD-E5F6G7H8', 'status': 'cancelled'}

--- Final Stock ---
  Gaming Laptop             stock: 14
  Wireless Mouse            stock: 48
  Design Patterns Book      stock: 99
  Python T-Shirt            stock: 200

============================================================
  DEMO COMPLETE
```

```
  Patterns demonstrated: Singleton, Factory, Builder,
  Strategy, Observer, Command, Chain of Responsibility,
  Repository, CQRS, Circuit Breaker, Saga
==============================================================
```

Architecture Decision Map

When building a system like this, use this decision map to choose patterns:

```
"I need to create objects..."
  - Complex construction?           -> Builder
  - Multiple types from one family? -> Factory Method
  - Copy an existing template?      -> Prototype

"I need to structure behavior..."
  - Interchangeable algorithms?     -> Strategy
  - Pipeline of handlers?           -> Chain of Responsibility
  - Undoable operations?            -> Command
  - React to state changes?         -> Observer

"I need to access data..."
  - Abstract storage?               -> Repository
  - Separate read/write?            -> CQRS
  - Cross-service transaction?      -> Saga

"I need reliability..."
  - External service might fail?    -> Circuit Breaker
  - Need a single entry point?      -> API Gateway
  - Dynamic service locations?      -> Service Discovery

"I need shared resources..."
  - One instance globally?          -> Singleton
  - Share common state?             -> Flyweight
  - Pool expensive resources?       -> Object Pool
```

Quick Summary

This project demonstrates how design patterns work together in a real system. No pattern exists in isolation. Singleton provides configuration. Repository abstracts data access. Builder constructs complex orders. Strategy handles variable business rules. Observer enables event-driven communication. Command enables undoable operations. Chain of Responsibility

validates data. Factory creates processors. CQRS optimizes reads and writes. Circuit Breaker adds resilience. Saga manages distributed transactions.

Key Points

- Patterns are tools that compose: Builder creates objects that Strategy processes
- The Repository pattern is the foundation that every other pattern depends on for data access
- Events (Observer) are the glue that connects loosely coupled components
- Saga coordinates across service boundaries what transactions do within one
- Circuit Breaker prevents one failing service from taking down the whole system
- CQRS lets you optimize reads and writes independently
- Start simple: add patterns only when the complexity they address actually exists
- Every pattern has a cost: do not use patterns just because you can

---

Practice Questions

1. In this system, why do we use both the Command pattern and the Saga pattern? What does each handle that the other cannot?
2. If we removed the Event Bus (Observer), what would change? Which components would need direct references to each other?
3. The Circuit Breaker wraps payment calls. What other external calls in an e-commerce system should have circuit breakers?
4. How would you add a "gift wrapping" option to the order? Which patterns would be involved and how?
5. The system uses in-memory repositories. What changes if you switch to a real database? Which patterns protect you from this change?

Exercises

Exercise 1: Add a Coupon System

Add coupon support to the e-commerce system:

- Create a `CouponRepository` to store and validate coupons
- Add a `CouponDiscountStrategy` that applies coupon codes
- Extend `OrderBuilder` with a `.apply_coupon(code)` method
- Add coupon validation to the Chain of Responsibility
- Publish a "coupon.redeemed" event when a coupon is used
- Support: percentage coupons, fixed-amount coupons, and minimum-order coupons

Exercise 2: Add Product Reviews

Add a review system:

- `Review` entity with rating, comment, customer_id, product_id
- `ReviewRepository` with CQRS: write side saves reviews, read side returns aggregated ratings
- Chain of Responsibility for review validation (profanity filter, rating range check, purchase verification)
- Observer events: "review.posted" triggers notification to product owner
- Strategy for sorting reviews: by date, by rating, by helpfulness

Exercise 3: Add Inventory Alerts

Build an inventory monitoring system:

- Observer that watches stock levels after every order
- When stock drops below threshold (5 items), publish "inventory.low" event
- When stock reaches zero, publish "inventory.depleted" event
- NotificationHandler sends alerts to the inventory team
- Command to restock products with undo capability
- Dashboard view (print) showing all products with stock warnings

---

What Is Next?

The final chapter provides a comprehensive Glossary of all design pattern terms used throughout this book, giving you a quick reference to look up any concept.

Chapter 32: Glossary

A comprehensive reference of design pattern terms and concepts used throughout this book. Each entry includes a concise definition and the chapter(s) where it is covered in depth.

Abstract Factory A creational pattern that provides an interface for creating families of related objects without specifying their concrete classes. Produces objects that are designed to work together. (Chapter 5)

Adapter A structural pattern that allows objects with incompatible interfaces to collaborate by wrapping one object to make it look like another. Also called a Wrapper. (Chapter 7)

Anti-Corruption Layer A boundary that translates between two systems or contexts to prevent one system's design from leaking into another. Often implemented using Adapter or Facade patterns. (Chapter 26)

API Gateway An architectural pattern that provides a single entry point for client requests to a microservice architecture. Handles routing, authentication, rate limiting, and response aggregation. (Chapter 29)

Behavioral Pattern A category of design patterns concerned with algorithms and the assignment of responsibilities between objects. Examples include Strategy, Observer, and Command. (Chapters 14-22)

BFF (Backend for Frontend) A variation of the API Gateway pattern where separate gateway instances are created for different client types (mobile, web, admin), each tailored to that client's data needs. (Chapter 29)

Bridge A structural pattern that separates an abstraction from its implementation so the two can vary independently. Useful when both the abstraction hierarchy and implementation hierarchy need to grow. (Chapter 9)

Builder A creational pattern that constructs complex objects step by step. Allows producing different types and representations of an object using the same construction process. (Chapter 4)

Bulkhead A resilience pattern that isolates components so that a failure in one does not cascade to others. Named after ship compartments that prevent flooding from sinking the

entire vessel. (Chapter 27)

Chain of Responsibility A behavioral pattern that passes a request along a chain of handlers. Each handler decides either to process the request or pass it to the next handler in the chain. (Chapter 18)

Choreography A saga coordination approach where each service listens for events and reacts independently. No central coordinator. Contrast with Orchestration. (Chapter 28)

Circuit Breaker A resilience pattern that prevents an application from repeatedly calling a failing service. Tracks failures and "opens" the circuit to reject calls, then periodically allows test calls to check recovery. Three states: Closed, Open, Half-Open. (Chapter 27)

Client-Side Discovery A service discovery approach where the client queries the service registry directly and selects which instance to call, performing its own load balancing. (Chapter 30)

Cloneable A Java interface that indicates an object can be cloned using `Object.clone()`. Used in the Prototype pattern. Python equivalent uses the `copy` module. (Chapter 6)

Command A behavioral pattern that encapsulates a request as an object, allowing parameterization of clients with different requests, queuing of requests, and support for undoable operations. (Chapter 17)

Compensating Transaction A transaction that undoes the effects of a previously completed transaction. Used in the Saga pattern to maintain consistency when a step in a distributed transaction fails. (Chapter 28)

Composite A structural pattern that composes objects into tree structures to represent part-whole hierarchies. Lets clients treat individual objects and compositions uniformly through a shared interface. (Chapter 12)

Concrete Class A class that implements all abstract methods and can be instantiated. Contrast with abstract class or interface. Used throughout all pattern implementations.

Copy Constructor A constructor that creates a new object by copying fields from an existing object of the same class. An alternative to implementing `Cloneable` in Java for the Prototype pattern. (Chapter 6)

CQRS (Command Query Responsibility Segregation) An architectural pattern that separates the read model (queries) from the write model (commands). Allows independent optimization and scaling of reads and writes. (Chapter 25)

Creational Pattern A category of design patterns that deal with object creation mechanisms, trying to create objects in a manner suitable to the situation. Examples include Singleton, Factory Method, and Builder. (Chapters 1-6)

Decorator A structural pattern that attaches additional responsibilities to an object dynamically. Provides a flexible alternative to subclassing for extending functionality. (Chapter 10)

Deep Copy A copy operation that duplicates an object and all objects it references, recursively. The result is fully independent of the original. Contrast with Shallow Copy. (Chapter 6)

Dependency Injection A technique where an object receives its dependencies from external code rather than creating them internally. Not a GoF pattern but widely used in modern backend development. (Chapter 23)

Design Pattern A general, reusable solution to a commonly occurring problem within a given context in software design. Not finished code but a template for solving problems.

Double Dispatch A technique where the operation performed depends on the runtime types of two objects. Used in the Visitor pattern: the element calls a type-specific method on the visitor. (Chapter 22)

Eager Initialization Creating a Singleton instance when the class is loaded, rather than waiting until first use. Simple and thread-safe but wastes resources if the instance is never used. (Chapter 1)

Event Bus A communication mechanism where components publish events and subscribe to events without knowing about each other. An implementation of the Observer pattern for decoupled communication. (Chapters 20, 31)

Event Sourcing An architectural pattern that stores the full history of changes to application state as a sequence of events, rather than storing just the current state. Often used with CQRS. (Chapter 25)

Extrinsic State In the Flyweight pattern, data that varies between instances and is stored externally (by the client or context object). Examples: position, size, user-specific data. Contrast with Intrinsic State. (Chapter 13)

Facade A structural pattern that provides a simplified interface to a complex subsystem. Hides the complexity of interacting with multiple classes behind a single, easy-to-use class. (Chapter 8)

Factory Method A creational pattern that defines an interface for creating objects but lets subclasses decide which class to instantiate. Defers instantiation to subclasses. (Chapter 3)

Flyweight A structural pattern that uses sharing to support large numbers of fine-grained objects efficiently. Splits object state into shared (intrinsic) and unique (extrinsic) parts. (Chapter 13)

Flyweight Factory A factory that manages the creation and caching of Flyweight objects. Ensures that shared instances are reused rather than duplicated. (Chapter 13)

Gang of Four (GoF) The four authors of the seminal book "Design Patterns: Elements of Reusable Object-Oriented Software" (1994): Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. The book defined 23 classic design patterns.

Generator (Python) A function that uses the `yield` keyword to produce a sequence of values lazily. Each call to `next()` resumes execution until the next `yield`. A natural implementation of the Iterator pattern in Python. (Chapter 21)

God Object An anti-pattern where one class does too much, accumulating too many responsibilities. The Mediator pattern can degrade into a God Object if too much logic is placed in the mediator. (Chapter 20)

Half-Open State In the Circuit Breaker pattern, a state where the breaker allows a limited number of test requests to check if the failing service has recovered. (Chapter 27)

Handler In the Chain of Responsibility pattern, an object that processes a request or passes it to the next handler. Each handler in the chain has a single responsibility. (Chapter 18)

Health Check A mechanism where services report their operational status (healthy/unhealthy) to a service registry or monitoring system. Used in Service Discovery to route traffic only to healthy instances. (Chapter 30)

Heartbeat A periodic signal sent by a service instance to the service registry to indicate it is still alive. If heartbeats stop, the registry removes the instance. (Chapter 30)

Idempotent An operation that produces the same result whether executed once or multiple times. Critical for compensating transactions in the Saga pattern and for retry mechanisms. (Chapter 28)

Intrinsic State In the Flyweight pattern, data that is shared across many instances and stored inside the flyweight object. Must be immutable. Examples: texture data, font definition, icon SVG. Contrast with Extrinsic State. (Chapter 13)

Iterable An object that can produce an Iterator. In Java, implements `Iterable<T>`. In Python, implements `__iter__()`. Enables use in for-each loops. (Chapter 21)

Iterator A behavioral pattern that provides a way to access elements of a collection sequentially without exposing its underlying representation. In Java: `Iterator<T>`. In Python: `__iter__()` and `__next__()`. (Chapter 21)

Lazy Initialization Creating a Singleton instance only when first requested, rather than at class load time. Saves resources but requires thread-safety handling. (Chapter 1)

Lazy Evaluation Computing values only when they are needed, not in advance. Generators and iterators use lazy evaluation to handle datasets larger than memory. (Chapter 21)

Leaf In the Composite pattern, an object that has no children. Represents the end of a branch in the tree structure. Performs operations directly rather than delegating. (Chapter 12)

Load Balancer A component that distributes incoming requests across multiple service instances. Strategies include round-robin, random, least connections, and weighted distribution. (Chapters 29, 30)

Mediator A behavioral pattern that reduces chaotic dependencies between objects by restricting direct communications and forcing them to collaborate through a mediator object. (Chapter 20)

Memento A behavioral pattern that captures an object's internal state so it can be restored later without violating encapsulation. Used for undo/redo functionality. (Chapter 19)

Null Object A behavioral pattern that provides an object with a defined neutral ("null") behavior. Eliminates the need for null checks by providing a do-nothing implementation of an interface.

Observer A behavioral pattern that defines a one-to-many dependency between objects so that when one object changes state, all dependents are notified and updated automatically. Also known as Publish-Subscribe. (Chapter 15)

Open/Closed Principle A SOLID principle stating that software entities should be open for extension but closed for modification. The Strategy, Observer, and Visitor patterns embody this principle.

Orchestration A saga coordination approach where a central orchestrator directs each service when to execute its step. The orchestrator contains the workflow logic. Contrast with Choreography. (Chapter 28)

Polymorphism The ability of different classes to respond to the same method call in their own way. Fundamental to nearly every design pattern, enabling interface-based programming.

Prototype A creational pattern that creates new objects by cloning existing ones. Useful when object creation is expensive or when many objects share similar configurations. (Chapter 6)

Prototype Registry A catalog of pre-configured prototype objects stored by name. Clients look up a prototype by name and clone it, rather than configuring objects from scratch. (Chapter 6)

Proxy A structural pattern that provides a surrogate or placeholder for another object to control access to it. Types include virtual proxy, protection proxy, and remote proxy. (Chapter 11)

Rate Limiting Controlling the number of requests a client can make within a time period. Common algorithms include token bucket and sliding window. Typically enforced at the API Gateway. (Chapter 29)

Repository An architectural pattern that mediates between the domain and data mapping layers using a collection-like interface for accessing domain objects. Abstracts data storage details from business logic. (Chapter 23)

Saga A distributed transaction pattern that breaks a long-lived transaction into a sequence of local transactions. Each local transaction has a compensating transaction to undo its effects on failure. (Chapter 28)

Server-Side Discovery A service discovery approach where the client sends requests to a load balancer or router, which queries the registry and forwards the request to an available instance. (Chapter 30)

Service Discovery An architectural pattern where services dynamically register themselves with a central registry, and clients look up service locations at runtime. Enables dynamic scaling and deployment. (Chapter 30)

Service Mesh An infrastructure layer that handles service-to-service communication, providing features like service discovery, load balancing, encryption, and observability without changes to application code. Examples: Istio, Linkerd.

Service Registry A database of available service instances with their network locations. Services register on startup and deregister on shutdown. The registry removes instances that fail health checks. (Chapter 30)

Shallow Copy A copy operation that duplicates an object but shares references to nested objects. The copy and original point to the same nested data. Contrast with Deep Copy. (Chapter 6)

Simple Factory Not a formal GoF pattern but a commonly used technique where a single method creates objects based on a parameter. Simpler than Factory Method but less flexible. (Chapter 2)

Singleton A creational pattern that ensures a class has only one instance and provides a global point of access to it. Thread-safe implementations use double-checked locking or eager initialization. (Chapter 1)

SOLID Principles Five principles of object-oriented design: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. Design patterns often embody one or more of these principles.

State A behavioral pattern that lets an object alter its behavior when its internal state changes. The object appears to change its class. (Chapter 16)

StopIteration A Python exception raised by `__next__()` to signal that an iterator has no more elements. For loops handle this automatically. (Chapter 21)

Strategy A behavioral pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable. Lets the algorithm vary independently from the clients that use it. (Chapter 14)

Stream API (Java) Java 8+ API for functional-style operations on sequences of elements. Provides lazy evaluation, filtering, mapping, and reducing. Built on the Iterator concept. (Chapter 21)

Structural Pattern A category of design patterns concerned with how classes and objects are composed to form larger structures. Examples include Adapter, Decorator, and Composite. (Chapters 7-13)

Template Method A behavioral pattern that defines the skeleton of an algorithm in a superclass but lets subclasses override specific steps without changing the algorithm's structure. (Chapter 19)

Thread Safety The property of code that functions correctly when accessed by multiple threads simultaneously. Critical for Singleton (double-checked locking), Flyweight (immutable shared state), and Service Registry implementations. (Chapters 1, 13, 30)

Token Bucket A rate limiting algorithm that represents available capacity as tokens. Each request consumes a token; tokens are replenished at a fixed rate. When no tokens remain, requests are rejected. (Chapter 29)

Visitor A behavioral pattern that lets you add operations to objects without modifying them. Uses double dispatch to call the right method based on both the visitor type and the element type. (Chapter 22)

Pattern Categories Quick Reference

Creational Patterns (How objects are created)

Pattern	Purpose	Chapter
Singleton	Ensure one instance globally	1
Simple Factory	Centralize object creation	2

Factory Method	Defer creation to subclasses	3
Builder	Construct complex objects step by step	4
Abstract Factory	Create families of related objects	5
Prototype	Clone existing objects	6

Structural Patterns (How objects are composed)

Pattern	Purpose	Chapter
Adapter	Make incompatible interfaces work together	7
Facade	Simplify complex subsystem interface	8
Bridge	Separate abstraction from implementation	9
Decorator	Add responsibilities dynamically	10
Proxy	Control access to an object	11
Composite	Tree structures with uniform interface	12
Flyweight	Share state to save memory	13

Behavioral Patterns (How objects interact)

Pattern	Purpose	Chapter
Strategy	Interchangeable algorithms	14
Observer	Notify dependents of state changes	15
State	Behavior changes with internal state	16
Command	Encapsulate requests as objects	17

Chain of Responsibility	Pass requests along a handler chain	18
Template Method / Memento	Algorithm skeleton / state capture	19
Mediator	Centralize complex communications	20
Iterator	Sequential collection access	21
Visitor	Add operations without modifying classes	22

Architectural Patterns (System-level design)

Pattern	Purpose	Chapter
Repository	Abstract data access	23
Dependency Injection	Externalize dependencies	23
MVC/MVVM	Separate concerns in applications	24
CQRS	Separate read and write models	25
Event Sourcing	Store state as event history	25
Anti-Corruption Layer	Translate between bounded contexts	26
Circuit Breaker	Resilient external service calls	27
Bulkhead	Isolate failure domains	27
Saga	Distributed transaction management	28
API Gateway	Single entry point for microservices	29
Service Discovery	Dynamic service registration and lookup	30